

A Parallel Programming Perspective

Citation for published version (APA):

Franken, T. T. P. (2026). *A Parallel Programming Perspective: Designing AuDaLa and AuDaLogic*. [Phd Thesis 1 (Research TU/e / Graduation TU/e), Mathematics and Computer Science]. Eindhoven University of Technology.

Document status and date:

Published: 03/06/2026

Document Version:

Publisher's PDF, also known as Version of Record (includes final page, issue and volume numbers)

Please check the document version of this publication:

- A submitted manuscript is the version of the article upon submission and before peer-review. There can be important differences between the submitted version and the official published version of record. People interested in the research are advised to contact the author for the final version of the publication, or visit the DOI to the publisher's website.
- The final author version and the galley proof are versions of the publication after peer review.
- The final published version features the final layout of the paper including the volume, issue and page numbers.

[Link to publication](#)

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal.

If the publication is distributed under the terms of Article 25fa of the Dutch Copyright Act, indicated by the "Taverne" license above, please follow below link for the End User Agreement:

www.tue.nl/taverne

Take down policy

If you believe that this document breaches copyright please contact us at:

openaccess@tue.nl

providing details and we will investigate your claim.

A Parallel Programming Perspective Designing AuDaLa and AuDaLogic

PROEFSCHRIFT

ter verkrijging van de graad van doctor aan de
Technische Universiteit Eindhoven, op gezag van de
rector magnificus prof.dr. S.K. Lenaerts, voor een
commissie aangewezen door het College voor
Promoties, in het openbaar te verdedigen op
woensdag 03 juni 2026 om 11:00 uur

door

Thomas Theo Paul Franken

geboren te Rotterdam

Dit proefschrift is goedgekeurd door de promotoren en de samenstelling van de promotiecommissie is als volgt:

Voorzitter:	prof.dr. E.R. van den Heuvel
Promotor:	prof.dr.ir. J.F. Groote
Copromotor:	dr.ir. T.S. Neele
Leden:	prof.dr.ir. N. Meratnia
	prof.dr. M.R. Mousavi (King's College London)
	prof.dr. R.V. van Nieuwpoort (Universiteit Leiden)
	prof.dr. J.A. Pérez Parra (Rijksuniversiteit Groningen)
	dr.ing. A.J. Wijs

Het onderzoek of ontwerp dat in dit proefschrift wordt beschreven is uitgevoerd in overeenstemming met de TU/e Gedragscode Wetenschapsbeoefening.

Copyright © 2026 by Tom T.P. Franken. All Rights Reserved.

A catalogue record is available from the Eindhoven University of Technology Library

ISBN 978-90-386-6701-0.

Printed by ADC Nederland.

Cover design by Tom T.P. Franken.

The work in this thesis has been carried out under the auspices of the research school IPA (Institute for Programming research and Algorithmics).

IPA dissertation series 2026-07.



Preface and Acknowledgement

May 2026

Although I do not remember the exact time I decided it, the decision to do a PhD was made in high school, as a challenge to myself. The decision had very little to do with computer science. Though my grandfather's stories in my youth had kindled a bit of an interest in me for science, most of my youth, I wanted to be a medical doctor, like my parents, and that changed only at the end of high school. It was a choice between computer science and English, for I had more of an affinity with fiction and stories. I chose computer science, for it was expedient (I expected to be able to at least get a job in computer science) and because I liked the clarity of purpose of it.

I almost did not come to Eindhoven either; I was set on Utrecht, but decided to at least check out Eindhoven, where the open day's mini-course on applied logic drew me in. For most of my study, I then fixated on algorithms, for I love their theoretical elegance, only to realise during the second year of my master that formal system analysis was just as cool. On what was essentially a whim, I decided to see if I could do some extra coursework for the Formal System Analysis group, and on a whim, I decided to ask whether the supervisors of those courses knew of a PhD position.

I asked at the right time; not even two days later, Jan Friso contacted me via mail, to my considerable surprise and excitement. It turns out that the Accelerated Verification and Verified Acceleration project group, at the time consisting of Jan Friso Groote, Anton Wijs, Pieter Hijma, Jan Martens and Lars van den Haak, had had the idea to make a parallel programming language for GPUs based on graphs, which they called "graph machines". I was brought in with the task of making that happen, starting with some preliminary designs by Jan Martens and Lars van den Haak. And so I did; supervised by Jan Friso

and Thomas, I spearheaded the creation of AuDaLa from the initial ideas.

This was not as straightforward as I initially thought. I put my own spin on the project by keeping myself to the theoretical side of programming language design and the other related subjects I had to dive into during my PhD. This seems to have been a somewhat unconventional approach to parallel programming language research, to the consternation and rare (but not absent) scorn of reviewers and other parallel programming language researchers, and even worse, as a result of my spin, my contributions did not include an implementation or a benchmark.

A not insignificant amount of my PhD was therefore ultimately spent in justifying the project's existence, motivating my theoretical results and refuting reviewers which considered it only natural that any theoretical programming language design result must be in want of an implementation, including a benchmark. (Personally, I'd argue that this is merely conventional.) Luckily, at some point Gijs Leemrijse came along and obliged the reviewers by creating an AuDaLa to CUDA compiler, which made these efforts quite a bit easier. Not doing this myself gave me time to create AuDaLogic, a proof system for AuDaLa, which I enjoyed far more.

Now, I've got a thesis, I have laid the groundwork for AuDaLa and AuDaLogic and I can consider myself the premier expert on both (out of the four people who have been involved during my PhD). I have risen to my challenge, and now, due to the PhD and the excellent supervision by Jan Friso and Thomas on different fronts, I also consider myself a researcher. After my defense, my challenge will be completed.

Though the journey I went on to solve this challenge was turbulent at times, and I would have been able to avoid many professional, personal and social difficulties by taking the safer and more boring path, I do not regret walking the path I did. Though my current situation and place has its drawbacks, I would not want to have it any other way. Without a shadow of a doubt, it was worth it.

In the future too, I think I will keep challenging myself, as it turns out that challenges are the spice of life. My next adventure will be in Singapore, where I have accepted a postdoc position under Abhik Roychoudhury on agentic verification using theorem provers.

Before I go on that adventure, now, at the end of my PhD, there are many people I would like to thank and to give a mention. Firstly, I would like to thank the original AVVA project group for giving me this opportunity. Furthermore, I would like to thank my supervisor Thomas Neele, my promotor Jan Friso Groote and my brief colleague Gijs Leemrijse.

Jan Friso, you were a great promotor and secondary supervisor. I appreciate and will miss your stories, your humour and your insights in academia as a field. Your stories and insights gave colour and character to the for me

sometimes somewhat abstract place that is academia; thanks especially for that. Though it took some balancing, I can happily say that we found that balance; explaining something to you, especially in the later years, was engaging, challenging, thorough and rewarding. You have a fantastic capacity for real, genuine care about those you supervise, without which I am not sure whether I would have finished the PhD. Thank you for your supervision; I hope you enjoyed it as well.

Thomas, you are an excellent second author and you are a born supervisor. During our collaboration, you were sharp, precise and calm, and you supervised me with a grace I would not (easily) be able to replicate. Though we differ in many things, you had the patience to learn to effectively communicate with me, while also subtly training and challenging me to improve my thought processes and academic discussion without that ever becoming too intrusive or forceful. For that, and for bearing with my random walk-ins and random discussions without complaint, I thank you; I am very glad to have had you as a supervisor. I hope supervising me led to some useful insights, and that you also enjoyed it.

Gijs, we worked together only a bit, but I'm in awe of your intellect and competence. Thank you for your efforts on the weak-memory semantics and the implementation of AuDaLa, which I'm sure would not exist or be as good without you.

During my time in Eindhoven, I have been able to participate in some truly amazing things. I have been part of some great social groups, and I have been able to witness and interact with some amazing people. Thank you all.

I would like to thank some people more specifically. First of all, my family. Mam and pap, Anne-Marie and Ralph, thank you. You've always supported me, and I would not have been able to do half the things I have done without you. I love you both. Thanks to Myrthe, Jule and Jelle as well, and thanks to Loek and Pepijn too.

Then, the extended families Franken and van Haelst. Fons and Bep, Kees Jan, Ome Donic and Tante Hanneke, Steven and Maxime, Jord and Abby, Robert and Lida, Bieke and Steef, Saskia and Erik, Tijmen and Aliena, thanks for all the fun times we have had during family outings, visits and the occasional boardgame night. Kees Jan and Ome Donic, thank you for being secondary guiding figures during my time in Eindhoven as well, I really appreciated that. To the new van Haelstjes, Dominique, Bente, Vicky, Hannah and Lukas, welcome! Lastly, a posthumous thank you to my late grandparents. Opa Paul, thank you for the stories and the walks. Oma Elly Franken-Swenker, thank you for your practical way, the walks and your kindness and tenacity. Oma Elly van Haelst-Snoek, thank you for your care, your sharpness and your honesty. I am glad to have been able to visit you as I did for most of my time in Eindhoven. I miss you.



Figure 1: Behold! Poekie!

Next, I thank my lady and ruler, my dearest little gremlin boo of a cat Poekie Juno Mauwmans. Behold her majesty in Figure 1. Caring for her made my life easier when hard, and better when already good. She's an idiot, but a cute one, and I hope I'll be able to enjoy her shenanigans for a long time yet. On that note, I want to briefly mention Bob the dog and Strepie Mauwmans the cat, my parents' adorable but sadly departed pets during my time in Eindhoven.

Then, I would like to thank the great people I met and interacted with while working as a PhD at the university of Eindhoven, starting with my workgroup, Formal System Analysis. Anna Stramaglia, my second and office mate, you have been a consistent pillar and friend my entire PhD. You have been there through thick and thin and you have always cared, treating me with respect and honesty, but also calling out my bullshit when required. Now at the end of my PhD, you are still here, and I am happy to be able to consider you one of my best friends now and hopefully for a long time yet. Grazie mille. Also thank you Pavlo Burda, for the nice discussions, the events and the barbecues.

Jan Martens, Menno Bartels and Jore Booy, my other office mates, thank you for our time together. Jan, in the office you are who I looked up to as a young researcher; you were (and most likely still are) brilliant, sharp and clear, with many strengths. It was a pleasure to share an office with you, and thank you for the discussions we had during this time; though they may have been a bit intense, they were always worthwhile. Menno, I have no doubt that whatever you are aiming for, you will get. There are few people I have met as quietly competent as you, and I thank you for sharing an office with me and allowing me to explain my research to someone beyond Thomas at times. Jore, you have the spark of unconventionality, and a great sense of humour. You'll be just fine as well. Thank you for sitting next to me, and throwing balls with me as well as munching smints and for our discussions, whether whimsical or more philosophical.

Rick Erkens, Olav Bunte and Maurice Laveaux, thank you for your company, which I greatly enjoyed during my PhD. Rick, I thank you for your straightforward but kind honesty and your excellent taste in music and cats.

Olav, I thank you for your easygoing affability and peacefulness, your vibing with me and sharing stories and books. Maurice, I thank you for your kindness and your wicked sense of humour, which blindsides me in surprising ways. Let our drinks and our games and our concert going continue after my defense.

Clemens Dubslaff, thank you for your advice and the fun discussions we had over the years, as well as going to Lima, Peru with me and Anna for ICTAC. Matthias Volk, thank you for more recent discussions about academia and the future, and for organising the CS research days. Valentina Castiglioni, thank you for your efforts in keeping the group fun. Also thank you the time during FORTE in Groningen.

Sebastian Betancourt Arias and Myrthe Spronck, thanks for the fun times boardslaying the Spire and thank you Sebastian for succeeding me in the PhD council. Thanks Mara Miulescu, Tar van Krieken, Lars van den Haak, Edward Liem, Eduardo Costa Martins, Kevin Jilissen and Kevin van de Glind and those of FSA already mentioned for the nice movie nights, drinks and dinners we had together. Also thank you Jeroen Keiren for that nice Fish coverband concert we went to, and thanks Roel Bloo, Herman Geuvers, Bas Lutтик, Erik de Vink, Tim Willemse, Natasja Betting-van Liempt, Agnes van den Reek, Hans Zantema (may he rest in peace), Flip van Spaendonck, Ferry Timmers and Isabelle Cooijmans, along with all others of FSA already mentioned, for the nice times, drinks, events and colloquia over the years.

I would also like to thank the SET group since I started my PhD, most notably Nathan Cassee and Lavinia Paganini, for joining in with the fun. Notably, for Nathan I remember my first IPA Fall Days, and for Lavinia, the time we went to the Van Gogh museum. Also thanks to Satrio Rukmono, and Hossain Muctadir, for the mangoes and the oranges and the fun.

During my PhD, I also had the pleasure of meeting a lot more people than just those of my group. I want to thank the PhD council as led by Nathan van Beusekom and by Sanne van Kempen. I also want to thank PhDs and colleagues of our neighbouring groups, like Matteo Asuni, Martin Rosso, Leon Kersten, Arpan Ray and Dominik George from SEC, Pieter Cuijpers, Adam Watkins and Jolande Matthijsse from IRIS, Fabrizio Conca, Jolijn Cottaar, Elisa Pioldi and Heloise Guarise Vieira from Discrete Mathematics for the sports days, the occasional movie night and other fun meetings during my PhD.

Thanks the IPA PhD council under my leadership for the three lovely IPA Fall Days events we were able to plan, and the IPA PhD students that I met during those weeks. Specifically, I would like to thank Ivo Gabe de Wolff for being my right hand man (and Taskmaster assistant) on the council and Philip Tasche for his efforts on the council.

I also want to thank the fellow PhDs I met during the IPA Fall Days, Formal Method Days, ICT.Opens, the university strike and other occasions I visited as a PhD. Notably, thank you Christopher Esterhuyse, Dennis Sprockholt, Bob

Rubbens, Niels Doorn, Damian Frölich, Amir Mir, Ali Al-Kaswan, Ömer Şakar and Tom Dennings. Bob, thank you also for your ongoing interest in my work.

Furthermore, thanks to the PhDs I met during ICTAC 2023 and FORTE 2024, notably Bas van den Heuvel, Erik Voogd, Amazigh Amrane, Åsmund Kløvstad, Ziggy Atala and Florian Dorfhuber. May the (Theoretical) Computer Scientists release a second album.

Thanks also to the people from Nexus, for which I had the pleasure to be involved with since before it was founded. Nazanin Farid Mohajer and Charles Wesemann, thanks for all the times with Nexus and after. Nazanin, thank you for the dinners and being the chairman of my first board; you kept us all on track. Charles, thank you for your company and especially the relaxing tea breaks and our conversations recently. I'm happy to have gotten to know both of you, and I am happy to have you as friends.

Also a special thanks to the entirety of Circus, past and present: Ava Sw-evels, Héctor Ortega González, Michela Salinaro, Mitchel Brunings, Elena Zancato and Jan-Willem van Maanen, Sim Bouwmans, Enrique Montero Franco, Nanditha Mudunuru, Silvia Ritsch and Muskan, and to Cédric Roy from the boardgame nights group. Thank you for the fun we had, the boardgames we played and the silliness we caused. I also want to thank my fellow board members not already mentioned: Pavan Telluri, Babak Azkaei, Deovrat Phal and Yaz Armağan Arslankaya. Thank you for standing at the helm of Nexus with me for a while. Furthermore thanks to Yorick Spenrath and Max van Mulken for founding Nexus. Yorick, thanks also for the many financial discussions we had during my board year. Also thanks to the people of the CCCs I was involved with, Nicky van den Berg, Steven Beumer, Luca Begnardi and Marc Kopf, for following me and later checking my finances, and Aurélien Boland and the 2025 party committee for organising a great Nexus party.

Before I move on beyond my work, I would like to lastly thank my committee, Anton Wijs, Nirvana Meratnia, Rob van Nieuwpoort, Jorge Pérez and Mohammad Mousavi, for reading my thesis. I would also like to thank Hans-Dieter Hiep for reading my thesis as well. Thank you for your time and comments.

Next, I would like to thank the friends and people I enjoyed the company of outside of my PhD. Joep van Dam, my second, you are my oldest friend and the one I consider my honorary brother. Visiting you is like coming home away from home; I know I'll always be welcome at yours, and you shall always be welcome at mine, both of us with open arms. It has a way of making things very simple, which I always greatly appreciate. I am happy to see that you have found your equal and counterpart in Annelies; thank you as well, Annelies Riezebos. I'm glad to count you among my friends as well. Thank you both for your company over the years and hopefully for many years to come. Also thanks to Karin and Jeroen van Dam; I've enjoyed joining you and

Joep on several outings and occasions during my years in Eindhoven.

Lars van Duin, you too are one of my oldest friends. Your stalwart presence is a blessing, and your positivity, kindness and optimistic outlook on life always feel like a breeze of fresh air whenever I see you. Luckily for me, you are always present somewhere, be it with boardgames, vacations, joining my DnD campaigns and currently trouncing me in chess. You are a good man, and a great friend, now and hopefully in the future. Never change, and thank you. Also thank you, Linske Huijssen, for the nice conversations and company during my visits to you and Lars; I've greatly enjoyed following you both on Polarsteps during your journey to New Zealand.

Pim Snel, thank you for your calm and level-headedness over the years. Thank you for our sometimes philosophical conversations, our (board)gaming and to have someone to relisten Doctor Who audio plays with. Jasper Mooren, thank you for your care, exuberance and enthusiasm. Thank you for the engaging discussions, the boardgames, carnaval and our visit to Vienna. Bram van de Berg (Brabraham), thank you for your optimism and can-do attitude. Thanks for always being able to challenge my self-imposed boundaries, our discussions and your good taste in fun. I am glad to have you as a semi-constant part in my life, which I hope will stay the case in the future, including vacations, Warhammer and the occasional online multiplayer grand strategy. Maybe one day there will even be a bookshop/bed-and-breakfast/bar somewhere. Also thank you Louisa Snel, for the good times we had during DnD, for the book discussions and for the nice conversations during my visits to you and Pim.

Mark Schapendonk, thank you for being my neighbour and buddy. Easily the thing I miss most about my time in Florastate is the way we just hopped over to each other's apartments to watch series and eat together. Though it's a bit harder now we both moved, I am glad that we still watch movies and series and enjoy whiskies or a nice Brouwers 0.0 together. I hope we can continue doing so for a long time, and rope many more friends into it.

Stan Peters, Jan Ebbers, Esther Dommissie and Benoît Balmaekers, Patrick Schmiermann, Doeke Jelle Haasakker, Inge van Creij-Maas and Marijn van Creij-Maas, Thomas van Dijk and Pleunie Sluijter, Alexandra van Setten and David Vos, thank you for your ongoing friendship. Specifically thank you for the great new year's eves, and the fun social events we do. Stan, thank you for the fun times in Den Bosch, for fanboying about Zelda and for joining my DnD campaigns. I expect great things from you; until and after you achieve them I hope you will join me and I you in many more fun events. Inge and Marijn, thanks for showing me the wonders of carnival and your company in the whiskygroup. Thomas, thank you for playing in my Discordia DnD campaigns. Esther, thanks for the opera's and the concerts and the book discussions. Furthermore, thank you Mathijs Andeweg for joining my Discordia DnD

campaigns, and thank you Frank Wensink for joining us in the whiskygroup.

Jan Ebbers and Janine van Buel, Tom Houtekamer (Plank), Daniel de Malo Aguilar, Sidney den Engelsman, Olaf Brasser, Christiaan Goossens and Merel Jung, thank you for the nights in Hubble, the time at Da Vinci and more recently the nights with the boiis; I am always surprised by how effortless it is to have a good time in your company, where we have candid discussions and joke within the same breath. Thank you. Christiaan, thank you for your calm and kindness and good company, when watching series, making servers or going to St. Jozefdoelen together, as well as our epic bike trips. Furthermore, thank you for joining me for my campaign in Drakkenheim. Daniel, thank you for joining me for my campaign in Drakkenheim; you are an exceptional player. Jan, thank you for your friendship over the years, both in the context of the high school group as well as this group. Thanks for our book conversations, and the intense rounds of boardgames we have played; I'm always up for more.

On the topic of St. Jozef Doelen, Carool Harks, Joke van Montfort, Henry van der Leest, Mark van der Leest, Ash Meerloo, Raymond Meerloo, Gabi Sondagh, Kirsten van der Sanden, Freek Wijnands, Willem Verbeek and the others I met at St. Jozef Doelen, thank you for the company and the many enjoyable and relaxing evenings. Dat we er nog veel mogen lusten! (En lusten doen we ze al!) Also thank you and the Mudhunters for the many fun times we had at the 3D competitions. You brought a lot of life and fun to the concept of 3D competitions for me and I hope to be able to join you again in the future.

Furthermore, I would like to thank Kevin Cleijne of the 13th Da Vinci board, to those already mentioned who were part of the 13th and 17th board of Da Vinci with me and to Marcel Moreaux and the others I met at Da Vinci. I would also like to thank those I have recently met at De Counter Vught.

Jeroen Berkvens, Jenneke Berkvens and Niels Heere, thank you for the fun, the series and the DnD. I've been happy to join in with your chaos and I've enjoyed visiting you, playing boardgames and marathons and helping with the house. Thanks for joining me in Drakkenheim; I'm looking forward to our next campaign, and to when Jeroen and Jenneke will join a MMORPG with Niels and me, even though it is sadly not TurtleWOW. I hope I will be able to enjoy our brand of craziness for quite some time with you all.

René Verhoef-Schutgens and Linde Schutgens-Verhoef, Dennis Arets, Maikel Heetveld, Eric Arts and Henk van der Laan, thank you for the fun we had during the 13th Age campaign and the Oddylords campaign, as well as the boardgames we have played together over the years. Linde, thanks for the great food you always make for the sessions, and thanks for introducing me to LARPing. René, thank you for joining me in my own campaigns, the nice DnD discussions and Slay the Spire discussions and for bringing baked goods to sessions. Henk, thanks also for your friendship during my student days, and

our time during the bachelor endproject, and for helping me and Ome Donic with my computer.

On that note, also thanks to Haico Dorenbos, Stefan Tanja and Dylan ter Veen for your friendship during my student days. Haico, thanks for the easy relaxation and our summer visits to swimming pools. Stefan, thanks for the great and challenging bike trips.

Nander Bras, thank you for our nice walks, the visits to Utrecht and the Ayreon concerts. Also thank you for meeting up with me and Joep at Joep's big band concerts, and for hosting us in Utrecht. To many more meetings.

Stijn Smits, thank you for the movies we watched during corona and after. Stijn Slot, thank you for the courses we did together and thank you and Ellen Rijtsma for the nice visits and boardgames we played during the start of my PhD. Ivar Leenders and Meike Noordijke, thanks for the times we played DnD together.

Thank you to my old housemates Iris Messing, Iris Brouwer, Tom Frissen, Martijn Hilders and Eloy Jegers from the Laboratoriumstraat, as well as Ellen Kleipool, Arno Daamen and Jeanne Daamen, Annie Schapendonk-de Groot (may she rest in peace) and Jan van Randwijk and my other neighbours in the Florastate. Ralf Schmidt and Team T.E.S.T., Tamás Harcsa, Gijs van de Sanden, Wessel van der Heijden, Geert van Ieperen and Didier Rothuizen, Famke Berg, Hans van de Ven and the people of the St. Lambertus in Engelen, thank you as well.

Done? Not quite. Thanks to all those I may have forgotten to mention, and again thanks to everyone in general. It has been a pleasure to have you in my life and it has been a pleasure to feature in Poekie's life.

Last but not least, I want to thank you, for reading this thesis. Enjoy!

Tom T.P. Franken

Contents

1	Introduction	1
1.1	Complexity of Parallel Programming	2
1.2	Research Questions and Contributions	4
1.3	Origin of Chapters	12
1.4	Related Work	14
I	AuDaLa	21
2	Defining AuDaLa	23
2.1	Concepts	23
2.2	Syntax	30
2.3	Type System	33
2.4	Semantics	40
2.5	Conclusion	52
3	Examples of AuDaLa	53
3.1	Reachability	54
3.2	Creating a Directed Spanning Tree	56
3.3	The 3-SUM Problem	59
3.4	Sorting in AuDaLa	61
3.5	A Turing Machine in AuDaLa	70
3.6	Conclusion	75
4	Properties of AuDaLa	77
4.1	Properties of AuDaLa Programs	77
4.2	Properties of Well-Formed Programs	81
4.3	Corollaries and Type Soundness	95

4.4	Conclusion	100
5	Proving in AuDaLa	101
5.1	Proving Prefix Sum Correct	101
5.2	Proving Linked List Copy Sort Correct	106
5.3	Proving the AuDaLa Turing Machine Implementation Correct .	117
5.4	Conclusion	124
6	Adapting AuDaLa Semantics	125
6.1	Potential Programming Adaptations	127
6.2	The Weak-Memory Semantics	137
6.3	Connecting the Semantics	146
6.4	Conclusion	169
II	AuDaLogic	171
7	Defining AuDaLogic	173
7.1	Preliminaries - Proving Programs with Triples	175
7.2	Defining AuDaLogic Expressions and Predicates	180
7.3	From AuDaLa Syntax to AuDaLogic Predicates	193
7.4	Proof Rules	199
7.5	Proving Linear Sort with AuDaLogic	206
7.6	Conclusion	230
8	Proving AuDaLogic Sound	231
8.1	Connecting the semantics of AuDaLa and AuDaLogic	231
8.2	Correctness of Syntax Processing for AuDaLogic	240
8.3	Soundness of the Proof Rules	250
8.4	Conclusion	269
9	Adapting AuDaLogic for Write-Write Race Conditions	271
9.1	Adapting AuDaLogic for Write-Write Race Conditions	272
9.2	Proof Tree Example for Adapted AuDaLogic	288
9.3	Conclusion	295
10	Conclusion and future work	297
10.1	Discussion of Part I	297
10.2	Discussion of Part II	304
	Appendices	307

Appendix A	Rules Summary	309
A.1	Type System Rules	309
A.2	Operational Semantics Rules	311
A.3	AuDaLogic Proof Rules	313
Appendix B	Glossaries	317
B.1	Part I - AuDaLa	317
B.2	Part II - AuDaLogic	331
Bibliography		343
Summary		361
Samenvatting		363
Curriculum Vitae		365

Chapter 1

Introduction

Since the introduction of the personal computer after the invention of the microchip in 1971 [142], life has changed considerably. To find a location, we now use our phones to display maps and navigate us there. Instead of painstakingly planning out and writing documents on paper, we now use document editors to dynamically write, change and layout documents. The internet and social media have completely changed our interaction with the world, allowing us to send messages that arrive nearly instantaneously, look up information on the fly and communicate with people we have never met and will probably never meet in person within seconds. All of these are a result of the integration of computers in our daily lives.

However, when they were first introduced, computers were leagues away from being capable of what they can be used for today. A large part of this improvement of computers is due to a consistent effort to optimize computer hardware to become faster and smaller. For a long time, these increases in performance were reasonably predictable and guided by two laws, Moore's Law [47, 112, 131] and Dennard Scaling [27, 51]. Moore's Law states that miniaturizing transistors would allow the amount of transistors put on chips to be doubled roughly every two years. Dennard Scaling states that shrinking transistors by half their size can be done without increasing the power requirements, which allows for a 40% increase in frequency per transistor [56]. Following these two laws, the performance of chips doubled roughly every two years.

Dennard Scaling broke down around 2005 [27, 80, 112], which limited the performance gains per generation of smaller transistors. These days, we are also reaching the limits of Moore's Law [112], as the miniaturizing of the transistors is hindered by the laws of physics. Due to this, researchers started to explore other ways for improving computing performance. One way

of doing this is by exploiting parallelism, where multiple processors work at the same time to execute a program as opposed to one processor executing a program sequentially.

To facilitate exploiting parallelism, one needs programming languages that allow parallelism in the first place. Research on how these parallel languages should be designed already started before the 1970s [144]. Early research into parallel programming language design yielded many extensions of existing sequential languages to work in a parallel setting, and also led to some new principles and languages based on these principles. This second group was better connected to the formal theory of parallel programming than the extensions of sequential languages [103].

Additionally, theoretical research was done into which computation models should be used to reason about parallel programming and parallel algorithms [20, 44, 61] and how to approach designing parallel programs [37]. The research into the approaches to the design of parallel programs gave rise to new parallel programming *paradigms*, categories that programming languages can fall under, characterised by a set of principles or assumptions on the programming environment. Like other programming paradigms, many of the parallel programming paradigms are well-suited for specific kinds of programs [174]. Nowadays, many frameworks and languages for parallel programming have been created, and many parallel paradigms have been specified [41, 48, 55, 101].

1.1 Complexity of Parallel Programming

Even with the current state of the art of parallel programming, the problem remains that designing parallel programs, especially high-performance parallel programs, is harder than designing sequential equivalents [5, 39, 41, 67, 70, 107]. One reason for this is that in parallel programming, multiple different parallel processes are executing code at the same time, independently of each other. It follows that where in sequential programming, there is a single, straightforward execution of each program, in parallel programming, there are exponentially many potential executions for any program, depending on the interactions of the independent processors.

It is hard for a programmer to keep track of all possible executions of the program, and all unintended behaviour that may stem from those possible executions, like for example conflicts on memory resources [138]. As an example of this, consider computing the sum of a list of numbers. In sequential programming, this is trivially done by going through the list. However, this only works because only one processor is executing the program. If multiple processors are to be used, the programmer must also keep track of how the list is divided between those processors, and how the processors will combine

their results. If this is done incorrectly, one processor may overwrite the result from another processor in a memory conflict, in which case the final result returned by the program will not be the sum of all numbers in the list.

The complexity of parallel programming is made worse by the current state of the art of parallel programming languages and frameworks, which mostly support *explicit* parallelism [41], which leaves it to the user to divide the tasks and its resources over the parallel elements throughout the whole program. In contrast, *implicit* parallelism divides tasks and resources automatically.

Dividing tasks and resources effectively requires more knowledge about the underlying hardware, memory layout and movement of data than is necessary for sequential programming [67]. Additionally, explicit parallelism management often bleeds into the code for the parallel programs, which in our opinion makes parallel code unintuitive and hard to comprehend. For an example of this, consider the example given in Section 2.1.2. We are of the opinion that the comparison of parallel programming with an ‘arcane skill’ [39] can still be made today, which is corroborated by the call for parallel programming languages at a higher level of abstraction [41, 70, 75].

Another factor in the complexity of parallel programming is that it is hard to establish whether a parallel program is *correct*. As many errors and faults may depend on very specific execution behaviours that almost never happen, it is harder to find all unintended behaviour through testing [183]. Though we can use formal verification instead of testing, this is also harder for parallel programs due the vast amount of possible behaviours they exhibit. Furthermore, the difficulty of parallel testing and verification is further complicated by the many different approaches to the parallelism problem in state of the art languages and frameworks [185], as verification methods are often developed independently of programming languages [21, 115, 185].

The degree of the complexity of the state of the art of current parallel programming research cannot just be attributed to parallelism being a difficult concept. For example, parallel algorithm design research uses relatively simple computational models [11, 97, 114] or abstract machines [19, 44, 61] to find nice parallel algorithms [122, 163]. For some of these models, algorithms designed for them can even be directly implemented [72].

We think that the state of the art of parallel programming research is as complex as it is because, to quote Mattson *et al.* [124], “Parallel programming research is dominated by an engineering perspective: build it, show that ‘it’ works, and move on”. This leads to a large focus on performance in parallel programming research, large enough to create a ‘hardware lottery’ [95], which favours parallel language innovations that have an immediate, short-term performance benefit. Examples of this sentiment are considering performance the only relevant part of parallel computing and programming [111], and a tendency for new innovations in parallel language research to require

benchmarking and an implementation to be considered worthwhile.

This would explain why most state of the art parallel programming languages support explicit parallelism, as that allows for better performance [41]. As defining languages as extensions for existing languages allows users to reuse performance and hardware optimizations, code and libraries already defined for these languages, it would also explain why languages are still often defined as extensions for existing languages like C, C++ and Java [41]. It furthermore explains why these extensions ignore the fact that some existing languages, like C and C++, were originally designed for sequential programming and can be considered ill-suited for massively parallel programming [75, 107].

Instead of keeping the focus of parallel programming on the performance, we agree that it would be better to focus on *programmability*, a measure of how easy it is to program in a language [39, 124]. Additionally, we consider that it would be useful to consider verification during the language design process, to make it easier to check the correctness of the programs created in the language.

1.2 Research Questions and Contributions

In this thesis, we explore other facets of parallel programming than performance to make parallel programming easier. To do this, we believe focusing on programmability is the correct approach. However, instead of trying to bring parallel programming closer to sequential programming [39], we believe it is more important to make parallel programming more intuitive for programmers than it currently is. This will allow them to practice their parallel programming and become more proficient at it without overly relying on sequential programming concepts and conventions. For this, it is good to have a more algorithm-focused view of parallel programming and abstract away from the large focus on hardware, memory and thread management. This leads to a focus on the data over the processes, which suits us well; as a function over time, we consider a process to be inherently sequential, where data is more parallel due to its existence among other data.

To make our view and focus more concrete, we introduce the *data-autonomous* paradigm. In the data-autonomous paradigm, *data elements* not only store data and references locally, but also execute their own code. These executions are always carried out in parallel with other data elements by default, and are governed by a *schedule*. Data elements can cooperate through stored references. Most importantly, the data-autonomous paradigm abstracts away from processors and memory, to focus fully on data elements.

This paradigm leads to a high *separation of concerns*, as the code concerning data structures, algorithms and orchestration is properly separated and

not entwined with memory and process management. Furthermore, it forces programmers to think in parallel, as by default, execution of code by data elements is considered to be parallel. Lastly, it promotes a bottom-up design process, from data element design to orchestration via the schedule. Through these benefits, the paradigm and languages based on the paradigm facilitate an intuitive understanding of how the data interacts in a parallel program and therefore of what happens when the program is executed. We foresee that the paradigm is particularly well suited for programs implementing graph algorithms and other procedures that operate on unstructured data.

This leads us to our first research question for this thesis:

Research Question 1. What does a parallel programming language following the data-autonomous paradigm look like, and what properties does that language have?

As a response to this question, we create the parallel programming language AuDaLa (Autonomous Data Language), the first data-autonomous language. We mainly focus on the theoretical basis, which includes a syntax, a formal operational semantics, a type system, an exploration of the properties of the semantics and the type system and some theoretic extensions to the syntax and semantics.

As we believe verification to be relevant for any parallel programming language, we also answer the following research question:

Research Question 2. Can we create formal methods to prove programs of AuDaLa correct?

To answer this question, we devise both the basis to prove AuDaLa programs correct using the semantics, as well as AuDaLogic, a deductive verification proof system for proving AuDaLa programs correct based on separation logic.

The scope of this thesis is the theoretical foundation and the verification of AuDaLa. The thesis consists of two parts and eight chapters with content, the dependencies of which have been visualised in Figure 1.1.

The first part deals with Research Question 1, while the second part deals with Research Question 2. Generally, the thesis can be read from front to back, which we recommend to get the full picture of AuDaLa and AuDaLogic. If one is mostly interested in AuDaLogic, we recommend skipping Chapters 3, 5 and 6.

If one is mostly interested in hardware and language implementations, we recommend first and foremost the work by Leemrijse *et al.* on AuDaLa [109, 110], which can be introduced by Chapter 2 and Chapter 6.2. Note that the scope of this thesis does not include the implementation of AuDaLa [109]. Though the author of the thesis was present and somewhat involved with the

code during parallel execution. Furthermore, loop termination is undecidable, which means that loop analysis is required to figure out its behavioural impact on parallelly executed code. As the loop may also impact other unrelated code executed in parallel with the loop, this complicates the analysis of all code executed in parallel with a loop.

Therefore, AuDaLa facilitates the understanding of the results of executing a parallel program by its enforced structure and through limiting loops to the schedule, which coordinates the execution of programs in AuDaLa. This way, parallelism is encountered when designing simple functions for the data elements, of which the results are easier to understand, while these functions can be looped in the schedule. AuDaLa furthermore does not prevent resource conflicts from occurring in the simple functions; we leave dealing with them to the programmer, which we consider to help build intuition for parallel programming. We consider this to be possible as without loops, the added complexity of these resource conflicts is limited.

As stated earlier, we consider that the focus of parallel programming on explicit parallelism and the resulting focus on the underlying hardware and the management is opposed to our focus of keeping parallel programming programmable. See also Section 2.1.2. As a design principle, we therefore only allow for minimal hardware, performance and implementation considerations in the design of the language concepts and syntax, to keep the language conceptually simple and programmable. We make two exceptions regarding AuDaLa's independence from hardware and implementations.

Firstly, AuDaLa is meant to be hardware-neutral. This means that we want it to work on GPUs as well as CPUs. To make AuDaLa more suitable for GPUs, we impose the requirement that the local memory of each data element cannot contain dynamically-sized data structures. This way, the memory required per data element will be statically determinable, which makes it easier to conceptualize the implementation for GPUs. Furthermore, this has the added benefit that the data represented by the data elements in AuDaLa must be kept reasonably small, as they can only have finitely many parameters. Without this assumption, a programmer would be able to put arbitrary amounts of data into data elements, which would essentially devolve back into processes as the element ceases to really be defined by the data it contains. In return, however, AuDaLa does not support arrays (or similar dynamic indexing data structures), as that would require an indexer for which the number of parameters can only be determined during runtime. We discuss this further in Section 6.1.

Secondly, the semantics of AuDaLa contains *atomic actions*, actions which execute without interference from other actions, and our atomic actions for reading and writing to variables are reasonably strict. We assume the atomicity is correctly implemented, which would mean that no memory conflicts dependent on access at the same time, like *data races* [77, 138], can occur.

Though this is a strong assumption, taking it allows us to keep the semantics relatively simple, which makes our work on determining the properties of AuDaLa and creating verification methods for AuDaLa more feasible. Similarly, we decided to have a single strict looping mechanism in our schedule, as opposed to a more flexible looping mechanism.

We see two main alternatives for this choice. On the one hand, we can make the semantics more complicated to lift this problem from the implementation from the semantics. Though this could be directly integrated into the standard semantics, we are in favour of doing this similarly to how we integrate the Weak-memory semantics in Section 6.2 and Section 6.3, with an alternative semantics. As a drawback, verification methods would become a lot harder to define and harder to apply if this new semantics is not functionally equivalent in some way to the standard semantics. Secondly, we could lift this problem even further and lift it to the syntax, making it the programmer's problem, for example by using locks. Following from our focus on programmability and our opposition to explicit parallelism, we are not particularly in favour of this approach, but it would be a valid alternative. Investigating these alternatives and their need for them we leave to future work.

Other than these two exceptions, in this thesis and during the design of AuDaLa, we use the shared-memory model to conceptualize executions of AuDaLa as a frame of reference. AuDaLa should however also be usable with other memory models, like for example the message passing model, with perhaps some adaptations. We do not investigate this in the thesis, so we leave this to future work.

We talk about resource conflicts in terms of *race conditions*. These occur when two parallel processes a and b access the same resource x , and at least one of the processes writes to x , which we take to be a . Then, if b writes to x as well, the resulting value of x becomes non-deterministic, since we do not know whether a or b writes first. If b reads from x , the value read is non-deterministic, as we do not know whether a writes first or b reads first. We distinguish race conditions and data races by the fact that race conditions do not access the variable at the same time, while data races do.

We do not prevent occurrences of race conditions to happen in AuDaLa, as we believe they have their uses. One example of such a use is an elegant write-write race condition nomination protocol approach first used by Wijs *et al.* [178] and also used by Martens *et al.* [122] in their bisimulation algorithm.

We then study the properties of AuDaLa, starting with programmability. As discussed previously, programmability is a measure of how easy a programming language is to program in [39, 124]. However, it is currently not a clearly specified measure, with consensus on what is and what is not programmable. To explore programmability in AuDaLa, we have therefore made

our own interpretation of programmability, where we consider AuDaLa to be *programmable* if it is simple, structured and expressive enough to be intuitive to use.

This leads to the following two subquestions:

Subquestion 2. To what extent is AuDaLa simple and structured?

Subquestion 3. How expressive is AuDaLa?

We have designed AuDaLa to be both simple and structured, as discussed above and in Chapter 2. Therefore, to explore Subquestion 2, we demonstrate the structure and simplicity of AuDaLa in Chapter 3, by giving multiple examples of programs created for AuDaLa. This ranges from programs for computing reachability in a directed graph to programs for solving the 3-SUM problem and sorting lists of numbers.

For expressivity, from related work on programming language expressivity [49, 58, 130], we found that there are different ways of approaching expressivity.

On the one hand, a programming language A can be considered more expressive than a programming language B if A allows for the creation of programs that cannot in any way be expressed in B [130]. That is, however, not the full story, because expressivity also includes some kind of judgement on how easy it is to use a programming language. On the other hand, therefore, A can be considered more expressive than B if A makes it easier to express programs than B , for example by having more flexible or powerful language features [58].

We can consider the first measure to be the *relative measure*, and the second measure to be the *practical measure*.

In this thesis, we explore the expressivity of AuDaLa in two ways. Following the formal measure, we first establish an expressivity class for AuDaLa by proving AuDaLa Turing complete. We do this by simulating a Turing machine in AuDaLa in Section 3.5 and proving that simulation to be correct in Section 5.3.

Secondly, following the second measure, in Section 6.1 we identify some possible extensions of AuDaLa which would allow for some programs that would otherwise be hard to express with the standard semantics, following the second part of the expressivity definition. We then create full semantic adaptations to allow for those features.

Another property we study is the verifiability of AuDaLa, for which we focus on the following subquestion:

Subquestion 4. How can we verify AuDaLa programs using the semantics and how difficult is doing so?

In Chapter 4, we take the first steps to answer this question by formalizing some properties of AuDaLa programs. As we are generally interested in AuDaLa programs that make sense, we limit most of these properties to programs which conform to the type system of AuDaLa as given in Chapter 2. The type system makes sure that the program syntax is well-typed, that there is no unwanted overlap between names used in the program and that local variables are declared and used properly. This allows us to make some assumptions on the programs for which we establish the properties. From these properties, we can derive type soundness for our type system, and we also formulate some corollaries to make proving AuDaLa programs a bit more intuitive and high-level.

In Chapter 5, we apply the properties from Chapter 4 to prove our program for *prefix sum*, one of our sorting programs and our implementation of a Turing machine correct. Though these proofs are quite application-dependent and close to the semantics, they already show some standard steps that most AuDaLa proofs should go through. Furthermore, the proof for the Turing machine makes use of a *stutter trace equivalence*, which is a relevant technique to prove execution results equivalent across different semantics.

The last property of AuDaLa we explore is implementability. Generally, as AuDaLa's semantics is finite and completely defined, we can quickly establish that AuDaLa is indeed implementable (as confirmed by Leemrijse *et al.* [109]). However, from further investigation into compiling AuDaLa to CUDA C++, a commonly used parallel programming model and language [69], we found that the standard semantics of AuDaLa is relatively strict and does not allow for common compiler and hardware optimizations, which inhibits the performance of the compiled AuDaLa programs [109]. We therefore explore the following research question:

Subquestion 5. How can we improve AuDaLa's semantics to be more compatible with common compiler and hardware optimisations?

To answer this question, we define a weak-memory semantics in Section 6.2, which is less restrictive than the original semantics and supports more execution optimizations. As a drawback, however, this semantics is more complex than the original semantics. As we want to keep AuDaLa as simple as possible, we also formally prove the original semantics and the weak memory semantics equivalent for AuDaLa programs without read-write race conditions in Section 6.3. This permits programmers to use the easier original semantics for programming and verification, while the implementation can benefit from the performance benefits of the more complex weak-memory semantics.

1.2.2 Contributions of part II

In the second part of the thesis, we reconsider the verification of AuDaLa programs. Though the semantic verification in Chapter 5 does exhibit a structure, it is still lengthy and precise work to prove even the most basic of programs. In this part, we therefore create AuDaLogic, a deductive verification *proof system* for AuDaLa based on concurrent separation logic. This proof system consists of a logic with which we can express properties about AuDaLa states, and inference rules with which we can prove properties of AuDaLa executions.

We formulate three subquestions for this part, each of which is discussed in a separate chapter. The first subquestion is as follows:

Subquestion 6. How can we define a proof system for AuDaLa?

We answer this question in Chapter 7, where we define the syntax and semantics of AuDaLogic using the papers by Vafeiadis and Reynolds [159, 173] as a starting point. This includes the definition of two functions to translate AuDaLa expressions and the stability property to AuDaLogic.

The main differences between conventional concurrent separation logic and AuDaLogic is the addition of the *label triple*, and the fact that AuDaLogic does not use *resource invariants*. The label triple is a construct in the AuDaLogic state reflecting which data elements are currently considered active for the program, which allows us to abstract away from irrelevant data elements as well as irrelevant memory resources.

In conventional separation logic, some resources may be shared between processes, but may only be accessed by one process at a time, which is facilitated by a lock. Using these locks prevents for example data races to happen. Resource invariants guarantee properties of the resources both before and after they are accessed by a process. Currently, AuDaLa does support locks, so AuDaLogic does not use resource invariants.

We give an example of how AuDaLogic can be used in practice in Section 7.5 by proving one of our sorting algorithms correct.

With AuDaLogic fully defined, the next subquestion is the following:

Subquestion 7. Is AuDaLogic *sound*?

Here, soundness is an important property of proof systems, which establishes that whatever can be proven in the proof system is indeed true. For AuDaLogic, this means proving that anything we can prove in AuDaLogic is correct with regards to the semantics of AuDaLa.

In Chapter 8, we prove the soundness of AuDaLogic by connecting the semantics of AuDaLa and AuDaLogic, proving our predicate generation functions correct and then proving the soundness for the proof rules of AuDaLogic. For this, we use the methods first described by Vafeiadis [173], adapted to

fit AuDaLogic and AuDaLa. This is somewhat comparable to the stuttering equivalence proofs of part I, if more complicated.

This gives us a sound proof system for AuDaLa. However, in its original form, the proof system assumes that there are no race conditions in the AuDaLa program on which it is used. As there are programs in AuDaLa which use benevolent write-write race conditions, which can be proven with the semantics, we therefore consider the last subquestion:

Subquestion 8. How can we verify AuDaLa programs with write-write race conditions?

In Chapter 9, we show how to adapt AuDaLogic to allow for write-write race conditions. We demonstrate the adaptation with a small example.

We conclude the thesis with Chapter 10, where we discuss what the next steps are for AuDaLa and AuDaLogic.

1.3 Origin of Chapters

This thesis is divided into two parts, with Part I discussing the basics of AuDaLa and the exploration of its properties, and the second discussing the definition and soundness of AuDaLogic.

The original idea for the data-autonomous paradigm and AuDaLa was originally conceived by the AVVA project group, which consisted of Jan Friso Groote, Thomas Neele, Anton Wijs, Pieter Hijma, Jan Martens and Lars van den Haak.

Part I draws from several publications. Firstly, Chapter 2, 3, 4 and 5, except for the Turing machine program given in Section 3.5 and its proof in Section 5.3 originate from the publication which originally introduced AuDaLa and its journal extension:

- [64] T. T. P. Franken, T. Neele, and J. F. Groote. “An Autonomous Data Language”. In: *Theoretical Aspects of Computing – ICTAC 2023*. Cham: Springer Nature Switzerland, 2023, pp. 158–177. ISBN: 978-3-031-47963-2. DOI: 10.1007/978-3-031-47963-2_11.
- [65] T. T. P. Franken, T. Neele, and J. F. Groote. “The Autonomous Data Language – Concepts, design and formal verification”. In: *Theoretical Computer Science* 1057 (2025), p. 115560. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2025.115560>.

For the thesis, we have polished and slightly extended these contents. First of all, where the journal version given above gave the six properties of well-formed programs in one large theorem ([65, Theorem 7.14]), we have split

them into separate lemmas for the thesis. We furthermore found that the case distinction for the postcondition of operator commands in Property 5 in the journal paper missed a case, which we added in this thesis.

We have additionally polished the proof of the properties, and split the proof of property 5 into more accessible subproofs. We included the new auxiliary Lemma 4.2.19 for property 5 (as given in Lemma 4.2.18). The proof for type soundness, not present in the journal extension, has been included in the thesis.

The Turing machine implementation (Section 3.5) and its correctness proof (Section 5.3), and the semantic adaptations exploring expressivity given in Section 6.1 originate from the following publication and its journal extension:

- [62] T. T. P. Franken and T. Neele. “AuDaLa is Turing Complete”. In: *Formal Techniques for Distributed Objects, Components, and Systems*. Cham: Springer Nature Switzerland, 2024, pp. 221–229. ISBN: 978-3-031-62645-6. DOI: 10.1007/978-3-031-62645-6_12.
- [63] T. T. P. Franken and T. Neele. “Expressivity of AuDaLa: Turing Completeness and Possible Extensions”. In: *Logical Methods in Computer Science* Volume 22, Issue 2 (2026). ISSN: 1860-5974. DOI: 10.46298/lmcs-22(2:5)2026.

We have polished these contents, and made the fact that the Turing completeness proof is a stuttering equivalence proof more explicit than it was in these publications, to connect it to the proofs in Chapter 6.

Lastly, the weak-memory semantics presented in Section 6.2 has been adapted from the weak-memory semantics given in the following publication:

- [110] G. P. Leemrijse, T. T. P. Franken, and T. Neele. “Formalisation of a New Weak Semantics for AuDaLa”. In: *Automated Technology for Verification and Analysis*. Cham: Springer Nature Switzerland, 2025, pp. 93–116. ISBN: 978-3-031-78750-8. DOI: 10.1007/978-3-031-78750-8_5.

For the thesis, we have slightly adapted the semantics as given in the paper, the details of which are discussed in Section 6.2. This was done to remove some small side cases in the proofs of Section 6.3 which would otherwise make the proof harder to follow. The proof found in Section 6.3.2 is a new contribution of the thesis, which proves Theorem 1 of the paper. Due to the use of the adapted properties, we establish in the thesis that the type system is also sound for the adapted weak-memory semantics. Though the author contributed significantly to the proof of Theorem 2 in the paper, Theorem 2 and 3 of the paper and their respective contexts have not been included in the thesis, as the author did not significantly contribute to the axiomatic memory consistency model both theorems apply to.

Part II is unpublished thus far.

1.4 Related Work

In this section, we give an overview of the related work we have explored for the different contributions in this thesis.

1.4.1 The paradigm and AuDaLa language design

Conceptually, the data-autonomous paradigm is closely related to the Parallel Pointer Machine (PPM) [44, 78], which models memory as a graph that is traversed by processors. We consider the PPM to be a main inspiration for the data-autonomous paradigm, and one can consider the paradigm to be based on the PPM. The main difference between the paradigm and the PPM is that we do not consider the processors to be separate from the data as the PPM does, and we do not require that data elements execute synchronously.

The data-autonomous paradigm shares the same focus on data as *message passing* [42, 118, 127] languages like Active Pebbles [179], ParCel-2 [33] and AL-1 [119], but differs in using shared variables instead of synchronisation and messages. It also does not allow the use of data as passive elements, like in the messages of MPI [42].

The data-autonomous paradigm can be considered related to the *coordination paradigm* [8, 71, 148]. Like with data-driven coordination, the data-autonomous paradigm considers the state of computation dependent on the data, their values and their configuration relative to other data. Like with control-driven coordination, the data-autonomous paradigm strongly separates coordination and computation; this is evidenced by the inclusion of a schedule in the data-autonomous paradigm, which can be considered an integrated coordination language.

However, the data-autonomous paradigm is not just concerned with coordination, but also with computation; it requires the execution of code to be done by the data elements, which is distinct from the more process-minded coordination paradigm. By encapsulating both the computation and the coordination, the data-autonomous paradigm also imposes more structure than coordination languages, which are meant to be integrated with other existing base languages when used. Furthermore, the goal of the data-autonomous paradigm is not to define an independent model to be added to base languages [71], but to be a model for fully newly defined parallel programming languages. The two models therefore conceptually differ, but they do share some outlooks.

As AuDaLa follows the data-autonomous paradigm, it follows that it can also be considered related to some coordination languages. It is less related to coordination languages like Linda [37, 38, 71] and KLAIM [50], which are more concerned with how processes access and resolve operations on data in

some shared space. It also does not need to create channels between components, like for example Reo [9].

However, an interesting related coordination language to both AuDaLa and the data-autonomous paradigm is the Gamma language [14, 15, 16] and their chemical reaction model. In this model, data can cooperate in parallel by undergoing ‘reactions’, for which they may need to fulfil certain conditions. This model is very compatible with the data-autonomous paradigm, even though it, following the coordination paradigm, does not specify how the reactions should take place. The standard three problems for implementing the Gamma language, as outlined in [15, Section 2.4], take a more process-minded view, where data is to be manipulated, but that does not rule out a data-autonomous interpretation. On the AuDaLa level, AuDaLa is less abstract, as it requires the user to specify how the ‘reactions’ happen and how the data elements manipulate their references to achieve intended results. Additionally, AuDaLa is far more structured, as evidenced by the immutability of structs during execution. Nevertheless, the two are very related, if distinct ways of trying to tackle a more data centered approach on parallel programming.

Similar to the Γ -language, the paradigm and AuDaLa are related to the Chemical Abstract machine [19] and the Γ -Calculus Parallel Programming Framework [68].

Another model close to the data-autonomous paradigm is the *actor model* by Hewitt [86, 87]. Like actors, to make an AuDaLa program, one must first define the templates for the data elements in the system, and like actors, a data element can contain finitely many references to other data elements. The difference between the model and the data-autonomous paradigm stems from their approach to communication and execution. In the actor model, actors send other actors messages, on which the recipients can act. In comparison, the actions of the data elements in the data-autonomous paradigm is not reactive with regards to other data elements, but governed by a schedule. It follows that the data-autonomous paradigm does not have as large a focus on message passing as the actor model.

It also follows that AuDaLa is related to actor languages, like ABS [99], Ly [172], ParCel-1 [175], POB++ [153] and A-NETL [10]. However, AuDaLa does not use the message passing model for communication, where these languages often do. Of those who do not, OpenABL [46] uses agents similar to data elements, but gives the agents to functions instead of functions to agents. *Active Object* languages [23, 25] do give their objects functions, which is very closely related to data elements. The execution of functions in these objects however, is fully asynchronous: objects can activate other objects by calling methods in them for them to execute. This is less structured than in AuDaLa, in which the functions to be executed are defined in the schedule. As a result, AuDaLa does not currently use futures, unlike many active object languages,

though there may be benefits in considering how the concept of futures may improve AuDaLa.

The motivation of our work is similar to the motivation of the creation of the ICE language [72]. In the paper, the authors essentially bridge the gap between the Parallel Random Access Machine [61], used for parallel algorithm design, and hardware and implementation, with the goal to make parallel programming easier. We can be considered to do something similar, except with a paradigm based mainly on the PPM and defined for parallel programs, instead of with PRAMs. This is evident in the algorithmic focus of AuDaLa code.

In AuDaLa, the relations between data elements can be viewed as a graph, which is also the case for *graph based* languages, such as DDG [171], a scheduling language, and GraphGrind [169], a graph partitioning language. The Connection Machine [89] uses a graph-based hardware architecture for parallel computation. Similarly, the way data is expressed in Legion [17] and OP2 [132] is similar to AuDaLa. However, these two languages work top down from a main process that calls functions on data, which is unlike the data-autonomous paradigm.

The data-autonomous paradigm has similarities with the data-parallel problem decomposition paradigm, except that the data-autonomous paradigm is more extreme with regards to the role of data in the program and the role of parallelism in the program. It follows that AuDaLa shares concepts with other data-parallel languages like CUDA [69, 82] and OpenCL [40]. It has the most in common with object-oriented approaches to data-parallelism, like the POOL family of languages [4], languages in which small elements do parallel computations based on their neighbours, like RELACS [158], PPC [120], Chestnut [167] and the ParCel languages [33, 175], and *actor languages*.

The use of a schedule in the data-autonomous paradigm relates AuDaLa to some more functional data-parallel languages as well. For example, AuDaLa uses a schedule, like Halide [157]. Furthermore, the manipulation of an array in Futhark [85] has some similarity to calling a step in AuDaLa.

1.4.2 Programmability and expressivity

We consider parallel *programmability* to be a measure of how easy it is to actually use the parallel programming language for programming [39, 124].

Though we present no specific measure of programmability, which seems to be a study all on its own, we do consider a part of this to be the *expressivity* of the parallel language, which we define as the measure of what can and cannot be expressed in the language.

As described in Section 1.2.1, we consider there to be at least two aspects of the expressivity of parallel programming languages, following the related

work by Mitchell [130], Felleisen [58] and Davidson [49]; *relative* expressivity and *practical* expressivity.

We took the name relative expressivity from related work, where it is used for example in the context of process algebras [150], fulfilling a similar function as in our context. For parallel languages, though this kind of expressivity of actor languages has been studied before [24] and there is research into suitable Turing machine-like models for concurrency [104, 155, 177], there surprisingly does not seem to be a large focus on relative expressivity research for parallel languages, including work on proving parallel languages Turing complete. In contrast, there is a lot of work on relative expressivity for process calculi and algebras [150, 151].

If we restrict ourselves to proving parallel languages Turing complete, we can think of several reasons why this does not have a large focus in research. One of these reasons can be that many of these languages extend other languages, e.g., CUDA and OpenCL are built upon C++. For these languages, Turing completeness is inherited from their base language. Other languages, for example domain specific languages like Halide [157], are simple by design, and if the functionality is more important than the applicability, Turing completeness may be of lesser importance. Some languages are also not Turing complete on purpose [53, 74], for example to make automated verification decidable. As AuDaLa is not an extension of a sequential language, Turing completeness for AuDaLa is not automatic. It has been noted that Turing completeness also does not establish as much about expressivity as it does in the domain of sequential languages, as Turing completeness does not account for the impact of concurrent and distributive operations, which can lead to differences in behaviour in two Turing complete concurrent languages [54].

Though we agree with this, the fact stands that the full behaviour of a parallel language is a superset of the behaviour of the language in the sequential setting, so establishing AuDaLa to be Turing complete is a good starting point for analysing the expressivity of AuDaLa. Our method for doing so follows the same line as the proof for the Turing completeness of Circa [52]. Other parallel systems that have been proven Turing complete include water systems [84] and asynchronous non-camouflage cellular automata [182].

Beyond Turing completeness, the notion that Turing completeness does not account for the impact of concurrent and distributive operations on behaviour seems to suggest that more can be done to better define relative expressivity for programming languages, but we have not found much work on that beyond the work we base our definition of expressivity off of, especially recent work. We do not know precisely why this is the case, and invite anyone who does have work on this to let us know.

We consider *practical* expressivity to have been studied before [164]. Notably, the fact that it requires some measure of what languages are easy to use

or not directly relate it to the study of programming language usability [121, 134, 136, 162] and program comprehension [108, 181].

1.4.3 Semantic equivalences

In Section 5.3, we prove our implementation of a Turing machine correct by proving that for every Turing machine execution there is an equivalent implementation execution and vice versa. In Chapter 6, we define the weak-memory semantics, and prove it is equivalent to the original semantics for programs without read-write race conditions by proving that for those programs, every execution of the original semantics has an equivalent execution in the weak-memory semantics. In general, we consider that for hardware and optimization purposes, defining a new semantics and connecting it to the standard semantics is a preferred approach to keep the original language simple while still allowing for optimizations and implementations. In Chapter 8, the soundness of a proof rule or a function implies an equivalence between the postcondition of a proof rule and the executions of the code in the proof rule executed from states satisfying the precondition of the proof rule, which also implies an equivalence between the semantics of AuDaLa and the proof system.

In literature, this approach is related to the approach for determining the determinism of parallel languages as outlined by Lu and Scott [116], with the difference that we are not trying to establish determinism between different executions of a program in the same semantics, but correspondence of executions of a program using two distinct semantics. There are multiple papers on trying to equate two semantics with each other that have similarities with our approach [117, 123, 139, 161].

Often, the related work relates a denotational semantics to an operational semantics of the same language [117, 139, 161]. We do not use denotational semantics in our work, instead reasoning about possible executions. However, the step from reasoning about executions to a full denotational semantics is not a large one, for which we would be able to use established notions like the expression result values as defined in Chapter 4.

The paper by Mason and Talcott [123] wants to translate a higher level actor language into a lower level language, which must preserve interaction semantics. They prove this by extracting interaction semantics in a configuration for both languages and proving that the translation and the original program have the same interaction semantics. This is similar to our approach for all of the above mentioned sections; in our case we extract all data and existing data elements from the states of the program which we then use in the proof.

What we do can also be compared to the methods used to prove compiler

optimizations sound [113, 168], with our method being especially similar to the one used by Kundu *et al.* [168]. This is most apparent in the weak-memory semantics equivalence proof of Chapter 6, even though no translation is applied. A difference from the method used by Kundu *et al.* [168] is that we use *stutter trace equivalence* [12] instead of bisimulation. This is due to the fact that when a function is called by the schedule, its execution is asynchronous for the data elements.

1.4.4 Verification of AuDaLa

In this thesis, we verify several of the examples given in Chapter 3. Other than the verification of the Turing machine, which follows our semantic equivalence approach, we consider the proofs of Chapter 5 to be contract-based, as opposed to the AuDaLogic proofs in Part II.

Both the proofs in Chapter 5 and the AuDaLogic proofs are related to *Hoare logic* [6, 91]. The proofs in Chapter 5 revolve around specifying a precondition and finding the results of executing individual steps and chaining them together to establish a postcondition, and AuDaLogic is based on *concurrent separation logic* [29, 31, 32, 141, 143, 154, 159, 160], an extension of Hoare logic and Owicki-Gries logic [145, 146] used for verifying concurrent programs. Separation logic is not the only adaptation of Hoare logic and Owicki-Gries logic for concurrent verification [57, 106, 166, 176], but it is probably the most successful adaptation, and its focus on separating the available context suits our purposes.

Generating the step contracts in the contract-based proofs of Chapter 5 is akin to *symbolic execution* [13, 102], albeit a variant which does not need to consider loops and needs to include parallelism. Our approach, like separation logic, has a local focus: contracts are generated relative to their executing elements, consider only the parameters accessed by their executing elements and work with pointers. However, the way this locality is achieved is orthogonal; where separation logic partitions the memory to achieve locality for processes, our approach partitions the process into the steps and achieves the locality by only considering one step at a time.

For AuDaLa, we make a distinction between write-write race conditions and read-write race conditions, where we consider write-write race conditions benign enough to include and read-write race conditions as harmful. Distinctions between race conditions, whether it is good to have them and methods to find the harmful ones have been discussed before [22, 133, 137].

There is quite a body of related work on separation logic and concurrent separation logic. Many of these are adaptations of separation logic for specific goals [26, 76, 92, 93, 98, 105, 147, 156, 170, 184]. Due to this, there has also been work warning to not create too many different separation logics [149] or

work trying to unify different separation logics, for example in classes [34].

Though AuDaLogic has been created to function as a proof system for AuDaLa, it is still an adaptation of concurrent separation logic. As an adaptation, we would say it is related to *probabilistic separation logic* [76], which also uses the separator to split something other than the heap, though what is split is different. We use it to split parallelism by splitting active elements from each other, however, while the paper splits probabilities. Furthermore, in AuDaLogic, the splits are *fictional* [98]; they are abstractions from the actual state that focus in on the current active elements and the memory that we have assigned to them.

We have created a new separation logic, AuDaLogic, instead of making AuDaLa fit with another, already existing separation logic. This is in contrast to the warnings of too many separation logics as given by Parkinson [149]. In our case, we believe this is justified; we have had to make substantial changes to the core of concurrent separation logic as presented by Vafeiadis [173] to make the resulting logic fit AuDaLa, including an entirely new axis along which to split resources. We agree that finding the right core principles and creating a “right core logic” is important [34, 100, 149], however. We think that for such a core logic, it is important to not be too hardware focused, as while separation logic is great for reasoning about memory resources [154], it is also great for other resources, like the active elements or probabilities [76].

Some related work is also concerned with finding ways to prove separation logics and their proof systems sound [29, 59, 83, 88, 126, 165, 173]. We follow the method first proposed by Vafeiadis [173], which has some similarities to stuttering equivalence proofs. For example, to prove AuDaLogic sound, we essentially compared the traces of AuDaLa to traces implied by the proof system (traces of the proof system semantics) through a safety predicate for which we extract the relevant data from AuDaLa states (configurations).

Interestingly, we did not have to introduce environment transitions, which are introduced by Soares *et al.* [165]. These were introduced to remove a requirement in Vafeiadis’s proof system that a parallel statement $C_1 \parallel C_2$ can only be split over two separated preconditions (and postconditions) P_1 and P_2 if the variables in P_1 and C_2 , and P_2 and C_1 are disjoint. In AuDaLogic, this never comes up, which we attribute to the local statement execution perspective of AuDaLa; variable names do not refer to fixed variables in our fictional memory, but are relative to the struct instance which executes the statement.

For AuDaLogic future work, it is notable that there have also been efforts to make separation logic compatible with automated verification [18, 100, 125].

Part I

AuDaLa

Chapter 2

Defining AuDaLa

In this chapter, we give the basics of AuDaLa. We first discuss the intuition of AuDaLa in Section 2.1. The syntax is given in Section 2.2, and we define a type system for AuDaLa on the syntax in Section 2.3. We then give the first semantics for AuDaLa in Section 2.4 and conclude the chapter in Section 2.5

2.1 Concepts

In this section, we discuss the concepts of AuDaLa. We furthermore design a program for the prefix sum problem in AuDaLa as a motivating example to demonstrate the concepts. We will first discuss the basics, after which we discuss the motivating example. Lastly, we discuss some important design considerations and benefits.

2.1.1 The basics

In AuDaLa, programs are designed using three main components: the *structs*, the *steps* and a *schedule*. For a visual overview of the three main components, their purposes and relation to the other concepts, see Figure 2.1.

The *structs* are used to define the *system* of the program by defining the types of the data elements that can occur in them. The structs in an AuDaLa program are dependent on the context of the program. For example, to create a parallel program that works on a graph, our system can consist of nodes and edges. Defining a struct consists of giving the name of the struct and the parameters available to that struct.

As an example, we can define a struct for nodes and edges as shown in Listing 2.1. Here, the nodes have a *value* parameter, and the edges have a

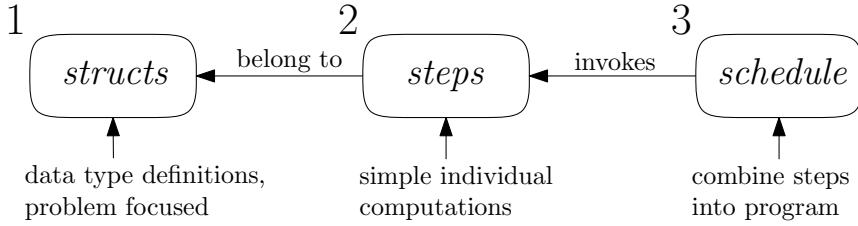


Figure 2.1: The three main components of an AuDaLa program. The numbers signify the order in which they should be finalized in an AuDaLa program.

source and a *target* parameter.

```

1 struct Node(value: Int){}
2 struct Edge(source: Node, target: Node){
3   add {
4     target.value := target.value + source.value;
5   }
6   increment {
7     source.value := source.value + 1;
8   }
9 }
10
11 Fix(increment < add)

```

Listing 2.1: A small working example AuDaLa program

When executing our program, the execution will be done by *struct instances*, which are single elements created from the template as given by a struct. At the start of an AuDaLa program, a special *null-instance* is assumed to exist for every struct. These are special struct instances which represent the boundaries of the system, with each null-instance representing all undefined instances of a specific struct. It can be considered a constant *default* instance of a struct. As a null-instance is a default element, their parameters cannot be changed during execution. One of the things null-instances can be used for is initialisation, since they are the only struct instances that exist at the launch of a program.

Each struct contains zero or more *steps*, which represent operations a struct instance of that struct can do. A step contains simple, algorithmic code, consisting of conditions and assignments, without loops. Struct instances may define and use *local variables* during a step, but those cannot be used by other struct instances and do not carry over between steps. This makes steps finite and easy to reason about. As a struct instance represents a piece of data

executing a piece of code, the struct instance always has access to its own parameters, as defined by the struct. Some of these parameters can be *references*, which refer to other struct instances, which can be followed to access the parameters of those struct instances.

As an example step, we can give our edge a step to add the value of its source node to its target node, and a step to increment the value of its source node. As an example, see the steps *add* and *increment* as shown in Listing 2.1.

The *schedule* prescribes an execution order on the steps. It contains step names and fixpoint operators (*Fix*).

Whenever a step name occurs in the schedule, it is executed in parallel by all struct instances of which the struct contains a definition for that step name. It is also possible to have only the struct types of a single struct instance execute the step by using a specific syntax construct.

The fixpoint operator is our main looping mechanism, containing a nested schedule. It will run the contained schedule until the entire existing system (composed of all existing struct instances) is *stable*. Here, we define *stable* to mean that no additional struct instances have been created nor have any parameters been changed during the execution of the schedule in the fixpoint. If the system is not stable after an iteration of a fixpoint, the fixpoint will be executed again. Once a fixpoint has been run at least once and the last iteration of the fixpoint was stable, the fixpoint will finish executing and the schedule can move on. In this case, we say that the fixpoint *terminates*.

The occurrences of step references and fixpoint operators are separated by synchronization barriers ('<'). Execution only proceeds past a barrier when all computations that precede the barrier have concluded.

For an example schedule, consider the schedule *Fix(increment < add)* in Listing 2.1. In the schedule, we loop over the steps *add* and *increment*. As the value of *source.value* and *target.value* will keep increasing, this loop will never terminate. To have it terminate, we can put a guard in the step using an if-statement.

Note that nested fixpoints are allowed, which corresponds to nested loops in other parallel programming languages. For example, *Fix(Fix(step) < check)* is a schedule which contains a loop in which it will first execute the step *step* until the system is stable, and then execute *check* once. The program will continue with this loop until after the single execution of *check*, the system is stable.

Through the workings of the schedule, it follows that AuDaLa programs are inherently parallel. With some extra effort, however, it is still possible to define sequential programs in AuDaLa as well.

2.1.2 A motivating example - prefix sum

To give an example of these concepts in action, we consider the *prefix sum* problem:

Problem 2.1.1 (Prefix Sum). Given a sequence of n integer values $v_1, \dots, v_n$, compute for each position $1 \leq k \leq n$ in the sequence the sum $\sum_{i=1}^k v_i$. We assume that the sequence is given as a single, finite list without loops.

For example, consider the sequence of integers 3, 4, 2, 1. We would like to compute for each position the sum of all integers up to that position. In our example, this means we would like to compute the sequence 3, 3+4, 3+4+2, 3+4+2+1 = 3, 7, 9, 10.

To design an AuDaLa program for prefix sum, we follow the design structure suggested in Figure 2.1. We omit the initialization of the sequence, as that depends on the exact context in which the program will need to be applied. To that end, our program will be comparable to the design of a parallel function for prefix sum in other programming languages. As we still need to define the structs, we can still conclude things about the system which executes our program, even though we do not know the exact initialization.

In the prefix sum problem, the input is a sequence of integers. We model an element of this sequence with a struct *Position* containing a value *val*. We also give every *Position* a reference to the preceding *Position*, contained in parameter *prev*, as seen in Listing 2.2. This reference is then used to go through the list to gain the values of all previous positions.

For this motivating example, we will assume that during the initialization a single sequence of n integer values without cycles is initialized. We therefore assume the value of *prev* for the first *Position* in the sequence is set to *null*, as it has no previous *Position*. As *prev* is a reference to a *Position*, *null* is a reference to the null-*Position*, which we use in this program to signify the beginning of the sequence. This null-instance has the values 0 for *val* and *null* for *prev*. As we can assume that the original list is given as a single, finite list without loops, we know that every other *Position* can reach the *Position* instance which reflects the first position in the list by following the references stored in *prev*.

```
1 struct Position(val: Int, prev: Position){ [...] }
```

Listing 2.2: Partial AuDaLa code for the structs for Prefix Sum

In Listing 2.3 the steps *read* and *write* of the *Position* struct are shown. These steps are based on the method for computing prefix sum in parallel which was introduced by Hillis and Steele [90]. In Figure 2.2, we show how with these steps, we compute the prefix sum of our example sequence 3, 4, 2, 1.

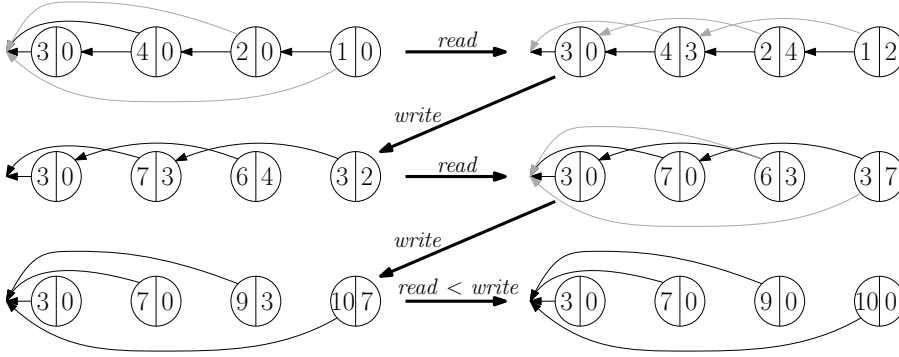


Figure 2.2: Execution of Prefix Sum on a small list containing the example sequence 3, 4, 2, 1. The left side of a list element holds the parameter *val*, while the right side holds the parameter *auxval*. The parameter *prev* is shown as unmarked black arrows, while the parameter *auxprev* is shown as unmarked grey arrows.

As local variables are only defined in the step they are used in, we introduce the additional parameters *auxprev* and *auxval* to be able to carry over the values read in the *read* step to the *write* step.

In the steps, every *Position* first reads *prev.prev* and *prev.val* from their predecessor in the step *read*. After synchronisation, every position updates their *prev* to *prev.prev* and their *val* to *val* + *prev.val* in the step *write*.

By repeating these steps, we get the execution pattern as shown in Figure 2.2. To do so, we put the steps in a fixpoint **Fix**(*read* < *write*) as shown in Listing 2.3. This fixpoint will execute the schedule *read* < *write* until all *Positions* are stable, synchronising in between *read* and *write*.

After one execution of the two steps, the *k*th position will contain the sum $\sum_{i=k-1}^k x_i$, and have its *prev* set to the *k* – 2nd position. The next execution will then combine this with the sum $\sum_{i=k-3}^{k-2} x_i$, as the *k* – 2th Position will also have executed the first iteration of the steps, so the total sum will be $\sum_{i=k-3}^k x_i$, and so on. It follows that after $\log(n)$ iterations of *read* followed by *write*, *prev* is set to null for the *n*th *Position*.

It follows that in $\log(n)$ iterations of *read* followed by *write*, the program will have computed the prefix sum for all positions of the list, as for all *Position* elements, in at most $\log(n)$ iterations, *prev* will be null, and *value* will contain the prefix sum for that *Position* element.

Furthermore, if the previous *Position* is the null-instance, the *Position* executing the step will not add anything to the sum, as the null-value of an integer is defined as 0 in AuDaLa. It will also set its *prev* to null again, as null-parameters are automatically set to null in AuDaLa, so the *prev* of the

null-instance is the null-instance.

As a result, if the *prev* parameter of a *Position* is set to null, the *Position* in question is stable. It follows that after $\log(n)$ iterations, all *Positions* are stable, and the fixpoint terminates.

```

1  struct Position(val: Int, prev: Position, auxval: Int, auxprev: Position){
2      read {                                     /*step definition*/
3          auxval := prev.val;
4          auxprev := prev.prev;
5      }
6      write {                                     /*step definition*/
7          val := val + auxval;
8          prev := auxprev;
9      }}
10
11 Fix(read < write)                             /*schedule*/

```

Listing 2.3: AuDaLa code for Prefix Sum with steps and a schedule

2.1.3 Design considerations

The design of AuDaLa leads to some directly visible benefits. To illustrate this, consider the OpenCL and CUDA kernel implementations of the prefix sum problem in Listings 2.4 and 2.5, which have been used in the literature [40, 82]. As with our program, the initialization is omitted from these programs.

In Listing 2.4, the kernel first copies the input array to the output array, after which it uses an offset variable to check whether an element of the array needs to keep updating itself with other elements. It uses an auxiliary element “temp” to facilitate this update. In Listing 2.5, the kernel uses a single array which is twice as big as the original array to alternate the updates between the front half and the back half of the array. The method is similar to the method of the OpenCL example, but it does need to synchronise less due to the alternation in the array. Both kernels require synchronization barriers in their code, as well as an offset variable to check which data needs to be operated on, against which the thread ids need to be checked multiple times per execution. These hardware-specific concerns are mixed with the more general code of the functions and obfuscate the actual goal of the function. This makes the functions harder to write and understand.

Observe however that these programs have a low *separation of concerns* between hardware and function: the kernels are at the same time concerned with correctly encoding a way to solve prefix sum as well as making sure the data used during this encoding is mapped correctly on the memory. This

```

1 kernel void koggeStone(const local T *in, local T *out) {
2   out[tid] = in[tid];
3   barrier();
4   for (unsigned offset = 1; offset < n; offset *= 2){
5     T temp;
6     if (tid ≥ offset) temp = out[tid - offset];
7     barrier();
8     if (tid ≥ offset) out[tid] = temp ⊕ out[tid];
9     barrier();
10  }}

```

Listing 2.4: OpenCL kernel for Prefix Sum (from [40])

may lead in part to the complexity of the code. In comparison, as illustrated by Listing 2.3 and by Figure 2.1, AuDaLa has a high *separation of concerns*: structs model data and their attributes, steps contain the algorithmic code and the schedule contains the execution. Hardware concerns are abstracted away and replaced with a focus on defining the right system using structs and steps. Because of AuDaLa’s design and separation of concerns, AuDaLa requires no synchronization barriers in the step code, no variables to find the right indices for memory access and no offset variables to avoid going out of bounds. Instead, these are straightforwardly incorporated in the schedule. Though AuDaLa and its enforced method of program structure may still take a bit of practice to get used to, we are confident that the threshold for attaining familiarity with AuDaLa is generally lower than hardware-focused languages like CUDA and OpenCL.

That being said, there are three design considerations which are important for attaining this familiarity as a programmer. First of all, null-instances and null-values in AuDaLa are distinct from the use of null-pointers in other common programming languages. Where null-pointers point to an abstract undefined address in the memory, null-values are defined default values, and every null-instance for a struct is a defined instance which represents the undefined struct instances for that struct. It has the parameters as defined in the struct, all set to null as well. In the case of a parameter with a struct as type, this refers again to the null-instance of that struct, and in the case of a parameter with a basic type, this is a specific defined null-value. This means that reading from a null-instance is possible and does not warrant an interruption to halt the program, like reading null-pointers does in other programming languages.

Secondly, we do not make assumptions about a global execution order of statements executed in parallel. Thus, synchronisation only happens when imposed by a barrier or between iterations of a fixpoint, at the schedule level.


```

1  __global__ void scan(float *g_odata, float *g_idata, int n){
2  extern __shared__ float temp[];
3  int thid = threadIdx.x;
4  int pout = 0, pin = 1;
5  temp[pout*n + thid] = (thid > 0) ? g_idata[thid-1] : 0;
6  __syncthreads();
7  for (int offset = 1; offset < n; offset *= 2){
8      pout = 1 - pout; // swap double buffer indices
9      pin = 1 - pout;
10     if (thid >= offset)
11         temp[pout*n+thid] += temp[pin*n+thid - offset];
12     else
13         temp[pout*n+thid] = temp[pin*n+thid]
14     __syncthreads();
15 }
16 g_odata[thid] = temp[pout*n+thid];
17 }

```

Listing 2.5: CUDA kernel for Prefix Sum (or Scan) (from [82])

Therefore, AuDaLa code is not executed in lock-step, where all struct instances all execute statements together as a single large multi-action. It follows that we allow the occurrence of *race conditions* within the scope of an AuDaLa step.

Race conditions are conflicts which occur when struct instances a and b that want to access the same variable x and at least one struct instance writes to x , for example a . Then, if b writes to x as well, the resulting value of x becomes non-deterministic, since we do not know whether a or b writes first. If b reads from x , the value read is non-deterministic, as we do not know whether a writes first or b reads first. If this cannot lead to undesirable behaviour, we consider the race condition *benign*. For an example of this, see also Section 3.2.

Lastly, though the hardware is abstracted away from the programmer, the intention is for AuDaLa to be implementable on hardware. To keep AuDaLa compatible with GPUs or other structures with many small but simple processors with limited memory, we require that struct instances of AuDaLa only use a fixed amount of memory. To that end, AuDaLa does not allow the use of dynamic parameters and local variables, like dynamic arrays, in structs and steps.

2.2 Syntax

In this section, we give the concrete syntax of AuDaLa. In the definitions below, non-terminals are indicated with $\langle \cdot \rangle$ and terminals with quotes; the empty

word is ϵ . The non-terminal *Id* describes identifiers, and the non-terminal *Type* describes type names, which are either *Int*, *Nat* (natural number), *Bool*, *String* or an identifier (the name of a struct).

An AuDaLa *Program* consists of a list of definitions of *structs* and a schedule:

$$\begin{aligned}\langle Program \rangle &::= \langle Defs \rangle \langle Sched \rangle \\ \langle Defs \rangle &::= \langle Struct \rangle \mid \langle Struct \rangle \langle Defs \rangle\end{aligned}$$

A *struct* definition gives the struct a type name (*Id*), a list of parameters (*Pars*) and a number of steps (*Steps*):

$$\begin{aligned}\langle Struct \rangle &::= \text{'struct'} \langle Id \rangle \text{'('} \langle Pars \rangle \text{' '}' \text{'{'} \langle Steps \rangle \text{'}'}, \\ \langle Pars \rangle &::= \langle Par \rangle \langle ParList \rangle \mid \epsilon \\ \langle ParList \rangle &::= \text{' ,' } \langle Par \rangle \langle ParList \rangle \mid \epsilon \\ \langle Par \rangle &::= \langle Id \rangle \text{' :' } \langle Type \rangle\end{aligned}$$

Here, $\langle Type \rangle$ has the syntax $\langle Type \rangle ::= \langle Id \rangle \mid \text{'String'} \mid \text{'Nat'} \mid \text{'Int'} \mid \text{'Bool'}$, and though the exact syntax of $\langle Id \rangle$ is not important for the results of this thesis and can be exchanged for other syntax, we use the syntax $\langle Id \rangle ::= [\text{A-Za-z}][\text{A-Za-z0-9_}]^*$, which states that identifiers start with a letter (upper case or lower case) and can then consist of zero or more letters, numbers or underscores.

Steps are defined with a step name (*Id*) and a list of statements:

$$\begin{aligned}\langle Steps \rangle &::= \langle Id \rangle \text{'{' } \langle Stats \rangle \text{'}' } \langle Steps \rangle \mid \epsilon \\ \langle Stats \rangle &::= \langle Stat \rangle \langle Stats \rangle \mid \epsilon\end{aligned}$$

A *statement* adheres to the following syntax:

$\langle Stat \rangle ::=$	$\text{'if' } \langle Exp \rangle \text{' then' '{' } \langle Stats \rangle \text{'}' }$	if-then statement
	$\mid \langle Type \rangle \langle Id \rangle \text{' := ' } \langle Exp \rangle \text{' ;' }$	variable assignment
	$\mid \langle Var \rangle \text{' := ' } \langle Exp \rangle \text{' ;' }$	variable update
	$\mid \langle Id \rangle \text{' (' } \langle Exps \rangle \text{')' ;' }$	constructor statement

The *Id* in the variable assignment is a variable name. The constructor statement spawns a new struct instance of the type determined by *Id*, with parameter values determined by the expressions *Exps*. We define the syntax of a list of expressions as:

$$\begin{aligned}\langle Exps \rangle &::= \langle Exp \rangle \langle ExpList \rangle \mid \epsilon \\ \langle ExpList \rangle &::= \text{' ,' } \langle Exp \rangle \langle ExpList \rangle \mid \epsilon\end{aligned}$$

A single expression *Exp* then has the following syntax:

$\langle \text{Exp} \rangle ::=$	$\langle \text{Exp} \rangle \langle \text{BOP} \rangle \langle \text{Exp} \rangle$	binary operator expression
	$ \text{'('} \langle \text{Exp} \rangle \text{'})'}$	brackets
	$ \text{'!'} \langle \text{Exp} \rangle$	negation
	$ \langle \text{Id} \rangle \text{'('} \langle \text{Exps} \rangle \text{'})'}$	constructor expression
	$ \langle \text{Var} \rangle$	variable expression
	$ \langle \text{Literal} \rangle$	literal expression
	$ \text{'null'}$	'null' expression
	$ \text{'this'}$	'this' expression

We consider `this` to refer to the struct instance from which the code which includes `this` is executed.

Binary Operators follow the syntax

$$\langle \text{BOP} \rangle ::= \text{'='} | \text{'!='} | \text{'<='} | \text{'>='} | \text{'<'} | \text{'>'} | \text{'*'} | \text{'/'} | \text{'\%'} | \text{'+'} | \text{'-'} | \text{'^'} | \text{'\&\&'} | \text{'||'},$$

which represent the binary operations for which the notation is commonly used.

A *variable* reference follows the syntax:

$$\langle \text{Var} \rangle ::= \langle \text{Id} \rangle \text{'.'} \langle \text{Var} \rangle | \langle \text{Id} \rangle,$$

where in both cases *Id* is the name of a variable. Through the first case, one can access the parameters of parameters. For example, `prev.prev.val` would have been valid AuDaLa in Listing 2.3, and would access the value of the *Position* before the previous *Position* of the current *Position*.

Although the exact syntax for literals and strings is not important for the content of this thesis, we use the following syntax for literals and strings. Literals can be strings, zero, positive or negative whole numbers and true or false. The string syntax allows any character to occur within the string (bounded by two double quotes) except more double quotes.

$$\begin{aligned} \langle \text{Literal} \rangle &::= \langle \text{String} \rangle | 0 | -?[1-9][0-9]^* | \text{'true'} | \text{'false'} \\ \langle \text{String} \rangle &::= \text{'\"}[^"]^* \text{'\"} \end{aligned}$$

The *Schedule* consists of the variants as given in the following syntax:

$\langle \text{Sched} \rangle ::=$	$\langle \text{Id} \rangle$	step execution
	$ \langle \text{Id} \rangle \text{'.'} \langle \text{Id} \rangle$	typed step execution
	$ \langle \text{Sched} \rangle \text{'<'} \langle \text{Sched} \rangle$	barrier composition
	$ \text{'Fix'} \text{'('} \langle \text{Sched} \rangle \text{'})'}$	fixpoint calculation

The *Id* in the step execution is a step name. In the typed step execution, the first *Id* is a type name, while the second is a step name. The typed step execution is used to schedule a step executed by only one specific struct type.

In the rest of the thesis, we often assume that we are working on an *abstract syntax tree*, a tree representation of the code of an AuDaLa program

generated from the concrete syntax of that program. This allows us to ignore the brackets when working with expressions, and to annotate the occurrences of `null` and `this` with more information for the type system and the semantics. The exact ordering of the expression binary operators is not relevant to the results discussed in the thesis. Therefore, the exact abstract syntax is left to the implementation.

2.3 Type System

Like many other programming languages, we put type constraints and other syntactical constraints on the syntax of an acceptable AuDaLa program. To formalise what is and what is not considered well-formed for an AuDaLa program, we give a *type system* [35, 36, 129, 152] for AuDaLa in this section. With the type system of AuDaLa, we root out standard *type errors*, where there is a mismatch between the expected type of an expression or a variable and the actual type presented, as well as any breach of the following requirements:

1. Identifiers may not be keywords.
2. A step name is declared at most once within each struct definition.
3. A parameter name is used at most once within each struct definition.
4. Names of local variables do not overlap with parameter names of the surrounding struct definition.
5. Variable assignment statements (as defined in the syntax) are only used to declare new local variables.
6. A local variable is only used after its declaration in a variable assignment statement.

We design our type system to encode the above requirements and the requirement that a program should be correctly typed, and formally define what it means for a program to be well-formed:

Definition 2.3.2 (AuDaLa well-formedness). An AuDaLa program $\mathcal{P}$ is *well-formed* iff according to the type rules as presented in this section, $\emptyset, \emptyset \vdash \mathcal{P}$.

In the rest of this section, we give the type rules of the type system, highlighting how it enforces the six requirements given above. It follows that if a program is well-formed, the six requirements above must be met. Later, in Chapter 4, we also prove that the type system is *sound*, which means that if a program is well-formed according to this type system, the program does not contain type errors.

We have seven different categories of rules in our type system:

- The Environment category makes sure that the environment is well-formed, by checking for overlap in variable names, parameters and

struct types in the environment. In this category, we enforce our first requirement and third requirements.

- The Types category processes the types encountered in a program, including those declared in a struct definition. This includes a new meta-type `Step`, which we introduce to be able to check the second requirement of well-formedness. By considering the step names variables with the type `Step` in the type system, we can check for overlap between steps with the type system as we can for any other variable type.
- The Standard Values category encodes our requirements on standard values. Note that `null` can be of any type, as we work with defined null-values and null-instances.
- The Expressions category processes the types encountered in expressions. It contains rules for the binary and unary operators in AuDaLa, variables and constructor expressions.
- The Statements category processes the types encountered in statements, and makes sure that newly declared struct instances and variables are well-typed. For our six requirements, this category makes sure that variable assignment statements are only used to declare new local variables, and that a local variable can only be used after it is declared in a variable assignment statement.
- The Schedule category processes whether a schedule is well-typed by checking if all steps in it are well-typed.
- The Program category ties the whole thing together. It uses the definitions of the program to check the definitions of the struct types, the contents of the statements and the schedule, and to define the environments under which these should be checked. By adding the parameters of a struct type to the environment when checking the statements of a step of that struct type, this category, together with the statements category, prevents local variables to overlap with the parameters of the surrounding struct definition.

We continue with giving some standard notations and definitions, after which we will discuss every category separately.

Notations and definitions

The rules are given using *inference rule notation*, where if the premises above the line hold, the conclusion below the line holds. In these rules, we consider free variables to be quantified universally.

In the type system, but also in the semantics and in the rest of the thesis, we regularly use lists. List concatenation is denoted with a semicolon, and we

identify a singleton list with its only element. The empty list is denoted ε . If we work with lists of statements, we assume that in this list, the statements are also separated by a semicolon, even though if-statements do not have a semicolon at the end in the syntax.

The type rules use a *syntax variable environment* Γ and a *struct type environment* Ω , both of which are ordered lists. The environment Γ consists of pairs of variable names and their types; the variable names present in Γ can be accessed using $\text{dom}(\Gamma)$. The environment Ω saves struct type definitions and their parameters, of which the struct types can be accessed using $\text{dom}(\Omega)$.

We denote struct types using variations of the letter θ . We assume that for every occurrence of `this`, the struct in which it occurs is recorded in such a way that every occurrence of `this` is annotated with the name of its struct, like `this $_{\theta}$` . We also assume that any list, like a list of parameters, is separated by semicolons even if the syntax specifies commas.

We denote schedules using variations of the greek letter Δ . To make working with the schedule easier, we treat occurrences of F_{θ} for some step F as a variable name for a variable of a special newly introduced type `Step`. Here, an occurrence of F_{θ} denotes that F is defined for θ . This is used to enforce the second requirement for well-formedness, which is further discussed in Section 2.3.

We use *Str* to denote any sequence of characters admissible in a string literal and let N_1 and N_2 such that $N_1, N_2 \in \{\text{Nat}, \text{Int}\}$. Additionally, let S be a statement and let $\mathcal{S}$ be a sequence of statements as defined in the syntax. Let E, E_1, E_2 be expressions, let x be a single variable name and let X be a variable following the non-terminal $\langle \text{Var} \rangle$. Lastly, let Δ, Δ_1 and Δ_2 be schedules and let $K = \{\text{this}, \text{if}, \text{then}, \text{null}, \text{Fix}, \text{struct}\}$ be the set of keywords.

The Environment category

The category *Environment* deals with the well-formedness of the environment and the types and struct types contained in it, by checking whether they overlap with each other and with keywords. Here, the symbol $\diamond$ is used to denote that the environments are well-formed. It consists of three rules. The first rule, **Empty**, tells us that if both environments are empty, the environments are well-formed.

$$\text{(Empty)} \frac{}{\emptyset, \emptyset \vdash \diamond}$$

The other two rules, **AddType** and **AddVar**, deal with adding struct types and variables to the struct type environment and the syntax variable environment respectively. These both require that the new variable or type is not a keyword and are not already in the environments. For new types, we also require that the parameters for that struct type are distinct from other pa-

parameters. This way, we enforce the third requirement of well-formedness that parameter names are used at most once in each struct instance definition.

By requiring that the new variables and types are not in the set of keywords K , we implement our first requirement of well-formedness that identifiers may not be keywords. By checking whether all the parameter names are distinct in **AddType**, we furthermore check our third requirement of well-formedness that a parameter name may occur only once in a struct definition.

$$\begin{array}{c}
 \frac{\Gamma, \Omega \vdash \diamond \quad \theta \notin \text{dom}(\Omega) \quad \theta \notin K \quad \forall 1 \leq i < j \leq n. (x_i \neq x_j)}{(\text{AddType}) \quad \Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n) \vdash \diamond} \quad \frac{\Gamma, \Omega \vdash T \quad x \notin \text{dom}(\Gamma) \quad x \notin K}{(\text{AddVar}) \quad \Gamma; x : T, \Omega \vdash \diamond}
 \end{array}$$

The Types category

The category *Types* deals with checking whether types are well-formed and are admitted by the environment. This is also done by three rules, **TBasic**, **TStruct** and **DefStruct**, where **TBasic** checks standard types, **TStruct** checks struct types and **DefStruct** checks the types of the parameters of a struct type when it is declared.

We introduce a new basic type *Step* here, which we use to type step names, which is used to enforce the second requirement of well-formedness.

$$\begin{array}{c}
 (\text{TBasic}) \quad \frac{\Gamma, \Omega \vdash \diamond \quad T \in \{\text{Nat}, \text{Int}, \text{Bool}, \text{String}, \text{Step}\}}{\Gamma, \Omega \vdash T} \\
 (\text{TStruct}) \quad \frac{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \diamond}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta} \\
 (\text{DefStruct}) \quad \frac{\Gamma, \Omega \vdash \theta \quad \forall 1 \leq i \leq n. \Gamma, \Omega \vdash T_i}{\Gamma, \Omega \vdash \theta(x_1 : T_1, \dots, x_n : T_n)}
 \end{array}$$

We use a universal quantifier in the rule **DefStruct**, which is possible as it is bounded by $1 \leq i \leq n$. The quantifier can therefore be unfolded into the separate proof obligations $\Gamma, \Omega \vdash T_1 \dots \Gamma, \Omega \vdash T_n$, which we can process further using the type system.

The Standard Values category

The category *Standard Values* deals with the typing requirements of literals and *null* and *this*. Here, we divide the numbers into natural numbers (0 or larger) and negative integers (less than 0). We do this because natural numbers are generally weakened to integers in our type system, but not the other way around.

$$\begin{array}{c}
\text{(Bool)} \frac{\Gamma, \Omega \vdash \diamond \quad b \in \{\text{true}, \text{false}\}}{\Gamma, \Omega \vdash b : \text{Bool}} \\
\text{(NInt)} \frac{\Gamma, \Omega \vdash \diamond \quad c < 0}{\Gamma, \Omega \vdash c : \text{Int}} \\
\text{(Null)} \frac{\Gamma, \Omega \vdash T}{\Gamma, \Omega \vdash \text{null} : T} \\
\text{(String)} \frac{\Gamma, \Omega \vdash \diamond}{\Gamma, \Omega \vdash \text{"Str"} : \text{String}} \\
\text{(PNat)} \frac{\Gamma, \Omega \vdash \diamond \quad c \geq 0}{\Gamma, \Omega \vdash c : \text{Nat}} \\
\text{(This)} \frac{\Gamma, \Omega \vdash \theta}{\Gamma, \Omega \vdash \text{this}_\theta : \theta}
\end{array}$$

The Expressions category

The category *Expressions* deals with checking whether expressions are well-formed by checking that they are well-typed. We have two rules for arithmetic, which encodes the relation between when something is deemed a natural number or an integer. We assume that arithmetic with integers or arithmetic using subtraction leads to integers, and only preserve the natural number type when no subtraction is used and both numbers are natural numbers. The rule **VarR** deals with when a reference $X.x_i$ is well-typed, which is when X has the type θ for some struct type θ , and x_i is a parameter of θ with the right type as assigned to the parameter in the struct type declaration. The rule **ConsE** deals with constructor expressions, which are well-typed if they have the right struct type and all declared values for the parameters have the right type.

$$\begin{array}{c}
\text{(Eq)} \frac{\Gamma, \Omega \vdash E_1 : T \quad \Gamma, \Omega \vdash E_2 : T \quad \circ \in \{=, !=\}}{\Gamma, \Omega \vdash E_1 = E_2 : \text{Bool}} \\
\text{(NArith)} \frac{\Gamma, \Omega \vdash E_1 : \text{Nat} \quad \Gamma, \Omega \vdash E_2 : \text{Nat} \quad \circ \in \{*, /, \%, +, ^\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Nat}} \\
\text{(BinLog)} \frac{\Gamma, \Omega \vdash E_1 : \text{Bool} \quad \Gamma, \Omega \vdash E_2 : \text{Bool} \quad \circ \in \{\&\&, ||\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Bool}} \\
\text{(Neg)} \frac{\Gamma, \Omega \vdash E : \text{Bool}}{\Gamma, \Omega \vdash !E : \text{Bool}} \\
\text{(Comp)} \frac{\Gamma, \Omega \vdash E_1 : N_1 \quad \Gamma, \Omega \vdash E_2 : N_2 \quad \circ \in \{<=, >=, <, >, =, !=\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Bool}} \\
\text{(IArith)} \frac{\Gamma, \Omega \vdash E_1 : N_1 \quad \Gamma, \Omega \vdash E_2 : N_2 \quad \circ \in \{*, /, \%, +, ^, -\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Int}} \\
\text{(Brackets)} \frac{\Gamma, \Omega \vdash E : T}{\Gamma, \Omega \vdash (E) : T} \\
\text{(VarS)} \frac{\Gamma_1; (x : T); \Gamma_2, \Omega \vdash \diamond}{\Gamma_1; (x : T); \Gamma_2, \Omega \vdash x : T}
\end{array}$$

$$\begin{array}{c}
\text{(VarR)} \frac{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_i : T_i, \dots, x_n : T_n); \Omega' \vdash X : \theta}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_i : T_i, \dots, x_n : T_n); \Omega' \vdash x_i : T_i} \\
\\
\text{(ConsE)} \frac{\forall_{0 \leq i \leq n}. \Gamma, \Omega \vdash E_i : T_i}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta(E_1, \dots, E_n) : \theta}
\end{array}$$

The Statements category

The category *Statements* deals with checking whether statements are well-formed. In AuDaLa, expressions have types, while statements do not, which is exemplified by the two distinct rules for constructor expressions and constructor statements. Furthermore, variable assignment statements have their own sequence rule which adds the newly assigned variable to the environment. As discussed before, we assume that though if-then statements do not have a ‘;’ in the syntax, they will have one in lists of statements in the type system, so there is no separate sequence rule for if-then statements.

$$\begin{array}{c}
\begin{array}{cc}
\text{(IfThen)} \frac{\Gamma, \Omega \vdash E : \text{Bool} \quad \Gamma, \Omega \vdash S}{\Gamma, \Omega \vdash \text{if } E \text{ then } \{S\}} & \text{(LVar)} \frac{\Gamma, \Omega \vdash E : T \quad \Gamma_1; (x : T); \Gamma_2, \Omega \vdash \diamond}{\Gamma, \Omega \vdash Tx := E} \\
\\
\text{(Update)} \frac{\Gamma, \Omega \vdash X : T \quad \Gamma, \Omega \vdash E : T}{\Gamma, \Omega \vdash X := E} & \text{(Seq)} \frac{S \neq 'Tx := E' \quad \Gamma, \Omega \vdash S}{\Gamma, \Omega \vdash S; S} \\
\\
\text{(SeqLVar)} \frac{\Gamma, \Omega \vdash Tx := E \quad \Gamma; (x : T), \Omega \vdash S}{\Gamma, \Omega \vdash Tx := E; S} \\
\\
\text{(ConsS)} \frac{\forall_{0 \leq i \leq n}. \Gamma, \Omega \vdash E_i : T_i}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta(E_1, \dots, E_n)}
\end{array}
\end{array}$$

The rules **LVar** and **SeqLVar** together enforce the fifth requirement for well-formedness, by requiring that the environments are well-formed if the local variable from the assignment statement is added to them. This can only be true if the local variable was not already declared, as then due to **SeqLVar**, the local variable would already be in the variable environment Γ . If the local variable is then added to Γ again, the resulting environment would not be well-formed.

These rules also straightforwardly enforces the sixth requirement, as in **LVar**, the expression of the assignment statement is checked using the local

variable environment which does not yet contain the local variable declared in the assignment statement. The local variable is only added to the variable environment in the rule **SeqLVar**.

The Schedule category

The category *Schedule* deals with checking whether the schedule is well-formed. We do not consider an empty schedule to be well-formed in an AuDaLa program, as in that case the AuDaLa program will never execute anything.

$$\begin{array}{cc}
 (\text{SBar}) \frac{\Gamma, \Omega \vdash \Delta_1 \quad \Gamma, \Omega \vdash \Delta_2}{\Gamma, \Omega \vdash \Delta_1 < \Delta_2} & (\text{SFix}) \frac{\Gamma, \Omega \vdash \Delta}{\Gamma, \Omega \vdash \text{Fix}(\Delta)} \\
 (\text{STypCall}) \frac{\Gamma, \Omega \vdash F_\theta : \text{Step}}{\Gamma, \Omega \vdash \theta.F} & (\text{SNormCall}) \frac{\Gamma, \Omega \vdash F_\theta : \text{Step}}{\Gamma, \Omega \vdash F}
 \end{array}$$

With the rules **STypCall** and **SNormCall**, we check whether the steps occurring in the schedule occur in the environments. For the rule **SNormCall**, the struct type θ is arbitrary. Though as an unbounded variable it is universally quantified, as it only occurs in the premise, the proof rule states that from any struct type θ such that $\Gamma, \Omega \vdash F_\theta : \text{Step}$ we can conclude that $\Gamma, \Omega \vdash F$. It follows that to prove $\Gamma, \Omega \vdash F$, there must exist at least one θ such that $\Gamma, \Omega \vdash F_\theta : \text{Step}$. It follows that θ can be chosen when proving a program well-formed. Note that step names can be processed using the **VarS** rule from the Expressions category.

The Program category

The last category, *Program*, consists of only one rule, **Prog**, which ties our type system together. For this rule, we split a program $\mathcal{P}$ into two syntactic elements: $D_{\mathcal{P}}$, which represents the topmost syntactic *Defs* element and $\Delta_{\mathcal{P}}$, which represents the schedule of $\mathcal{P}$. Let $\text{Structs}(D_{\mathcal{P}})$ be a function that returns struct type definition information from $D_{\mathcal{P}}$, which consists of all struct types and their parameters formatted as $\theta(x_1 : T_1, \dots, x_n : T_n)$. Let $\text{Pars}_{\mathcal{P}}(\theta)$ return the parameters of θ as defined in $D_{\mathcal{P}}$ including type information. Let $\text{Steps}(D_{\mathcal{P}})$ be a function to extract step definition information from $D_{\mathcal{P}}$, consisting of the step and its struct type. We consider these steps to have the type *Step*, so the output is formatted as a list consisting of elements with format $F_\theta : \text{Step}$, where F is a step. Let $\text{StepDef}(D_{\mathcal{P}})$ be a function to extract step definition information coupled with their statements, which outputs a list of elements of

the format $(F_\theta, \mathcal{S})$. The rule **Prog** is given below (label omitted for space):

$$\begin{array}{c}
 \mathcal{P} = D_{\mathcal{P}} \Delta_{\mathcal{P}} \\
 \forall \theta(x_1 : T_1, \dots, x_n : T_n) \in \text{Structs}(D_{\mathcal{P}}). \Gamma, \Omega; \text{Structs}(D_{\mathcal{P}}) \vdash \theta(x_1 : T_1, \dots, x_n : T_n) \\
 \forall (F_\theta, \mathcal{S}) \in \text{StepDef}(D_{\mathcal{P}}). \Gamma; \text{Pars}_{\mathcal{P}}(\theta), \Omega; \text{Structs}(D_{\mathcal{P}}) \vdash \mathcal{S} \\
 \Gamma; \text{Steps}(D_{\mathcal{P}}), \Omega \vdash \Delta_{\mathcal{P}} \\
 \hline
 \Gamma, \Omega \vdash \mathcal{P}
 \end{array}$$

This is the rule which decomposes checking a program into multiple subtasks to check. In this rule, we define that a program consisting of definitions and a schedule is well-formed iff all struct types are defined well (top line), all steps are well defined (second line left) and the schedule is well defined (second line right). Here, we again use universal quantifiers, but these are all bounded, as $\mathcal{P}$ is a finitely sized program and can therefore only have a finite amount of defined struct types and steps.

Combined with the other rules which deal with the **Step** type, this rule enforces the second requirement for well-formedness, that a step is declared at most once within each struct definition, as if this is not the case, $\Gamma; \text{Steps}(D), \Omega \vdash \Delta$ will never be true, as $\Gamma; \text{Steps}(D), \Omega$ will not be well-formed. Furthermore, together with the statement rule **LVar**, this rule enforces the fourth requirement for well-formedness, that names of local variables do not overlap with parameter names of the surrounding struct definition. If there is a local variable name that overlaps with a parameter name of the surrounding struct definition, then, with x being this name and with the local variable assignment statement in the step F as defined for θ , with statements $\mathcal{S}$, the requirement $\Gamma; \text{Pars}_{\mathcal{P}}(\theta), \Omega; \text{Structs}(D) \vdash \mathcal{S}$ would never hold, as x would already be in $\Gamma; \text{Pars}_{\mathcal{P}}(\theta)$, which means that the rule **LVar** can never be applied for the assignment statement that initializes x as a local variable.

With the type system defined and enforcing the six requirements for well-formedness, we now continue by defining the semantics for AuDaLa.

2.4 Semantics

In this section, we present the semantics of AuDaLa. We have chosen to express the semantics of AuDaLa as an interleaving semantics, as if we make the timesteps small enough, we will almost always be able to order simultaneous actions by the exact time they are started. As stack machines suitably expose interleaving between actions, and because they are naturally suited for expressing the evaluation of expressions, we have chosen to express the semantics of AuDaLa as a stack machine.

Throughout this section and in the thesis in general, we frequently make use of *function updates*. For this, we use the following definition:

Definition 2.4.3 (Function update). Given a total function $f : A \rightarrow B$ and $a \in A$ and $b \in B$, then $f[a \mapsto b](a) = b$ and $f[a \mapsto b](x) = f(x)$ for all $x \neq a$.

We extend this to partial functions straightforwardly: If $a \in A$ and $f(a)$ is undefined, then with $b \in B$, $f[a \mapsto b](a) = b$ as well. We lift this operation to sets of updates: $f[\{a_1 \mapsto b_1, a_2 \mapsto b_2, \dots\}] = f[a_1 \mapsto b_1][a_2 \mapsto b_2] \dots$. Since the order of applying updates is relevant, this is only well-defined if the left-hand sides $a_1, a_2, \dots$ are pairwise distinct. If B contains tuples, that is, $B = B_1 \times \dots \times B_n$, we can also update a single element of a tuple: if $f(a) = \langle b_1, \dots, b_n \rangle$, then we define $f[a, i \mapsto b](a) = \langle b_1, \dots, b_{i-1}, b, b_{i+1}, \dots, b_n \rangle$ and $f[a, i \mapsto b](x) = f(x)$ for all $x \neq a$.

We regularly use lists, as defined in Section 2.3. For convenience, we repeat our definition here. List concatenation is denoted with a semicolon, and we identify a singleton list with its only element. The empty list is denoted ε . If we work with lists of statements, we assume that in this list, the statements are also separated by a semicolon, even though if-statements do not have a semicolon at the end in the syntax.

To define the semantics, we start by giving some sets and definitions used to bridge the syntax and the semantics. We then define *commands*, used as primitives for the behaviour encoded in the syntax, and a function from the syntax to commands. We define the states of an AuDaLa program, and then use the commands and the schedule to define operational behaviour of an AuDaLa program. We conclude with Definition 2.4.10.

From syntax to semantics

In our semantics, we make use of several elements of the syntax. First of all, our semantics is defined relative to an AuDaLa program $\mathcal{P}$, and we assume we work on the abstract syntax tree of $\mathcal{P}$. To this end, we do not concern ourselves with operator precedence and parentheses, and we assume that polymorphic elements such as `null` and `42` are labelled with the right type for their context, viz., $null_T$ is the expression *null* of type T .

We have a number of sets containing AST elements: ID is the set of all identifiers, LT is the set of all literals, SC is the set of all schedules, ST is the set of all statements and EX contains all expressions. The set containing all syntactic types is $\mathcal{T} = \{\text{Nat}, \text{Int}, \text{Bool}, \text{String}\} \cup ID$, corresponding to the non-terminal $\langle \text{Type} \rangle$, and the set $O = \{<=, >=, <, >, =, !=, *, /, \%, +, -, \wedge, \&\&, ||\}$ contains all syntactic binary operators, corresponding to the non-terminal $\langle \text{Bop} \rangle$.

While the schedules are represented using their syntax, in the semantics, we do consider schedules to be lists separated by $<$. It follows that schedules can be empty (ε) and that $\Delta = \Delta < \varepsilon$. However, note that in the syntax, an empty schedule is not admitted in the type system, so every program starts with a non-empty schedule.

In our semantics, *labels* reference concrete struct instances (as opposed to struct definitions). We assume some sufficiently large set $\mathcal{L}$ containing these labels. We also have the semantic types $\mathbb{N}$, $\mathbb{Z}$, $\mathbb{B}$ and *String* corresponding to the natural numbers, the integers, the booleans and the set of all strings, respectively. We consider the natural numbers to be a subset of the integers containing all positive numbers and 0. All semantic values are collected in $\mathcal{V} = \mathcal{L} \sqcup \mathbb{Z} \sqcup \mathbb{B} \sqcup \text{String}$ (where $\sqcup$ denotes the disjoint union). The set of all semantic types is $\mathbb{T} = \{\mathcal{L}, \mathbb{N}, \mathbb{Z}, \mathbb{B}, \text{String}\}$, and the set of all semantic operators is $\mathcal{O} = \{=, \neq, \leq, \geq, <, >, *, /, \%, +, -, \wedge, \vee\}$. The semantic value of a literal $g \in LT$ is $\text{val}(g)$. We consider the semantic value of **this** to be the label $\ell \in \mathcal{L}$ of the struct instance executing the line of code in which **this** occurs.

In addition, we assume for every struct type θ the existence of a *null-label* $\ell_\theta^0 \in \mathcal{L}$, so that we can provide a default value for each syntactical type with the function $\text{defaultVal}: \mathbb{T} \rightarrow \mathcal{V}$, defined as:

$$\text{defaultVal}(T) = \begin{cases} 0 & \text{if } T = \text{Nat or } T = \text{Int} \\ \text{false} & \text{if } T = \text{Bool} \\ \varepsilon & \text{if } T = \text{String} \\ \ell_T^0 & \text{if } T \in ID \end{cases}.$$

We define the set of all null-labels to be $\mathcal{L}^0$, with $\mathcal{L}^0 \subset \mathcal{L}$.

Commands and the interpretation function

To facilitate conciseness in our operational semantics, we break down statements and expressions into *commands*, which can be viewed as atomic actions in the semantics. We define commands as follows:

Definition 2.4.4 (Commands). A *command* c is constructed according to the following grammar:

$$c ::= \text{push}(v) \mid \text{rd}(x) \mid \text{wr}(x) \mid \text{cons}(x) \mid \text{if}(C) \mid \text{not} \mid \text{op}(o)$$

where $v \in \mathcal{V} \cup \{\text{this}\}$ is a semantic value or **this**, a special value, $x \in ID$ is an identifier, C is a list of commands, and $o \in \mathcal{O}$ is an operator. The set of all commands is $\mathbb{C}$.

Intuitively, **this** is the semantic equivalent to the syntactic **this**-expression. The precise effect of each command is discussed later in this section when the operational rules are given.

Statements and expressions are compiled into a list of commands according to the following recursive interpretation function:

Definition 2.4.5 (Interpretation function). Let $x, x_1, \dots, x_n \in ID$ be variables, $E, E_1, \dots, E_m \in EX$ expressions, $g \in LT$ a literal, $\theta \in ID$ a struct type, $S \in ST$ a statement, $S \in ST^*$ a list of statements, $T \in \mathcal{T}$ a type and $\circ \in O$ an operator from the syntax. Let the list $x_1; \dots; x_n$ be the list of n variables from x_1 to x_n . We define the *interpretation function* $\llbracket \cdot \rrbracket : ST^* \cup EX^* \rightarrow \mathbb{C}^*$ transforming a list of statements or expressions into a list of commands:

$$\begin{aligned}
\llbracket g \rrbracket &= \mathbf{push}(\mathit{val}(g)) \\
\llbracket \mathbf{this} \rrbracket &= \mathbf{push}(\mathbf{this}) \\
\llbracket \mathbf{null}_T \rrbracket &= \mathbf{push}(\mathit{defaultVal}(T)) \\
\llbracket x_1 \cdots x_n \rrbracket &= \mathbf{push}(\mathbf{this}); \mathbf{rd}(x_1); \dots; \mathbf{rd}(x_n) \\
\llbracket !E \rrbracket &= \llbracket E \rrbracket; \mathbf{not} \\
\llbracket E_1 \circ E_2 \rrbracket &= \llbracket E_1 \rrbracket; \llbracket E_2 \rrbracket; \mathbf{op}(\circ) \\
\llbracket \mathbf{if } E \mathbf{ then } \{S\} \rrbracket &= \llbracket E \rrbracket; \mathbf{if}(\llbracket S \rrbracket) \\
\llbracket T \ x := E \rrbracket &= \llbracket x_1 \cdots x_n. x := E \rrbracket \\
\llbracket x_1 \cdots x_n. x := E \rrbracket &= \llbracket E \rrbracket; \llbracket x_1 \cdots x_n \rrbracket; \mathbf{wr}(x) \\
\llbracket \theta(E_1, \dots, E_m) \rrbracket &= \llbracket E_1 \rrbracket; \dots; \llbracket E_m \rrbracket; \mathbf{cons}(\theta) \\
\llbracket \epsilon \rrbracket &= \epsilon \\
\llbracket S; S \rrbracket &= \llbracket S \rrbracket; \llbracket S \rrbracket
\end{aligned}$$

We fix that if $n = 0$, $\llbracket x_1 \cdots x_n \rrbracket = \mathbf{push}(\mathbf{this})$ as a special case. We require **push(this)** to give us the label of the current executing struct instance at the start of resolving any reference.

States, struct instances and the initial state

We then give the definitions of the structures of our semantics, states and *struct instances*. Struct instances are instantiated from the structs during runtime and can be considered the *data elements* of AuDaLa.

We define them as follows:

Definition 2.4.6 (Struct instance). A *struct instance* is a tuple $\langle \theta, \gamma, \chi, \xi \rangle$ where:

- $\theta \in ID$ is the type of the struct,
- $\gamma \in \mathbb{C}^*$ is a list of commands that are to be executed,
- $\chi \in \mathcal{V}^*$ is a stack that stores values during the evaluation of an expression,
- $\xi : ID \rightarrow \mathcal{V}$ is an environment that stores the values of local variables as well as parameters.

We define $\mathbb{S}$ as the set of all possible struct instances.

A state of a program is the combination of a schedule that remains to be executed, a collection with all the struct instances that currently exist and a

stack of Boolean values that are required to determine whether a fixpoint has been reached. Every label can refer to at most one distinct struct instance. This is formalised in the following definition:

Definition 2.4.7 (State). A *state* is a tuple $\langle \Delta, \sigma, s\chi \rangle$, where:

- $\Delta \in SC$ is a schedule expressed as a list,
- $\sigma : \mathcal{L} \rightarrow \mathbb{S} \cup \{\perp\}$ is a *struct environment*,
- $s\chi \in \mathbb{B}^*$ is a *stability stack*.

The set of all states is defined as $\mathcal{G} = SC \times (\mathcal{L} \rightarrow \mathbb{S} \cup \{\perp\}) \times \mathbb{B}^*$.

Intuitively, the stability stack keeps track of whether a fixpoint should terminate. When starting execution of this fixpoint, a new variable is placed on the stability stack corresponding to this fixpoint. This variable is set to true every time the fixpoint iterates, and is set to false when a parameter is changed during the execution of the fixpoint. Consider for example the code for Prefix Sum (Listing 2.3). In it, the fixpoint for *read < write* will place a single variable on the stability stack, which is reset to *true* at the start of an iteration and only remains *true* if no element updates its *prev* or *val* parameters to new values. Note that the state space is not dependent on the program.

With a notion of states and struct instances, we define *null-instances*, which function as a default instance for every struct:

Definition 2.4.8 (Null-instances). Let $p = \langle \Delta, \sigma, s\chi \rangle \in \mathcal{G}$ be a state. Then the set of *null-instances* in state p is defined as $\{\sigma(\ell) \mid \sigma(\ell) \neq \perp \wedge \ell \in \mathcal{L}^0\}$.

Thus, each struct instance that is labelled with a null-label is a null-instance.

Our initial state of $\mathcal{P}$ is dependent on the definition of $\mathcal{P}$. As in Section 2.3, let our program $\mathcal{P}$ be defined as $\mathcal{P} = D_{\mathcal{P}} \Delta_{\mathcal{P}}$, with $D_{\mathcal{P}}$ the definitions of the program and $\Delta_{\mathcal{P}}$ the schedule of the program. We define $\Theta_{\mathcal{P}} \subseteq ID$ to be the set of all struct types defined in $\mathcal{P}$. The initial variable environment for a struct instance of type θ is ξ_{θ}^0 , defined as $\xi_{\theta}^0(x) = \text{defaultVal}(T)$ for all $x : T \in \text{Pars}_{\mathcal{P}}(\theta)$, where $\text{Pars}_{\mathcal{P}}(\theta)$ returns the parameters of θ in $\mathcal{P}$ including the types of those parameters in a list. For other variables $x \in ID$, $\xi_{\theta}^0(x)$ is left arbitrary.

We then define the initial state of $\mathcal{P}$ as follows:

Definition 2.4.9 (Initial state). The *initial state* of $\mathcal{P}$ is $p_{\mathcal{P}}^0 = \langle \Delta_{\mathcal{P}}, \sigma_{\mathcal{P}}^0, \varepsilon \rangle$, where $\sigma_{\mathcal{P}}^0(\ell_{\theta}^0) = \langle \theta, \varepsilon, \xi_{\theta}^0 \rangle$ for all $\theta \in \Theta_{\mathcal{P}}$ and $\sigma_{\mathcal{P}}^0(\ell) = \perp$ for all other labels.

Intuitively, this definition states that the initial state of a program $\mathcal{P}$ consists of the schedule as found in the program, a struct environment filled with null-instances for every struct type declared in $\mathcal{P}$ and an empty stack.

The transition relation

We define the behaviour of our semantics in terms of a transition relation $\Rightarrow_{\mathcal{P}}$, which is dependent on the program $\mathcal{P}$. We present $\Rightarrow_{\mathcal{P}}$ using *inference rule notation* to give operational rules, where if the premises above the line are satisfied, the transition below the line is enabled. In this relation, we consider every transition to be labelled by the operational rule that induced the transition. The dependence of $\Rightarrow_{\mathcal{P}}$ on $\mathcal{P}$ stems from the dependence on parameter information from $\mathcal{P}$.

There are rules that define the execution of commands and rules for the execution of a schedule. We start with the former, starting with the **push** command. Command **push**(v) pushes value v on the stack χ , and **push**(**this**) pushes the label of the structure instance on χ :

$$\begin{aligned} (\text{ComPush}) \quad & \frac{\sigma(\ell) = \langle \theta, \mathbf{push}(v); \gamma, \chi, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; v, \xi \rangle], s\chi \rangle} \\ (\text{ComPushThis}) \quad & \frac{\sigma(\ell) = \langle \theta, \mathbf{push}(\mathbf{this}); \gamma, \chi, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \ell, \xi \rangle], s\chi \rangle} \end{aligned}$$

The command **rd**(x) reads the value of variable x from environment ξ' of ℓ' and places it onto the stack:

$$(\text{ComRd}) \quad \frac{\begin{array}{l} \sigma(\ell) = \langle \theta, \mathbf{rd}(x); \gamma, \chi; \ell', \xi \rangle \\ \sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \end{array}}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \xi'(x), \xi \rangle], s\chi \rangle}$$

For normal struct instances, **wr**(x) takes a label ℓ' and a value v from the stack and writes this to $\xi'(x)$, the environment of the struct instance corresponding to ℓ' . If x is a parameter (as opposed to a local variable) and writing v changes its value, then any surrounding fixpoint in the schedule becomes unstable. In that case, we set the auxiliary value su (for *stability update*) to false and clear the stability stack by setting it to $s\chi_1 \wedge su; \dots; s\chi_{|s\chi|} \wedge su$. This leaves the stack unchanged if su is true. Below, in the update “ $[\ell', 4 \mapsto \xi'[x \mapsto v]]$ ”, recall that $f[a, i \mapsto b]$ denotes the update of a function that returns a tuple.

$$\begin{aligned} (\text{ComWr}) \quad & \frac{\begin{array}{l} \sigma(\ell) = \langle \theta, \mathbf{wr}(x); \gamma, \chi; v; \ell', \xi \rangle \\ \sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\ \ell' \notin \mathcal{L}^0 \vee x \notin \text{Pars}_{\mathcal{P}}(\theta') \\ su = (x \notin \text{Pars}_{\mathcal{P}}(\theta') \vee \xi'(x) = v) \end{array}}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle][\ell', 4 \mapsto \xi'[x \mapsto v]], \\ & s\chi_1 \wedge su; \dots; s\chi_{|s\chi|} \wedge su \rangle} \end{aligned}$$

The next rule skips the write if the target is a parameter of a null-instance, which ensures that the parameters of a null-instance cannot be changed. This

rule does not apply on local variables, which can be used by null-instances. This is useful for the initialization of a system, for example.

$$\begin{array}{c}
 \sigma(\ell) = \langle \theta, \mathbf{wr}(x); \gamma, \chi; v; \ell', \xi \rangle \\
 \sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\
 \ell' \in \mathcal{L}^0 \wedge x \in \text{Pars}_{\mathcal{P}}(\theta') \\
 \text{(ComWrNSkip)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle], s\chi \rangle}
 \end{array}$$

Let b be a boolean value on the top of the stack χ . A **not** command negates b :

$$\text{(ComNot)} \frac{\sigma(\ell) = \langle \theta, \mathbf{not}; \gamma, \chi; b, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \neg b, \xi \rangle], s\chi \rangle}$$

An **op**($\circ$) command applies the semantic equivalent $\circ \in \mathcal{O}$ of the syntactic operator $\circ \in \mathcal{O}$ to the two values at the top of χ (here taken to be the values a and b), of which the result is put on top of the stack:

$$\text{(ComOp)} \frac{\sigma(\ell) = \langle \theta, \mathbf{op}(\circ); \gamma, \chi; a; b, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; (a \circ b), \xi \rangle], s\chi \rangle}$$

Let $\theta' \in \Theta_{\mathcal{P}}$ be the type of a struct with n parameters. The command **cons**(θ') creates a new struct instance of type θ' in the struct environment σ with a fresh label ℓ' , and initializes the parameters to the top n values of the stack:

$$\begin{array}{c}
 \sigma(\ell) = \langle \theta, \mathbf{cons}(\theta'); \gamma, \chi; v_1; \dots; v_n, \xi \rangle \\
 \text{Pars}_{\mathcal{P}}(\theta') = x_1 : T_1; \dots; x_n : T_n \\
 \sigma(\ell') = \perp \\
 \text{(ComCons)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \gamma, \chi; \ell', \xi \rangle, \\
 \ell' \mapsto \langle \theta', \varepsilon, \varepsilon, \xi_{\theta'}^0[\{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\}]\}], false^{|s\chi|} \rangle}
 \end{array}$$

The command **if**(C) with $C \in \mathbb{C}^*$ adds commands C to the start of γ if the top value of the stack is *true*. If the top value is *false*, the command does nothing:

$$\begin{array}{c}
 \sigma(\ell) = \langle \theta, \mathbf{if}(C); \gamma, \chi; \text{true}, \xi \rangle \\
 \text{(ComIfT)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, C; \gamma, \chi, \xi \rangle], s\chi \rangle} \\
 \sigma(\ell) = \langle \theta, \mathbf{if}(C); \gamma, \chi; \text{false}, \xi \rangle \\
 \text{(ComIfF)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle], s\chi \rangle}
 \end{array}$$

In the remaining rules, let $\text{Done}(\sigma) = \forall \ell. (\sigma(\ell) = \perp \vee \exists \theta, \gamma, \xi. \sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle)$, let sc be a (possibly empty) schedule. The predicate $\text{Done}(\sigma)$ holds when all commands have been executed in all struct instances in σ .

We can initiate steps globally and locally. The global step initiation converts all statements in a step to commands for any structure instance that has that step and adds the commands to γ . Recall that $\mathcal{P} = D \Delta$ and recall from last

section that $StepDef(D)$ returns all pairs (F_θ, S) such that F is a step defined for $\theta \in \Theta_{\mathcal{P}}$ as the list of statements S . Let $S_\theta^F = \varepsilon$ if there is no list of statements S such that $(F_\theta, S) \in StepDef(D)$, and let S_θ^F be a list of statements such that $(F_\theta, S_\theta^F) \in StepDef(D)$ otherwise.

$$\text{(InitG)} \frac{Done(\sigma)}{\langle F < \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \llbracket S_\theta^F \rrbracket, \varepsilon, \xi \rangle \mid \sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle\}], s\chi \rangle}$$

The local step initiation converts the step to commands and adds those commands to γ only for struct instances of a specified struct θ :

$$\text{(InitL)} \frac{Done(\sigma)}{\langle \theta.F < \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \llbracket S_\theta^F \rrbracket, \varepsilon, \xi \rangle \mid \sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle\}], s\chi \rangle}$$

Neither of the step initializations flush the previous local variables from the struct environments. Instead, the type system of AuDaLa as found in Section 2.3 limits the scope of local variables to the steps they are declared in. If required, it can also be enforced in the semantics by defining a function which takes the value of the struct instance environment if the given variable is a parameter, and $\perp$ otherwise.

Let Δ_1 be a schedule. Fixpoints are initiated when first encountered:

$$\text{(FixInit)} \frac{Done(\sigma)}{\langle Fix(\Delta) < \Delta_1, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle}$$

The symbol $aFix$ is a semantic symbol used to denote a fixpoint which has been initiated. When an initiated fixpoint is encountered again, the stability stack is used to determine whether the body should be executed again:

$$\begin{aligned} \text{(FixIter)} & \frac{Done(\sigma)}{\langle aFix(\Delta) < \Delta_1, \sigma, s\chi; false \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle} \\ \text{(FixTerm)} & \frac{Done(\sigma)}{\langle aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle \Rightarrow_{\mathcal{P}} \langle \Delta_1, \sigma, s\chi \rangle} \end{aligned}$$

The full operational semantics

Using the definitions of this section, in particular the states and state space, the initial state and the transition relation, we give an operational semantics for an AuDaLa program $\mathcal{P}$:

Definition 2.4.10 (Operational semantics). The *semantics* of $\mathcal{P}$ is the directed graph $\langle \mathcal{G}, \Rightarrow_{\mathcal{P}}, p_{\mathcal{P}}^0 \rangle$, where $\mathcal{G}$ is the set of all states as defined in Def. 2.4.7, $\Rightarrow_{\mathcal{P}}$ is the labelled transition relation for $\mathcal{P}$ as given above and $p_{\mathcal{P}}^0$ is the initial state of $\mathcal{P}$ as defined in Def. 2.4.9.

2.4.1 Example: prefix sum

As an example of how the semantics works, consider the program PSum in Listing 2.6, which contains the code for Prefix Sum and an initialization function for the sequence 3, 4, 2, 1.

```

1  struct Position(val: Int, prev: Position, auxval: Int, auxprev: Position){
2    read {
3      auxval := prev.val;
4      auxprev := prev.prev;
5    }
6    write {
7      val := val + auxval;
8      prev := auxprev;
9    }
10   init {
11     Position one := Position(3, null, null, null);
12     Position two := Position(4, one, null, null);
13     Position three := Position(2, two, null, null);
14     Position four := Position(1, three, null, null);
15   }
16 }
17
18 init < Fix(read < write)                                     /*schedule*/

```

Listing 2.6: AuDaLa code for Prefix Sum initialized for the sequence 3, 4, 2, 1

For this program, the semantics is the directed graph $\langle \mathcal{G}, \Rightarrow_{\text{PSum}}, p_{\text{PSum}}^0 \rangle$, following Definition 2.4.10. Here $\Rightarrow_{\text{PSum}}$ is the transition relation, with the parameter function $\text{Pars}_{\text{PSum}}$, which is defined as $\text{Pars}_{\text{PSum}}(\text{Position}) = \text{val} : \text{Int}, \text{prev} : \text{Position}, \text{auxval} : \text{Int}, \text{auxprev} : \text{Position}$ and the empty list for all other inputs. Furthermore, the initial state p_{PSum}^0 is the state $\langle \text{init} < \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0, \varepsilon \rangle$, with σ_{PSum}^0 defined as:

$$\sigma_{\text{PSum}}^0(\ell) = \begin{cases} \langle \text{Position}, \varepsilon, \varepsilon, \xi_{\text{Position}}^0 \rangle & \text{if } \ell = \ell_{\text{Position}}^0 \\ \perp & \text{otherwise} \end{cases}$$

and ξ_{Position}^0 defined as:

$$\xi_{\text{Position}}^0(x) = \begin{cases} 0 & \text{if } x = \text{val} \\ \ell_{\text{Position}}^0 & \text{if } x = \text{prev} \\ 0 & \text{if } x = \text{auxval} \\ \ell_{\text{Position}}^0 & \text{if } x = \text{auxprev} \\ \perp & \text{otherwise} \end{cases}.$$

The execution of PSum then follows the transition relation $\Rightarrow_{\text{PSum}}$. As $\text{Done}(\sigma_{\text{PSum}}^0(\ell))$ holds and the first schedule element in the schedule is a step call *init*, only the **InitG** transition relative to *init* is enabled in the initial state. Let the four statements of *init* be represented by S_1 , S_2 , S_3 and S_4 . By applying that rule, we get the state p_1 , which is defined as

$$\langle \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0[\ell_{\text{Position}}^0 \mapsto \langle \text{Position}, \llbracket S_1; S_2; S_3; S_4 \rrbracket, \epsilon, \xi_{\text{Position}}^0 \rangle], \epsilon \rangle.$$

According to the definition of the interpretation function, we know that

$$\begin{aligned} \llbracket S_1; S_2; S_3; S_4 \rrbracket &= \llbracket S_1 \rrbracket; \llbracket S_2 \rrbracket; \llbracket S_3 \rrbracket; \llbracket S_4 \rrbracket \\ \llbracket S_1 \rrbracket &= \text{push}(\text{val}(3)); \text{push}(\text{defaultVal}(\text{Position})); \\ &\quad \text{push}(\text{defaultVal}(\text{Int})); \text{push}(\text{defaultVal}(\text{Position})); \\ &\quad \text{cons}(\text{Position}); \text{push}(\text{this}); \text{wr}(\text{one}); \end{aligned}$$

with $\llbracket S_2 \rrbracket$, $\llbracket S_3 \rrbracket$ and $\llbracket S_4 \rrbracket$ straightforwardly adapted from $\llbracket S_1 \rrbracket$. Then from p_1 , the only transitions we can follow are the many push transitions, processed as defined in the semantic rule **ComPush** and **ComPushThis** until we end up in the state p_2 , defined as

$$\begin{aligned} \langle \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0[\ell_{\text{Position}}^0 \mapsto \\ \langle \text{Position}, \text{cons}(\text{Position}); \text{push}(\text{this}); \text{wr}(\text{one}); \llbracket S_2 \rrbracket; \llbracket S_3 \rrbracket; \llbracket S_4 \rrbracket, \\ 3; \ell_{\text{Position}}^0; 0; \ell_{\text{Position}}^0, \xi_{\text{Position}}^0 \rangle], \epsilon \rangle \end{aligned}$$

Here, we know that the value stack is equal to $3; \ell_{\text{Position}}^0; 0; \ell_{\text{Position}}^0$ by the definition of *defaultVal*.

From p_2 , we can make a **ComCons** transition, creating a new *Position* struct instance, for which we take some fresh label ℓ_1 . This results in the state p_3 , given below.

$$\begin{aligned} \langle \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0[\{ \\ \ell_{\text{Position}}^0 \mapsto \langle \text{Position}, \text{push}(\text{this}); \text{wr}(\text{one}); \llbracket S_2 \rrbracket; \llbracket S_3 \rrbracket; \llbracket S_4 \rrbracket, \ell_1, \\ \xi_{\text{Position}}^0 \rangle \\ \ell_1 \mapsto \langle \text{Position}, \epsilon, \epsilon, \\ \xi_{\text{Position}}^0[\{ \text{val} \mapsto 3, \text{prev} \mapsto \ell_{\text{Position}}^0, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{\text{Position}}^0 \}] \} \\ \}, \epsilon \rangle \end{aligned}$$

On this transition, there are a few remarks. First of all, note that there are actually infinitely many transitions **ComCons**, depending on our choice of label (for now, ℓ_1). As the exact label chosen does not matter, we generally make

sure our definitions and further work on the semantics allow for any of the chosen labels, as can be seen in the definitions in Chapter 4. We shall take p_3 as a representative state for the purpose of this example.

Furthermore, the rule specifies to update the stability stack to a sequence of *false* the length of the stability stack. As the stability stack is empty, it follows that it stays empty.

Lastly, neither the transition nor the semantics as a whole makes sure that the values written to variables are actually compatible with the variables. This is done by the type system instead.

As the new struct instance does not have any commands, the only transitions enabled from p_3 are those of the rest of the commands of S_1 . Resolving **push(this)** then changes the value stack of the struct instance of $\ell_{Position}^0$ to $\ell_1; \ell_{Position}^0$.

After that, the semantics resolve the command **wr(one)** using the transition rule **ComWr**, as *one* is not a parameter according to $\text{Pars}_{\text{PSum}}(\text{Position})$. This transition can only occur because there is a struct instance associated with the label at the top of the stack, in this case $\ell_{Position}^0$. The transition then updates the variable *one* as owned by the struct instance of $\ell_{Position}^0$ to the new value as defined in the value stack, ℓ_1 . This results in state p_5 :

$$\begin{aligned}
 &\langle \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0[\{ \\
 &\quad \ell_{Position}^0 \mapsto \langle \text{Position}, \llbracket S_2 \rrbracket; \llbracket S_3 \rrbracket; \llbracket S_4 \rrbracket, \varepsilon, \\
 &\quad \quad \xi_{Position}^0[\text{one} \mapsto \ell_1] \rangle \\
 &\quad \ell_1 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
 &\quad \quad \xi_{Position}^0[\{ \text{val} \mapsto 3, \text{prev} \mapsto \ell_{Position}^0, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{Position}^0 \}] \rangle \\
 &\quad \}, \varepsilon \rangle
 \end{aligned}$$

The program then continues with S_2 , S_3 and S_4 , of which the execution is similar to S_1 . One exception is that the execution of S_2 , S_3 and S_4 read the label for their previous element from the variable environment of $\ell_{Position}^0$ when required, along the transition **ComRd**. The way this happens is reasonably straightforward, and follows the definition of **ComRd**. Let the new labels chosen for the struct instances created in S_2 , S_3 and S_4 be ℓ_2 , ℓ_3 and ℓ_4 respectively.

After S_4 is executed, the resulting state is p_6 :

$$\begin{aligned}
&\langle \text{Fix}(\text{read} < \text{write}), \sigma_{\mathcal{P}}^0[\{ \\
&\quad \ell_{\text{Position}}^0 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
&\quad \quad \xi_{\text{Position}}^0[\{\text{one} \mapsto \ell_1, \text{two} \mapsto \ell_2, \text{three} \mapsto \ell_3, \text{four} \mapsto \ell_4\}] \rangle \\
&\quad \ell_1 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
&\quad \quad \xi_{\text{Position}}^0[\{\text{val} \mapsto 3, \text{prev} \mapsto \ell_{\text{Position}}^0, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{\text{Position}}^0\}] \rangle \\
&\quad \ell_2 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
&\quad \quad \xi_{\text{Position}}^0[\{\text{val} \mapsto 4, \text{prev} \mapsto \ell^1, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{\text{Position}}^0\}] \rangle \\
&\quad \ell_3 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
&\quad \quad \xi_{\text{Position}}^0[\{\text{val} \mapsto 2, \text{prev} \mapsto \ell^2, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{\text{Position}}^0\}] \rangle \\
&\quad \ell_4 \mapsto \langle \text{Position}, \varepsilon, \varepsilon, \\
&\quad \quad \xi_{\text{Position}}^0[\{\text{val} \mapsto 1, \text{prev} \mapsto \ell^3, \text{auxval} \mapsto 0, \text{auxprev} \mapsto \ell_{\text{Position}}^0\}] \rangle \\
&\quad \}, \varepsilon \rangle
\end{aligned}$$

Let the struct environment as used in p_6 be σ_1 . From the definition of p_6 , it follows that $\text{Done}(\sigma_1)$ holds. The next transitions will then be schedule transitions. In this case, from p_6 , only the transition **FixInit** is allowed, which leads to p_7 , following the definition of **FixInit**:

$$\langle \text{read} < \text{write} < \text{aFix}(\text{read} < \text{write}), \sigma_1, \text{true} \rangle$$

From there, the program will execute the steps of *read* and *write* in a similar manner as how it executed *init*. However, as in this case the commands of a step, for example the first *read* step executed, are loaded in in all the struct instances of σ_1 , the commands executed by the different struct instances can interleave. It is therefore harder to determine the exact effects of *read* and *write* than it is to determine the effects of the initial execution of *init*. Additionally, where in *init* the stability stack was not updated due to it being empty, in the execution of *read* and *write*, the stability stack is updated as defined in the transition rules.

After a single iteration of *read* and *write*, the program will not be stable, and **FixIter** will be used to call for another iteration of the fixpoint. Following Figure 2.2, after three iterations, the system will be stable and the fixpoint operator will be removed from the schedule using the **FixTerm** transition. At that point, program PSum will terminate. For a proof of the exact effects of the non-initialization part of PSum, see also Section 5.1.

2.5 Conclusion

In this chapter, we have discussed the concepts, related work, the syntax, the type system and the first semantics of AuDaLa. The syntax and the type system have been given in full, and we have argued that the type system indeed tests for the requirements we want to enforce other than well-typedness. We furthermore gave the full operational semantics of AuDaLa, as seen in Definition 2.4.10.

In Chapter 4, we shall define important properties for AuDaLa programs based on the type system and the semantics, as well as establish that the type system is *sound*, by proving that a well-formed program cannot contain type errors. While the semantics given according to Definition 2.4.10 is considered to be the *standard* semantics of AuDaLa programs, we adapt this semantics for a better implementation and some other purposes in Chapter 6. In the next chapter, we discuss some examples of AuDaLa programs.

Chapter 3

Examples of AuDaLa

In the last chapter, we defined AuDaLa by giving an overview of its design, a motivating example and defining a syntax, type system and a semantics for the language. In this chapter, we give several examples of AuDaLa code for various applications, using the syntax and semantics of AuDaLa as defined in Chapter 2. By doing this, we demonstrate how to use AuDaLa. We also argue that the examples in the chapter show that AuDaLa is reasonably programmable: the examples show that programming AuDaLa is reasonably simple and structured. In particular, we think that the method for programming in AuDaLa is similar to designing parallel algorithms for an abstract machine.

We first design a program to compute reachability in directed graphs as a second basic example for AuDaLa in Section 3.1. Then, we discuss two more difficult examples by designing a program for creating a spanning tree in Section 3.2 and designing programs for the 3-SUM problem in Section 3.3. Furthermore, we discuss sorting in AuDaLa in Section 3.4. We give a template for implementing arbitrary Turing machines in AuDaLa in Section 3.5, and conclude the chapter with Section 3.6.

Chapter conventions and preliminaries.

In this chapter, we often do not give initialisation code for the programs created, as those are dependent on the specific cases for which the program is applied. Instead, we give expectations of the precondition before applying the program. An exception for this is our program for computing reachability, where we do give an example of what the initialisation looks like for a specific case.

Furthermore, we give the number of struct instances used, the *work* and the *span* of the programs given in this section. Here, *work* and *span* are terms

from analysing parallel algorithms: in our setting, *work* represents the number of executed commands required to execute the program, and *span* is the largest number of commands that have to be executed sequentially during the execution of the program. The amount of struct instances used gives a bound on the required space. Recall that we designed AuDaLa to require that every struct instance uses a finite amount of space, which makes the number of struct instances a usable metric for estimating space requirements.

We use the *big O notation* to give the upper bounds on the growth rate of the struct instances, *work* and *span* as a function of the input of the program. The big *O* notation is used to give upper bounds on functions. For two functions $f(x)$ and $g(x)$, $f(x) = O(g(x))$ iff there exists a positive real number c and a real number a such that $|f(x)| \leq c * g(x)$ for all $x \geq a$. Generally, we abstract away from choosing the a , and just assume that it exists. In this case, we say that $f(x) = O(g(x))$ for large enough x . If for example $f(x) = x + 5$, then $f(x) = O(g(x))$ for large enough x when $g(x) = x$, x^2 or other faster growing functions, but not if for example $g(x) = 5$ or $g(x) = \log(x)/2$. From this point on we omit the “for large enough x ” part.

As an example, the prefix sum program created in Section 2.1.2 used on a list of size n uses $O(n)$ struct instances, as we need one struct instance for every list element. The prefix sum executes $\log n$ sequential iterations. We can bound the commands executed per iteration for every struct instance by a constant, so we can disregard the precise amount of commands for our upper bound. During an iteration, every struct instance executes these commands, so the total amount of commands executed per iteration is bounded by the list size n . It follows that the prefix sum program requires $O(n \log n)$ work and has a span of $O(\log n)$.

3.1 Reachability

In Section 2.1, we discussed how to code a program for the Prefix Sum problem as a motivating example. Another problem for which it is equally simple to create a program in AuDaLa is the *reachability* problem:

Problem 3.1.1 (Reachability). Given a directed graph $G = (V, E)$, where V is the set of vertices and E is a set of directed edges, and a source $v \in V$, compute for all vertices whether they can be reached through a path of edges starting from v .

To solve this problem with an AuDaLa program, we create two structs: *Node* for nodes and *Edge* for edges. We give *Node* a single parameter *reach*, which saves whether the node is reachable, and we give *Edge* two parameters, *source* to save the source node and *target* to save the target node.

```

1 struct Node (reach: Bool) {}
2
3 struct Edge (source: Node, target: Node) {
4   reachability {
5     if (source.reach = true) then {
6       target.reach := true;
7     }
8   }
9 }
10
11 init < Fix(reachability)

```

Listing 3.1: AuDaLa code for a reachability program on a small graph, with example initialization

We then design a step *reach*, which sets the *reach*-parameter of the *target* to true if the *reach*-parameter of the *source* is true. This way, edges can pass on the reachability property from the source to the target. For the schedule, we use *Fix(reach)*, which makes the edges transmit the reachability property until every node that is reachable has been set to reachable. This results in the code displayed in Listing 3.1.

The program uses $O(|V| + |E|)$ struct elements. The span of the program is $O(d)$, with d the *diameter* of the graph: the farthest distance in edges between two nodes. As the reachability property can be passed through one edge at a time, the reachability property needs to be passed on at most d sequential times to guarantee that all possible reachable nodes have been reached. While this means that there can be at most $d + 1$ iterations in AuDaLa, as AuDaLa fixpoints require one additional iteration to establish stability, we do not have to include that iteration in the upper bound as the function $d + 1$ is bounded by $O(d)$. The program then requires $O(d * |E|)$ work, as in at most $d + 1$ iterations, every edge executes a constant number of commands to check if they need to update anything, and then a constant number of commands to update their target if required.

During initialisation, the input should be a directed graph, reflected by the values of the *source* and *target* parameters of the edges. Furthermore, one vertex should be set to reachable. Setting more than one vertex to reachable makes the program a solution for the many-source reachability problem, which has a source set of nodes instead of a single source node. Furthermore, while the code currently assumes a directed graph, the code can easily be extended for an undirected graph, by also setting the *source* vertex's *reach* parameter to true if the *target* vertex *reach* parameter is true.

An execution of the program on a small graph is shown in Figure 3.1, where

the edges and nodes of the graph are modelled by their respective structs. An example of an initialization step for the execution shown in Figure 3.1 is shown in Listing 3.2. Here, the step *init* will be executed by the null-instance of *Edge*, as that is the only *Edge* struct instance to exist when the step is called.

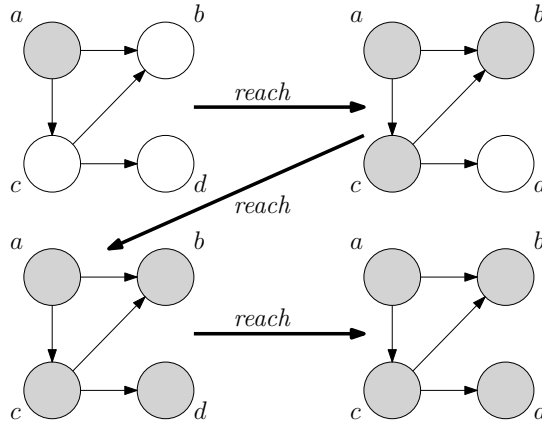


Figure 3.1: Execution of Listing 3.1 on a small graph. Grey Nodes are considered reachable.

```

1  init {
2    Node a := Node(true);
3    Node b := Node(false);
4    Node c := Node(false);
5    Node d := Node(false);
6
7    Edge ab := Edge(a, b);
8    Edge ac := Edge(a, c);
9    Edge cb := Edge(c, b);
10   Edge cd := Edge(c, d);
11 }

```

Listing 3.2: Example initialization step for reachability, which can be added to the struct *Edge* in Listing 3.1.

3.2 Creating a Directed Spanning Tree

Our next example deals with creating a *spanning tree* from a directed graph. A spanning tree of a graph G is a subgraph G' of G such that G' is a tree and

```

1 struct Node(dist: Int, in: Edge){}
2
3 struct Edge(s: Node, t: Node){
4   linkEdge{
5     if s.dist != -1 && t.dist = -1 then {
6       t.in := this;
7     }
8   }
9   handleEdge {
10    if t.in != null then {
11      if t.in = this then {
12        t.dist := s.dist + 1;
13      }
14      if t.in != this then {
15        s := null;
16        t := null;
17      }
18    }
19  }
20  }

```

Listing 3.3: AuDaLa code for creating a spanning tree

G' contains all the nodes of G . Spanning trees have nice properties: they are connected, have $n - 1$ edges for n vertices, they do not have loops and they are useful in other algorithms. Generally spanning trees are undirected, but we consider a directed variant, to keep the code simpler. Extending this to undirected graphs is straightforward.

We define the problem of creating a spanning tree as follows:

Problem 3.2.2 (Oriented Spanning Tree Creation). Given a connected directed graph $G = (V, E)$ and a root node $u \in V$, create a directed spanning tree G' rooted at u .

We create the directed spanning tree using *breadth-first search*. This means we will consider the nodes in the order of how many edges we need to traverse to reach the node from the source. In this tree, for every node v , the path from u to v is a shortest path in G . We do this by incrementally adding nodes from G with a higher distance to u to the spanning tree.

We first sketch our approach. In the i th breadth-first search iteration, the program adds all edges (s, t) to the tree such that the distance from u to s is $i - 1$ and the distance from u to t is still unknown. If multiple such edges lead to the same t , the program uses a race condition to determine which edge is chosen. As any edge will suffice, this race condition is benign. The distance

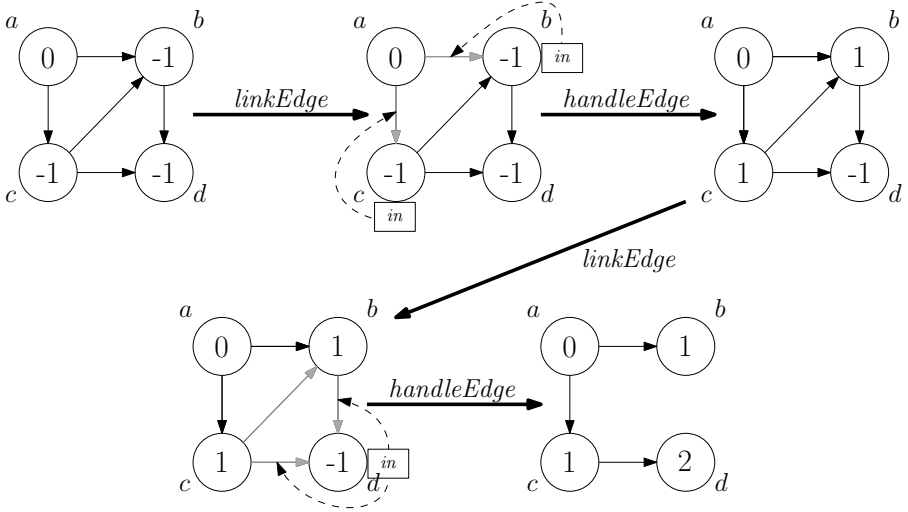


Figure 3.2: Execution of Listing 3.3 on a small graph.

from u to t is then set to i and we continue with the next iteration.

Contained in Listing 3.3 is an AuDaLa program that implements this approach. The program defines the struct `Node` (line 1) with parameters `dist`, to store the distance from root node u , and `in`, a reference to its incoming spanning tree edge. The struct `Edge` (line 3) has a source s and a target t .

During initialisation, the input should be a directed graph, with a root `Node` u with `dist` 0 and with the `dist` parameter of the other `Nodes` set to -1 . For every `Node`, the parameter `in` should be set to null.

Both steps in the program belong to `Edge`. The first step, `linkEdge`, first determines whether an `Edge` e from `Node` s to `Node` t is at the frontier of the tree in line 5. This is the case when s is in the tree, but t is not. If so, e nominates itself as the `Edge` connecting t to the tree, $t.in$ (line 6). This is a race condition won by only one edge for t , the edge which applies the semantic rule **ComWr** last. In the second step, `handleEdge`, if the nomination for t has finished (line 10) and e has won the nomination (line 11), e will update t 's distance to the root. If e has lost, it will remove itself from the graph, here coded as setting the source and target parameters to `null` in lines 14 to 18.

To create a full spanning tree, this must be executed until all `Nodes` have a positive `dist` and all `Edges` are either $t.in$ for their target `Node` t or have `null` as their source and target. To this end, the schedule (line 20) contains a fixpoint, in which `Edges` first nominate themselves and then update the distances of new `Nodes`. This fixpoint terminates, as `Edges` in the spanning tree will continuously update their targets with the same information and `Edges` which lost their

nomination will not get past the first conditions of the two steps, causing the data elements to stabilize after all *Nodes* have received a distance from u . Initialization steps should be placed before the shown schedule. An execution of the program on a small graph is shown in Figure 3.2, where the edges and nodes of the graph are modelled by their respective structs. In the example, every *Edge* newly considered in the current step is grey. Considered *Edges* stay considered, but stable. The dotted arrows denote the possible new values for $t.in$ of a target node t . Note that the *Edge* from c to d wins the race condition to the reference $d.in$.

The program uses $O(|V| + |E|)$ struct instances. The span of the program is $O(d)$, with d the diameter of the graph, as in the worst case all edges in the path which determines the diameter of the graph need to be considered sequentially. It follows that the worst case running time for the algorithm is $O(d)$. The amount of work performed by this program is $O(d * |E|)$.

3.3 The 3-SUM Problem

In this section, we create two programs for the 3-SUM problem [66, 94]:

Problem 3.3.3. Given a set S of n integers, are there three elements $a, b, c \in S$ such that $a + b + c = 0$? These elements a, b and c do not need to be distinct.

The first program, in Listing 3.4, contains a naive solution, and uses a nested fixpoint. This serves as an example of the use of nested fixpoints. The second program, in Listing 3.4 solves this in linear time (but with $O(n^2)$ work). This program is based upon the program given by Hoffman [94]. Both programs require n struct instances.

We assume that the set has been given to us as a list. If not, we can convert the set to a list in linear time using a fixpoint in which we nominate an element still in the set and then put it at the back of the list. Furthermore, our second program assumes that this list is sorted, which can be achieved by applying the example code from Section 3.4.1 or 3.4.2.

Both programs define the struct *Elem*, which represents an element of the list. In the first program, it has a value *val*, a reference to the next element *next*, a reference to the first element of the list *first* and a reference to two other mutable elements, *p1* and *p2*. They also have a parameter *ans*, which holds the answer to the question. After initialisation, *val*, *next* and *first* are set as specified above. The parameter *ans* is set to *false* and *p1* and *p2* are set to *first*.

To solve the 3SUM problem, the first program will make every list element a walk through the list in a nested fashion to find elements b and c s.t a, b and c are not the null-*Element*. For every element b chosen by a , a will walk through the list to find c , if it exists. If not, it will replace b with its

```

1  struct Elem(val: Int, next: Elem, first: Elem, p1: Elem, p2: Elem, ans: Bool){
2    next{
3      if !ans then {
4        p1 := p1.next;
5        p2 := first;
6      }
7    }
8    walk {
9      if p1 != null && p2 != null then {
10       if val + p1.val + p2.val = 0 then {
11         ans := true;
12       }
13     }
14     if !ans then {
15       p2 := p2.next
16     }
17   }
18 }
19 Fix(Fix(walk) < next)

```

Listing 3.4: AuDaLa implementation for naive 3SUM solution

next element in the step *next*. This program performs $O(n^3)$ work (n elements check at most n^2 combinations), with a span of $O(n^2)$: in the worst case, where no triple exists, $O(n)$ elements in the list need to be traversed $O(n)$ times. The code therefore executes in $O(n^2)$ time. Note that this program contains a nested fixpoint: every iteration of the outer fixpoint, the elements will walk through the entire list in the inner fixpoint, and then replace *b* with the next in next after having done so. The inner fixpoint becomes stable when a suitable *b* is found, or the whole list has been traversed (as then the null-instance is reached).

In the second program (Listing 3.5), *Elem* has the parameters *next* and *prev* which refer to those respective elements in the list. It also has candidate references *p1* and *p2*, initialized as the first and last element in the list respectively, a value *val* and an integer *ans* to give the answer to the problem (initialized to 0).

The second program first computes the current value of the three saved elements. If the current value is 0, a solution is found and *ans* can be set to 1. Otherwise, it will either move *p1* to a node with a larger value or *p2* to a node with a smaller value, depending on the current value. If *p1* or *p2* passes the current node, then either there is a solution with the current node as one of the side nodes or there is no solution with the current node. Either way, the node may stop checking for a solution. This is reflected in *ans* being set

```

1 struct Elem(val: Int, next: Elem, prev: Elem, p1: Elem, p2: Elem, ans: Int){
2   iterate {
3     if ans = 0 then {
4       Int cur := val + p1.val + p2.val;
5       if (p2 = prev || p1 = next) then {
6         ans := -1;
7       }
8       if cur = 0 then {
9         ans = 1;
10      }
11      if ans = 0 && cur > 0 then {
12        p2 := p2.prev;
13      }
14      if ans = 0 && cur < 0 then {
15        p1 := p1.next;
16      }
17    }
18  }
19 }
20
21 Fix(iterate)

```

Listing 3.5: AuDaLa implementation for a more efficient 3SUM solution

to -1. At the end of the program, if there is a solution, then there will be at least one node per solution with *ans* set to 1, which is the middle node of the solution. We have depicted the fixpoint process for a single element in the list in the second program in Figure 3.3.

This program performs $O(n^2)$ work; every *Elem* does a single (possibly incomplete) walk through the list. The span is $O(n)$, constrained by a walk through the list of an *Elem*. The total running time is then $O(n)$.

3.4 Sorting in AuDaLa

In this section, we discuss sorting in AuDaLa, which we define as follows:

Problem 3.4.4 (Sorting). Given a list of values $L_1 = v_1, \dots, v_n$ such that $v_1, \dots, v_n$ are distinct, positive values, output a list of values $L_2 = v'_1, \dots, v'_n$ such that $v'_1 < \dots < v'_n$ and for all $v_i \in L_1$, it holds that $v_i \in L_2$.

We give two programs to solve this problem in AuDaLa. The first program, presented in Section 3.4.1, has all elements of the list walk through the list to find the order, after which they rearrange themselves. This program has a span

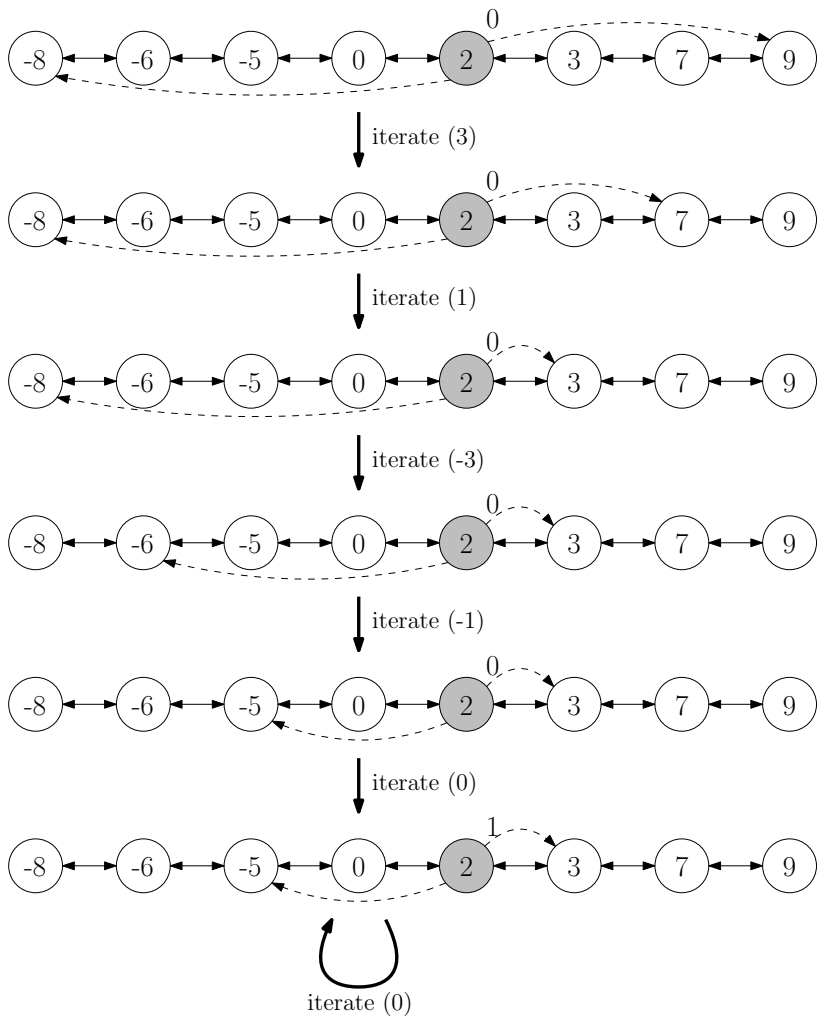


Figure 3.3: Execution of Listing 3.5 for a single element on a small list. The element considered is grey. The upper arrow points at $p2$, and the lower arrow points at $p1$. The value ans has been displayed above and right of the element, and the computed value cur for an iterate step has been displayed next to the iterate steps.

```

1 struct ListElem(val: Int, next: ListElem, newNext: ListElem, comp: ListElem){
2   compareElement {
3     if comp != null then {
4       if (comp.val >= val && ((newNext = null) || comp.val < newNext.val)) then {
5         newNext := comp;
6       }
7       comp := comp.next;
8     }
9   }
10  reorder {
11    next := newNext;
12  }
13 }
14
15 Fix(compareElement) < reorder

```

Listing 3.6: AuDaLa code for sorting

(and running time) of $O(n)$. The second program, presented in Section 3.4.2, adapts binary search to dynamically create a new, sorted list from the old list. This program has a span (and running time) of $O(\log m)$, where m is the largest value in the list.

Furthermore, we have investigated implementing Cole's Algorithm [43] in AuDaLa. Though this work is not finished, we have added a discussion of our findings in Section 3.4.3.

3.4.1 Linear-time sorting

A concise example of an AuDaLa program for sorting a linked list of n elements can be found in Listing 3.6. In it, the elements of the list traverse the list together, during which each element e is looking for its successor in the sorted list. After the traversal, the successor element is saved and the link is updated to the saved element. This reorders the list to the sorted list.

The program defines the struct *ListElem*, modeling the nodes of the list, with parameters *val*, *next*, a reference to the next *ListElem*, *newNext*, a reference to the *ListElem* that should come next in the sorted list, and *comp*, a reference to the current *ListElem* *newNext* is compared to. The initialisation needs to make sure that for every element in the list, there is a corresponding *ListElem* with the right value and the right next *ListElem*, and that in every element, *comp* is set to the first element of the list and *newNext* is set to *null*.

To facilitate our strategy we give our *ListElem* two steps, one to check an element in the list called *compareElement* and one to reorder the list at the end

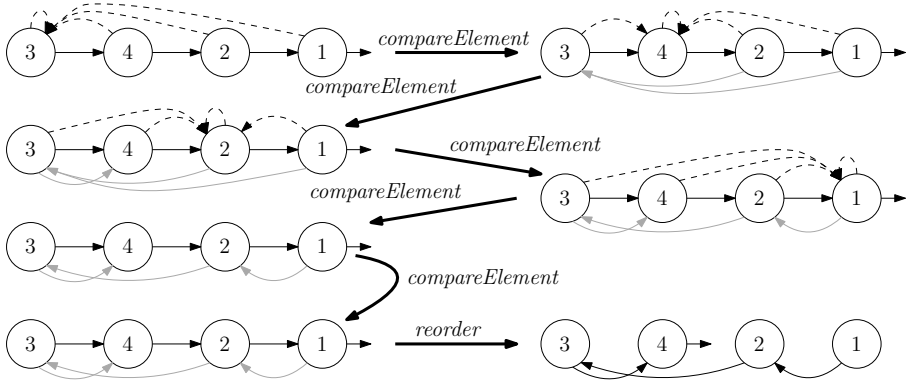


Figure 3.4: Execution of Listing 3.6 on a small list. The parameters *next*, *newNext* and *comp* are shown as black, grey and dashed unmarked arrows respectively, and the null-references for *newNext* and *comp* are not shown. The nodes corresponding to *ListElems* contain the value of parameter *val*.

called *reorder*. With the step *compareElement*, an element checks whether the element to which the *comp* reference leads is a better next element than the current element saved in *newNext* (line 4) and updates *newNext* if that is the case. Afterwards, the *comp* reference is updated to the next element in the list (line 7). With the *reorder* step, an element replaces their old *next* reference with *newNext* (line 11).

To have the program execute our strategy, we call a fixpoint on *compareElement*, such that every element checks all elements in the list. After that is done, the schedule tells the elements to *reorder* (line 15). An execution of the program on a small list is shown in Figure 3.4.

The program requires n struct instances, one for every list element. It performs $O(n^2)$ work, as every list element is compared to every other list element. Its span is $O(n)$, as it walks through a list of size n sequentially. Therefore, the program runs in $O(n)$ time.

3.4.2 Linked List Copy Sort

A more complex example for sorting a linked list of n elements in AuDaLa can be found in Listing 3.7, which makes a sorted copy of the given list. The idea of the algorithm is to do tree-insert for every data element at the same time, using a tree of size $O(\log m)$, which is dynamically created and repurposed to use as few new data elements as possible.

At the start of execution, this copy consists of a single element, which encompasses the full range of the old list. In every iteration, all elements of

the new list split and divide their range (when necessary), and elements from the old list linked to an element of a new list redivide themselves over that element and its new split. This continues until every new element has exactly one old element linked to it. At that point, the new list is a sorted copy of the old list.

The program defines the struct *OldElem*, which represents an element of the old list, and *NewElem*, which represents an element of the new list. Struct *OldElem* has parameters *val*, which holds its value, *place*, which holds the *NewElem* it is currently linked to, and *move*, which saves whether the old element should move to the right neighbour of its current *place* during a split. Struct *NewElem* has the parameters *min*, *spl* (split) and *max*, which hold the current range of the new element and the value it will split the range on during the next split. Additionally, it has parameters *next*, which holds the next new element in the list, *p1*, which holds a representative old element which is in the first half of its range, *p2*, which holds a representative for the second half of its range, *hasSplit*, which signals to the old elements that the new element has split and *done*, which saves whether the element will have to split again.

During initialisation, there should be a single new element *b* to which all old elements of the linked list are linked. For all old elements, *move* should be set to *false*. For *b*, *min* should be set to the minimum value of the linked list, *max* should be set to the maximum value of the linked list, *spl* should be set to $(\max - \min + 1)/2 + \min$. The parameter *hasSplit* should be *false*, *done* should be *true* and *next*, *p1* and *p2* should be set to *null*. Finding *max* and *min* can be done with a method akin to Prefix Sum.

To facilitate the strategy of splitting the new elements and dividing the old elements after the split, we use three steps. The first step, *checkStable* is used to check whether a new element should split its range further. This is the case if there is still an element on the right side of the range of a new element or if there are multiple elements still on the left side of the range of a new element, which the step encodes. For the first part of the second step, *migrate*, let *e* be a new element which just split into *e* and *e*₂. In the first part of *migrate*, the old elements distribute themselves over *e*₁ and *e*₂ by either moving to *e*₂ or staying at *e*. In the second part of *migrate* the old elements prepare themselves for the next split and distribute themselves over the left and right sides of the range of their current element. The third step, *split*, regulates the splitting of new elements. Every new element halves the range, but based on the contents of the ranges, they either change their minimum or maximum if only one half is populated, or they make a new element if both halves are populated. They then reset their representatives of both ranges and the *done* parameter.

During execution, the method will execute through a fixpoint $\text{migrate} < \text{Fix}(\text{checkStable} < \text{split} < \text{migrate})$. The first call to *migrate* divides the old elements over the range of the new elements. The fixpoint then keeps splitting

```

1  struct OldElem(val: Int, place: NewElem, move: Bool){
2    checkStable {
3      if (place.p2 != null || (val <= place.spl && place.p1 != this)) then {
4        place.done := false;
5      }
6    }
7    migrate {
8      if (move) then {
9        if (place.hasSplit) then {
10         place := place.next;
11       }
12       move := false;
13     }
14     if (val <= place.spl) then {
15       place.p1 := this;
16     }
17     if (val > place.spl) then {
18       place.p2 := this;
19       move := true;
20     }
21   }
22 }
23 struct NewElem(min: Int, spl: Int, max: Int, next: NewElem, p1: OldElem, p2:
    OldElem, hasSplit: Bool, done: Bool){
24   split {
25     if (!done) then {
26       if (p1 != null && p2 != null) then {
27         next:=NewElem(spl+1,spl+1+(max-(spl+1))/2,max,next,null,null,false,true)
28       ;
29       max := spl;
30       hasSplit := true;
31     }
32     if (p1 != null && p2 = null) then {
33       max := spl;
34       hasSplit := false;
35     }
36     if (p2 != null && p1 = null) then {
37       min := spl + 1;
38       hasSplit := false;
39     }
40     spl := min + (max - min)/2;
41     p1 := null;
42     p2 := null;
43   }
44   done := true;
45 }
46
47 migrate < Fix(checkStable < split < migrate)

```

Listing 3.7: AuDaLa code for Linked List Copy Sort

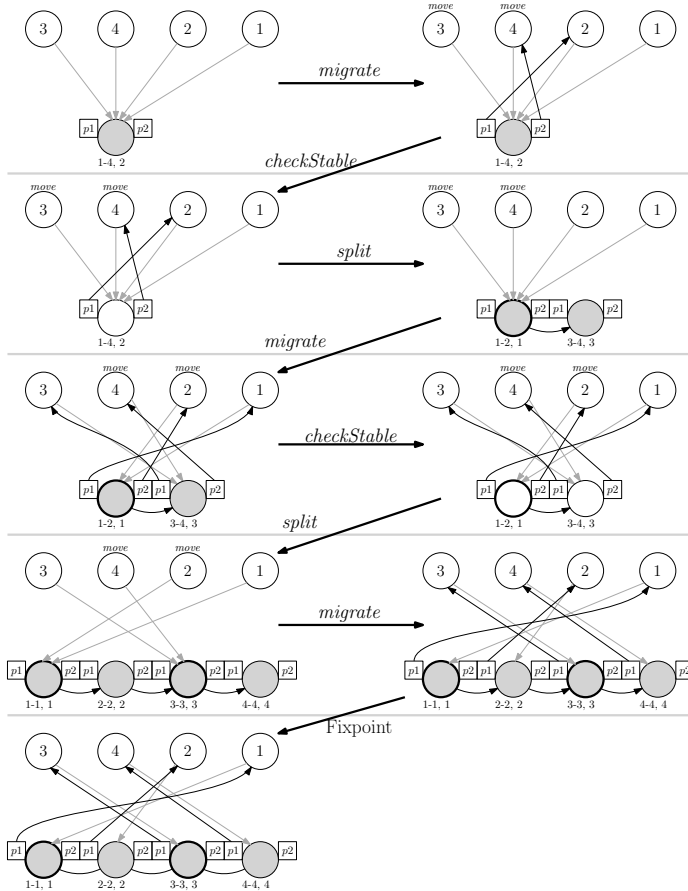


Figure 3.5: Execution of Listing 3.7 on a small list. The upper rows are the old elements, the lower rows the new elements. For the old elements, the parameter *place* is depicted by the grey arrows. For the new elements, the numbers beneath every element are formatted “[*min*]-[*max*], [*spl*]”. New elements for which *hasSplit* is true have a bold border, and a new element for which *done* is true is grey.

until for every new element, there is only one old element (which is stored in $p1$). This is helped by the parameter *done*, which only allows for splits if the new element is unstable. The execution of the first iteration of the program on a small list is shown in Figure 3.5.

For a list with n data elements with maximum value m , the program uses exactly $2n$ struct instances, as it does not create a struct instance for the new list unless there is an element of the old list within its range, and struct instances are repurposed if there is no element of the old list within range. The program performs $O(n \log m)$ work, as all list elements compare themselves to at most $\log m$ other elements. The span of the algorithm is $O(\log m)$, for in the worst case, an element must go through $\log m$ comparisons before it is placed correctly in the new list. It follows that the program runs in $O(\log m)$ time with n data elements with maximum value m .

3.4.3 Cole's algorithm discussion

Parallel sorting has an optimal theoretic running time of $O(\log n)$. To establish that that theoretic running time can also be achieved in AuDaLa, which does not use arrays and therefore does not have easy ways to enforce constant time access, we have investigated how to implement Cole's Algorithm [43] in AuDaLa. (We chose Cole's Algorithm for this as our main other option, the AKS network, is more impractical [1, 81].)

We found that we can adapt Cole's Algorithm for linked lists without impacting the running time, while keeping the parameter restriction of AuDaLa in mind. This makes us confident that Cole's algorithm can be implemented in AuDaLa, which means that sorting in AuDaLa can be done in $O(\log n)$ time. Because of that, though we have not yet formally adapted our result to AuDaLa, we discuss what we found in this section. To discuss our findings, we first give a rough overview on how Cole's Algorithm works, followed by a discussion of what we were able to change and what we found we could not change.

Cole's Algorithm Overview

Cole's Algorithm is essentially a pipelined sampling merge sort. The algorithm incrementally creates a binary merge tree from the original array. This tree consists of $\log n$ levels containing nodes, where every node represents a part of the original array. It starts by dividing the original array into arrays of size four (which are sorted in constant time). These form the nodes of the first level. Then in $O(\log n)$ iterations, the algorithm will fill in the rest of the merge tree, which leads to the sorted list on the top.

Let k be the number of the current iteration, let i be a level, and let arrays j_k and j'_k be neighbouring arrays on level i with parent array m_k on level $i+1$.

Then during iteration k , samples are taken from j_k and j'_k using the indexes of those arrays, which are then merged into a new array m_{k+1} using the ordering information of m_k . If j_k and j'_k are finished, then either every fourth element, every second element or every element is sampled, depending on whether this is the first, second or third iteration in which they are finished. If j_k and j'_k are not yet finished, then every fourth element is sampled. At the end of the iteration, j_k , j'_k and m_k are replaced with arrays j_{k+1} , j'_{k+1} and m_{k+1} , which are either finished or two times as big as their predecessors, and fully sorted. The doubling of the size of j_{k+1} , j'_{k+1} and m_{k+1} compared to j_k , j'_k and m_k is due to a finely tuned pipelining balance, which is powered by the sampling strategy. In the next iteration, the old arrays j_k , j'_k and m_k become irrelevant.

To merge j_k and j'_k , *rankings* are used, which save ordering information: if an element a ranks into an element b , then a should be inserted after b and before b 's right neighbour if a is merged into b 's list. These need to be maintained and updated for the new arrays of the next iteration

Important is that m_{k+1} , j_{k+1} and j'_{k+1} do not have to contain all elements of m_k , j_k and j'_k . The path of an element through all the arrays to the top of the merge tree is not constant, and an element may reach the top level of the merge tree in one time step only to be removed from the top level the next timestep. The only guarantees we have are the relations between the arrays as a whole.

Adaptations

We have put some effort to adapt Cole's Algorithm to work with linked lists, while keeping AuDaLa's restrictions in mind. The main difficulty for doing this is that we cannot use index access, which makes rankings the only method for accessing other elements in constant time. Due to the erratic path of single elements through the merge tree, maintaining the rankings throughout the timesteps turned out to be difficult.

To solve this, our linked list version keeps the progress of an element through the merge tree monotone. We therefore merge the linked lists representing m_k , j_k and j'_k into m_{k+1} , j_{k+1} and j'_{k+1} as well, and use some extra rankings to keep track of which elements are new and which are holdovers from past lists. This is facilitated by the fact that linked lists are dynamic and that elements can copy themselves in constant time with the AuDaLa restrictions of finite memory per element. This allows us to correctly update the rankings in the linked list version as well, and also to make the algorithm a bit simpler by not throwing away most of the merge tree every iteration. The adaptation focuses on making sure that all required information for sampling and merging is locally available and correctly maintained. Furthermore, the adaptation does not use arrays or dynamic amounts of information locally. We

are therefore confident that we would be able to implement this adaptation in AuDaLa.

We have studied ways to improve the above adaptation further in two ways. Firstly, while the merge tree is maintained in our adaptation, we still need to add some extra structure to be able to sample exactly as in the original. Secondly, now that the merge tree is maintained, to make a space-efficient version, we would like to remove elements from lower levels of the merge tree when they are sampled. However, the growth of the lists in each level needs to remain a constant factor to maintain the running time of the algorithm, which is a precarious balance, easily upset by changes introduced to achieve either of these goals. Therefore, we have not been able to find a way to make versions of the algorithm with these features.

In conclusion, though we are confident that Cole's Algorithm can be implemented in AuDaLa with a running time (and span) of $O(\log n)$, our investigation has not led to further results. Furthermore, AuDaLa code for the adaptation has not yet been made nor proven correct.

3.5 A Turing Machine in AuDaLa

In this section, we define a Turing machine in AuDaLa. We first give some background information on Turing machines in Section 3.5.1, before giving the design and the code in Section 3.5.2. This example is proven correct in Chapter 5, from which it follows that AuDaLa is Turing complete.

3.5.1 Turing machines

We define a Turing machine following the definition of Hopcroft *et al.* [96]. Let $\mathbb{D} = \{L, R\}$ be the set of the two directions *left* and *right*. A Turing machine M is a 7-tuple $M = (Q, q_0, A, \Lambda, \Sigma, B, \delta)$, with a finite set of control states Q , an initial state $q_0 \in Q$, a set of accepting states $A \subseteq Q$, a set of tape symbols Λ , a finite set of input symbols $\Sigma \subseteq \Lambda$, a blank symbol $B \in \Lambda \setminus \Sigma$ and a partial transition function $\delta : Q \times \Lambda \rightarrow Q \times \Lambda \times \mathbb{D}$.

Every Turing machine M operates on an infinite *tape* divided into *cells*. Initially, this tape contains an input string $\mathcal{I} = \varsigma_0 \dots \varsigma_n$ with symbols from Σ . All other cells are set to the blank symbol B . The cell the Turing machine operates on is called the *head*. We represent the tape as a function $t : \mathbb{Z} \rightarrow \Lambda$, where cell i contains symbol $t(i) \in \Lambda$. In this function, cell 0 is the head, cells i such that $i < 0$ are the cells left from the head and cells i such that $i > 0$ are the cells right from the head. We restrict ourselves to deterministic Turing machines. We also assume the input string is not empty, without loss of generality.

We define a *configuration* to be a tuple (q, t) , with q the current state of the Turing machine and t the current tape function. Given input string $\mathcal{I} = \varsigma_0 \dots \varsigma_n$,

the *initial configuration* of a Turing machine M is $(q_0, t_{\mathcal{I}})$, with q_0 as defined for M , and $t_{\mathcal{I}}(i) = \varsigma_i$ for $0 \leq i \leq n$ and $t_{\mathcal{I}}(i) = B$ otherwise.

During the execution, a Turing machine M performs *transitions*, defined as:

Definition 3.5.5 (Turing machine transition). Let $M = (Q, q_0, A, \Lambda, \Sigma, B, \delta)$ be a Turing machine with input string $\mathcal{I}$ and let (q, t) be a configuration such that $\delta(q, t(0)) = (q', \varsigma', D)$, with $D \in \mathbb{D}$. Then $(q, t) \rightarrow (q', t')$, where t' is defined as

$$t'(i) = \begin{cases} \varsigma' & \text{if } i = 1 \\ t(i-1) & \text{otherwise} \end{cases} \quad \text{if } D=L \text{ and } t'(i) = \begin{cases} \varsigma' & \text{if } i = -1 \\ t(i+1) & \text{otherwise} \end{cases} \quad \text{if } D=R.$$

We say a Turing machine M *accepts* a string $\mathcal{I}$ iff, starting from $(q_0, t_{\mathcal{I}})$ and taking transitions while possible, M halts in a configuration (q, t) such that $q \in A$.

3.5.2 The implementation of a Turing machine in AuDaLa

In this section, we describe the implementation of a Turing machine in AuDaLa.

Intuitively, this implementation consists of two components. The tape is modelled as a linked list of cells, which is extendable at both edges to make it functionally infinite. Furthermore, we define a *control* to actually execute the Turing machine, which contains the transition information and which links to the head of the tape. Note that the definition of the AuDaLa program is dependent on the Turing machine it implements. In the rest of this section, we give the template for this implementation, including an example at the end.

Let $M = (Q, q_0, A, \Lambda, \Sigma, B, \delta)$ be a Turing machine and $\mathcal{I}$ an input string. We implement M and initialize the tape to $\mathcal{I}$ in AuDaLa. We assume w.l.o.g. that $Q \subseteq \mathbb{Z}$ with $q_0 = 0$ and that $\Lambda \subseteq \mathbb{Z}$ with $B = 0$.

As M 's tape is infinite and AuDaLa does not allow dynamic amounts of parameters, we instead model M 's tape as a connected group of tape *cells*, as that can be done with a finite amount of parameters. We then add new tape cells to the end of the tape when necessary, which allows for an arbitrarily large tape.

For the tape cells, we create the struct *TapeCell*, with a left cell (parameter *left*), a right cell (*right*) and a cell symbol (*symbol*).

The control of M is modeled by a struct *Control*, which saves a tape head (variable *head*), a state $q \in Q$ (*state*) and whether $q \in A$ (*accepting*). See Listing 3.8. The step *transition* in the *Control* struct models the transition function δ . For every pair $(q, \varsigma) \in Q \times \Lambda$ such that $\delta(q, \varsigma) = (q', \varsigma', D)$ with $D \in \mathbb{D}$, *transition* contains a clause as shown in Listing 3.9 (assuming $D = R$). This clause updates the state and symbol, and saves whether the new state is accepting.

```

1  struct TapeCell (left: TapeCell, right: TapeCell, symbol: Int){} //def. of TapeCell
2
3  struct Control (head: TapeCell, state: Int, accepting: Bool) {
4    transition {
5      see Listing 3.9 and 3.10      //definition of the step "transition"
6    }
7    init {
8      see Listing 3.11              //definition of the step "init"
9    }
10  }
11
12  init < Fix(transition)      //schedule: run "init" once and then iterate "transition"

```

Listing 3.8: The AuDaLa Turing machine structure

```

1  if (state == q && head.symbol ==  $\varsigma$ ) then {
2    head.symbol :=  $\varsigma'$ ;      //update the head symbol
3    state := q';           //update the state
4    accepting := (q' ∈ A); //the new state is accepting or rejecting
5
6    if (head != null && head.right == null) then {
7      head.right := TapeCell(head, null, 0); //call constructor to create a new TapeCell
8    }
9    head := head.right;     //move right
10 }

```

Listing 3.9: A clause for $\delta(q, \varsigma) = (q', \varsigma', R)$.

It also moves the head and creates a new *TapeCell* if there is no next *TapeCell* instance, which we check in line 6. Note that we also check whether the head is not null, for while the null-instance of head cannot update its own parameter, it can still make new *TapeCells*, which is undesirable. For this, as ς can be *null*, we need to explicitly check whether *head* is a null-instance. Note that $B = 0$, and that if $D = L$ the code only minimally changes.

The clauses for the transitions are combined using an if-else if structure (syntactic sugar for a combination of ifs and variables), so only one clause is executed each time *transition* is executed. See Listing 3.10.

In the step *init* in the *Control* struct, we create a *TapeCell* for every symbol $\varsigma \in \mathcal{I}$ from left to right, which are linked together to create the tape. We also create a *Control*-instance. Listing 3.11 shows this for an example tape $\mathcal{I} = \varsigma_0, \varsigma_1, \varsigma_2$. A call of *init* in the schedule causes the null-instance of *Control* to initialize the tape. It also initializes a single non-*null* instance of *Control*.

```

1  transition {
2    if (state ==  $q_1$  && head.symbol ==  $\varsigma_1$ ) then{
3      /*clause 1*/
4    }
5    else if (state ==  $q_2$  && head.symbol ==  $\varsigma_2$ ) then {
6      /*clause 2*/
7    }
8    else if (state ==  $q_3$  && head.symbol ==  $\varsigma_3$ ) then {
9      /*clause 3*/
10   }
11   // etc.
12 }

```

Listing 3.10: The *transition* step. The shown pairs all have an output in δ .

```

1  init {
2    TapeCell cell0 := TapeCell(null, null,  $\varsigma_0$ );           // initialize the tape
3    TapeCell cell1 := TapeCell(null, null,  $\varsigma_1$ );
4    TapeCell cell2 := TapeCell(null, null,  $\varsigma_2$ );
5
6    cell1.left := cell0;                                     // connect the tape
7    cell0.right := cell1;
8    cell2.left := cell1;
9    cell1.right := cell2;
10
11   Control(cell0, 0, ( $q_0 \in A$ ));                          //initialize the control
12 }

```

Listing 3.11: Initializing input string $\mathcal{I}$.

```

1  struct TapeCell (left: TapeCell, right: TapeCell, symbol: Int){} //def. of TapeCell
2  struct Control (head: TapeCell, state: Int, accepting: Bool) {
3    transition {
4      if (state == 0 && head.symbol == 0) then{ // transition  $\delta(0,0)$ 
5        head.symbol := 0;
6        state := 0;
7        accepting := false;
8        if (head != null && head.right == null) then {
9          head.right := TapeCell(head, null, 0);
10         }
11        head := head.right;
12      }
13      else if (state == 0 && head.symbol == 1) then{ // transition  $\delta(0,1)$ 
14        head.symbol := 1;
15        state := 0;
16        accepting := false;
17        if (head != null && head.right == null) then {
18          head.right := TapeCell(head, null, 0);
19        }
20        head := head.right;
21      }
22      else if (state == 0 && head.symbol == 2) then{ // transition  $\delta(0,2)$ 
23        head.symbol := 2;
24        state := 1;
25        accepting := true;
26        if (head != null && head.right == null) then {
27          head.right := TapeCell(head, null, 0);
28        }
29        head := head.right;
30      }
31    }
32    init {
33      TapeCell cell0 := TapeCell(null, null, 1);           // initialize the tape
34      TapeCell cell1 := TapeCell(null, null, 1);
35      TapeCell cell2 := TapeCell(null, null, 2);
36      TapeCell cell3 := TapeCell(null, null, 1);
37      cell1.left := cell0;                               // connect the tape
38      cell0.right := cell1;
39      cell2.left := cell1;
40      cell1.right := cell2;
41      cell3.left := cell2;
42      cell2.right := cell3;
43      Control(cell0, 0, false);
44    }
45  }
46  init < Fix(transition) //schedule: run "init" once and then iterate "transition"

```

Listing 3.12: The AuDaLa program for M_{ex} with input string 1121.

The schedule will then make that instance of *Control* run the *transition* step until the program stabilizes. Listing 3.8 shows the final structure of the program.

Example 3.5.6. As an example, consider a Turing machine $M_{ex} = (\{0, 1\}, 0, \{1\}, \{0, 1, 2\}, \{1, 2\}, 0, \delta)$, with $\delta(0, 0) = (0, 0, R)$, $\delta(0, 1) = (0, 1, R)$ and $\delta(0, 2) = (1, 2, R)$, with no transitions defined for state 1. This Turing machine walks through the input and stops after finding a 2. In Listing 3.12, we have created an AuDaLa program for this Turing machine with input string ‘1121’. The Turing machine with input string ‘1121’ is visualised in Figure 3.6.

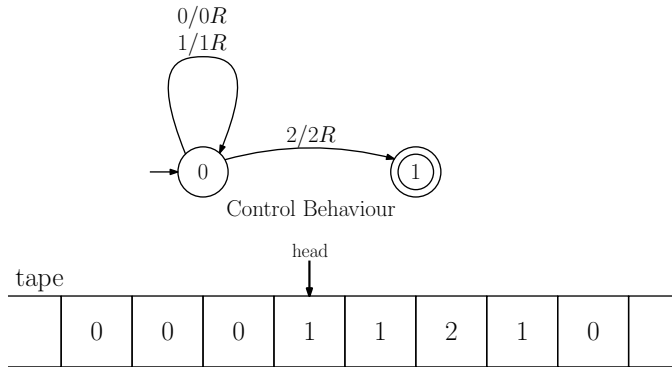


Figure 3.6: The Turing Machine of Example 3.5.6 with input string ‘1121’.

3.6 Conclusion

In this chapter, we have presented and discussed multiple examples of programs in AuDaLa, to demonstrate certain features and as a basis for those reading it to start programming with AuDaLa. We gave another basic example of AuDaLa in Section 3.1. This example is considered again in Chapter 6, as it leads to a potential adaptation of the AuDaLa semantics. We demonstrated a benign race condition in Section 3.2, a nested fixpoint in Section 3.3. We have given some sorting algorithms in Section 3.4, including a discussion on adapting Cole’s Algorithm [43] in AuDaLa. Along with the program for Prefix Sum, we prove the linked list copy sorting algorithm of Section 3.4.2 correct in Chapter 5, and we create a proof tree for the linear sorting algorithm in Chapter 7. We leave an implementation and verification of Cole’s Algorithm to future work. In the next chapter, we will be considering the semantics further and give and prove some properties for AuDaLa programs.

Chapter 4

Properties of AuDaLa

This chapter explores the behaviour of AuDaLa and AuDaLa programs by giving definitions and proving properties for AuDaLa. The goal of this is to get an insight in what generic facts we can establish for the behaviour of AuDaLa as defined by the semantics in Chapter 2. We also use the properties established in this section in Section 5 to prove some of our example algorithms correct.

We start out with some standard properties and definitions (Section 4.1), like *reference notation* and some lemmas and definitions on race conditions and (non-)determinism.

Then, we analyze well-formed programs in Section 4.2. This results in a set of six properties on the precise results of the execution of commands in well-formed programs.

After that, we explore the implications of these six properties in Section 4.3, leading to three main results. Firstly, we formalize the notion of an *expression result*, which simplifies semantic proofs and other formal reasoning about AuDaLa. Secondly, we prove our type system *sound* using the six properties. Thirdly, we define some collaries that simplify reasoning about well-formed AuDaLa programs in proofs by lifting the proof from transitions to syntax statements and expressions.

After Section 4.3, we conclude the chapter in Section 4.4.

4.1 Properties of AuDaLa Programs

In this section, we give some standard definitions and properties of any AuDaLa program. First, to simplify reasoning about semantic values and variables, we define *reference notation* for semantic values, based on the dot notation as used in the syntax:

Definition 4.1.1 (Reference Notation). Let σ be a structure environment and let s be a struct instance with label ℓ_s such that $\sigma(\ell_s) = \langle \theta, \gamma, \chi, \xi \rangle$. We define the notation $\ell_s.x^i$ inductively on i , with a variable x :

1. $\ell_s.x^1 = \xi(x)$,
2. $\ell_s.x^i = \xi'(x)$ for $i > 1$, with $\ell_s.x^{i-1} \in \mathcal{L}$ and $\sigma(\ell_s.x^{i-1}) = \langle \theta', \gamma', \chi', \xi' \rangle$.

We write $\ell_s.x$ for $\ell_s.x^1$ and $s.x$ for $\ell_s.x$, where $\sigma(\ell_s) = s$ and x is a parameter of s .

We then define the notions of *executions* and *accesses*:

Definition 4.1.2 (AuDaLa Execution). Let p be an AuDaLa state reachable in some program $\mathcal{P}$. An *execution* from p refers to a possibly infinite chain of semantic rule transitions that start in a state p , characterized by the labels of those transitions. A *finite execution* refers to a finite chain of semantic rule transitions that starts in a state p and ends in a state p_1 .

We extend this definition to fit our use of it: As steps are finite by the definition of steps (see Chapter 2), an execution of a step F from p refers to a chain of rule transitions from p starting with an **Init**-transition for F and ending in the first state that contains a struct environment σ for which $\text{Done}(\sigma)$ holds. A step F can be executed from p iff there exists an execution of F from p .

As expressions and statements are finite, an execution of an expression E , a statement S or a command list C by some struct instance s from p refers to a chain of rule transitions from p starting with the transition enabled by the first command of $\llbracket E \rrbracket$, $\llbracket S \rrbracket$ or C for s and ending with the transition enabled by the last command of, respectively, $\llbracket E \rrbracket$, $\llbracket S \rrbracket$ or C for s . Additionally, for every command in $\llbracket E \rrbracket$, $\llbracket S \rrbracket$ or C , there needs to be a corresponding transition induced by that command taken by s . This allows for interleaving with other statements or expressions; not all transitions need to correspond to a command of $\llbracket E \rrbracket$, $\llbracket S \rrbracket$ or C in the command list of s . An expression E , a statement S or a command list C can be executed from p by a struct instance s iff there exists an execution of, respectively, E , S or C from p for s .

An execution of a program $\mathcal{P}$ refers to a possibly infinite chain of semantic rule transitions that start in the state $p_{\mathcal{P}}^0$. A finite execution of a program $\mathcal{P}$ starts in $p_{\mathcal{P}}^0$ and ends when no transition is possible. Lastly, we consider *bounded executions*, where an execution of size i from p starts at p and ends at some state p_1 reached after exactly i transitions.

Recall that parameters and local variables are both considered variables. We then define the following:

Definition 4.1.3 (AuDaLa Semantic Access). Let x be a variable. We define every application of the semantic rules **ComRd** and **ComWr** with respect to x to be an *access* of x .

Here, an application of **ComWrNSkip** for x does not constitute an access.

We use this to introduce some other definitions and lemmas, starting with a notion of determinism. For this, note that we non-deterministically assign new labels to newly created struct instances. As the exact labels we assign does not matter as long as all labels are unique, which is enforced by the semantics, we can ignore the values of newly assigned labels for a notion of determinism. We use the following notions:

Definition 4.1.4 (AuDaLa Determinism). Let $\mathcal{P}$ be an AuDaLa program containing step F . Let p be some state from which F can be executed. Then F is *deterministic* for p iff there exists exactly one state modulo newly assigned labels which is reached by executing F from p .

Additionally, a variable x is *deterministic* for F and p iff in all states reached by executing F from p the value for x is the same modulo newly assigned labels. Furthermore, F is (fully) *deterministic* iff for all states p that can execute F , F is deterministic for p , and x is (fully) *deterministic* for F if x is deterministic when executing F from all states that can execute F .

Definition 4.1.5 (Race Condition). Let $\mathcal{P}$ be an AuDaLa program, let F be a step in $\mathcal{P}$ and let p be a state in $\mathcal{P}$ from which F can be executed. Then F executed from p contains a *race condition* for parameter x iff there exist distinct struct instances s_a and s_b that both access a parameter x in the same struct instance during an execution of F from p and at least one of the two instances writes to x . If both s_a and s_b write to x , this is a *write-write race condition*, otherwise this is a *read-write race condition*. If any step F in the execution of $\mathcal{P}$ has a race condition for some parameter, then $\mathcal{P}$ has a race condition.

Note that in the semantics, writes are atomic, which means that race conditions cannot be *data races*, which are race conditions with overlapping accesses. It follows that the result of a race condition still depends on the context, but cannot be fully random.

With the above definitions of race conditions and determinism, we are able to prove the following lemma:

Lemma 4.1.6 (AuDaLa Determinism). A step F from an AuDaLa program $\mathcal{P}$ is deterministic for some state p if F does not contain a race condition from p .

Proof. We prove the lemma by contradiction.

Assume that F does not contain a race condition from p . Then assume that F is not deterministic from p . As F is not deterministic, there exist multiple distinct states modulo newly assigned labels which can be reached by executing F from p , following Definition 4.1.4. Let p_1 and p_2 be two of these states, and let π_1 be an arbitrary execution of F from p reaching p_1 and let π_2 be an arbitrary execution of F from p reaching p_2 .

As p_1 and p_2 are distinct (modulo newly assigned labels), they must differ in either their schedule or their struct instance contents. As π_1 and π_2 are both executions of F from p , they only contain a single schedule transition (either **InitL** or **InitG**), which is taken from p . As both π_1 and π_2 start from p , it follows that this is the same schedule transition, and as the other transitions in π_1 and π_1 do not impact the schedule, the difference between p_1 and p_2 must lie in their struct instance contents. Let the state resulting from the schedule transition from p be p' .

As F has only a single definition, it only has a single translation to commands. It follows that π_1 and π_2 execute the same command transitions. Furthermore, as the semantics does not allow for reordering, these command transitions are executed in the same order. By definition of the AuDaLa semantics, any one executed command transition does not introduce non-determinism modulo newly assigned labels locally. It follows that π_1 and π_2 start their command transitions from p' and execute the same command transitions in the same order, with none of those transitions inducing local non-determinism (modulo newly assigned labels). Therefore, any non-determinism must have been introduced by the interleaving of the execution of F by different struct instances. This contradicts with the fact that F does not contain race conditions from p . Therefore, if F does not contain a race condition p , an execution of F from p is deterministic, which contradicts our assumptions $\square$

We can adjust the focus from a whole step to a single parameter, leading to the following corollary:

Corollary 4.1.7 (AuDaLa Parameter Determinism). A parameter x is deterministic during the execution of a step F (of an AuDaLa program $\mathcal{P}$) from a state p iff F does not contain a race condition for x from p .

Proof. This is true along the same lines of the proof for Lemma 4.1.6, except that we only consider differences in the value for x in p_1 and p_2 and consider x to be the variable used in the last two alineas of the proof. $\square$

Finding race conditions can be difficult. To this end, we define *potential race conditions*, which can be found on a syntax level. To do so, we first define the following:

Definition 4.1.8 (Direct and indirect syntax access). Let X be a parameter reference in the syntax, following $\langle Var \rangle$. If X is of the form " $x_1 \dots x_n.x$ ", X is an *indirect* syntax access of x . If X is of the form " x ", X is a *direct* syntax access of x . We consider a syntax access to be a *write* access if it occurs in an update or variable assignment statement to the left of the assignment operator.

Note that indirect syntax accesses can only be write syntax accesses in update statements. We use this to define *potential data races*:

Definition 4.1.9 (Potential Race Condition). Let $\mathcal{P}$ be an AuDaLa program and let F be a step in $\mathcal{P}$. Let θ_a and θ_b be a pair of struct types for which F is defined, allowing for $\theta_a = \theta_b$. Let there be a pair of statements S_1 in F for θ_a and S_2 in F for θ_b which include syntax accesses X_a and X_b of some parameter x of a struct type θ' . Let either X_a or X_b be indirect and let either X_a or X_b be a syntax write access to x . Then the pair (S_1, S_2) is a *potential race condition* on x .

These can be used to consider which parameters can possibly be in a data race:

Lemma 4.1.10. Let $\mathcal{P}$ be an AuDaLa program and let F be a step in $\mathcal{P}$. If F does not have any potential race conditions on parameter x , F does not have any race conditions on parameter x .

Proof. If F does not have any potential race conditions, there are no pairs of statements (S_1, S_2) that can be taken from any executions of F in any structs such that S_1 and S_2 are executed by different struct instances but access x in the same struct instance of which at least one access is a write access. Then it follows that F cannot have any race conditions on x . $\square$

We also define a notion of *instability*:

Definition 4.1.11 (Instability). Let $\mathcal{P}$ be an AuDaLa program containing a step F . Let p be a state of $\mathcal{P}$ from which F can be executed, and let s be a struct instance in p . A parameter $s.x$ with value v before the execution of F is *unstable* after the execution of F from p , denoted as $s.x^*$, iff a write command is executed during F that sets $s.x$ to a value $v' \neq v$.

Lastly, as we also want to prove partial programs, which may contain race conditions in the undefined parts of the program, we give a more specific definition for what we consider a program without read-write race conditions:

Definition 4.1.12 (RW Race Condition Free from Premise State). We consider an AuDaLa program $\mathcal{P}$ to be without read-write race conditions from some state p_1 iff any execution of $\mathcal{P}$ starting in p_1 does not contain a read-write race condition. When considering executions from p_1 , we consider p_1 to be the *premise state*.

4.2 Properties of Well-Formed Programs

In this section, we analyse the properties of well-formed programs specifically. To do this, we give six properties of states which can be reached during executions of well-formed programs.

Before we do this, however, we need to introduce an auxiliary notion, to relate well-formedness to schedules using the semantics-only schedule element *aFix*, as those are not considered well-formed by the type system. For our properties and our proofs, however, it is useful to be able to make assertions on the relative well-formedness of the schedule and the program even after execution has started. We therefore define *well-formed modulo aFix*:

Definition 4.2.13 (Well-Formed modulo *aFix*). A schedule Δ is *well-formed modulo aFix*, notation $\Gamma, \Omega \vdash_{aF} \Delta$ for some environments Γ and Ω , if replacing all occurrences of *aFix* in Δ with *Fix* yields a well-formed schedule Δ' .

Furthermore, let $\mathbb{P}$ be the set of all AuDaLa programs. We define a function $sem: \mathcal{T} \times \mathbb{P} \rightarrow \mathbb{T} \cup \{\perp\}$ to translate between syntax types and semantic types more smoothly. Let sem be defined as:

$$\begin{aligned} sem(\text{Nat}, \mathcal{P}) &= \mathbb{N}, \\ sem(\text{Int}, \mathcal{P}) &= \mathbb{Z}, \\ sem(\text{Bool}, \mathcal{P}) &= \mathbb{B}, \\ sem(\text{String}, \mathcal{P}) &= \text{String}, \\ sem(\theta, \mathcal{P}) &= \theta \text{ if } \theta \in \Theta_{\mathcal{P}} \\ sem(\theta, \mathcal{P}) &= \perp \text{ if } \theta \notin \Theta_{\mathcal{P}}. \end{aligned}$$

The six properties of well-formed programs we will prove can be intuitively stated as:

1. For all states reachable in a well-formed program, either the schedule is empty or it is well-formed modulo *aFix* (Lemma 4.2.14).
2. For all states reachable in a well-formed program, if the *Done*-predicate holds and the schedule is not empty, then there is a schedule transition that can be taken from the state (Lemma 4.2.15).
3. For all states reachable in a well-formed program, if the *Done*-predicate holds and the schedule is not empty, then after a finite number of schedule transitions from the state, a step will be called (Lemma 4.2.16).
4. For all states reachable in a well-formed program, when executing a command, the command is always the last command of some expression or the last command of a statement that it is generated from (Lemma 4.2.17).
5. For all states reachable in a well-formed program, when executing a command, we know specific information about the current state depending on the command, we know that a command transition must be enabled (due to the specific information we know about the state) and we

know specific information about the state resulting from this transition depending on the command (Lemma 4.2.18).

6. For all states reachable in a well-formed program, when executing a step, this execution is finite; after a finite number of transitions, the *Done*-predicate will hold (Lemma 4.2.20).

For the rest of this subsection, we will prove the properties separately, before combining them in a theorem at the end of the subsection. Each property depends upon different parts of the semantics, but all parts of the semantics are used in the proofs.

We start by giving the first property and its proof. For readability, we distinguish between schedules Δ and *schedule elements* sc , where schedule elements can be fixpoints or step calls but not of the form $sc = sc_1 < sc_2$.

Lemma 4.2.14 (Property 1 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Then either $\Delta = \varepsilon$ or $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta$.

Proof. We prove Property 1 by induction on the states traversed in any execution of $\mathcal{P}$. As a base case, the execution starts in the initial state $p_{\mathcal{P}}^0$; by the premise of the lemma, this state has a well-formed schedule. This proves the base case.

Then, let $p_i = \langle \Delta, \sigma, s\chi \rangle$ be the i th state traversed in some execution of $\mathcal{P}$ with definitions D , from which some transition is taken to a state $p_{i+1} = \langle \Delta', \sigma', s\chi' \rangle$. By the induction hypothesis, we assume it holds that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta$ or that $\Delta = \varepsilon$. Then, we do a case distinction:

- If the transition taken is a command transition it holds that $\Delta' = \Delta$, so $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta'$ or $\Delta' = \varepsilon$.
- If the transition taken is a schedule transition, we know that $\Delta \neq \varepsilon$, as this is required by every schedule transition.

As Δ is well-formed, we know that it has a well-defined non-empty first element sc , such that $\Delta = sc$ or $\Delta = sc < Sched_1$.

Then, if $\Delta = sc$, we know that $\Gamma, \Omega \vdash_{aF} sc$, as Δ is well-formed modulo *aFix*.

Then, we have a case distinction on the form of sc :

1. If $sc = F$ for some step F or $sc = \theta.F$ for some struct type θ and some step F , then the schedule transition that is taken is *InitG* or *InitL* respectively. These will remove sc from the schedule.
2. If $sc = Fix(\Delta_1)$ for some schedule Δ_1 , the transition must be *FixInit*, resulting in $\Delta' = \Delta_1 < aFix(\Delta_1)$ which replaces sc . As $\Gamma; Steps(D), \Omega \vdash$

$Fix(\Delta_1)$, it holds that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1$. It follows that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1 < aFix(\Delta_1)$.

3. If $sc = aFix(\Delta_1)$ for some schedule Δ_1 , the transition must be either *FixIter* or *FixTerm*. If the transition is *FixIter*, then $\Delta' = \Delta_1 < aFix(\Delta_1)$ replaces sc . As $\Gamma; Steps(D), \Omega \vdash_{aF} aFix(\Delta_1)$, $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1$. It follows that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1 < aFix(\Delta_1)$. If the transition is **FixTerm**, sc is removed from the schedule.

From this, we can conclude that either $\Delta' = \varepsilon$ or $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta'$, so the induction step holds.

Then if $\Delta = sc < \Delta_1$, with sc being a single schedule element, note that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta$ iff $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1$ and $\Gamma; Steps(D), \Omega \vdash_{aF} sc$. Any schedule transition only affects the first schedule element, so after a single transition, Δ_1 will remain unchanged. This means that we can consider sc as a separate schedule. We then apply same case distinction as above, from which we can conclude that either the first element is removed after a transition or the transition results in a schedule Δ'' such that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta''$. In the first case, $\Delta' = \Delta_1$ and $\Gamma, \Omega \vdash_{aF} \Delta'$. In the second case, we know that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta''$ and $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta_1$, so we know that $\Gamma; Steps(D), \Omega \vdash_{aF} \Delta'$. Therefore, in this case, the induction step holds.

It follows that the induction step holds, so Property 1 also holds. $\square$

We use this first property to prove the second property:

Lemma 4.2.15 (Property 2 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Then if $Done(\sigma)$ holds and $\Delta \neq \varepsilon$, there exists at least one schedule transition that can be taken from p with definitions D .

Proof. Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a state reached in an execution of $\mathcal{P}$ such that $Done(\sigma) = true$. Assume that $\Delta \neq \varepsilon$. It follows from Property 1 that Δ is well-formed modulo $aFix$, so we know that Δ has a well-formed and well-defined first schedule element sc . Then, we do a case distinction on what this element is:

1. If $sc = F$ for some step F , then it follows that **InitG** is enabled: $Done(\sigma)$ holds, and beyond that, the only requirement is that $sc = F$. Same holds for $sc = \theta.F$ and **InitL**.

2. If $sc = \text{Fix}(\Delta_1)$ for some schedule Δ_1 , the transition **FixInit** can be taken, as $\text{Done}(\sigma)$ holds and sc fits the pattern in the schedule which is the other requirement.
3. If $sc = a\text{Fix}(\Delta_1)$ for some schedule Δ_1 , then whether a transition can be taken depends on whether there is a value on top of the stability stack. This is, however, the case: The only rule which introduces a pattern which includes $a\text{Fix}$ in the semantics is the rule **FixInit**, which also adds a value on top of the stability stack. The only rule which removes a value of from the stability stack is **FixTerm**, which also removes an $a\text{Fix}$ pattern. It follows that the amount of values in the stability stack corresponds exactly to the amount of $a\text{Fix}$ patterns. It follows that there is a value on the stability stack when a $a\text{Fix}$ pattern is encountered. Then, either this value is *true* or *false*. If the value is *true*, note that $\text{Done}(\sigma)$ holds and that an $a\text{Fix}$ pattern is the topmost schedule element, so **FixTerm** is enabled. Along the same lines, if the value is *false*, **FixIter** is enabled.

It follows that if the schedule is not empty and $\text{Done}(\sigma)$ holds, there must be a well-formed first schedule element and for all forms that element can take there is an enabled transition. $\square$

This, again, is used to prove the third property:

Lemma 4.2.16 (Property 3 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a well-formed AuDaLa state reached by some program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\text{Done}(\sigma)$ holds and $\Delta \neq \varepsilon$, then after a finite amount of transitions from p either the schedule is empty or there is an **InitG** or **InitL** transition to some state $p' = \langle \Delta', \sigma', s\chi' \rangle$ for which $\text{Done}(\sigma')$ does not hold.

Proof. We prove Property 3 by first proving through induction that after any sequence of transitions from p either the schedule is empty or there must be an **InitG** or **InitL** transition. We then prove that after this **InitG** or **InitL** transition to, w.l.o.g., state p' , $\text{Done}(\sigma')$ does not hold.

If either **InitG** or **InitL** are enabled, by definition of the schedule and the transitions, no other transitions are enabled, so the **InitL** or **InitG** transition must also be taken. We therefore make no distinction between the an **InitG** or **InitL** transition being enabled and that transition being taken.

We first do induction on the structure of the schedule. As base case, we take that $\Delta = \varepsilon$, which directly satisfies our property. We then assume as induction hypothesis 1 that for all but the first element of the schedule, any sequence of schedule transitions eventually leads to either the schedule being empty or an **InitL** or **InitG** transition being taken.

Let sc be the first element of Δ , and assume that **InitG** and **InitL** are not enabled. It follows that $sc = a\text{Fix}(\Delta_1)$ or $sc = \text{Fix}(\Delta_1)$. Let the next transition

taken be t , from p to $p' = \Delta', \sigma', s\chi'$. This transition exists, by Property 2. We then do induction on the number of occurrences k of both Fix and $aFix$ in sc . If $k = 1$, It follows that Δ_1 is either a step call, a typed step call or a sequence of step and typed step calls, as Δ is well-formed modulo $aFix$. Then if $t = \mathbf{FixInit}$, (so $sc = Fix(\Delta_1)$), Δ' will start with the sequence $\Delta_1 < aFix(\Delta_1)$, and as Δ_1 must have a step call as first element, either **InitL** or **InitG** must be enabled. The same argument holds if $t = \mathbf{FixIter}$, but with $sc = aFix(\Delta_1)$. If $t = \mathbf{FixTerm}$, so $sc = aFix(\Delta_1)$, sc is removed. Then either $\Delta' = \varepsilon$, or we use induction hypothesis 1 to conclude that any sequence of schedule transitions eventually leads to either the schedule being empty or an **InitL** or **InitG** transition being taken.

We then assume as induction hypothesis 2 that if $k = i - 1$, any sequence of schedule transitions eventually leads to either the schedule being empty or an **InitL** or **InitG** transition being taken. Let $k = i$. Then if $t = \mathbf{FixInit}$, so $sc = Fix(\Delta_1)$, Δ' will start with the sequence $\Delta_1 < aFix(\Delta_1)$. As the amount of occurrences of both Fix and $aFix$ in Δ_1 must then be $i - 1$, by induction hypothesis 2, any sequence of schedule transitions eventually leads to either the schedule being empty or an **InitL** or **InitG** transition being taken. The same argument holds if $t = \mathbf{FixIter}$, but with $sc = aFix(\Delta_1)$. If $t = \mathbf{FixTerm}$, so $sc = aFix(\Delta_1)$, sc is removed. Then again either $\Delta' = \varepsilon$, or we use induction hypothesis 1 to conclude that any sequence of schedule transitions eventually leads to either the schedule being empty or an **InitL** or **InitG** transition being taken.

It follows that after a finite number of transitions from p either the schedule is empty or there must be an **InitG** or **InitL** transition to some state $p' = \langle \Delta', \sigma', s\chi' \rangle$. By the definition of the schedule transitions, when transition **InitG** is taken for some step F , by definition of **InitG**, the commands of F are put into the command list of every struct instance of a struct type for which F is defined. The same holds for when transition **InitL** is taken for some step call $\theta.F$. As Δ is well-formed modulo $aFix$, there is at least one struct type for which F is defined, and as there always exists a null-instance for every struct type, there is then at least one struct instance which will have commands in their command list. It follows that $\neg Done(\sigma')$. $\square$

This concludes the schedule properties. We continue with the first of the step execution properties:

Lemma 4.2.17 (Property 4 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg Done(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ and $\gamma \neq \varepsilon$. Let $\gamma = c_j; \gamma$, such that c_j has been generated from a statement S , with $\llbracket S \rrbracket = c_1; \dots; c_m$. Then there exists a c_k with $k \leq j$ such that either $c_k; \dots; c_j = \llbracket E \rrbracket$ for some expression E or $c_k; \dots; c_j = \llbracket S \rrbracket$ for some statement S .

Proof. To prove the lemma, we do a case distinction on which command c_j is and choose an appropriate c_k :

1. If $c_j = \mathbf{push}(val(g))$, it has been generated by the interpretation function in response to either the expression g , with $g \in LT$, or, if $g = \mathbf{defaultVal}(T)$ for some T , to $null_T$. It follows that with $k = j$, the lemma holds. Analogously, the lemma holds if $c_j = \mathbf{push}(\mathbf{this})$, which is generated for the expression $\mathbf{this}$.
2. If $c_j = \mathbf{rd}(x_i)$ for some variable x_i , then we know that c_j is generated by the interpretation function in response to some variable expression $x_1 \cdots x_n$, with $x_i \in x_1 \cdots x_n$. Let c_k be the first command of $[x_1 \cdots x_n]$. Then $c_k; \dots; c_j = [x_1 \cdots x_i]$ and the lemma holds.
3. If c_j is a **wr**, **cons**, **if**, **not** or **op** command, it is automatically the last generated command for some expression or statement as per the definition of the interpretation function. The lemma will then hold for c_k being the first command generated for the same expression or statement.

As in all cases, a k can be chosen, the lemma holds. $\square$

We use this to make the proof of the next property easier. We then prove the fifth property, which is the most important property for step executions. Intuitively, this property allows us to reason about the execution of statements and expressions in terms of their preconditions and postconditions on a semantic level. This property cannot be split further, as the proof for what we can expect as preconditions depends on the proof of what we can expect as postconditions and vice versa. Furthermore, the cases also intermingle in their dependencies.

Lemma 4.2.18 (Property 5 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a well-formed AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta \mathcal{P}$ such that $\Gamma, \Omega \vdash D \Delta \mathcal{P}$ for some Γ, Ω . If $\neg \mathbf{Done}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ and $\gamma \neq \varepsilon$. Let $\gamma = c_j; \gamma$, such that c_j has been generated from a statement S_i in a step F , with $\llbracket S_i \rrbracket = c_1; \dots; c_m$. Let $S_1; \dots; S_{i-1}$ be the statements executed by the struct instance of ℓ before S_i as a part of F .

Then, with c_k with $k \leq j$ such that $c_k; \dots; c_j = \llbracket E \rrbracket$ or $c_k; \dots; c_j = \llbracket S_i \rrbracket$, let $p_k = \langle \Delta_k, \sigma_k, s\chi_k \rangle$ be the state right before the transition induced by c_k and let $\sigma_k(\ell) = \langle \theta, \gamma_k, \chi_k; \xi_k \rangle$. Then at least one of the following holds:

1. $c_j = \mathbf{push}(v)$ for some $v \in \mathcal{V}$ or $c_j = \mathbf{push}(\mathbf{this})$,
2. $c_j = \mathbf{rd}(x)$ and all of the following hold:
 - (a) $\chi = \chi_k; \ell'$ and $\sigma(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$, and if $x \notin \mathbf{ParsP}(\theta_{\ell'})$ then $\ell' = \ell$,

- (b) either $S_q = (T \ x := E)$ for some $1 \leq q < i$, $T \in \mathcal{T}$ and $E \in EX$ or $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$,
 - (c) either $T \in \Theta_{\mathcal{P}}$, $\xi_{\ell'}(x) \in \mathcal{L}$ and $\sigma(\xi_{\ell'}(x)) = \langle T, \gamma_x, \chi_x, \xi_x \rangle$ (for some γ_x, χ_x and ξ_x) or $T \in \mathcal{T} \setminus ID$ and $\xi_{\ell'}(x) \in \text{sem}(T)$.
3. $c_j = \mathbf{wr}(x)$ and all of the following hold:
- (a) $\chi = \chi_k; v; \ell'$ and $\sigma(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$ and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ then $\ell' = \ell$,
 - (b) either $S_q = (T \ x := E)$ for some $1 \leq q \leq i$, $T \in \mathcal{T}$ and $E \in EX$ or $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta')$ for some $T \in \mathcal{T}$,
 - (c) either $T \in \Theta_{\mathcal{P}}$, $v \in \mathcal{L}$ and $\sigma(v) = \langle T, \gamma'', \chi'', \xi'' \rangle$ (for some γ'', χ'' and ξ'') or $T \in \mathcal{T} \setminus ID$ and $v \in \text{sem}(T)$.
4. $c_j = \mathbf{cons}(\theta)$, with $\theta \in \Theta_{\mathcal{P}}$ and $\text{Pars}_{\mathcal{P}}(\theta) = x_1 : T_1; \dots; x_c : T_c$ for some c , and $\chi = \chi_k; v_1; \dots; v_c$ and for all $x_h : T_h$, either $T_h \in \mathcal{T} \setminus ID$ and $v_h \in \text{sem}(T_h)$ or $T_h \in \Theta_{\mathcal{P}}$ and $v_h \in \mathcal{L} \wedge \sigma(v_h) = \langle T_h, \gamma'', \chi'', \xi'' \rangle$ for some γ'', χ'' and ξ'' .
5. $c_j = \mathbf{if}(C)$ and $\chi = \chi_k; b$ such that $b \in \mathbb{B}$.
6. $c_j = \mathbf{not}$ and $\chi = \chi_k; b$ such that $b \in \mathbb{B}$.
7. $c_j = \mathbf{op}(\circ)$ and $\chi = \chi_k; a; b$ and all of the following hold:
- (a) If $\circ \in \{=, !=\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$ or there exists a $T \in \{\mathbb{N}, \mathbb{Z}, \mathbb{B}, \text{String}\} \cup \Theta_{\mathcal{P}}$ such that either $T \notin \Theta_{\mathcal{P}}$ and $a, b \in T$ or $T \in \Theta_{\mathcal{P}}$ and $\sigma(a) = \langle T, \gamma_a, \chi_a, \xi_a \rangle$ and $\sigma(b) = \langle T, \gamma_b, \chi_b, \xi_b \rangle$,
 - (b) If $\circ \in \{<, >, =, <=, >=, *, /, \%, +, ^, -\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$,
 - (c) If $\circ \in \{\&\&, ||\}$ then $a, b \in \mathbb{B}$.

Additionally, there exists a transition induced by c_j from p to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ such that $\sigma'(\ell) = \langle \theta, \gamma', \chi', \xi' \rangle$ and all of the following hold:

1. If $c_j = \mathbf{push}(v)$ for some $v \in \mathcal{V}$, then it holds that $\chi' = \chi_k; v$ and $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$.
2. If $c_j = \mathbf{push}(\mathbf{this})$, then $\chi' = \chi_k; \ell$ and $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$.
3. If $c_j = \mathbf{rd}(x)$, with $\chi = \chi_k; \ell'$ and $\sigma(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$, then $\chi' = \chi_k; \xi_{\ell'}(x)$ and $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$.
4. If $c_j = \mathbf{wr}(x)$, then, with $\chi = \chi_k; v; \ell'$, $\sigma'(\ell') = \langle \theta_{\ell'}, \gamma'_{\ell'}, \chi'_{\ell'}, \xi'_{\ell'} \rangle$ such that either $\ell' \notin \mathcal{L}_0$, $\xi'_{\ell'} = \xi_{\ell'}[x \mapsto v]$ and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ or $\xi_{\ell'}(x) \neq v$ then $s\chi' = \text{false}^{|s\chi|}$, or $\ell' \in \mathcal{L}_0$, $\xi'_{\ell'} = \xi_{\ell'}$ and $s\chi' = s\chi$. Additionally, $\chi' = \chi_k$, $\gamma' = \llbracket S_{i+1}; \dots; S_n \rrbracket$.
5. If $c_j = \mathbf{cons}(\theta)$, then, with $\chi = \chi_k; v_1; \dots; v_c$ and $\text{Pars}_{\mathcal{P}}(\theta) = x_1 : T_1; \dots; x_c : T_c$, it holds that $\chi' = \chi_k; \ell'$ for some ℓ' such that $\sigma(\ell') = \perp$ and $\sigma'(\ell') =$

$\langle \theta, \varepsilon, \varepsilon, \xi_\theta^0[\{x_1 \mapsto v_1, \dots, x_c \mapsto v_c\}] \rangle$, $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$, and it holds that $s\chi' = \text{false}^{\llbracket s\chi \rrbracket}$.

6. If $c_j = \mathbf{if}(C)$, with $\chi = \chi_k; b$, $\chi' = \chi_k$ and if $b = \text{true}$, $\gamma' = C; \llbracket S_{i+1}; \dots; S_n \rrbracket$ and if $b = \text{false}$, $\gamma' = \llbracket S_{i+1}; \dots; S_n \rrbracket$.
7. If $c_j = \mathbf{not}$, with $\chi = \chi_k; b$, $\chi' = \chi_k; \neg b$ and $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$.
8. If $c_j = \mathbf{op}(o)$, with $\chi = \chi_k; a; b$, then $\gamma' = c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket$ and, with o the semantic equivalent of o , $\chi' = \chi_k; c$ for some c such that all of the following hold:
 - (a) If $a \in T$ and $b \in T$ with $T \in \mathbb{T}$ and $o \in \{=, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
 - (b) If $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $o \in \{<=, >=, <, >, =, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
 - (c) If $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $o \in \{*, /, \%, +, ^, -\}$, then $c = a \circ b$ and $c \in \mathbb{Z}$,
 - (d) If $a, b \in \mathbb{N}$ and $o \in \{*, /, \%, +, ^\}$, then $c = a \circ b$ and $c \in \mathbb{N}$.
 - (e) If $a, b \in \mathbb{B}$ and $o \in \{\&, ||\}$, then $c = a \circ b$ and $c \in \mathbb{B}$.

Note that if at least one of the cases in the first case distinction of Property 5 holds, exactly one case holds, as due to the fact that they require c_j to be a specific command, all cases are mutually exclusive.

To prove this lemma, we first prove an auxiliary lemma:

Lemma 4.2.19 (Expression/Semantics Type Correspondence). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg \text{Done}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ and $\gamma \neq \varepsilon$. Let $\gamma = c_j; \gamma$, such that c_j has been generated from a statement S , with $\llbracket S \rrbracket = c_1; \dots; c_m$. Let $c_k \in c_1; \dots; c_j$ exist such that $c_k; \dots; c_j = \llbracket E \rrbracket$ for some expression E . Let E have type T in $\mathcal{P}$ according to the type system. Then if Property 5 holds for c_j , there exists a single value v that is put on the stack by the transition induced by c_j . Furthermore, either $T \in \mathcal{T} \setminus ID$ and v have the semantic type $\text{sem}(T)$, or $T \in \Theta_{\mathcal{P}}$ and v will be a label ℓ such that the struct type of $\sigma(\ell)$ is T .

Proof. This follows from a case distinction on the command c_j is and what expression E generates the commands $c_k; \dots; c_j$:

- If c_j is a push-command, then according to the interpretation function and the rules **ComPush** and **ComPushThis**, $v = \text{val}(g)$ and $E = g$, in which case $v \in \text{sem}(T)$, or $v = \ell$ and $E = \text{this}$, in which case E has type θ , $v = \ell$ and $\sigma(v)$ has the type θ .
- If c_j is a read-command, then according to Property 5, it is a read command of some variable x with type T_x . It follows that E has type T_x , as

$\mathcal{P}$ is well-formed (following rule **VarR**). Then, from Property 5, we then know that if $T_x \in \Theta_{\mathcal{P}}$, $v \in \mathcal{L}$ and $\sigma(v)$ has struct type T_x . Furthermore, if $T_x \in \mathcal{T} \setminus ID$, according to Property 5, we know that $v \in \text{sem}(T_x)$.

- If c_j is a constructor-command, then according to Property 5, we know that $v = \ell_x$ such that $\sigma(\ell_x)$ has struct type θ_x . As c_j is generated from E through the interpretation function, it follows by that function and the type system that E has type θ_x .
- If c_j is a not-command or an op-command, then by the fact that $\mathcal{P}$ is well-formed and c_j is generated by the interpretation function applied on E , E has a type $T \in \mathcal{T} \setminus ID$ and $v \in \text{sem}(T)$.

As the lemma holds in all cases, the lemma holds. $\square$

We use this auxiliary lemma in the proof of Property 5.

The proof of Property 5 is intricate and has a lot of overlapping cases, due to which we have decided to not give a single full proof, but to divide the proof into subproofs, including a full proof made up of these subproofs.

The proof takes the form of a strong induction. As there are many cases to consider in the step case of this proof, we have split the proof into subproofs for accessibility.

Full proof of Property 5. Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a well-formed AuDaLa state reached by some program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \text{ Sc}_{\mathcal{P}}$ for some Γ, Ω . Then assume that $\neg \text{Done}(\sigma)$. Let $\ell \in \mathcal{L}$ be a label such that $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ and $\gamma \neq \varepsilon$. Let $\gamma = c_j; \gamma$, such that c_j has been generated from a statement S , with $\llbracket S \rrbracket = c_1; \dots; c_m$.

Let c_k with $k \leq j$ such that $c_k; \dots; c_j = \llbracket E \rrbracket$ or $c_k; \dots; c_j = \llbracket S_i \rrbracket$. According to Property 4, this c_k is guaranteed to exist, so we assume c_k has the largest k possible for c_j . Let $p_k = \langle \Delta_k, \sigma_k, s\chi_k \rangle$ be the state right before the transition induced by c_k and let $\sigma_k(\ell) = \langle \theta, \gamma_k, \chi_k; \xi_k \rangle$.

To prove the lemma, we perform a strong induction on the total number of commands for which a transition induced by that command has been taken during the execution of $\mathcal{P}$. For the base case, refer to Subproof 4.2.19.A. For the step case, refer to Subproof 4.2.19.B. As the strong induction holds, Property 5 holds. $\square$

Subproof 4.2.19.A - Base case for strong induction. In the base case, we assume that c_j is the first command to be processed in the execution of $\mathcal{P}$. Then it follows that $c_j = c_k = c_1$, as c_j must be the first command of $\llbracket S \rrbracket$. This means that c_j is both the first and the last command of some expression or statement. By the definition of the interpretation function, this can only happen for the expressions g with $g \in LT$, **this**, null_T and $x_1 \dots x_0$. In all of those cases, $c_j = \text{push}(v)$ for some value $v \in \mathcal{V}$ or $c_j = \text{push}(\text{this})$.

This satisfies the first part of Property 5. Then, if $c_j = \mathbf{push}(v)$ for some $v \in \mathcal{V}$, a transition **CompPush** will be enabled to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket; \gamma', \chi; v, \xi' \rangle$. As $\chi = \chi_k$, this satisfies the property. If $c_j = \mathbf{push}(\mathbf{this})$, then a transition **CompPushThis** will be enabled to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket; \chi; \ell, \xi' \rangle$. As $\chi = \chi_k$, this satisfies the property. ■

Subproof 4.2.19.B - Step case for strong induction. In the step case of the second strong induction, we assume as the induction hypothesis that Property 5 holds for all previously processed commands. In particular, it holds for all commands $c_1; \dots; c_{j-1}$ in $\llbracket S \rrbracket$, and it holds for all write commands processed before c_j during the execution of $\mathcal{P}$.

We then do a case distinction on the commands c_j can be.

- The case where c_j is a **push**-command is analogous to the base case, given in Subproof 4.2.19.A.
- For the case where c_j is a **rd**-command, refer to Subproof 4.2.19.C.
- For the case where c_j is a **wr**-command, refer to Subproof 4.2.19.D.
- For the case where c_j is a **cons**-command, refer to Subproof 4.2.19.E.
- For the case where c_j is an **if**-command, refer to Subproof 4.2.19.F.
- For the case where c_j is a **not**-command, refer to Subproof 4.2.19.G.
- For the case where c_j is an **op**-command, refer to Subproof 4.2.19.H.

As all the cases hold, it follows that the step case for the strong induction over d also holds. ■

Subproof 4.2.19.C - Detailed step case rd. If $c_j = \mathbf{rd}(x)$ for some x , then $c_k = \mathbf{push}(\mathbf{this})$ and the expression E is a variable expression $x_1. \dots .x_h.x$, as seen in the proof for Property 4. As $\mathbf{rd}(x)$ was generated by the interpretation function, it follows that there exists a variable x_h with $h \geq 0$ such that $E = x_1. \dots .x_h.x$. It follows that $E = E'.x$ for $E' = x_1. \dots .x_h$, and that we can apply the induction hypothesis on command c_{j-1} , as $c_k; \dots; c_{j-1} = \llbracket E' \rrbracket$ and the distance between c_k and c_{j-1} is $d-1$.

From Property 5 and Lemma 4.2.19 for c_{j-1} we then know that there was a transition from a state $p_h = \langle \Delta_h, \sigma_h, s\chi_h \rangle$ with some ℓ_h such that $\sigma_h(\ell_h) = \langle \theta_h, \gamma_h, \chi_h, \xi_h \rangle$ (for some $\theta_h, \gamma_h, \chi_h, \xi_h$) and $\chi = \chi_k; \xi_h(x_h)$, for some x_h defined in S_i . We also know that $\xi_h(x_h) \in \mathcal{L}$, as E is well-formed through $\mathcal{P}$, so $x_1. \dots .x_h$ leads to a label.

Furthermore, we know from the induction hypothesis that Property 5 holds for all previously processed write commands **wr**(y) during the execution of $\mathcal{P}$, where y has syntactic type $T_y \in \mathcal{T}$ and v_y is the new value written by the command. Due to that, if $T_y \in \Theta_{\mathcal{P}}$, then $v_y \in \mathcal{L}$ and v_y represents a struct

instance with type T_y , and if $T_y \in \mathcal{T} \setminus ID$, then $v_y \in \text{sem}(T_y)$. We also know by the definition of the initial state of $\mathcal{P}$ and the fact that null-parameters cannot be overwritten during the execution of $\mathcal{P}$, that for any null-parameter z of syntactic type $T_z \in \mathcal{T}$, if $T_z \in \Theta_{\mathcal{P}}$ then $v_z = \ell_{T_z}^0$, which represents the null-instance of type T_z , and if $T_z \in \mathcal{T} \setminus ID$, then $v_z = \text{defaultVal}(T_z)$, so $v_z \in \text{sem}(T_z)$.

As AuDaLa does not delete struct instances, it follows that $\sigma(\xi_h(x_h)) = \langle \theta', \gamma', \chi', \xi' \rangle$ (for some $\theta', \gamma', \chi', \xi'$) and we also know, with θ' the struct type of $\sigma(\xi_h(x_h))$, that $x \notin \text{Pars}_{\mathcal{P}}(\theta')$ implies that $\xi_h(x_h) = \ell$, as otherwise $x_1 \cdots x_h.x$ would not be admitted by the type rule **VarR**. With this, the first precondition for the **rd** command is satisfied.

As $\mathcal{P}$ is well-formed, we know that there was either a statement $S_q = (T \ x := E_x)$ with $1 \leq q < i$ and $E_x \in EX$ or that $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta')$ for some $T \in \mathcal{T}$, as otherwise $x_1 \cdots x_h.x$ would not be provable in the type system. This satisfies the second precondition for the **rd** command.

From the earlier application of the induction hypothesis on all previous write-commands, we also know that if $T \in \Theta_{\mathcal{P}}$, it holds that $\xi(x) \in \mathcal{L}$ and as AuDaLa does not delete struct instances, it holds that $\sigma(\xi'(x)) = \langle T, \gamma_x, \chi_x, \xi_x \rangle$ (for some γ_x, χ_x, ξ_x). Furthermore, if $T \in \mathcal{T} \setminus ID$, it holds that $\xi(x) \in \text{sem}(T)$. With this, the third precondition for the **rd** command is satisfied.

These preconditions also satisfy the requirements for the **ComRd** transition, so there exists an enabled transition **ComRd** to a state $p' = \langle \Delta', \sigma', s_{\chi'} \rangle$ with

$\sigma'(\ell) = \langle \theta, c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k; \xi_h(x), \xi' \rangle$, by definition of **ComRd**. This satisfies the postconditions of the command **rd**. ■

Subproof 4.2.19.D - Detailed step case wr. If $c_j = \mathbf{wr}(x)$ for some x , then $c_j = c_m$, by the definition of the interpretation function. It follows that $c_k = c_1$ and that $c_k; \dots; c_m = \llbracket S_i \rrbracket$. By the interpretation function, it follows that $\llbracket S_i \rrbracket = \llbracket E' \rrbracket; \llbracket x_1 \cdots x_h \rrbracket; \mathbf{wr}(x)$ for some expression E' and some variable expression $x_1 \cdots x_h$. By the induction hypothesis on the commands $\llbracket E' \rrbracket = c_1; \dots; c_a$, we know that after command c_a , we are in a state $p_a = \langle \Delta_a, \sigma_a, s_{\chi_a} \rangle$, with $\sigma_a(\ell) = \langle \theta, c_{a+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k; v, \xi_a \rangle$ for some value $v \in \mathcal{V}$. By the induction hypothesis on the commands $\llbracket x_1 \cdots x_h \rrbracket = c_{a+1}; \dots; c_{m-1}$ and analogous reasoning as in the case above, we know that $\sigma(\ell) = \langle \theta, c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k; v; \ell_h, \xi_a \rangle$ for some $\ell_h \in \mathcal{L}$, with $\sigma(\ell_h) = \langle \theta_h, \gamma_h, \chi_h, \xi_h \rangle$ for some $\theta_h, \gamma_h, \chi_h, \xi_h$ and, if $x \notin \text{Pars}_{\mathcal{P}}(\theta_h)$, with $\ell' = \ell$. This satisfies the first precondition of the **wr** command.

Again, as $\mathcal{P}$ is well-formed, we know that there was either a statement $S_q = (T \ x := E_x)$ with $1 \leq q \leq i$ and $E_x \in EX$ or that $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_h)$ for some $T \in \mathcal{T}$, as otherwise $x_1 \cdots x_h.x$ would not be provable in the type system. This satisfies the second precondition of the **wr** command.

Then, to establish the third precondition, note that by the induction hy-

pothesis for the strong induction, we know that Property 5 holds for commands $c_1; \dots; c_a$. We then know that v has the semantic type which reflects E' in p_a , due to Lemma 4.2.19. Furthermore, as AuDaLa does not remove struct instances and keeps struct types of struct instances immutable, we can extend this to p . As $\mathcal{P}$ is well-formed, we also know that E' has the same syntactic type as x . It follows that the third precondition is satisfied.

Then if $\ell_h \notin \mathcal{L}_0$, there exists a transition **ComWr** to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k, \xi' \rangle$ and $\sigma'(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi'_{\ell'} \rangle$ such that $\xi'_{\ell'} = \xi_{\ell'}[x \mapsto v]$ and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'}) \vee \xi_{\ell'}(x) \neq v$ then $s\chi' = \text{false}^{|s\chi|}$ by definition of **ComWr**.

If $\ell_h \in \mathcal{L}_0$, there exists instead a transition **ComWrNSkip** to a state $p' = \langle \Delta', \sigma', s\chi \rangle$ with $\sigma'(\ell) = \langle \theta, \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k, \xi' \rangle$ and $\sigma'(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ by definition of **ComWrNSkip**. ■

Subproof 4.2.19.E - Detailed step case cons. If $c_j = \text{cons}(\theta)$ for some struct type θ , then it must have been generated as the last command of an expression or statement $\theta(E_1, \dots, E_h)$. It then follows that the largest possible c_k is the first command of $\theta(E_1, \dots, E_h)$. Then let $\text{Pars}_{\mathcal{P}}(\theta) = (x_1 : T_1); \dots; (x_h : T_h)$. It follows from repeated application of the induction hypothesis and Lemma 4.2.19 that every expression $E_i \in E_1, \dots, E_h$ leaves a value $v \in \mathcal{V}$ on the stack, such that $\chi = \chi_k; v_1; \dots; v_h$. Due to Lemma 4.2.19, as Property 5 holds for all commands in $\llbracket E_1; \dots; E_h \rrbracket$ due to the induction hypothesis, we know that for every expression $E_i \in E_1, \dots, E_h$ that v_i has the semantic type which reflects E_i after it has been added to the stack, which we extend to p as AuDaLa does not remove struct instances or change their struct types. As $\mathcal{P}$ is well-formed, x_i has the same syntactic type as E_i , so it follows that for all $x_i : T_i$, either $T_i \in \mathcal{T} \setminus ID$ and $v_i \in \text{sem}(T_i)$ or $T_i \in \Theta_{\mathcal{P}}$ and $v_i \in \mathcal{L}$ such that $\sigma(v)$ has struct type T_i .

Then by the definition of the transition **ComCons** with a fresh label ℓ' , there exists a transition to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, c_{j+1}; \dots; c_m; \llbracket S_{i+1}; \dots; S_n \rrbracket, \chi_k; \ell', \xi' \rangle$ and $\sigma'(\ell') = \langle \theta, \varepsilon, \varepsilon, \xi_{\theta}^0[\{p_1 \mapsto v_1, \dots, p_h \mapsto v_h\}] \rangle$ and $s\chi' = \text{false}^{|s\chi|}$. ■

Subproof 4.2.19.F - Detailed step case if. If $c_j = \text{if}(C)$ for some command list C , then c_j must have been generated as part of an if-statement, such that $c_j = c_m$ and $\llbracket S_i \rrbracket = \llbracket E' \rrbracket; \text{if}(\llbracket S \rrbracket)$ for some expression E' and some list of statements S . It follows that $c_k = c_1$. Then by application of the induction hypothesis and Lemma 4.2.19 on $\llbracket E' \rrbracket = c_1; \dots; c_{m-1}$, we know that $\chi = \chi_k; b$, where $b \in \mathbb{B}$.

Then by the definition of the transitions **ComIfT** and **ComIfF**, there exists a transition to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, \gamma', \chi_k, \xi' \rangle$ such that if $b = \text{true}$, $\gamma' = C; \llbracket S_{i+1}; \dots; S_n \rrbracket$. and if $b = \text{false}$, $\gamma' = \llbracket S_{i+1}; \dots; S_n \rrbracket$. ■

*Subproof 4.2.19.G - Detailed step case **not**.* If $c_j = \mathbf{not}$, it must have been generated as the last command of an expression $!E'$ for some expression E' , where E' is of type $\mathbf{Bool}$ as $\mathcal{P}$ is well-formed. It follows that the largest possible c_k is the first command of $[[E'] = [E]; \mathbf{not}$. By application of the induction hypothesis on $[[E']]$ together with Lemma 4.2.19, we then know that $\chi = \chi_k; b$, and we know that $b \in \mathbb{B}$ due to Lemma 4.2.19.

Then by the definition of the transition **ComNot** there exists a transition to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, [S_{i+1}; \dots; S_n], \chi_k; \neg b, \xi' \rangle$. ■

*Subproof 4.2.19.H - Detailed step case **op**.* If $c_j = \mathbf{op}(\circ)$ for some syntactic operator $\circ$, it must have been generated as the last command of an expression $E_1 \circ E_2$. It follows that the largest possible c_k is the first command of $[[E_1 \circ E_2] = [E_1]; [E_2]; \mathbf{op}(\circ)]$. Then, by induction hypothesis on $[[E_1]]$ and $[[E_2]]$, and the relation between the interpretation function and the stack in the semantics, we know that $\chi = \chi_k; a; b$ for $a, b \in \mathcal{V}$. From Lemma 4.2.19, we know that if E_1 has type T_1 , then either $T_1 \notin \Theta_{\mathcal{P}}$, and $a \in \text{sem}(T_1)$ or $T_1 \in \Theta_{\mathcal{P}}$, $a \in \mathcal{L}$ and, as AuDaLa does not delete struct instances, $\sigma(a)$ is a struct instance with type T_1 . We know the same for E_2 and b .

We then know that if $\circ \in \{=, !=\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$ or there exists a $T \in \{\mathbb{N}, \mathbb{Z}, \mathbb{B}, \text{String}\} \cup \Theta_{\mathcal{P}}$ s.t. either $T \notin \Theta_{\mathcal{P}}$ and $a, b \in T$ or $T \in \Theta_{\mathcal{P}}$ and a and b are labels of struct instances with type T , by the type rules **Eq** and **Comp**.

By the type rules **Comp**, **NArith** and **IArith**, we know that if $\circ \in \{<=, >=, <, >, =, !=, *, /, \%, +, ^, -\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$, and by the type rule **BinLog** we know that if $\circ \in \{\&\&, ||\}$ then $a, b \in \mathbb{B}$.

Then by the definition of **ComOp**, there exists a transition to a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, c_{j+1}; \dots; c_m [S_{i+1}; \dots; S_n], \chi_k; c\xi' \rangle$ such that:

1. if $a \in T$ and $b \in T$ with $T \in \mathbb{T}$ and $\circ = \{=, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
2. if $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $\circ = \{<=, >=, <, >, =, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
3. if $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $\circ = \{*, /, \%, +, ^, -\}$, then $c = a \circ b$ and $c \in \mathbb{Z}$,
4. if $a, b \in \mathbb{N}$ and $\circ = \{*, /, \%, +, ^\}$, then $c = a \circ b$ and $c \in \mathbb{N}$ and
5. if $a, b \in \mathbb{B}$ and $\circ \in \{\&\&, ||\}$, then $c = a \circ b$ and $c \in \mathbb{B}$.

It follows that this step case holds. ■

With this, all the subproofs have been given; we conclude that Property 5 as given in Lemma 4.2.18 holds. Then the only property which we still need to prove is the sixth property:

Lemma 4.2.20 (Property 6 of well-formed programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a well-formed AuDaLa state reached by some program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that

$\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $Done(\sigma)$ does not hold in p , then after a finite amount of transitions there must be a transition to some state $p' = \langle \Delta', \sigma', s\chi' \rangle$ for which $Done(\sigma')$ holds.

Proof of Property 4.2.20. Let $p = \langle \Delta, \sigma, s\chi \rangle$ be a well-formed AuDaLa state reached by some program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Then assume that $\neg Done(\sigma)$. Let k be the total amount of commands still present in some command list of some struct instance. As $Done(\sigma) = false$ we know that $k > 0$. Then the amount of transitions enabled directly corresponds to the commands c such that for some $\ell' \in \mathcal{L}$, $\sigma(\ell') = \langle \theta', c; \gamma', \chi', \xi \rangle$. It follows that after a transition to a state p_1 , there are $k - 1$ commands in total still present in some command list of some struct instance, and the transitions enabled are similarly bound to commands as in p . As $\mathcal{P}$ is well-formed, steps in $\mathcal{P}$ are finite, so k must be a finite number. Then it follows that after a finite amount of transitions, we arrive in a state $p' = \langle \Delta', \sigma', s\chi' \rangle$ such that $Done(\sigma') = true$. $\square$

We then conclude this section with the following theorem:

Theorem 4.2.21 (Properties of well-formed AuDaLa programs). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Then, for p , the following properties hold:

- Property 1, as specified by Lemma 4.2.14,
- Property 2, as specified by Lemma 4.2.15,
- Property 3, as specified by Lemma 4.2.16,
- Property 4, as specified by Lemma 4.2.17,
- Property 5, as specified by Lemma 4.2.18,
- Property 6, as specified by Lemma 4.2.20.

4.3 Corollaries and Type Soundness

In this section, we give some corollaries resulting from Theorem 4.2.21, including the *type soundness* of our type system. We firstly define *result values* for expressions and discuss those in the scope of programs without read-write race conditions. Then, we assert that our type system is sound. Lastly, we give some corollaries to reason directly about the result of executing syntactic statements.

Expression results

First, we define *result values* to extrapolate the results on command executions to expressions:

Definition 4.3.22 (Expression Results). We define the *result (value)* of an execution of an expression E by a struct instance s in some step F in a well-formed program $\mathcal{P}$ to be a value v resulting from the transition induced by the last command c_m of $\llbracket E \rrbracket$ in the command list of s as per Theorem 4.2.21.

There always is at least one resulting value for every expression, by multiple applications of Property 5 on all commands of the expression. However, there may be multiple values, because we allow for interleaving during the execution of an expression E by struct instance s . If E contains a read to a variable which is not deterministic during F , this can lead to multiple result values for E . The following holds, however:

Lemma 4.3.23. Let E be an expression in a well-formed program $\mathcal{P}$ executed by a struct instance s from some state p . Let $x_1, \dots, x_n$ be the parameters referenced during E . If all parameters $x_i \in x_1, \dots, x_n$ are deterministic during the execution, then there exists only a single result value for E executed by s during $\mathcal{P}$.

Proof. If all of the parameters of E are deterministic, it follows that any computations on them have a deterministic outcome, so the value returned by the execution must also be deterministic. $\square$

In this case, we say that the result (value) of E is deterministic.

In this thesis, we are interested in proving programs without read-write race conditions from some premise state p_1 . In program executions without read-write race conditions from p_1 , all resulting values from all expressions are deterministic, as the fact that parameters are read in an expression E executed by a struct instance s during the execution of a step F from some state p means they cannot be written to during F . It follows that in a program without read-write race conditions from p_1 , expression results are deterministic.

Type soundness

We then establish *type soundness* for AuDaLa, starting with a small overview of what that means for a type system. Type soundness was first defined by Milner [128] as the property that well-typed programs do not go “wrong”. This is further expanded upon by Wright and Felleisen [180], which identify this as *weak type soundness*, and more specifically define it as the absence of type errors during execution. They give a definition of *strong type soundness*, stating that if a program (or expression) is evaluated to be of some type by

the type system, the value returned from evaluating that program also has that type. They then give the well-known syntactic approach for proving type soundness based on progress and preservation.

Here, weak type soundness is indeed weaker than strong type soundness. Weak type soundness will be satisfied if there are no type errors, even if there are type mismatches in the program (as long as they don't incur a type error). Strong type soundness does not allow for type mismatches between the program and its evaluation.

In our language, there are essentially two levels of the semantics. The first level works directly on the syntax and deals with the execution of the schedule. The second level works on commands generated by the syntax and deals with the execution of the statements of steps occurring in the schedule. To establish type soundness for AuDaLa, we have to consider the type soundness of both levels.

Due to the fact that a good part of the semantics does not directly work on the syntax of the program, we cannot use progress and preservation. As defining a type system on the commands does not have a use beyond proving type soundness using the progress and preservation method (for commands only, not for the schedule), we have decided to instead establish type soundness using the semantics directly.

To prove the type soundness of our type system, we will need to prove that in every state of a well-formed program $\mathcal{P}$, weak and strong soundness hold for the execution of $\mathcal{P}$. We first establish weak soundness. To do this, we first have to define when a type error occurs in AuDaLa:

Definition 4.3.24 (Type Errors in AuDaLa). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state. We define a *type error* to occur in p iff one of the following holds:

1. The schedule Δ is neither empty nor well-formed modulo *aFix*.
2. The values on the stack of a struct instance in σ are not compatible with a command transition enabled for that struct instance.
3. A transition is enabled in some struct instance of σ that writes a value to a variable environment such that the variable written to and the value have different types.

Note that as the transitions on the schedule match with the first entry in the schedule, a schedule that is neither empty nor well-formed will terminate the program early, instead of incur a visible type error. We still count it though, as the error has been incurred by a mismatch in expected schedule content and actual schedule content.

Note then that AuDaLa's type system is weakly sound:

Corollary 4.3.25 (Weak type soundness for the type system of AuDaLa). There are no type errors in any state encountered during any execution of a well-formed AuDaLa program $\mathcal{P}$.

Proof. This follows from Theorem 4.2.21. The first property directly contradicts that an occurrence of the first possible type error can happen in $\mathcal{P}$. The fifth property contradicts the occurrence of the second and third types of type error with its strong requirements on the types of the values on the stack and the values written to variable environments. $\square$

Then, to prove that strong soundness holds, note that the only dynamic parts of an AuDaLa program that have types are the schedule and expressions used in steps. As we already established that the schedule is well-formed modulo *aFix* right until it is empty (which is expected, as if the schedule is empty the program terminates), we then only need to establish that the expressions evaluated during a program execution are evaluated to a value which matches type with the expression.

To do this, we define the following corollary, an addendum to Corollary 4.3.25, which is derived from Lemma 4.2.19:

Corollary 4.3.26 (Strong type soundness addendum). Let $\mathcal{P}$ be a well-formed program, and let E be an expression defined in a step of a struct type with type θ . Let E have type T in $\mathcal{P}$ according to the type system. Let p be an AuDaLa state reached by $\mathcal{P}$ from which E can be executed. Then either $T \in \mathcal{T} \setminus ID$ and the result value of E executed from p will have the semantic type $sem(T)$, or $T \in \Theta_{\mathcal{P}}$ and the result value of E executed from p will be a label ℓ such that the struct type of $\sigma(\ell)$ is T .

Proof. As Property 5 holds for every command executed during a well-formed program $\mathcal{P}$, we can apply Lemma 4.2.19 on all the commands constituting an expression during the execution of $\mathcal{P}$, from which the corollary follows. $\square$

With this, we then conclude the following corollary:

Corollary 4.3.27. The type system of AuDaLa is *sound*: it is both strongly and weakly sound.

Proof. By Corollary 4.3.25 and Corollary 4.3.26. $\square$

Execution of syntactic statements to proofs

Lastly, we continue with some more corollaries to make proving in AuDaLa more streamlined. Firstly, on a more general level, we know that the execution of a statement S in a step F in a well-formed program $\mathcal{P}$ by a struct instance s leads to the effects in Property 4.2.18 depending on the last command in $\llbracket S \rrbracket$. Intuitively, this means the following:

Corollary 4.3.28 (Execution Effects). Let S be a statement executed during the execution of a step F in a well-formed program $\mathcal{P}$ by a struct instance s . Let p' be a state resulting from the transition induced by the last command of $\llbracket S \rrbracket$ for s . Then:

1. If S is an update statement $x_1 \dots x_n. x := E$, then, with x the variable in the struct instance belonging to a result of an execution of $x_1 \dots x_n$, if x is not a parameter of a null-instance, x will be updated to a result of an execution of E during the execution of S in p' , and all values in the stability stack will be reset to *false* if this means that a parameter has changed. If x is a parameter of a null-instance, E is executed and any struct instances created during the execution of E are still present in p' , but x has not been updated.
2. If S is a variable assignment statement $T \ x := E$, then it has the same effect as executing the update statement $x := E$.
3. If S is an if-statement, in p' , E will be executed to a result $b \in \mathbb{B}$. If $b = \text{true}$, the if-clause will be taken. If $b = \text{false}$, the if-clause will be skipped.
4. If S is a constructor statement $\theta(E_1, \dots, E_m)$, then in p' , there will be a new struct instance for a fresh label ℓ with type θ and its parameters set to results of executions of $E_1, \dots, E_m$, and all values in the stability stack will be reset to *false*. Additionally, ℓ will be the last value on the stack of s .

Additionally, let $\mathcal{E}$ be the expressions executed as a part of S . For any expression $E \in \mathcal{E}$, if E is a constructor expression $\theta(E_1, \dots, E_m)$, the fourth point above will also hold for E , except that the fresh label ℓ will not necessarily be the last value on the stack of s . Lastly, the effects of a statement will be deterministic if all results from all expressions in S are deterministic.

Proof. Follows from applications of Theorem 4.2.21 and Lemma 4.3.23. The fact that the if-condition will resolve to a boolean value follows from Corollary 4.3.26. $\square$

In this thesis, we often consider only AuDaLa programs without read-write race conditions from some premise state, in which case the direct effects of all statements are deterministic. However, we do not exclude write-write race conditions, so we need to weaken the above Corollary to find the permanent results of executing update statements in a step when write-write race conditions can occur:

Corollary 4.3.29 (Permanent Update Results). Let $\mathcal{P}$ be a well-formed program without read-write race conditions from some premise state p_1 , with a step F . Let F be executed from some state p and let s be a struct instance in

p . Let S be the last update statement of some parameter $s'.x$ of some struct instance s' by s , which updates $s'.x$ to a value v . Let S be the last update statements of $s'.x$ across all definitions of F for any struct type θ . Then we can guarantee that after the execution of F from p :

- a. If s' is a null-instance, $s'.x = \text{null}$.
- b. If s' is not a null-instance:
 - i. If $s'.x$ is not involved in a write-write race condition, $s'.x = v$.
 - ii. If $s'.x$ is involved in a write-write race condition in F , let N be the set of all possible values written to $s'.x$ during the execution of any statement from S by any struct instance during the execution of F from p . Then $s'.x \in N$.

Additionally, we know that if $s'.x$ has had its value changed during the execution of F , all values in the stability stack have been reset to *false* during the execution of F .

Proof. Follows from applying Corollary 4.3.28 on all statements executed during the execution of F from p . $\square$

4.4 Conclusion

In this section, we have established some basic definitions for analysing AuDaLa programs and some basic features of AuDaLa programs and well-formed AuDaLa programs, including Theorem 4.2.21. We have established the soundness of our type system with Corollary 4.3.27, and we have given some more corollaries and auxiliary definitions which make it easier to reason about programs.

With the corollaries proven in this section, one can prove the effects of an AuDaLa step F by hand on the level of the syntactic code of F in the program, without transforming that behaviour into commands, using the above corollaries to formally establish the behaviour of statements. For variables x deterministic during the execution of F , one only has to consider the final value of x relative to a single struct instance, which can then be generalised to all struct instances. This can be done by sequentially walking through the code of F . For variables x involved in write-write race conditions, one will need to construct the set N as defined in Corollary 4.3.29.

With that, we are ready to prove some of our example programs correct in Chapter 5.

Chapter 5

Proving in AuDaLa

In this chapter, we prove some of our examples from Chapter 3 correct by using the properties of well-formed AuDaLa programs as established in Chapter 4. During the proofs, we consider the program in a vacuum; we assume that if something is not changed by the program as a whole, it is not changed at all. With this, we rule out outside interference by hardware. As we want to prove the correctness of AuDaLa programs, not AuDaLa program implementations, we consider this to be a fair assumption. We also omit the type system proof derivation to prove both programs well-formed.

In this chapter, we prove three example programs correct. In Section 5.1, we prove our AuDaLa program for prefix sum as given in Section 2.1.2 correct. In Section 5.2, we prove the sorting AuDaLa program given in Section 3.4.2 correct. In Section 5.3, we prove the Turing machine implementation of Section 3.5 correct, hereby also proving AuDaLa Turing complete. We conclude the chapter in Section 5.4.

The first two proofs straightforwardly use the results from Chapter 4. The last proof proves our implementation correct by establishing a *stuttering equivalence* between the behaviours of the program and the behaviours of the Turing machine, which will later be used in Chapter 6 to prove another version of the AuDaLa semantics conditionally equivalent to the standard semantics.

5.1 Proving Prefix Sum Correct

In this subsection, we prove that the code as given in Listing 2.3 correctly computes the prefix sum. To prove the code in Listing 2.3 correct, we first consider the steps of this code, for which we prove the execution results. The strategy is to first consider which parameters are in a race condition, if any,

using information on the states from which the step is executed. Then, we can establish the results of the execution of the step on the parameters using Corollary 4.3.29. For parameters not involved in race conditions, this consists of walking through the step sequentially and establishing the last value written to the parameter. After we have considered the execution results of all the steps, we combine them into contracts for one iteration of the fixpoints present in the schedule. These we then use to prove the execution results of fixpoints, from which we establish the final result of executing the code. If this result corresponds to the requirements as stated in the problem, it is correct provided the state from which it is executed is a valid premise state.

The rest of the section follows the outline as explained above, but first we restate the problem and give the initialization requirements.

Problem statement and initialization

The problem is restated as given in Problem 5.1.1:

Problem 5.1.1 (Prefix Sum). Given a sequence of n integer values $v_1, \dots, v_n$, compute for each position $1 \leq k \leq n$ in the sequence the sum $\sum_{i=1}^k v_i$. We assume that the sequence is given as a single, finite list without loops.

As the code does not describe initialization, we need to specify the requirements of a premise state from which this code can be executed with the intended effect. These requirements have already been stated in Section 2.1.2, but are formalized here:

Definition 5.1.2 (Premise of Listing 2.3). We define a premise state of the prefix sum code as a state $p_1 = \langle \Delta, \sigma, s, \chi \rangle$ such that there are labels $\ell_1, \dots, \ell_n$ for positions $s_1, \dots, s_n$ where $\sigma(\ell_1) = \langle \text{Position}, \varepsilon, \varepsilon, \xi_1 \rangle$ is the first position s_1 , with $s_1.\text{val} = v_1$ and $s_1.\text{prev} = \text{null}$, and for every s_i with $1 < i \leq n$, $s_i = \sigma(\ell_i) = \langle \text{Position}, \varepsilon, \varepsilon, \xi_i \rangle$, with $s_i.\text{val} = v_i$ and $s_i.\text{prev} = \ell_{i-1}$.

Step execution results

We then prove the execution results of the steps. For Listing 2.3, instead of considering variables separately we prove that all steps are deterministic as a whole:

Lemma 5.1.3. All steps in Listing 2.3 are deterministic.

Proof. This follows from the fact that no step in Listing 2.3 has potential race conditions:

- In *read*, a potential race condition would need to include either *auxval* or *auxprev*, as those are the parameters written to in *read*. However, these

parameters are not accessed indirectly, so there can be no potential race conditions on them, following from Definition 4.1.9. Therefore, *read* has no potential race conditions.

- In *write*, no parameters used are accessed indirectly, which is required for a potential race condition to exist, according to Definition 4.1.9. Therefore, *write* has no potential race conditions.

Therefore, no step in Listing 2.3 has potential race conditions. It follows from Lemma 4.1.10 that no step in Listing 2.3 has race conditions. It follows by Lemma 4.1.6 that all steps in Listing 2.3 are deterministic. $\square$

As all steps are deterministic, we can walk through a step sequentially to determine its effects, as established in Section 4.4. That way, we establish the following contracts for *read* and *write* in Listing 2.3. Here, we make use of *reference notation*, as defined in Definition 4.1.1.

Lemma 5.1.4 (*read Contract*). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{\text{Position}}$ be the labels of *Position* structs in p . Executing *read* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every $\ell_s \in \mathcal{L}_{\text{Position}}$ such that $s = \sigma(\ell_s) = \langle \text{Position}, \gamma, \chi, \xi_s \rangle$, let s' be $\sigma'(\ell_s)$. Then $s'.auxval = s.prev.val$ and $s'.auxprev = s.prev.prev$.

Proof. Let p, p', s and s' be as defined in the lemma. As *read* is deterministic, we can sequentially walk through the statements of the *read* step.

By applying Corollary 4.3.28 to the first statement of *read*, it follows that $s'.auxval = s.prev.val$. By applying it to the second statement of *read*, we know that $s'.auxprev$ is equal to $s.prev.prev$. Note that if $\ell_s \in \mathcal{L}_0$, all the parameters of s are set to null, so $s.prev.val = 0$ and $s.prev.prev = \ell_{\text{Position}}^0$, and additionally $s.auxval = 0$ and $s.prev = \ell_{\text{Position}}^0$. Therefore, while the updates are ignored, the lemma still follows. $\square$

Lemma 5.1.5. Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{\text{Position}}$ be the labels of *Position* structs in p . Executing *write* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle \text{Position}, \gamma, \chi, \xi_s \rangle$, let s' be $\sigma'(\ell_s)$. Then $s'.val = s.val + s.auxval$ and $s'.prev = s.auxprev$.

Proof. Let p, p', s and s' be defined in the lemma. As *write* is deterministic, we can sequentially walk through the statements of the *write* step. By applying Corollary 4.3.28 to all statements, we get that $s'.val = s.val + s.auxval$ and $s'.prev = s.auxprev$. Again, if $\ell_s \in \mathcal{L}_0$, as all parameters are set to null, $s.val + s.auxval = 0 + 0 = 0$, and $s.auxprev = \ell_{\text{Position}}^0$, but also $s.val = 0$ and $s.prev = \ell_{\text{Position}}^0$. Therefore, while the updates are ignored, the lemma still follows. $\square$

Schedule execution results

We then consider the schedule. As the fixpoint in the schedule first executes *read* and then *write*, we define the following combined contract for *read* < *write*:

Lemma 5.1.6 (Fixpoint execution). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{\text{Position}}$ be the labels of *Position* structs in p . Executing *read*<*write* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi' \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_p) = \langle \text{Position}, \gamma, \chi, \xi_s \rangle$, let s' be $\sigma'(\ell_s)$. Then $s'.val = s.val + s.prev.val$, $s'.prev = s.prev.prev$, $s'.auxval = s.prev.val$ and $s'.auxprev = s.prev.prev$.

Proof. Follows from Lemmas 5.1.4 and 5.1.5. $\square$

We now prove the following two lemmas. As the proof for these lemmas can be efficiently combined, we prove them together:

Lemma 5.1.7 (Termination). Executing *Fix*(*read* < *write*) terminates.

Lemma 5.1.8 (Result). Executing *Fix*(*read* < *write*) starting at premise state $p_1 = \langle \Delta_1, \sigma_1, s\chi_1 \rangle$ results in a state $p' = \langle \Delta', \sigma', s\chi' \rangle$. In p' , for every ℓ_i such that position $s_i = \sigma(\ell_i)$ with $1 \leq i \leq n$, let s'_i be $\sigma'(\ell_{s_i})$. Then $s'_i.val = \sum_{k=1}^i v_k$.

Proof of Termination and Result. For the first lemma, the fixpoint terminates when an iteration is reached which does not change any of the parameters. We need to prove that this iteration is eventually reached. The second lemma is more straightforward.

To prove both, we will first prove that after every iteration j of the fixpoint, $s'_i.val = \sum_{k=h}^i v_k$, with $h = \max(1, i - 2^j + 1)$ and $s'_i.prev = \ell_{i-2^j}$ or *null* if $i - 2^j \leq 0$ for any $1 \leq i \leq n$, by induction on j .

$j = 1$. For the first iteration, we start at premise state p_1 and end at a state p_2 . In p_2 , according to Lemma 5.1.6, $s'_i.val = s_i.val + s_i.prev.val = s_i.val + s_{i-1}.val$, with $s_{i-1} = \sigma_1(\ell_{i-1})$. If $i - 1 < 1$, then $s_{i-1} = \text{null}$, as $s_1.prev = \text{null}$, so $s_{i-1}.val = 0$ and then $s_i.val + s_{i-1}.val = v_1$. Then due to this and the premise, $s'_i.val = s_i.val + s_{i-1}.val = \sum_{k=h}^i v_k$, with $h = \max(1, i - 1) = \max(1, i - 2^j + 1)$, as $j = 1$.

We also know from Lemma 5.1.6 that $s'_i.prev = s_i.prev.prev$. If $i - 2 > 0$, then $i - 1 > 0$, so with $s_i.prev = \ell_{i-1}$ and $s_{i-1} = \sigma(\ell_{i-1})$, $s_{i-1}.prev = \ell_{i-2}$, so $s'_i.prev = \ell_{i-2} = \ell_{i-2^j}$. If $i - 2 \leq 0$, then either $i - 1 \leq 0$, in which case $s_i.prev.prev = \text{null}$, or $i - 1 > 0$, in which case $s_{i-1}.prev = \text{null}$, so $s_i.prev.prev = \text{null}$ and $s'_i.prev = \text{null}$.

$j > 1$. By the induction hypothesis, when we start at state $p_{j-1} = \langle \Delta, \sigma, s\chi \rangle$ for iteration j , for any $s_i = \sigma(\ell_i)$, $s_i.val = \sum_{k=h}^i v_k$ with $h = \max(1, i - 2^{j-1} + 1)$ and $s_i.prev = \ell_{i-2^{j-1}}$ or *null* if $i - 2^{j-1} \leq 0$. Due to Lemma 5.1.6, it follows

that we end up in a state $p_j = \langle \Delta', \sigma', s\chi' \rangle$ such that for $s'_i = \sigma(\ell_i)$, $s'_i.val = s_i.val + s_i.prev.val$ and $s'_i.prev = s_i.prev.prev$.

From the IH we know that $s_i.val = \sum_{k=h}^i v_k$ with $h = \max(1, i - 2^{j-1} + 1)$. Then if $i - 2^{j-1} \leq 0$ we know that $i - 2^j \leq 0$ and that $s_i.prev = null$, so $s'_i.prev = null$, $s'_i.prev.prev = null$. Additionally, $h = 1$, so $s_i.val = \sum_1^i v_k$, and $s'_i.val = s_i.val + s_i.prev.val = \sum_1^i v_k + 0 = \sum_1^i v_k$.

If $i - 2^{j-1} > 0$, then we know that $s_i.prev = \ell_{i-2^{j-1}}$ with $s_{i-2^{j-1}} = \sigma(\ell_{i-2^{j-1}})$. Then either $i - 2^j \leq 0$ or not.

If $i - 2^j \leq 0$, then $(i - 2^{j-1}) - 2^{j-1} \leq 0$, and then by the IH we know that $s_{i-2^{j-1}}.prev = null$, so $s'_i.prev = s_i.prev.prev = null$. Otherwise, $s_{i-2^{j-1}}.prev = \ell_{i-2^j}$, so $s'_i.prev = \ell_{i-2^j}$.

As $i - 2^{j-1} > 0$, we know that $s_i.val = \sum_{k=i-2^{j-1}+1}^i v_k$. We also know that $s_{i-2^{j-1}}.val = \sum_{k=g}^{i-2^{j-1}} v_k$ with $g = \max(1, i - 2^j + 1)$ by the IH and the facts established in the last paragraph. Then $s'_i.val = s_i.val + s_{i-2^{j-1}}.val = \sum_{k=i-2^{j-1}+1}^i v_k + \sum_{k=g}^{i-2^{j-1}} v_k = \sum_{k=g}^i v_k$.

As this proves all parts of the IH for iteration j , the induction step holds.

Note that $null.prev = null$ and $null.val = 0$. As per the induction, for any s_i and any iteration j such that $2^j \geq i$, $s_i.prev = null$. Therefore the *prev* parameter for s_i will be stable in iteration $\lceil \log_2(i) \rceil + 1$.

Also note that in iteration $\log_2(i) + 1$, $s_i.val$ will be stable, as per the induction. Therefore, Lemma 5.1.7 holds; the fixpoint will terminate after $\lceil \log_2 n \rceil + 1$ iterations, as the last of the n elements is the last to become stable.

From the induction it holds that after $\log_2(n) + 1$ iterations $s'_i.val = \sum_{k=1}^i v_k$. Therefore Lemma 5.1.8 holds. $\square$

Conclusion

With the execution results of the schedule proven, we can conclude the following:

Theorem 5.1.9 (Correctness of Listing 2.3). When executing the AuDaLa code as given in Listing 2.3 from a premise state according to Definition 5.1.2 for a list of integer values $v_1, \dots, v_n$, in the resulting state, every *Position* s_i corresponding to value v_i with $1 \leq i \leq n$ has its *val* parameter set to $\sum_{k=1}^i v_k$.

Proof. Follows from Lemma 5.1.8 and the premise. $\square$

5.2 Proving Linked List Copy Sort Correct

In this subsection, we prove that the code as given in Listing 3.7 sorts correctly. We follow the same general outline as in Section 5.1, starting with restating the problem and initialization requirements.

Problem statement and initialization

The problem is restated as given in Problem 5.2.10:

Problem 5.2.10 (Sorting). Given a list of values $L_1 = v_1, \dots, v_n$ such that $v_1, \dots, v_n$ are distinct, positive values, output a list of values $L_2 = v'_1, \dots, v'_n$ such that $v'_1 < \dots < v'_n$ and for all $v_i \in L_1$, it holds that $v_i \in L_2$.

Following the informal requirements stated in Section 3.4, we define the premise state of Listing 3.7 as:

Definition 5.2.11 (Premise of Listing 3.7). We define a premise state of the Linked List Copy Sort code as a state $p_1 = \langle \Delta, \sigma, s\chi \rangle$ s.t:

- For every value v_i in $v_1, \dots, v_n$ there exists a struct instance s_i (in $s_1, \dots, s_n$) with label ℓ_i (in $\ell_1, \dots, \ell_n$) such that $s_i = \langle \text{OldElem}, \varepsilon, \varepsilon, \xi_i \rangle$ and $\sigma(\ell_i) = s_i$.
- There exists a non-null struct instance t_1 with label ℓ_{t_1} such that $t_1 = \langle \text{NewElem}, \varepsilon, \varepsilon, \xi_b \rangle$.
- For all s_i , $s_i.\text{val} = v_i$ with $v_i \in \{v_1, \dots, v_n\}$, $s_i.\text{place} = t_1$ and $s_i.\text{move} = \text{false}$.
- $t_1.\text{min} = \min(v_1, \dots, v_n)$, $t_1.\text{max} = \max(v_1, \dots, v_n)$ (which can be done with a method akin to prefix-sum), $t_1.\text{spl} = \lfloor (t_1.\text{max} - t_1.\text{min}) / 2 \rfloor + t_1.\text{min}$, $t_1.\text{next} = \ell_{\text{NewElem}}^0$, $t_1.p1 = \ell_{\text{OldElem}}^0$, $t_1.p2 = \ell_{\text{OldElem}}^0$, $t_1.\text{hasSplit} = \text{false}$ and $t_1.\text{done} = \text{true}$.

Step execution results

We then prove the results of executing the steps. We start by determining race conditions. Considering the steps, note that to both *checkStable* and *migrate*, write-write race conditions are integral to its function. Therefore, the determinism of steps in Listing 3.7 can be encapsulated by the following lemma:

Lemma 5.2.12. All of the following hold:

1. Only the parameter *place.done* can be involved in a race condition in the step *checkStable*, and if it is, it is involved in a write-write race condition.
2. Only the parameters *place.p1* and *place.p2* can be involved in a race condition in the step *migrate*, and if they are, they are involved in a write-write race condition.

3. The step *split* is deterministic.

Proof. We prove all of the points separately:

1. From Listing 3.7, the only struct executing *checkStable* is the struct *OldElem*, and through the implementation of *checkStable* in *OldElem*, we know that *place.done* is the only parameter that is written to, so only *place.done* has potential race conditions. As *place.done* is not read in the code of *checkStable*, any race condition *place.done* is involved in must be a write-write race condition.
2. This point holds following the same reasoning as the first point, except with the parameters *place.p1* and *place.p2* and the step *migrate*.
3. In *split*, no parameters used are accessed indirectly, which is required for a potential race condition to exist, according to Definition 4.1.9. Therefore, *split* has no potential race conditions. It follows that *split* has no race conditions (Lemma 4.1.10) and therefore, *split* is deterministic (Lemma 4.1.6). $\square$

With this lemma, we can establish contracts for the steps in Listing 3.7.

Lemma 5.2.13 (*migrate Contract*). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{OldElem} \subseteq \mathcal{L}$ be the labels of *OldElem* structs in p . Executing *migrate* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle OldElem, \gamma, \chi, \xi_s \rangle$, $s' = \sigma'(\ell_s)$, all of the following hold:

1. $s.move \wedge s.place.hasSplit \Leftrightarrow s'.place = s.place.next$,
2. $(s.val > s.place.spl) \Leftrightarrow s'.move = true$
3. $(s.val \leq s.place.spl) \Rightarrow$
 $\exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r \wedge r.place = s'.place \wedge s'.place.p1 = \ell),$
4. $(s.val > p.place.spl) \Rightarrow$
 $\exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r' \wedge r'.place = s'.place \wedge s'.place.p2 = \ell).$

Proof. Let p , p' , s and s' be as defined in the lemma. As the only parameters accessed by s involved in a race condition are $s'.place.p1$ and $s'.place.p2$, which do not have any bearing on point 1 of the lemma, we can walk deterministically through the lines relevant for point 1, lines 7 to 13. Through application Corollary 4.3.28, point 1 follows, as *place* can only be updated when the if-statement resolves to true.

For point 2 and 3, note that the parameter *move* is not involved in any race conditions (as per Lemma 5.2.12), we then know through application Corollary 4.3.28 that *move* is set to *false* for the struct instance of ℓ_s after line 13 and that $s.val > s.place.spl \Leftrightarrow s'.move = true$.

Then to prove that $(s.val \leq s.place.split) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r \wedge r.place = s'.place \wedge s'.place.p1 = \ell)$, consider that the update of the variable $s.place.p1$ may or may not be in a race condition with another struct instance q with a label ℓ_q updating the variable $r.place.p1$, as long as $r.place = s.place$ at the time of the update. Therefore, by Corollary 4.3.28 and Corollary 4.3.29, we know that after executing line 15, there exists a struct instance r such that $r.place = s.place$ and $s.place.p1 = \ell_r$.

As after line 10, the $place$ parameter is not further updated in the step code, it follows by Corollary 4.3.28 and Corollary 4.3.29 that after executing line 15, $\exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r' \wedge r'.place = s'.place \wedge s'.place.p1 = \ell)$.

When $s.val > s.place.split$, we know that $\ell_s \notin \mathcal{L}_0$, as then $s.val = 0$ and $s.place.split = 0$. Therefore, we know through Corollary 4.3.28 with line 19 that if $(s.val > s.place.split)$, $s'.move = true$. Proving that $(s.val > s.place.split) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r' \wedge r'.place = s'.place \wedge s'.place.p2 = \ell)$ uses the same reasoning as the last case, so point 4 holds. $\square$

Lemma 5.2.14 (*checkStable Contract*). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{OldElem} \subseteq \mathcal{L}$ be the labels of *OldElem* structs in p . Executing *checkStable* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle OldElem, \gamma, \chi, \xi_s \rangle$ and $s' = \sigma'(\ell_s)$, it holds that

$$(s.place.p2 \neq \ell_{OldElem}^0) \vee (s.val \leq s.place.split \wedge s.place.p1 \neq \ell_s) \Rightarrow s'.place.done = false$$

Proof. Holds by application of Corollaries 4.3.28 and 4.3.29. Note that applying Corollary 4.3.29 is this simple because only *false* can be written to $place.done$. $\square$

Corollary 5.2.15 (*checkStable Reverse Contract*). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{NewElem} \subseteq \mathcal{L}$ be the labels of *NewElem* structs in p and let $\mathcal{L}_{OldElem} \subseteq \mathcal{L}$ be the labels of *OldElem* structs in p . Executing *checkStable* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle NewElem, \gamma, \chi, \xi_s \rangle$ and $s' = \sigma'(\ell_s)$, it holds that

$$s'.done = false \Leftrightarrow (s.p2 \neq \ell_{OldElem}^0) \vee \exists \ell \in \mathcal{L}_{OldElem}. (\sigma(\ell).val \leq s.split \wedge s.p1 \neq \ell)$$

Proof. Follows from applying Lemma 5.2.14 on all old elements from the perspective of the result, which allows us to make the implication a bi-implication. $\square$

Lemma 5.2.16 (*split Contract*). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let $\mathcal{L}_{NewElem} \subseteq \mathcal{L} \setminus$ be the labels of *NewElem* structs in p . Let $\mathcal{L}_p$ be all labels used in p . Executing *split* starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In

p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle OldElem, \gamma, \chi, \xi_s \rangle$ and $s' = \sigma'(\ell_s)$, all of the following hold:

1. If $\ell_s \in \mathcal{L}_0$, then $s = s'$ and no struct instance is created during the execution of *split* by the struct instance of ℓ_s .
2. Iff $\ell_s \notin \mathcal{L}_0$, all of the following hold:
 - (a) $s'.done = true$.
 - (b) Iff $s.done = false$:
 - i. $s.p1 \neq \ell_{OldElem}^0 \wedge s.p2 \neq \ell_{OldElem}^0 \Leftrightarrow$
 $\exists \ell \notin \mathcal{L}_p. (\sigma'(\ell) = \langle NewElem, \varepsilon, \varepsilon, \xi_\ell \rangle \wedge$
 $\xi_\ell = \xi_{NewElem}^0 [\{min \mapsto s.spl + 1, spl \mapsto s.spl + (s.max - s.spl) / 2,$
 $max \mapsto s.max, next \mapsto s.next, done \mapsto true\}] \wedge$
 $s'.next = \ell) \wedge s'.max = s.spl \wedge s'.hasSplit = true,$
 - ii. $s.p1 \neq \ell_{OldElem}^0 \wedge s.p2 = \ell_{OldElem}^0 \Leftrightarrow s'.max = s.spl \wedge$
 $s'.hasSplit = false,$
 - iii. $s.p1 = \ell_{OldElem}^0 \wedge s.p2 \neq \ell_{OldElem}^0 \Leftrightarrow s'.min = s.spl + 1 \wedge$
 $s'.hasSplit = false,$
 - iv. $s'.spl = s'.min + (s'.max - s'.min) / 2 \wedge s'.p1 = \ell_{OldElem}^0 \wedge$
 $s'.p2 = \ell_{OldElem}^0.$

Proof. As, by Lemma 5.2.12, *split* does not have any race conditions, point 2 follows from multiple applications of Corollary 4.3.28. For the first point, note that if $\ell_s \in \mathcal{L}_0$, $s.p1 = \ell_{OldElem}^0$ and $s.p2 = \ell_{OldElem}^0$. Therefore, the struct instance of ℓ_s will not create a new struct instance as per line 27. That $s = s'$ again follows from multiple applications of Corollary 4.3.28. Therefore, point 1 holds. $\square$

Schedule execution results

We then consider the schedule. We first fix the state after the first execution of *migrate*:

Lemma 5.2.17 (Fixpoint premise). Let p_1 be the premise state and let p_2 be the state after the first execution of *migrate*. Then in p_2 , all of the following hold:

1. For every value v_i in $v_1, \dots, v_n$ there exists a struct instance s_i with label ℓ_i such that $s_i = \langle OldElem, \varepsilon, \varepsilon, \xi_i \rangle$ and $\sigma(\ell_i) = s_i$.
2. There exists a non-*null* struct instance t_1 with label ℓ_{t_1} such that $t_1 = \langle newElem, \varepsilon, \varepsilon, \xi_b \rangle$.

3. For all s_i , $s_i.val = v_i$ with $v_i \in v_1, \dots, v_n$ and $s_i.place = t_1$.
4. $t_1.min = \min(v_1, \dots, v_n)$, $t_1.max = \max(v_1, \dots, v_n)$,
 $t_1.spl = (t_1.max - t_1.min)/2 + t_1.min$, $t_1.next = \ell_{NewElem}^0$, $t_1.hasSplit = false$
and $t_1.done = true$.
5. Let G_1 be the set of s_i such that $s_i.val \leq t_1.spl$, with labels $\mathcal{L}_{G_1}$ and let G_2 be the other s_i , with labels $\mathcal{L}_{G_2}$. Then for all $s_i \in G_1$, $s_i.move = false$, for all $s_i \in G_2$, $s_i.move = true$, if $|G_1| > 0$, $\exists \ell_i \in \mathcal{L}_{G_1}.(t_1.p1 = \ell_i)$, and if $|G_2| > 0$, $\exists \ell_i \in \mathcal{L}_{G_2}.(t_1.p2 = \ell_i)$.

Proof. The first three points are directly derived from Definition 5.2.11. The last point follows from Lemma 5.2.13, as when $|G_1| > 0$, there is at least one old element for which point 2 of Lemma 5.2.13 applies, and if $|G_2| > 0$, there is at least one old element for which point 3 of Lemma 5.2.13 applies. Note that point 1 applies to none of the elements from the premise. $\square$

We then prove the following lemma specifying the effect of a single fixpoint execution:

Lemma 5.2.18 (Fixpoint execution contract). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state, let $\mathcal{L}_{OldElem} \subseteq \mathcal{L}$ be the labels of *OldElem* structs in p and let $\mathcal{L}_{NewElem} \subseteq \mathcal{L}$ be the labels of *NewElem* structs in p . Let $\mathcal{L}_p$ be all labels used in p . Executing an iteration of the fixpoint starting at state p results in a state $p' = \langle \Delta, \sigma', s\chi \rangle$. In p' , for every ℓ_s such that $s = \sigma(\ell_s) = \langle OldElem, \gamma, \chi, \xi_s \rangle$ and $s' = \sigma'(\ell_s)$ and ℓ_t such that $t = \sigma(\ell_t) = \langle NewElem, \gamma, \chi, \xi_t \rangle$, $t' = \sigma'(\ell_t)$ and $s.place = t$, all of the following hold:

1. If $\ell_s \in \mathcal{L}_0$, $s = s'$, and if $\ell_t \in \mathcal{L}_0$, $t = t'$ and no structure instance is created during the execution of split by the struct instance of ℓ_t .
2. Iff $\ell_s \notin \mathcal{L}_0$ and $\ell_t \notin \mathcal{L}_0$, all of the following hold:

- (a) $t'.done = true$
- (b) Iff $(t.p2 \neq \ell_{OldElem}^0) \vee \exists \ell_r \in \mathcal{L}_{OldElem}(\sigma(\ell_r).val \leq t.split \wedge t.p1 \neq \ell_t)$ all of the following hold:
 - i. $t.p1 \neq \ell_{OldElem}^0 \wedge t.p2 \neq \ell_{OldElem}^0 \Leftrightarrow$
 $\exists \ell \notin \mathcal{L}_p.(\sigma'(\ell) = \langle NewElem, \varepsilon, \varepsilon, \xi_\ell \rangle \wedge$
 $\xi_\ell = \xi_{NewElem}^0[\{min \mapsto t.spl + 1, max \mapsto t.max,$
 $spl \mapsto t.spl + 1 + (t.max - (t.spl + 1))/2,$
 $next \mapsto t.next, done \mapsto true\}]) \wedge$
 $t'.next = \ell) \wedge t'.max = t.spl \wedge t'.hasSplit = true,$
 - ii. $t.p1 \neq \ell_{OldElem}^0 \wedge t.p2 = \ell_{OldElem}^0 \Leftrightarrow t'.max = t.spl \wedge t'.hasSplit = false,$

- iii. $t.p1 = \ell_{OldElem}^0 \wedge t.p2 \neq \ell_{OldElem}^0 \Leftrightarrow t'.min = t.spl + 1 \wedge$
 $t'.hasSplit = false,$
- iv. $t'.spl = t'.min + (t'.max - t'.min) / 2 \wedge t'.p1 = \ell_{OldElem}^0 \wedge$
 $t'.p2 = \ell_{OldElem}^0,$
- v. $t'.done^*.$
- (c) $s.move \wedge t'.hasSplit \Leftrightarrow t'.place = t'.next,$
- (d) $(s.val > s'.place.spl) \Leftrightarrow s'.move = true,$
- (e) $(s.val \leq s'.place.spl) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r \wedge r.place = s'.place \wedge$
 $s'.place.p1 = \ell),$
- (f) $(s.val > s'.place.spl) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r \wedge r.place = s'.place \wedge$
 $s'.place.p2 = \ell),$
- (g) $s.place \neq \ell_{NewElem}^0 \Rightarrow s'.place \neq \ell_{NewElem}^0.$

Proof. This is a straightforward combination of the three contracts that make up the contracts for the fixpoint. If $\ell_s \in \mathcal{L}_0$, then all of its parameters are *null*, so none of the updates of ℓ_s will lead to changed parameters. If $\ell_t \in \mathcal{L}_0$, then point 2 holds by Lemma 5.2.16. Point 2a holds by Lemma 5.2.16. The condition of 2b is from Corollary 5.2.15; as *checkStable* determines whether *done* is *false* and the execution of *split* directly depends on that, the clauses *i-iv* are lifted from *split* and therefore hold due to Lemma 5.2.16. Clause *v* is a result of the interaction between *checkStable* and *split*; if the condition in 2b is true, then *checkStable* will change *t.done* to *false*, which is later overwritten in *split* with *true*. If the condition is *false*, then *t.done* will not be changed, but will stay constantly *true*. Clauses 2c, 2d, 2e, and 2f hold due to Lemma 5.2.13. Clause 2g holds due to a combination of Lemma 5.2.16 and Lemma 5.2.13: if $s.place \neq \ell_{NewElem}^0$, then according to Lemma 5.2.16, if $t.hasSplit = true$, then also $t.next \neq \ell_{NewElem}^0$. As the only update of $s.place$ happens in *migrate*, and only if $t.hasSplit = true$, it follows that $s'.place \neq \ell_{NewElem}^0$. $\square$

We then use this lemma to prove the following lemma for executions of the fixpoint from the fixpoint premise:

Lemma 5.2.19 (Fixpoint execution properties). Let $q_{i-1} = \langle \Delta, \sigma, s\chi \rangle$ be the state before the *i*th execution of the fixpoint from the state p_2 , as defined in Lemma 5.2.17, with $q_0 = p_2$. Let $\mathcal{L}_{OldElem} \subseteq \mathcal{L}$ be the labels of *OldElem* structs in q_{i-1} and let $\mathcal{L}_{NewElem} \subseteq \mathcal{L}$ be the labels of *NewElem* structs in q_{i-1} . Let $\mathcal{L}_p$ be all labels used in p . Let ℓ_s be a label of an *OldElem* struct instance, where $\sigma(\ell_s) = s$. Then the following all hold:

1. $s.val > s.place.spl \Leftrightarrow s.move = true.$
2. $(s.val \leq s.place.spl) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma(\ell) = r \wedge r.place = s.place \wedge$
 $s.place.p1 = \ell),$

3. $(s.val > s.place.spl) \Rightarrow \exists \ell \in \mathcal{L}_{OldElem}. (\sigma(\ell) = r \wedge r.place = s.place \wedge s.place.p2 = \ell).$
4. $s.place.spl = s.place.min + \lfloor (s.place.max - s.place.min) / 2 \rfloor.$
5. If $s.place.next = \ell_{NewElem}^0$, then $\nexists \ell_t \in \mathcal{L}_{NewElem}. (\sigma(\ell_t) = t \wedge s.place \neq \ell_t \wedge t.next = \ell_{NewElem}^0).$
6. If $s.place.next \neq \ell_{NewElem}^0$, then $s.place.max < s.place.next.min.$
7. $s.place.min \leq s.val \leq s.place.max.$

Additionally, let $q_i = \langle \Delta', \sigma', s\chi' \rangle$ be the state after the i th execution of the fixpoint from p_2 , with $\sigma'(\ell_s) = s'$. Then all of the the following properties, describing the relation between q_{i-1} and q_i , also hold:

C1 $s'.val = s.val$ and $s'.val$ is stable.

C2 $s.place \neq \ell_{NewElem}^0 \wedge s'.place \neq \ell_{NewElem}^0.$

R1 If $\exists \ell_r \in \mathcal{L}_{OldElem} \setminus \{\ell_s\}. (\sigma(\ell_r) = r \wedge r.place = s.place) \vee s.val > s.place.spl$, then $s'.place.max - s'.place.min \leq \lceil s.place.max - s.place.min / 2 \rceil.$

R2 If $\nexists \ell_r \in \mathcal{L}_{OldElem} \setminus \{\ell_s\}. (\sigma(\ell_r) = r \wedge r.place = s.place) \wedge s.val \leq s.place.spl$, with $s.place = \ell_t$, then the struct instances of ℓ_s and ℓ_t do not make new struct instances in execution i and all parameters of ℓ_s and ℓ_t are stable after execution i , $s'.place = \ell_t$ and $t'.p1 = \ell_s.$

Proof. We first assert **C1** and **C2**. The point **C1** directly follows from the fact that $s'.val$ is not assigned a value in the contract in Lemma 5.2.18. The point **C2** directly follows from property 2 of p_2 and a straightforward induction applying point 2g of Lemma 5.2.18 to every subsequent iteration of the fixpoint. We use this to prove points 1-7 hold for all possible q_i by induction on i :

$i = 0$. Then $q_i = p_2$. Point 1, 2 and 3 hold by point 5 of the properties of p_2 . Point 4 holds by point 4 of the properties of p_2 as given in Lemma 5.2.17. As there is only one $NewElem$ t_1 such that for all possible s , $s.place = t_1$ in p_2 (point 5), and $t_1.next = \ell_{NewElem}^0$ in p_2 (point 6), points 5 and 6 of the lemma hold. As $t_1.min$ is the minimum value of all old elements and $t_1.max$ is the maximum value of all old elements, point 7 of the lemma also holds.

$i > 0$. Then as induction hypothesis, we assume points 1-7 hold for q_{i-1} . Then we prove the points separately:

1. Assume that $s'.val > s'.place.spl$. Then as the val parameter never gets written to in Listing 3.7, $s'.val > s'.place.spl \Leftrightarrow s.val > s'.place.spl$. Then by point 2d of Lemma 5.2.18, it holds that $s'.val > s'.place.spl \Leftrightarrow s'.move = true.$

2. We assume that $(s'.val \leq s'.place.spl)$, and prove that
 $\exists \ell \in \mathcal{L}_{OldElem}. (\sigma'(\ell) = r \wedge r.place = s'.place \wedge s'.place.pl = \ell)$. From Lemma 5.2.18, due to the fact that $s'.val = s.val$, we can use point 2e directly to establish this point.
3. Proven along the same lines as the above point, only using point 2f from Lemma 5.2.18 instead.
4. To prove that $s'.place.spl = s'.min + [(s'.max - s'.min)/2]$, note that $s'.place$ is either equal to $s.place$ or not. We do a case distinction on these cases. If $s'.place = s.place$, according to point 2b-iv in Lemma 5.2.18 and the fact that AuDaLa only does integer calculations (and therefore the result is automatically rounded down), this point holds. If $s'.place \neq s.place$, let $s.place = \ell_t$ and let $t = \sigma(\ell_t)$ and $t' = \sigma'(\ell_t)$. Then as $s'.place \neq s.place$ and as the only possible value is then $s'.place = t'.next$ by point 2c of Lemma 5.2.18, we know from 2c that $s.move = true$ and $t'.hasSplit = true$. It then follows by point 2b-i of Lemma 5.2.18 that $t'.next.spl = t.spl + 1 + (t.max - (t.spl + 1))/2$, which is equal to $t'.next.min + (t'.next.max - t'.next.min)/2$. Then, as again AuDaLa only does integer calculations, the point holds in this case.
5. By the induction hypothesis, if $s.place.next = \ell_{NewElem}^0$, then $\nexists \ell_t \in \mathcal{L}_{NewElem}. (\sigma(\ell_t) = t \wedge s.place \neq \ell_t \wedge t.next = \ell_{NewElem}^0)$. Then as no *next* parameter is set to *null* in Listing 3.7, if there are two *NewElems* in q_i with an empty *next* parameter, then both must be related to $s.place$. Let $s.place = \ell_{t_s}$, with $\sigma'(\ell_{t_s}) = t'_s$. Then as the struct instance of ℓ_{t_s} can create only one new struct instance in execution i , the *NewElem* struct instances with an empty *next* parameter must be ℓ_{t_s} and a struct instance with a label ℓ made by ℓ_{t_s} . But then by Lemma 5.2.18, this can only be $t'_s.next$, which means that ℓ_{t_s} does have a non-*null* value for *next*. Therefore, this point holds.
6. By the induction hypothesis, if $s.place.next \neq \ell_{NewElem}^0$, then $s.place.max < s.place.next.min$. Let $\ell_t = s.place.next$, with $\sigma(\ell_t) = t$ and $\sigma'(\ell_t) = t'$. Then $t.min \leq t'.min$ and $s'.max \leq s.max$, as seen in cases 2b-(i-iv) of Lemma 5.2.18. It follows that $s'.max < s'.min$ so this point holds.
7. By the induction hypothesis, $s.place.min \leq s.val \leq s.place.max$. Let $s.place = \ell_t$. Let $t = \sigma(\ell_t)$ and $t' = \sigma'(\ell_t)$. Then either $s'.place = \ell_t$ or not. We use a case distinction: If $s'.place = \ell_t$, then if the condition of 2b of Lemma 5.2.18 does not hold for t , $t' = t$, and as $s'.val = s.val$, this point holds. We therefore assume that the condition of 2b does hold for t .

Then either $s.move = true$ or not. In the case where $s.move = true$,

we know that $s'.val = s.val \geq t.spl + 1 \geq t'.min$. To prove that $s'.val \leq t'.max$, note that by the induction hypothesis, it follows from $s.val \geq t.spl + 1$ that $t.p2 \neq \ell_{OldElem}^0$ (point 3 of this lemma). As we know that $s.move = true$ but that $s'.place = \ell_t$, it follows that $t'.hasSplit = false$. From this it follows that $2b-iii$ holds and $2b-i$ and $2b-ii$ do not hold, from which we conclude that $t'.max = t.max$. It follows that $s'.place.min \leq s'.val \leq s'.place.max$. If $s.move = false$, it follows that $s'.val = s.val \leq t.spl \leq t'.max$. We then also know that $t.p1 \neq \ell_{OldElem}^0$ (point 2 in this lemma), from which it follows that either $2b-i$ or $2b-iii$ of Lemma 5.2.18 can hold for ℓ_s and ℓ_t , but not $2b-ii$. In both cases, $t'.min = t.min$, from which it follows that $s'.place.min \leq s'.val \leq s'.place.max$.

It rests to prove the point if $s'.place \neq \ell_t$. In this case, we know that $s'.place = t'.next$ (the only value other than ℓ_t it can have according to Lemma 5.2.18), from which it follows that $s.move = true$ and $t'.hasSplit = true$. From $s.move = true$, we get that $s.val \geq t.spl + 1$ (point 1 of this lemma). Then we know that $t.p2 \neq \ell_{OldElem}^0$ (point 3 of this lemma), from which it follows that the condition for $2b$ holds for ℓ_t . As $t'.hasSplit = true$, it follows that $2b-i$ holds for t , from which it follows that $s'.place.min = t.spl + 1$ and $s'.place.max = t.max$. As $s'.val = s.val$, it follows that $s'.place.min \leq s'.val \leq s'.place.max$. As then in all cases $s'.place.min \leq s'.val \leq s'.place.max$, this point holds.

As all points hold, the step case holds.

As the induction holds, points 1-7 hold. We use these points to prove points **R1** and **R2**, for q_{i-1} and q_i :

- R1.** If $\exists \ell_r \in \mathcal{L}_{OldElem}. (\sigma(\ell_r) = r \wedge r.place = s.place) \vee s.val > s.place.spl$, let $s.place = \ell_t$, with $\sigma(\ell_t) = t$ and $\sigma'(\ell_t) = t'$. Then if $s.val > s.place.spl$, we know from point 3 of this lemma that $t.p2 \neq \ell_{OldElem}^0$ and from point 1 of this lemma that $s.move = true$. Then either $2b-i$ or $2b-iii$ of Lemma 5.2.18 holds for s , s' , t and t' . Regardless of the exact case, it follows that $s'.place.min = t.spl + 1$ and $s'.place.max = t.max$. From point 4 of this lemma, we know that $t.spl = t.min + \lfloor (t.max + t.min)/2 \rfloor$, so

$$\begin{aligned}
 & s'.place.max - s'.place.min \\
 &= t.max - (t.min + \lfloor (t.max - t.min)/2 \rfloor + 1) \\
 &\leq (t.max - t.min) - (\lfloor (t.max - t.min)/2 \rfloor) \\
 &= \lceil (t.max - t.min)/2 \rceil \\
 &= \lceil (s.place.max - s.place.min)/2 \rceil.
 \end{aligned}$$

Then if $s.val \leq s.place.spl$, it holds that $\exists \ell_r \in \mathcal{L}_{OldElem}. (\sigma(\ell_r) = r \wedge r.place = s.place)$, so we know that there are multiple *OldElems* which

qualify for $t.p1$. Then the condition of $2b$ holds, and either $2b-i$ holds or $2b-ii$ holds. In both cases it follows that $s'.place.max = t.spl$ and $s'.place.min = t.min$. From point 4 of this lemma, we know that $t.spl = t.min + \lfloor (t.max + t.min)/2 \rfloor$, so

$$\begin{aligned}
 & s'.place.max - s'.place.min \\
 &= t.spl - t.min \\
 &= t.min - t.min + \lfloor (t.max + t.min)/2 \rfloor \\
 &\leq \lfloor (t.max + t.min)/2 \rfloor \\
 &= \lfloor s.place.max - s.place.min/2 \rfloor.
 \end{aligned}$$

It follows that **R1** holds.

- R2.** Assume $\nexists \ell_r \in \mathcal{L}_{OldElem} \setminus \{\ell_s\}.(\sigma(\ell_r) = r \wedge r.place = s.place) \wedge s.val \leq s.place.spl$, and let $s.place = \ell_t$. The condition holds straightforwardly when $\ell_s \in \mathcal{L}_0$ or $\ell_t \in \mathcal{L}_0$, by point 1 of Lemma 5.2.18 and the fact that no struct instances are ever created by s during the execution of the fixpoint as per the same lemma. Then if both ℓ_s and ℓ_t are not null-labels, as s does not create struct instances and the condition for $2b$ does not hold for t due to the assumption, no struct instances are created.

To prove that the parameters are stable, we also consider Lemma 5.2.18. As the condition for $2b$ does not hold, we know that only $t.p1$ and $t.p2$ can have changed, through $2e$ and $2f$. The value for $t'.done$ cannot have changed, as only when the condition of $2b$ holds $t'.done$ can be unstable.

Due to our assumption, we know that $2f$ cannot apply and that $t'.p1$ can only be ℓ_t . Additionally, from point 2 of this lemma and our assumption, we also know that $t.p1 = \ell_s$. Therefore, all parameters of t are stable and $t'.p1 = t.p1 = \ell_s$.

For the parameters of ℓ_s , note that $s.place = \ell_t$. Due to point 1 of this lemma, we know due to the assumption that $s.move = false$, from which it follows by $2c$ of Lemma 5.2.18 that $s'.place = s.place = \ell_t$. Additionally, from the assumption and $2d$ of Lemma 5.2.18 it follows that $s'.move = false$ as well. From point **C** of this lemma, $s.val$ has not changed in the iteration and is stable. Therefore, all of the parameters of ℓ_s are stable in s' and $s'.place = \ell_t$.

Then if $\nexists \ell_r \in \mathcal{L}_{OldElem} \setminus \{\ell_s\}.(\sigma(\ell_r) = r \wedge r.place = s.place) \wedge s.val \leq s.place.spl$, all parameters of ℓ_s and ℓ_t are stable after the execution of the fixpoint iteration, $s'.place = \ell_t$ and $t.p1 = \ell_s$.

With all points proven, the lemma holds. $\square$

We use this lemma to prove the following lemma:

Lemma 5.2.20 (Fixpoint termination and result). The execution of the fixpoint starting at P_2 as specified in Lemma 5.2.17 terminates. Let $p' = \langle \Delta, \sigma', s\chi \rangle$ be the state after the execution of the fixpoint. Then the following all hold:

1. Every *OldElem* element $s \in \sigma'$ with label ℓ_s is uniquely paired with an *NewElem* element $t \in \sigma'$ with label ℓ_t , such that $s.place = \ell_t$ and $s.place.min \leq s.val \leq s.place.max$ and $t.p1 = \ell_s$.
2. For every *NewElem* element $t \in \sigma'$ such that $t.next \neq \ell_{NewElem}^0$, it holds that $t.max < t.next.min$.
3. There is exactly one *NewElem* element $t \in \sigma'$ such that $t.next = \ell_{NewElem}^0$.

Proof. First of all, in any state p with struct environment σ during or after the execution of the fixpoint, there is always at least one *OldElem* $s \in \sigma$ such that $s.place = \ell_t$ for every *NewElem* $t \in \sigma$ with label ℓ_t . This is due to new elements only being made if there are *OldElems* in both $t.p1$ and $t.p2$ as representatives of both halves of the range of t and the range of t being dynamically updated otherwise (Lemma 5.2.18).

Due to **R1** of Lemma 5.2.19 We know that if there are multiple elements with ℓ_t as value of their *place* parameter before an iteration of the fixpoint, the range of ℓ_t will halve during the iteration of the fixpoint. As we deal with distinct numbers, after enough halvings of the range, there will always eventually be exactly one *OldElem* that has ℓ_t as its *place*, and due to the dynamic range halving, this element is eventually referenced by the *p1* parameter of ℓ_t . Due to **R2** of Lemma 5.2.19 it follows that that element and ℓ_t are then stable. As this holds for any ℓ_t , all *OldElems* and *NewElems* will eventually be stable and the fixpoint will terminate.

From this and from point 7 of Lemma 5.2.19, it also follows that point 1 of this lemma holds. Point 2 follows from point 6 of Lemma 5.2.19, and point 3 holds from point 5 of Lemma 5.2.19. Therefore, the entire lemma holds. $\square$

Conclusion

From the step and schedule execution results, we can conclude the following:

Theorem 5.2.21 (Correctness of Listing 3.7). When executing the AuDaLa code as given in Listing 3.7 from a premise state according to Definition 5.2.11 for a set of elements $s_1, \dots, s_n$ created around a list of distinct positive values $L_1 = v_1, \dots, v_n$, it results in a state with a second sequence of elements $t_1, \dots, t_n$ associated with the list of values $L_2 = v'_1, \dots, v'_n$ such that $v'_1 < \dots < v'_n$ and for all $v_i \in L_1$ it holds that $v_i \in L_2$.

Proof. From Lemma 5.2.20, we know that after execution there exists a sequence $t_1; \dots; t_n$ such that for every s_i there is a paired t_j , for which $t_1.pl.val < \dots t_n.pl.val$, which means that this sequence is sorted on values. This satisfies Problem 5.2.10. $\square$

5.3 Proving the AuDaLa Turing Machine Implementation Correct

In this section, we show that AuDaLa is Turing complete, by proving our implementation of an arbitrary Turing machine in Section 3.5.2 correct. For this, we first introduce *stutter trace equivalence* and adapt it to our context and we define the notion of *implementation configurations*. We then prove our implementation correct by inductively proving it equivalent according to our adaptation of stutter trace equivalence. For this, we first give an outline of the proof. After this, we prove that the initialization is correct and that the transition execution is correct for our induction. With this proof, we conclude that AuDaLa is Turing complete.

Stutter trace equivalence and the implementation configuration

To prove our implementation correct, we will use a form of *stutter trace equivalence*, adapted from Baier and Katoen [12, Definition 7.89]. This equivalence uses the notion of a *path*, which is an execution sequence consisting of the states traversed during execution, and a *trace*, which is a sequence of *atomic propositions* encountered during a path. *Atomic propositions* are an encoding of the information contained in the states, often expressed as a true statement over the contents of the state. This can range from just the entire state to an abstraction of the information in the state, for example to a letter or a number.

We extend the notion of paths to AuDaLa and Turing machines via the following definition, which adapts the definition of paths from Baier and Katoen [12] to operational semantics.

Definition 5.3.22 (Paths and Prefixes). Let $G = (Q, \rightarrow, I)$ be an operational semantics with state space Q , transition relation $\rightarrow$ and an initial state I . A *path* is a sequence $\rho = q_1, q_2, \dots$ such that $q_1 \rightarrow q_2 \rightarrow \dots$ is valid in G . If $q_1 = I$, ρ is a *prefix*.

The equivalence can then be explained using the following definition, adapted from Baier and Katoen [12, Definition 7.86]:

Definition 5.3.23 (Stutter equivalence for paths). Let L be a set of state labels, let $f : Q \rightarrow L$ be a labelling function from a set of states Q to L , and let, with a label $l \in L$, the sequence l^d for some $d > 0$ be a sequence of d instances of l .

Two paths $\rho_1 = A_1, \dots$ and $\rho_2 = B_1, \dots$ over a set of states Q such that $A_1, \dots, B_1, \dots \in Q$ are *stutter-equivalent* with regards to f , or *f-stutter-equivalent* if applying f on the states of ρ_1 results in a sequence $L_1^{a_1} L_2^{a_2} \dots$ with $a_1, a_2, \dots > 0$ and applying f on the states of ρ_2 results in a sequence $L_1^{b_1} L_2^{b_2} \dots$ with $b_1, b_2, \dots > 0$, for some $L_1, L_2, \dots \in L$.

As an example, if we have two paths ρ_1 and ρ_2 and a labelling function f such that the results from applying f on ρ_1 and ρ_2 are $xxxxxyyzzzzzy$ and $xxxxxyyzzzzzyy$, then ρ_1 and ρ_2 are *f-stutter-equivalent*, as these traces can be expressed as $x^{a_1} y^{a_2} z^{a_3} y^{a_4}$ and $x^{b_1} y^{b_2} z^{b_3} y^{b_4}$ respectively, with $a_1, a_2, \dots > 0$ and $b_1, b_2, \dots > 0$. Note that here, $xxxxxyyzzzzzy$ and $xxxxxyyzzzzzyy$ are our *traces*, using a set of atomic propositions $\{x, y, z\}$.

Classic stutter trace equivalence then considers two systems ST_1 and ST_2 equivalent iff every path in ST_1 has a stutter-equivalent path in ST_2 and vice versa. In our case, we want to compare our Turing machine implementation execution to the execution of the original Turing machine. To do so, we define a function to label the states of the implementation to correspond to the states of the Turing machine, which we use to establish equivalence between the two types of states.

Henceforth, let $\mathcal{P}_{(M, \mathcal{I})}$ be the implementation of a Turing machine M with an input string $\mathcal{I} = \varsigma_0 \dots \varsigma_n$ as specified in Section 3.5. As $\mathcal{P}_{(M, \mathcal{I})}$ is well-typed and satisfies the requirements given in Section 2.3, $\mathcal{P}_{(M, \mathcal{I})}$ is well-formed.

We first define the notion of *idle* states:

Definition 5.3.24 (Idle States). We define any AuDaLa state $p = \langle \Delta_p, \sigma_p, s\chi_p \rangle$ to be *idle* iff $Done(\sigma_p) = true$. We define *busy* states as states that are not idle.

As $\mathcal{P}_{(M, \mathcal{I})}$ is well-formed, it does not get stuck, according to Corollary 4.3.25. It follows that the execution of $\mathcal{P}_{(M, \mathcal{I})}$ consists of the execution of a step *init* from the initial state, and then executions of the step *transition*. For the idle states in between these steps, we define *implementation configurations*:

Definition 5.3.25 (Implementation Configuration). Let p be an idle state of $\mathcal{P}_{(M, \mathcal{I})}$ containing a single non-*null* instance c of *Control*. Then we define the *implementation configuration* of p as a tuple (q_p, t_p) such that q_p is the value of the *state* parameter of c and $t_p : \mathbb{Z} \rightarrow \mathbb{Z}$ defined as:

$$t_p(i) = \begin{cases} c.head.symbol & \text{if } i = 0 \\ c.head.left^i.symbol & \text{if } i < 0, \\ c.head.right^i.symbol & \text{if } i > 0 \end{cases}$$

If p does not contain a single non-*null* instance of *Control*, then the implementation configuration is undefined.

Furthermore, let p_1 be a busy state encountered, and let p' be the last idle state encountered before p_1 which has a defined implementation configuration. Then the implementation configuration of p_1 is defined to be equal to the implementation configuration of p' .

Note that this implementation configuration has the same format as a Turing machine configuration. We therefore unify Turing machine configurations with implementation configurations into a single notion of *configurations*.

We would like to compare the configurations of a Turing machine execution to the configurations of a Turing machine implementation execution. We therefore define the function *conf*:

Definition 5.3.26 (The function *conf*). Let Q_1 be the set of all possible configurations. Let $\mathbb{I}$ be the set of all possible Turing machine implementations following the template of Section 3.5.2, and let Q_2 be the set of all states of any implementation in $\mathbb{I}$ with a defined implementation configuration. Then we define the function $\text{conf}: Q_1 \cup Q_2 \rightarrow Q_1$ such that:

$$\text{conf}(a) = \begin{cases} a & \text{if } a \in Q_1 \\ b & \text{if } a \in Q_2 \text{ and } b \text{ is the implementation configuration of } a \end{cases}$$

This then leads to the following definition of *stutter trace equivalence*, adapted from Baier and Katoen [12, Definition 7.89] for our purpose of proving the Turing machine implementation correct:

Definition 5.3.27 (Configuration Stutter Prefix Equivalence). Let M be a Turing machine with input string $\mathcal{I}$ and let $\mathcal{P}_{(M,\mathcal{I})}$ be the AuDaLa implementation of M . Let p_1 be the state after the execution of the initialization in $\mathcal{P}_{(M,\mathcal{I})}$, s.t. the *Done*-predicate holds and the schedule is equal to $\text{Fix}(\text{transition})$. Then M with $\mathcal{I}$ and $\mathcal{P}_{(M,\mathcal{I})}$ are *configuration stutter prefix equivalent* iff the following all holds:

1. For all possible prefixes ϱ_1 that can be encountered while executing M , there is a path ϱ_2 starting from p_1 that can be encountered while executing $\mathcal{P}_{(M,\mathcal{I})}$ after initialization such that ϱ_1 and ϱ_2 are *conf-stutter equivalent*.
2. For all possible paths ϱ_2 starting from p_1 that can be encountered while executing $\mathcal{P}_{(M,\mathcal{I})}$, there is a prefix ϱ_1 that can be encountered while executing M after initialization such that ϱ_1 and ϱ_2 are *conf-stutter equivalent*.

Note that we only consider the implementation states after initialization as the Turing machine does not need to be initialized.

Proof outline

If an Turing machine M with $\mathcal{I}$ and its implementation $\mathcal{P}_{(M,\mathcal{I})}$ are configuration stutter prefix equivalent, it follows that any transition of M with input string $\mathcal{I}$ can be mirrored by $\mathcal{P}_{(M,\mathcal{I})}$ and vice versa, and the resulting states have the same configuration. This we consider enough to prove that $\mathcal{P}_{(M,\mathcal{I})}$ correctly implements M with $\mathcal{I}$.

The proof is a proof by induction on the transitions taken. We prove that after i transitions, any path in $\mathcal{P}_{(M,\mathcal{I})}$ that starts in the state p_1 after initialization and executes i transitions has a *conf*-stutter-equivalent prefix which executes i transitions in M with $\mathcal{I}$ and vice versa.

As the implementation configuration is defined on the struct environment of idle states only, we abstract to collections of idle states between *transition* step executions. This does not matter for our proof as all idle states in the same collection will all have the same configuration: there is no semantic transition that changes the struct environment enabled when the *Done*-predicate holds. Furthermore, we can abstract away from proving busy states equivalent to Turing machine states, as that follows from idle states being equivalent to Turing machine states.

Our induction is over the amount of transitions executed by M with $\mathcal{I}$ and $\mathcal{P}_{(M,\mathcal{I})}$. For our induction proof, we first need to prove as a base case that after the initialization we find ourselves in idle state p_1 with the initial configuration of M with $\mathcal{I}$ as its implementation configuration.

We then have to prove as a step case that every transition taken by either M with $\mathcal{I}$ or $\mathcal{P}_{(M,\mathcal{I})}$ can be exactly mirrored by $\mathcal{P}_{(M,\mathcal{I})}$ or M with $\mathcal{I}$. As induction hypothesis we assume that the Turing configuration before the transition is equal to the implementation configuration of the group of idle states of $\mathcal{P}_{(M,\mathcal{I})}$ before the transition. As a conclusion, it then follows that for any path from p_1 in $\mathcal{P}_{(M,\mathcal{I})}$, there is a *conf*-stutter-equivalent prefix of M and vice versa.

During all steps of the proof, we have to also maintain that there is at most a single non-null *Control* instance in the idle states reached in $\mathcal{P}_{(M,\mathcal{I})}$. This guarantees that there are no race conditions in $\mathcal{P}_{(M,\mathcal{I})}$.

Proving the initialization correct

We prove that after the initialization we find ourselves in idle state p_1 with the initial configuration of M with $\mathcal{I}$ as its implementation configuration. For this, we first prove that the step *init* is deterministic. We then use this to prove the exact result of executing *init*, by using Corollary 4.3.28.

Lemma 5.3.28. The execution of *init* in $\mathcal{P}_{(M,\mathcal{I})}$ is deterministic.

Proof. The step *init* is only executed once, at the start of the program, by the null-instance of *Control* (as no other instances exist). As only one instance ex-

ists, there cannot be a race condition between two struct instances. Therefore *init* contains no race conditions. It then follows by Lemma 4.1.6 that *init* is deterministic. $\square$

Lemma 5.3.29 (Executing *init* in the initial state). Let $p_{\mathcal{P}_{(M,\mathcal{I})}}^0$ be the initial state at the start of executing $\mathcal{P}_{(M,\mathcal{I})}$ and let the input string $\mathcal{I} = \varsigma_0 \dots \varsigma_n$. Executing the step *init* on $p_{\mathcal{P}_{(M,\mathcal{I})}}^0$ results in a state p_1 with a single non-null *Control* instance such that (q_0, t_S) is the implementation configuration of p_1 .

Proof. First, note that the initial state for $\mathcal{P}_{(M,\mathcal{I})}$, as defined in the AuDaLa semantics, contains the schedule of $\mathcal{P}_{(M,\mathcal{I})}$, the null-instances of all structs, and a stability stack. The stability stack has no bearing on this proof, and will be disregarded. From Lemma 5.3.28, we know that as only the null-instance of *Control* executes *init*, the execution of *init* is deterministic.

Then let p_1 be an idle state reached by executing *init* from $p_{\mathcal{P}_{(M,\mathcal{I})}}^0$. As written in line 43 of $\mathcal{P}_{(M,\mathcal{I})}$, the null-instance of *Control* will create one additional *Control* instance. As no other struct instances execute *init*, it follows that there will be a single non-null *Control* instance in p_1 . Let this instance be c . From line 43 of Listing 3.12, we also know that c will have its state set to 0, which is the representation of q_0 .

We then prove that the function made according to Definition 5.3.25 in p_1 from the *TapeCells* is t_S . Due to Lemma 5.3.28, we can walk through Listing 3.11 sequentially. Then, through multiple applications of Corollary 4.3.28, we know that the first part makes one *TapeCell* instance for every $\varsigma_i \in \mathcal{I}$, and the second part then connects these to the correct *left* and *right* neighbours. It follows that in p_1 , there exists a *TapeCell* for all symbols $\varsigma_i \in \mathcal{I}$, and no others, such that every *TapeCell* ς_i is connected to ς_{i-1} and ς_{i+1} (if they exist) through parameters *left* and *right* respectively. The last line of *init* then sets the *head* parameter of c to the *TapeCell* for ς_0 . It follows that the function made according to Definition 5.3.25 is indeed t_S .

Then the lemma holds: the implementation configuration of p_1 is (q_0, t_S) . $\square$

As the configuration of p_1 is (q_0, t_S) , the initialization configuration of the Turing machine, the initialization of $\mathcal{P}_{(M,\mathcal{I})}$ is correct. We then move on to proving the transitions correct.

Proving transition executions correct

We then prove the transition execution of our implementation correct. We do this by first establishing that the step *transition* taken from a state with a single non-null *Control* instance is deterministic, after which we prove that all executions of *transition* taken in $\mathcal{P}_{(M,\mathcal{I})}$ are deterministic (as there will always

be only a single *Control* instance). We then prove that executing some transition from a configuration (q, t) in $\mathcal{P}_{(M, \mathcal{I})}$ has the same resulting configuration as executing the same transition from (q, t) in M with $\mathcal{I}$. As a part of this, we also prove that no new *Control* instances are made, as that would mean that the idle states reached would not have an implementation configuration.

Lemma 5.3.30. Let p be an idle state reachable in $\mathcal{P}_{(M, \mathcal{I})}$ with a single non-null *Control* instance. Any execution of *transition* executed from p is deterministic.

Proof. As per Lemma 4.1.6, it suffices to prove that the execution contains no race conditions. Due to the definition of *transition* (Listing 3.9), only one clause of the step will be executed. Let c be this clause. As there is only one non-null *Control* instance, let c_0 be the null-instance of *Control* and let c_1 be the instance of *Control* that is not a null-instance.

During the execution of *transition* by the null-instance of *Control* c_0 , all of the updates executed by c_0 will be updates to parameters of null-instances (recall that $c_0.\text{head} = \text{null}$). As there is only one non-null *Control* instance c_1 , any race condition must involve both c_0 and c_1 , and must then be a race for a null-parameter. However, race conditions require at least one write to be performed to the variable and writes to null-parameters are not performed (as per the semantic rule **ComWrNSkip**). It follows that the execution contains no race conditions. $\square$

Lemma 5.3.31. Every *transition* step executed in $\mathcal{P}_{(M, \mathcal{I})}$ is deterministic.

Proof. We prove by induction that every execution of *transition* in $\mathcal{P}_{(M, \mathcal{I})}$ starts from a state with a single non-null *Control* instance. As a base case, the first execution of *transition* starts from p_1 as defined in Lemma 5.3.29, which has a single non-null *Control* instance.

Then, let p_i be a state with a single non-null *Control* instance from which *transition* is executed for the i th time. Due to Lemma 5.3.30, we know that the execution of *transition* is deterministic, so we can consider the sequential execution of *transition*. The step *transition* is made up of multiple mutually exclusive clauses by definition. Let Cl be the clause executed, w.l.o.g., for this execution of *transition*. By the definition of Cl , in none of the statements in Cl a *Control* instance is created, as seen in Listing 3.9. It follows that the state p'_i resulting from the execution of *transition* will also have only a single non-null *Control* instance. By Corollary 4.3.25 and Property 3 (Lemma 4.2.16), we know that eventually, from p'_i , a state p_{i+1} is reached such that either the schedule is empty or *transition* is executed again. If the schedule is empty in p_{i+1} , every execution of the step *transition* in $\mathcal{P}_{(M, \mathcal{I})}$ will have been executed from a state with a single non-null *Control* instance. Else, note that the Schedule Transitions as defined in the semantics of $\mathcal{P}_{(M, \mathcal{I})}$ (Section 2.4) do

not change the struct environment, so p_{i+1} will again have a single non-null *Control* instance.

It follows that every step *transition* executed in $\mathcal{P}_{(M, \mathcal{I})}$ will be executed from a state with a single non-null *Control* instance. By Lemma 5.3.30, the lemma follows. $\square$

Lemma 5.3.32 (Result of a *transition* execution). Let p be an idle state of $\mathcal{P}_{(M, \mathcal{I})}$ from which *transition* can be executed and let (q, t) be the implementation configuration of p . Let (q, t) also be the current Turing machine configuration of M with input string $\mathcal{I}$. Then the result of a transition in M is a configuration (q', t') iff the result of executing the *transition* step from p in $\mathcal{P}_{(M, \mathcal{I})}$ is an idle state p' such that (q', t') is its implementation configuration.

Proof. Let P and (q, t) be as defined in the lemma. Then by definition of $\mathcal{P}_{(M, \mathcal{I})}$, there exists a single transition that can be executed from p for $\mathcal{P}_{(M, \mathcal{I})}$ iff $\delta(q, t)$ is defined. Additionally, δ is a partial function, so if $\delta(q, t)$ is defined for (q, t) it is always uniquely defined for (q, t) .

Let $\delta(q, t) = (q', s', D)$. Let $D = R$ (the proof of $D = L$ is analogous). Then as a transition in M with $\mathcal{I}$ is deterministic, by Definition 3.5.5, the resulting Turing machine configuration of taking a transition from (q, t) is (q', t') , with

$$t'(i) = \begin{cases} s' & \text{if } i = -1 \\ t(i+1) & \text{otherwise} \end{cases}.$$

Taking the transition from p for $\mathcal{P}_{(M, \mathcal{I})}$ is also deterministic (Lemma 5.3.31), and therefore we can walk through the statements of the clause to determine its result. By definition of $\mathcal{P}_{(M, \mathcal{I})}$, this clause is based on the template shown in Listing 3.9. Let c be the single *Control* instance that exists in p , and let h be the *TapeCell* instance which is referenced in the *head* parameter of c . As per multiple applications of Corollary 4.3.28, the result of the transition is that the *state* of c is updated to q' , the *symbol* of h is updated to s' , the *accepting* parameter of c is updated to whether $q' \in A$ and that the *head* parameter of c shifts one *TapeCell* to the right (making a new *TapeCell* if required).

Creating a function of the *TapeCells* as in Definition 5.3.25 then results in function t'' , such that $t''(-1) = s'$ and $t''(i) = t(i+1)$ for all $i \neq -1$. It follows that $t'' = t'$. Then, by Definition 5.3.25, the implementation configuration of the resulting state is (q', t') . $\square$

Conclusion

With the lemmas for the initialization and the transitions proven, we combine them in the following theorem:

Theorem 5.3.33 (Configuration stutter prefix equivalence of Turing machine and implementation). A Turing machine M with $\mathcal{I}$ and its AuDaLa implementation $\mathcal{P}_{(M,\mathcal{I})}$ are configuration stutter prefix equivalent.

Proof. We prove this by induction: after i transitions, any path in $\mathcal{P}_{(M,\mathcal{I})}$ that starts in the state p_1 after initialization and executes i transitions has a *conf-stutter*-equivalent prefix which executes i transitions in M and vice versa. As the base case, after initialization, both the $\mathcal{P}_{(M,\mathcal{I})}$ and the Turing machine M with input string $\mathcal{I}$ have configuration (q_0, t_S) . For M with $\mathcal{I}$, this follows by definition, and for $\mathcal{P}_{(M,\mathcal{I})}$, this follows from Lemma 5.3.29. Then, as the step case, we prove that when M with $\mathcal{I}$ and $\mathcal{P}_{(M,\mathcal{I})}$ both start from configuration (q, t) and take a transition, both end up in configuration (q', t') . This follows from Lemma 5.3.32. We conclude that M with $\mathcal{I}$ and $\mathcal{P}_{(M,\mathcal{I})}$ are configuration stutter prefix equivalent. $\square$

As discussed before, we consider this sufficient to prove that the AuDaLa implementation of Turing machines as given in Section 3.5 is correct.

From Theorem 5.3.33, we also conclude the following:

Theorem 5.3.34 (Turing completeness). AuDaLa is Turing complete.

Proof. As for every possible Turing machine M with input string $\mathcal{I}$, our implementation in AuDaLa is configuration stutter prefix equivalent, we can simulate all Turing machines in AuDaLa, from which we conclude that AuDaLa is Turing complete. $\square$

It follows that AuDaLa can compute all effectively computable functions following the Church-Turing thesis [45].

5.4 Conclusion

In this section, we have proven the correctness of the AuDaLa programs for Prefix Sum (Theorem 5.1.9), Linked List Copy Sort (Theorem 5.2.21), and our AuDaLa implementations of Turing machines (Theorem 5.3.33), using the properties as proven in Chapter 4. This has also led to the proof that AuDaLa is Turing complete (Theorem 5.3.34).

In Chapter 6, we will give some adaptations for the AuDaLa semantics, with a specific focus on an alternative semantics which can be used for a decently performant implementation of AuDaLa on the hardware. We prove this semantics equivalent to the original semantics under some conditions, for which we again adapt stutter trace equivalence, like we did in Section 5.3.

Chapter 6

Adapting AuDaLa Semantics

In the previous chapters, we have given the foundations of AuDaLa and demonstrated their use. However, like most programming languages, its design has some limitations. In this chapter, we explore adapting the semantics of AuDaLa to adapt it to overcome some limitations inherent to the design.

We consider two directions for our adaptations. Firstly, we would like AuDaLa to be *programmable*. As discussed in Chapter 1, we interpret this as that we want AuDaLa to be simple, structured and expressive. From related work, we have defined two layers of expressivity, where programming language A can be considered more expressive than programming language B if one of the following holds:

1. Language A allows for the creation of programs that cannot in any way be expressed in language B .
2. Languages A and B are operationally equivalent and language A makes it easier to express programs than language B , for example by having more flexible or powerful language features.

We consider AuDaLa simple and structured through its design, which is demonstrated by the examples of Chapter 3. Furthermore, we have given an initial expressivity class for AuDaLa by proving it Turing complete in Section 3.5 and Section 5.3. However, we have not yet explored how we can extend the semantics of AuDaLa to allow for some more flexible and powerful language features.

In Section 6.1, we give three adaptations of AuDaLa to increase its flexibility. Two of the adaptations are for making the fixpoint more flexible, and one is for making AuDaLa support some kind of array, as those are commonly used in other programming languages. As flexibility is also subjective, these can be considered example adaptations, as we encourage the user to adapt AuDaLa

further depending on their requirements. At the end of the section, we give a small discussion on the adaptations, their merits and their drawbacks.

Secondly, as also discussed in Chapter 1, we would like AuDaLa to be usable on conventional hardware. However, in related work [109] it became clear that the standard semantics of AuDaLa is too strict in the order of reads and writes for decently fast execution of programs on conventional hardware.

AuDaLa can be compiled using a *shared-memory* memory layout, which assumes that all memory is accessible by all processors through a shared memory space. In the quest for performance, *memory consistency models* [73, 135] were introduced for shared-memory languages, which specify how memory accesses can appear to be executed to the programmer when executing the program. These are then enforced for the execution of programs on the hardware.

Strong memory consistency models do not allow for many hardware optimisations to improve the execution performance, but are simpler to program in and to formalize due to stronger guarantees on program behaviour. *Relaxed* (or *weak*) memory models, in comparison, allow for reordering of memory accesses, which allows for optimisations in the execution of programs. This leads to better performance, but added difficulty in the programs and formalizations to handle the additional possible behaviours.

Furthermore, the weak memory models can be *incoherent*; the parallel processors do not need to agree on what values are stored in the memory at all times. As an example, the processor may have local caches in which it stores recently read values, and these values do not necessarily have to be up to date with the value in the global memory. This allows for less synchronisation between the parallel processors and further improves performance.

The standard semantics of AuDaLa assumes *sequential consistency*, a strong and intuitive memory consistency model which requires that the execution of a parallel program appears as an interleaving of the sequential executions of the parallel parts. To improve performance, we can adapt the standard semantics to work with a relaxed memory consistency model that supports incoherency.

Therefore, in Section 6.2, we adapt the standard semantics to a *weak-memory semantics*, which uses a relaxed and incoherent memory consistency model. In this new semantics, we allow for accesses to be reordered, provided that this does not conflict with access dependencies as defined by the syntax. In related work [109], it is demonstrated that compiling this weak-memory semantics to CUDA leads to better performance than when sequential consistency is assumed, which is progress to making AuDaLa decently performant.

However, though the weak-memory semantics supports more execution optimizations, it is harder to verify due to its increased complexity. As one of our other focus points for AuDaLa is verifiability, this poses a problem. As a solution, we prove in Section 6.3 that as long as there are no read-write race conditions, any program execution using the standard semantics is mirrored

by an execution of the same program in the weak-memory semantics and vice versa, making them functionally equivalent. This allows us to consider only the standard semantics for programming in AuDaLa and verifying AuDaLa, provided there are no read-write race conditions.

We conclude the chapter in Section 6.4.

6.1 Potential Programming Adaptations

In this section, we discuss several potential reasonably simple adaptations for the AuDaLa semantics to make AuDaLa more flexible by supporting more features. First, we discuss the adaptation of the schedule to allow for more diverse looping structures in Section 6.1.1, and then we will discuss the addition of array-like structures to AuDaLa in Section 6.1.2. This will include additions to the syntax and semantics of AuDaLa. Here, we consider the adaptations in this section separately from each other, and that for all adaptations we extend the original semantics of AuDaLa, as given in Section 2.4. As examples, we adapt our code for the AuDaLa reachability program as given in Section 3.1.

6.1.1 Schedule adaptations

In AuDaLa, the schedule controls the flow of the program, using steps, barriers and fixpoints. The fixpoint has two main components as a looping structure. First, it requires synchronisation between all struct instances in each iteration of the loop. Second, it requires that the whole system is stable for the loop to terminate. For the schedule adaptations, we will first relax the second requirement, introducing a possible addition of *parameter-specific fixpoints*, which only require certain parameters to be stable. Then, we will relax the first requirement and give a possible addition of *iterators*, which removes the synchronisation requirement for fixpoints.

Parameter-specific fixpoints

In AuDaLa, fixpoints require that after the execution of the fixpoint every parameters in the entire system is stable. This is convenient theoretically, as this is a strong guarantee for the contents of a state after fixpoint termination. In practice, however, it could be useful to limit the relevant parameters for the fixpoint.

To illustrate this, we have created a program based on the reachability example given in Section 3.1. It is displayed in Listing 6.1. The goal of this program is not only to compute which nodes are reachable, but also to count the iterations necessary to do so. The program does this by counting the iterations of the fixpoint in a parameter *count*.

```

1  struct Node (reach: Bool) {}
2
3  struct Edge (in: Node, out: Node, diam: Int) {
4    reachability {
5      count := count + 1;
6      if (in.reach = true) then {
7        out.reach := true;
8      }
9    }
10   init {
11     Node node1 := Node(true);
12     Node node2 := Node(false);
13     Node node3 := Node(false);
14     Node node4 := Node(false);
15     Node node5 := Node(false);
16
17     Edge edge12 := Edge(node1, node2, -1);
18     Edge edge13 := Edge(node1, node3, -1);
19     Edge edge23 := Edge(node2, node3, -1);
20     Edge edge34 := Edge(node3, node4, -1);
21     Edge edge51 := Edge(node5, node1, -1);
22   }
23 }
24
25 init < Fix(reachability)

```

Listing 6.1: Counting Reachability.

This program, however, will never become stable, as *count* is a parameter. Furthermore, as the value of *count* stays relevant between iterations of the fixpoint, *count* must be a parameter. The only way to mitigate that this program never becomes stable, is to make the program more complicated by computing stability of the graph during the step reachability and somehow making that stability information available to all edges, which would be undesirable. To that end, it may be useful to limit the relevant parameters for the fixpoint to exclude the parameter *count*, which is not stable by design.

To limit the relevant parameters for a fixpoint, we add *parameter-specific fixpoints*: special fixpoints which allow us to specify the parameters the fixpoint requires to be stable. Note that we still consider fixpoints unstable when new struct instances are made during the execution of an iteration of the fixpoint. For this, we expand the syntax of schedules to allow fixpoints of the form: “*Fix*(*sc* : $x_1, x_2, \dots$)”, where *sc* is the subschedule to be executed and $x_1, x_2, \dots$ are the variables which need to be stable for the fixpoint to

terminate. Note that this could be straightforwardly extended in the case the variables need to be uniquely identified with their struct types. In our example in Listing 6.1, we replace the fixpoint with “*Fix(reachability: reach)*”, which makes the program work as intended.

To allow the parameter-specific fixpoints in the syntax of AuDaLa, we have to add an additional entry ‘*Fix*’(‘(‘*Sched*’ : ‘(‘*Vars*’)’’)’ to the syntax for the schedule, which represents a parameter specific fixpoint. Here, $\langle Vars \rangle$ gives the parameters relevant for this fixpoint, with $\langle Vars \rangle ::= \langle Var \rangle \langle AVars \rangle$ and $\langle AVars \rangle ::= ‘,’ \langle Var \rangle \langle AVars \rangle \mid \epsilon$.

Let $\mathcal{P}$ be a program for which the semantics is defined. We then change the definition of the semantics as follows to be able to use parameter-specific fixpoints. First, we adapt the state to include a *stability function*, which is a total function which keeps track of which fixpoints depend on which variables. This function has type $\mathbb{N} \rightarrow ID \rightarrow \mathbb{B}$ and is initialized as *false* for all input pairs $(i, x) \in \mathbb{N} \times ID$. This function is called during a write to check whether this write is relevant for the stability of the fixpoints. Generally, it is only used for parameters, as it is initialized and updated outside of steps. To avoid unnecessary complication, we assume that every parameter name is unique; to remove this assumption, we can extend the stability function type to $\mathbb{N} \rightarrow (Type \times ID) \rightarrow \mathbb{B}$ instead. The function sf can be seen as a stack of functions, where $sf(k)$ denotes which parameters are relevant for the k th fixpoint currently initialized. The new state is then defined as a four-tuple $\langle \Delta, \sigma, s\chi, sf \rangle$, with Δ the schedule, σ the struct type environment, $s\chi$ the stability stack and $sf : \mathbb{N} \rightarrow ID \rightarrow \mathbb{B}$ the *stability function*.

For our initial state, we define the function sf^0 , which returns false for all input pairs $(i, x) \in \mathbb{N} \times ID$, which is added as the fourth element of the initial state.

For the transition rules, we define the function $sf_t^0 : ID \rightarrow \mathbb{B}$, which returns true for all inputs $x \in ID$. We change the rule **ComWr** to the following rule **ComWrN**, which ensures that the stability stack is only reset when the relevant variables are changed. The changed parts of the rule are highlighted.

$$\begin{array}{c}
 \sigma(\ell) = \langle \theta, \mathbf{wr}(x); \gamma, \chi; v; \ell', \xi \rangle \\
 \sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\
 \ell' \notin \mathcal{L}^0 \vee x \notin \text{Pars}_{\mathcal{P}} \theta' \\
 su = (x \notin \text{Pars}_{\mathcal{P}}(\theta') \vee \xi'(x) = v) \\
 \hline
 (\text{ComWrN}) \frac{}{\langle \Delta, \sigma, s\chi, sf \rangle \Rightarrow \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle][\ell', 4 \mapsto \xi'[x \mapsto v]],} \\
 s\chi_1 \wedge (su \vee \neg sf(1)(x)) ; \dots ; s\chi_{|s\chi|} \wedge (su \vee \neg sf(|s\chi|)(x)) , sf \rangle
 \end{array}$$

In **ComWrN**, as in **ComWr**, su is true when x is not a parameter changed by the application of this rule. However, due to the addition of parameter-specific fixpoints, even if x is a changed parameter, whether the entries of the stability

stack need to be reset to *false* also depends on whether x is relevant for the entries. Let $s\chi_k$ be one of the entries of the stability stack. Then whether x is relevant for the k th currently initialized fixpoint is saved in $sf(k)(x)$: if x is relevant, $sf(k)(x) = \text{true}$. Then $s\chi_k$ should only be reset to false (if it isn't false already) if x is a relevant, changed parameter, which is reflected in updating $s\chi_k$ to $s\chi_k \wedge (su \vee sf(k)(x))$.

We also adapt the rule **FixInit** into the rules **FixInitG** (for the general case) and **FixInitS** (for parameter-specific fixpoints), which properly initiates the function sf for every fixpoint. Let $X \subseteq ID$ be some list of parameters. The differences between the new rules and **FixInit** are highlighted.

$$\begin{array}{c}
 \text{Done}(\sigma) \\
 k = |s\chi| + 1 \\
 \hline
 \text{(FixInitG)} \quad \langle \text{Fix}(F); F_1, \sigma, s\chi, sf \rangle \Rightarrow \langle F; a\text{Fix}(F); F_1, \sigma, s\chi; \text{true}, sf[k \mapsto sf_t^0] \rangle \\
 \\
 \begin{array}{c}
 \text{Done}(\sigma) \qquad k = |s\chi| + 1 \\
 X = x_1, \dots, x_n \qquad x_1, \dots, x_n \in \text{ParP} \\
 sf' = sf[x \mapsto sf_f^0[\{x_1 \mapsto \text{true}, \dots, x_n \mapsto \text{true}\}]] \\
 \hline
 \text{(FixInitS)} \quad \langle \text{Fix}(F : X); F_1, \sigma, s\chi, sf \rangle \Rightarrow \langle F; a\text{Fix}(F); F_1, \sigma, s\chi; \text{true}, sf' \rangle
 \end{array}
 \end{array}$$

In these rules, we correctly initialize normal fixpoints to fixpoints dependent on every parameter (as shown by setting $sf(k)$ to sf_t^0) and parameter specific fixpoints to fixpoints only dependent on the given parameters and no others (as shown by the definition of sf' and overwriting $sf(k)$ with sf'). We adapt the rules **FixIter** and **FixTerm** into the rules **FixIterN** and **FixTermN** as follows. Differences are again highlighted.

$$\begin{array}{c}
 \text{Done}(\sigma) \\
 \hline
 \text{(FixIterN)} \quad \langle a\text{Fix}(sc) < sc_1, \sigma, s\chi; \text{false}, sf \rangle \Rightarrow_P \langle sc < a\text{Fix}(sc) < sc_1, \sigma, s\chi; \text{true}, sf \rangle \\
 \\
 \text{Done}(\sigma) \\
 \hline
 \text{(FixTermN)} \quad \langle a\text{Fix}(sc) < sc_1, \sigma, s\chi; \text{true}, sf \rangle \Rightarrow_P \langle sc_1, \sigma, s\chi, sf \rangle
 \end{array}$$

These rules are not significantly different from **FixIter** and **FixTerm**, as both resetting and initializing levels of sf happens simultaneously in **FixInitG** and **FixInitS**. For the command rules, we then also add sf to both the state before and after the transition, much like in **FixIterN** and **FixTermN**.

Iterators

In AuDaLa, fixpoints require that between iterations, there is a synchronisation for all struct instances, through the use of the *Done*-predicate in the transition

rules. This is due to the fact that the rules **FixIter** and **FixTerm** require the *Done*-predicate to be true for the state it is taken from in the semantics of AuDaLa, as defined in Section 2.4. This makes the program more understandable, as it provides a strong guarantee for the status of variables between the executions of the fixpoint.

However, this also inhibits performance, as synchronisation leads to overhead and there are programs for which a strong guarantee on the status of variables between executions is not necessary for understandability. For example, consider again Listing 3.1 for computing reachability. While we could synchronise between the different iterations of the reachability step, this does not lead to a better understanding, less write-write race conditions or other benefits. Though a straightforward extension may be an alternative barrier which does not require synchronisation, we believe that this will remove much of the structure and simplicity of AuDaLa. Therefore, we instead introduce an unsynchronised loop: the *iterator*, where the lack of synchronisation is contained within the iterator only.

In this version of the extension, we do not allow iterators or fixpoints nested into iterators (though an iterator nested into a fixpoint is allowed). We do this because both iterators and fixpoints as we define them require synchronisation at their beginnings and ends. This contradicts our intended use of the iterator, which should execute all steps within it without synchronisation.

The iterator takes the form $Iter(F_1; F_2; F_3; \dots)$, with every occurrence F_i a step name. During execution, the iterator will loop over these steps. With S_i the statements of the step F_i , every iteration, all struct instances execute the commands generated by $\llbracket S_1; S_2; S_3; \dots \rrbracket$, if they have any. If any struct instances which had commands are finished executing their commands, the iterator can check whether the system is stable. If it is not, then the iterator will make sure that for all struct instances that have to execute commands, there is at least one more iteration of the commands at the end of the command list. It does this by adding the commands at the end of the command list of every struct instance which does not have the full command sequence of an iteration in their command list anymore.

The iterator terminates when there are no more commands to execute and the system is stable. This results in a loop where struct instances asynchronously execute the sequence of steps included in the iterator until the system is stable.

For example, if we were to execute the step *reachability* as given in Section 3.1 with an iterator, we would put “ $Iter(reachability)$ ” in the schedule. This would cause the struct instances to asynchronously execute multiple iterations of the step *reachability*, until the system is stable. Here, as the struct instances of *Node* do not execute any commands for the step *reachability*, they are not taken into account for the iterator. If we had another step *step*, we could then

also call “*Iter(reachability;step)*”. This would cause all struct instances that have commands for any of the steps *reachability* and *step* to asynchronously execute multiple iterations of the commands of *reachability* followed by the commands of *step*, until the system is stable.

To implement our iterator, we extend AuDaLa’s syntax by adding the option ‘*Iter*’(‘*StepList*’)

 for the schedule, where $\langle \textit{StepList} \rangle ::= \langle \textit{Id} \rangle \mid \langle \textit{Id} \rangle; \langle \textit{StepList} \rangle$, where the $\langle \textit{Id} \rangle$ occurrences are type checked to be step names. This step list holds the steps to be executed during the iterator, which are executed without synchronisation in between.

In the remainder of this section, let $\mathcal{P}$ be some AuDaLa program. We need to add transition rules to be able to implement the iterator, which are based on the transition rules for fixpoints. First, we add a rule to initialize iterators. This rule, **IterInit**, initialises the iterator and also introduces the meta-schedule element *alter*, which denotes an iterator which has been started but has not finished yet. We use F^+ to denote a list of one or more steps. We define the function $I(\ell, F^+)$ with $\ell \in \mathcal{L}$ and $F^+ = F_1; \dots; F_n$, which returns the list of commands $\llbracket S_{\theta_\ell}^{F_1} \rrbracket; \dots; \llbracket S_{\theta_\ell}^{F_n} \rrbracket$. Recall from Section 2.4 that $\mathbb{C}$ is the set of all commands. Changes from the rule **FixInit** are highlighted.

$$\begin{array}{c}
 \text{Done}(\sigma) \\
 \hline
 \text{(IterInit)} \quad \frac{}{\langle \textit{Iter}(F^+) \rangle < sc_1, \sigma, s\chi \rangle \Rightarrow \mathcal{P} \langle \textit{alter}(F^+) \rangle < sc_1,} \\
 \sigma[\{\ell \mapsto \langle \theta_\ell, I(\ell, F^+) \rangle, \varepsilon, \xi_\ell, \textit{true} \} \mid \sigma(\ell) = \langle \theta_\ell, \gamma_\ell, \chi_\ell, \xi_\ell \rangle], s\chi; \textit{true}
 \end{array}$$

In the second rule, we define when an iterator needs to do another iteration. This rule works as follows: When a struct instance is done executing all steps of the iterator, and the stability stack is false, then either some other struct instance caused this instability or the current struct instance did. As we cannot discount either option, it follows that all struct instances should execute the steps from the iterator at least one more time before the iterator can be considered stable. Therefore, the rule tops up the commands of all struct instance s.t. their command lists end in exactly one full list of commands as generated by the steps in the iterator. The rule also sets the stability stack to *true*. This is safe: every struct instance will execute the steps at least one more time, so the execution of the iterator cannot stop before at least one full iteration has happened in which no parameter was changed and no struct instances were created. This means that the iterator successfully computes a fixed point when it terminates with this iteration transition rule. Note that in this rule, ℓ' is universally quantified.

$$\begin{array}{c}
\text{(IterIter)} \frac{\exists \ell \in \mathcal{L}. (\sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle \wedge I(\ell, F^+) \neq \varepsilon)}{\langle \text{aIter}(F^+) < sc_1, \sigma, s\chi; \text{false} \rangle \Rightarrow_{\mathcal{P}} \langle \text{aIter}(F^+) < sc_1, \\
\sigma[\{\ell' \mapsto \langle \theta_{\ell'}, \gamma_{\ell'}; I(\ell', F^+), \chi_{\ell'}, \xi_{\ell'} \rangle \mid \\
\sigma(\ell') = \langle \theta_{\ell'}, \gamma_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle \wedge \nexists \gamma' \in \mathbb{C}^*. (\gamma_{\ell'} = \gamma'; I(\ell', F^+))\}, s\chi; \text{true} \rangle}
\end{array}$$

In the third rule, which can be taken when the iterator is stable and every struct instance is done, the iterator terminates.

$$\begin{array}{c}
\text{(IterTerm)} \frac{\text{Done}(\sigma)}{\langle \text{alter}(F^+) < sc_1, \sigma, s\chi; \text{true} \rangle \Rightarrow_{\mathcal{P}} \langle sc_1, \sigma, s\chi \rangle}
\end{array}$$

With the first rule, we satisfy that the iterator requires a synchronisation before starting its execution. In the second rule, we allow struct instances to start new iterations of the iterator step without synchronisation, and with the third rule, we require a synchronisation before the loop terminates. This satisfies the first requirement. The commands put on the command lists of struct instances in the first and second rules are generated from the entire subschedule inside the iterator. As commands are run asynchronously, it follows that the second requirement also holds. The third requirement holds because AuDaLa syntax does not permit fixpoints or iterators nested in iterators when expanded with the syntax additions given above. It follows that these extensions correctly implement the iterator as intended into the semantics of AuDaLa.

We can use the iterator to create better code for AuDaLa reachability, by changing Listing 3.1's schedule to the schedule given below:

`1 init < Iter(reachability)`

Listing 6.2: New Schedule for Listing 3.1

6.1.2 Arrays

Arrays offer a quick way of grouping elements which allows for quick access due to insights in how things are stored in the memory. AuDaLa does not naturally support dynamically-sized arrays. This is due to the fact that while AuDaLa does not assume a memory implementation, it does require that all struct instances only have a constant-sized amount of memory to their disposal, as AuDaLa focuses on small data elements.

As arrays are widely used in other programming languages and algorithms, supporting arrays may make it easier for people used to those other languages

and algorithms to adopt AuDaLa. Furthermore, it may make it easier to translate existing implementations to AuDaLa. It would allow open up the possibility to leverage the constant time access arrays are known for in AuDaLa. Therefore, we propose one way to extend AuDaLa with support for arrays. This would allow people to use arrays with AuDaLa, and would allow people to leverage the constant time access inherent to them as well.

To adopt arrays in AuDaLa, we introduce a new, special *array instance*. The idea is that this array instance owns an array and saves the array starting address and size. As with conventional arrays, while the size may be chosen dynamically upon initialization, it may not be changed after the array has been initialized. The array is then filled with labels of struct instances, functioning as links to those struct instances. Each element can be accessed through indirect access.

Note that this keeps the assumption of constant-sized memory intact for non-array instances, which we prefer over abolishing the assumption and allowing arrays through allowing dynamically-sized parameters for every struct. In this extension, we also consider every entry of an array to be a parameter, to keep the extension simple.

First, we add the new expression $\text{'array'}(\langle \text{Exp} \rangle)$ to the syntax of AuDaLa. This expression reflects the creation of a new array. The expression contained in it represents the size of the array. When type checking, the inner expression should resolve to a natural number larger than 0, and the whole program should again be well-typed.

When accessing variables, we allow for variable accesses of the type $a[E]$, where $a[E]$ is an array access with E an expression. Accesses can be chained with other variable accesses when necessary. This is reflected by changing the syntax for variables to

$$\langle \text{Var} \rangle ::= \langle \text{Var} \rangle . \langle \text{Id} \rangle \mid \langle \text{Var} \rangle ' [\langle \text{Exp} \rangle '] ' \mid \langle \text{Id} \rangle$$

We add the type checking requirement that the expression in the square brackets has to resolve to a natural number. We also add the requirement that if there is an array access $a[E_1] \cdots [E_n]$ in the syntax, a must be of type $\text{Array}_1(\cdots(\text{Array}_n(T)))$ for some T . We furthermore require updates of arrays to be well-typed. Lastly, we consider “array” to be a protected word in the syntax.

We also allow for a new expression $\langle \text{Var} \rangle . \text{s}'$, which returns the size of the array given for $\langle \text{Var} \rangle$. During type checking, we require $\langle \text{Var} \rangle$ to refer to an array.

Note that in this extension, we have not included array initializations like $\text{Int}[2] \ a := \{0, 1\}$, as we would like to keep our commands as simple as possible. We are therefore not inclined to introduce some kind of multiwrite command. This means that the commands generated from a statement like the one given

above would not be significantly different from the statements “ $\text{Array}(\text{Int}) \ a := \text{array}(2); a[0] := 0; a[1] := 1;$ ”.

Let $\mathcal{P}$ be an arbitrary program. We create a new set $\text{Types}_{\mathcal{P}}^a = \Theta_{\mathcal{P}} \cup \{\mathbb{N}, \mathbb{Z}, \mathbb{B}, \text{String}\} \cup \text{Types}_{\mathcal{P}}^1 \cup \text{Types}_{\mathcal{P}}^2 \cup \dots$, where

$$\text{Types}_{\mathcal{P}}^1 = \{\text{Array}(\theta) \mid \theta \in \Theta_{\mathcal{P}}\} \cup \{\text{Array}(\mathbb{N}), \text{Array}(\mathbb{Z}), \text{Array}(\mathbb{B}), \text{Array}(\text{String})\}$$

and $\text{Types}_{\mathcal{P}}^i = \{\text{Array}(T) \mid T \in \text{Types}_{\mathcal{P}}^{i-1}\}$ in the semantics. We also define that in the semantics, arrays are labelled using the set $\mathcal{L}^a \subset \mathcal{L}$.

Arrays are dependent on the memory. To that end, and to keep the struct environment separate from hardware dependent memory, we add a *memory function* $\mathcal{M} : \mathcal{A} \rightarrow \mathcal{V}$ to the semantic state, with $\mathcal{A}$ a set of memory addresses and $\mathcal{V}$ the set of all values (with $\mathcal{A} \subset \mathbb{N}$). This function models the hardware memory, where the addresses for which it is defined are allocated addresses. We also define that our set $\mathbb{N}$ must include the number 0. We consider the empty address to be an abstract, non-existent address α_0 .

We define an *array instance* a as a tuple $\langle \alpha, s \rangle$, which contains the address α at which a starts and the size s of a . We save array instances in the struct environment (and extend the struct environment to facilitate this). Lastly, we extend the *null*-instances with a *null*-instance $\langle \alpha_0, 0 \rangle$, with label ℓ_0^a . We extend *defaultVal* such that $\text{defaultVal}(\text{Array}(T)) = \ell_0^a$ for any $T \in \Theta_{\mathcal{P}}^a$.

To work with arrays, we create the commands **rdA** and **wrA** with $x \in ID$ a variable, to read from and write to a location of an array. We also add the new commands **arr**(T), with $T \in \Theta_{\mathcal{P}}^a$ to make a new array, and **asize** which takes a variable $x \in ID$ and retrieves the size of the array saved under x .

For the interpretation function, we remove the interpretation function clauses $\llbracket x_1 \dots x_n \rrbracket$, $\llbracket T \ x := E \rrbracket$ and $\llbracket x_1, \dots, x_n.x := E \rrbracket$ and we add the following clauses for variables, with expressions E and E' , sequences of variables X , X_1 and X_2 and single variable x . Here, X is a variable expression as constructed by the syntax above. We use the notation $X[E]$ to denote that the last $\langle \text{Var} \rangle$ element of X is an array access element “[E]” and we use the notation $X.x$ to denote that the last $\langle \text{Var} \rangle$ element of X is a variable access element “. x ”.

$$\begin{aligned} \llbracket x \rrbracket &= \text{push}(\text{this}); \text{rd}(x) \\ \llbracket X[E] \rrbracket &= \llbracket X \rrbracket; \llbracket E \rrbracket; \text{rdA} \\ \llbracket X.x \rrbracket &= \llbracket X \rrbracket; \text{rd}(x) \\ \llbracket x := E' \rrbracket &= \llbracket E' \rrbracket; \text{push}(\text{this}); \text{wr}(x) \\ \llbracket X[E] := E' \rrbracket &= \llbracket E' \rrbracket; \llbracket X \rrbracket; \llbracket E \rrbracket; \text{wrA} \\ \llbracket X.x := E' \rrbracket &= \llbracket E' \rrbracket; \llbracket X \rrbracket; \text{wr}(x) \\ \llbracket T \ X := E \rrbracket &= \llbracket X := E \rrbracket \end{aligned}$$

We also add the following clauses to deal with array creation and requesting

the size of an array:

$$\begin{aligned} \llbracket \text{'array'}(E) \rrbracket &= \llbracket E \rrbracket; \mathbf{arr}(T) \\ \llbracket X.s \rrbracket &= \llbracket X \rrbracket; \mathbf{asize} \end{aligned}$$

We then add new operational semantics rules to reflect reading from an array (**ComRdA**), writing to an array (**ComWrA**), getting the size of the array (**ComArrs**) and creating an array (**ComArr**). We also adapt the other operational semantics to include $\mathcal{M}$ into the state. The rule **ComRdA** given below is based on the rule **ComRd**, with changes to access the memory instead of variable environments and to use the command **RdA**. Note that the location is realized as a displacement from the memory address of the array. Changes from **ComRd** are highlighted.

$$\begin{array}{c} \sigma(\ell) = \langle \theta, \mathbf{rdA}; \gamma, \chi; \ell'; v, \xi \rangle \\ \sigma(\ell') = \langle \alpha, s \rangle \quad v < s \quad \mathcal{M}(\alpha + v) \text{ is defined} \\ \text{(ComRdA)} \frac{}{\langle \Delta, \sigma, \mathcal{M}, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \mathcal{M}(\alpha + v), \xi \rangle], \mathcal{M}, s\chi \rangle} \end{array}$$

The rule **ComWrA** given below is based on **ComWr**. Like **ComWr**, it keeps track of whether it has to update the stability stack if the array is changed and the array is a parameter, though this means that in the end, three entries of the struct environment are integral to the functioning of this command.

$$\begin{array}{c} \sigma(\ell) = \langle \theta, \mathbf{wrA}; \gamma, \chi; v_1; \ell'; v_2, \xi \rangle \quad \sigma(\ell') = \langle \alpha, s \rangle \\ \ell' \notin \mathcal{L}^0 \quad v_2 < s \quad su = (\mathcal{M}(\alpha + v_2) = v_1) \\ \text{(ComWrA)} \frac{}{\langle \Delta, \sigma, \mathcal{M}, s\chi \rangle \Rightarrow \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle], \mathcal{M}, s\chi \rangle} \\ \mathcal{M}[(\alpha + v_2) \mapsto v_1], s\chi_1 \wedge su; \dots; s\chi_{|s\chi|} \wedge su \end{array}$$

The rule **ComAsize** given below is also based on **ComRd**, and returns the size of an array. The differences with **ComRd** are highlighted.

$$\begin{array}{c} \sigma(\ell) = \langle \theta, \mathbf{asize}; \gamma, \chi; \ell', \xi \rangle \\ \sigma(\ell') = \langle \alpha, s \rangle \\ \text{(ComAsize)} \frac{}{\langle \Delta, \sigma, \mathcal{M}, s\chi \rangle \Rightarrow \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; s, \xi \rangle], \mathcal{M}, s\chi \rangle} \end{array}$$

The last new rule is **ComArr**, which creates a new array. This rule is based on **ComCons**, and differences with that rule are highlighted. Note that ℓ' is a

fresh label in this rule, signified by $\sigma(\ell') = \perp$.

$$\begin{array}{c}
 \sigma(\ell) = \langle \theta, \mathbf{arr}(T); \gamma, \chi; s, \xi \rangle \\
 \hline
 \text{(ComArr)} \quad \frac{s \in \mathbb{N} \quad s \geq 1 \quad \mathcal{M} \text{ is not defined for } \alpha, \dots, (\alpha + s) \quad \sigma(\ell') = \perp}{\langle \Delta, \sigma, \mathcal{M}, s\chi \rangle \Rightarrow \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \gamma, \chi; \ell', \xi \rangle, \ell' \mapsto \langle \alpha, s \rangle\}], \mathcal{M}[\{\alpha_i \mapsto \text{DefaultVal}(T) \mid \alpha_i \in \alpha, \dots, (\alpha + s)\}], s\chi \rangle}
 \end{array}$$

With the rule **ComArr**, we can now make array instances, allocating them a sequence of unallocated addresses. We can also get the size of the array through the addition `.s` behind an array variable `x`, which is then handled by the transition **ComASize**, induced by the **asize** command. We can lastly read and write from those arrays using conventional block-parenthesis notation, and the locations we read and write to can be determined dynamically. This satisfies our expectations for an array. Note that this extension does not include the destruction and deallocation of arrays.

With this extension, we are able to reimplement reachability to use arrays, as shown in Listing 6.3 (based on Listing 3.1). As you can see in the Listing, arrays make the system smaller, but as loops are contained in the schedule, one cannot employ a for-loop to walk through all array instances in a single step, and the code must be adapted for this.

6.2 The Weak-Memory Semantics

In this section, we adapt the standard semantics as given in Chapter 2 into the *weak-memory semantics*, which assumes a weaker memory consistency model than sequential consistency. This semantics allows for the reordering of command executions, where commands may be executed if all commands they depend on have already been executed. Notably, it allows for the execution of read and write commands if all preceding commands that reference the same variable have been executed. Furthermore, the weak-memory semantics allows for incoherence between the variable values established by different struct instances.

We base this new semantics on the standard semantics and syntax as presented in Chapter 2. This leads to a slightly different semantics than the semantics given in the paper this section is based on [110]. We discuss the differences in depth at the end of the section, but all of them can be mitigated by transforming the syntax of the programs only slightly between the semantics presented in the thesis and the semantics presented by Leemrijse [110].

To adapt the standard semantics to the weak semantics, we first discuss the differences between the design of the original semantics and the weak-memory semantics as presented in the thesis. We then give all the definitions,

```

1  struct Node (reach: Bool, succ: Array(Node), done: Nat) {
2    reachability {
3      if (reach = true) then {
4        if (done < succ.s){
5          succ[done].reach := true;
6          done := done + 1;
7        }
8      }
9    }
10
11   init {
12     Node node1 := Node(true, array(2), 0);
13     Node node2 := Node(false, array(1), 0);
14     Node node3 := Node(false, array(1), 0);
15     Node node4 := Node(false, null, 0);
16
17     node1.succ[0] := node2;
18     node1.succ[1] := node3;
19     node2.succ[0] := node3;
20     node3.succ[0] := node4;
21   }
22 }
23 init < Fix(reachability)

```

Listing 6.3: AuDaLa code for a reachability program on a small graph using arrays

functions and rules changed from Section 2.4. All definitions and functions not given are assumed to be the same in the two semantics.

The first big change, to be able to reorder accesses without destroying the structure of the command list, we unify the command list with the value stack, resulting in a new stack where commands are replaced by their results. For this, we adapt the commands: we remove the **push**-command and we change how certain other commands are processed by the operational semantics. Furthermore, we add an extra write command, **wrP**. When writing to a parameter, **wr** is then split to a read and a write using the **wrP** command, to make explicit that the parameter needs to be read before it is written to, to check if the stability stack needs to be reset. This is beneficial for interleaving and performance, as this only requires **rd** and **wrP** to be atomic, and not **wr** as a whole.

The second big change, to allow for incoherence, we extend the state with an *initial value function* ι and a *global memory* μ , which contains new values written to parameters, combined with the label of the struct instance that wrote the value. The variable environments in the struct instances are then

considered to be caches. During the execution of steps, there are then two possibilities for reading from a variable. The first is reading from the cache, or, if the cache does not have a value for the current variable, retrieving a value from the initial value function. The second is to retrieve a value for the variable from the global memory (if the global memory contains values for the variable). Which one of the possibilities is taken is non-deterministic in the semantics, to reflect that in practice, with incoherent memory and reshuffled accesses, reads can happen in a lot of ways, depending on the occasion.

New parameters for constructors are added to the initial value function, and writes to a parameter are added to both the cache and the global memory. To process the global memory back into an initial value function for the next step, we add a new rule **Finish** to the operational semantics.

For the actual adaptation of the standard semantics, we keep everything up till the definition of the commands the same as in Section 2.4. We have to change the commands, however, to accomodate the fact that we now work with only a single value/command stack (so **push**-commands are superfluous), that we need to add the **wrP** command for writing to parameters and that we need to change the **if**-command to take a value/command stack as argument. Therefore, we adapt Definition 2.4.4 to the following:

Definition 6.2.1 (Commands (Weak-Memory)). A *command* c is constructed according to the following grammar:

$$c ::= \mathbf{this} \mid \mathbf{rd}(x) \mid \mathbf{wr}(x) \mid \mathbf{wrP}(x) \mid \mathbf{cons}(x) \mid \mathbf{if}(VC) \mid \mathbf{not} \mid \mathbf{op}(o)$$

where **this** is a special value, $x \in ID$ is an identifier, VC is a list of values and commands, and $o \in O$ is an operator. The set of all commands is $\mathbb{C}$.

We then have to change the interpretation function to deal with the removal of the **push**-command and to reflect that we now only have one stack. The **wrP**-command is not generated from the syntax, but introduced during execution.

Definition 6.2.2 (Interpretation function (Weak-Memory)). Let $x, x_1, \dots, x_n \in ID$ be variables, $E, E_1, \dots, E_m \in EX$ expressions, $g \in LT$ a literal, $\theta \in ID$ a struct type, $S \in ST$ a statement, $S \in ST^*$ a list of statements, $T \in \mathcal{T}$ a type and $o \in O$ an operator from the syntax. Let the list $x_1; \dots; x_n$ be the list of n variables from x_1 to x_n . We define the *interpretation function* $\llbracket \cdot \rrbracket : ST^* \cup EX \rightarrow (\mathbb{C} \cup \mathcal{V})^*$

transforming a list of statements or expressions into a list of commands:

$$\begin{array}{ll}
\llbracket g \rrbracket = \text{val}(g) & \llbracket \text{if } E \text{ then } \{S\} \rrbracket = \llbracket E \rrbracket; \mathbf{if}(\llbracket S \rrbracket) \\
\llbracket \text{this} \rrbracket = \mathbf{this} & \llbracket T \ x := E \rrbracket = \llbracket x_1 \cdots x_0. x := E \rrbracket \\
\llbracket \text{null}_T \rrbracket = \text{defaultVal}(T) & \llbracket x_1 \cdots x_n. x := E \rrbracket = \llbracket E \rrbracket; \llbracket x_1 \cdots x_n \rrbracket; \mathbf{wr}(x) \\
\llbracket x_1 \cdots x_n \rrbracket = \mathbf{this}; \mathbf{rd}(x_1); \dots; \mathbf{rd}(x_n) & \llbracket \theta(E_1, \dots, E_m) \rrbracket = \llbracket E_1 \rrbracket; \dots; \llbracket E_m \rrbracket; \mathbf{cons}(\theta) \\
\llbracket !E \rrbracket = \llbracket E \rrbracket; \mathbf{not} & \llbracket \varepsilon \rrbracket = \varepsilon \\
\llbracket E_1 \circ E_2 \rrbracket = \llbracket E_1 \rrbracket; \llbracket E_2 \rrbracket; \mathbf{op}(\circ) & \llbracket S; S \rrbracket = \llbracket S \rrbracket; \llbracket S \rrbracket
\end{array}$$

We fix that if $n = 0$, $\llbracket x_1 \cdots x_n \rrbracket = \mathbf{this}$ as a special case. We require **this** to give us the label of the current executing struct instance at the start of dereferencing any pointer.

We change the semantics of struct instances to reflect our new value/-command stack and the adjusted role of ξ as a cache. As the symbol for the value/command stack, we overload χ , and we ξ to represent the cache instead of the variable environment of a struct instance.

Definition 6.2.3 (Struct instance (Weak-Memory)). A *struct instance* is a tuple $\langle \theta, \chi, \xi \rangle$ where:

- $\theta \in ID$ is the type of the struct,
- $\chi \in (\mathbb{C} \cup \mathcal{V})^*$ is a stack that stores values during the evaluation of an expression,
- $\xi : \mathcal{L} \times ID \rightarrow \mathcal{V} \cup \{\perp\}$ is a value cache that stores the values of local variables as well as parameters. The special value $\perp$ is used to denote that variables do not have a value in the cache.

We define $\mathbb{S}$ as the set of all possible struct instances and $\xi_\perp$ as the empty cache.

We also change the definition of a AuDaLa state:

Definition 6.2.4 (State (Weak-Memory)). A *state* is a tuple $\langle \Delta, \sigma, \mu, \iota, s\chi \rangle$, where:

- $\Delta \in SC$ is a schedule expressed as a list,
- $\sigma : \mathcal{L} \rightarrow \mathbb{S} \cup \{\perp\}$ is a *struct environment*,
- $\mu : \mathcal{L} \times ID \rightarrow 2^{\mathcal{V} \times \mathcal{L}}$ is a *global memory* of parameter values written during the current step paired with their writer's label.
- $\iota : \mathcal{L} \times ID \rightarrow \mathcal{V}$ is an *initial value function* of parameter values, which records the value of each parameter before the start of the current step
- $s\chi \in \mathbb{B}^*$ is a *stability stack*.

The set of all states is defined as $\mathcal{G}$. The *empty global memory* μ_0 is defined as the global memory μ where all label-variable pairs map to the empty set.

We adapt the definition of null-instances straightforwardly to reflect the expanded state.

We redefine $\sigma_{\mathcal{P}}^0$ as $\sigma_{\perp}[\{\ell_{\theta}^0 \mapsto \langle \theta, \varepsilon, \xi_{\perp} \mid \theta \in \Theta_{\mathcal{P}} \}\}]$, with $\sigma_{\perp}$ the empty struct environment. We furthermore define $\iota_{\mathcal{P}}^0 = \iota_?[\{(\ell_{\theta}^0, x) \mapsto \text{defaultVal}(T) \mid \theta \in \Theta_{\mathcal{P}}, x : T \in \text{Pars}_{\mathcal{P}}(\theta)\}]$, where $\iota_?$ is an arbitrary initial value function. Using these, we give a new definition for the initial state:

Definition 6.2.5 (Initial state (Weak-Memory)). The *initial state* of $\mathcal{P}$ is $p_{\mathcal{P}}^0 = \langle \Delta_{\mathcal{P}}, \sigma_{\mathcal{P}}^0, \mu_0, \iota_{\mathcal{P}}^0, \varepsilon \rangle$.

We furthermore adapt the *Done*-predicate. We define the predicate $\text{End}(\sigma)$ to be equivalent to $\text{Done}(\sigma)$ as given in Section 2.4, and we redefine the *Done*-predicate as $\text{Done}(\sigma, \mu) = \text{End}(\sigma) \wedge \mu = \mu_0$.

Lastly, we add a new predicate:

Definition 6.2.6 (The *Refs* predicate). The predicate $\text{Refs} : (\mathcal{V} \cup \mathbb{C})^* \times \mathcal{L} \times ID \rightarrow \mathbb{B}$ is defined as:

$$\begin{aligned} \text{Refs}(\chi, \ell, x) = & \\ & \exists \chi_1, \chi_2 \in (\mathcal{V} \cup \mathbb{C})^*, \text{src} \in \{\ell\} \cup \mathbb{C}, \text{cmd} \in \{\mathbf{rd}(x), \mathbf{wr}(x), \mathbf{wrP}(x)\}. \\ & \chi = (\chi_1; \text{src}; \text{cmd}; \chi_2) \\ & \vee \exists \chi_1, \chi_2, \chi_3 \in (\mathcal{V} \cup \mathbb{C})^*. \\ & \chi = (\chi_1; \mathbf{if}(\chi_2); \chi_3) \wedge \text{Refs}(\chi_2, \ell, x) \end{aligned}$$

Intuitively, $\text{Refs}(\chi, \ell, x)$ checks if a read or write command is present in χ that references x owned by ℓ (if $\text{src} = \ell$) or can potentially reference x owned by ℓ (if src is a command which still has to be processed). When commands of the form $\mathbf{if}(\chi_2)$ are present in χ , the contents of the $\mathbf{if}$ -command will get checked recursively by Refs .

With this, we can define the transition function of the weak-memory semantics in full. The first rule is to replace the **this** command with the label of the current struct instance:

$$\text{(WMComThis)} \frac{\sigma(\ell) = \langle \theta, \chi_1; \mathbf{this}; \chi_2, \xi \rangle}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \ell; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle}$$

We continue with the rules for the **not**, **op** and **if**-commands, which are not changed notably from the version of the standard semantics. Let b be a boolean value, let $\circ \in O$ and let $o \in \mathcal{O}$ be the semantic equivalent of $\circ$. Let $a, b \in \mathcal{V}$, let $VC \in (\mathcal{V} \cup \mathbb{C})^*$.

$$\text{(WMComNot)} \frac{\sigma(\ell) = \langle \theta, \chi_1; b; \mathbf{not}; \chi_2, \xi \rangle}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \neg b; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle}$$

$$\begin{aligned}
(\text{WComOp}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; a; b; \mathbf{op}(\circ); \chi_2, \xi \rangle}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; (a \circ b); \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
(\text{WComIfT}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; \text{true}; \mathbf{if}(VC); \chi_2, \xi \rangle}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; VC; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
(\text{WComIfF}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; \text{false}; \mathbf{if}(VC); \chi_2, \xi \rangle}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle}
\end{aligned}$$

Then, we give the rules for reading in the weak-memory semantics: **WComRdInit** (initial), **WComRdInt** (internal) and **WComRdExt** (external). In these rules, the *Refs*-predicate ensures that read commands can only be processed if all of their dependencies in the earlier commands are resolved. The rule **WComRdInit** reads the initial value of a variable, for if the variable is not yet in the cache. The rule **WComRdInt** reads the value of a variable from the cache. Lastly, the rule **WComRdExt** reads from global memory. The global memory holds all values written to the variable paired with the label of the struct instance that wrote them, and the read takes one at random (as we do not assume any relation between the step executions of struct instances). This read can only happen when reading from another struct instance.

$$\begin{aligned}
(\text{WComRdInit}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; \ell'; \mathbf{rd}(x); \chi_2, \xi \rangle \quad \xi(\ell', x) = \perp \quad \neg \text{Refs}(\chi_1, \ell', x)}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi; \iota(\ell', x); \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
(\text{WComRdInt}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; \ell'; \mathbf{rd}(x); \chi_2, \xi \rangle \quad \xi(\ell', x) \neq \perp \quad \neg \text{Refs}(\chi_1, \ell', x)}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi; \xi(\ell', x); \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
(\text{WComRdExt}) \quad & \frac{\sigma(\ell) = \langle \theta, \chi_1; \ell'; \mathbf{rd}(x); \chi_2, \xi \rangle \quad (v, \ell'') \in \mu(\ell', x) \quad \ell'' \neq \ell \quad \neg \text{Refs}(\chi_1, \ell', x)}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi; v; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle}
\end{aligned}$$

Recall that the choice of which rule is applied is non-deterministic.

The next rules to discuss are the rules for writing to variables. The rule for local variables (**WComWrLoc**) and the rule for writing to null-parameters (**WComWrNSkip**) are relatively unchanged. We add, however, a rule for writing to parameters, which happens in two parts. The first part, given by the rule **WComWrP-I** decomposes the write into a read and a write denoted with the **wrP**-command. The second part, given by the rule **WComWrP-II**, resolves the **wrP**-command and sets the stability stack if necessary. Here, both the cache and the global memory are updated with the new value of the parameter.

$$(\text{WComWrLoc}) \quad \frac{\sigma(\ell) = \langle \theta, \chi_1; v; \ell; \mathbf{wr}(x); \chi_2, \xi \rangle \quad x \notin \text{Pars}_{\mathcal{P}}(\theta) \quad \neg \text{Refs}(\chi_1, \ell, x)}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \chi_2, \xi[(\ell, x) \mapsto v] \rangle], \mu, \iota, s\chi \rangle}$$

$$\begin{array}{c}
\text{(WComWrNSkip)} \frac{\sigma(\ell) = \langle \theta, \chi_1; v; \ell'; \mathbf{wr}(x); \chi_2, \xi \rangle \quad \sigma(\ell') = \langle \theta', \chi', \xi' \rangle \quad x \in \text{ParsP}(\theta') \quad \ell' \in \mathcal{L}_0}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
\\
\text{(WComWr-I)} \frac{\sigma(\ell) = \langle \theta, \chi_1; v; \ell'; \mathbf{wr}(x); \chi_2, \xi \rangle \quad \sigma(\ell') = \langle \theta', \chi', \xi' \rangle \quad x \in \text{ParsP}(\theta') \quad \ell' \notin \mathcal{L}_0}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \ell'; \mathbf{rd}(x); v; \ell'; \mathbf{wrP}(x); \chi_2, \xi \rangle], \mu, \iota, s\chi \rangle} \\
\\
\text{(WComWr-II)} \frac{\sigma(\ell) = \langle \theta, \chi_1; v_o; v_n; \ell'; \mathbf{wrP}(x); \chi_2, \xi \rangle \quad su = (v_o = v_n) \quad \neg \text{Refs}(\chi_1, \ell', x)}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \chi_1; \chi_2, \xi[(\ell', x) \mapsto v_n] \rangle], \mu[(\ell', x) \mapsto \mu(\ell', x) \cup \{(v_n, \ell)\}], \iota, s\chi_1 \wedge su; \dots; s\chi_{|s\chi|} \wedge su \rangle}
\end{array}$$

The next rule is an adaptation of the **ComCons** rule of the standard semantics. Instead of changing the variable environments, this rule adds the parameter values of the newly created struct instances to the initial value function.

$$\text{(WComCons)} \frac{\sigma(\ell) = \langle \theta, \chi_1; v_1; \dots; v_n; \mathbf{cons}(\theta'); \chi_2, \xi \rangle \quad \text{ParsP}(\theta') = x_1 : T_1; \dots; x_n : T_n \quad \sigma(\ell') = \perp}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \chi_1; \ell'; \chi_2, \xi \rangle, \ell' \mapsto \langle \theta', \varepsilon, \xi_{\perp} \rangle\}], \mu, \iota[\{(\ell', x_i) \mapsto v_i \mid 1 \leq i \leq n\}], \text{false}^{|s\chi|}] \rangle}$$

We then add a new rule, **Finish**, which must be taken at the end of each step. Recall that $\text{End}(\sigma)$ is equivalent to $\text{Done}(\sigma)$ as defined in Section 2.4. The rule updates ι with the values collected in μ . In the rule, the function $W(\ell, x)$ gives the defined cache values of the variable x as owned by ℓ accross all struct instances. Furthermore, we use $\text{fin}(\ell, x)$ to denote a non-deterministically chosen *final* value for the variable x as owned by ℓ , or, if x was not written to, the initial value.

$$\text{(WFinish)} \frac{\text{End}(\sigma) \quad \mu \neq \mu_0 \quad W(\ell, v) = \{\xi(\ell, x) \mid \sigma(\ell') = \langle \theta, \chi, \xi \rangle \setminus \{\perp\}\} \quad \text{fin}(\ell, x) \in W(\ell, x) \vee (W(\ell, v) = \emptyset \wedge \text{fin}(\ell, x) = \iota(\ell, x))}{\langle \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \chi_{\ell}, \xi_{\perp} \rangle \mid \sigma(\ell) = \langle \theta, \chi_{\ell}, \xi_{\ell} \rangle\}], \mu_0, \iota[\{(\ell, x) \mapsto \text{fin}(\ell, x) \mid (\ell, x) \in \mathcal{L} \times \text{ID}\}], s\chi \rangle}$$

With this, we have adapted all the command rules. The schedule rules are only adapted to reflect our new state and struct instance definition. Let $\mathcal{S}_{\theta}^F$ be the statements list of F for θ , as defined in Section 2.4.

$$\text{(WInitG)} \frac{\text{Done}(\sigma, \mu)}{\langle F < \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \llbracket \mathcal{S}_{\theta}^F \rrbracket, \xi \rangle \mid \sigma(\ell) = \langle \theta, \varepsilon, \xi \rangle\}], \mu, \iota, s\chi \rangle}$$

$$\begin{array}{c}
\text{(WMinItL)} \frac{Done(\sigma, \mu)}{\langle \theta.F < \Delta, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \llbracket S_{\theta}^F \rrbracket, \xi \rangle \mid \sigma(\ell) = \langle \theta, \chi, \xi \rangle\}], \mu, \iota, s\chi \rangle} \\
\\
\text{(WMFixInit)} \frac{Done(\sigma, \mu)}{\langle Fix(\Delta) < \Delta_1, \sigma, \mu, \iota, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, \mu, \iota, s\chi; true \rangle} \\
\\
\text{(WMFixIter)} \frac{Done(\sigma, \mu)}{\langle aFix(\Delta) < \Delta_1, \sigma, \mu, \iota, s\chi; false \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, \mu, \iota, s\chi; true \rangle} \\
\\
\text{(WMFixTerm)} \frac{Done(\sigma, \mu)}{\langle aFix(\Delta) < \Delta_1, \sigma, \mu, \iota, s\chi; true \rangle \Rightarrow_{\mathcal{P}} \langle \Delta_1, \sigma, \mu, \iota, s\chi \rangle}
\end{array}$$

We conclude with the definition for the weak-memory operational semantics for AuDaLa:

Definition 6.2.7 (Weak-Memory Operational Semantics). The weak-memory semantics of $\mathcal{P}$ is the directed graph $\langle \mathcal{G}, \Rightarrow_{\mathcal{P}}, p_{\mathcal{P}}^0 \rangle$, where $\mathcal{G}$ is the set of all states as defined in Def. 6.2.4, $\Rightarrow_{\mathcal{P}}$ is the labelled transition relation for $\mathcal{P}$ as given above and $p_{\mathcal{P}}^0$ is the initial state of $\mathcal{P}$ as defined in Def. 6.2.5.

Differences from the weak-memory semantics as presented in [110]

The weak-memory semantics given in Definition 6.2.7 is different from the weak-memory semantics as given in the paper by Leemrijse *et al.* [110] on which this section is based. The weak-memory semantics by Leemrijse *et al.* has some additional changes which we omitted in the weak-memory semantics given above:

1. It does not support strings.
2. The syntactic element *null* is only defined for struct types and cannot be used to refer to the default values of the other domains.
3. It uses an if-then-else construction, and the **if**-command, interpretation function and the operational rules for the **if**-command are extended to facilitate this.
4. It supports short-circuit evaluation of boolean operators, which leads to a different interpretation function for the boolean operators.

While we think these changes are beneficial, we have decided to leave these out of the weak-memory semantics presented here, to be able to compare it better to the standard semantics. Observe, however, that we can mitigate these differences through transforming AuDaLa programs syntactically.

1. We can encode strings as integers, which we can use to rewrite AuDaLa programs with strings to AuDaLa programs without strings.

2. We can rewrite syntactic instances of *null* not referring to the null-elements of struct types with the concretely defined default values for the other domains.
3. We can transform if-then-else constructs into AuDaLa code without if-then-else constructs using the format given in Listing 6.4.
4. Due to the support for short-circuiting, the weak-memory semantics by Leemrijse *et al.* may skip some expressions which are processed in the weak-memory semantics given above. Though the result value from processing the expression as a whole will not change, this does mean that some side effects from processing subexpressions may be lost. The only side effect that processing a subexpression can have that can be lost in this way is the creation of a new struct instance through a constructor expression. This occurs for example in the statement given in Listing 6.5. However, we can always transform expressions like this to create the struct instance before resolving the expression, for example through transforming the statement in Listing 6.5 to the code as given in Listing 6.6.

```

1 Bool a := /*condition*/;
2 if (a) then { /*if execution*/ }
3 if (!a) then { /*else execution*/ }

```

Listing 6.4: Expressing if-then-else constructs with if-then constructs. Assume the variable *a* is fresh

```

1 x := /*expression*/ && (Element(par1, par2) == this);

```

Listing 6.5: A constructor in a second argument.

```

1 x := /*expression*/;
2 if (x){
3   Element e := Element(par1, par2);
4   x := (e == this);
5 }

```

Listing 6.6: Second argument constructor compatible with the weak-memory semantics as presented in the thesis.

Other than the above changes, the weak-memory semantics by Leemrijse *et al.* has a localized interpretation function which takes the label of the current struct instance as argument and can therefore immediately translate **this** to the right label, where the semantics above has kept the interpretation function global and has the **this**-command and **WMComThis** rule. Though semantically significant, observe that the two semantics are equivalent, due to

the only change in executions being the additional presence of **WMComThis** transitions, which are implicit in the semantics of Leemrijse *et al.*

As this difference is not impactful enough to make a difference in behaviour, and we can mitigate all differences between the weak memory semantics as given above and the weak memory semantics as given in Leemrijse *et al.* syntactically, we consider the differences between the two superficial. We therefore consider the two functionally equivalent.

In the next section, we will prove that the weak-memory semantics as presented above can be considered equivalent to the standard semantics assuming there are no read-write race conditions in the program. This result extends to the semantics as given by Leemrijse *et al.*

6.3 Connecting the Semantics

In this section, we will establish the functional equivalence between the standard semantics and the weak-memory semantics for any AuDaLa program $\mathcal{P}$, *provided there are no read-write race conditions*. More specifically, we prove that for any AuDaLa program $\mathcal{P}$ without read-write race conditions, every execution with the standard semantics can be mirrored by an execution in the weak-memory semantics and vice versa. We consider two executions to mirror each other if they end up in final states that are equivalent to each other in their *contents* (as defined a bit further in the section).

We do not consider AuDaLa programs with read-write race conditions, as the two semantics are not functionally equivalent when read-write race conditions are allowed. As an example, consider the code snippets given in Figure 6.1. Consider an execution of these code snippets by two struct instances s_1 and s_2 such that the parameter r of s_1 points to s_2 and the parameter l of s_2 points to s_1 . There will be a read-write race for both x of s_1 and y from s_2 . As the standard semantics use the sequentially consistent memory model, in the standard semantics, the first lines have to be executed before the second lines. It can therefore not occur in the standard semantics that after the execution of this snippet, $r1 = r2 = 42$. The weak-memory semantics, however, allows for the reordering of the lines of both snippets, so after execution, there is a possibility that $r1 = r2 = 42$. Assuming the worst case, that $r1$ and $r2$ are parameters and that their values after the execution are their final values, the final states of the AuDaLa program executed by both semantics will not be equivalent. It follows that with read-write race conditions, the semantics are not functionally equivalent.

However, in the weak-memory semantics, **wr**-commands resolve to a **rd**-command and a **wrP**-command. We therefore make an exception and do consider read-write race conditions caused by resolving **wr**-commands, as those are not caused by the code but caused by the semantics. These read-write

$$\begin{array}{l|l} r1 := r.y; & r2 := l.x; \\ x = 42; & y = 42; \end{array}$$

Figure 6.1: Load buffering [3]

race conditions only impact the stability stack, by the definition of the weak-semantic rule **Wr-II**; as we prove later in this section that during such a read-write race condition, the resulting stability stack is deterministically updated (Lemma 6.3.16), we can otherwise ignore these read-write race conditions.

To prove the functional equivalence of the two semantics, we will use a form of *stutter trace equivalence*, adapted from Baier and Katoen [12, Definition 7.89], just like in Section 5.3. We again make a comparison between the states traversed during an execution, using a function to extract the relevant contents. In this proof, the comparison is made between states traversed during an execution of $\mathcal{P}$ using the original semantics and the states traversed during an execution of the same $\mathcal{P}$ using the weak-memory semantics.

For this proof, we first adapt Theorem 4.2.21 for the weak-memory semantics in Section 6.3.1. This allows us to streamline reasoning about values resulting from expressions and statements in the rest of the proof. The actual equivalence proof is given in Section 6.3.2.

6.3.1 Extending well-formed properties to the weak-memory semantics

In this subsection, we extend Theorem 4.2.21 to the weak-memory semantics, by adapting the properties and proofs given in Section 4.2 for the weak-memory semantics. This allows us to simplify reasoning about execution results in the equivalence proof given in Section 6.3.2. As an added bonus, this also proves that the type system is sound for the weak-memory semantics.

Recall that Theorem 4.2.21 proves six properties about the original semantics. For the intuitive description of these properties, refer to Section 4.2.

To extend Theorem 4.2.21 to the weak-memory semantics, note that properties 1, 2 and 3 (as given in Section 4.2) can be straightforwardly adapted for the weak-memory semantics, as the adaptations the weak-memory semantics makes to the original semantics do not really change the functionality with regards to handling the schedule. To adapt Property 4 and Property 5, we need to change our approach, as in the weak-memory semantics, commands can be resolved by a struct instance in a non-sequentially consistent order. To that end, we change Property 4 to the following lemma:

Lemma 6.3.8 (Property 4 of well-formed programs for the weak-memory semantics). Let F be a step of some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that

$\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be some state reached during the execution of F in the execution of $\mathcal{P}$. Let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Let S be a statement from F such that $\llbracket S \rrbracket \in \chi$, and let c_j be a command in $\llbracket S \rrbracket = c_1; \dots; c_m$. Then there exists a c_k with $k \leq j$ such that either $c_k; \dots; c_j = \llbracket E \rrbracket$ for some expression E or $c_k; \dots; c_j = \llbracket S \rrbracket$.

Proof. To prove the lemma, we do a case distinction on which command c_j is and choose an appropriate c_k :

1. If $c_j = \text{val}(g)$, it has been generated by the interpretation function in response to either the expression g , with $g \in LT$, or, if $g = \text{defaultVal}(T)$ for some T , to null_T . It follows that with $k = j$, the lemma holds. Analogously, the lemma holds for if $c_j = \text{this}$, which is generated for the expression this .
2. If $c_j = \text{rd}(x_i)$ for some variable x_i , then we know that c_j is generated by the interpretation function in response to some variable expression $x_1 \dots x_n$, with $x_i \in x_1 \dots x_n$. Let c_k be the first command of $\llbracket x_1 \dots x_n \rrbracket$. Then $c_k; \dots; c_j = \llbracket x_1 \dots x_i \rrbracket = \text{this}; \text{rd}(x_1); \dots; \text{rd}(x_i)$.
3. If c_j is a **wr**, **cons**, **if**, **not** or **op** command, it is automatically the last generated command for some expression or statement $\llbracket W \rrbracket = c_a; \dots; c_j$ as per the definition of the interpretation function.

As in all cases, a c_k exists fulfilling the requirements for the lemma, the lemma holds. $\square$

We use this to define the following:

Definition 6.3.9 (Stack Marking). Let F be a step of some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be some state reached during the execution of F in the execution of $\mathcal{P}$. Let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Let S be a statement or expression from F such that $\llbracket S \rrbracket$ is a contiguous subsequence of χ , and let c_j be a command or value in $\llbracket S \rrbracket = c_1; \dots; c_m$. Then if $j = m$, the stack marking of c_j is S . Otherwise, let c_k be the command or value in $\llbracket S \rrbracket$ with the largest $k \leq j$ such that $c_k; \dots; c_j = \llbracket E \rrbracket$ for some expression E contained in S . Then the stack marking of c_j is E .

Furthermore, we assume the stack marking is maintained inductively from the start of the execution of F . If a transition is taken induced by command c_j with a stack marking W , with W an expression or a statement, then any value put on the stack by that transition (if any) also have marking W , with two exceptions:

- If the transition is a **WMComWr-I** transition, we give the **rd**-command generated by the transition the special marker **WRP**. The **wrP**-command will still get mark W .

- If the transition is a **WComIfT** transition where the **if**-command contained a substack $VC = \llbracket S_1; \dots; S_n \rrbracket$ for some n , we mark the commands and values in VC as described above.

We prove the following for the stack marking:

Lemma 6.3.10 (Unique marking). Let F be a step of some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Let $p = \langle \Delta, \sigma, \mu, t, s\chi \rangle$ be some state reached during the execution of F in the execution of $\mathcal{P}$. Let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Let c_j be a command in χ . If c_j is marked with a statement or subexpression W , then c_j is the only command in χ marked with W .

Proof. In the interpretation function as defined in Definition 6.2.2, we define commands to always be the last command of some statement or expression W . The marking will consistently mark commands with the expression they are the last command of, by Definition 6.3.9. (In the case of $x_1, \dots, x_n$, due to the way marking works, $\mathbf{rd}(x_i)$ is marked with $E_i = x_1; \dots; x_i$, and **this** is always marked with **this**.) As we assume that W is uniquely defined by their syntax position, it follows that the lemma holds. $\square$

We use this to adapt Property 5, as given in Lemma 4.2.18. The two difficulties for this are the additional read-write behaviour of the weak-memory semantics, and the fact that the stack and command list have been merged and that the order of execution of commands cannot be assumed in the weak-memory semantics. The first we solve by adapting the case for the **wr**-command in the original semantics for the **wrP** command and making an extra, new case for the **wr**-command. The second we solve by assuming that we mark the stack according to Definition 6.3.9. We can then assert Property 5 for all commands of which their subexpressions consist of a single value. This allows us to keep Property 5 dependent on commands present instead of transitions taken, which preserves the corollary for weak type system soundness.

Note that according to the type system of AuDaLa, every local variable must be declared before it can be used. If we use a local variable x in statement or expression W during a step F , we can pinpoint the *origin statement* S_q of the x in F as the variable assignment statement that originally initializes x in F . In the old Property 5 we were able to pinpoint this statement by looking at the statements already executed for F , but in this adaptation, due to the random order of execution, we cannot be so precise. As in both cases the origin statement still exists, as guaranteed by the type system, we can however still use the notion in the lemma and the proof.

We first give an auxiliary lemma which helps us to define Property 5 more completely:

Lemma 6.3.11 (Choice of c_j). Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be an AuDaLa weak-memory state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg \text{End}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Assume that χ is marked according to an inductively maintained stack marking, as defined in Definition 6.3.9. Then if there is a command in χ , there exists at least one command c in χ such that all of the following hold:

1. the command is marked with **WRP** or the command is the only command of a subexpression or statement W .
2. either the command is not a **rd**, or **wrP** command or a **wr**-command for local variables or, if $\chi = \chi_1; \ell'; c; \chi_2$ with $\ell' \in \mathcal{L}$, $\text{Refs}(\chi_1, \ell', x) = \text{false}$, with x the variable name contained in c .
3. If the command is marked with W , there are no commands marked with any of W 's subexpressions. If $W = x$ for some variable x , $\chi \neq \chi_1; \text{this}; c; \chi_2$.

Proof. We use the definitions from the lemma. Let c_1 be the leftmost command encountered in χ . Then either it is marked with an expression or statement W (uniquely defined by its syntax position), or with **WRP**. If it is marked with W , then by Lemma 6.3.10, we know it is the only command of W . Then as there are no commands left of c_1 , it follows that if it is a **rd**, **wr** command for local variables or **wrP** command and $\chi = \chi_1; \ell'; c; \chi_2$, irrespective of the exact value of ℓ' and x , it must hold that $\text{Refs}(\chi_1, \ell', x) = \text{false}$. For the third point if c_1 is marked with W , due to the order in which commands are generated, any commands marked with subexpressions of W should show up to the left of c_1 . As there are no commands left of c_1 , this satisfies the third point in this case. $\square$

We then define Property 5:

Lemma 6.3.12 (Property 5 of well-formed programs for the weak-memory semantics). Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be an AuDaLa weak-memory state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg \text{End}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Assume that χ is marked according to an inductively maintained stack marking, as defined in Definition 6.3.9. Then for all commands c_j in χ that fulfills the criteria of Lemma 6.3.11, one of the following holds:

1. $c_j = \text{rd}(x)$ and all of the following hold:
 - (a) $\chi = \chi_1; \ell'; \text{rd}(x); \chi_2$ and $\sigma(\ell') = \langle \theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$, and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ then $\ell' = \ell$,
 - (b) $\text{Refs}(\chi_1, \ell', x) = \text{false}$,
 - (c) Either $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$, or the origin statement of x in W has the form $S_q = (T \ x := E)$ such that $T \in \mathcal{T}$ and $E \in EX$.

2. $c_j = \mathbf{wr}(x)$ and all of the following hold:

- (a) $\chi = \chi_1; v; \ell'; \mathbf{wr}(x); \chi_2$ and $\sigma(\ell') = \langle \theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$ and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ then $\ell' = \ell$,
- (b) If $\ell' \notin \mathcal{L}_0$ or $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$, then $\text{Refs}(\chi_1, \ell', x) = \text{false}$,
- (c) Either $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$, or the origin statement of x in W has the form $S_q = (T \ x := E)$ such that $T \in \mathcal{T}$ and $E \in EX$,
- (d) Either $T \in \Theta_{\mathcal{P}}$, $v \in \mathcal{L}$ and $\sigma(v) = \langle T, \chi'', \xi'' \rangle$ (for some χ'' and ξ'') or $T \in \mathcal{T} \setminus ID$ and $v \in \text{sem}(T)$.

3. $c_j = \mathbf{wrP}(x)$ and all of the following hold:

- (a) $\chi = \chi_1; v_o; v_n; \ell'; \mathbf{wr}(x); \chi_2$ and $\sigma(\ell') = \langle \theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ for some $\theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$ and if $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ then $\ell' = \ell$,
- (b) $\text{Refs}(\chi_1, \ell', x) = \text{false}$,
- (c) $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$,
- (d) Either $T \in \Theta_{\mathcal{P}}$, $v_o, v_n \in \mathcal{L}$ and $\sigma(v_n) = \langle T, \chi'', \xi'' \rangle$ (for some χ'' and ξ'') or $T \in \mathcal{T} \setminus ID$ and $v_o, v_n \in \text{sem}(T)$.

4. $c_j = \mathbf{cons}(\theta)$, with $\theta \in \Theta_{\mathcal{P}}$ and $\text{Pars}_{\mathcal{P}}(\theta) = x_1 : T_1; \dots; x_c : T_c$ for some c , and $\chi = \chi_1; v_1; \dots; v_c; \mathbf{cons}(\theta); \chi_2$ and for all $x_h : T_h$, either $T_h \in \mathcal{T} \setminus ID$ and $v_h \in \text{sem}(T_h)$ or $T_h \in \Theta_{\mathcal{P}}$ and $v_h \in \mathcal{L} \wedge \sigma(v_h) = \langle T_h, \chi'', \xi'' \rangle$ for some χ'' and ξ'' .

5. $c_j = \mathbf{if}(\gamma)$ and $\chi = \chi_1; b; \mathbf{if}(\gamma); \chi_2$ such that $b \in \mathbb{B}$.

6. $c_j = \mathbf{not}$ and $\chi = \chi_1; b; \mathbf{not}; \chi_2$ such that $b \in \mathbb{B}$.

7. $c_j = \mathbf{op}(\circ)$ and $\chi = \chi_1; a; b; \mathbf{op}(\circ); \chi_2$ and all of the following hold:

- (a) If $\circ \in \{=, !=\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$ or there exists a $T \in \{\mathbb{N}, \mathbb{Z}, \mathbb{B}, \text{String}\} \cup \Theta_{\mathcal{P}}$ such that either $T \notin \Theta_{\mathcal{P}}$ and $a, b \in T$ or $T \in \Theta_{\mathcal{P}}$ and $\sigma(a) = \langle T, \chi_a, \xi_a \rangle$ and $\sigma(b) = \langle T, \chi_b, \xi_b \rangle$,
- (b) If $\circ \in \{<=, >=, <, >, *, /, \%, +, ^, -\}$ then $a, b \in \{\mathbb{N}, \mathbb{Z}\}$,
- (c) If $\circ \in \{\&\&, ||\}$ then $a, b \in \mathbb{B}$.

Furthermore, there exists a transition induced by c_j from p to a state $p' = \langle \Delta', \sigma', \mu, \iota, s\chi' \rangle$ such that $\sigma'(\ell) = \langle \theta, \chi', \xi' \rangle$ and all of the following hold:

1. If $c_j = \mathbf{this}$, then $\chi' = \chi_1; \ell; \chi_2$.
2. If $c_j = \mathbf{rd}(x)$, with $\chi = \chi_1; \ell'; \mathbf{rd}(x); \chi_2$, one of the following holds:

- (a) $\xi(\ell', x) = \perp$ and $\chi' = \chi_1; \iota(\ell', x); \chi_2$.
- (b) $\xi(\ell', x) \neq \perp$ and $\chi' = \chi_1; \xi(\ell', x); \chi_2$.
- (c) There is a pair $(v, \ell'') \in \mu(\ell', x)$ such that $\ell'' \neq \ell$ and $\chi' = \chi_1; v; \chi_2$.

Additionally, let v be the value put on the stack according to the above cases. Then either $T \in \Theta_{\mathcal{P}}$, $v \in \mathcal{L}$ and $\sigma(v) = \langle T, \chi_x, \xi_x \rangle$ (for some χ_x and ξ_x) or $T \in \mathcal{T} \setminus ID$ and $v \in \text{sem}(T)$.

3. If $c_j = \mathbf{wr}(x)$, then, with $\chi = \chi_1; v; \ell'; \mathbf{wr}(x); \chi_2$, $\sigma'(\ell') = \langle \theta_{\ell'}, \gamma'_{\ell'}, \chi'_{\ell'}, \xi'_{\ell'} \rangle$ such that one of the following holds:
 - (a) $\ell \in \mathcal{L}_0$ and $\chi' = \chi_1; \chi_2$
 - (b) $\ell \notin \mathcal{L}_0$, $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$, $\chi' = \chi_1; \chi_2$ and $\xi' = \xi[(\ell, x) \mapsto v]$
 - (c) $\ell \notin \mathcal{L}_0$, $x \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ and $\chi' = \chi_1; \ell'; \mathbf{rd}(x); v; \ell'; \mathbf{wrP}(x); \chi_2$.
4. If $c_j = \mathbf{wrP}(x)$, then, with $\chi = \chi_1; v_o; v_n; \ell'; \mathbf{wr}(x); \chi_2$, $\sigma'(\ell') = \langle \theta_{\ell'}, \gamma'_{\ell'}, \chi'_{\ell'}, \xi'_{\ell'} \rangle$ such that $\chi' = \chi_1; \chi_2$, $\xi' = \xi[(\ell', x) \mapsto v]$, $\mu' = \mu[(\ell', x) \mapsto \mu(\ell', x) \cup (v_n, \ell)]$ and $s\chi' = s\chi_1 \wedge (v_o = v_n); \dots; s\chi_{|s\chi|} \wedge (v_o = v_n)$.
5. If $c_j = \mathbf{cons}(\theta)$, then, with $\chi = \chi_1; v_1; \dots; v_c; \mathbf{cons}(\theta); \chi_2$ and $\text{Pars}_{\mathcal{P}}(\theta) = x_1 : T_1; \dots; x_c : T_c$, it holds that $\chi = \chi_1; \ell'; \chi_2$ for some ℓ' such that $\sigma(\ell') = \perp$, $\sigma'(\ell') = \langle \theta, \varepsilon, \xi_{\perp} \rangle$, $\ell' = \iota[\{(\ell', x_a) \mapsto v_a \mid 1 \leq a \leq c\}]$ and $s\chi' = \text{false}^{|s\chi|}$.
6. If $c_j = \mathbf{if}(\gamma)$, with $\chi = \chi_1; b; \mathbf{if}(\gamma); \chi_2$, if $b = \text{true}$ then $\chi' = \chi_1; \gamma; \chi_2$ and if $b = \text{false}$, $\chi' = \chi_1; \chi_2$.
7. If $c_j = \mathbf{not}$, with $\chi = \chi_1; b; \mathbf{not}; \chi_2$, $\chi' = \chi_1; \neg b; \chi_2$.
8. If $c_j = \mathbf{op}(\circ)$, with $\chi = \chi_1; a; b; \mathbf{op}(\circ); \chi_2$, then with o the semantic equivalent of $\circ$, $\chi' = \chi_1; c; \chi_2$ for some c such that all of the following hold:
 - (a) If $a \in T$ and $b \in T$ with $T \in \mathbb{T}$ and $\circ \in \{=, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
 - (b) If $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $\circ \in \{<=, >=, <, >, =, !=\}$, then $c = a \circ b$ and $c \in \mathbb{B}$,
 - (c) If $a \in T_1$ and $b \in T_2$, with $T_1, T_2 \in \{\mathbb{N}, \mathbb{Z}\}$ and $\circ \in \{*, /, \%, +, ^, -\}$, then $c = a \circ b$ and $c \in \mathbb{Z}$,
 - (d) If $a, b \in \mathbb{N}$ and $\circ \in \{*, /, \%, +, ^\}$, then $c = a \circ b$ and $c \in \mathbb{N}$.
 - (e) If $a, b \in \mathbb{B}$ and $\circ \in \{\&\&, ||\}$, then $c = a \circ b$ and $c \in \mathbb{B}$.

To prove this, we prove an auxiliary lemma and we adapt the auxiliary lemma Lemma 4.2.19 for our purposes:

Lemma 6.3.13 (Expression/Semantics Type Correspondence in the weak-memory semantics). Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be an AuDaLa weak-memory state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg \text{End}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Assume that χ is marked according to an inductively maintained stack marking, as defined in Definition 6.3.9. Let Property 5 (as defined in Lemma 6.3.12) hold for p , and let c_j be a witness command of that, marked with either expression E , or with **WRP**. If marked with E , let E have type T in $\mathcal{P}$ according to the type system.

If marked with **WRP**, let the variable x read from the struct instance with label ℓ' by c_j have syntactic type T according to the type system. Then there exists a single value v that is put on the stack by the transition induced by c_j . Furthermore, either $T \in \mathcal{T} \setminus ID$ and v have the semantic type $\text{sem}(T)$, or $T \in \Theta_{\mathcal{P}}$ and v will be a label ℓ such that the struct type of $\sigma(\ell)$ is T .

Proof. This follows from a case distinction on the command c_j . As the marking does not come into play in the case that c_j is a read-command, the proof is straightforwardly adapted to the proof of Lemma 4.2.19. $\square$

We then prove Lemma 6.3.12. If a subproof is straightforwardly adaptable from the original proof of Lemma 4.2.18, we will refer to the original subproof of Lemma 4.2.18, with notes on what changes. First, we change the structure of the proof:

Full proof of Property 5 for the weak-memory semantics. Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be an AuDaLa weak-memory state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . If $\neg \text{End}(\sigma)$, let $\ell \in \mathcal{L}$ such that $\sigma(\ell) = \langle \theta, \chi, \xi \rangle$ and $\chi \neq \varepsilon$. Assume that χ is marked according to an inductively maintained stack marking, as defined in Definition 6.3.9.

To prove the lemma, we perform a strong induction on all the states traversed during the execution of $\mathcal{P}$. Let c_j be a command satisfying the conditions of Lemma 6.3.11. In both the base case and the step case, we do a case distinction on the possible commands. For the base case, refer to Subproof 6.3.13.A. For the step case, refer to Subproof 6.3.13.B. As the strong induction holds, Property 5 for the weak-memory semantics holds. $\square$

Subproof 6.3.13.A - Base case for strong induction. In the base case, we assume that p is the first state in which $\neg \text{End}(\sigma)$ holds. Then due to the definition of the interpretation function (Definition 6.2.2), we know that c_j can be any command except for **rd**, **wr** and **wrP** commands, as **wrP** commands can only be introduced by **wr** commands and no commands have been processed yet, and **wr** and **rd** commands must always be preceded by a **this**-command according to the interpretation function, which goes against the condition for c_j that there may not be any commands of any subexpression of W .

For all other commands, we can do a case distinction:

- For **this**, see Subproof 6.3.13.C,
- For **not**, **op**, **if** and **cons**, see Subproof 6.3.13.D.

As all the cases hold, it follows that the base case holds. $\blacksquare$

Subproof 6.3.13.B - Step case for strong induction. In the step case of the second strong induction, we assume as the induction hypothesis that Property 5 holds for all previous states p of the execution of $\mathcal{P}$.

We do a case distinction on the commands c_j can be:

- For **this**, see Subproof 6.3.13.C,
- For **not**, **op**, **if** and **cons**, see Subproof 6.3.13.D.
- For **rd**, see Subproof 6.3.13.E,
- For **wr**, see Subproof 6.3.13.F,
- For **wrP**, see Subproof 6.3.13.G.

As all the cases hold, it follows that the step case for the strong induction over d also holds. ■

Then, we prove the cases again. Most of these are straightforwardly adapted, but the cases for **rd**, **wr** and **wrP** are a bit less straightforward.

*Subproof 6.3.13.C - Case for **this**.* If c_j is **this**, the preconditions are vacuously satisfied. According to the weak-memory semantics, it follows that the transition **WMComThis** is enabled, from which it follows that the postconditions are also satisfied. ■

*Subproof 6.3.13.D - Case **not**, **op**, **if** and **cons**.* Straightforwardly adapted from their respective cases in the proof for Lemma 4.2.18, with Lemma 6.3.13 replacing Lemma 4.2.19.. In the base case we know that all subexpressions must have been literals, as c_j is the first command to be executed. It follows straightforwardly that they have the right stack positions and the right semantic types. In the step case we know that all subexpressions have already been executed as the third condition of Lemma 6.3.11 holds for c_j . This allows us to use the induction hypothesis on all subexpressions. ■

*Subproof 6.3.13.E - Step case **rd** for the weak-memory semantics.* If $c_j = \mathbf{rd}(x)$ for some x , then we know that c_j is marked with an expression E which is either a variable x or with a variable expression $x_1; \dots; x_h.x$. We consider x a special case $x_1; \dots; x_0.x$, where $x_1; \dots; x_0$ is an empty sequence, following the interpretation function. We also know that $\chi \neq \chi_1; \mathbf{this}; \mathbf{rd}(x); \chi_2$, as c_j satisfies the conditions of Lemma 6.3.11. it follows from the interpretation function on $\llbracket E \rrbracket$ and Lemma 6.3.11 that all other commands of E , including the **this**-command, have been processed.

It follows from the induction hypothesis and Lemma 6.3.13 on the commands of $x_1; \dots; x_h$ that $\chi = \chi_1; v; \mathbf{rd}(x); \chi_2$. From the fact that $\mathcal{P}$ is well-formed, we know that $v = \ell'$ for some $\ell' \in \mathcal{L}$.

We also know from the induction hypothesis that Property 5 holds for all previous states, and therefore for all previously processed **wr** and **wrP** commands to variables y during the execution of $\mathcal{P}$, where y has syntactic type

$T_y \in \mathcal{T}$ and v_y is the new value written by the command. Due to that, if $T_y \in \Theta_{\mathcal{P}}$, then $v_y \in \mathcal{L}$ and v_y represents a struct instance with type T_y , and if $T_y \in \mathcal{T} \setminus ID$, then $v_y \in \text{sem}(T_y)$. We also know by the definition of the initial state of $\mathcal{P}$ and the fact that null-parameters cannot be overwritten during the execution of $\mathcal{P}$, that for any null-parameter z of syntactic type $T_z \in \mathcal{T}$, if $T_z \in \Theta_{\mathcal{P}}$ then $v_z = \ell_{T_z}^0$, which represents the null-instance of type T_z , and if $T_z \in \mathcal{T} \setminus ID$, then $v_z = \text{defaultVal}(T_z)$, so $v_z \in \text{sem}(T_z)$.

As the weak-memory semantics does not delete struct instances, it follows that $\sigma(\ell') = \langle \theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ (for some $\theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$) and we also know, with $\theta_{\ell'}$ the struct type of $\sigma(\ell')$, that $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ implies that $\ell' = \ell$, as otherwise $x_1 \dots x_h.x$ would not be admitted by the type rule **VarR**. With this, the first precondition for the **rd** command is satisfied.

The second precondition is satisfied due to c_j satisfying the conditions of Lemma 6.3.11.

As $\mathcal{P}$ is well-formed, we know that there was either an origin statement $S_q = (T \ x := E_x)$ and $E_x \in EX$ or $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$, as otherwise $x_1 \dots x_h.x$ would not be provable in the type system. This satisfies the third precondition for the **rd** command.

These preconditions also satisfy the requirements for at least one of the **WComRd**-transitions, so there exists an enabled transition **WComRdInit**, **WComRdInt** or **WComRdExt** to a state $p' = \langle \Delta', \sigma', \mu', \iota', s\chi' \rangle$ with $\sigma'(\ell) = \langle \theta, \chi', \xi' \rangle$. For all the possible enabled transitions, $\chi' = \chi_1; v; \chi_2$.

Then, we do a case distinction on which transition led to p' . If it was a transition **WComRdInit**, then $\xi(\ell', x) = \perp$, or the transition would not be enabled. Then from the transition, we know that $\chi' = \chi_1; \iota(\ell', x); \chi_2$, which satisfies postcondition 2a.

If it was a transition **WComRdInt**, then $\xi(\ell', x) \neq \perp$, or the transition would not be enabled. Then from the transition, we know that $\chi' = \chi_1; \xi(\ell', x); \chi_2$, which satisfies postcondition 2b.

If it was a transition **WComRdExt**, then we know there exists a pair $(v', \ell'') \in \mu(\ell', x)$ such that $\ell'' \neq \ell$, or the transition would not be enabled. Then from the transition, we know that $\chi' = \chi_1; v'; \chi_2$, which satisfies postcondition 2c.

From the earlier application of the induction hypothesis on all previous write-commands **wr** and **wrP** and the definition of the initial state of $\mathcal{P}$, we also know that if $T \in \Theta_{\mathcal{P}}$, it holds that $v \in \mathcal{L}$ and as AuDaLa does not delete struct instances, it holds that $\sigma(v) = \langle T, \chi_x, \xi_x \rangle$ (for some χ_x, ξ_x). Furthermore, if $T \in \mathcal{T} \setminus ID$, it holds that $v \in \text{sem}(T)$. With this, the last postcondition for the **rd** command is satisfied. ■

*Subproof 6.3.13.F - Step case **wr** for the weak-memory semantics.* If $c_j = \text{wrP}(x)$ for some x , then it must be marked with some update or assignment statement

S , due to the definition of marking and the lemmas associated with it. It follows that S has subexpressions E and $x_1 \dots x_h$ for some h ; from Lemma 6.3.11, we know that all commands of those subexpressions have already been processed.

Then from the induction hypothesis and Lemma 6.3.13 on those subexpressions that $\chi = \chi_1; \nu_1; \nu_2; \mathbf{wr}(x); \chi_2$. From the fact that S is well-formed, we know that $\chi = \chi_1; \nu; \ell'; \mathbf{wr}(x); \chi_2$ for some value ν and label ℓ' .

We also know from the induction hypothesis that Property 5 holds for all previous states, and therefore for all previously processed **wr** and **wrP** commands to variables y during the execution of $\mathcal{P}$, where y has syntactic type $T_y \in \mathcal{T}$ and ν_y is the new value written by the command. Due to that, if $T_y \in \Theta_{\mathcal{P}}$, then $\nu_y \in \mathcal{L}$ and ν_y represents a struct instance with type T_y , and if $T_y \in \mathcal{T} \setminus ID$, then $\nu_y \in \text{sem}(T_y)$. We also know by the definition of the initial state of $\mathcal{P}$ and the fact that null-parameters cannot be overwritten during the execution of $\mathcal{P}$, that for any null-parameter z of syntactic type $T_z \in \mathcal{T}$, if $T_z \in \Theta_{\mathcal{P}}$ then $\nu_z = \ell_{T_z}^0$, which represents the null-instance of type T_z , and if $T_z \in \mathcal{T} \setminus ID$, then $\nu_z = \text{defaultVal}(T_z)$, so $\nu_z \in \text{sem}(T_z)$.

As the weak-memory semantics does not delete struct instances, it follows that $\sigma(\ell') = \langle \theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'} \rangle$ (for some $\theta_{\ell'}, \chi_{\ell'}, \xi_{\ell'}$) and we also know, with $\theta_{\ell'}$ the struct type of $\sigma(\ell')$, that $x \notin \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ implies that $\ell' = \ell$, as otherwise $x_1 \dots x_h \cdot x$ would not be admitted by the type rule **VarR**. With this, the first precondition for the **rd** command is satisfied.

The second property is straightforwardly satisfied by the fact that c_j satisfies the conditions of Lemma 6.3.11.

As $\mathcal{P}$ is well-formed, we know that there was either an origin statement $S_q = (T \ x := E_x)$ and $E_x \in EX$ or $(x : T) \in \text{Pars}_{\mathcal{P}}(\theta_{\ell'})$ for some $T \in \mathcal{T}$, as otherwise $x_1 \dots x_h \cdot x$ would not be provable in the type system. This satisfies the third precondition for the **rd** command.

Then, to establish the fourth precondition, note that by the induction hypothesis for the second strong induction, we know that Property 5 holds for the commands of E' . It follows that ν has the semantic type which reflects E' in p_a , due to Lemma 4.2.19. Furthermore, as AuDaLa does not remove struct instances and keeps struct types of struct instances immutable, we can extend this to p . As $\mathcal{P}$ is well-formed, we also know that E' has the same syntactic type as x . It follows that the third precondition is satisfied.

Then if $\ell' = \ell$ and $x \notin \text{Pars}_{\mathcal{P}}\theta_{\ell'}$, **WMComWrLoc** is enabled, if $\ell' \in \mathcal{L}_0$ and $x \in \text{Pars}_{\mathcal{P}}\theta_{\ell'}$, **WMComWrNSkip** is enabled and that if $\ell' \notin \mathcal{L}_0$ and $x \in \text{Pars}_{\mathcal{P}}\theta_{\ell'}$, **WMComWr-I** is enabled. These conditions cover all possible combinations of what ℓ' is and whether x is a parameter, so at least one transition is enabled.

We do a case distinction on which transition is enabled. Let the state $p' = \langle \Delta', \sigma', \mu', t', s\chi' \rangle$ be the state resulting from that transition, with $\sigma'(\ell) = \langle \theta, \chi', \xi' \rangle$.

Then if the transition was **WMComWrNSkip**, we know from that transition

that $\ell \in \mathcal{L}_0$ and that $\chi' = \chi_1; \chi_2$. If the transition was **WMComWrLoc**, we know that $\ell \notin \mathcal{L}_0$, $x \notin \text{Pars}_{\mathcal{P}}\theta_{\ell'}$, that $\chi' = \chi_1; \chi_2$ and that $\xi' = \xi[(\ell, x) \mapsto v]$. Lastly, if the transition was **WMComWr-I**, we know that $\ell \notin \mathcal{L}_0$, $x \in \text{Pars}_{\mathcal{P}}\theta_{\ell'}$ and that $\chi' = \chi_1; \ell'; \mathbf{rd}(x); v; \ell'; \mathbf{wrP}(x); \chi_2$.

This satisfies this case. ■

*Subproof 6.3.13.G - Step case **wrP** for the weak-memory semantics.* If $c_j = \mathbf{wrP}(x)$ for some x , then it must be marked with some update or assignment statement S , due to the definition of marking and the lemmas associated with it. It must have been generated by a **wr**-command $\mathbf{wr}(x)$ marked with that same statement, from which we know that the resulting commands were $\ell'; \mathbf{rd}(x); v_n; \ell'; \mathbf{wrP}(x)$. By the fact that c_j satisfies the conditions of Lemma 6.3.11, we know that $\mathbf{rd}(x)$ has been processed; by the induction hypothesis we know this results in a value v_o of the same type as v_n . For the preconditions, as all conditions on ℓ' are satisfied by the fact that they were satisfied for the $\mathbf{wr}(x)$ that generated c_j , the first precondition holds. The second precondition holds as c_j satisfies all conditions of Lemma 6.3.11. The third precondition holds, due to the third precondition of the write-command holding for $\mathbf{wr}(x)$ by the induction hypothesis, and the fact that because $\mathbf{wrP}(x)$ was generated, $x \in \text{Pars}_{\mathcal{P}}\theta_{\ell'}$ must be true. The fourth precondition follows from the induction hypothesis on $\mathbf{rd}(x)$ and $\mathbf{wr}(x)$.

Then we have all the preconditions to enable the transition **WMComWr-II**. The result of that transition straightforwardly satisfies the postconditions. ■

With Property 5 proven for the weak-memory semantics, adapting Property 6 is straightforward. The proof reasoning stays the same, except that now every transition either the total number of commands goes down, or the amount of **wr**-commands goes down. We know that we only run out of transitions to take if there are no commands left, as there is always a command for which Property 5 holds, through Lemma 6.3.11. If there are no commands left, **WMFinish** will be enabled, after which the *Done*-predicate will be true.

It follows that the following theorem holds:

Theorem 6.3.14 (Properties of well-formed AuDaLa states in the weak-memory semantics). Let $p = \langle \Delta, \sigma, \mu, \iota, s\chi \rangle$ be an AuDaLa state reached by some well-formed program $\mathcal{P} = D \Delta_{\mathcal{P}}$ such that $\Gamma, \Omega \vdash D \Delta_{\mathcal{P}}$ for some Γ, Ω . Then, for p , the following properties hold:

- Property 1, as straightforwardly adapted for the weak-memory semantics from Lemma 4.2.14,
- Property 2, as straightforwardly adapted for the weak-memory semantics from Lemma 4.2.15,

- Property 3, as straightforwardly adapted for the weak-memory semantics from Lemma 4.2.16,
- Property 4, as specified by Lemma 6.3.8,
- Property 5, as specified by Lemma 6.3.12,
- Property 6, as straightforwardly adapted for the weak-memory semantics from Lemma 4.2.20.

Implications and conclusions

With Theorem 6.3.14, we can straightforwardly adapt the corollaries of Section 4.3. Notably, we know that weak and strong type soundness also hold for the weak-memory semantics, which in the case of the strong type soundness follows from Lemma 6.3.13:

Theorem 6.3.15. The type system of AuDaLa is sound for the weak-memory semantics.

We also know that every expression leads to a single *result value*, just as in the original semantics. If there are no race conditions in the weak-memory semantics of a program $\mathcal{P}$, observe that **WMComRdExt** is never used, as only one struct instance writes to μ , and it cannot read its own values from μ . This allows us to assert that the result values of expressions in $\mathcal{P}$ are deterministic.

To extend this to programs with only write-write race conditions, as discussed at the start of Section 6.3, as **wr**-commands generate both a **rd** and a **wrP** command, any write-write race condition induces a read-write race condition. From the definition of rule **WMComWr-II**, however, that this read-write race condition only has bearing on the stability stack. It follows that expression result values are deterministic in programs without read-write race conditions not induced by a write-write race condition through the resolution of the **wr**-command.

Furthermore, the read-write race condition has no bearing on updating the stability stack in these cases. We prove this by proving the lemma given below, which expresses that the read-write race condition caused by the resolution of the **wr**-command has no bearing on updating the stability stack if the race condition is between two struct instances. The lemma scales straightforwardly to more than two struct instances by considering all struct instances that write pairwise with the struct instance that reads. We therefore omit the lemma for potential read-write conditions caused by resolving a **wr** command which involve more than two struct instances.

Lemma 6.3.16 (Stack Resets are deterministic in the weak-memory semantics (2 struct instances)). Let $\mathcal{P}$ be an AuDaLa program without read-write race conditions not induced by a write-write race condition through the resolution

of the **wr**-command. Let s_1 and s_2 be two struct instances in a write-write race condition for a parameter x owned by a struct instance with label $\ell' \notin \mathcal{L}_0$ with old value v_o . Let s_1 write v_1 to x and s_2 write v_2 to x . Then after both writes have been completed, the stack is reset to $false^{|\mathcal{S}\chi|}$ if $v_1 \neq v_o$ or if $v_2 \neq v_o$.

Proof. Assume that $v_1 \neq v_o$ or $v_2 \neq v_o$. As x is a parameter of a non-null instance, we know that the writes will happen through the transitions **WMComWr-I** followed by **WMComWr-II**, by Theorem 6.3.14. For both s_1 and s_2 , the write will be resolved by resolving the substack $\ell'; \mathbf{rd}(x); v_n; \ell'; \mathbf{wr}(x)$

Let without loss of generality $v_1 \neq v_o$. Then either s_1 reads the old v_o first or s_2 does. If s_1 reads the value first, it will update the stack to $false^{|\mathcal{S}\chi|}$ in **Wr-II**, as $v_1 \neq v_o$ and **Wr-II** is atomic, no matter what s' does to the stack meanwhile. As the stack reset cannot be undone by s_2 by the definition of **Wr-II**, the lemma holds.

If s_2 reads the value first, then either $v_2 = v_o$ or $v_2 \neq v_o$. If $v_2 \neq v_o$, so s_2 will reset the stack, which cannot be undone during the write by s_1 , so the lemma holds. If $v_2 = v_o$, then it will not update the stability stack. However, as the value of x after the write of s_2 is still v_o and s_1 does read that value for its own write (either before or after the write by s_2), s_1 will reset the stability stack and the lemma holds. $\square$

It follows that for AuDaLa programs without read-write race conditions not induced by a write-write race condition through the resolution of the **wr**-command processed using the weak-memory semantics:

- We can find deterministic result values for the expressions executed in $\mathcal{P}$.
- The execution results of the statements are the same as in Corollary 4.3.28
- Corollary 4.3.29 can be applied to find permanent update result over steps.

In the next section, we use this to prove the weak-memory semantics stuttering equivalent for AuDaLa programs without read-write race conditions not induced by a write-write race condition through the resolution of the **wr**-command.

6.3.2 Proving the semantics stutter trace equivalent

In this subsection, we prove that the weak-memory semantics are stuttering equivalent to the original semantics if there are no read-write race conditions other than those induced by a write-write race condition through the resolution of the **wr**-command. For brevity, we make the assumption on the data

paces explicit for all the following lemmas and proofs:

Race Condition Assumption. We assume that there are no read-write race conditions not induced by a write-write race condition through the resolution of the **wr**-command in the programs we consider.

In the proof, we consider the names of the derivation rules which induce transitions in both semantics to be the labels of those transitions. We use the terms SC-state and WM-state to refer to states from the standard (sequentially consistent) semantics and the weak-memory semantics respectively. We use the subscripts *SC* and *WM* to distinguish the elements of both semantics which use the same notation where necessary.

We extend the notion of an *idle* state as used in Section 5.3 to also encompass WM-states:

Definition 6.3.17 (Idle States). We define any SC-state $p = \langle \Delta_p, \sigma_p, s\chi_p \rangle$ to be *idle* iff $Done_{SC}(\sigma_p) = true$, with $Done_{SC}$ the predicate *Done* as defined in the standard semantics. We define any WM-state $q = \langle \Delta_q, \sigma_q, \mu_q, \iota_q, s\chi_q \rangle$ to be *idle* iff $Done_{WM}(\sigma_q, \mu_q) = true$, with $Done_{WM}$ the proposition *Done* as defined in the weak-memory semantics. We define *busy* states as states that are not idle.

Then, we define the *contents* of both struct instances and states.

Definition 6.3.18 (Contents of struct instances and states). Let the state $p_1 = \langle \Delta_1, \sigma_1, s\chi_1 \rangle$ be a SC-state. Then for any struct instance with label $\ell_1 \in \mathcal{L}$ such that $\sigma_1(\ell_1) = \langle \theta_1, \gamma_1, \chi_1, \xi_1 \rangle$, let $z_{\ell_1} : ID \rightarrow \mathcal{V} \cup \{\perp\}$ be the function defined as

$$z_{\ell_1}(x) = \begin{cases} \xi_1(x) & \text{if } x \in Pars_{\mathcal{P}}(\theta_1) \\ \perp & \text{otherwise.} \end{cases}$$

We define the *contents* Con_{ℓ} of ℓ as the triple $(\ell_1, \theta_1, z_{\ell_1})$

Let $p_2 = \langle \Delta_2, \sigma_2, \mu_2, \iota_2, s\chi_2 \rangle$ be an WM-state. Then for any struct instance with label ℓ_2 such that $\sigma_2(\ell_2) = \langle \theta_2, \chi_2, \xi_2 \rangle$, let $z_{\ell_2} : ID \rightarrow \mathcal{V} \cup \{\perp\}$ be the function defined as

$$z_{\ell_2}(x) = \begin{cases} \iota_2(\ell_2, x) & \text{if } x \in Pars_{\mathcal{P}}(\theta_2) \\ \perp & \text{otherwise.} \end{cases}$$

We define the *contents* Con_{ℓ_2} of ℓ_2 as the triple $(\ell_2, \theta_2, z_{\ell_2})$.

For any SC-state or WM-state p with struct environment σ_p we define the *contents function* $Z_p : \mathcal{L} \rightarrow (\mathcal{L} \times ID \times (ID \rightarrow \mathcal{V} \cup \{\perp\})) \cup \{\perp\}$ for any struct instance of p with label ℓ as $Z_p(\ell) = Con_{\ell}$. We define $Z_p(\ell) = \perp$ for all other labels.

Intuitively, this extracts information on which struct instances exist and what their parameter values are from both the SC-states and the WM-states.

For WM-states, however, the values given for the parameters are the values at the start of the step.

We use this to define our version of atomic propositions for this proof, using a *state labelling function*:

Definition 6.3.19 (State labelling function). We define *contents labelling function* $\mathcal{C}$. This function takes a SC- or WM-state as input, and returns either a double containing the schedule and the *busy symbol* β if the state is a busy state, or it returns a triple containing the schedule, the contents of the state and the stability stack. It is formally defined for any SC-state or WM-state p with schedule Δ_p and stability stack $s\chi_p$ as:

$$\mathcal{C}(p) = \begin{cases} (\Delta_p, \beta) & \text{if } p \text{ is busy} \\ (\Delta_p, Z_p, s\chi_p) & \text{otherwise} \end{cases}$$

We adapt the definitions of *stutter trace equivalence* by Baier and Katoen [12, Definition 7.89] to apply them to the standard and weak-memory semantics. To do this, we give our definitions of paths and prefixes, stutter equivalence for prefixes and stutter prefix equivalence relative to semantics. Note that the definitions for paths and prefixes and for stutter equivalence for paths are restated from Section 5.3.

Definition 5.3.22 (Paths and Prefixes). Let $G = (Q, \rightarrow, I)$ be an operational semantics with state space Q , transition relation $\rightarrow$ and an initial state I . A *path* is a sequence $\varrho = q_1, q_2, \dots$ such that $q_1 \rightarrow q_2 \rightarrow \dots$ is valid in G . If $q_1 = I$, ϱ is a *prefix*.

This follows the common definition of paths, which we repeat due to the importance of paths to the proof. Note that a prefix is essentially an initial path, but prefix fits better with the constructive nature of the proofs. Furthermore, we assume we can label the transitions due to the transition relation being labelled.

We then define *stutter-equivalence for paths* and *stutter prefix equivalence for semantics*, adapted from Baier and Katoen [12], definitions 7.86 and 7.89.

Definition 5.3.23 (Stutter equivalence for paths). Let L be a set of state labels, let $f : Q \rightarrow L$ be a labelling function from a set of states Q to L , and let, with a label $l \in L$, the sequence l^d for some $d > 0$ be a sequence of d instances of l .

Two paths $\varrho_1 = A_1, \dots$ and $\varrho_2 = B_1, \dots$ over a set of states Q such that $A_1, \dots, B_1, \dots \in Q$ are *stutter-equivalent* with regards to f , or *f-stutter-equivalent* if applying f on the states of ϱ_1 results in a sequence $L_1^{a_1} L_2^{a_2} \dots$ with $a_1, a_2, \dots > 0$ and applying f on the states of ϱ_2 results in a sequence $L_1^{b_1} L_2^{b_2} \dots$ with $b_1, b_2, \dots > 0$, for some $L_1, L_2, \dots \in L$.

Definition 6.3.20 (Stutter prefix equivalence for semantics). Let $\langle \cdot \rangle_1$ and $\langle \cdot \rangle_2$ be two semantics of languages G_1 and G_2 respectively, let $\mathcal{Z}$ be a set of programs such that all $\mathcal{P} \in \mathcal{Z}$ are valid in both G_1 and G_2 . Let Q_{G_1} be all possible states of any program in $\mathcal{Z}$ in $\langle \cdot \rangle_1$, let Q_{G_2} be all possible states of any program in $\mathcal{Z}$ in $\langle \cdot \rangle_2$ and let L be a set of labels. Let $f : Q_{G_1} \cup Q_{G_2} \rightarrow L$ be a labelling function. Then $\langle \cdot \rangle_1$ and $\langle \cdot \rangle_2$ are *stutter prefix equivalent on f for $\mathcal{Z}$* iff for all programs $\mathcal{P} \in \mathcal{Z}$, for all prefixes ρ_1 in $\langle \mathcal{P} \rangle_1$, a prefix ρ_2 exists in $\langle \mathcal{P} \rangle_2$ such that ρ_1 and ρ_2 are f -stutter-equivalent, and vice versa.

Of course, for us, $G_1 = G_2 = \text{AuDaLa}$, just with different semantics. We continue with the main theorem we want to prove in this section:

Theorem 6.3.21 ($\mathcal{C}$ -SPEq). Let $\langle \cdot \rangle_{SC}$ be the standard semantics and let $\langle \cdot \rangle_{WM}$ be the weak-memory semantics. Then $\langle \cdot \rangle_{SC}$ and $\langle \cdot \rangle_{WM}$ are stutter prefix equivalent on $\mathcal{C}$ ($\mathcal{C}$ -SPEq) for all AuDaLa programs without read-write race conditions other than those induced by **wr**-commands in the weak-memory semantics.

To prove this, we fix an arbitrary program $\mathcal{P}$ without read-write race conditions other than those induced by **wr**-commands in the weak-memory semantics. Then, as outlined by Definition 6.3.20, we prove that for all prefixes ρ_1 possible for $\mathcal{P}$ in the sequentially consistent semantics, there exists a prefix $\mathcal{C}$ -stutter-equivalent ρ_2 for $\mathcal{P}$ in the weak-memory semantics and vice versa. Note that the traces are implicit in this approach; they are the result of applying the labelling function $\mathcal{C}$ to all states in a path.

We split up this proof in lemmas, which we combine together in a final proof of the theorem. We fix $\langle \cdot \rangle_{SC}$ and $\langle \cdot \rangle_{WM}$ to be the original semantics and the weak-memory semantics respectively. Let $\langle \mathcal{P} \rangle_{SC}$ and $\langle \mathcal{P} \rangle_{WM}$ be the semantic interpretations of $\mathcal{P}$ in the original semantics and the weak-memory semantics respectively.

We begin by proving that the initial states are $\mathcal{C}$ -stutter-equivalent. For this, it is important that the two semantics use the same labels for $\mathcal{L}_0$. If they do not, the contents for the null-instances will never match, which is a requirement for prefixes to be $\mathcal{C}$ -stutter equivalent.

Lemma 6.3.22 (Initial state prefixes are $\mathcal{C}$ -SPEq). Let $(P_{\mathcal{P}}^0)_{SC}$ and $(P_{\mathcal{P}}^0)_{WM}$ be the initial states of $\langle \mathcal{P} \rangle_{SC}$ and $\langle \mathcal{P} \rangle_{WM}$, and assume that $\langle \mathcal{P} \rangle_{SC}$ and $\langle \mathcal{P} \rangle_{WM}$ use the same set $\mathcal{L}_0$ for the null-labels. Then $\mathcal{C}((P_{\mathcal{P}}^0)_{SC}) = \mathcal{C}((P_{\mathcal{P}}^0)_{WM})$.

Proof. Let $(P_{\mathcal{P}}^0)_{SC}$ be the initial state of $\langle \mathcal{P} \rangle_{SC}$ and let $(P_{\mathcal{P}}^0)_{WM}$ be the initial state of $\langle \mathcal{P} \rangle_{WM}$. We prove that $\mathcal{C}((P_{\mathcal{P}}^0)_{SC}) = \mathcal{C}((P_{\mathcal{P}}^0)_{WM})$. Let $\mathcal{L}_0^{SC}$ be the null-labels used by $\langle \mathcal{P} \rangle_{SC}$ and let $\mathcal{L}_0^{WM}$ be the null-labels used by $\langle \mathcal{P} \rangle_{WM}$.

As per Definition 6.2.5, the initial state $(P_{\mathcal{P}}^0)_{WM}$ is defined as:

$$\begin{aligned} (P_{\mathcal{P}}^0)_{WM} &= \langle \Delta_{\mathcal{P}}, \sigma_{\mathcal{P}}^0, \mu_{\emptyset}, l_{\mathcal{P}}^0, \varepsilon \rangle \\ &\text{with} \\ \sigma_{\mathcal{P}}^0 &= \sigma_{\perp} [\{\ell_{\theta}^0 \mapsto \langle \theta, \varepsilon, \xi_{\perp} \mid \theta \in \Theta_{\mathcal{P}} \rangle\}] \\ l_{\mathcal{P}}^0 &= l_{\perp} [\{\langle \ell_{\theta}^0, x \rangle \mapsto \text{defaultVal}(T) \mid \theta \in \Theta_{\mathcal{P}}, x : T \in \text{Pars}_{\mathcal{P}}(\theta) \}] \end{aligned}$$

for all $\ell_{\sigma}^0 \in \mathcal{L}_0^{WM}$.

Then, following Definition 6.3.18, the function $Z_{(P_{\mathcal{P}}^0)_{WM}}$ for the initial state is defined as

$$Z_{(P_{\mathcal{P}}^0)_{WM}}(\ell) = \begin{cases} (\ell, \theta, z_{\perp} [\{x \mapsto \text{defaultVal}(T) \mid x : T \in \text{Pars}_{\mathcal{P}}(\ell)\}]) & \text{if } \ell \in \mathcal{L}_0^{WM} \wedge \ell = \ell_{\theta}^0, \\ \perp & \text{otherwise} \end{cases},$$

with $z_{\perp}$ an empty function which returns $\perp$ for all values $x \in ID$. Then, the full label of $(P_{\mathcal{P}}^0)_{WM}$ is $(\Delta_{\mathcal{P}}, Z_{(P_{\mathcal{P}}^0)_{WM}}, \varepsilon)$, as the initial state is idle, by virtue of χ being empty for all null-instances in it and as $\mu = \mu_{\emptyset}$.

Paraphrased from Definition 2.4.9, the initial state $(P_{\mathcal{P}}^0)_{SC}$ is defined as:

$$\begin{aligned} (P_{\mathcal{P}}^0)_{SC} &= \langle \Delta_{\mathcal{P}}, \sigma_{\mathcal{P}}^0, \varepsilon \rangle \\ &\text{with} \\ \sigma_{\mathcal{P}}^0 &= \sigma_{\perp} [\{\ell_{\theta}^0 \mapsto \langle \theta, \varepsilon, \xi_{\theta}^0 \mid \theta \in \Theta \rangle\}] \\ &\text{with} \\ \xi_{\theta}^0 &= \xi_{\perp} [\{x \mapsto \text{defaultVal}(T) \mid x : T \in \text{Pars}_{\mathcal{P}}(\theta)\}] \end{aligned}$$

for all $\ell_{\sigma}^0 \in \mathcal{L}_0^{SC}$.

Then, following Definition 6.3.18, the function $Z_{(P_{\mathcal{P}}^0)_{SC}}$ for the initial state is defined as

$$Z_{(P_{\mathcal{P}}^0)_{SC}}(\ell) = \begin{cases} (\ell, \theta, z_{\perp} [\{x \mapsto \text{defaultVal}(T) \mid x : T \in \text{Pars}_{\mathcal{P}}(\theta)\}]) & \text{if } \ell \in \mathcal{L}_0^{SC} \wedge \ell = \ell_{\theta}^0, \\ \perp & \text{otherwise} \end{cases},$$

with $z_{\perp}$ an empty function which returns $\perp$ for all values $x \in ID$. In this case, the full label of $(P_{\mathcal{P}}^0)_{SC}$ is $(\Delta_{\mathcal{P}}, Z_{(P_{\mathcal{P}}^0)_{SC}}, \varepsilon)$, as the initial state is idle.

As $\mathcal{L}_0^{SC} = \mathcal{L}_0^{WM}$ by the assumption in the lemma, it follows by construction that $Z_{(P_{\mathcal{P}}^0)_{WM}} = Z_{(P_{\mathcal{P}}^0)_{SC}}$. $\square$

We prove that we can extend any two $\mathcal{C}$ -SPEq prefixes with traces consisting only of busy states, and the extended prefixes will once again be $\mathcal{C}$ -SPEq.

Lemma 6.3.23 (*C-SPEq of prefixes to a busy state*). Let ϱ_{SC} be a prefix to an idle state p_{SC} in $(\mathcal{P})_{SC}$ and let ϱ_{WM} be a prefix to an idle state p_{WM} in $(\mathcal{P})_{WM}$ such that ϱ_{SC} and ϱ_{WM} are *C-stutter-equivalent*. Then for any paths ϱ_{SC} and ϱ_{WM} in $(\mathcal{P})_{SC}$ and $(\mathcal{P})_{WM}$ respectively such that both only contain busy states and $\varrho_{SC};\varrho_{SC}$ and $\varrho_{WM};\varrho_{WM}$ are prefixes, $\varrho_{SC};\varrho_{SC}$ and $\varrho_{WM};\varrho_{WM}$ are *C-stutter-equivalent*.

Proof. To prove that $\varrho_{SC};\varrho_{SC}$ and $\varrho_{WM};\varrho_{WM}$ are *C-stutter-equivalent*, note that for all states p_1 in ϱ_{SC} , $\mathcal{C}(p_1) = (\Delta_{SC}, \beta)$ and for all states p_2 in ϱ_{WM} , $\mathcal{C}(p_2) = (\Delta_{WM}, \beta)$, by definition of *C* and the fact that no transitions from busy to busy states change the schedule. As ϱ_{SC} and ϱ_{WM} are already *C-stutter-equivalent*, it suffices to prove that $\Delta_{SC} = \Delta_{WM}$. To prove this, we know that ϱ_{SC} and ϱ_{WM} end in an idle state with the same schedule, and that the transition from P_{SC} and P_{WM} to the first state of ϱ_{SC} and ϱ_{WM} is a transition to a busy state. Then we know that this transition must be either **InitG** or **InitL** for P_{SC} and either **WMInitG** or **WMInitL** for P_{WM} . In both cases, the schedule of both P_{SC} and P_{WM} must be of the format $F < \Delta'$, where F is a step. As ϱ_{SC} and ϱ_{WM} are *C-stutter-equivalent*, we know that P_{SC} and P_{WM} have the same schedule, so fix the schedule to be $F < \Delta'$. Then the resulting schedule of any of the enabled transitions is the schedule Δ' . Therefore, $\Delta_{SC} = \Delta' = \Delta_{WM}$. $\square$

Next, we prove that transitions from idle to idle states preserve *C-SPEq*:

Lemma 6.3.24 (*C-SPEq preserved for idle-idle transitions*). Let ϱ_{SC} be a prefix to an idle state p_{SC} in $(\mathcal{P})_{SC}$ and let ϱ_{WM} be a prefix to an idle state p_{WM} in $(\mathcal{P})_{WM}$ such that ϱ_{SC} and ϱ_{WM} are *C-stutter-equivalent*. Then for all prefixes $\varrho_{SC}; p'_{SC}$ such that p'_{SC} is an idle F-state there exists a *C-stutter-equivalent* prefix $\varrho_{WM}; p'_{WM}$ such that p'_{WM} is an idle WM-state and vice versa.

Proof. To prove the lemma, note that the transition from P_{SC} to p'_{SC} and the transition from P_{WM} to p'_{WM} must both be transitions from idle states to idle states. There are three transitions that can be taken in both semantics from idle to idle state:

- **FixInit** in the standard semantics, which is mirrored by **WMFixInit** in the weak-memory semantics.
- **FixIter** in the standard semantics, which is mirrored by **Fix-Iter** in the weak-memory semantics.
- **FixTerm** in the standard semantics, which is mirrored by **Fix-Resolve** in the weak-memory semantics.

All three transition pairs change the schedule and stability stack in the same way and do not change the contents functions, from which it follows that any

prefix $\varrho_{SC}; p'_{SC}$ such that p'_{SC} is an idle state a prefix $\varrho_{WM}; p'_{WM}$ exists such that p'_{WM} is an idle state and $\mathcal{C}(p'_{SC}) = \mathcal{C}(p'_{WM})$. From this the lemma follows. $\square$

The following lemmas and definitions have the goal to prove that if an SC-state p_{SC} and a WM-state p_{WM} with $\mathcal{C}$ -stutter-equivalent prefixes ϱ_{SC} and ϱ_{WM} both execute a step F , for all resulting prefixes $\varrho'_{SC}; \varrho'_{SC}$ where ϱ'_{SC} is a path of an execution of F from p_{SC} , there exists a $\mathcal{C}$ -stutter-equivalent prefix $\varrho'_{WM}; \varrho'_{WM}$, where ϱ'_{WM} is a path of an execution of F from p_{WM} . See also Lemma 6.3.30.

We first define the notion of *corresponding* states and struct instances. Intuitively, a pair of states or a pair of struct instances is corresponding iff the contents of both elements of the pair is the same.

Definition 6.3.25 (Corresponding states). Let p_{SC} be an idle SC-state with struct environment σ_{SC} and contents function $Z_{p_{SC}}$ and let p_{WM} be an idle WM-state with struct environment σ_{WM} and contents function $Z_{p_{WM}}$ such that $\mathcal{C}(p_{SC}) = \mathcal{C}(p_{WM})$. Then p_{SC} and p_{WM} are *corresponding* states.

Lemma 6.3.26 (Correspondence of struct instances). Let p_{SC} be an idle SC-state with struct environment σ_{SC} and contents function $Z_{p_{SC}}$ and let p_{WM} be a corresponding idle WM-state with struct environment σ_{WM} and contents function $Z_{p_{WM}}$. Then for any label $\ell \in \mathcal{L}$, $\sigma_{SC}(\ell) \neq \perp$ iff $\sigma_{WM}(\ell) \neq \perp$ and $Z_{p_{SC}}(\ell) = Z_{p_{WM}}(\ell)$.

Proof. Follows directly from Definition 6.3.18. $\square$

It follows that for all corresponding states p_{SC} and p_{WM} , we can pair up the struct instances into *corresponding* struct instances:

Definition 6.3.27 (Corresponding struct instances). Let SC-state p_{SC} and WM-state p_{WM} be two corresponding states, with the struct environments of the states being σ_{SC} and σ_{WM} respectively. Then for any $\ell \in \mathcal{L}$ s.t. $\sigma_{SC}(\ell) \neq \perp$, the struct instances $s_{SC} = \sigma_{SC}(\ell)$ and $s_{WM} = \sigma_{WM}(\ell)$ are *corresponding* struct instances.

We then define *step execution paths*:

Definition 6.3.28 (Step Execution Path). Let $\varrho = p_1, p'_1, \dots, p_2$ be a path of a program $\mathcal{P}$ from idle state p_1 to idle state p_2 such that p_1 and p_2 are the only idle states in ϱ and p'_1 is a busy state reached from p_1 through a transition t_1 . Then the path from p'_1 to p_2 is a *step path of step F* , where F is the step removed from the schedule in t_1 .

Note that in this definition, p_1 is idle and p'_1 is busy, so t_1 must be one of the transitions **InitL**, **InitG**, **WMinL** or **WMinG**, all of which remove a step F from the schedule. It follows that F is always defined.

We continue by connecting the step resolutions along the two semantics, using the adaptation of Theorem 4.2.21 for the original semantics into Theorem 6.3.14 for the weak-memory semantics, as given in Section 6.3.1. This is done by proving the following lemma:

Lemma 6.3.29 (Step Resolution). Let p_{SC} be an idle F-state with struct environment σ_{SC} and let p_{WM} be an idle WM-state with struct environment σ_{WM} such that $\mathcal{C}(p_{SC}) = \mathcal{C}(p_{WM}) = (\Delta, Z, s\chi)$. Let ϱ_{SC} and ϱ_{WM} be step paths of step F in $(\mathcal{P})_{SC}$ and $(\mathcal{P})_{WM}$ respectively, such that $p_{SC};\varrho_{SC}$ and $p_{WM};\varrho_{WM}$ are also paths in $(\mathcal{P})_{SC}$ and $(\mathcal{P})_{WM}$. Let ℓ be a label of struct instances $s_{SC} = \sigma_{SC}(\ell)$ and $s_{WM} = \sigma_{WM}(\ell)$. Then for all statements S in F for s_{SC} and s_{WM} :

1. The stability stack is reset before S is fully resolved by s_{WM} iff s_{SC} resets the stability stack during the resolution of S .
2. if S is an if-statement, the statements contained in S are executed by s_{SC} iff they are executed by s_{WM} .
3. s_{WM} writes a value val to a variable v of a struct instance with label ℓ' during the resolution of S iff s_{SC} writes val to v of ℓ' during the resolution of S .
4. For all constructor statements or expressions W included in S , let s'_{SC} be the struct instance created for W by s_{SC} and let s'_{WM} be the struct instance created for W by s_{WM} . Let ℓ_{SC} be the label of s'_{SC} and let ℓ_{WM} be the label of s'_{WM} . If $\ell_{SC} = \ell_{WM}$, then $Con_{s'_{SC}} = Con_{s'_{WM}}$.

Proof. Due to Theorem 6.3.14, we know the preconditions and the postconditions of all transitions taken in the execution of $\mathcal{P}$. Furthermore, we have established in Section 6.3.1 that the result values of expressions executed in a program without read-write race conditions not induced by processing a **wr** command are deterministic and that the stability stack updates are also deterministic and that statement results are also deterministic. (Note that we use deterministic as defined in the thesis here, which means that the newly introduced labels are ignored.) Due to the *Refs*-predicate, statements are executed sequentially consistently with the statements they depend on. It follows that without loss of generality we can assume that s_{WM} executed its commands sequentially consistently in ϱ_{WM} .

This allows us to disregard the requirements on the *Refs* predicate of Property 5 as defined for the weak-memory semantics. All other preconditions and postconditions defined in Property 5 for the weak-memory semantics (Lemma 6.3.12) have an equivalent property in Property 5 as defined for the original semantics (Lemma 4.2.18). For most preconditions and postconditions this is straightforward, with three exceptions:

- The pre and postconditions of the **wr**-commands and **wrP**-commands

combined are the weak-memory equivalent to the pre and postconditions of the **wr**-commands in the original semantics.

- In the weak semantics, Property 5 only asserts the type of a value read by a **rd**-command as a postcondition, due to the fact that the environment read from depends on the transition. However, as the type of a value read depends on the type the value has in the environment it is read from directly, this is equivalent to the precondition on the type of the value read in Property 5 as asserted for the original semantics.

From this it follows that if s_{SC} and s_{WM} are from states with equivalent contents and execute the same statement S in the same step F , all of the requirements of the lemma hold, as we can directly match the transitions taken by s_{SC} with those taken by s_{WM} according to the principles outlined above. It follows that the lemma holds. $\square$

In the proof, note that we cannot assert that s_{SC} and s_{WM} use the same labels for newly created struct instances, as the semantics leaves the choice of which labels used arbitrary. Assuming that they do use the same labels is therefore necessary to conclude that $Con_{s'_{SC}} = Con_{s'_{WM}}$.

Then, we prove that if two idle states in both semantics are $\mathcal{C}$ -stutter-equivalent, then after a step execution the resulting idle states are also $\mathcal{C}$ -stutter equivalent.

Lemma 6.3.30 (Extending $\mathcal{C}$ -stutter-equivalence over step paths). Let ϱ_{SC} be a prefix to an idle state p_{SC} in $(\mathcal{P})_{SC}$ and let ϱ_{WM} be a prefix to an idle state p_{WM} in $(\mathcal{P})_{WM}$ such that ϱ_{SC} and ϱ_{WM} are $\mathcal{C}$ -stutter-equivalent. Then any prefix $\varrho_{SC};\varrho'_{SC}$ such that ϱ'_{SC} is a step path for step F exists if there exists a $\mathcal{C}$ -stutter-equivalent prefix $\varrho_{WM};\varrho'_{WM}$ such that ϱ'_{WM} is a step path for step F and vice versa.

Proof. Let p'_{SC} be the idle state at the end of ϱ'_{SC} and let p'_{WM} be the idle state at the end of ϱ'_{WM} . Let ϱ''_{SC} and ϱ''_{WM} be the paths ϱ'_{SC} and ϱ'_{WM} but with the states p'_{SC} and p'_{WM} removed. As proven in Lemma 6.3.23, $\varrho_{SC};\varrho''_{SC}$ and $\varrho_{WM};\varrho''_{WM}$ are already $\mathcal{C}$ -stutter-equivalent.

Therefore, proving the lemma comes down to proving that any prefix $\varrho_{SC};\varrho'_{SC}$ where ϱ'_{SC} is a step path for step F ending at p'_{SC} exists iff there exists a prefix $\varrho_{WM};\varrho'_{WM}$ where ϱ'_{WM} is a step path for step F ending at p'_{WM} such that $\mathcal{C}(p'_{SC}) = \mathcal{C}(p'_{WM})$. We prove this by proving that for ϱ'_{SC} , we can specify ϱ'_{WM} ; the reverse is analogous. For this proof, note that $\mathcal{C}(p_{SC}) = \mathcal{C}(p_{WM})$.

By the definition of ϱ'_{SC} and the hypothetical ϱ'_{WM} , we know that both are reached by calling F from the schedule. As no transitions from busy states change the schedule any further, we know that p'_{SC} and p'_{WM} share the same schedule no matter which ϱ'_{WM} we choose such that $\varrho_{WM};\varrho'_{WM}$ is a prefix.

Let s_{SC} be a struct instance present in p_{SC} . From Lemma 6.3.26, it follows that s_{SC} corresponds to a struct instance s_{WM} in p_{WM} , and that both are present all through ρ'_{SC} and any ρ'_{WM} chosen. As the labels s_{SC} introduces during ρ'_{SC} are arbitrary, there is always a step path ρ'_{WM} in $(\mathcal{P})_{WM}$ such that $\rho_{WM};\rho'_{WM}$ is a prefix and s_{WM} introduces the same labels for the same constructors, so we choose ρ'_{WM} such that s_{SC} and s_{WM} introduce the same labels for the same constructors. Then it follows from point 2 of Lemma 6.3.29 that during the resolution of F , s_{WM} and s_{SC} follow the same execution flow and execute the same statements. It follows from the other points of Lemma 6.3.29 that s_{SC} creates struct instances if s_{WM} creates the same struct instances, s_{SC} writes a value to a variable if s_{WM} writes the same value to the same variable and s_{SC} updates the stability stack if s_{WM} updates the stability stack, and vice versa. We therefore choose ρ'_{WM} such that all corresponding struct instances s_{SC} and s_{WM} introduce the same labels for the same constructors.

With this choice, for any corresponding struct instances s_{SC} and s_{WM} , when resolving a statement, s_{SC} writes v to x iff s_{WM} writes v to x for any variable x . Due to this, if there are no race conditions in F , x will reflect the write from s_{SC} in $Z_{p'_{SC}}$ iff x reflects the write from s_{WM} in $Z_{p'_{WM}}$. It follows that in this case, for this ρ'_{WM} , $Z_{p'_{SC}} = Z_{p'_{WM}}$.

If there is a write-write race condition, this does not necessarily need to hold. However, among the possible choices for an ρ'_{WM} such that all corresponding struct instances s_{SC} and s_{WM} introduce the same labels at the same places for any ρ'_{SC} , there always exists one such that ρ'_{SC} and ρ'_{WM} resolve the race conditions the same way. This follows from the fact that race condition resolution is independent from the step execution, so all possible resolutions will have their respective paths. Due to Lemma 6.3.16, we know that the stability stack will be reset in ρ'_{SC} iff it is reset in ρ'_{WM} . Then, when choosing that ρ'_{WM} , even with write-write race conditions and read-write race conditions induced by the write-write conditions, $Z_{p'_{SC}} = Z_{p'_{WM}}$.

Therefore, p'_{SC} and p'_{WM} have the same schedule, contents and stability stack. It follows that $\mathcal{C}(p'_{SC}) = \mathcal{C}(p'_{WM})$. Therefore, the lemma holds. $\square$

From this, we conclude the following theorem:

Theorem 6.3.21 ($\mathcal{C}$ -SPEq). Let $(\cdot)_{SC}$ be the standard semantics and let $(\cdot)_{WM}$ be the weak-memory semantics. Then $(\cdot)_{SC}$ and $(\cdot)_{WM}$ are stutter prefix equivalent on $\mathcal{C}$ ($\mathcal{C}$ -SPEq) for all AuDaLa programs without read-write race conditions other than those induced by **wr**-commands in the weak-memory semantics.

Proof. We prove the theorem by proving that a prefix ρ_{SC} exists in $(\mathcal{P})_{SC}$ if a $\mathcal{C}$ -stutter-equivalent prefix ρ_{WM} exists in $(\mathcal{P})_{WM}$ and vice versa. We do this via induction on the number of idle states i in ρ_{SC} and ρ_{WM} .

- $i = 1$: In this case, ϱ_{SC} and ϱ_{WM} consist of only an initial state (which is an idle state). As the set $\mathcal{L}_0$ is not specified for the initial states, we can fix $\mathcal{L}_0$ to be the same set for both initial states. Then, from Lemma 6.3.22, we know that the initial states have the same label in $\mathcal{C}$. It follows that ϱ_{SC} and ϱ_{WM} are $\mathcal{C}$ -stutter-equivalent.
- $i > 1$: By induction hypothesis, let ϱ_{SC} and ϱ_{WM} be $\mathcal{C}$ -stutter-equivalent. Let p_{SC} and p_{WM} be the last idle states of ϱ_{SC} and ϱ_{WM} respectively. Then we do a case distinction on the next transition taken by p_{SC} and p_{WM} :
- If the next transition of p_{SC} (analogously p_{WM}) is a transition to an idle state p'_{SC} , then the resulting prefix $\varrho_{SC}; p'_{SC}$ can be mirrored by some prefix $\varrho_{WM}; p'_{WM}$, as proven in Lemma 6.3.24.
 - If the next transition of p_{SC} (analogously p_{WM}) is a transition to a busy state, then any prefixes $\varrho_{SC}; pr'_{SC}$ where ϱ'_{SC} ends in a busy state exists iff a $\mathcal{C}$ -stutter equivalent prefix $\varrho_{WM}; \varrho'_{WM}$ exists where ϱ'_{WM} ends in a busy state as proven in Lemma 6.3.23. As this is a step case for an induction on idle states, we can split up paths with multiple idle states in it. We then only need to prove that any prefixes $\varrho_{SC}; \varrho'_{SC}$ where ϱ'_{SC} is a step path exist iff a $\mathcal{C}$ -stutter equivalent prefix $\varrho_{WM}; \varrho'_{WM}$ exists where ϱ'_{WM} is a step path. This follows from Lemma 6.3.30.

As the induction holds, a prefix ϱ_{SC} exists in $(\langle \mathcal{P} \rangle)_{SC}$ if a $\mathcal{C}$ -stutter-equivalent prefix ϱ_{WM} exists in $(\langle \mathcal{P} \rangle)_{WM}$ and vice versa. This means that the theorem holds. $\square$

With this, we can conclude the following:

Corollary 6.3.31. The original semantics and the weak-memory semantics are functionally equivalent as long as the program executed does not contain read-write race conditions other than those induced by the resolution of **wr**-commands in the weak-memory semantics.

6.4 Conclusion

In this section, we have given several adaptations of the AuDaLa semantics, including one more extensive adaptation to make the semantics more compatible with conventional hardware. We have proven this last, weak-memory semantics to be functionally equivalent to the original semantics as given in Section 2.4.

Sections 6.2 and 6.3 are based on our contributions to the paper by Leemrijse *et al.* [110]. The paper goes beyond the definition of their weak-memory

semantics and also gives an axiomatisation of the memory model of the weak-memory semantics, which is proven to be a correct axiomatisation encoding all possible sequences of transitions that may occur during the execution of an AuDaLa program. This axiomatisation is then compared with the axiomatic model PTX, which encodes the possible executions in CUDA. Tests indicate that the two memory models are compatible, making AuDaLa using the weak-memory model compatible with PTX, as long as no “out of thin air” behaviour (nonsensical behaviour that does not translate to actual executions) occurs.

The weak-memory semantics and the proof of $\mathcal{C}$ -SPEq of the original and the weak-memory semantics imply that to some functional extent, we can decouple theoretical language design, focused on verification and simplicity, from practical language design, focused on performance and hardware implementation, without sacrificing too much of either. Though it takes a bit more work, one could use the method used above to create an entire web of functionally equivalent semantics, all focused on specific hardware or theoretic properties. By using the web of semantics, we can separate the analysis of AuDaLa and its programs for different properties more easily while keeping the language the same.

Part II

AuDaLogic

Chapter 7

Defining AuDaLogic

In Part I, we have created AuDaLa, the Autonomous Data Language and explored it. Notably, we have defined some properties on well-formed AuDaLa programs in Chapter 4, and proven some example programs correct in Chapter 5. These proofs are lengthy and require precision in how the program is approached, but they also demonstrate that there is a structure to proving AuDaLa programs correct.

In this chapter, we develop verification of AuDaLa further by formalising AuDaLa proofs with a *proof system*, AuDaLogic (Autonomous Data Logic), to prove AuDaLa programs correct. This proof system gives us a more elegant framework to consider AuDaLa programs in, which also does a lot of heavy lifting for us, like considering how to decompose AuDaLa programs and how the available resources should be represented. It is based on *concurrent separation logic* [29, 31, 143] (CSL) with permissions, with as starting point the work by Vafeiadis [173] on proving proof systems using CSL sound.

We chose CSL with permissions as a starting point due to its capability to split resources between processes until there is only one process to be considered locally, which we found important to be able to do in an AuDaLa system with decentralized struct instances. Conventional CSL mostly considers these resources to be memory resources, but the logic also supports other interpretations of splitting the state, which we use to split the active struct instances as well. This leads to an elegant logic where one can continuously keep splitting the state until only one struct instance and its resources are left. Here, the struct instance can specifically be considered to execute steps separately from all other struct instances, like in the proofs of Chapter 5. This is extremely useful for finding the effects of steps.

There are differences between conventional CSL and AuDaLogic, however. First of all, most applications of CSL consider memory resources to be passive

and moved around between processes. In AuDaLa, however, every process is instead owned by a memory resource, and manipulates the links between memory resources. We have bridged this conceptual gap in AuDaLogic using a fictional *memory heap* connected to the AuDaLa semantics, to be able to consider memory resources similarly to conventional CSL.

Furthermore, AuDaLa execution and the more process-focused execution assumed for more conventional CSL do not quite match. As in AuDaLa there are no directly related threads or processes, but decentralized struct instances, we have added another construct to our adaptation of CSL, the *label triple*. This construct specifies which struct instances we are currently considering to be active. Like the memory heap, we have connected the label triple to the AuDaLa semantics. Additionally, as the relations between struct instances in AuDaLa are far more important to the functioning of the program than for conventional programming languages, we have also decided to utilize higher order logic in AuDaLogic. This allows us to for example specify that there exists a set of struct instances which form a list, which can be a vital property to prove the correctness of a list initialization method in AuDaLa.

Though AuDaLogic bridges the differences between the assumptions of conventional CSL and AuDaLogic reasonably well, there are still some differences which would benefit from more study. For example, AuDaLogic does not use *resource invariants*, as AuDaLa does not use critical sections and locks, for which these resource invariants were introduced [30, 146]. This also means that AuDaLogic does not need a notion of precision for the conjunction rule of AuDaLogic [79, 173]. However, AuDaLa and AuDaLogic could benefit from critical sections and locks and what that would look like without sacrificing the design principles of AuDaLa. This we leave to future work.

Despite the complications of adapting CSL for AuDaLa, we consider the current version of AuDaLogic to be surprisingly elegant. Most notably, it has a natural split between a proof on the schedule level and a proof on the statement level, mirroring Chapter 5.

In Section 7.1 we first discuss the relevant preliminaries. We continue in Section 7.2 with the definition of the syntax and semantics of our proof system, AuDaLogic. We define some methods for translating AuDaLa to AuDaLogic in Section 7.3 and we give the proof rules for AuDaLogic in Section 7.4. Lastly, we give an example proof tree proving linear time sorting (as discussed in Section 3.4.1) in Section 7.5 and we conclude in Section 7.6.

7.1 Preliminaries - Proving Programs with Triples

7.1.1 Hoare logic

One of the most famous and well-known formal systems for proving programs correct is Hoare logic [7, 60, 91], which uses *Hoare triples* to reason about the effects of executing code. Hoare triples have the form $\{P\}\mathcal{X}\{Q\}$, where P is a precondition, $\mathcal{X}$ is a program or a part of a program to be executed, and Q is a postcondition. The Hoare triple $\{P\}\mathcal{X}\{Q\}$ holds if and only if executing the code $\mathcal{X}$ from a state which satisfies the precondition P always leads to a state in which the postcondition Q holds upon termination of $\mathcal{X}$.

As an example, if our code $\mathcal{X}$ consists of the statement $x := y + 1$, and our precondition P is defined as $y = 1 \wedge x = 5$, and we can assume that $\mathcal{X}$ will be executed without intervention from other processes, we can express that with the Hoare triple $\{y = 1 \wedge x = 5\}x := y + 1\{y = 1 \wedge x = 2\}$ for the execution of $\mathcal{X}$ from any state satisfying P .

The strength of Hoare logic is that it defines *axioms* and *proof rules* to establish the correctness of Hoare triples. Axioms are basic assumptions assumed to always hold. Proof rules specify a number of premises and a conclusion such that if the premises hold, the conclusion must also hold. We denote proof rules using *inference rule notation*, where if the premises of the inference above the line hold, the conclusion below the line also holds. Axioms are given as inference rules without premises, which intuitively state that no premises have to hold for the conclusion to be true. For example, consider the following axiom, established by Hoare (taken from [7]):

$$\text{(ASSIGNMENT)} \frac{}{\{P[x \mapsto t]\}x := t\{P\}}$$

This axiom states that if we have a postcondition P from the execution of the code $x := t$, then we must have started in a state where the precondition formed by replacing all instances of x by t in P holds. This allows us to directly establish the truth of some Hoare triples, for example the following Hoare triple:

$$\{5 \leq 10\}x := 5\{x \leq 10\}$$

A second example of a proof rule is the following rule, also established by Hoare ([7]):

$$\text{(CONSEQUENCE)} \frac{P \Rightarrow P_1 \quad \{P_1\}\mathcal{X}\{Q_1\} \quad Q_1 \Rightarrow Q}{\{P\}\mathcal{X}\{Q\}}$$

This proof rule states that if we execute $\mathcal{X}$ from a state satisfying P , then if P implies P_1 , if the Hoare triple $\{P_1\}\mathcal{X}\{Q_1\}$ holds and if Q_1 implies Q , then the

Hoare triple $\{P\}\mathcal{X}\{Q\}$ also holds. Using this proof rule, we can establish that even more Hoare triples hold. As an example, consider the Hoare triple given below:

$$\{x \leq 8\}x := x + 5\{x \leq 15\}$$

The correctness of this triple follows from the fact that $x \leq 8 \Rightarrow (x + 5) \leq 13$, and $(x + 5) \leq 13 \Rightarrow (x + 5) \leq 15$. It follows that with the CONSEQUENCE and ASSIGNMENT rules, we can prove this Hoare triple to be correct.

Deduction proofs using Hoare proof rules can be shown as proof trees. For example, to prove $\{x \leq 10\}x := x + 5\{x \leq 15\}$, we can create the following proof tree, where A stands for an application of ASSIGNMENT, and C stands for an application of CONSEQUENCE.

$$\frac{x \leq 8 \Rightarrow (x + 5) \leq 15 \quad \frac{\{(x + 5) \leq 15\}x := x + 5\{x \leq 15\}}{x \leq 8 \Rightarrow (x + 5) \leq 15} A \quad x \leq 15 \Rightarrow x \leq 15}{\{x \leq 8\}x := x + 5\{x \leq 15\}} C$$

So far, we have assumed that we can just apply our proof rules on the code given, but in reality, these proof rules have to be *sound* for the programming language that is being considered before they can be applied. A *proof system*, or a set of proof rules, is sound for a language if the conclusions based on the premises hold with regards to the semantics of the language. Intuitively, this means that anything we prove using Hoare logic is actually true for the language we are using.

For example, if in our language $x := 5$ actually updates x to 11, the axiom ASSIGNMENT does not hold for our language and we cannot derive the conclusion that $\{5 \leq 10\}x := 5\{x \leq 10\}$ using that axiom.

Hoare logic is widely applied, but for programs with pointers the method fell short and became hard to use [28]. To solve that, Hoare logic was extended to *separation logic* [141, 159].

7.1.2 Separation logic

Data is stored in the memory of a computer during program execution, and indexed using memory addresses. Many programming languages work with *pointers*, which allow the program to access the memory address defined by some expression to retrieve the value therein directly.

Let for the context of this section the notation of a pointer in some simple programming language be $[\cdot]$. As an example, if we write $[x] := 5$, we do not overwrite the value of the variable x with 5, but the memory at the address x .

This can lead to problems with Hoare logic, as we cannot straightforwardly apply the assignment axiom above on pointer programs. This is because if we have the program $[x] := 5; [y] := x + 1$, we do not know the values of x and y ,

so we do not know if the value of $[x]$ after these statements will be 5 or 6. To conclusively prove something about $[x]$ and $[y]$ after the execution of our program, we need to take into account that x and y may point to the same address, which is called *aliasing*.

This problem grows in complexity with the complexity of the program. While it is possible to reason about programs like this in Hoare logic [28], separation logic presents an elegant way to deal with this problem.

The memory heap and the separator

Separation logic adds a model of the memory to Hoare logic, the *memory heap*. Generally, this is a function of memory addresses to values. Furthermore, separation logic adds variable environment to the logic, which is a function from variables to their values. To reason a bit more concretely about the memory, Separation logic uses *memory assertions* of the form $x \mapsto a$ to assert that the memory heap consists of a single memory cell with the address saved in variable x , which contains the value a .

Hoare logic generally uses assertions with the standard logical operators, like $\wedge, \vee, \neg$ and $\Rightarrow$. In separation logic, there are two additional logical operators, the *separating conjunction* $*$ and the *separating implication* $\multimap$. These are used to reason about the memory heap. We generally shorten “separating conjunction” to just “separator”.

For the predicates P and Q , the predicate $P * Q$ can be intuitively explained as *P and separately Q*, in the sense that no memory locations used in P and Q overlap. If P and Q are *pure*, which is the case when no memory locations are used in both P and Q , $P * Q$ is equivalent to $P \wedge Q$. In a sense, by using $P * Q$, the memory heap is split in two, with one part being available for the predicate P and one for the predicate Q .

As an example, $x \mapsto a * y \mapsto b$ states that x and y are distinct memory cells, where address x has value a and address y has value b . If this predicate holds, there will be no aliasing between x and y , due to the separator.

The predicate $P \multimap Q$ states that, with a current memory heap b , if we add the memory heap a satisfying P to b , then the full memory heap consisting of both a and b satisfies Q . If both P and Q are pure, then $P \multimap Q$ is equivalent to $P \Rightarrow Q$.

As an example of a use for the separating implication, $(x \mapsto a) * ((x \mapsto 3) \multimap Q)$ (adapted from [141]) states that we have a memory heap containing a cell x which has value a , and furthermore that if $a = 3$, Q holds.

In separation logic, there is also an additional constant **emp**, which is true iff the memory heap is empty. The relation between **emp** is comparable to the relation between *true* and logical conjunction. For example, $P \wedge \text{true} \Leftrightarrow P$, and $P * \text{emp} \Leftrightarrow P$.

Note that $P * \text{true} \not\Rightarrow P$. For example, $x \mapsto a$ states that the memory heap should consist of a single memory cell with address x and value a , where $x \mapsto a * \text{true}$ only states that the aforementioned memory cell should be present in the heap. This is because we can split the heap into that memory cell and a heap which satisfies true , for all heaps satisfy true .

Furthermore, just like $P \wedge \text{false} \Leftrightarrow \text{false}$, $P * \text{false} \Leftrightarrow \text{false}$, as we will never be able to split the heap in such a way that it satisfies false . For the relation between **emp** and $*$, this is not as simple, as **emp** $\rightarrow P \Leftrightarrow P$ and $P * \text{emp}$ is false if P is not pure, as by adding a non-empty heap to the current heap, the resulting heap will not satisfy **emp**.

By definition of $x \mapsto a$, $x \mapsto a \wedge y \mapsto b$ is false unless $x = y$ and $a = b$, as the assertion requires that only a single memory cell is in the memory heap. To be able to just reason about the contents of the heap without putting requirements on the concrete structure, separation logic often uses a weakened memory assertion which we call *memory inclusion*, denoted as $x \hookrightarrow a$, to assert that the memory cell x is in the heap and has value a . The memory inclusion assertion is formally defined as $x \hookrightarrow a * \text{true}$, so we can split the heap such that one part of the heap is the single cell x which has value a .

Memory inclusions are more flexible and work better with logical conjunction. For example, $x \hookrightarrow a \wedge y \hookrightarrow b$ is true if there exist both a memory cell with address x and value a and a memory cell with address y and value b in your memory heap, regardless of whether these are the same cell or not.

As memory assertions $x \mapsto a$ essentially contain both the statement that the memory contains a single memory cell with address x and that the value of this cell is a , Separation logic also often uses the shorthand $x \mapsto -$, which is short for $\exists a. x \mapsto a$. It follows that as memory cells have values by definition, $x \mapsto -$ only asserts that the memory heap is a single memory cell with address x .

The frame rule and concurrent separation logic

Due to the definition of the separator and the memory heap, proof systems using separation logic have a *frame rule*, which generally has the following shape:

$$\text{(FRAME RULE)} \frac{\{P\} \mathcal{X} \{Q\} \quad \mathcal{X} \text{ does not modify anything in } R}{\{P * R\} \mathcal{X} \{Q * R\}}$$

This rule allows the proof system to discard parts of the memory context which are not relevant for the current piece of code $\mathcal{X}$, which can greatly simplify the pre- and postcondition and therefore the proof of $\mathcal{X}$.

As an example, the frame rule can be used as follows [141]:

$$\frac{\{y \mapsto 0\}[y] := x\{y \mapsto x\}}{\{y \mapsto 0 * x \mapsto a\}[y] := x\{y \mapsto x * x \mapsto a\}} \text{ FRAME RULE}$$

This is particularly useful in one of the major applications of separation logic, the verification of concurrent programs. When used in this way, the proof system is usually referred to as *concurrent separation logic* [29, 31, 143] (CSL). In CSL, the frame rule can be adapted to split concurrently executed processes to consider them on their own. Consider the program $\mathcal{X}_1 \parallel \mathcal{X}_2$. We can then establish the following *concurrency rule*:

$$\text{(CONCURRENCY RULE)} \frac{\begin{array}{ll} \{P_1\}\mathcal{X}_1\{Q_1\} & \mathcal{X}_1 \text{ does not modify anything in } P_2 \\ \{P_2\}\mathcal{X}_2\{Q_2\} & \mathcal{X}_2 \text{ does not modify anything in } P_1 \end{array}}{\{P_1 * P_2\}\mathcal{X}_1 \parallel \mathcal{X}_2\{Q_1 * Q_2\}}$$

This then allows us to consider $\mathcal{X}_1$ and $\mathcal{X}_2$ separately. For example:

$$\frac{\{y \mapsto 0\}[y] := x\{y \mapsto x\} \quad \{x \mapsto a\}[x] := y\{x \mapsto y\}}{\{y \mapsto 0 * x \mapsto a\}[y] := x \parallel [x] := y\{y \mapsto x * x \mapsto y\}} \text{ CONCURRENCY RULE}$$

One drawback of using CSL to verify concurrent programs is that it requires a clean and total split of the memory. While this prevents race conditions, it also prevents different processes to read from the same resources, even if they do not otherwise change the resources. To at least allow processes to share some read-only resources, a common extension to CSL is to add permissions to CSL [173].

With permissions, a memory assertion has the form $x \xrightarrow{\zeta} a$, where ζ is the permission within a range $(0, 1]$ (so excluding 0). CSL then allows for writes to x iff $\zeta = 1$. The separator is adapted such that $x \xrightarrow{\zeta} a \Leftrightarrow x \xrightarrow{\zeta_1} a * x \xrightarrow{\zeta_2} a$ is true iff $\zeta_1 + \zeta_2 = \zeta$ and $\zeta \leq 1$. This way, when the memory heap is split, some resources can be present in the heap at both sides of the split, but only to read from.

This admits for example the following use of the concurrency rule, adapted for permissions.

$$\frac{\{y \xrightarrow{1} 0 * z \xrightarrow{\frac{1}{2}} b\}[y] := z\{y \xrightarrow{1} z * z \xrightarrow{\frac{1}{2}} b\} \quad \{x \xrightarrow{1} a * z \xrightarrow{\frac{1}{2}} b\}[x] := z\{x \xrightarrow{1} z * z \xrightarrow{\frac{1}{2}} b\}}{\{y \xrightarrow{1} 0 * x \xrightarrow{1} a * z \xrightarrow{1} b\}[y] := z \parallel [x] := z\{y \xrightarrow{1} z * x \xrightarrow{1} z * z \xrightarrow{1} b\}}$$

We found CSL with permissions to be a good starting point for creating a proof system for AuDaLa, due to the possibility to consider processes, and therefore struct instances, separately from the other struct instances, as the processes are considered separately when applying the concurrency rule. In the following sections, we adapt and extend CSL extended with permissions to be a fully fledged proof system for AuDaLa. To do this, we add another

model alongside the memory model, which tracks the currently considered elements, and give our own proof rules which we prove sound with regards to the semantics of AuDaLa.

7.2 Defining AuDaLogic Expressions and Predicates

In this section, we define the syntax and semantics of AuDaLogic. AuDaLogic expands upon CSL by separating not only the memory heap, but also the struct instances of AuDaLa currently considered active through the use of a *label triple*. For this, it expands the CSL memory heap to a full AuDaLogic state, and it extends the functionality of the separator and separating implication to be able to split both the memory heap and this label triple.

To define AuDaLogic, we first define some general sets and functions. We then define the AuDaLogic states. With this, we define ways to merge the AuDaLogic state, an ordering on the memory heap, as well as the syntax and semantics of AuDaLogic expressions. After that, we give the syntax and semantics of AuDaLogic predicates, which we then use to define some shorthands and an activation relation between predicates.

General sets and functions

Like the semantics of AuDaLa, the semantics of AuDaLogic are defined relative to an AuDaLa program $\mathcal{P}$. We use the following sets and functions which contain information extracted from $\mathcal{P}$ or from the definition of AuDaLa with regards to $\mathcal{P}$. We define *simple* expressions to be expressions in which no constructor expression occurs.

- The set $\Theta_{\mathcal{P}}$, which contains all struct identifiers of $\mathcal{P}$.
- The set ID_S , which contains all step names of $\mathcal{P}$.
- The set ID_V , which contains all variable names of $\mathcal{P}$.
- The set $\mathcal{L}$, which contains all labels in $\mathcal{P}$.
- The set $\mathcal{L}_0$, such that $\mathcal{L}_0 \subseteq \mathcal{L}$, which contains all null-labels in $\mathcal{P}$.
- The set ST , the set of all possible statements of AuDaLa.
- The set EX , the set of all possible expressions of AuDaLa.
- The set EX_C such that $EX_C \subseteq EX$, which contains all constructor expressions of AuDaLa.
- The set EX_S such that $EX_S \subseteq EX$, which contains all *simple* expressions of AuDaLa

- The set $\mathcal{T}$ of syntactic types of $\mathcal{P}$.
- The set $\mathbb{T}$ of semantic types of AuDaLa.
- The set $\mathcal{V}$, which contains all semantic values of AuDaLa.
- The function $StepStats_{\mathcal{P}} : ID_S \times \Theta_{\mathcal{P}} \rightarrow ST^*$, defined such that for some step F and some $\theta \in \Theta_{\mathcal{P}}$, $StepStats_{\mathcal{P}}(F, \theta)$ returns the possibly empty list of statements of the step F as defined in the struct with type θ in $\mathcal{P}$.
- The function $Pars_{\mathcal{P}} : \mathcal{L} \cup \Theta_{\mathcal{P}} \rightarrow ID_P^*$, defined such that for some input $\theta \in \Theta_{\mathcal{P}}$, $Pars_{\mathcal{P}}(\theta)$ returns the parameters of θ as defined in $\mathcal{P}$, and for some input $\ell \in \mathcal{L}$, such that the struct instance of ℓ has the struct type θ_{ℓ} , $Pars_{\mathcal{P}}(\ell)$ returns the parameters of θ_{ℓ} .

We define the functions $dom(f)$ to give the domain of a function f and we define that $def(f(a)) = true$ iff $a \in dom(f)$. We extend this to binary operators, such that $def(a \circ b) = true$ iff the binary operator $\circ$ is defined for a and b . We also define the set of all AuDaLogic variables as $\mathcal{A}$, with a subset $\mathcal{L} \subset \mathcal{A}$ containing AuDaLogic label variables. Lastly, we define a set of *permissions* $Z = (0, 1] \cup \{\zeta_0\}$, with ζ_0 a special *null-permission*, which we will use to encode null-parameters from the AuDaLa semantics in AuDaLogic.

The AuDaLogic state

In AuDaLogic, we use predicates that are either true or false for the AuDaLogic state, the state which reflects the current state of the AuDaLa execution. The AuDaLogic state is defined as follows:

Definition 7.2.1 (AuDaLogic State). An *AuDaLogic state* is a triple (α, h, ς) , where

1. $\alpha \subseteq (\mathcal{L} \times \Theta_{\mathcal{P}}) \times (\mathcal{L} \times \Theta_{\mathcal{P}}) \times \mathcal{L}$ is the *label triple*, with the first set containing the active labels and their struct types, the second set containing the newly created labels and their struct types and the third set containing unused labels. We denote the first set as $\alpha(1) \subseteq \mathcal{L} \times \Theta_{\mathcal{P}}$, the second set as $\alpha(2) \subseteq \mathcal{L} \times \Theta_{\mathcal{P}}$ and the third set as $\alpha(3) \subseteq \mathcal{L}$. For any label triple α , it holds that α is defined iff $\nexists \ell. \exists 1 \leq i < j \leq 3. \ell \in \alpha(i) \wedge \ell \in \alpha(j)$.
2. $h : (\mathcal{L} \times ID_V) \rightarrow Z \times \mathcal{V}$ is the *data heap*, which takes a label and an AuDaLa variable name, and returns the permission and the value of the variable as owned by the struct instance of the label.
3. $\varsigma : \mathcal{A} \rightarrow \Theta_{\mathcal{P}} \cup Z \cup ID_V \cup \mathcal{V} \cup \bigcup_n 2^{\mathcal{V}^n}$ is the *AuDaLogic variable environment* containing the context for our predicates. It takes an AuDaLogic variable and returns a struct type, variable name, permission, value or values relation (first or higher order).

Generally, we use “AuDaLogic environment” as a shortening for the AuDaLogic variable environment. Additionally, we assume that $\alpha(1)$, $\alpha(2)$ and h are *finite*, as we are working with finite programs $\mathcal{P}$, and finite programs imply finite amounts of active struct instances and parameters to be in the heap. In contrast, we assume $\alpha(3)$ to be infinite.

Example 7.2.2. As an example state, consider the state (α_e, h_e, ζ_e) , which we shall use to illustrate some of the following definitions of this section. For this state, let $\alpha_e = (\{(\ell_1, \theta_1), (\ell_2, \theta_2), (\ell_3, \theta_3)\}, \{(\ell_4, \theta_2)\}, \alpha_e(3))$, such that there exist two labels $\ell_a \notin \alpha_e(3)$ and $\ell_b \in \alpha_e(3)$. Intuitively, this means we have three active elements for executing steps, one new element (not yet executing steps) and a set of unused labels which includes ℓ_b but not ℓ_a .

Furthermore, let h_e be the heap defined as

$$\begin{aligned} h_e(\ell_1, \text{var1}) &= (0.5, 6) & h_e(\ell_1, \text{var6}) &= (1, 42) \\ h_e(\ell_2, \text{var2}) &= (1, \ell_3) & h_e(\ell_2, \text{var4}) &= (0.125, \ell_1) \\ h_e(\ell_4, \text{var4}) &= (1, \text{true}) \end{aligned}$$

and undefined otherwise. Intuitively, that means we have permissions for reading the *var1* variable of ℓ_1 and the *var4* variable of ℓ_2 , as well as full permissions for the other three variables in the heap.

Lastly, let ζ_e be a AuDaLogic environment in which $\zeta_e(a_1) = \ell_1$, $\zeta_e(L) = \{\ell_1, \ell_2\}$, and $\zeta_e(a_2) = \text{true}$. Intuitively, this denotes that we have some AuDaLogic variables which are relevant in our context; a_1 refers to ℓ_1 , L represents a set $\{\ell_1, \ell_2\}$, and a_2 representing *true*.

Merging label triples, heaps and states

Merging (and splitting) memory is a distinguishing feature of separation logic and CSL. In AuDaLogic, we extend this to the label triple and to the AuDaLogic state as a whole, which we will use for the semantic definition of the separator in AuDaLogic, later in this section. To do this, we will first define ways to merge label triples and heaps, which we then extend to the states.

First of all, to work with the label triple α , we define that $\ell \in \alpha(i)$ with $i \in \{1, 2\}$ iff $\exists \theta \in \Theta_{\mathcal{P}}. (\ell, \theta) \in \alpha(i)$. We define that $\ell \in \alpha$ iff $\ell \in \alpha(i)$ for some $i \in \{1, 2, 3\}$. We then define a merge operator for label triples:

Definition 7.2.3 (Merging label triples). We define a merge operator $\diamond$ for our label triples such that $\text{def}(\alpha_1 \diamond \alpha_2)$ iff $\nexists \ell \in \mathcal{L}. \ell \in \alpha_1 \wedge \ell \in \alpha_2$. If $\text{def}(\alpha_1 \diamond \alpha_2)$, $\alpha_1 \diamond \alpha_2 = (\alpha_1(1) \cup \alpha_2(1), \alpha_1(2) \cup \alpha_2(2), \alpha_1(3) \cup \alpha_2(3))$.

Secondly, for our memory heap, we use permissions in much the same way as they are used by Vafeiadis [173], except for the addition of ζ_0 .

Definition 7.2.4 (Merging permissions and heaps). We define $\oplus$ as addition within the range $(0, 1]$, extended for ζ_0 such that $\zeta_0 \oplus \zeta_0 = \zeta_0$ and $\forall \zeta_1 \in (0, 1). \zeta_0 \oplus \zeta_1 = \zeta_1 \oplus \zeta_0 = \zeta_1$.

We extend this for permission/value pairs as follows:

$$(\zeta_1, v_1) \oplus (\zeta_2, v_2) = \begin{cases} (\zeta_1 \oplus \zeta_2, v_1) & \text{if } v_1 = v_2 \wedge \text{def}(\zeta_1 \oplus \zeta_2) \\ \text{undefined} & \text{otherwise.} \end{cases}$$

We further extend the operator for heaps, and define that $h_1 \oplus h_2$ is defined iff $h_1(a) \oplus h_2(a)$ is defined for all $a \in \text{dom}(h_1) \cap \text{dom}(h_2)$. Then $h_1 \oplus h_2$ is defined with $\text{dom}(h_1 \oplus h_2) = \text{dom}(h_1) \cup \text{dom}(h_2)$ and for all $a \in \text{dom}(h_1 \oplus h_2)$:

$$(h_1 \oplus h_2)(a) = \begin{cases} h_1(a) \oplus h_2(a) & \text{if } a \in (\text{dom}(h_1) \cap \text{dom}(h_2)) \\ h_1(a) & \text{if } a \in (\text{dom}(h_1) \setminus \text{dom}(h_2)) \\ h_2(a) & \text{if } a \in (\text{dom}(h_2) \setminus \text{dom}(h_1)). \end{cases}$$

We use the definitions of label triple merges and heap merges to define the full AuDaLogic state merge:

Definition 7.2.5 (AuDaLogic state merge). We define the binary operator $\star$ for AuDaLogic states such that $\text{def}((\alpha_1, h_1, \varsigma) \star (\alpha_2, h_2, \varsigma))$ iff $\text{def}(\alpha_1 \diamond \alpha_2)$ and $\text{def}(h_1 \oplus h_2)$, with the resulting state $(\alpha_1, h_1, \varsigma) \star (\alpha_2, h_2, \varsigma) = (\alpha_1 \diamond \alpha_2, h_1 \oplus h_2, \varsigma)$.

This merge is only defined if both states have the same AuDaLogic environment ς , as AuDaLogic environments are not always comparable, for example if they have conflicting mappings for the same AuDaLogic variables.

Example 7.2.6. To illustrate adding and subtracting permissions, consider the permissions $\zeta_1 = 0.45$, $\zeta_2 = 0.65$, $\zeta_3 = 0.25$ and $\zeta_4 = 1$. Then, $\zeta_1 \oplus \zeta_3 = 0.7$, as the resulting value is within the range $(0, 1]$. In contrast, $\zeta_1 \oplus \zeta_2$ is undefined, as $0.45 + 0.65 > 1$. We also know that $\zeta_1 \oplus \zeta_0 = \zeta_1$, which also holds if we replace ζ_1 with ζ_2 or ζ_3 . However, $\zeta_4 \oplus \zeta_0$ is not defined, as $\zeta_4 \notin (0, 1)$.

To illustrate merging and splitting label triples, first note that our existing label triple α_e is equal to $\alpha_e^1 \diamond \alpha_e^2$, where $\alpha_e^1 = (\{(\ell_1, \theta_1), (\ell_3, \theta_3)\}, \emptyset, \alpha_e^1(3))$ and $\alpha_e^2 = (\{(\ell_2, \theta_2)\}, \{(\ell_4, \theta_2)\}, \alpha_e^2(3))$, where $\alpha_e^1(3) \cap \alpha_e^2(3) = \emptyset$ and $\alpha_e(3) = \alpha_e^1(3) \cup \alpha_e^2(3)$. As α_e and α_e^1 overlap, $\text{def}(\alpha_e \diamond \alpha_e^1) = \text{false}$, which means that the merge of α_e and α_e^1 is undefined. Similarly, $\alpha_e \diamond \alpha_e^2$ is undefined as well.

Furthermore, to illustrate merging and splitting heaps, we can split our existing heap h_e into heaps h_e^1 and h_e^2 , with

$$\begin{aligned} h_e^1(\ell_1, \text{var1}) &= (0.5, 6) & h_e^2(\ell_1, \text{var6}) &= (1, 42) \\ h_e^1(\ell_2, \text{var2}) &= (1, \ell_3) & h_e^2(\ell_2, \text{var4}) &= (0.125, \ell_1) \\ h_e^1(\ell_4, \text{var4}) &= (1, \text{true}) \end{aligned}$$

and any other inputs for either heap undefined. It then holds that $h_e^1 \oplus h_e^2 = h_e$. With these extra heaps, note that $h_e \oplus h_e^1$ is undefined, as for example the entries $h_e(\ell_2, \text{var2}) = (1, \ell_3)$ and $h_e^1(\ell_2, \text{var2}) = (1, \ell_3)$ cannot be merged due to the permissions summing up to 2, which is not in the permission range. Furthermore, we would also not be able to merge h_e with a heap h' containing the entry $h'(\ell_1, \text{var1}) = (0.25, \ell_1)$, as then the values of the variable var1 as owned by ℓ_1 would not match between the heaps.

From the above two illustrations, it also holds that

$$(\alpha_e, h_e \zeta_e) = (\alpha_e^1, h_e^1, \zeta_e) \star (\alpha_e^2, h_e^2, \zeta_e).$$

The Null-Heap and heap orderings

In AuDaLogic for $\mathcal{P}$, we reason about the memory contents during the execution of $\mathcal{P}$. As by the definition of $\mathcal{P}$ null-instances and their parameters are immediately available, constant in their values and always accessible for reading through `null`, we can restrict ourselves to heaps which always include the null-parameters when reasoning about memory heaps. For this reason, we define the *null-heap*:

Definition 7.2.7 (null-heap). We define the *null-heap* h_0 of AuDaLogic for some program $\mathcal{P}$ as the heap where exactly for all $\theta \in \Theta_{\mathcal{P}}$ and all parameters $x \in \text{Pars}_{\mathcal{P}}(\theta)$ such that $x : T$, $h_0(\ell_\theta^0, x) = (\zeta_0, \text{DefaultVal}(T))$.

The semantics of AuDaLogic for $\mathcal{P}$ are defined to treat the null-heap of $\mathcal{P}$ as the empty heap is treated in general CSL. One can consider that AuDaLogic treats the null-heap as a static extension of the empty heap. This is fitting, as all null-parameters are static and can be read by any struct instance at any time. The null-permission ζ_0 ensures that the null-heap can be split at will into two null-heaps with read-only permissions.

We require that if $h(\ell, x) = (\zeta_0, v)$, then $(\ell, x) \in \text{dom}(h_0)$. This ensures that the null-permission is only used for null-parameters.

We define a partial ordering $\sqsubseteq$ between AuDaLogic heaps as follows:

Definition 7.2.8 (Heap ordering). Let h_1 and h_2 be heaps. We define that $h_1 \sqsubseteq h_2$ iff $\text{dom}(h_1) \subseteq \text{dom}(h_2)$ and for all $(\ell, x) \in \mathcal{L} \times \text{ID}_V$ such that $h_1(x) \neq \perp$, it holds that $h_1(x) = (\zeta_1, v)$ iff $h_2(x) = (\zeta_2, v)$ for some $\zeta_1, \zeta_2 \in Z$ such that $\zeta_1 = \zeta_2$ iff $\zeta_1 \notin (0, 1]$ or $\zeta_2 \notin (0, 1]$ and $\zeta_1 \leq \zeta_2$ if $\zeta_1, \zeta_2 \in (0, 1]$.

Note that this means that for all heaps h such that $h_0 \sqsubseteq h$, null-parameters may only have the null-permission in h , and there cannot be entries for null-parameters in h that are not also in h_0 .

Example 7.2.9. For our example heap h_e , the null-heap would at least contain entries for the parameters of the null-instances of θ_1 , θ_2 and θ_3 , as those are mentioned in the label triple α_e . The full definition of the null-heap for our

example depends on $\mathcal{P}$, however, which we currently do not know. Note that currently, with the null-heap h'_0 for our example, we do know that $h_e \not\sqsubseteq h_0$, as there are no entries with the null-permission ζ_0 in h_e .

For the following examples, let us assume that $h_e \sqsubseteq h_0$, and that the defined entries for h_e are the only entries in h_e that are not also in h'_0 .

AuDaLogic expressions

In AuDaLogic, we maintain a level of abstraction between AuDaLogic and AuDaLa. This is useful, for example when we want to reason abstractly about labels or struct instances, which can only be concrete in the AuDaLa semantics. We furthermore need to be able to reason about permissions, which are not reflected in AuDaLa. Though we can maintain our context in the AuDaLogic environment, we still need to define the syntax and semantics of AuDaLogic expressions to be able to use these abstractions and the AuDaLogic environment in the predicates.

Recall that $\mathbb{T}$ is the set of semantic types of AuDaLa. We define the set of AuDaLogic types as $\mathfrak{T} = \mathbb{T} \cup \{\Theta_{\mathcal{P}}, Z, ID_V, \mathcal{V}\}$. We define the syntax of AuDaLogic expressions as follows:

Definition 7.2.10 (AuDaLogic Expressions Syntax). Let $o_q \in \{=, \neq, \in\}$, $o_c \in \{\leq, \geq, <, >\}$, $o_a \in \{*, /, \%, +, -, \wedge\}$ and $o_b \in \{\wedge, \vee\}$ be binary operators. Let $b \in \mathbb{B}$, $s \in \text{String}$, $n \in \{\mathbb{Z}, \mathbb{N}\}$, $\ell^0 \in \mathcal{L}_0$, $\theta \in \Theta_{\mathcal{P}}$, $\zeta \in Z$, $x \in ID_V$, $t \in \mathfrak{T}$, $f \in \bigcup_n 2^{\mathcal{V}^n}$ and $a \in \mathfrak{A}$. We define AuDaLogic arithmetic expressions E_M , boolean expressions E_B , label expressions E_L , struct type expressions E_T , permission expression E_Z , variable expression E_X , function expressions E_F and execution expressions E as follows. Note that E_M includes the absolute value notation $|E_M|$ in the syntax.

$$\begin{array}{ll}
 E_F ::= f \mid a \mid E_F(E) & E_B ::= b \mid a \mid E_B o_b E_B \mid \neg E_B \mid E_M o_c E_M \mid E o_q E \\
 E_L ::= \ell^0 \mid a & E_M ::= n \mid a \mid |E_M| \mid E_M o_a E_M \\
 E_T ::= \theta \mid a & E_Z ::= E_M \mid \zeta \mid a \\
 E_X ::= x \mid a & E ::= s \mid t \mid a \mid E_F \mid E_Z \mid E_L \mid E_B \mid E_M
 \end{array}$$

We denote the set of as $\mathfrak{E}_E$, with $\mathfrak{E}_M$, $\mathfrak{E}_B$, $\mathfrak{E}_L$, $\mathfrak{E}_T$, $\mathfrak{E}_Z$, $\mathfrak{E}_F$ and $\mathfrak{E}_X$ being the arithmetic (math), boolean, label, struct type, permission, function and variable AuDaLogic expressions respectively.

We extend *def* to AuDaLogic expressions as a type checker, where *def*(E) for $E \in \mathfrak{E}$ iff for all subexpressions $E_1 o E_2$, $\neg E'$, $|E'|$ and $E_F(E')$ it holds that *def*($E_1 o E_2$), *def*($\neg E$), *def*($|E'|$) and *def*($E_F(E')$) respectively. For this, we treat $E_F(E)$ as function application, which is not defined if E_F is not a function or if E has the wrong type for the function in E_F . We use this to define how

AuDaLogic expressions are resolved by the AuDaLogic environment, which is done using the function *res*:

Definition 7.2.11 (Resolving AuDaLogic Expressions). Let $o_q \in \{=, \neq, \in\}$, $o_c \in \{=, \neq, \leq, \geq, <, >\}$, $o_a \in \{*, /, \%, +, -, \wedge\}$ and $o_b \in \{\wedge, \vee\}$. Let $b \in \mathbb{B}$, $s \in \text{String}$, $n \in \{\mathbb{Z}, \mathbb{N}\}$, $\ell^0 \in \mathcal{L}_0$, $\theta \in \Theta_{\mathcal{P}}$, $\zeta \in Z$, $x \in ID_V$, $t \in \mathcal{T}$ and $a \in \mathcal{A}$. Let ς be a AuDaLogic variable environment.

We define the function $res: \mathcal{E} \times (\mathcal{A} \rightarrow \Theta_{\mathcal{P}} \cup \mathcal{V} \cup \bigcup_n 2^{\mathcal{V}^n}) \rightarrow \Theta_{\mathcal{P}} \cup \mathcal{V} \cup \bigcup_n 2^{\mathcal{V}^n}$ such that $def(res(E))$ iff $def(E)$. If $def(E)$, we define $res(E)$ with the following case distinction on E :

$$\begin{array}{ll}
 res(n, \varsigma) = n & res(M o_a E_M, \varsigma) = res(E_M, \varsigma) o_a res(E_M, \varsigma) \\
 res(b, \varsigma) = b & res(E_B o_b E_B, \varsigma) = res(E_B, \varsigma) o_b res(E_B, \varsigma) \\
 res(s, \varsigma) = s & res(E_M o_c E_M, \varsigma) = res(E_M, \varsigma) o_c res(E_M, \varsigma) \\
 res(\ell^0, \varsigma) = \ell^0 & res(E o_q E, \varsigma) = res(E, \varsigma) o_q res(E, \varsigma) \\
 res(\theta, \varsigma) = \theta & res(\neg E_B, \varsigma) = \neg res(E_B, \varsigma) \\
 res(\zeta, \varsigma) = \zeta & res(|a|, \varsigma) = |res(a, \varsigma)| \\
 res(x, \varsigma) = x & res(E_F(E), \varsigma) = res(E_F, \varsigma)(res(E, \varsigma)) \\
 res(t, \varsigma) = t & \\
 res(f, \varsigma) = f & \\
 res(a, \varsigma) = \varsigma(a) &
 \end{array}$$

To give an intuition of the relation between AuDaLa expressions and AuDaLogic expressions, any AuDaLa expression without constructors and without indirect variable references (of the form $a.b.c$) is also an AuDaLogic expression. For our example, $5+5$, $var1=5$ and $var4=false$ can syntactically be considered as AuDaLa and AuDaLogic expressions, while $var4.var6$ can be an AuDaLa expression, but not an AuDaLogic expression. We do not allow indirect variable references in AuDaLogic expressions to avoid aliasing problems. In Section 7.3, we discuss how we bridge the remaining gap between AuDaLa and AuDaLogic expressions, such that we can translate all relevant AuDaLa expressions to AuDaLogic expressions.

AuDaLogic expressions can however also express things AuDaLa expressions cannot express. For example, in our example, we can also have the AuDaLogic expression $a_1 = \ell_1$, which is true according to $res(a_1, \varsigma_e)$, and we can also have $a_4 = \{1, 2, 3\}$. Whether the last one is true or not depends on the entry of a_4 in our AuDaLogic environment ς_e , however, as though ς_e is a total function and will therefore have an entry for a_4 , we did not specify the value of that entry in our example. Furthermore, if we have some defined function, like the domain function *dom* (in the context of the AuDaLa values),

we can have the expression $\text{dom}(f) = \{1, 2, 3\}$. Whether this is true depends on the definition of f , however.

AuDaLogic predicates syntax

With the definitions, functions and concepts above, we define predicates in AuDaLogic:

Definition 7.2.12 (AuDaLogic Predicate Syntax). We define the syntax for AuDaLogic predicates, with predicates P and Q , AuDaLogic variable a , AuDaLogic label expression $E_L \in \mathfrak{E}_L$, AuDaLogic struct type expression $E_T \in \mathfrak{E}_T$, AuDaLogic permission expression $E_Z \in \mathfrak{E}_Z$, AuDaLogic variable expression $E_X \in \mathfrak{E}_X$, boolean AuDaLogic expression $B \in \mathfrak{E}_B$ and AuDaLogic expression $E \in \mathfrak{E}_E$ and indicator $i \in \{1, 2\}$, as follows:

$$\begin{array}{l|l}
 P ::= & \\
 B \mid P \wedge P \mid \neg P \mid \exists a. P & \text{standard} \\
 \mid \mathbf{emp} \mid (E_L, E_X) \xrightarrow{E_Z} E \mid P * P \mid P \multimap P & \text{CSL} \\
 \mid \emptyset_i \mid \blacksquare_i(E_L, E_T) \mid \text{fresh}(E_L) & \text{AuDaLa}
 \end{array}$$

where the CSL clauses come from separation logic, and the AuDaLa clauses are added specifically for AuDaLogic. We define the set of all AuDaLogic predicates to be $\mathbb{P}$.

Note that we consider the separator and separating implication to bind stronger than the other boolean expression operators, and we consider quantifiers to bind stronger than the separator and separating implication.

As discussed before, in AuDaLogic, we expand the use of the separator and separating implication to split full AuDaLogic states as opposed to only memory heaps. We therefore straightforwardly extend the meaning of a *pure* AuDaLogic predicate to not contain any assertions on either the label triple or the memory heap. The use of pure predicates and how they interact with the separator is otherwise unchanged with regards to AuDaLogic, as splits using the separator still have no bearing on the truth of these predicates.

We also introduce *triple-free* and *heap-free* predicates, which are pure with regards to only the label triple or the memory heap respectively:

Definition 7.2.13 (Triple- and heap-free predicates). We define a *triple-free* predicate as a predicate that does not contain any AuDaLogic syntax in the “AuDaLa” row of the syntax (or shorthands derived from them).

We furthermore define *heap-free* predicates as predicates that do not contain memory assertions or occurrences of **emp** in any way (including through shorthands).

We use this in some of the proof rules to give adequately loose constraints on the precondition, as requirements on the memory heap do not need to constrain the predicate with regards to the label triple and vice versa. Note that we have not defined shorthands yet; this will be done later in the section. We included them here to have a complete definition.

Example 7.2.14. Following our ongoing example, we can create some predicates using the above syntax. We create the predicates $(\ell_1, var1) \xrightarrow{0.5} 6 * true$ and $\exists a.(\ell_2, var4) \xrightarrow{0.1} a$ using the CSL category of syntax. Furthermore, we create the predicates $\blacksquare_1(\ell_1, \theta_1) * true$, $\forall(l, a).(\blacksquare_2(l, a) * true \Rightarrow (a = \theta_2))$ and $fresh(\ell_a)$ using the AuDaLa category of syntax.

Here, the predicates made using the CSL category of syntax are *triple-free*, and the predicates made using the AuDaLa category of the syntax are *heap-free*. As an example of a pure predicate, consider the predicate $\exists a.a = \{\ell_1, 2, true\}$. As an example of a predicate which is neither triple-free nor heap-free, consider the predicate $(\blacksquare_1(\ell_2, \theta_2) \wedge (\ell_4, var4) \xrightarrow{1} true) * true$. We will discuss the meaning of the predicates after giving the semantics of AuDaLogic predicates.

AuDaLogic predicates semantics

We then define the semantics per category of the AuDaLogic syntax. We denote the semantics of AuDaLogic predicates as $(\alpha, h, \varsigma) \models P$, which states that the AuDaLogic state (α, h, ς) satisfies the predicate P .

For the predicates in the “standard” row of the syntax, we define the following semantics, which are reasonably straightforward. We use the notation $a : T$ with $T \in \Theta_P$, $T = Z$, $T = ID_V$, $T = \mathcal{V}$ or $T = 2^{\mathcal{V}^n}$ for some n to denote whether AuDaLogic variable a refers to a struct type, a permission, a variable, a value or a relation of arity n .

$$\begin{aligned}
 (\alpha, h, \varsigma) \models B & \text{ iff } res(B, \varsigma) = true \\
 (\alpha, h, \varsigma) \models P \wedge Q & \text{ iff } (\alpha, h, \varsigma) \models P \wedge (\alpha, h, \varsigma) \models Q \\
 (\alpha, h, \varsigma) \models \neg P & \text{ iff } \neg(\alpha, h, \varsigma) \models P \\
 (\alpha, h, \varsigma) \models \exists a.P & \text{ iff, for } a : T, \text{ it holds that } \exists v \in T. (\alpha, h, \varsigma[a \mapsto v]) \models P
 \end{aligned}$$

For the “CSL” row of the syntax, we defined the following semantics:

$$\begin{aligned}
(\alpha, h, \zeta) &\models \mathbf{emp} \text{ iff } h = h_0 \\
(\alpha, h, \zeta) &\models (E_L, E_X) \xrightarrow{E_Z} E \text{ iff } \exists \ell, x, \zeta, v. (\\
&\quad \text{res}(E_L, \zeta) = \ell \wedge \text{res}(E_X, \zeta) = x \wedge \text{res}(E_Z, \zeta) = \zeta \wedge \\
&\quad (\zeta = \zeta_0 \Rightarrow h = h_0) \wedge (\zeta \neq \zeta_0 \Rightarrow (h_0 \sqsubseteq h \wedge \text{dom}(h) = \{(\ell, x)\} \cup \text{dom}(h_0))) \wedge \\
&\quad \text{res}(E, \zeta) = v \wedge h(\ell, x) = (\zeta, v)) \\
(\alpha, h, \zeta) &\models P * Q \text{ iff } \exists \alpha_1, \alpha_2, h_1, h_2. (\alpha, h, \zeta) = (\alpha_1, h_1, \zeta) \star (\alpha_2, h_2, \zeta) \wedge \\
&\quad (\alpha_1, h_1, \zeta) \models P \wedge (\alpha_2, h_2, \zeta) \models Q \\
(\alpha, h, \zeta) &\models P \multimap Q \text{ iff } \forall (\zeta_1, h_1, \zeta). (\text{def}((\alpha, h, \zeta) \star (\alpha_1, h_1, \zeta)) \wedge (\alpha_1, h_1, \zeta) \models P) \Rightarrow \\
&\quad ((\alpha \diamond \alpha_1, h \oplus h_1, \zeta) \models Q))
\end{aligned}$$

In the semantics, we consider the empty heap to be the null-heap, as defined in the semantics for **emp** and memory assertions. Furthermore, the semantics of our memory assertion has two cases: either the permission used is the null-permission and the memory cell should be a null-parameter cell from the null-heap, or the heap should be exactly the null-heap combined with the memory cell as given in the assertion. The value may only be a single value (no sets or second-order logic constructs, which could be allowed through E), and the memory cell should correspond to the assertion. Note that this semantics for the memory assertion ensures that only null-parameters can have the null-permission in AuDaLogic predicates.

The semantics of the separator requires there to be a possible split of the state such that one half satisfies P and the other satisfies Q . The semantics of the separation implication states that for all states that satisfy P and that can be merged with the current state, the new state satisfies Q .

For the AuDaLogic specific syntax, we have the following semantics.

$$\begin{aligned}
(\alpha, h, \zeta) &\models \emptyset_i \text{ iff } \alpha(i) = \emptyset \\
(\alpha, h, \zeta) &\models \blacksquare_i(E_L, E_T) \text{ iff } \alpha(i) = \{(\text{res}(E_L, \zeta), \text{res}(E_T, \zeta))\} \\
(\alpha, h, \zeta) &\models \text{fresh}(E_L) \text{ iff } \text{res}(E_L, \zeta) \in \alpha(3)
\end{aligned}$$

Here, $\emptyset_i$ takes the same role as the empty heap, but for the label triple. It states that the label triple is empty in the facet specified by i , which can be either 1 or 2. The assertion $\blacksquare_i$ is equivalent to the memory assertion, but for the label triple. It states that the facet of the label triple specified by i (which can be 1 or 2), consists of a single entry specified by E_L and E_T . Lastly, *fresh* checks whether the label specified by E_L is in $\alpha(3)$, which means that it has not been used and is available in the current state.

Note that due to our use of the null-permission and the null-heap in the AuDaLogic semantics, and the fact that AuDaLogic label expressions can be

null-labels, it holds that if $(\ell, x) \in \text{dom}(h_0)$, then $P \Leftrightarrow P * (\ell, x) \xrightarrow{\zeta_0} \text{DefaultVal}(T)$, with T the type of x in $\mathcal{P}$. This is because due to the definition of ζ_0 and $\star$, we can always split a state (α, h, ζ) into the states (α, h, ζ) and (α_0, h_0, ζ) , where α_0 is the label triple $(\emptyset, \emptyset, \emptyset)$.

Also note that there is no AuDaLogic state satisfying $(\ell, x) \xrightarrow{\zeta} v$ for some $(\ell, x) \in \text{dom}(h_0)$, $\zeta \in Z \setminus \{\zeta_0\}$ and $v \in \mathcal{V}$. This is because our definition specifies that for this to hold, with the heap of the AuDaLogic state being h , $h_0 \sqsubseteq h$. From this it follows, however, that $h(\ell, x) = (\zeta_0, v)$ for some value $v \in \mathcal{V}$.

Example 7.2.15. We can now evaluate the predicates we made in the last example with regards to (α_e, h_e, ζ_e) . It follows from the above semantics that the predicate $(\ell_1, \text{var1}) \xrightarrow{0.5} 6 * \text{true}$ is true, as we can split h_e into a heap containing only the entry $h_e(\ell_1, \text{var1}) = (0.5, 6)$ and a heap containing the rest of h_e . The predicate $\exists a. (\ell_2, \text{var4}) \xrightarrow{0.1} a$ is false, however, as $h_e(\ell_2, \text{var4})$ has permission 0.125, and not 0.1. If it had the right permission, the predicate would still not be true, as $(\ell_2, \text{var4})$ is not the *only* entry in our heap that is not also in the null-heap.

The predicate $\blacksquare_1(\ell_1, \theta_1) * \text{true}$ is true, as we can split $\alpha_e(1)$ into two sets $\{(\ell_1, \theta_1)\}$ and $\alpha_e(1) \setminus \{(\ell_1, \theta_1)\}$. Similarly, the predicate $\forall(l, a). (\blacksquare_2(l, a) * \text{true} \Rightarrow (a = \theta_2))$ is also true. This predicate states that all struct instances in $\alpha_e(2)$ have struct type θ_2 , which is true.

The predicate $\exists a. a = \{\ell_1, 2, \text{true}\}$ is also true, due to the existence quantifier. Lastly, we have the predicate $(\blacksquare_1(\ell_2, \theta_2) \wedge (\ell_4, \text{var4}) \xrightarrow{1} \text{true}) * \text{true}$. It states that we can split the full state such that one side of the split has only the struct instance of ℓ_2 with type θ_2 in $\alpha_e(1)$, and only an entry for $(\ell_4, \text{var4})$, resolving to $(1, \text{true})$ in the heap. This we can do with α_e and h_e , so this predicate is true. Note the use of the conjunction in this predicate; by using it, we connect our requirements on the label triple to the requirements on the heap.

AuDaLogic shorthands

To make AuDaLogic a bit more intuitive, we standardly use some simple syntactic shorthands. Among these shorthands are $\vee$, $\Rightarrow$ and $\forall$ as commonly defined from $\wedge$, $\neg$ and $\exists$, $\subseteq$ and $\subset$ as commonly defined using $\forall$, $\in$ and $=$, $\cup$ and $\cap$ as defined using the aforementioned operators and $\wedge$, $\vee$ and $\otimes$ as shorthands for repeated conjunctions, disjunctions and separators, taken over finite sets. As an example, for a finite set $L = \{(l_1, x_1), \dots, (l_n, x_n)\}$, $\otimes_{(l_i, x_i) \in L} (l_i, x_i) \hookrightarrow -$ is equivalent to $(l_1, x_1) \hookrightarrow - * \dots * (l_n, x_n) \hookrightarrow -$. We furthermore accept typing notation like $f: A \rightarrow B$ and the relation ‘ $\times$ ’ to type our relations as shorthands. Other shorthands used are clarified when they are relevant. Note that shorthands are taken into consideration for whether a predicate is triple- or heap-free, as per Definition 7.2.13

We also define the following more specific shorthands, to make it easier to reason about the components of our state. For the memory heap, we have the following shorthands, which can be straightforwardly combined:

$$\begin{aligned}
(E_L, E_X) \xrightarrow{E_Z} - &= \exists a. (E_L, E_X) \xrightarrow{E_Z} a \\
(E_L, E_X) \hookrightarrow E &= \exists a. (E_L, E_X) \xrightarrow{a} E \\
(E_L, E_X) \not\xrightarrow{E_Z} E &= (E_L, E_X) \xrightarrow{E_Z} - \wedge \neg((E_L, E_X) \xrightarrow{E_Z} E) \\
(E_L, E_X) \xrightarrow{E_Z} E &= (E_L, E_X) \xrightarrow{E_Z} E * \text{true} \\
(E_L, E_X) \xrightarrow{E_Z}_s E &= (E_L, E_X) \xrightarrow{E_Z} E * \neg(E_L, E_X) \hookrightarrow -
\end{aligned}$$

Note that $(E_L, E_X) \not\xrightarrow{E_Z} E$ is distinct from $\neg((E_L, E_X) \xrightarrow{E_Z} E)$, as the last one is also true if (E_L, E_X) is not in the current heap. Additionally, $(E_L, E_X) \xrightarrow{E_Z}_s E$ is the *strict inclusion*. This shorthand makes it a bit easier to reason about permissions, as $(E_L, E_X) \xrightarrow{E_Z} E$, with $\varsigma(E_Z) = \zeta$ does not exclude that for any ζ' such that $\text{def}(\zeta \oplus \zeta')$ and $\varsigma(E'_Z) = \zeta \oplus \zeta'$, $(E_L, E_X) \xrightarrow{E'_Z} E$. Note that $(E_L, E_X) \hookrightarrow_s E \Leftrightarrow (E_L, E_X) \hookrightarrow E$.

For the label triple, we define the following shorthands. Unbound expressions and variables are assumed to be bound with a universal quantifier.

$$\begin{aligned}
\Box_i(E_L, E_T) &= \blacksquare_i(E_L, E_T) * \text{true} \\
\blacksquare_i E_T &= \forall(l, a). (\Box_i(l, a) \Rightarrow (a = E_T)) \\
\Box_i E_T &= \exists l. (\Box_i(l, E_T)) \\
\blacksquare(E_L, E_T) &= (\blacksquare_1(E_L, E_T) \wedge \emptyset_2) \vee (\blacksquare_2(E_L, E_T) \wedge \emptyset_1) \\
\blacksquare E_T &= \blacksquare_1 E_T \wedge \blacksquare_2 E_T \\
\Box E_T &= \Box_1 E_T \wedge \Box_2 E_T \\
\emptyset &= \emptyset_1 \wedge \emptyset_2
\end{aligned}$$

Intuitively, $\Box_i(E_L, E_T)$ means that $\alpha(i)$, with $i \in \{1, 2\}$, contains the label-struct instance pair as defined by E_L and E_T . This serves the same purpose as $\hookrightarrow$ does for $\hookrightarrow$ in memory assertions. The predicate $\blacksquare_i E_T$ denotes that all struct instances in $\alpha(i)$ have the struct type as specified by E_T and $\Box_i \theta$ denotes that in $\alpha(i)$, there exists a struct instance which has the struct type as specified by E_T . Additionally, note that $\emptyset$ is distinct from the empty set symbol $\emptyset$.

Example 7.2.16. Some of the predicates we defined in earlier examples can be simplified using the shorthands here. The predicate $(\ell_1, \text{var}1) \xrightarrow{0.5} 6 * \text{true}$ can be simplified to $(\ell_1, \text{var}1) \xrightarrow{0.5} 6$. The predicate $\exists a. (\ell_2, \text{var}4) \xrightarrow{0.1} a$ can be simplified to $(\ell_2, \text{var}4) \xrightarrow{0.1} -$.

Of the triple predicates, the predicate $\blacksquare_1(\ell_1, \theta_1) * \text{true}$ can be simplified to $\blacksquare_1(\ell_1, \theta_1)$. Furthermore, the predicate $\forall(l, a) \in \mathcal{L} \times \mathcal{A}. (\blacksquare_2(l, a) * \text{true} \Rightarrow (a = \theta_2))$ can be simplified to $\blacksquare_2 \theta_2$.

The predicate $(\blacksquare_1(\ell_2, \theta_2) \wedge (\ell_4, \text{var4}) \xrightarrow{1} \text{true}) * \text{true}$ cannot be simplified.

The activation relation

Lastly, in AuDaLa, created struct instances will be able to execute steps after the step they have been created in has finished. To mirror that in AuDaLogic, we define the activation relation. For this, we first define, for any label triple α , the *maximally active* label triples:

Definition 7.2.17 (Maximally active label triples). For any label triple α , let the maximally active label triple $m(\alpha) = \alpha'$ be defined as the label triple such that $\alpha(1) \cup \alpha(2) = \alpha'(1)$, $\alpha'(2) = \emptyset$.

This is used to reflect that in AuDaLa, newly created struct instances are set to active when a new step is executed.

We use this to define the *activation relation* S (“Set (active)”):

Definition 7.2.18 (Activation relation S). Let P and P' be AuDaLogic predicates. Then the predicate $S(P, P')$ is true iff $P' \Rightarrow \emptyset_2$ and $(\alpha, h, \zeta) \models P$ iff $(m(\alpha), h, \zeta) \models P'$.

Intuitively, the activation relation relates a predicate with active elements and newly created elements to predicates which are the same except that all existing struct instances are considered active. Syntactically, a good strategy for finding P' can be to change all occurrences of $\blacksquare$ into $\blacksquare_1$, all occurrences of $\blacksquare_2$ into $\blacksquare_1$ and all occurrences of $\blacksquare$ and $\blacksquare_2$ to $\blacksquare_1$. Whether this works, however, depends on the exact predicate. We require this for the proof rules, where we set the new elements to active when a new step is executed.

Example 7.2.19. As an example of a maximally active label triple, for α_e , $m(\alpha_e)$ is the heap defined as $(\{(\ell_1, \theta_1), (\ell_2, \theta_2), (\ell_3, \theta_3), (\ell_4, \theta_2)\}, \emptyset, \alpha_e(3))$.

For the activation relation, consider that $S(\blacksquare_2 \theta_2, \blacksquare_1 \theta_1)$ is false, as $(m(\alpha_e), h_e, \zeta_e) \not\models \blacksquare_1 \theta_2$. As an example where we can straightforwardly find a good P' for the activation relation, consider the predicate $\blacksquare_2(\ell_2, \theta_2)$, which is true for α_e . For this predicate, we can define P' as $\blacksquare_1(\ell_2, \theta_2)$, which is true for $m(\alpha_e)$. It follows that $S(\blacksquare_2(\ell_2, \theta_2), \blacksquare_1(\ell_2, \theta_2))$.

With this, we have given a full definition of the AuDaLogic syntax and semantics. In the next section, we give some notations and functions we use during our proof rules to represent the AuDaLa syntax in AuDaLogic.

7.3 From AuDaLa Syntax to AuDaLogic Predicates

In this section, we build a bridge between AuDaLa syntax and AuDaLogic. We first discuss how to transform well-formed AuDaLa programs into *constructor normal form*, which makes the programs easier to process and reason about in AuDaLogic. Then we shall discuss transforming AuDaLa expressions into AuDaLogic predicates for use in the proof rules. Lastly we shall define the *stable* function, which generates an AuDaLogic predicate which encodes the stability termination requirement for a fixpoint in AuDaLa. We do not feature many examples in this section; the expression conversion method and the stable function are used in the large example in Section 7.5.

7.3.1 Constructor Normal Form

In AuDaLa, most expressions have no side effects when executed with regards to the state the program $\mathcal{P}$ is currently in. There is one exception: the constructor expression. While this expression makes creating new struct instances easier, having expressions containing constructor expressions makes the definition of proof rules for AuDaLa in AuDaLogic far harder. However, constructor expressions can be removed from expressions and statements and confined to local variable assignments, which solves this problem. Confining constructor expressions to special variable assignment statements leads to a program in *constructor normal form*, for which we can define easier proof rules and functions.

In this section, we give two intertwined functions, *ConsElim* and *ConsElimE*, to transform a well-formed statement into one or multiple statements in *constructor normal form*. The function *ConsElim* processes statements, and uses the function *ConsElimE* to process expressions. The method to transform a program into constructor normal form is then to apply *ConsElim* on all statements of the program. Both functions assume the existence of a set $V \in ID$ of currently unused variable names. Such a set can be constructed using the program syntax, and should be provided by the user.

We first give the definition of *ConsElimE*. Intuitively, the output for *ConsElimE* consists of the new expression, a list of required statements to be executed before the new expression and a set of remaining unused variables.

Definition 7.3.20 (The function *ConsElimE*). Let $\circ$ be a syntactic operator, let $\mathcal{E}$ be a list of expressions, let $T \in \Theta_{\mathcal{P}}$ be a type and let $\theta \in \Theta_{\mathcal{P}}$ be a struct type, let $E_v \in EX_V$ be a variable expression, let g be a literal. Let $V \subseteq ID$ be a set of unused variable names. The function $ConsElimE : EX \times 2^{ID} \rightarrow EX \times ST^*, 2^{ID}$ is

then defined as follows:

$$\begin{aligned}
\text{ConsElim}(E_1 \circ E_2, V) &= (E'_1 \circ E'_2, \mathcal{S}_1; \mathcal{S}_2, V'') \\
&\text{where } \text{ConsElim}(E_1, V) = (E'_1, \mathcal{S}_1, V') \\
&\text{and } \text{ConsElim}(E_2, V') = (E'_2, \mathcal{S}_2, V'') \\
\text{ConsElim}(E, V) &= (E', \mathcal{S}, V') \\
&\text{where } \text{ConsElim}(E, V) = (E', \mathcal{S}, V') \\
\text{ConsElim}(!E, V) &= (!E', \mathcal{S}, V') \\
&\text{where } \text{ConsElim}(E, V) = (E', \mathcal{S}, V') \\
\text{ConsElim}(\theta(E_1, \dots, E_n), V) &= (x_{\text{new}}, \mathcal{S}'_1; \dots; \mathcal{S}'_n; \theta \ x_{\text{new}} = \theta(E'_1, \dots, E'_n), V') \\
&\text{where } \forall_{1 \leq i \leq n} \text{ConsElim}(E_i, V_{i-1}) = (E'_i, \mathcal{S}'_i, V_i), \text{ with } V_0 = V, \\
&\quad x_{\text{new}} \in V_n \text{ and } V' = V_n \setminus \{x_{\text{new}}\} \\
\text{ConsElim}(E_v, V) &= (E_v, \varepsilon, V) \\
\text{ConsElim}(g, V) &= (g, \varepsilon, V) \\
\text{ConsElim}(\text{null}, V) &= (\text{null}, \varepsilon, V) \\
\text{ConsElim}(\text{this}, V) &= (\text{this}, \varepsilon, V)
\end{aligned}$$

We now give the definition of *ConsElim*. The output for *ConsElim* consists of the augmented set of statements and a set of remaining unused variables.

Definition 7.3.21 (the function *ConsElim*). Let $E, E_1, \dots, E_n \in EX$ be AuDaLa expressions, and let $V \subseteq ID$ be a set of unused variable names. Then we define $\text{ConsElim}: ST^* \times 2^{ID} \rightarrow ST^* \times 2^{ID}$ as follows:

$$\begin{aligned}
\text{ConsElim}(\varepsilon, V) &= (\varepsilon, V) \\
\text{ConsElim}(\text{if } E \text{ then } \{S_1\}, V) &= (\mathcal{S}'; \text{if } E' \text{ then } \{S'_1\}, V'') \\
&\text{where } \text{ConsElim}(E, V) = (E', \mathcal{S}', V') \text{ and } \text{ConsElim}(\mathcal{S}_1, V') = (\mathcal{S}'_1, V'') \\
\text{ConsElim}(Tx := E, V) &= (\mathcal{S}'; Tx := E', V') \\
&\text{where } \text{ConsElim}(E, V) = (E', \mathcal{S}', V') \\
\text{ConsElim}(x := E, V) &= (\mathcal{S}'; x := E', V') \\
&\text{where } \text{ConsElim}(E, V) = (E', \mathcal{S}', V') \\
\text{ConsElim}(\theta(E_1, \dots, E_n), V) &= (\mathcal{S}', V') \\
&\text{where } \text{ConsElim}(\theta(E_1, \dots, E_n), V) = (E', \mathcal{S}', V') \\
\text{ConsElim}(\mathcal{S}; \mathcal{S}, V) &= (\mathcal{S}_1; \mathcal{S}_2, V'') \\
&\text{where } \text{ConsElim}(\mathcal{S}, V) = (\mathcal{S}_1, V') \text{ and } \text{ConsElim}(\mathcal{S}, V') = (\mathcal{S}_2, V'')
\end{aligned}$$

From this point on, as we define AuDaLogic for $\mathcal{P}$, we shall assume $\mathcal{P}$ to be

well-formed and in constructor normal form. We continue by giving a method to transform AuDaLa expressions into AuDaLogic expressions.

7.3.2 Converting expressions

In this subsection, we bridge the gap between AuDaLa syntax expressions and AuDaLogic expressions: if we want to make AuDaLogic predicates for if-statements and updates, the expressions used in those statements must be converted to AuDaLogic expressions.

To do so, we first define the function $pred(a_\epsilon, E)$ for a simple expression E and $a_\epsilon \in \mathcal{L}$ to return the equivalent *semi-semantic expression*, in which:

1. All literals g are replaced with $val(g)$.
2. All occurrences of $!$ are replaced with $\neg$, and all binary operators are replaced by their semantic equivalents.
3. All occurrences of `this` are replaced by a_ϵ .
4. Assuming all occurrences of `null` are annotated with its type as $null_T$, all occurrences of $null_T$ are replaced by $DefaultVal(T)$.

The result is a semi-expression which is almost an AuDaLogic expression but still contains variable expressions.

As AuDaLogic expressions do not contain a notion of AuDaLa variable expressions, we still have to translate the AuDaLa variable expressions. We do this by assigning an AuDaLogic variable for every AuDaLa variable expression. However, by replacing all AuDaLa variable expressions with AuDaLogic variables, we add in a level of abstraction, and potentially lose the relations inherent in the AuDaLa variable expressions. For example, the AuDaLa expression $x.y + x.y.z$ not only resolves to a value, but also gives information on how the references to the values for the expression relate to each other. When replacing $x.y$ and $x.y.z$ with AuDaLogic variables a and b , we transform the expression $x.y + x.y.z$ to $a + b$, which does not maintain the relation between $x.y$ and $x.y.z$.

For the AuDaLogic expression resolved from an AuDaLa expression to truly be equivalent, the mapping from AuDaLa variables to AuDaLogic variables must retain the structure of the references of the AuDaLa variable expressions. We call this the *expression correspondence requirement*. For example, for $a + b$ to be equivalent to $x.y + x.y.z$, we need to know that a accurately represents $x.y$, which is true if the state fulfills the additional requirement $(l, x) \hookrightarrow c$, with $\zeta(l)$ the label of the struct instance executing $x.y + x.y.z$, and $(c, y) \hookrightarrow a$. Additionally, to ensure that b represents $x.y.z$, we also need to assert that $(a, z) \hookrightarrow b$. We can then be sure that the value resolved from the AuDaLa expression is the same as the value resolved from the AuDaLogic expression.

To deconstruct AuDaLa variable expressions, we define the *occurrence function* $occ : EX^* \rightarrow 2^{EX_V}$, which takes a list of AuDaLa expressions $\mathcal{E}$ and returns all occurrences of maximal variable expressions in $\mathcal{E}$ in a set. For example, $occ(x.y + y.z + x.y.z)$ returns the set $\{x.y, y.z, x.y.z\}$. Furthermore, we define the *prefixes function* $prefixes : EX_V \rightarrow 2^{EX_V}$, which returns a set containing all non-empty prefixes of a variable expression. For example, $prefixes(x.y.z) = \{x, x.y, x.y.z\}$.

We can then formalize the requirements on the context for the correctness of the translation from AuDaLa expression to AuDaLogic expression:

Definition 7.3.22 (Expression correspondence requirement function). Let $E_1, \dots, E_n$ be AuDaLa expressions. Let $x \in ID_V$ be a single variable in AuDaLa and let X be an AuDaLa variable expression. We define the *expression correspondence requirement function* for an AuDaLogic variable a_ϵ and expressions $E_1, \dots, E_n$ as

$$A(a_\epsilon, E_1, \dots, E_n) = \bigwedge_{Y \in occ(E_1, \dots, E_n)} \left(\left(\bigwedge_{x \in prefixes(Y)} (a_\epsilon, x) \hookrightarrow a_x \right) \wedge \left(\bigwedge_{X.x \in prefixes(Y)} (a_X, x) \hookrightarrow a_{X.x} \right) \right).$$

The full translation we then define as follows:

Definition 7.3.23 (AuDaLa-AuDaLogic Expression Correspondence). Let E be an AuDaLa expression. We define that for all states (α, h, ζ) such that $(\alpha, h, \zeta) \models \blacksquare_1(a_\epsilon, \theta)$ for some struct type θ and $(\alpha, h, \zeta) \models A(a_\epsilon, E)$, the *AuDaLogic equivalent expression* to E is $eqi(a_\epsilon, E) = pred(a_\epsilon, E)[Y \mapsto a_Y]_{Y \in occ(E)}$, where $pred(a_\epsilon, E)[Y \mapsto a_Y]_{Y \in occ(E)}$ is the simultaneous syntactical substitution of all maximal variables in $pred(a_\epsilon, E)$.

In Chapter 8, Section 8.2, we prove that we can indeed consider $eqi(a_\epsilon, E)$ to be equivalent to E in AuDaLogic states that satisfy the requirements. This is used when proving our proof system sound. We also use the *eqi* function to define a function to generate *stability predicates*.

7.3.3 The definition of the *stable* function

In AuDaLa, fixpoints only terminate when the system is *stable*. This is defined to be the case when no new struct instances have been created and no parameters have been assigned a new value in the last iteration of the fixpoint. To be able to give proofs about programs using fixpoints in AuDaLogic, we need to give a formal definition of what this means in AuDaLogic terms. To this end, we define a function $stable_P$, which takes a predicate and a schedule and generates a predicate which states the requirements for the schedule to be stable for the active struct instances as implied by the predicate in the next iteration.

Note that this works on the coming iteration, and not on the previous iteration, but as nothing should be changed in the last iteration of a fixpoint this does not impact the postcondition of a fixpoint.

When defining the $stable_{\mathcal{P}}$ function, we assume that $\mathcal{P}$ is in constructor normal form. We use this mainly in the subfunction for processing statements to cover constructors simply with a single extra clause for a variable assignment statement which contains a constructor expression. We can then disregard constructor expressions when dealing with expressions.

An important part of the definition of the $stable_{\mathcal{P}}$ function is the rewrite function $rew: ID_V \rightarrow EX_S$, which is used to rewrite occurrences of local variables with simple expressions that express the local variables in terms of parameters. This is because local variables do not exist in between iterations of a fixpoint, where this function is relevant. To simplify our notation, let $\mathfrak{R}$ be the domain of all possible rewrite functions.

The $stable$ function consists of three layers. Firstly, the top layer, $stable_{\mathcal{P}}$, takes a predicate P and a schedule Δ and splits up the schedule into steps: if the full schedule is stable, then every step executed by the schedule must be stable. Secondly, the second layer, $stableF_{\mathcal{P}}$, takes a predicate P and a step name F and extracts the active struct elements from P to check whether the execution of the statements of F is stable for every active struct instance implied by P . The statements of F are determined by the struct type of the struct instance. Thirdly, the bottom layer, $PrS_{\mathcal{P}}$, takes a label variable a_{ϵ} , a struct type θ , a list of statements S and a rewrite function rew and checks whether the list of statements is stable for the struct instance specified by a_{ϵ} and θ . When called by $stableF_{\mathcal{P}}$, $PrS_{\mathcal{P}}$ is called with the identity rewrite function rew_0 , which does not rewrite anything. We give the definitions from bottom to top.

Definition 7.3.24 (The function $PrS_{\mathcal{P}}$). We define $PrS_{\mathcal{P}}: \mathfrak{E}_L \times \Theta_{\mathcal{P}} \times ST^* \times \mathfrak{R} \rightarrow \mathbb{P}$ as a function with $a_{\epsilon} \in \mathfrak{E}_L$, $\theta' \in \Theta_{\mathcal{P}}$, $E \in EX$, $\mathcal{E} \in EX^*$, $S, S_1 \in ST^*$, $x \in ID_V$, $X \in EX_v, ID_V$ and $E_v \in EX_V$. We assume the empty statement can only occur at the end of any list of statements. If not, the empty statements not at the end of the lists of statements can be removed without issue.

Let $PrE_{\mathcal{P}}(E, rew) = E[Y \mapsto rew(Y)]_{Y \in occ(E)}$. As when transforming expressions, $E[Y \mapsto rew(Y)]_{Y \in occ(E)}$ is the simultaneous substitution of all maximal variables in E . Let for the purposes of this definition, for any expression E , $E' = PrE_{\mathcal{P}}(E, rew)$.

The function is then defined along the following cases:

$$\begin{aligned}
PrSP(a_\varepsilon, \theta, \varepsilon, rew) &= true \\
PrSP(a_\varepsilon, \theta, \theta' x := \theta'(\mathcal{E}); \mathcal{S}, rew) &= false \\
PrSP(a_\varepsilon, \theta, \text{if } E \text{ then } \{S_1\}; \mathcal{S}, rew) &= \\
&\quad A(a_\varepsilon, E') \wedge ((eqi(a_\varepsilon, E') \wedge PrSP(a_\varepsilon, \theta, S_1; \mathcal{S}, rew)) \\
&\quad \vee (\neg eqi(a_\varepsilon, E') \wedge PrSP(a_\varepsilon, \theta, \mathcal{S}, rew))) \\
PrSP(a_\varepsilon, \theta, T x := E; \mathcal{S}, rew) &= A(a_\varepsilon, E') \wedge PrSP(a_\varepsilon, \theta, \mathcal{S}, rew[x \mapsto E']) \\
PrSP(a_\varepsilon, \theta, E_v := E; \mathcal{S}, rew) &= \begin{cases} A(a_\varepsilon, X, E') \wedge (a_X, x) \xrightarrow{1} eqi(a_\varepsilon, E') \wedge \\ \quad PrSP(a_\varepsilon, \theta, \mathcal{S}, rew) \\ \text{if } E_v = X.x \wedge X \neq \varepsilon \\ \\ A(a_\varepsilon, E') \wedge (a_\varepsilon, x) \xrightarrow{1} eqi(a_\varepsilon, E') \wedge \\ \quad PrSP(a_\varepsilon, \theta, \mathcal{S}, rew) \\ \text{if } E_v = x \in ParsP(\theta) \\ \\ A(a_\varepsilon, E') \wedge PrSP(a_\varepsilon, \theta, \mathcal{S}, rew[x \mapsto E']) \\ \text{otherwise, with } E_v = x \\ \text{for some } x \in ID_V \setminus ParsP(\theta) \end{cases}
\end{aligned}$$

Intuitively, the function $PrSP$ gives a predicate for a list of statements $\mathcal{S}$ that can hold iff there are no constructor expressions in $\mathcal{S}$ (second case) and for every parameter, any value it is updated with in $\mathcal{S}$ should be equal to the value the parameter already had. The predicate also checks if the heap contains the right permission to do the accesses required for $\mathcal{S}$, as otherwise, whether $\mathcal{S}$ is stable or not, AuDaLogic will not be able to process the next execution of $\mathcal{S}$. In conclusion, if $PrSP(a_\varepsilon, \theta, \mathcal{S}, rew)$ holds for a state (α, h, ς) , such that α contains only a struct instance s as active struct instance, reflected by (a_ε, θ) , $\mathcal{S}$ is stable for the s and processing all statements during an execution of $\mathcal{S}$ by s is supported by the memory heap h .

We then define $stableFP : ID_S \rightarrow \mathbb{P}$ as follows, with $id \in \mathfrak{A}$ as the identity rewrite function:

$$\begin{aligned}
stableFP(F) &= \\
&\quad \exists L \subseteq \mathcal{L} \times \Theta_P. \bigotimes_{(a_\varepsilon, \theta) \in L} (\blacksquare(a_\varepsilon, \theta) \wedge PrSP(a_\varepsilon, \theta, StepStat_P(F, \theta), id))
\end{aligned}$$

This function divides the assertion that a step must be stable when executed into the assertion that the execution of the step by all separate struct instances

must be stable. We consider all variables a_i to be distinct from any other variables $a_j \in L$.

The intuition is that as there are a finite amount of active elements in the label triple, there will always exist a set L such that $\otimes_{(a_\epsilon, \theta) \in L} (\blacksquare(a_\epsilon, \theta))$ holds. There will also always exist only a single set L for which it holds, as the predicate specifies exactly which struct instance labels should be in $\alpha(1) \cup \alpha(2)$. For this set we should check that all individual struct instances are stable for the statements of F , which is done with $PrS_{\mathcal{P}}(a_\epsilon, \theta, StepStat_{\mathcal{P}}(F, \theta), id)$. See also Lemma 8.2.13 for a correctness proof.

Note that while P may imply multiple heaps of active elements, in the end, there will be only one actual heap of active elements, so the condition will only be true for one of the generated clauses.

The full function $stable_{\mathcal{P}} : SC \rightarrow \mathbb{P}$ then breaks up the schedule into individual stability predicates for each step. It is defined as follows:

$$\begin{aligned} stable_{\mathcal{P}}(F) &= stableF_{\mathcal{P}}(F) \\ stable_{\mathcal{P}}(\theta'.F) &= \blacksquare \theta' \wedge stable_{\mathcal{P}}(F) * \neg \Box \theta' \\ stable_{\mathcal{P}}(\Delta_1 < \Delta_2) &= stable_{\mathcal{P}}(P, \Delta_1) \wedge stable_{\mathcal{P}}(P, \Delta_2) \\ stable_{\mathcal{P}}(Fix(\Delta_1)) &= stable_{\mathcal{P}}(\Delta_1) \end{aligned}$$

We prove the *stable* function to be correct in Chapter 8, Section 8.2.2.

In the next section, we define the proof rules for AuDaLogic.

7.4 Proof Rules

In this section, we give the proof rules for AuDaLogic. As noted at the start of the chapter, these are adapted from proof rules for CSL [29, 31, 143] (CSL) with permissions, with as starting point the work by Vafeiadis [173]. For example, the parallel rules are adapted from the concurrency rule as for example given in Section 7.1, and the general rules are based on Vafeiadis's work.

Recall from Section 7.2 that these are to be defined relative to the program $\mathcal{P}$ from which the syntax comes, and from Section 7.3 that we assume $\mathcal{P}$ to be well-formed and in constructor normal form. It follows that constructors can only occur in variable assignment statements.

We use the program-relative functions as given in Section 7.2, and the predicates A , eqi and *stable* defined in Section 7.3. Additionally, we define the function $fv : \mathbb{P} * \rightarrow 2_V^{ID}$ to return the set of free variable identifiers from the list of predicates given in the input.

In the proof rules, let the different occurrences of E be simple expressions (from EX_S). Let occurrences of S be statements (from ST). We extend this to $\mathcal{E}, \mathcal{S}$ for lists of E and S . Let occurrences of Δ be schedules, and let occurrences of F be steps. Note that a schedule can also be a step. Let $\mathcal{X}$ be an *executable*,

where $\mathcal{X}$ is a schedule or a list of statements. Let P, Q and R be predicates and let occurrences of a with subscripts or otherwise be AuDaLogic variables, such that $a \in \mathfrak{A}$. Furthermore, we define the function fv to return the set of free variables in a predicate, and we define the function ov to return the set of all variables used in a predicate.

We assume unbound variables, predicates or mappings in the proof rules to be universally qualified in the scope of the entire rule, where the rule can be read as “for all (unbound elements), if it holds that (premises), it holds that (conclusion).” In practice, this means that the user can fill in the precise instances of the unbound variables they want to use for the rule, which should satisfy the pattern of the rule.

We then define the proof rules in four groups: statement rules, parallel rules, schedule rules and general rules.

7.4.1 Statement rules

The statement rules deal with the execution of statements in steps. The first two rules, given below, are for the update statements encountered in $\mathcal{P}$.

The first rule, **UPDATE**, deals with updating variables with simple expressions (without constructors) if the variable is not a null-parameter. It requires that the precondition can be split into two distinct parts, one heap-free and one triple-free (see Definition 7.2.13), which ensure that the label triple contains only one active struct instance, and the heap contains the memory cell that is to be written to. The heap should furthermore also fulfill the expression correspondence requirement (see Section 7.3.2) such that the value written to the heap indeed corresponds to the expression E written to the variable in AuDaLa.

$$\begin{array}{c}
 P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
 P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta) \quad P_H \Rightarrow A(a_\epsilon, E, X) \\
 P = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} -) \\
 Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} eqi(a_\epsilon, E)) \\
 \text{(UPDATE)} \frac{}{\vdash \{P\} X.x := E \{Q\}}
 \end{array}$$

The second rule, **NULL UPDATE**, deals with update statements which would update a parameter of a null-instance. The requirements on the precondition are similar to the **UPDATE** rule, except that in this case the permission for the memory cell should be the null-permission ζ_0 , and the write does not actually take place, following the AuDaLa semantics.

$$\begin{array}{c}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta) \quad P_H \Rightarrow A(a_\epsilon, E, X) \\
P = P_L \wedge (P_H * (a_X, x) \xrightarrow{\zeta_0} E') \\
\text{(NULL UPDATE)} \frac{}{\vdash \{P\} X.x := E\{P\}}
\end{array}$$

Note that as our empty heap is the null-heap and $\zeta_0 \oplus 1$ is undefined, we know that the UPDATE rule is not applicable on parameters of null-instances, and as we require that only parameters of null-instances can have the permission ζ_0 , NULL UPDATE is only ever applicable on parameters of null-instances. Note that the preliminaries for both of the rules are the same because even though the update does not happen in NULL UPDATE, all accessed variables need to be in the heap to resolve the expression in the update statement. We consider both the update rules to also apply for the syntax ' $x := E$ ', which we consider a special variant of ' $X.x := E$ ' with an empty X .

The next rules are three rules dealing with variable assignment statements. The first, CONSTRUCTOR, deals with constructor variable assignments. The requirements on the precondition for this rule are similar to those for the UPDATE rule, except that the expression correspondence requirement (Section 7.3.2) must now hold for all expressions in the constructor, and the postcondition must correctly reflect the initialisation of the new struct instance according to the constructor.

$$\begin{array}{c}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
\text{Pars}_{\mathcal{P}}(\theta) = x_1, \dots, x_e \quad P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta) \\
P_H \Rightarrow \wedge \neg(a_\epsilon, x) \hookrightarrow - \wedge A(a_\epsilon, E_1, \dots, E_e) \\
Q = (P_L * \blacksquare_2(l', \theta')) \wedge (P_H * (a_\epsilon, x) \xrightarrow{1} l' * Q') \\
Q' = \bigotimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} eqi(a_\epsilon, E_i) \\
\text{(CONSTRUCTOR)} \frac{}{\vdash \{(P_L * \text{fresh}(l')) \wedge P_H\} \theta \ x := \theta'(E_1, \dots, E_e)\{Q\}}
\end{array}$$

As $\mathcal{P}$ is in constructor normal form, we only need CONSTRUCTOR to deal with constructors; there are no constructor statements in $\mathcal{P}$, nor are there any constructor subexpressions (which means that for all update statements, either UPDATE or NULL UPDATE should be applicable).

The second and the third rule, ASSIGNMENT-1 and ASSIGNMENT-2, deal with variables assigned a simple expression, the first when a local variable is not yet defined and the second when it already has been defined. These rules make sure that when an assignment statement is encountered, the local variable will get an entry in the heap when it does not already have one. For

the assignment rules, let $T \in \mathcal{T} \setminus \Theta_{\mathcal{P}}$.

$$\text{(ASSIGNMENT-1)} \frac{\begin{array}{l} P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\ P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \quad P_H \Rightarrow \neg(a_\varepsilon, x) \hookrightarrow - \\ \vdash \{P_L \wedge (P_H * (a_\varepsilon, x) \xrightarrow{1} -)\} x := E\{Q\} \end{array}}{\vdash \{P_L \wedge P_H\} T \ x := E\{Q\}}$$

$$\text{(ASSIGNMENT-2)} \frac{\begin{array}{l} P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\ P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \quad P_H \Rightarrow (a_\varepsilon, x) \hookrightarrow - \\ \vdash \{P_L \wedge P_H\} x := E\{Q\} \end{array}}{\vdash \{P_L \wedge P_H\} T \ x := E\{Q\}}$$

For the if-then statement, we then have the following rule. This rule requires that the expression correspondence requirement is met for the condition expression, and that the postcondition encapsulates the case in which the if-condition is not met.

$$\text{(IF-STATEMENT)} \frac{\begin{array}{l} P \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \wedge A(a_\varepsilon, E) \\ P \wedge \neg eqi(a_\varepsilon, E) \Rightarrow Q \\ \vdash \{P \wedge eqi(a_\varepsilon, E)\} S\{Q\} \end{array}}{\vdash \{P\} \text{if } E \text{ then } \{S\}\{Q\}}$$

Lastly, we have a rule to deal with a list of statements by splitting of the first statement of the list.

$$\text{(SEQUENCE)} \frac{\begin{array}{l} P \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \\ \vdash \{P\} S\{R\} \quad \vdash \{R\} S\{Q\} \end{array}}{\vdash \{P\} S; S\{Q\}}$$

7.4.2 Parallel rules

We have two parallel rules to zoom in on the effects of single elements on the program. The first rule, Split Parallel, allows the user to more divide the system in two parts. This rule is straightforwardly adapted from the concurrency rule of separation logic.

$$\text{(SPLIT PARALLEL)} \frac{\vdash \{P_1\} F\{Q_1\} \quad \vdash \{P_2\} F\{Q_2\}}{\vdash \{P_1 * P_2\} F\{Q_1 * Q_2\}}$$

The second rule, Gen Parallel (stands for Generalised Parallel), is useful when considering a homogeneous system consisting of multiple very similar parts, for example when considering all the elements of a single struct type. The rule allows the user to prove a property over all parts of the system by

proving the property for only one part of the system. Some flexibility in the shape of the parts may be expressed using the parameter i , which denotes the i th predicate of the split.

$$(\text{GEN PARALLEL}) \frac{\vdash \{P(i)\}F\{Q(i)\}}{\vdash \{\otimes_{1 \leq i \leq n}(P(i))\}F\{\otimes_{1 \leq i \leq n}(Q(i))\}}$$

We have chosen to present both rules as a part of our proof rule system, as Gen Parallel can be used on an arbitrarily big system, which may require an unknown amount of splits. Split Parallel is particularly useful for splitting structs and splitting off null-instances, while Gen Parallel is particularly useful for considering the non-null struct instances of a struct.

7.4.3 Schedule rules

The schedule rules encode the interactions of the schedule with the whole. They consist of a rule about schedule barriers, three rules about step calls and a rule to prove fixpoints.

We first present a rule about barriers, which is very straightforward.

$$(\text{BARRIER}) \frac{\vdash \{P\}\Delta_1\{R\} \quad \vdash \{R\}\Delta_2\{Q\}}{\vdash \{P\}\Delta_1 < \Delta_2\{Q\}}$$

The next rule is for step calls, with F a step, given below. Note that a step can only be called if only a single struct instance is left, as there are no statement proof rules defined if there are multiple struct instances executing them at the same time. Note that during a step call, all the new struct instances are made active, which is represented by the activation relation clause $S(P, P')$ in the rule, which requires that all existing struct instances in P are active in P' (See also Definition 7.2.18).

$$(\text{STEP CALL}) \frac{\begin{array}{l} S(P, P') \quad P' \Rightarrow \blacksquare_1(a_e, \theta) \\ \text{StepStat}_{\mathcal{P}}(F, \theta) = \mathcal{S} \quad \vdash \{P'\}\mathcal{S}\{Q\} \end{array}}{\vdash \{P\}F\{Q\}}$$

For typed step calls, we have the following decomposition rules, which transforms it into a case for step calls by splitting off the part of the state which contains all struct instances not of the relevant struct type.

$$(\text{TYPED CALL}) \frac{\begin{array}{l} \vdash \{P_1\}F\{Q\} \quad P_1 \Rightarrow \blacksquare \theta \\ P_2 \Rightarrow \neg \Box \theta \end{array}}{\vdash \{P_1 * P_2\}\theta.F\{Q * P_2\}}$$

Lastly, we give the fixpoint rule, which proves correctness and termination at the same time. Note that we use the *stable*-predicate for this rule, as that

is the fixpoint termination condition. As AuDaLa loops are fixpoints whose termination depends on the stability of the steps that are executed in the loop, termination is an important aspect of proving fixpoints to be correct. To this end, we require users to define a *variant generator function*, which encodes what meaningfully changes every iteration of the fixpoint. Note that this variant generator function is a function from AuDaLogic expressions to predicates, where the expression denotes the current iteration of the fixpoint. Two common expressions as input would be natural numbers for directly defining the iteration, or AuDaLogic variables for abstractly defining the iteration. We then require that there exists a natural number (iteration) after which the invariant precondition and the variant predicates generated imply that the fixpoint has become stable. This is represented in the second requirement for the fixpoint rule.

The rule requires that the variant is correct, which is the case if for any iteration j such that $Va(j-1)$ holds at the start, $Va(j)$ holds at the end, as then the variant correctly encodes changes per iteration. This can be proven by proving that this is the case for every possible value of j , but it is more beneficial to prove this only once for an abstract j . To encode j as an abstract value, we require that j is a AuDaLogic variable. As j should not be able to be influenced by the defined predicates, it should be *fresh*: it should not overlap with the names used for any variables of the AuDaLogic predicates that already exist, nor should it be used as a variable not relative to the input in the variant generator function Va . Recall that $ov(P)$ returns the set of all predicates occurring in predicate P . We define that $ov(Va)$ returns the set of variables occurring in Va which are not relative to the input.

We then define the fixpoint proof rule as follows.

$$\begin{array}{c}
 j \in \mathfrak{A} \quad j \notin ov(P, stable_P(\Delta), Va) \\
 \vdash \{P \wedge Va(j) \wedge \neg stable_P(\Delta)\} \Delta \{P \wedge Va(j+1)\} \\
 n \geq 0 \quad \forall k \geq n. P \wedge Va(k) \Rightarrow stable_P(\Delta) \\
 \text{(FIXPOINT)} \frac{}{\vdash \{P \wedge Va(0)\} Fix(\Delta) \{P \wedge Va(n+1) \wedge stable_P(\Delta)\}}
 \end{array}$$

7.4.4 General rules

With all the rules above, we are still in need of some general rules to work with predicates. We first include a rule for the empty executable, which is required to reason about step executions if a struct instance has no statements for the step:

$$\text{(EMPTY)} \frac{}{\vdash \{P\} \varepsilon \{P\}}$$

Similarly, we give a rule about what we can prove when our precondition is *false*. This rule works on the principle that the proof system wants to establish

that for each state, if the state satisfies a precondition then executing some code from that state leads to a postcondition. If the precondition is false, then there are no states that satisfy the precondition, so the implication holds vacuously.

$$\text{(FALSE PRECONDITION)} \frac{}{\vdash \{false\} \mathcal{X} \{P\}}$$

Below, we give three standardly introduced rules for pre- and postcondition: weakening, disjunctive postcondition and conjunctive postcondition. Note that the conjunction rule does not include the requirement that the resource invariants are precise, like in the work by Vafeiadis [173], as we do not have resource invariants.

$$\text{(WEAKENING)} \frac{\begin{array}{c} \vdash \{P\} \mathcal{X} \{Q\} \\ P' \Rightarrow P \quad Q \Rightarrow Q' \end{array}}{\vdash \{P'\} \mathcal{X} \{Q'\}}$$

$$\text{(DISJUNCTION)} \frac{\begin{array}{c} \vdash \{P_1\} \mathcal{X} \{Q_1\} \\ \vdash \{P_2\} \mathcal{X} \{Q_2\} \end{array}}{\vdash \{P_1 \vee P_2\} \mathcal{X} \{Q_1 \vee Q_2\}}$$

$$\text{(CONJUNCTION)} \frac{\begin{array}{c} \vdash \{P_1\} \mathcal{X} \{Q_1\} \\ \vdash \{P_2\} \mathcal{X} \{Q_2\} \end{array}}{\vdash \{P_1 \wedge P_2\} \mathcal{X} \{Q_1 \wedge Q_2\}}$$

Let in the next rule a be an AuDaLogic variable.

$$\text{(EXAMPLE)} \frac{\vdash \{P\} \mathcal{X} \{Q\}}{\vdash \{\exists a.P\} \mathcal{X} \{\exists a.Q\}}$$

The weakening and example rules allow us to abstract away from exact values in cases where the exact value is not important or not given. As an example, we can weaken $P * (\ell_1, E_1) \mapsto -$ to $\exists a.(P * (\ell_1, E_1) \mapsto a)$ and then remove the existential quantifier using the example rule.

We also use the frame rule. Recall that the set of free variables of a predicate R is $fv(R)$. We split it in two; one for a list of statements and one for a schedule, as for lists of statements newly created elements are inert, and for schedules, they are not. Recall that S is a list of statements and Δ is a schedule.

$$\text{(FRAME STATEMENTS)} \frac{\begin{array}{c} \vdash \{P\} S \{Q\} \\ fv(P) \cap fv(R) = \emptyset \\ R \Rightarrow \emptyset_1 \end{array}}{\vdash \{P * R\} S \{Q * R\}}$$

$$\text{(FRAME SCHEDULE)} \frac{\begin{array}{c} \vdash \{P\} \Delta \{Q\} \\ fv(P) \cap fv(R) = \emptyset \\ R \Rightarrow \emptyset \end{array}}{\vdash \{P * R\} \Delta \{Q * R\}}$$

These are all the proof rules of AuDaLogic. In the next chapter, we shall prove the proof system presented in this chapter sound.

7.5 Proving Linear Sort with AuDaLogic

In this section, we prove the results of executing linear sort, as given in Listing 3.6 and Section 3.4.1, using AuDaLogic. We do this with the intent of proving that the program correctly sorts a list of struct instances containing distinct values in ascending order, which we establish by proving the program's effects and then concluding from those effects that the intended effect is reached.

Here, we generally take an observant approach instead of a deterministic one where we can. Where possible, we do not assert postconditions before starting on the proof tree, as the postcondition should follow from the effects of the execution. We cannot do this for fixpoints, however, where we will have to define a variant and invariant with our intentions for the effects of the loop.

In Section 7.5.1, we first give some preliminaries and define a precondition using information from AuDaLa and Section 3.4.1. Furthermore, we decompose the schedule. In Section 7.5.2, we deal with the fixpoint in the schedule, and in Section 7.5.3 we process the step *compareElements*. Following this, we conclude the fixpoint by determining its postcondition in Section 7.5.4. Then, we process the step *reorder* in Section 7.5.5 and conclude our proof tree in Section 7.5.6.

Due to its size, we do not attempt to give the proof tree as a whole. Instead, let any occurrence of ∇ be a reference to another part of a proof tree.

As it is useful to show the work in progress, we often give unfinished versions of the partial proof trees. The finished trees are labeled in their figures using the label “**Finished:**”. Unknown predicates are given using a question mark. While sometimes the contents of an unknown postcondition can be elaborated upon relatively to the precondition, we do not elaborate upon partially known postconditions until we can fully define them, to keep the example clear. In contrast, we do note down fully defined postconditions if the proof rules require them, as we can use those as intermediate checks for correctness so far. We use ... above a rule if we discuss the part of the proof tree above that rule later. Lastly, we often have to remove multiple existential quantifiers in sequence, which we denote by the label `EXAMPLE+`.

While we can give an ordered list as a relation $f_0[1 \mapsto e_1, \dots, n \mapsto e_n]$, where f_0 is the appropriate empty function, we use a notational shorthand and write lists as $f = e_1; \dots; e_n$, to make it align with the rest of the thesis. Note that this means that a list element e_i can be more accurately represented as $f(i)$, which means that i can be represented as an AuDaLogic variable, even though it is represented as a subscript. We also use $e_i \in f$ instead of $i \in \text{dom}(f) \wedge e_i = f(i)$, and similarly extend set notation to lists for this section.

7.5.1 Preliminaries and schedule

To create a proof tree to prove the effects of the program given in Listing 3.6, we first have to consider the program and its explanation as given in Section 3.4.1. To make the proof tree more readable, we give the following designations to all the statements in Listing 3.6:

$S_1 = \text{"if } comp \neq null \text{ then } \{ S_2 \}"$
 $S_2 = \text{"if } (comp.val \geq val \ \&\& \ ((newNext = null) \parallel comp.val < newNext.val))$
 $\quad \text{then } \{ S_3 \}"$
 $S_3 = \text{"newNext := comp;"}$
 $S_4 = \text{"comp := comp.next;"}$
 $S_5 = \text{"next := newNext;"}$

As stated in Problem 5.2.10, we are given a list of distinct positive values. Listing 3.6 does not include an initialisation, but the expected results are stated in Section 3.4.1: after the initialisation, the program assumes that the list is correctly made, and that for all *ListElem* elements, *comp* is set to the *ListElem* representing the first element of the list and *newNext* is set to null.

We use this to define a precondition P_0 , as given below. In P_0 , we assert that there is an ordered list of distinct, positive values represented by an ordered list of AuDaLogic variables and an ordered list of label variables which represent the *ListElem* struct instances of the initialised list. We then assert that every label variable l_i should correctly represent the i th element of the list, with the parameters initialised as expected. Lastly, we also assert that there is a null-instance active for *ListElem*, which follows from the semantics of AuDaLa. This precondition is strict; it asserts that the list and the null-instance exist, but also that no other active elements exist. Similarly, the heap is also kept strict: only the parameters (and null-parameters) of the *ListElem* struct instances exist.

$$\begin{aligned}
 P_0 = & \exists Vals: \mathbb{Z} \rightarrow \mathcal{V}. Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \\
 & \forall 0 \leq x < n. a_x > 0 \wedge \exists L \subset \mathbb{Z} \rightarrow \mathcal{L}. L = l_0; \dots; l_{n-1} \wedge \\
 & \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne} \in \mathcal{L}. (((l_i, next) \xrightarrow{1} l_{ne} * \\
 & \quad (l_i, newNext) \xrightarrow{1} \ell_{ListElem}^0 * (l_i, comp) \xrightarrow{1} l_0 * (l_i, val) \xrightarrow{1} a_i) \\
 & \quad \wedge ((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1}))) \\
 & * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2 \wedge \mathbf{emp})
 \end{aligned}$$

As it is beneficial for the fixpoint variant generator function later, we have decided to index our values and struct instances from 0 to $n-1$. It is important to note, though, that l_0 is distinct from the null-instance reference $\ell_{ListElem}^0$.

$$\frac{\frac{\nabla_1}{\vdash \{P_1\}\text{Fix}(\text{compareElement})\{?\}} \quad \frac{\dots}{\vdash \{?\}\text{reorder}\{?\}}}{\frac{\vdash \{P_1\}\text{Fix}(\text{compareElement}) < \text{reorder}\{?\}}{\{P_0\}\text{Fix}(\text{compareElement}) < \text{reorder}\{?\}} \text{EXAMPLE+}} \text{BARRIER}$$

Figure 7.1: Proof Tree ∇_0 , first unfinished version

With this precondition, we start our proof tree by decomposing the schedule. Recall that the schedule of linear sort is $\text{Fix}(\text{compareElement}) < \text{reorder}$, with compareElement composed of statements S_1 to S_4 , and reorder composed of statement S_5 .

We first remove the two existential quantifiers for the value AuDaLogic variables and the label variables of P_0 using two applications of the example rule. We depict this in Figure 7.1. For our readability, we denote sequences of applications of the same rule only once. Removing the existential quantifiers leads to P_1 :

$$\begin{aligned}
P_1 = & (\text{Vals} = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& L = l_0; \dots; l_{n-1} \wedge \\
& \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{ne} \in \mathcal{L}. (((l_i, \text{next}) \xrightarrow{1} l_{ne} * \\
& (l_i, \text{newNext}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * (l_i, \text{comp}) \xrightarrow{1} l_0 * (l_i, \text{val}) \xrightarrow{1} a_i) \\
& \wedge ((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1}))) \\
& * (\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2 \wedge \mathbf{emp})
\end{aligned}$$

We furthermore use the BARRIER rule to split the schedule, as also depicted in Figure 7.1. The next step is to deal with the fixpoint.

7.5.2 Fixpoints, variants and the *stable* predicate

To prove the result of the fixpoint, we want to use the FIXPOINT rule, for which we must transform P_1 into a form $P_2 \wedge \text{Va}(1)$, for some P and some *variant* Va . Here, a variant is a predicate relative to the iteration, which should encapsulate the changes that happen in every iteration. As for proving a fixpoint we also need to prove that a program terminates, the variant should eventually imply that the system is stable as well.

As described in Section 3.4.1, the *ListElem* elements in the program traverse the list and during that traversal, find the element which has the smallest value bigger than its own value. We define a variant generator function Va based on this intent, which generates a predicate for every natural number,

dependent on the parameters *comp*, *val* and *next*. Here, $Va(0)$ gives the variant as it holds before execution, and $Va(i)$ with $i > 0$ gives the variant as it should hold after i iterations.

$$\begin{aligned}
Va(h) = & \forall_{0 \leq i \leq n-1}. \exists l_C, l_{NN} \in \mathcal{L}. ((l_i, comp) \hookrightarrow l_C \wedge \\
& (h < n) \Rightarrow l_C = l_h \wedge ((h \geq n) \Rightarrow l_C = \ell_{ListElem}^0) \wedge \\
& (l_i, newNext) \hookrightarrow l_{NN} \wedge (l_{NN} \neq \ell_{ListElem}^0 \Rightarrow \\
& (\exists a_{NN} \in \mathcal{V}. ((l_{NN}, val) \hookrightarrow a_{NN} \wedge a_{NN} > a_i \wedge \\
& (\forall_{0 \leq t < \min(h, n)}. a_t > a_i \Rightarrow a_t > a_{NN})))) \wedge \\
& (l_{NN} = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(h, n)}. a_t \leq a_i))
\end{aligned}$$

This states that for all struct instances in the list (represented by L), the label currently in the *comp*-parameter (represented by l_C) is the label of the h th struct instance of the list if there have been less or equal to n iterations. Furthermore, this states that for all struct instances in the list with value v_i , if the label referenced by the *newNext* parameter (represented by l_{NN}) does not point to the null-instance of *ListElem*, it references the struct instance in the first h or n elements (whichever is smaller) in the list with the smallest value larger than v_i . Additionally, if the *newNext* parameter does refer to null, then all values in the first h or n (whichever is smaller) elements is smaller than v_i .

If we take h to be 0, then we get that $Va(0)$ is the following predicate:

$$\begin{aligned}
Va(0) = & \forall_{0 \leq i < n}. \exists l_{NN} \in \mathcal{L}. (l_i, comp) \hookrightarrow l_0 \wedge (l_i, newNext) \hookrightarrow l_{NN} \wedge \\
& (l_{NN} \neq \ell_{ListElem}^0 \Rightarrow (\exists a_{NN} \in \mathcal{V}. ((l_{NN}, val) \hookrightarrow a_{NN} \wedge a_{NN} > a_i)))
\end{aligned}$$

We weaken P_1 into P_2 , given below, to redistribute some permissions. We make sure that all struct instances have write permissions for their *newNext* and *comp* parameters, and read permissions for the values of the struct instances of their *newNext* and *comp* values, which they will need to execute the *compareElement* step. Furthermore, we give every struct instance read permissions for the next variable of all struct instances. This is required because we need to keep the permissions for the *next* parameters explicit, as we still want to write to those in the step *reorder*. We add some clauses to encode the possible values of the *next* parameter of the current struct instance and the *comp* struct instance, and lastly assert that there is a null-instance active. Note that we have weakened away the **emp** assertion for the null-instance; this is because the leftover permissions not made explicit in P_2 can then be assigned to the null-instance.

$$\begin{aligned}
P_2 = & (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& L = l_0; \dots; l_{n-1} \wedge \\
& \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\
& \quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \mapsto l_{ne} \wedge (l_c, next) \mapsto l_{cn}) \\
& \quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \quad \wedge ((l_c \neq \ell_{ListElem}^0) \Rightarrow (\exists 0 \leq z < n. \ell_c = \ell_z \wedge \\
& \quad ((z = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((z < n - 1) \wedge l_{cn} = l_{z+1})))) \\
& * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

As $P_2 \Rightarrow Va(0)$, we abstract $P_2 \wedge Va(0)$ to just P_2 . We then show that for all $k \geq n$, $P \wedge Va(k) \Rightarrow stable_P(compareElement)$. For this, we adapt $Va(h)$ for k as follows:

$$\begin{aligned}
Va(k) = & \forall 0 \leq i < n. (l_i, comp) \mapsto \ell_{ListElem}^0 \wedge \\
& \exists l_{NN} \in \mathcal{L}. (l_i, newNext) \mapsto l_{NN} \wedge (l_{NN} \neq \ell_{ListElem}^0) \Rightarrow \\
& (\exists a_{NN} \in \mathcal{V}. ((l_{NN}, val) \mapsto a_{NN} \wedge a_{NN} > a_i \wedge \\
& \quad \forall 0 \leq t < n. a_t > a_i \Rightarrow a_t > a_{NN})) \wedge \\
& (l_{NN} = \ell_{ListElem}^0) \Rightarrow (\forall 0 \leq t < n. a_t \leq a_i)
\end{aligned}$$

We then make $stable_P(compareElement)$ concrete by unfolding its definition. In particular, we know that $PrSP(l_i, ListElem, S_1; S_4, id)$ resolves to the predicate given below.

$$\begin{aligned}
& (l_i, comp) \mapsto a_C \wedge (a_C \neq \ell_{ListElem}^0 \wedge \\
& \quad (l_i, val) \mapsto a_V \wedge (a_C, val) \mapsto a_{CV} \wedge \\
& \quad (l_i, newNext) \mapsto a_{NN} \wedge (a_{NN}, val) \mapsto a_{NNV} \wedge \\
& \quad (a_{CV} > a_V \wedge (a_{NN} = \ell_{ListElem}^0 \vee a_{CV} < a_{NNV})) \wedge (\\
& \quad (l_i, newNext) \xrightarrow{1} a_{NN} \wedge a_{NN} = a_C \\
& \quad \wedge (l_i, comp) \xrightarrow{1} a_C \wedge (a_C, next) \mapsto a_{CN} \wedge a_C = a_{CN}) \\
&) \vee (\neg(a_{CV} > a_V \wedge (a_{NN} = \ell_{ListElem}^0 \vee a_{CV} < a_{NNV})) \\
& \quad \wedge (l_i, comp) \xrightarrow{1} a_C \wedge (a_C, next) \mapsto a_{CN} \wedge a_C = a_{CN}) \\
&) \vee (a_C = \ell_{ListElem}^0 \wedge (l_i, comp) \xrightarrow{1} a_C \wedge (a_C, next) \mapsto a_{CN} \wedge a_C = a_{CN})
\end{aligned}$$

We then simplify this and map all introduced a -variables to the existentially introduced variables of P_2 . The mapping is straightforward and can be found by matching the memory assertion requirements on the existentially introduced variables of P_2 with the existentially introduced variables of $\text{PrSp}(l_i, \text{ListElem}, S_1; S_4, id)$. The mapping is enforced by the fact that with any other mapping, $P_2 \wedge V(k) \Rightarrow \text{stable}_{\mathcal{P}}(\text{compareElement})$ cannot hold due to aforementioned requirements. As a result, we get the following predicate for $\text{PrSp}(l_i, \text{ListElem}, S_1; S_4, id)$:

$$\begin{aligned}
 (l_i, \text{comp}) &\xrightarrow{1} l_c \wedge (l_c \neq \ell_{\text{ListElem}}^0 \Rightarrow \\
 &\quad (l_i, \text{val}) \hookrightarrow a_i \wedge (l_c, \text{val}) \hookrightarrow a_c \wedge (l_i, \text{newNext}) \hookrightarrow l_{nn} \wedge (l_{nn}, \text{val}) \hookrightarrow a_{nn} \wedge \\
 &\quad (((a_{nn} > a_i) \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_c < a_{nn})) \Rightarrow \\
 &\quad \quad (l_i, \text{newNext}) \hookrightarrow l_{nn} \wedge (l_{nn} = l_c)) \\
 &\quad) \wedge (l_c, \text{next}) \xrightarrow{l} \hookrightarrow_{cn} \wedge l_c = l_{cn}
 \end{aligned}$$

The full stable predicate $\text{stable}_{\mathcal{P}}(\text{compareElement})$ for the system that satisfies P_2 is $\bigotimes_{(l_i, \text{ListElem}) \in L \times \{\text{ListElem}\}} \text{PrSp}(l_i, S_1; S_4) * \text{PrSp}(\ell_{\text{ListElem}}^0, \text{ListElem}, S_1; S_4, id)$. For $\text{PrSp}(\ell_{\text{ListElem}}^0, \text{ListElem}, S_1; S_4, id)$, we map the introduced variables to reflect the null-values of the null-instance parameters. This results in the following predicate, which resolves to *true*, due to the semantics of AuDaLogic:

$$(\ell_{\text{ListElem}}^0, \text{comp}) \hookrightarrow \ell_{\text{ListElem}}^0 \wedge (\ell_{\text{ListElem}}^0, \text{next}) \hookrightarrow \ell_{\text{ListElem}}^0 \wedge \ell_{\text{ListElem}}^0 = \ell_{\text{ListElem}}^0$$

To prove that $P_2 \wedge Va(k) \Rightarrow \text{stable}_{\mathcal{P}}(\text{compareElement})$, we can distribute the clauses of $Va(k)$ over the separator of P_2 . In AuDaLogic (as in separation logic [140]), it holds that

$$(P_A * P_B) \wedge (a_1, x) \hookrightarrow a_2 \Leftrightarrow ((P_A \wedge (a_1, x) \hookrightarrow a_2) * P_B) \vee (P_A * (P_B \wedge (a_1, x) \hookrightarrow a_2)).$$

It follows that distributing $Va(k)$ over the big separator of P_2 leads to a lot of or-clauses. However, the clauses in $Va(k)$ require that $(l_i, \text{comp}) \hookrightarrow -$ and that $(l_i, \text{newNext}) \hookrightarrow -$, so every clause has only one separator clause for which it is not immediately false, leading the entire separator to be false. To that end, with $Va(k) = Va(k)_1 \wedge \dots \wedge Va(k)_n$, we define the predicate $P_2 \wedge Va(k)$, given below.

$$\begin{aligned}
P_2 \wedge Va(k) = & (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& L = l_0; \dots; l_{n-1} \wedge \\
& \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\
& \quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \quad \wedge ((l_c \neq \ell_{ListElem}^0) \Rightarrow (\exists 0 \leq z < n. \ell_c = \ell_z \wedge \\
& \quad ((z = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((z < n - 1) \wedge l_{cn} = l_{z+1}))) \\
& \quad \wedge (l_i, comp) \hookrightarrow \ell_{ListElem}^0 \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < n. a_t \leq a_i)) \\
& \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall 0 \leq t < n. a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \quad * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

This we resolve to the following predicate.

$$\begin{aligned}
P_2 \wedge Va(k) = & (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& L = l_0; \dots; l_{n-1} \wedge \\
& \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_{nn} \in \mathcal{L}. \exists a_{nn} \in \mathcal{V}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} \ell_{ListElem}^0 * \\
& \quad (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne}) \\
& \quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall 0 \leq t < n. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < n. a_t \leq a_i))) \\
& \quad * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

It follows that $P_2 \wedge Va(k) \Rightarrow stable_{\mathcal{P}}(compareElement)$, due to the clause $(l_i, comp) \xrightarrow{1} \ell_{ListElem}^0$ for every $l_i \in L$ in $P_2 \wedge Va(k)$. We can then continue our proof tree, as shown in Figure 7.2. We give the postcondition of the application of $FIXPOINT$ as $P_2 \wedge Va(k) \wedge stable_{\mathcal{P}}(compareElement)$, as due to the definition of $Va(n+1)$, $Va(n+1) = Va(k)$. We omit $stable_{\mathcal{P}}(compareElement)$ from the postcondition as it is implied by $P_2 \wedge Va(k)$, and as $Va(n+1) = Va(k)$.

We then choose a fresh ghost variable j for the iteration we are going to prove, and resolve $P_2 \wedge Va(j) \wedge \neg stable_{\mathcal{P}}(compareElement)$ into P_3 . As the input for Va , we may assume that j is a natural number.

$$\frac{\frac{\nabla_2}{\frac{\vdash \{P_2 \wedge Va(j) \wedge \neg \text{stable}_{\mathcal{P}}(\text{compareElement})\} \text{compareElement} \{P_2 \wedge Va(j+1)\}}{\vdash \{P_2\} \text{Fix}(\text{compareElement}) \{P_2 \wedge Va(k) \wedge \text{stable}_{\mathcal{P}}(\text{compareElement})\}} \text{FP}}{\vdash \{P_1\} \text{Fix}(\text{compareElement}) \{?\}} \text{WK}$$

Figure 7.2: Proof Tree ∇_1 , unfinished version. Here, WK stands for WEAKENING and FP stands for FIXPOINT.

We distribute $Va(j)$ over P_2 to get $P_2 \wedge Va(j)$, as given below. We unify the existential variables of $Va(j)$ and P_2 , with a straightforward mapping. As $V(j)$ requires that $((j < n) \Rightarrow l_c = l_j) \wedge ((j \geq n) \Rightarrow l_c = \ell_{ListElem}^0)$, we slightly simplify the predicate part for l_{cn} , as if $l_c \neq \ell_{ListElem}^0$, $(j < n)$ must hold, so $l_c = l_j$.

$$\begin{aligned}
P_2 \wedge Va(j) = & (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& (L = l_0; \dots; l_{n-1}) \wedge \\
& (\otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c \in \mathcal{V}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\
& \quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \quad \wedge (((i = n-1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n-1) \wedge l_{ne} = l_{i+1})) \\
& \quad \wedge (j < n \Rightarrow l_c = l_j \wedge ((j \geq n) \Rightarrow l_c = \ell_{ListElem}^0)) \\
& \quad \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n-1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n-1) \wedge l_{cn} = l_{j+1}))) \\
& \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < \min(j, n)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(j, n)}. a_t \leq a_i))) \\
& \quad * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

We can optionally also distribute the $\neg \text{stable}_{\mathcal{P}}(\text{compareElement})$ over P_2 , as the precondition allows this, but as we do not need it and it makes the predicate bigger, we will weaken it away instead. See the end of the subsection for an example of how to distribute $\neg \text{stable}_{\mathcal{P}}(\text{compareElement})$ over P_2 .

The next step is to put the first two lines of P_2 into the separator, for which we utilize an rule of separation logic from given by Reynolds [159], which states that if P_A is pure, $P_A \wedge (P_B * P_C) \Leftrightarrow (P_A \wedge P_B) * P_C$. We therefore duplicate C and add it to the clauses of the struct instances, which results in P_3 :

$$\begin{aligned}
C = & (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& (L = l_0; \dots; l_{n-1})
\end{aligned}$$

$$\begin{aligned}
P_3 = & \textcircled{*}_{0 \leq i < n} (\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} l_c * \\
& (l_c, \text{val}) \mapsto a_c * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i * \\
& ((\textcircled{*}_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_c, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (j < n \Rightarrow l_c = l_j \wedge (j \geq n) \Rightarrow l_c = \ell_{\text{ListElem}}^0)) \\
& \wedge (l_c \neq \ell_{\text{ListElem}}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < \min(j, n)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < \min(j, n)}. a_t \leq a_i))) \\
& \wedge C) * (\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2)
\end{aligned}$$

From this point on, we omit C for readability, as it does not change until the very end.

We split off the null-instance, which we will prove separately. This results in P_4 , which is the same as P_3 except that the last line is removed. We split P_4 into P_5 for each struct instance using GEN PARALLEL:

$$\begin{aligned}
P_5 = & \blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} l_c * \\
& (l_c, \text{val}) \mapsto a_c * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i * \\
& ((\textcircled{*}_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_c, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (j < n \Rightarrow l_c = l_j \wedge (j \geq n) \Rightarrow l_c = \ell_{\text{ListElem}}^0)) \\
& \wedge (l_c \neq \ell_{\text{ListElem}}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < \min(j, n)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < \min(j, n)}. a_t \leq a_i))) \wedge C
\end{aligned}$$

Continuing the proof tree with these steps results in Figure 7.3. In the figure, we use P_N to denote the predicate $\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2$.

The next step is to process the step *compareElement* in Section 7.5.3. The rest of this subsection will describe distributing the negated stability predicate over P_3 . We have included this as an example, but as it is not used in our proof tree, consider it optional.

$$\frac{
\frac{
\frac{\nabla_3}{\vdash \{P_5\}compareElement\{?\}}
}{\vdash \{P_4\}compareElement\{?\}}
GP
\quad
\frac{\dots}{\vdash \{P_N\}compareElement\{?\}}
SP
}{
\vdash \{P_3\}compareElement\{P_2 \wedge Va(j+1)\}
}
WK
\vdash \{P_2 \wedge Va(j) \wedge \neg stable_{\mathcal{P}}(compareElement)\}compareElement\{P_2 \wedge Va(j+1)\}$$

Figure 7.3: Proof Tree ∇_2 , first unfinished version, where WK stands for WEAKENING, SP stands for SPLIT PARALLEL and GP stands for GEN PARALLEL.

Extra example - distributing $\neg stable_{\mathcal{P}}(compareElement)$ over $P_2 \wedge Va(j)$

In other proofs, it may be required to distribute $\neg stable_{\mathcal{P}}(compareElement)$ over a precondition. To show an example of how this works, we give an example of how $\neg stable_{\mathcal{P}}(compareElement)$ can be distributed over P_3 .

Consider that

$$\begin{aligned}
\neg stable_{\mathcal{P}}(compareElement) = \\
\neg(\otimes_{(l_i, ListElem) \in L \times \{ListElem\}} PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id) * true).
\end{aligned}$$

This states that the split we wish to make does not exist. This can only be if for all possible partial splits over the elements of L , for some $l_i \in L$, $PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id)$ does not hold. This includes the split made for P_2 .

To be able to reason about all instances of $ListElem$ during the execution of the step, we generalize the above. For convenience's sake, we extend set notation to lists. There must exist a non-empty sublist $L_0 \subseteq L$ such that for the split in P_2 , the predicate $PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id)$ does not hold iff $l_i \in L_0$. Let $L_1 = L \setminus L_0$; for the elements l_i in L_1 , it follows that $PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id)$ holds in the split of P_2 . Then we conclude that

$$\begin{aligned}
\neg stable_{\mathcal{P}}(compareElement) \Rightarrow \exists L_0, L_1 \subseteq L. (L_0 \cap L_1 = \emptyset \wedge L_0 \neq \emptyset \wedge \\
\forall l_i \in L_0. \neg PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id) \wedge \\
\forall l_i \in L_1. PrS_{\mathcal{P}}(l_i, ListElem, S_1; S_4, id)).
\end{aligned}$$

We add this to $P_2 \wedge Va(j)$ to make $P_2 \wedge Va(j) \wedge \neg stable_{\mathcal{P}}$, which we call P_{E1} , as given below.

$$\begin{aligned}
P_{E1} = & (Vals = \{a_0, \dots, a_{n-1}\} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
& (L = \{l_0, \dots, l_{n-1}\}) \wedge \\
& \exists L_0, L_1 \subset L. (L_0 \cap L_1 = \emptyset \wedge L_0 \neq \emptyset \wedge \forall l_i \in L_0. \neg PrSp(l_i, ListElem, S_1; S_4, id \wedge \\
& \quad \forall l_i \in L_1. PrSp(l_i, ListElem, S_1; S_4, id))) \wedge \\
& (*_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\
& \quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \quad \wedge (j < n \Rightarrow l_c = l_j \wedge ((j \geq n) \Rightarrow l_c = \ell_{ListElem}^0)) \\
& \quad \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < \min(j, n)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(j, n)}. a_t \leq a_i))) \\
& * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

We then remove the existential quantifier for L_0 and L_1 using the EXAMPLE rule, resulting in P'_{E1} . We furthermore distribute the universal quantifiers associated with L_0 and L_1 over the separator.

For this, we can distribute the universal quantifier for L_1 over the separator as explained before, with a lot of or-clauses where the faulty distribution makes the whole clause false by default. Here, for a single label l_i , the predicate $PrSp(l_i, ListElem, S_1; S_4, id)$ resolves to the following predicate, as all memory heap requirements are satisfied for that l_i :

$$(l_c \neq \ell_{ListElem}^0 \Rightarrow ((a_{nn} > a_i) \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn}) \Rightarrow (l_{nn} = l_c))) \wedge l_c = l_{cn}$$

For the universal quantifier for L_0 , we first establish that the predicate $\neg PrSp(l_i, ListElem, S_1; S_4, id)$ resolves to:

$$\begin{aligned}
& \neg(l_i, comp) \xrightarrow{1} l_c \vee (l_c \neq \ell_{ListElem}^0 \wedge \\
& \quad (\neg(l_i, val) \hookrightarrow a_i \vee \neg(l_c, val) \hookrightarrow a_c \vee \neg(l_i, newNext) \hookrightarrow l_{nn} \vee \\
& \quad \quad \neg(l_{nn}, val) \hookrightarrow a_{nn} \vee \\
& \quad ((a_c > a_i) \wedge ((l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn}) \wedge \\
& \quad \quad (\neg(l_i, newNext) \xrightarrow{1} l_{nn} \vee (l_{nn} \neq l_c)))) \\
&) \vee \neg(l_c, next) \xrightarrow{1} l_{cn} \vee l_c \neq l_{cn}
\end{aligned}$$

To distribute this over the separator, we use the insight that negated inclusion statements do in fact distribute down over separators. If something is not in the larger memory heap, it is not going to be in any of its parts. An disjunction of negated inclusions and pure statements then also distributes down over a separator, as whatever part of the disjunction makes the disjunction true will also hold for all parts of any split of the heap of the predicate.

As this is trivially true for all parts of the split induced by the separator that are not assigned to l_i , as those will not contain a heap entry $(l_i, comp) \xrightarrow{1} -$, the only clause any part of the predicate will then show up is the clause for l_i . For the part of the split of l_i , this resolves to the following predicate:

$$(l_c \neq \ell_{ListElem}^0 \wedge ((a_c > a_i) \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn}) \wedge (l_{nn} \neq l_c))) \vee l_c \neq l_{cn}$$

We then conclude with the predicate P_{E2} , given below:

$$\begin{aligned} P_{E2} = & (Vals = \{a_0, \dots, a_{n-1}\} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\ & (L = \{l_0, \dots, l_{n-1}\}) \wedge L_0 \cup L_1 = \emptyset \wedge L_0 \neq \emptyset \wedge \\ & (\otimes_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn} \in \mathcal{V}. (\\ & \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\ & \quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\ & \quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\ & \quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\ & \quad \wedge (j < n) \Rightarrow l_c = l_j \wedge ((j \geq n) \Rightarrow l_c = \ell_{ListElem}^0) \\ & \quad \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\ & \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall 0 \leq t < \min(j, n). (a_t > a_i \Rightarrow a_t > a_{nn}))) \\ & \quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < \min(j, n). a_t \leq a_i)) \\ & \quad \wedge (l_i \in L_0 \Rightarrow ((l_c \neq \ell_{ListElem}^0 \wedge ((a_c > a_i) \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn}) \wedge \\ & \quad (l_{nn} \neq l_c))) \vee l_c \neq l_{cn})) \\ & \quad \wedge (l_i \in L_1 \Rightarrow ((l_c \neq \ell_{ListElem}^0 \Rightarrow \\ & \quad ((a_{nn} > a_i) \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn}) \Rightarrow (l_{nn} = l_c))) \wedge l_c = l_{cn}))) \\ & * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2) \end{aligned}$$

7.5.3 Processing the step *compareElement*

To process the step *compareElement*, we first transform P_5 into P_6 :

$$\begin{aligned}
P_6 = & (\blacksquare_1(l_i, \text{ListElem}) \wedge \mathcal{O}_2) \wedge \\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} l_c * \\
& (l_c, \text{val}) \mapsto a_c * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i) * \\
& ((\otimes_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_c, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge ((j < n) \Rightarrow l_c = l_j \wedge ((j \geq n) \Rightarrow l_c = \ell_{\text{ListElem}}^0)) \\
& \wedge (l_c \neq \ell_{\text{ListElem}}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < \min(j, n)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < \min(j, n)}. a_t \leq a_i)) \wedge C
\end{aligned}$$

The only change between the two is the removal of the existential quantifier $\exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_j, a_c \in \mathcal{V}. (\dots)$.

Following this, we use the rule **STEP CALL**, for which we take P_6 to be P' as well, as $P_6 \rightarrow \blacksquare_1(l_i, \text{ListElem})$. We then use the **SEQUENCE** rule to split the statements S_1 and S_4 . We then use the **IF-STATEMENT** rule twice, first for S_1 and then for S_2 . After the first application, we have as precondition $P_6 \wedge l_c \neq \ell_{\text{ListElem}}^0$, and after the second application, we have the precondition $P_6 \wedge l_c \neq \ell_{\text{ListElem}}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_c < a_{nn})$. We use the if-conditions to simplify our precondition into P_7 .

$$\begin{aligned}
P_7 = & (\blacksquare_1(l_i, \text{ListElem}) \wedge \mathcal{O}_2) \wedge \\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} l_j * \\
& (l_j, \text{val}) \mapsto a_j * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i) * \\
& ((\otimes_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_j, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1})) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < j}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < j}. a_t \leq a_i)) \\
& \wedge C \wedge a_j > a_i \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_j < a_{nn})
\end{aligned}$$

Notably, we use that $l_c \neq \ell_{\text{ListElem}}^0$ to assert that $j < n$, which states that $l_c = l_j$, both of which we use to simplify our l_{cn} clause and the universal quantifiers.

While the rule requires us to verify that the precondition coupled with the negated if-condition implies the postcondition, we wait in specifying the exact

postcondition until after we have proven the triples required from the rules. This leads to the tree as depicted in Figure 7.4.

We then apply the update rule. For this, note that we can add the last five lines to the first line for the heap-free predicate, as the last five lines are pure. This results in P_8 , with the change in bold:

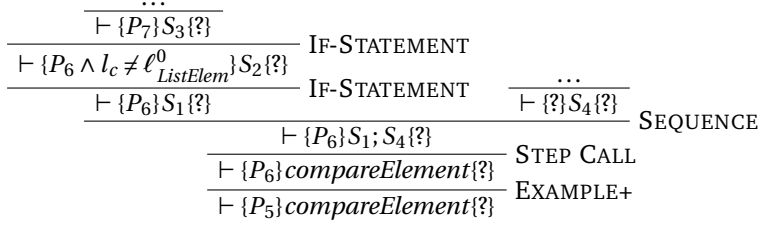
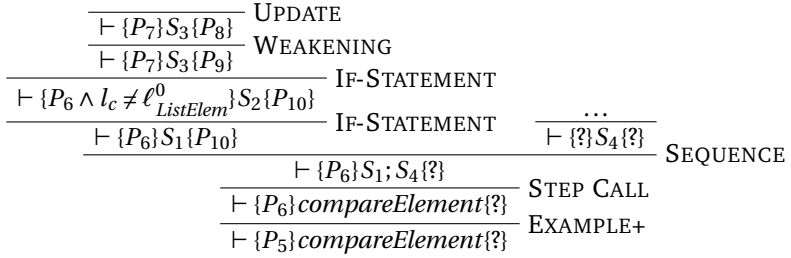
$$\begin{aligned}
P_8 = & (\blacksquare_1(l_i, \text{ListElem}) \wedge \oslash_2) \wedge \\
& (l_i, \text{newNext}) \xrightarrow{1} \mathbf{l_j} * (l_i, \text{comp}) \xrightarrow{1} l_j * \\
& (l_j, \text{val}) \mapsto a_j * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i) * \\
& ((\otimes_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_j, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1})) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < j}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < j}. a_t \leq a_i)) \\
& \wedge C \wedge a_j > a_i \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_j < a_{nn})
\end{aligned}$$

As we know that $a_j > a_i \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_j < a_{nn})$, we know that $(l_j \neq \ell_{\text{ListElem}}^0)$, as $a_j > a_i > 0$. Furthermore, as we know that $(l_{nn} \neq \ell_{\text{ListElem}}^0) \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < j}. (a_t > a_i \Rightarrow a_t < a_{nn}))$ and that $(l_{nn} = \ell_{\text{ListElem}}^0) \Rightarrow (\forall_{0 \leq t < j}. a_t \leq a_i)$, we know that $\forall_{0 \leq t \leq j}. (a_t > a_i \Rightarrow a_t > a_j)$, as $(l_{nn} = \ell_{\text{ListElem}}^0 \vee a_j < a_{nn})$. It follows that $(l_j \neq \ell_{\text{ListElem}}^0) \wedge (a_j > a_i \wedge \forall_{0 \leq t \leq j}. (a_t > a_i \Rightarrow a_t > a_j))$. We also know that $j + 1 \leq n + 1$, as $j < n$. We use this to weaken P_8 into P_9 :

$$\begin{aligned}
P_9 = & (\blacksquare_1(l_i, \text{ListElem}) \wedge \oslash_2) \wedge \\
& (l_i, \text{newNext}) \xrightarrow{1} l_j * (l_i, \text{comp}) \xrightarrow{1} l_j * \\
& (l_j, \text{val}) \mapsto a_j * (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i * \\
& ((\otimes_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \hookrightarrow l_{ne} \wedge (l_j, \text{next}) \hookrightarrow l_{cn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (((j = n - 1) \wedge l_{cn} = \ell_{\text{ListElem}}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1})) \\
& \wedge (l_j \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_j > a_i \wedge \forall_{0 \leq t < \min(j+1, n+1)}. (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
& \wedge (l_j = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < \min(j+1, n+1)}. a_t \leq a_i)) \\
& \wedge C \wedge a_j > a_i \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \vee a_j < a_{nn})
\end{aligned}$$

Here, we can weaken in $(l_j = \ell_{\text{ListElem}}^0) \Rightarrow (\forall_{0 \leq t < \min(j+1, n+1)}. a_t \leq a_i)$ because we know that $(l_j = \ell_{\text{ListElem}}^0)$ is false.

We can use this to determine the postcondition of the if-statements. We remove the if-conditions from P_9 , yielding P'_9 , and define the postcondition of

Figure 7.4: Proof Tree ∇_3 , first unfinished version.Figure 7.5: Proof Tree ∇_3 , second unfinished version.

the if-statement to be

$$\begin{aligned}
& (l_c \neq \ell_{ListElem}^0 \wedge a_j > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_j < a_{nn}) \wedge P'_9) \vee \\
& ((l_c = \ell_{ListElem}^0 \vee a_j \leq a_i \vee (l_{nn} \neq \ell_{ListElem}^0 \wedge a_j \geq a_{nn})) \wedge P'_6)
\end{aligned}$$

In this postcondition, P'_6 is weakened from P_6 by setting the counter in the variant-induced universal quantifiers to $j+1$.

It then follows that P_6 combined with $l_c = \ell_{ListElem}^0$ implies P'_6 , as that implies that $j \geq n$, which means that $\min(j, n) = \min(j+1, n) = n$. Furthermore, P_6 combined with $(a_c \leq a_i \vee (l_{nn} \neq \ell_{ListElem}^0 \wedge a_c \geq a_{nn}))$ implies P'_6 , as this negated condition implies that l_{nn} is a better choice for *newNext* than l_c , which means that we can expand the universal quantifiers induced by the variant to $j+1$ and they will still hold.

We conclude that P_6 combined with either negated if-conditions implies this postcondition, so the postcondition suffices. We designate the postcondition as P_{10} , and continue our proof tree as shown in Figure 7.5.

To resolve S_4 , we first weaken P_{10} to P_{11} , as given below.

$$\begin{array}{c}
\frac{}{\vdash \{P_7\}S_3\{P_8\}} \text{UPDATE} \\
\frac{}{\vdash \{P_7\}S_3\{P_9\}} \text{WEAKENING} \\
\frac{}{\vdash \{P_6 \wedge l_c \neq \ell_{ListElem}^0\}S_2\{P_{10}\}} \text{IF-STATEMENT} \\
\frac{}{\vdash \{P_6\}S_1\{P_{10}\}} \text{IF-STATEMENT} \\
\frac{}{\vdash \{P_{11}\}S_4\{P_{12}\}} \text{UPDATE} \\
\frac{}{\vdash \{P_{10}\}S_4\{P_{12}\}} \text{WEAKENING} \\
\frac{}{\vdash \{P_6\}S_1; S_4\{P_{12}\}} \text{SEQUENCE} \\
\frac{}{\vdash \{P_6\}compareElement\{P_{12}\}} \text{STEP CALL} \\
\frac{}{\vdash \{P_5\}compareElement\{P_{13}\}} \text{EXAMPLE+}
\end{array}$$

Figure 7.6: **Finished:** Proof Tree ∇_3

$$\begin{aligned}
P_{11} = & (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_u \in \mathcal{L} \exists a_u \in \mathcal{V}. (\\
& (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_j * (l_u, val) \mapsto a_u * \\
& (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& ((\otimes_{0 \leq q < n} (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \wedge ((l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_c \wedge a_u = a_c) \\
& \wedge (\neg(l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_{nn} \wedge a_u = a_{nn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge ((j < n \Rightarrow l_c = l_j) \wedge (j \geq n \Rightarrow l_c = \ell_{ListElem}^0)) \\
& \wedge (l_u \neq \ell_{ListElem}^0 \Rightarrow (a_u > a_i \wedge \forall_{0 \leq t < \min(j+1, n)}. (a_t > a_i \Rightarrow a_t > a_u))) \\
& \wedge (l_u = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(j+1, n)}. a_t \leq a_i))) \\
& \wedge C
\end{aligned}$$

Here, we add the separation clause $(l_u, val) \mapsto a_u$, which can be done because $l_u = l_c \vee l_u = l_{nn}$, and for both of these there are permissions in the heap. Both of these permissions are not full permissions, nor are they permissions that need to be defined, so we can implicitly split the permission corresponding to the permission of (l_u, val) without knowing the permission or the exact label l_u is.

We then apply the update rule on the memory assertion $(l_i, comp) \xrightarrow{1} l_j$ to replace it with $(l_i, comp) \xrightarrow{1} l_{cn}$, yielding P_{12} . This is once again done by considering all lines starting with a $\wedge$ a part of the heap-free predicate.

We add all the other existential quantifiers from P_5 back in to get P_{13} :

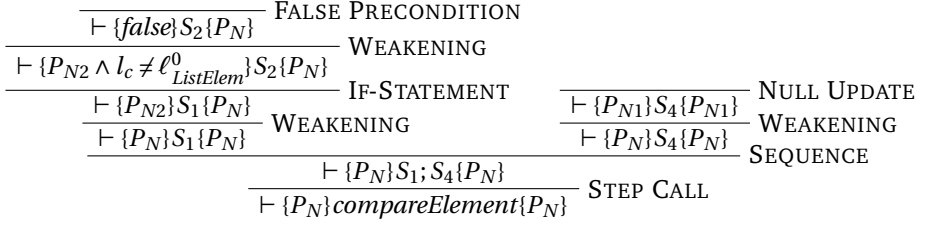


Figure 7.7: **Finished:** Proof Tree ∇_4 , for the execution of *compareElement* by the null-instance.

$$\begin{aligned}
P_{13} = & (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_u, l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, a_u \in \mathcal{V}. (\\
& (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_{cn} * (l_u, val) \mapsto a_u * \\
& (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \wedge ((l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_c \wedge a_u = a_c) \\
& \wedge (\neg(l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_{nn} \wedge a_u = a_{nn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge ((j < n \Rightarrow l_c = l_j) \wedge (j \geq n \Rightarrow l_c = \ell_{ListElem}^0)) \\
& \wedge (l_u \neq \ell_{ListElem}^0 \Rightarrow (a_u > a_i \wedge \forall_{0 \leq t < \min(j+1, n)}. (a_t > a_i \Rightarrow a_t > a_u))) \\
& \wedge (l_u = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(j+1, n)}. a_t \leq a_i))) \\
& \wedge C
\end{aligned}$$

with this, we finish this part of the proof tree, as shown in Figure 7.6.

We then return to part ∇_2 of the proof tree. For this part, we still need to prove what happens when the null-instance executes the step. This can be adapted from ∇_3 , as seen in ∇_4 as given in Figure 7.7. In the figure, we use P_N to denote the predicate $\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2$, we use P_{N1} to denote the predicate $P_N \wedge true * (\ell_{ListElem}^0, comp) \xrightarrow{\zeta_0} \ell_{ListElem}^0$ and we use P_{N2} to denote the predicate $P_N \wedge \exists l_c \in \mathcal{L}. (true * (\ell_{ListElem}^0, comp) \xrightarrow{\zeta_0} l_c)$. Furthermore, note that we can weaken P_N into P_{N1} and P_{N1} and P_N because we only add or remove null parameters, which are by default included in any heap. As the next step, we have to work towards the postcondition of the fixpoint.

$$\frac{\frac{\frac{\nabla_3}{\vdash \{P_5\}compareElement\{P_{13}\}}}{\vdash \{P_4\}compareElement\{P_{14}\}} \text{ GP} \quad \frac{\nabla_4}{\vdash \{P_N\}compareElement\{P_N\}} \text{ SP}}{\vdash \{P_3\}compareElement\{P_{15}\}} \text{ SP} \\
\vdash \{P_2 \wedge Va(j) \wedge \neg stable_P(compareElement)\}compareElement\{P_2 \wedge Va(j+1)\} \text{ WK}$$

Figure 7.8: **Finished:** Proof Tree ∇_2 , where WK stands for “WEAKENING”, SP stands for SPLIT PARALLEL and GP stands for GEN PARALLEL.

7.5.4 Back to the fixpoint postcondition

We need to align the postcondition from executing the step back into the postcondition of the fixpoint. To do this, we first add the big separator back to P_{13} to yield P_{14} , and add in ∇_4 and its postcondition into ∇_2 . Recombining P_{14} with the null-instance then leads to P_{15} .

$$\begin{aligned}
P_{15} = & \textcircled{*}_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_u, l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, a_u \in \mathcal{V}. (\\
& (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_{cn} * (l_u, val) \mapsto a_u * \\
& (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
& ((\textcircled{*}_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
& \wedge ((l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_c \wedge a_u = a_c) \\
& \wedge (\neg(l_c \neq \ell_{ListElem}^0 \wedge a_c > a_i \wedge (l_{nn} = \ell_{ListElem}^0 \vee a_c < a_{nn})) \Rightarrow l_u = l_{nn} \wedge a_u = a_{nn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
& \wedge ((j < n \Rightarrow l_c = l_j) \wedge (j \geq n \Rightarrow l_c = \ell_{ListElem}^0)) \\
& \wedge (l_u \neq \ell_{ListElem}^0 \Rightarrow (a_u > a_i \wedge \forall_{0 \leq t < \min(j+1, n)}. (a_t > a_i \Rightarrow a_t > a_u))) \\
& \wedge (l_u = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < \min(j+1, n)}. a_t \leq a_i))) \\
& \wedge C) * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

We update the proof tree of ∇_2 as seen in Figure 7.8. Though the tree is finished, we still have to prove it is valid, by showing that $P_{15} \Rightarrow P_2 \wedge Va(j+1)$. For this, we first define $P_2 \wedge Va(j+1)$, given below.

$$\begin{aligned}
P_2 \wedge Va(j+1) &= (Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \forall 0 \leq x < n. a_x > 0) \wedge \\
&\quad (L = l_0; \dots; l_{n-1}) \wedge \\
&\quad \otimes_{0 \leq i < n} (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_c, l_{cn}, l_{nn} \in \mathcal{L}. \exists a_{nn}, a_c, \in \mathcal{V}. (\\
&\quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} l_c * \\
&\quad (l_c, val) \mapsto a_c * (l_{nn}, val) \mapsto a_{nn} * (l_i, val) \mapsto a_i * \\
&\quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
&\quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
&\quad \wedge (j + 1 < n \Rightarrow l_c = l_{j+1} \wedge ((j + 1 \geq n) \Rightarrow l_c = \ell_{ListElem}^0)) \\
&\quad \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j + 1 = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j + 1 < n - 1) \wedge l_{cn} = l_{j+2}))) \\
&\quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall 0 \leq t < \min(j+1, n). (a_t > a_i \Rightarrow a_t > a_{nn}))) \\
&\quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < \min(j+1, n). a_t \leq a_i))) \\
&\quad * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

The first thing we need to do to transform P_{15} into $P_2 \wedge Va(j+1)$ is to replace l_{nn} with l_u , the new *newNext*. This can be done by weakening away the first two $\wedge$ clauses and redistributing the permission for (l_{nn}, val) over the big separator and the remaining heap in the clause for $(\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)$. This redistribution is only possible due to the fact that (l_{nn}, val) does not have full permission in any clause. At that point, the l_{nn} and a_{nn} variables are not used and can be removed. The result is P_{16} , given below.

$$\begin{aligned}
P_{16} &= \otimes_{0 \leq i < n} (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_u, l_{ne}, l_c, l_{cn}, \in \mathcal{L}. \exists a_c, a_u \in \mathcal{V}. (\\
&\quad (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_{cn} * \\
&\quad (l_c, val) \mapsto a_c * (l_u, val) \mapsto a_u * (l_i, val) \mapsto a_i * \\
&\quad ((\otimes_{0 \leq q < n}. (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \hookrightarrow l_{ne} \wedge (l_c, next) \hookrightarrow l_{cn}) \\
&\quad \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
&\quad \wedge ((j < n \Rightarrow l_c = l_j) \wedge (j \geq n \Rightarrow l_c = \ell_{ListElem}^0)) \\
&\quad \wedge (l_c \neq \ell_{ListElem}^0 \Rightarrow (((j = n - 1) \wedge l_{cn} = \ell_{ListElem}^0) \vee ((j < n - 1) \wedge l_{cn} = l_{j+1}))) \\
&\quad \wedge (l_u \neq \ell_{ListElem}^0 \Rightarrow (a_u > a_i \wedge \forall 0 \leq t < \min(j+1, n). (a_t > a_i \Rightarrow a_t > a_u))) \\
&\quad \wedge (l_u = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < \min(j+1, n). a_t \leq a_i))) \\
&\quad \wedge C) * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

Then, if $j + 1 < n$, then $j < n$, $l_c = l_j$ and $l_{cn} = l_{j+1}$. Furthermore, if $j + 1 \geq n$, if $j \geq n$ then $l_c = \ell_{ListElem}^0$ and $l_{cn} = \ell_{ListElem}^0$, and if $j = n - 1$, then $l_c = l_n$ and

$l_{cn} = \ell_{ListElem}^0$. Then $P_{16} \Rightarrow ((j+1 < n \Rightarrow l_{cn} = l_{j+1}) \wedge (j+1 \geq n \Rightarrow l_{cn} = \ell_{ListElem}^0))$. We replace the second and third $\wedge$ -clauses of P_{16} with the above conclusion. We also redistribute the permissions of the *ListElems* to replace $(l_c, val) \mapsto a_c$ with $(l_{cn}, val) \mapsto a_{cn}$ and $(l_c, next) \mapsto l_{cn}$ with $(l_{cn}, next) \mapsto l_{cnn}$ and remove l_c and a_c , which leads to P_{17} :

$$\begin{aligned}
P_{17} = & \textcircled{*}_{0 \leq i < n} (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_{cn}, l_{cnn}, l_u \in \mathcal{L}. \exists a_u, a_{cn} \in \mathcal{V}. \\
& (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_{cn} * \\
& (l_{cn}, val) \mapsto a_{cn} * (l_u, val) \mapsto a_u * (l_i, val) \mapsto a_i) * \\
& (((\textcircled{*}_{0 \leq q < n} (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \mapsto l_{ne} \wedge (l_{cn}, next) \mapsto l_{cnn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge ((j + 1 < n) \Rightarrow l_{cn} = l_{j+1}) \wedge ((j \geq n) \Rightarrow l_{cn} = \ell_{ListElem}^0) \\
& \wedge (l_u \neq \ell_{ListElem}^0) \Rightarrow (a_u > a_i \wedge \forall_{0 \leq t < \min(j+1, n)}. (a_t > a_i \Rightarrow a_t > a_u)) \\
& \wedge (l_u = \ell_{ListElem}^0) \Rightarrow (\forall_{0 \leq t < \min(j+1, n)}. a_t \leq a_i) \\
& \wedge C) * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

Then, we can also infer from the entire separator, which requires that $((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})$ for all l_i , that $(l_{cn} \neq \ell_{ListElem}^0) \Rightarrow ((j + 1 = n - 1) \wedge l_{cnn} = \ell_{ListElem}^0) \vee ((j + 1 < n - 1) \wedge l_{cnn} = l_{j+2})$ holds, leading to P_{18} :

$$\begin{aligned}
P_{18} = & \textcircled{*}_{0 \leq i < n} (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{ne}, l_{cn}, l_{cnn}, l_u \in \mathcal{L}. \exists a_u, a_{cn} \in \mathcal{V}. \\
& (l_i, newNext) \xrightarrow{1} l_u * (l_i, comp) \xrightarrow{1} l_{cn} * \\
& (l_{cn}, val) \mapsto a_{cn} * (l_u, val) \mapsto a_u * (l_i, val) \mapsto a_i) * \\
& (((\textcircled{*}_{0 \leq q < n} (l_q, next) \xrightarrow{1/n} -) \wedge (l_i, next) \mapsto l_{ne} \wedge (l_{cn}, next) \mapsto l_{cnn}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (l_{cn} \neq \ell_{ListElem}^0) \Rightarrow (((j + 1 = n - 1) \wedge l_{cnn} = \ell_{ListElem}^0) \vee ((j + 1 < n - 1) \wedge l_{cnn} = l_{j+2})) \\
& \wedge ((j + 1 < n) \Rightarrow l_{cn} = l_{j+1}) \wedge ((j + 1 \geq n) \Rightarrow l_{cn} = \ell_{ListElem}^0) \\
& \wedge (l_u \neq \ell_{ListElem}^0) \Rightarrow (a_u > a_i \wedge \forall_{0 \leq t < \min(j+1, n)}. (a_t > a_i \Rightarrow a_t > a_u)) \\
& \wedge (l_u = \ell_{ListElem}^0) \Rightarrow (\forall_{0 \leq t < \min(j+1, n)}. a_t \leq a_i) \\
& \wedge C) * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

The only difference between P_{18} and $P_2 \wedge Va(j+1)$ are the variable names, so we can conclude that $P_{18} \Rightarrow P_2 \wedge Va(j+1)$. With that, the proof tree ∇_2 is done and validated.

To finish proof tree ∇_1 , we know the postcondition of the fixpoint is $P_2 \wedge Va(k) \wedge \text{stable}_{\mathcal{P}}(\text{compareElement})$. As $P_2 \wedge Va(k) \Rightarrow \text{stable}_{\mathcal{P}}(\text{compareElement})$, we

$$\frac{\frac{\nabla_2}{\frac{\vdash \{P_2 \wedge Va(j) \wedge \neg \text{stable}_P(\text{compareElement})\} \text{compareElement} \{P_2 \wedge Va(j+1)\}}{\vdash \{P_2\} \text{Fix}(\text{compareElement}) \{P_2 \wedge Va(k) \wedge \text{stable}_P(\text{compareElement})\}} \text{FP}}{\vdash \{P_1\} \text{Fix}(\text{compareElement}) \{P_{19}\}} \text{WK}$$

Figure 7.9: **Finished:** Proof Tree ∇_1 . Here, WK stands for WEAKENING and FP stands for FIXPOINT.

just use $P_2 \wedge Va(k)$ as a postcondition, which we update slightly to put the constants in the separator. This results in P_{19} :

$$\begin{aligned}
P_{19} = & \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{ne}, l_{nn} \in \mathcal{L}. \exists a_{nn} \in \mathcal{V}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * \\
& (l_{nn}, \text{val}) \mapsto a_{nn} * (l_i, \text{val}) \mapsto a_i * \\
& ((\otimes_{0 \leq q < n}. (l_q, \text{next}) \xrightarrow{1/n} -) \wedge (l_i, \text{next}) \mapsto l_{ne}) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < n}. a_t \leq a_i))) \\
& \wedge C) * (\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2)
\end{aligned}$$

With this, we finish proof tree ∇_1 , as seen in Figure 7.9. This concludes the first part of the schedule of Linear Time Sort.

7.5.5 Processing the step *reorder*

With the fixpoint done, we prove the postcondition of the step *reorder*. We first use the intermediate postcondition of ∇_1 to update ∇_0 , as seen in Figure 7.10. We then weaken P_{19} into P_{20} by redistributing some permissions over the separator, specifically the permission of (l_i, next) for all $l_i \in L$.

$$\begin{aligned}
P_{20} = & \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{ne}, l_{nn} \in \mathcal{L}. \exists a_{nn}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * (l_i, \text{next}) \xrightarrow{1} l_{ne} * \\
& (l_i, \text{val}) \mapsto a_i * (l_{nn}, \text{val}) \mapsto a_{nn} \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < n}. a_t \leq a_i))) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{\text{ListElem}}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1})) \\
& \wedge C) * (\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2)
\end{aligned}$$

$$\frac{\frac{\frac{\nabla_1}{\vdash \{P_1\}\text{Fix}(\text{compareElement})\{P_{19}\}}} \quad \frac{\frac{\nabla_5}{\vdash \{P_{20}\}\text{reorder}\{?\}}}{\vdash \{P_{19}\}\text{reorder}\{?\}} \text{WEAKENING}}{\vdash \{P_1\}\text{Fix}(\text{compareElement}) < \text{reorder}\{?\}} \text{BARRIER}}{\frac{\vdash \{P_1\}\text{Fix}(\text{compareElement}) < \text{reorder}\{?\}}{\{P_0\}\text{Fix}(\text{compareElement}) < \text{reorder}\{?\}} \text{EXAMPLE+}}$$

Figure 7.10: Proof Tree ∇_0 , second unfinished version

$$\frac{\frac{\frac{\dots}{\vdash \{P_{23}\}\text{reorder}\{?\}}}{\vdash \{P_{22}\}\text{reorder}\{?\}} \text{EXAMPLE+}}{\vdash \{P_{21}\}\text{reorder}\{?\}} \text{GEN PARALLEL} \quad \frac{\frac{\dots}{\vdash \{P_N\}\text{reorder}\{?\}}}{\vdash \{P_{20}\}\text{reorder}\{?\}} \text{SPLIT PARALLEL}}{\vdash \{P_{20}\}\text{reorder}\{?\}}$$

Figure 7.11: Proof Tree ∇_5 , first unfinished version

With this, we update proof tree ∇_0 , as also shown in Figure 7.10.

We then prove the postcondition of the step *reorder* when executed with the precondition P_{20} . For this, we first use the rule SPLIT PARALLEL to split P_{20} into P_{21} and P_N (where $P_N = (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)$, as defined earlier). We then use the rule GEN PARALLEL to split P_{21} into a predicate P_{22} for a single struct instance l_i . This is analogous to how we split the precondition for executing *compareElement*. We then remove the existential quantifiers using the rule EXAMPLE, leading to P_{23} as given below and the proof tree depicted in Figure 7.11.

$$\begin{aligned}
P_{23} = & (\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge (\\
& (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} \ell_{ListElem}^0 * (l_i, next) \xrightarrow{1} l_{ne} * \\
& (l_i, val) \mapsto a_i * (l_{nn}, val) \mapsto a_{nn} \\
& \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall_{0 \leq t < n}. a_t \leq a_i)) \\
& \wedge (((i = n - 1) \wedge l_{ne} = \ell_{ListElem}^0) \vee ((i < n - 1) \wedge l_{ne} = l_{i+1}))) \wedge C
\end{aligned}$$

We use STEP CALL, with P_{23} as P' , to unfold *reorder* into S_5 . We then use the UPDATE rule to process S_5 , resulting in P_{24} . We weaken the clause about l_{ne} away now l_{ne} has been replaced, resulting in P_{25} , as given below.

$$\begin{array}{c}
\frac{}{\vdash \{P_{23}\} S_5 \{P_{24}\}} \text{UPDATE} \\
\frac{}{\vdash \{P_{23}\} S_5 \{P_{25}\}} \text{WEAKENING} \\
\frac{}{\vdash \{P_{23}\} \text{reorder}\{P_{25}\}} \text{STEP CALL} \\
\frac{}{\vdash \{P_{22}\} \text{reorder}\{P_{26}\}} \text{EXAMPLE+} \\
\frac{}{\vdash \{P_{22}\} \text{reorder}\{P_{27}\}} \text{WEAKENING} \\
\frac{}{\vdash \{P_{21}\} \text{reorder}\{P_{28}\}} \text{GEN PARALLEL} \quad \frac{\dots}{\vdash \{P_N\} \text{reorder}\{?\}} \text{SPLIT PARALLEL} \\
\hline
\vdash \{P_{20}\} \text{reorder}\{?\}
\end{array}$$

Figure 7.12: Proof Tree ∇_5 , second unfinished version

$$\begin{aligned}
P_{25} = & (\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * (l_i, \text{next}) \xrightarrow{1} \mathbf{l}_{nn} * \\
& (l_i, \text{val}) \mapsto a_i * (l_{nn}, \text{val}) \mapsto a_{nn} \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \wedge C
\end{aligned}$$

Putting the existential quantifiers back results in P_{26} . We then weaken the quantifier of l_{ne} away again using weakening, resulting in P_{27} . Lastly, we readd the big separator to make P_{28} , as given below:

$$\begin{aligned}
P_{28} = & \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{nn} \in \mathcal{L}. \exists a_{nn}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * (l_i, \text{next}) \xrightarrow{1} \mathbf{l}_{nn} * \\
& (l_i, \text{val}) \mapsto a_i * (l_{nn}, \text{val}) \mapsto a_{nn} \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \wedge C)
\end{aligned}$$

This leads to the proof tree depicted in Figure 7.12.

For P_N , we also call the step, after which we weaken P_N to $P_{N1} = P_N \wedge \text{true} * (\ell_{\text{ListElem}}^0, \text{next}) \xrightarrow{\zeta_0} \ell_{\text{ListElem}}^0$. We then use the NULL UPDATE rule, and reverse our weakenings. We then recombine P_N and P_{28} , resulting in P_{29} :

$$\begin{aligned}
P_{29} = & \otimes_{0 \leq i < n} ((\blacksquare_1(l_i, \text{ListElem}) \wedge \emptyset_2) \wedge \exists l_{nn} \in \mathcal{L}. \exists a_{nn}. (\\
& (l_i, \text{newNext}) \xrightarrow{1} l_{nn} * (l_i, \text{comp}) \xrightarrow{1} \ell_{\text{ListElem}}^0 * (l_i, \text{next}) \xrightarrow{1} l_{nn} * \\
& (l_i, \text{val}) \mapsto a_i * (l_{nn}, \text{val}) \mapsto a_{nn} \\
& \wedge (l_{nn} \neq \ell_{\text{ListElem}}^0 \Rightarrow (a_{nn} > a_i \wedge \forall_{0 \leq t < n}. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \wedge (l_{nn} = \ell_{\text{ListElem}}^0 \Rightarrow (\forall_{0 \leq t < n}. a_t \leq a_i))) \\
& \wedge C) * (\blacksquare_1(\ell_{\text{ListElem}}^0, \text{ListElem}) \wedge \emptyset_2)
\end{aligned}$$

This results in the finished proof tree for ∇_5 , depicted in Figure 7.13.

$\frac{}{\vdash \{P_{23}\}S_5\{P_{24}\}}$	UPDATE		
$\frac{}{\vdash \{P_{23}\}S_5\{P_{25}\}}$	WEAKENING		
$\frac{}{\vdash \{P_{23}\}reorder\{P_{25}\}}$	STEP CALL	$\frac{}{\vdash \{P_{N1}\}S_5\{P_{N1}\}}$	NULL UPDATE
$\frac{}{\vdash \{P_{22}\}reorder\{P_{26}\}}$	EXAMPLE+	$\frac{}{\vdash \{P_N\}S_5\{P_N\}}$	WEAKENING
$\frac{}{\vdash \{P_{21}\}reorder\{P_{27}\}}$	GEN PARALLEL	$\frac{}{\vdash \{P_N\}reorder\{P_N\}}$	STEP CALL
			SPLIT PARALLEL
$\vdash \{P_{20}\}reorder\{P_{28}\}$			

Figure 7.13: **Finished:** Proof Tree ∇_5

$\frac{}{\vdash \{P_1\}Fix(compareElement)\{P_{19}\}}$	$\frac{}{\vdash \{P_{20}\}reorder\{P_{29}\}}$	$\frac{}{\vdash \{P_{19}\}reorder\{P_{30}\}}$	WEAKENING
			BARRIER
$\vdash \{P_1\}Fix(compareElement) < reorder\{P_{30}\}$			
$\{P_0\}Fix(compareElement) < reorder\{P_{31}\}$			
			EXAMPLE+

Figure 7.14: **Finished:** Proof Tree ∇_0

7.5.6 Conclusion of the Proof

We conclude by using P_{29} to finish proof tree ∇_0 , depicted in Figure 7.14. Here, we weaken P_{29} to P_{30} to move C back outside of the separator. We then return the existential quantifiers $Vals$ and L , which results in our final postcondition P_{31} , as given below.

$$\begin{aligned}
P_{31} = & \exists Vals: \mathbb{Z} \rightarrow \mathcal{V}. Vals = a_0; \dots; a_{n-1} \wedge \forall 0 \leq x < y < n. a_x \neq a_y \wedge \\
& \forall 0 \leq x < n. a_x > 0 \wedge \exists L \subset \mathbb{Z} \rightarrow \mathcal{L}. L = l_0; \dots; l_{n-1} \wedge \\
& \textcircled{*}_{0 \leq i < n} ((\blacksquare_1(l_i, ListElem) \wedge \emptyset_2) \wedge \exists l_{nn} \in \mathcal{L}. \exists a_{nn}. (\\
& \quad (l_i, newNext) \xrightarrow{1} l_{nn} * (l_i, comp) \xrightarrow{1} \ell_{ListElem}^0 * (l_i, next) \xrightarrow{1} l_{nn} * \\
& \quad (l_i, val) \mapsto a_i * (l_{nn}, val) \mapsto a_{nn} \\
& \quad \wedge (l_{nn} \neq \ell_{ListElem}^0 \Rightarrow (a_{nn} > a_i \wedge \forall 0 \leq t < n. a_t > a_i \Rightarrow a_t > a_{nn})) \\
& \quad \wedge (l_{nn} = \ell_{ListElem}^0 \Rightarrow (\forall 0 \leq t < n. a_t \leq a_i))) \\
& * (\blacksquare_1(\ell_{ListElem}^0, ListElem) \wedge \emptyset_2)
\end{aligned}$$

Due to the two properties on l_{nn} for every l_i and the fact that all values are distinct, it follows that for every l_i , a_{nn} is the next sorted value after a_i in a list sorted ascendingly. It follows that after executing the partial program of Listing 3.6, the list is sorted.

7.6 Conclusion

In this section, we have given the syntax and semantics of AuDaLogic, a proof system built for AuDaLa built on separation logic and the work by Vafeiadis [173]. AuDaLogic allows for the splitting of a fictional memory, containing all parameters and local variables of an AuDaLa program, as well as the splitting of a *label triple*, to be able to focus on specific active struct instances. It furthermore places a focus on reasoning about the environment, which is reflected in its use of higher-order logic, and it does not use resource invariants, as AuDaLa does not use locks.

In AuDaLogic, we make use of predicate generation, for example to relate AuDaLa expressions to AuDaLogic expressions, or to assert that a schedule is stable, or to define a variant for fixpoints. For the first two of these, we have given the methods or the functions to generate the predicate. As fixpoint variants are fixpoint dependent, we did not give a concrete function for that one.

As well as the formal language, we have also defined proof rules for AuDaLogic, which showed the separation between schedule and steps we are accustomed to from Chapter 5. We have furthermore given a proof rule for fixpoint correctness. As fixpoints are not as controllable as for-loops, we have made this rule require total correctness, by requiring a variant generation function from the user.

We have lastly given an extensive example of AuDaLogic in application, during which multiple situations occur which we consider typical for AuDaLogic, like handling the fixpoint, performing a variable update and decomposing the schedule and the active struct instances.

In the next chapter, we will formally connect AuDaLogic to AuDaLa, by relating the formal language of AuDaLogic to the states of AuDaLa and the proof rules to executions. We will then prove AuDaLogic sound.

Chapter 8

Proving AuDaLogic Sound

In this chapter, we prove that the proof system introduced in Chapter 7 is sound. To do this, first we bridge the gap between the semantics of AuDaLa and the semantics of AuDaLogic in Section 8.1. Then, we prove that our processing of AuDaLa syntax is correct by proving that the *eqi* transformation and the *stable* function is correct in Section 8.2. We then give the soundness proof of the proof rules in Section 8.3 and conclude the chapter in Section 8.4.

8.1 Connecting the semantics of AuDaLa and AuDaLogic

The definition of AuDaLogic creates a lot of constructs to reason about AuDaLa. However, to use these to reason about AuDaLa and to prove that AuDaLogic can indeed be used to prove AuDaLa programs, we should formalize what AuDaLogic constructs enforce in AuDaLa executions.

In this section, we connect the AuDaLogic semantics and structures to AuDaLa semantics, and define how the choice of AuDaLogic context impacts the AuDaLa semantics to be considered. We start with connecting AuDaLa states to AuDaLogic states. We then link AuDaLa executions to states and give some more definitions to reason about AuDaLa executions. We use this to define the notions of safety for transitions and a safety predicate for statements. We then define the safety predicate for the schedule, which we use for a formal definition of an AuDaLogic triple.

Connecting AuDaLa and AuDaLogic states

Firstly, we connect AuDaLa states with AuDaLogic states, thereby enabling us to reason about AuDaLa states using AuDaLogic. Informally, for an AuDaLogic state to be equivalent to an AuDaLa state, we want every active struct instance label in $\alpha(1)$ of the AuDaLogic state to be reflected by a struct instance in the AuDaLa state, we want every newly created struct instance label in $\alpha(2)$ to be reflected by an inactive struct instance in the AuDaLa state, and we want all labels in $\alpha(3)$ to be unused in the AuDaLa state. Furthermore, we want all heap cells in the heap to be reflected by the AuDaLa state. We formalise the correspondence between AuDaLogic states and the AuDaLa states as follows.

Definition 8.1.1 (State correspondence). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let (α, h, ς) be an AuDaLogic state. Then we define that p is *struct-equivalent* to α , notation $p \simeq \alpha$, iff for all labels $\ell \in \mathcal{L}$ and all struct types $\theta \in \Theta_{\mathcal{P}}$, all of the following hold:

1. If $(\ell, \theta) \in \alpha(1)$, then $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ for some γ, χ, ξ ,
2. If $(\ell, \theta) \in \alpha(2)$, then $\sigma(\ell) = \langle \theta, \varepsilon, \varepsilon, \xi \rangle$ for some ξ and
3. If $\ell_3 \in \alpha(3)$, $\sigma(\ell_3) = \perp$.

Furthermore, we define that p is *memory-equivalent* to h , notation $p \simeq h$, iff for all labels $\ell \in \mathcal{L}$ and variable names $x \in ID_V$, the following holds:

1. $h(\ell, x) = (\zeta, v)$ iff $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ for some $\theta, \gamma, \chi, \xi$ such that $\xi(x) = v$.

We also define that a memory heap h' is *derived* from p , notation $p > h'$, if $p \simeq h$ and $h' \sqsubseteq h$ for some memory heap h . We define $\Xi(h)$ to be the set of AuDaLa states p such that $p > h'$, and say that p is derived from h' iff $p \in \Xi(h')$.

We then extend the equivalence and the derivation relations to full AuDaLogic states and define that $p \simeq (\alpha, h, \varsigma)$ iff $p \simeq \alpha$ and $p \simeq h$, and that $p > (\alpha, h, \varsigma)$ iff $p \simeq \alpha$ and $p > h$. Additionally, we define that $p \in \Xi(\alpha, h, \varsigma)$ iff $p \in \Xi(h)$.

From the definition it follows that $p \simeq (\alpha, h, \varsigma)$ iff $p \simeq (m(\alpha), h, \varsigma)$ and that $p > (\alpha, h, \varsigma)$ iff $p > (m(\alpha), h, \varsigma)$.

As α contains the labels we consider in the AuDaLogic state, which translates better to program behaviour than to states, the impact of α is limited in this definition; we do not require that there exist no other struct instances in the AuDaLa state than those in α . Likewise, as ς is only relevant for the interpretation of AuDaLogic variables and has no meaning for AuDaLa states, ς is not relevant to this definition.

State correspondence gives a link between static AuDaLa and AuDaLogic states, but to reason about how states relate to each other we also require a link between AuDaLa behaviour and AuDaLogic. For this, we define AuDaLogic triples, using AuDaLa executions.

AuDaLa executions preliminaries and labelling

As defined in Chapter 4, with the semantics defined for a program $\mathcal{P}$, an AuDaLa execution from a state p refers to a possibly infinite chain of semantic rule transitions that start in a state p . Following Theorem 4.2.21, we know that these executions consist of alternating finite subsequences of schedule transitions (which are induced by the schedule of $\mathcal{P}$) and step executions, which are also finite. More specifically, an AuDaLa execution of a step F from p refers to a chain of rule transitions from p starting with an **Init**-transition for F and ending in a state that contains a struct environment σ for which $Done(\sigma)$ holds. A step F can be executed from p iff there exists an execution of F from p . As there are two different **Init**-transitions, we know that there are *local* step executions, which start with **InitL** and are only executed by a single struct type, and *global* step executions, which start with **InitG** and are executed by all struct types.

An execution of a step F consists of many possibly interleaving executions of the statements of F . An AuDaLa execution of an expression E or a statement S by some struct instance s from p refers to a chain of rule transitions from p starting with the transition enabled by the first command of $\llbracket E \rrbracket$ or $\llbracket S \rrbracket$ for s and ending with the transition enabled by the last command of $\llbracket E \rrbracket$ or $\llbracket S \rrbracket$ for s respectively. Additionally, for every command in $\llbracket E \rrbracket$ or $\llbracket S \rrbracket$, there needs to be a corresponding transition induced by that command taken by s . An expression E or a statement S can be executed from p by a struct instance s iff there exists an execution of, respectively, E or S from p for s . This is straightforwardly extended to lists of expressions $\mathcal{E}$ and lists of statements $\mathcal{S}$.

To relate the executed transitions induced by commands during executions, we can annotate every command transition with information relevant for the transition. We therefore annotate every command transition with an *execution label*, which is the label of the struct instance which executes the transition. We annotate every **ComRd** and **ComWr**-induced transition with $(\ell_1, \ell_2, x, v, \theta)$, where ℓ_1 is the label of the struct instance which executes the transition, the transition accesses the variable (ℓ_2, x) , the value written or read is v , and the struct type of ℓ_2 is θ . A **ComCons**-induced transition is annotated as $(\ell_1, \ell_2, \theta, v_1, \dots, v_e)$, with ℓ_1 the label of the struct instance which executes the transition, ℓ_2 as the new label of the newly created struct instance, θ as the struct type of the newly created struct instance and $v_1, \dots, v_e$ being the values assigned to the parameters $x_1, \dots, x_e \in Pars_{\mathcal{P}}(\theta)$. Note that transitions induced by the schedule are not annotated.

Preliminary definitions for the safety predicates

We use the annotation information of AuDaLa executions to define the semantic meaning of AuDaLogic triples. To do this, note that AuDaLogic proof

rules take their AuDaLa code to either be a statement, a list of statements or a schedule. We therefore define the semantic meaning of AuDaLogic triples using two *safety predicates*, one for a list of statements and one for a schedule, following the safety predicate example of Vafeiadis [173]. We generally refer to the safety predicate for lists of statements as the *statements predicate*, and to the safety predicate for schedules as the *schedule predicate*.

To define these safety predicates, we define one more correspondence between behaviour of AuDaLa and AuDaLogic, which specifies when a label triple α has actual bearing on an AuDaLa execution starting from a state corresponding to α . Intuitively, this is the case if only the active struct elements in $\alpha(1)$ can take transitions and only the labels in $\alpha(3)$ are used for new struct instances.

Definition 8.1.2 (Transition and Execution Correspondence). Let τ a sequence of command transitions. Then we define that τ *respects* an AuDaLogic state (α, h, ζ) , notation $\tau \sim (\alpha, h, \zeta)$, iff all of the following hold:

1. Every command-induced transition in π is annotated with a label $\ell \in \alpha(1)$
2. For every **ComCons**-induced transition annotated with (ℓ, ℓ_2, θ) , it holds that $\ell_2 \in \alpha(3)$.

Furthermore, let π be an AuDaLa execution from an AuDaLa state p , with transition sequence τ . Then we say that π *conforms* to an AuDaLogic state (α_1, h_1, ζ) , notation $\pi \curvearrowright (\alpha, h, \zeta)$ iff $\tau \sim (\alpha, h, \zeta)$ and $p > (\alpha, h, \zeta)$.

In the safety predicates and the triple semantics, we assume that the step or statement list executions conform to the state from which the step is executed. This is safe to do, which we will explain after giving the semantics of the AuDaLogic triple. This correspondence is defined just over sequences of command transitions, and therefore has to be renewed for every step execution in an execution; this cannot be avoided, as new active elements may be created during step executions.

Lastly, as we can split our active labels, we need a notion of splitting and combining sequences of transitions. We limit this to sequences of command-induced transitions, as the interleaving in executions happens only during step executions.

Definition 8.1.3 (Parallelization). Let τ_1 and τ_2 be a sequence of annotated transitions such that $\tau_1 \curvearrowright (\alpha_1, h_1, \zeta)$ and $\tau_2 \curvearrowright (\alpha_2, h_2, \zeta)$. We define that $\text{def}(\tau_1 \parallel \tau_2)$ iff $\text{def}((\alpha_1, h_1, \zeta) \star (\alpha_2, h_2, \zeta))$ and we define $\tau = \tau_1 \parallel \tau_2$ to be a sequence of transitions formed by interleaving τ_1 and τ_2 , such that for every transition t , $t \in \tau$ iff $t \in \tau_1$ or $t \in \tau_2$, and the order of the transitions in τ_1 and τ_2 is maintained.

Here, τ is not uniquely defined. This does not matter for our safety predicates. Furthermore, as α_1 and α_2 can be merged, there can be no overlapping transitions in τ_1 and τ_2 .

As defined, AuDaLogic states do not reflect the current state of the schedule. For the safety predicate for schedules, we must therefore connect specifically the step executions to changes of the AuDaLogic states. To that end, we can reduce the full execution to just the steps executed during an execution. This we call a *step sequence*:

Definition 8.1.4 (Step Sequence). Let the *step sequence* of an execution π executing a schedule be the possibly infinite ordered list $\mathcal{F}_\pi$ consisting of pairs (F_i, q_i) , where F_i is the i th step executed in π , by the struct types specified by q_i , with $q_i = \top$ iff F was globally executed and $q_i = \theta$ iff the step was locally executed by struct type θ .

By definition, $\mathcal{F}$ is finite iff π is finite.

Safety of transitions and the statements predicate

We use the above definitions to define the safety predicate for statements. For this, we make a distinction between the execution of a statement S , which in case of an if-statement includes the execution of contained statements if applicable, and the execution of the first statement of S , which, in case of an if-statement, is the execution of all commands directly generated by that if-statement. This includes the execution of the **if**-command generated by the if-statement, but does not include the commands of contained statements, as those are generated inside the **if**-command. For all other statements S , the execution of S and the execution of the first statement of S are identical.

We define the empty execution π_0 from any AuDaLa state p as a sequence of zero transitions, which then starts and vacuously ends in p .

Lastly, we define the *safety of transitions* w.r.t. of a precondition and postcondition AuDaLogic state:

Definition 8.1.5 (Safety of Transitions). We define transitions to be *safe* for two AuDaLogic states (α_1, h_1, ζ) and $(\alpha_2, h_2, \varsigma)$ iff one of the following holds:

1. The transition is not a **ComRd**, **ComWr** or **ComCons** transition.
2. The transition is a **ComRd** transition annotated with $(\ell, \ell_2, x, v, \theta)$ and $h_1(\ell_2, x) = (\zeta, v)$ for some $\zeta \in Z$.
3. The transition is a **ComWr** transition annotated with $(\ell, \ell_2, x, v, \theta)$ and $h_2(\ell_2, x) = (1, v)$ and if $x \in \text{Parsp}(\theta)$, $h_1(\ell_2, x) = (1, v')$ for some value $v' \in V$.
4. The transition is a **ComCons** transition annotated with $(\ell, \ell', \theta, v_1, \dots, v_e)$ and $\ell' \in \alpha_1(3)$, $\ell' \in \alpha_2(2)$ and for all parameters $x_i \in \text{Parsp}(\theta)$, $h_2(\ell', x_i) = (1, v_i)$.

This is a far stronger property than correspondence, which captures when a precondition state has the right context for the transition and whether the postcondition state reflects the execution of the transition correctly with regards to the precondition state. Note that we cannot require that local variables are present in the heap before a write, as that is not the case when a new local variable is written to for the first time.

We then define the statements predicate. Intuitively, this predicate checks *local safety*, relative to single struct instances and their individual executions. This predicate does not check against a postcondition, like in Vafeiadis's work [173], as it is used as a subpredicate in the schedule execution. Instead, it verifies whether the first AuDaLogic state is sufficient for the execution by requiring that it has the right permissions for all transitions, and it verifies whether the second AuDaLogic state is an accurate transformation of the first state to reflect the execution of the list of statements. The second state can then be checked against a postcondition.

Definition 8.1.6 (Safety Predicate for Statements). Let $S \in ST^*$ be a list of statements, let (α_1, h_1, ζ) and (α_2, h_2, ζ) be two AuDaLogic states and let π be an execution of S from some AuDaLa state p .

Then $\text{safeSt}_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds iff all of the following hold:

1. If $S = \varepsilon$, $(\alpha_1, h_1, \zeta) = (\alpha_2, h_2, \zeta)$.
2. If $S = S; S$ then let $\pi = \pi_S; \pi'$, where π_S is the sequence of transitions starting with the first transition of π and ending with the last transition of the execution of the first statement of S in π . Then there exists a AuDaLogic state (α', h', ζ) for which all of the following hold:
 - (a) $\alpha_1(1) = \alpha'(1) = \{(\ell, \theta)\}$ for some $\ell \in \mathcal{L}$ and some $\theta \in \Theta_{\mathcal{P}}$ and all transitions have execution label ℓ .
 - (b) All transitions in π_S are safe for (α_1, h_1, ζ) and (α', h', ζ) .
 - (c) If S is an if-statement and if π_S contains a **ComIfT**-transition annotated with ℓ , $\text{safeSt}_{\mathcal{P}}(\pi', S_1; S', (\alpha', h', \zeta), (\alpha_2, h_2, \zeta))$ holds. Otherwise $\text{safeSt}_{\mathcal{P}}(\pi', S', (\alpha', h', \zeta), (\alpha_2, h_2, \zeta))$ holds.

We prove the following auxiliary lemma about the statements predicate, which we use to slightly simplify the definitions of the safety predicate for schedules, the AuDaLogic triple semantics and the soundness proofs:

Lemma 8.1.7 (Safety Predicate for Statements Results). Let (α_1, h_1, ζ) be a AuDaLogic state and let π be an execution of a list of statements S such that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$. Then if there exists an AuDaLogic state (α_2, h_2, ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$, with p' as the final state of π , it holds that π is finite and $p' \succ (\alpha_2, h_2, \zeta)$.

Proof. We use the definitions of the lemma and assume the memory heap h_1^d and the AuDaLogic state (α_2, h_2, ζ) as defined in the lemma exists. As $\mathcal{P}$ is well-formed, it follows from the semantics of AuDaLa that π is finite (Theorem 4.2.21), which states that step executions are finite, from which it follows that π is finite if S is a list of statements). We then prove the lemma by structural induction on the list of statements:

- (Base) If $S = \epsilon$, π contains no transitions, so it must vacuously end in p . Then $\text{safeSt}_{\mathcal{P}}(\pi, \epsilon, (\alpha_1, h_1, \zeta), (\alpha_1, h_1, \zeta))$ holds due to property 1 of $\text{safeSt}_{\mathcal{P}}$, so $(\alpha_2, h_2, \zeta) = (\alpha_1, h_1, \zeta)$, which means that $p > (\alpha_2, h_2, \zeta)$.
- (Step) Let $S = S; S'$ for some list S' , then as $\text{safeSt}_{\mathcal{P}}(\pi, S; S', (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$, with $\pi = \pi_S; \pi'$, where π_S is the sequence of transitions starting with the first transition of π and ending with the last transition of the execution of the first statement of S in π , π_S results in a state p_1 . Additionally, there exists a state (α', h', ζ) for which the requirements 2a, 2b and 2c hold. Then as 2b holds, all statements in π_S are safe w.r.t. (α_1, h_1, ζ) and (α', h', ζ) , which implies that all changes to the memory and active labels of p during π_S have been mirrored in the changes from (α_1, h_1, ζ) to (α', h', ζ) . It follows that $p_1 > (\alpha', h', \zeta)$. Then as 2a and 2c hold, $\text{safeSt}_{\mathcal{P}}(\pi', S', (\alpha', h', \zeta), (\alpha_2, h_2, \zeta))$ holds (or, if S is an if-statement which resolves to true, $\text{safeSt}_{\mathcal{P}}(\pi', S_1 S', (\alpha', h', \zeta), (\alpha_2, h_2, \zeta))$ with S_1 the statements in S) and we know by the induction hypothesis that $p' > (\alpha_2, h_2, \zeta)$. $\square$

The schedule predicate

We then define the schedule safety predicate. The schedule predicate holds if presented with a finite execution π of a schedule Δ , a premise AuDaLogic state (α_1, h_1, ζ) such that π starts in (α_1, h_1, ζ) and a postcondition Q , such that after π has been executed, (α_1, h_1, ζ) has been transformed into a AuDaLogic state (α_2, h_2, ζ) such that Q holds for the latter. To do this, the schedule predicate splits up π in step executions $t_F; \pi_F$ for each step F in the step sequence of π , where t_F is the initialization transition of F and π_F the command transitions. Then for every step F , if $t_F; \pi_F$ conforms to its precondition state (α, h, ζ) , then whether the schedule predicate to hold (continues with the next step) depends on the existence of a specific combination of a valid split (α, h, ζ) and a parallelization of the command transitions π_F of the execution of F . For this combination, for every active struct instance executing the step, there must be an execution conforming to that struct instance's label. Furthermore, for all these executions, the statements executed during the step F are safe according to the statements predicate. The resulting states for all active struct instances executing the step should also be able to be recombined into a postcondition state for the step F .

If $t; \pi_F$ does not conform to its precondition state, the schedule predicate will be vacuously true; in this case we say that π *deviates* from F on. Note that unlike the statements predicate, we do not include the postcondition state in the predicate, but check the postcondition directly. This is because executions can deviate from the AuDaLogic states during steps, so a schedule predicate returning true for a postcondition state means that either the postcondition state is correct or the execution has deviated. The postcondition then not being assuredly correct leads to trouble when checking Q . This does not happen when we check Q inside the predicate, in which case the predicate returns true iff the execution deviates or the postcondition state satisfies Q .

We formally define the schedule predicate as follows:

Definition 8.1.8 (Safety Predicate for Schedules). Let Δ be a schedule, let (α_1, h_1, ζ) be an AuDaLogic state and let π be an execution of Δ from some AuDaLa state $p \in \Xi(\alpha_1, h_1, \zeta)$. Let $\mathcal{F}$ be the step sequence of π . Then $\text{safeScP}(\pi, \Delta, (\alpha_1, h_1, \zeta), Q)$ holds iff $\text{safeScP}(\pi, \mathcal{F}, (\alpha_1, h_1, \zeta), Q)$ holds, and $\text{safeScP}(\pi, \mathcal{F}, (\alpha_1, h_1, \zeta), Q)$ holds iff all of the following hold:

1. π is finite.
2. If $\mathcal{F} = \varepsilon$, $(\alpha_1, h_1, \zeta) \models Q$.
3. If $\mathcal{F} = (F, q); \mathcal{F}'$, let $\pi = \pi_\Delta; t; \pi_F; \pi'$, where π_Δ consists of all schedule transitions until the execution of F in π , $t; \pi_F$ is the execution of F in π , with t the **Init**-transition of F and π_F containing all the command transitions in the execution of F and π' as the rest of the execution of π . Let p_1 be the AuDaLa state after the transitions $\pi_\Delta; t$. Let $\alpha_u = m(\alpha_1)$.

Then if $\pi_F \rightsquigarrow (\alpha_u, h_1, \zeta)$, there exists a state (α_t, h_t, ζ) and there exist the splits $(\alpha_u, h_1, \zeta) = (\alpha'_u, h'_1, \zeta) \star (\alpha''_u, h''_1, \zeta) \star \dots \star (\alpha''_u, h''_1, \zeta)$ and $(\alpha_t, h_t, \zeta) = (\alpha'_t, h'_t, \zeta) \star (\alpha''_t, h''_t, \zeta) \star \dots \star (\alpha''_t, h''_t, \zeta)$ and $\pi_F = \tau^1 \parallel \dots \parallel \tau^n$ such that all of the following hold:

- (a) $\alpha'_u = \alpha'_t$ and $h'_1 = h'_t$.
- (b) For every $(\ell, \theta) \in \alpha_u(1)$, if $q = \theta \vee q = \top$, then for some $1 \leq i \leq n$, $\alpha''_u(1) = \{(\ell, \theta)\}$.
- (c) For every transition in π_F annotated with an execution label ℓ , there exists a $1 \leq i \leq n$ such that $\alpha''_u(1) = \{(\ell, \theta)\}$ for some struct type θ .
- (d) For all $1 \leq i \leq n$, let π^i be the execution from p_1 consisting of the transitions τ^i . Then $\pi^i \rightsquigarrow (\alpha''_u, h''_1, \zeta)$ and $\text{safeStP}(\pi^i, \text{StepStatP}(F, \theta), (\alpha''_u, h''_1, \zeta), (\alpha''_t, h''_t, \zeta))$.
- (e) $\text{safeScP}(\pi', \mathcal{F}', (\alpha_t, h_t, \zeta), Q)$ holds.

Note that in point 3, we first change h_1 to h_u , which we do due to **InitL** and **InitG** making formerly new instances active instances and that as the

schedule is not reflected in AuDaLogic states, $p_1 \succ (\alpha_u, h_1, \varsigma)$ iff $p \succ (\alpha_u, h_1, \varsigma)$. Additionally, the definition follows the AuDaLa semantics for $\mathcal{P}$; while there are limitations on the use of local variables in well-formed programs due to the type system, these are not in the semantics and therefore not in the safety predicate. Lastly, due to Lemma 8.1.7, we know that for requirement 3e, with p' as the starting state of π' , $p \in \Xi(\alpha_t, h_t, \varsigma)$, so we do not have to assert that in the requirement.

AuDaLogic triple semantics

We then define the semantics of a AuDaLogic triple:

Definition 8.1.9 (AuDaLogic Triple Semantics). Let $\mathcal{X}$ be either a list of statements or a schedule. Then we define that $\models \{P\}\mathcal{X}\{Q\}$ iff for all AuDaLogic states $(\alpha_1, h_1, \varsigma)$ such that $(\alpha_1, h_1, \varsigma) \models P$, $(\alpha_1, h_1, \varsigma) \models \{P\}\mathcal{X}\{Q\}$, and we define that $(\alpha_1, h_1, \varsigma) \models \{P\}\mathcal{X}\{Q\}$ iff one of the following holds:

1. $\mathcal{X}$ is a list of statements and for all executions π of $\mathcal{X}$ from an AuDaLa state p such that $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$, there exists a state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ holds and $(\alpha_2, h_2, \varsigma) \models Q$.
2. $\mathcal{X}$ is a schedule and for all executions π of $\mathcal{X}$ from an AuDaLa state p such that $p \succ (\alpha_1, h_1, \varsigma)$, $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), Q)$ holds.

Note that in the first case, we exclude executions for which $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$ does not hold when taking $(\alpha_1, h_1, \varsigma)$ to be our AuDaLogic precondition state. This is safe to do, because in a label triple where all struct instances of $\mathcal{P}$ are considered active, all transitions possible in any execution of the set of statements is permitted, and there is always a larger set $\alpha(3)$. It follows that the excluded executions will be considered for AuDaLogic states with more active labels or a larger set of unused labels. Furthermore, due to Lemma 8.1.7, we do not need to include a notion of π ending in a state p' and $p' \succ (\alpha_2, h_2, \varsigma)$ in Definition 8.1.9, as that follows from asserting that the safety predicate for statements holds.

In the second case, by the assumption in the third requirement of the safety predicates for schedules, we essentially return true vacuously for all executions which deviate from the accepted behaviour of an execution with regards to the active labels at that point. This is also not a problem, as due to the same reasoning as in the first case, these deviated executions will be considered for a AuDaLogic state in which they do not deviate, as the triple must hold for all AuDaLogic states. It follows that for all executions we check that for all contexts that fit them, the postcondition state must satisfy Q .

The safety predicates as they are now do not admit race conditions; the statement predicate looks at the execution with a single active struct instance

and has hard requirements on the resulting heap, which exclude race conditions, and the safety predicate for the schedule does not admit race conditions because of the split that is required for point 3 of the predicate to hold (our separator does as of yet have no permissions to facilitate a split with race conditions). It follows that we cannot prove AuDaLa programs with race conditions with the current system; we assume therefore that $\mathcal{P}$ does not have race conditions.

8.2 Correctness of Syntax Processing for AuDaLogic

In this section, we prove the correctness of the *eqi*-transformation and the *stable* function, starting with the *eqi*-transformation.

8.2.1 Correctness of the *eqi*-transformation

To prove the *eqi*-transformation correct, we prove the following lemma:

Lemma 8.2.10 (Expression correspondence). Let S be a statement containing a simple expression E in a well-formed program $\mathcal{P}$. Let (α, h, ς) be an AuDaLogic state such that for some $a_e \in \mathcal{L}$ such that $\varsigma(a_e) = \ell$, $\alpha(1) = \{(a_e, \theta)\}$ and $(\alpha, h, \varsigma) \models A(a_e, E)$. Let π be an execution of S such that $\pi \rightsquigarrow (\alpha, h, \varsigma)$. Then the result value of the expression E during π is v iff $\text{res}(\text{eqi}(a_e, E), \varsigma)$ resolves to v .

Proof. We use the definitions in the lemma. As $\pi \rightsquigarrow (\alpha, h, \varsigma)$ and $\alpha(1) = \{(a_e, \theta)\}$, there are no read-write race conditions in π and the execution of E by the struct instance with label ℓ from p has the same result value as the execution of E within π .

Let $p = \langle \Delta, \sigma, s\chi \rangle$ and let $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$. We do structural induction on E to prove this lemma, with the following base cases:

- If $E = g$, with $g \in LT$, then we know that $\text{occ}(E)$ returns an empty set, so $\text{res}(\text{eqi}(E, a_e), \varsigma) = \text{res}(\text{pred}(E, a_e), \varsigma) = \text{res}(\text{val}(g), \varsigma) = \text{val}(g)$.

Separately, from the interpretation function of AuDaLa, we know that $\llbracket E \rrbracket = \mathbf{push}(\text{val}(g))$, and by Theorem 4.2.21 we then know that the result value is $\text{val}(g)$, so the base case holds.

- If $E = \mathbf{this}$, then $\text{res}(\text{eqi}(E, a_e), \varsigma) = \text{res}(\text{pred}(E, a_e), \varsigma) = \text{res}(a_e, \varsigma) = \ell$.

Separately, we know that $\llbracket E \rrbracket = \mathbf{push}(\mathbf{this})$, so we know by Theorem 4.2.21 that the result value of executing E from p is ℓ . It follows that the base case holds.

- If $E = \text{null}_T$, then $\text{res}(\text{eqi}(E, a_\epsilon), ha) = \text{res}(\text{pred}(E, a_\epsilon), \zeta) = \text{res}(\text{pred}(E, a_\epsilon), \zeta)$ and $\text{res}(\text{pred}(E, a_\epsilon), \zeta) = \text{res}(\text{DefaultVal}(T), \zeta) = \text{DefaultVal}(T)$.

Separately, we also know that $\llbracket E \rrbracket = \mathbf{push}(\text{DefaultVal}(T))$, which, by Theorem 4.2.21, has the resulting value $\text{DefaultVal}(T)$. This base case therefore also holds.

- If $E = x$ with $x \in ID_V$, then $\text{pred}(E, a_\epsilon) = x$. As $x \in \text{occ}(E)$, it follows that $(\alpha, h, \zeta) \models (a_\epsilon, x) \hookrightarrow a_x$ and that $\text{pred}(a_\epsilon, E)[Y \mapsto a_Y]_{Y \in \text{occ}(E)} = a_x$. It then follows that $\text{res}(\text{eqi}(a_\epsilon, E), \zeta) = \zeta(a_x)$.

Separately, we know from the interpretation function that

$\llbracket E \rrbracket = \mathbf{push}(\mathbf{this}); \mathbf{rd}(x)$, of which (by Theorem 4.2.21) the result value is $\xi(x)$.

As $p \in \Xi(\alpha, h, \zeta)$, it follows that $h(\ell, x) = \xi(x)$. As $(\alpha, h, \zeta) \models (a_\epsilon, x) \hookrightarrow a_x$, we conclude that $\zeta(a_x) = \xi(x)$, so the lemma holds in this base case.

For the induction hypothesis, we assume that for all subexpressions of E , the lemma holds. Recall that $p = \langle \Delta, \sigma, s\chi \rangle$. The step cases are then as follows:

- If $E = X.x$, with $x \in ID_V$ and $X \in EX_V$, then $\text{pred}(a_\epsilon, X.x) = X.x$. By the assumptions, $(\alpha, h, \zeta) \models A(a_\epsilon, X.x)$, so with $X = X_1.x_1$, $(\alpha, h, \zeta) \models (a_{X_1}, x_1) \hookrightarrow a_X$ for $a_X \in \mathfrak{A}$. Then by the induction hypothesis we know that $\text{res}(\text{eqi}(a_\epsilon, X), \zeta) = \zeta(a_X) = v$, with v the result value of executing X .

As $\mathcal{P}$ is well-formed, v must be a label. Then let $\sigma(v) = \langle \theta', \gamma', \chi', \xi' \rangle$. From the interpretation function and Theorem 4.2.21, it follows that the result value of E is $\xi'(x)$.

At the same time, with $\text{res}(\text{eqi}(a_\epsilon, X), \zeta) = \zeta(a_X)$, we know that $(\alpha, h, \zeta) \models (a_X, x) \hookrightarrow a_{X.x}$, as $(\alpha, h, \zeta) \models A(a_\epsilon, X.x)$. It follows that $\text{res}(\text{eqi}(a_\epsilon, X.x), \zeta) = \zeta(a_{X.x})$. As $p \succ (\alpha, h, \zeta)$, $h(v, x) = \langle \zeta, \xi'(x) \rangle$, so $\zeta(a_{X.x}) = \xi'(x)$. Then, this step case holds.

- If $E = !E_1$, then we know that $\text{pred}(E, a_\epsilon) = \neg \text{pred}(E_1, a_\epsilon)$ from which we know that $\text{res}(\text{eqi}(a_\epsilon, E), \zeta) = \neg \text{res}(\text{eqi}(a_\epsilon, E_1), \zeta)$. From the induction hypothesis, we then know that $\text{res}(\text{eqi}(a_\epsilon, E_1), \zeta) = v$ for some value $v \in \mathcal{V}$ iff the result value of executing E_1 is v . By Theorem 4.2.21, we know that then the result value of executing $!E_1$ is $\neg v$. Then as $\text{res}(\text{eqi}(a_\epsilon, E), \zeta) = \neg v$, the lemma holds.
- If $E = E_1 \circ E_2$, with $\circ$ as the semantic equivalent of $\circ$, we know that $\text{pred}(a_\epsilon, E) = \text{pred}(a_\epsilon, E_1) \circ \text{pred}(a_\epsilon, E_2)$ and that $\text{res}(\text{eqi}(a_\epsilon, E), \zeta) = \text{res}(\text{eqi}(a_\epsilon, E_1), \zeta) \circ \text{res}(\text{eqi}(a_\epsilon, E_2), \zeta)$.

By the induction hypothesis, we know that $\text{res}(\text{eqi}(a_\epsilon, E_1), \zeta) = v_1$ iff the result value for E_1 in π is v_1 and that $\text{res}(\text{eqi}(a_\epsilon, E_2), \zeta) = v_2$ iff the result value for E_2 in π is v_2 . It then follows that $\text{res}(\text{eqi}(a_\epsilon, E), \zeta) = v_1 \circ v_2$ iff the result value for E in π is $v_1 \circ v_2$, so this step case holds.

As all step cases hold, the induction is complete and the lemma holds. $\square$

We use this in our proof of the correctness of the *stable* function.

8.2.2 Correctness of the *stable* function

In this subsection, we prove the correctness of the *stable* function. To do this, we first prove an auxiliary lemma. It simplifies the proof that **ComRd**-transitions are *safe* as defined in Definition 8.1.5.

Lemma 8.2.11 (Safety of **ComRd**-transitions). Let (α, h, ζ) be an AuDaLogic state such that $(\alpha, h, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$ and let π be an execution of a list of statements such that $\pi \rightsquigarrow (\alpha, h, \zeta)$. Let $\mathcal{E}$ be all expressions executed in π . Then if $(\alpha, h, \zeta) \models A(a_\varepsilon, \mathcal{E})$, all **ComRd** transitions executed as a part of $\mathcal{E}$ in π are safe, as defined in Definition 8.1.5.

Proof. Consider a **ComRd**-transition t of π annotated with $(\ell, \ell_2, x', v', \theta')$. Then t is generated as a part of a variable expression $E_v = E_v^1.x'.E_v^2$ in $\mathcal{E}$, where E_v^1 and E_v^2 are potentially empty.

Due to the annotation of the transition, we know that the return value of E_v^1 is ℓ_2 . As $(\alpha, h, \zeta) \models A(a_\varepsilon, \mathcal{E})$ and $(\alpha, h, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$, we can apply Lemma 8.2.10. From this lemma, it follows that $\zeta(a_{E_v^1}) = \text{res}(\text{eqi}(a_\varepsilon, E_v^1), \zeta)$, so $\zeta(a_{E_v^1}) = \ell_2$.

Likewise, we know that v' is the return value of $E_v^1.x'$, due to the annotation of the **ComRd** transition, and that $\zeta(a_{E_v^1.x'}) = \text{res}(\text{eqi}(a_\varepsilon, E_v^1.x'), \zeta)$. We conclude that $\zeta(a_{E_v^1.x'}) = v'$.

As $(\alpha_1, h_1, \zeta) \models A(a_\varepsilon, \mathcal{E})$, we know that $(\alpha_1, h_1, \zeta) \models (a_{E_v^1}, x') \hookrightarrow a_{E_v^1.x'}$. It follows that $h_1(\zeta(a_{E_v^1}), x') = (\zeta, \zeta(a_{E_v^1.x'}))$ for some $\zeta \in Z$, so $h_1(\ell_2, x') = (\zeta, v')$ for some $\zeta \in Z$. We conclude that the **ComRd** transition is safe. $\square$

This lemma is useful in this subsection, as well as in the soundness proofs for the statement rules of AuDaLogic as given in Section 8.3.1.

To prove the correctness of the stability predicate, note that by the definition of *stable_P* and *stableF_P*, the predicate *stable_P*(P, Δ) returns a predicate of the form $K_1 \wedge \dots \wedge K_n$, containing a clause K_i for every step F_i which occurs in Δ , without repeating steps due to fixpoints. Not looping on fixpoints is fine: if something is stable, then the first iteration of the fixpoint must be stable, which then terminates the fixpoint immediately.

Every clause K_i isolates the struct type which executes the step F_i , when required, and then splits up the remaining struct instances in a call to *stableF_P*. As discussed with the definition of *stableF_P*, there exists exactly one set L such that $\otimes_{(a_\varepsilon, \theta) \in L} (\blacksquare(a_\varepsilon, \theta))$ holds for any label triple α . Note that at the start of the step execution, newly created struct instances of the last step become active, so it is correct to use $\blacksquare(l_i, \theta_i)$ instead of $\blacksquare_1(l_i, \theta_i)$.

Then, $stableF_{\mathcal{P}}$ is true if for all struct instances of which the label triple α contains the label, executing F_i is stable. To prove this, there must exist a mapping such that for all struct instances, the predicate $PrS_{\mathcal{P}}$ generated for that struct instance holds.

Then the stable function as a whole is correct iff for all $(a_{\epsilon}, \theta) \in L$, $PrS_{\mathcal{P}}(a_{\epsilon}, \theta, StepStat_{\mathcal{P}}(F_i, \theta_i), id)$ correctly reflects that the execution of F_i by the struct instance coupled to (a_{ϵ}, θ_i) is stable. To prove this, we use the safety predicate for statements.

However, note that the original safety predicates for statements is not strong enough. Consider the case where we prove that the predicate $safeSt_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_1, h'_1, \zeta))$ holds, with an h'_1 identical to h_1 for all parameters. This can indeed mean that nothing has changed and the statements S are stable, but following requirement 2 of the safety predicate for statements, this can also mean that the intermediate AuDaLogic states chosen during the safety predicate for S do change some parameters; they just change the parameters back later.

To truly enforce that no parameter is ever changed in the safety predicate for statements, we define the *strict safety predicate for statements*:

Definition 8.2.12 (Strict safety predicate for schedules). We define the *strict safety predicate for statements* $safeStStr_{\mathcal{P}}$, where

$safeStStr_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ iff $safeSt_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ and the state chosen as (α', h', ζ) for requirement 2 of $safeSt_{\mathcal{P}}$ must be (α_1, h', ζ) , where for all entries $(\ell, x) \in dom(h') \setminus dom(h)$ and all entries (ℓ, x) such that $h(\ell, x) \neq h'(\ell, x)$, x is a local variable of the struct instance of ℓ .

By definition, if $safeStStr_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds, then $safeSt_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds as well. We use this to prove the correctness of $PrS_{\mathcal{P}}$:

Lemma 8.2.13 (Correctness of $PrS_{\mathcal{P}}$). Let (α, h, ζ) be an AuDaLogic state such that $(\alpha, h, \zeta) \models \blacksquare(a_{\epsilon}, \theta)$. Let S be a list of statements from a well-formed program $\mathcal{P}$. Then $(\alpha, h, \zeta) \models PrS_{\mathcal{P}}(a_{\epsilon}, \theta, S, id)$ iff for all executions π from an AuDaLa state p such that $\pi \sim (\alpha, h, \zeta)$, it holds that $safeStStr_{\mathcal{P}}(\pi_i, S_i, (\alpha, h, \zeta), (\alpha, h', \zeta))$ for some heap h' as constrained by $safeStStr_{\mathcal{P}}$.

To prove this, we use a rewrite function to rewrite local variables in expressions back to their value expressed as an expression containing only parameters. To prove this, we define that a rewrite function rew is *correctly maintained* for an AuDaLa state p iff for all parameters and local variables x such that $rew(x) = E$, the result values for x and $rew(E)$ executed from p are the same. Note that this is vacuously true of id , as $ID_V \subseteq EX_S$ and $id(x) = x$.

Additionally, we define that rew is *contextualized* for a AuDaLogic state (α, h, ζ) iff $(\alpha, h, \zeta) \models \blacksquare(a_{\epsilon}, \theta)$ and for all $x \in dom(rew)$, if $rew(x) = E$ with $E \neq x$, $(\alpha, h, \zeta) \models A(a_{\epsilon}, E)$.

We then prove the following slightly altered lemma:

Lemma 8.2.14 (*PrS_P proof lemma*). Let (α, h, ζ) be an AuDaLogic state such that $(\alpha, h, \zeta) \models \blacksquare(a_\epsilon, \theta)$. Let S be a list of statements from a well-formed program $\mathcal{P}$ and let rew be a rewrite function. Then if rew is correctly maintained for all states p such that S can be executed from p , and if rew is contextualized for (α, h, ζ) , it holds that $(\alpha, h, \zeta) \models PrS_{\mathcal{P}}(a_\epsilon, \theta, S, rew)$ iff for all executions π of S from an AuDaLa state p such that $\pi \rightsquigarrow (\alpha, h, \zeta)$, it holds that $\text{safeStStr}_{\mathcal{P}}(\pi_i, S_i, (\alpha, h, \zeta), (\alpha, h', \zeta))$ for some heap h' as constrained by $\text{safeStStr}_{\mathcal{P}}$.

With this lemma, and the fact that both assumptions vacuously hold for rewrite function id , we can then conclude that Lemma 8.2.13 holds if Lemma 8.2.14 holds.

The proof for Lemma 8.2.14 is as follows:

Proof. To prove the lemma, since the executions considered are the executions π such that $\pi \rightsquigarrow (\alpha, h, \zeta)$, and as $(\alpha, h, \zeta) \models \blacksquare_1(a_\epsilon, \theta)$, the only transitions that can be in π are those executed with execution label a_ϵ . It follows that there are no race conditions, and that we may take the result values of expressions to be deterministic. Additionally, any changes to the state can be directly attributed to executions by a_ϵ .

We prove the lemma by structural induction on S . As the induction hypothesis, we take that the lemma holds for S' , with S' the remainder of the statements after executing the first statement of S . Note that like with the safety predicate for statements, we do not consider the execution of statements enclosed in an if-statement to be part of the execution of the if-statement when executing the first statement of S .

Note that this means that during every case, we assume that rew is correctly maintained for p and contextualized for (α, h, ζ) . We then have two proof obligations: firstly, if we want to apply the induction hypothesis, we first need to establish that the rewrite function is still contextualized for (α, h, ζ) and correctly maintained for the beginning state of the execution of the remaining statements for which we apply the induction hypothesis. Secondly, we need to prove that the lemma holds.

As our base case, we consider the empty list of statements. If $S = \epsilon$, then $PrS_{\mathcal{P}}(a_\epsilon, \theta, S, rew) = \text{true}$. At the same time, by the definition of $\text{safeStStr}_{\mathcal{P}}$, it holds that $\text{safeStStr}_{\mathcal{P}}(\pi_i, \epsilon, (\alpha, h, \zeta), (\alpha, h, \zeta))$. As h satisfies the heap constraint of $\text{safeStStr}_{\mathcal{P}}$, this satisfies the lemma.

We have the following step cases. For these cases, the first requirement for $\text{safeStStr}_{\mathcal{P}}$ automatically holds, as $S \neq \epsilon$. We therefore only prove requirement 2 of the safety predicate for statements for the step cases, where relevant. Note that if we do not use the induction hypothesis, we do not need to prove that rew is correctly maintained.

- If $S = \theta' x := \theta'(E_1, \dots, E_n); S'$, then $\text{PrSP}(a_\epsilon, \theta, S, \text{rew}) = \text{false}$.

At the same time, for any execution π of S , π will include a **ComCons** transition. Then in first statement executed in S the **ComCons** transition is only safe if the new struct instance is reflected in α , which is not the case. It follows that $\text{safeStStrP}(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta)) = \text{false}$ (for any heap h'), so the lemma holds for this case.

- If $S = \text{if } E \text{ then } \{S_1\}; S'$, then

$$\begin{aligned} \text{PrSP}(a_\epsilon, \theta, \text{if } E \text{ then } \{S_1\}; S, \text{rew}) = \\ A(a_\epsilon, E') \wedge ((\text{eqi}(a_\epsilon, E') \wedge \text{PrSP}(a_\epsilon, \theta, S_1; S, \text{rew})) \\ \vee (\neg \text{eqi}(a_\epsilon, E') \wedge \text{PrSP}(a_\epsilon, \theta, S, \text{rew}))), \end{aligned}$$

where $E' = \text{PrEP}(E, \text{rew})$.

Recall that $\text{PrEP}(E, \text{rew})$ only rewrites E using rew . As rew is correctly maintained for p , we know that the result values of executing E and E' are the same result value, which we will fix to be v .

We prove the double implication by proving both sides of the implication. If $(\alpha, h, \zeta) \models \text{PrSP}(a_\epsilon, \theta, S, \text{rew})$, let π be an execution of S such that $\pi \rightsquigarrow (\alpha, h, \zeta)$. Then $\pi = \pi_E; \pi'$, by definition of S , where π_E is the execution of the first statement of S and π' is the rest of π . Let p' be the starting state of π' .

To prove that $\text{safeStStrP}(\pi_i, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some heap h' as constrained by safeStStrP , we prove that the execution of the first statement of S is safe according to safeStStrP requirements 2a and 2b and we use the induction hypothesis for π' to prove safeStStrP requirement 2c. As our intermediary AuDaLogic state, we use (α, h, ζ) .

From the premise of the lemma, we know that $(\alpha, h, \zeta) \models \blacksquare(a_\epsilon, \theta)$. Then due to that and the fact that $\pi \rightsquigarrow (\alpha, h, \zeta)$, we know that requirement 2a of safeStStrP holds.

For requirement 2b, the only transitions we have to prove safe in π_E are **ComRd**-transitions, as $\mathcal{P}$ is well-formed and the first statement of S is an if-statement (and therefore contains no **ComWr** transitions). These **ComRd**-transitions are then safe, due to Lemma 8.2.11, which applies in this case.

For requirement 2c, we prove that if there is a **ComIfT** transition in π_E then

$\text{safeStStrP}(\pi', S_1; S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some h' as constrained by safeStStrP and that if there is no **ComIfT** transition in π_E then $\text{safeStStrP}(\pi', S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some h' as constrained by safeStStrP .

As $(\alpha, h, \zeta) \models A(a_\epsilon, E')$ and due to Lemma 8.2.10, it follows that $\text{res}(\text{eqi}(a_\epsilon, E')) = v$. As $\mathcal{P}$ is well-formed and due to Theorem 4.2.21, $v \in \mathbb{B}$.

If $v = \text{true}$, we know that $(\alpha, h, \zeta) \models \text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, S_1; \mathcal{S}', \text{rew})$, as $(\alpha, h, \zeta) \models \text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}, \text{rew})$. Likewise, if $v = \text{false}$, we know that $(\alpha, h, \zeta) \models \text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}', \text{rew})$. For both subcalls, as no changes to parameters or variables have been made, rew is still correctly maintained for p' and contextualized for (α, h, ζ) .

We conclude that the rewrite assumption still holds for the subcall. Therefore by the induction hypothesis, if $v = \text{true}$ then $\text{safeStStr}_{\mathcal{P}}(\pi', S_1; \mathcal{S}', (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds and if $v = \text{false}$ then $\text{safeStStr}_{\mathcal{P}}(\pi', \mathcal{S}', (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds, for some h' as constrained by $\text{safeStStr}_{\mathcal{P}}$. With this, we know that the full requirement 2 of $\text{safeStStr}_{\mathcal{P}}$ holds, which proves the first implication.

Then assume that $\text{safeStStr}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some h' as constrained by $\text{safeStStr}_{\mathcal{P}}$. Then let $\pi = \pi_E; \pi'$ as above, with p' the starting state of π' . As $\text{safeStStr}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds, all transitions in π are safe, so all transitions in π_E are safe as well. Combined with the rewrite assumption on rew , we can conclude that $(\alpha, h, \zeta) \models A(a_\epsilon, E')$: all parameters in E are covered by the premise and all parameters introduced in E' are covered as rew is contextualized for (α, h, ζ) . Then as above, we know that $\text{res}(\text{eqi}(a_\epsilon, E'), \zeta) = v$, as rew is correctly maintained for p .

The execution π only takes one of the two branches of $\text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}, \text{rew})$, but as $\text{res}(E') = v$, we know that the branch π does not take will be trivially true for (α, h, ζ) . Let the subcall in the branch that π takes be the call $\text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}'', \text{rew})$, with $\mathcal{S}'' = S_1; \mathcal{S}'$ or $\mathcal{S}'' = \mathcal{S}'$, depending on the branch. As π does not change any variables during the execution of E , we know that rew is still correctly maintained for p' .

Then, $\text{safeStStr}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha, h, \zeta), (\alpha, h', \zeta))$ implies that for whatever subcall π takes, $\text{safeStStr}_{\mathcal{P}}(\pi, \mathcal{S}'', (\alpha, h'', \zeta), (\alpha, h', \zeta))$ holds. It follows that as π_E does not change any parameters nor local variables, rew is still contextualised for (α, h'', ζ) . Then we know by the induction hypothesis that $\text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}'', \text{rew})$.

With this we prove that, relative to π , $(\alpha, h, \zeta) \models \text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}, \text{rew})$. Then by generalization, $(\alpha, h, \zeta) \models \text{PrS}_{\mathcal{P}}(a_\epsilon, \theta, \mathcal{S}, \text{rew})$. With this, the bi-implication is proven.

- If $\mathcal{S} = T \ x := E; \mathcal{S}'$, let π be an execution of $\mathcal{S}$. Let $\pi = \pi_S; \pi'$, with π_S the execution of $T \ x := E$ and π' the execution of $\mathcal{S}'$. Let p' be the starting state of π' .

We know that $PrSp(a_\epsilon, \theta, T x := E; S', rew) = A(E') \wedge PrSp(a_\epsilon, \theta, S', rew[x \mapsto E'])$, with $E' = PrEp(E, rew)$.

We prove the double implication by proving both sides of the implication. Then if $(\alpha, h, \zeta) \models PrSp(a_\epsilon, \theta, T x := E; S', rew)$, to prove that $safeStStrP(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some h' constrained by $safeStStrP$ we define h' such that $h[(\zeta(a_\epsilon), x) \mapsto (1, v)]$, with v the result value of E in π . We prove that the execution of the first statement of S is safe according to $safeStStrP$ requirements 2a and 2b and we use the induction hypothesis for π' to prove $safeStStrP$ requirement 2c.

From the premise of the lemma, we know that $(\alpha, h, \zeta) \models \blacksquare(a_\epsilon, \theta)$. Then due to that and the fact that $\pi \prec (\alpha, h, \zeta)$, we know that requirement 2a of $safeStStrP$ holds.

For requirement 2b of $safeStStrP$, we prove all transitions in π_S safe. To prove this, we prove all **ComRd** and the single **ComWr** transition of $T x := E$ safe. As P is well-formed, there are no **ComCons** transitions. As $(\alpha, h, \zeta) \models A(a_\epsilon, E')$, we can apply Lemma 8.2.11 to conclude that all **ComRd** transitions are safe.

Let the **ComWr** transition of $T x := E$ be annotated with $(\ell_1, \ell_2, x, v', \theta')$, with x fixed due to the syntax. As $\pi \prec (\alpha, h, \zeta)$ and $(\alpha, h, \zeta) \models \blacksquare(a_\epsilon, \theta)$, it holds that $\ell_1 = \zeta(a_\epsilon)$. As P is well-formed, x is a local variable, so $\ell_2 = \zeta(a_\epsilon)$. As the return value of E is fixed to be v , it holds that $v = v'$. As $\ell_2 = \zeta(a_\epsilon)$, it holds that $\theta' = \theta$. From this, it follows that the **ComWr** transition is safe, as $x \notin ParsP(\theta)$ (as P is well-formed) and $h'(\ell_2, x) = (1, v')$. Therefore, requirement 2b of $safeStStrP$ holds.

To prove requirement 2c, consider that in p' , x and E must have the same result value, as p' is reached by executing $T x := E$. As rew is correctly maintained for p and x cannot occur in E due to the fact that P is well-formed, we know that the result value of E' from p is the same as the result value of E from p , so $rew[x \mapsto E']$ is correctly maintained for p' . Additionally, as $(\alpha, h, \zeta) \models A(a_\epsilon, E')$ and rew is contextualized for (α, h, ζ) , we know that $rew[x \mapsto E']$ is also contextualized for (α, h', ζ) . Then, by the induction hypothesis, $safeStStrP(\pi', S', (\alpha, h', \zeta), (\alpha, h'', \zeta))$, as $(\alpha, h, \zeta) \models PrSp(a_\epsilon, \theta, S', rew[x \mapsto E'])$.

Then $safeStStrP(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds, as the requirements 2a, 2b and 2c hold. This proves the first implication.

Then assume that $safeStStrP(\pi, S, (\alpha, h, \zeta), (\alpha, h'', \zeta))$ holds for some h'' constrained by $safeStStrP$. We prove that $PrSp(a_\epsilon, \theta, S', rew[x \mapsto E'])$ holds.

Let (α', h', ζ) be the state such that $safeStStrP(\pi, S', (\alpha', h', \zeta), (\alpha, h'', \zeta))$ holds, following the definition of $safeStStrP(\pi, S, (\alpha, h, \zeta), (\alpha, h'', \zeta))$.

Due to Lemma 8.1.7, it holds that $p' > (\alpha', h', \zeta)$. By the definition of p' as the state after the execution of $T \ x := E$, we know that $\alpha = \alpha'$, and as all transitions in π are safe, $h'(\zeta(a_\epsilon), x) = (1, \nu)$ for some ν which results from executing E in π . Then as all transitions during π are safe and as rew is contextualized for (α, h, ζ) , it holds that $(\alpha, h, \zeta) \models A(a_\epsilon, E')$. It follows that $res(eqi(a_\epsilon, E'), \zeta) = \nu$, with $E' = PrE_{\mathcal{P}}(E, rew)$, as rew is correctly maintained for p .

Then the result value of E and x is the same when they are executed from p' , so $rew[x \mapsto E']$ is correctly maintained for p' . Additionally, as $(\alpha, h, \zeta) \models A(a_\epsilon, E')$ and rew is contextualized for (α, h, ζ) , it holds that $rew[x \mapsto E']$ is contextualized for (α, h', ζ) .

Then as $\text{safeStStr}_{\mathcal{P}}(\pi', S', (\alpha, h', \zeta), (\alpha, h'', \zeta))$, it holds that $(\alpha, h', \zeta) \models PrS_{\mathcal{P}}(a_\epsilon, \theta, S', rew[x \mapsto E'])$. It follows that the lemma holds for this case.

- If $S = E_v := E; S'$, then there are three cases:
 - If E_v is a parameter $X.x$, then

$$\begin{aligned} PrS_{\mathcal{P}}(a_\epsilon, \theta, E_v := E; S, rew) = \\ A(a_\epsilon, X, E') \wedge (a_X, x) \xrightarrow{1} eqi(a_\epsilon, E') \wedge \\ PrS_{\mathcal{P}}(a_\epsilon, \theta, S, rew) \end{aligned}$$

with $E' = PrE_{\mathcal{P}}(E, rew)$.

Then let π be an arbitrary execution executing S from a state p such that $\pi \prec (\alpha, h, \zeta)$. As rew is correctly maintained, we know that the result values of E and E' from p are the same result value ν . Then as π executes S , we know that $\pi = \pi_S; \pi'$, with π_S the execution of $E_v := E$. Let p' be the starting state of π' . We will then prove that relative to π it holds that $(\alpha, h, \zeta) \models PrS_{\mathcal{P}}(a_\epsilon, \theta, E_v := E; S', rew)$ iff $\text{safeStStr}_{\mathcal{P}}(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ for a h' constrained by $\text{safeStStr}_{\mathcal{P}}$.

We prove the double implication by proving both sides of the implication. If relative to π it holds that $(\alpha, h, \zeta) \models PrS_{\mathcal{P}}(a_\epsilon, \theta, E_v := E; S', rew)$, we prove that $\text{safeStStr}_{\mathcal{P}}(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$. Let $\pi = \pi_S; \pi'$, where π_S is the execution of $X.x := E$ and π' is the execution of S' . We prove that $\text{safeStStr}_{\mathcal{P}}(\pi_S, X.x := E, (\alpha, h, \zeta), (\alpha, h, \zeta))$ and that $\text{safeStStr}_{\mathcal{P}}(\pi', S', (\alpha, h, \zeta), (\alpha, h', \zeta))$.

To prove that $\text{safeStStr}_{\mathcal{P}}(\pi_S, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds for some heap h' as constrained by $\text{safeStStr}_{\mathcal{P}}$, we prove that the execution of the first statement of S is safe according to $\text{safeStStr}_{\mathcal{P}}$ requirements 2a and 2b and we use the induction hypothesis for

π' to prove $\text{safeStStr}_{\mathcal{P}}$ requirement 2c. As our intermediary AuDaLogic state, we use (α, h, ζ) .

From the premise of the lemma, we know that $(\alpha, h, \zeta) \models \blacksquare(a_\varepsilon, \theta)$. Then due to that and the fact that $\pi \rightsquigarrow (\alpha, h, \zeta)$, we know that requirement 2a of $\text{safeStStr}_{\mathcal{P}}$ holds.

We then prove all transitions in π_S safe. As $\mathcal{P}$ is well-formed, there are no **ComCons** transitions in $X.x := E$.

For the **ComRd** transitions, As $(\alpha, h, \zeta) \models \text{PrSp}(a_\varepsilon, \theta, E_v := E; S', \text{rew})$, we know that $(\alpha, h, \zeta) \models A(a_\varepsilon, X, E') \wedge (a_X, x) \xrightarrow{1} E'$. It follows from Lemma 8.2.10 that $\text{res}(E', \zeta) = v$. Furthermore, due to Lemma 8.2.10, we also know that $\text{res}(\text{eqi}(a_\varepsilon, X)) = \ell$ such that the result value of X is ℓ . Then $\zeta(a_X) = \ell$, as $(\alpha, h, \zeta) \models A(a_\varepsilon, X, E')$. Then we know from the fact that rew is contextualized for (α, h, ζ) and the fact that $A(a_\varepsilon, X, E')$ holds for (α, h, ζ) . We then conclude from Lemma 8.2.11 that all **ComRd** transitions are safe.

Due to the syntax of $X.x := E$, we know that there is a single **ComWr** transition annotated with $(\ell_1, \ell_2, x, v', \theta)$. As $\pi \rightsquigarrow (\alpha, h, \zeta)$ and $(\alpha, h, \zeta) \models \blacksquare(a_\varepsilon, \theta')$, we know that $\ell_1 = \zeta(a_\varepsilon)$. As $\text{res}(\text{eqi}(a_\varepsilon, X)) = \ell$, we know that $\ell_2 = \ell$. As the result value of E is equal to $\text{res}(E', \zeta) = v$, we know that $v = v'$. Lastly, as θ' is then the struct type of the struct instance of ℓ , and $\mathcal{P}$ is well-formed, we know that $x \in \text{Pars}_{\mathcal{P}}(\theta')$.

As $(\alpha, h, \zeta) \models A(a_\varepsilon, E') \wedge (a_\varepsilon, x) \xrightarrow{1} E'$, we know that $h(\ell, x) = v$, both before (as $v = \text{res}(E, \zeta)$) and after the update (as the result value of E is v). We conclude the **ComWr** transition is safe. It follows that requirement 2b holds.

To prove requirement 2c, we use the induction hypothesis. As rew is unchanged for the call $\text{PrSp}(a_\varepsilon, \theta, S', \text{rew})$, it is contextualized for (α, h, ζ) . Additionally, as $(\alpha, h, \zeta) \models (a_X, x) \xrightarrow{1} E'$, the result value of E is $\text{res}(E', \zeta)$ and $X.x := E$ contains no constructors, no parameters nor local variables are changed in π_S . Then we know that rew is correctly maintained for p' . Then, by the induction hypothesis, we know that the lemma holds for $\text{PrSp}(a_\varepsilon, \theta, S', \text{rew})$. Then we know that $\text{safeStStr}_{\mathcal{P}}(\pi', S', (\alpha, h, \zeta), (\alpha, h', \zeta))$.

As all requirements 2a, 2b and 2c all hold, $\text{safeStStr}_{\mathcal{P}}(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds. This proves the first implication.

If $\text{safeStStr}_{\mathcal{P}}(\pi, S, (\alpha, h, \zeta), (\alpha, h', \zeta))$, as all transitions in π are safe, h contains all parameters accessed in π . As rew is contextualized for (α, h, ζ) , it holds that $(\alpha, h, \zeta) \models A(a_\varepsilon, X, E) \wedge (a_X, x) \xrightarrow{1} -$. As h' can only differ from h in the local variables, π changes no parameters. Then, by Lemma 8.2.10, we know that, with $\text{res}(E') = v$ and

$res(eqi(a_\varepsilon, X)) = \ell$, $h(\ell, x) = (1, v)$ must hold. It follows that $(\alpha, h, \zeta) \models (a_X, x) \xrightarrow{1} E'$.

Then with $\pi = \pi_S; \pi'$ and p' as defined above and as no local variables are initialized in π_S , the function rew is still correctly maintained for p' . Additionally, as $\text{safeStStrP}(\pi, \mathcal{S}, (\alpha, h, \zeta), (\alpha, h', \zeta))$ holds, and we know that π_S does not change any local variables, we know that $\text{safeStStrP}(\pi, X.x := E, (\alpha, h, \zeta), (\alpha, h'', \zeta))$ also holds for some state h'' as constrained by safeStStrP . As there have been no changes to rew and h and h'' agree on all the parameters, rew is still contextualized for (α, h'', ζ) . Then by the induction hypothesis, $(\alpha, h, \zeta) \models A(a_\varepsilon, E_\nu, E) \wedge (a_X, x) \xrightarrow{1} E' \wedge \text{PrSP}(a_\varepsilon, \theta, S', rew)$ relative to π . Then as π is arbitrary, we can conclude that the lemma holds for this case.

- If E_ν is a parameter x , the reasoning for the lemma is the same as the first case, except that $A(a_\varepsilon, X, E')$ is replaced with $A(a_\varepsilon, E')$ and that all reasoning about X can be removed, as $\ell = \zeta(a_\varepsilon)$.
- If $E_\nu = x$ for some local variable x , then the lemma holds analogously to the case of $T \ x := E$.

As all three cases hold, the lemma holds for the entire step case.

As all step cases hold, the lemma holds. $\square$

This proves Lemma 8.2.14, which in extension proves Lemma 8.2.13. In the next section, we will use Lemma 8.2.13 to prove the soundness of the fixpoint proof rule.

8.3 Soundness of the Proof Rules

In this section, we prove that our proof system as presented in Section 7.4 is *sound*. Recall that an executable $\mathcal{X}$ is a list of statements or a schedule. The soundness of a proof system is then defined as follows:

Definition 8.3.15 (Soundness). The proof system as presented in Section 7.4 is *sound* iff for all triples $\{P\}\mathcal{X}\{Q\}$, if $\vdash \{P\}\mathcal{X}\{Q\}$ in the proof system, then $\models \{P\}\mathcal{X}\{Q\}$.

We then prove the following theorem:

Theorem 8.3.16 (Soundness). AuDaLogic is sound.

To prove this, we need to prove all proof rules sound by structural induction. This takes the following general form.

Let **RULE** be a proof rule with conclusion $\vdash \{P\}\mathcal{X}\{Q\}$ for some precondition P , executable $\mathcal{X}$ and postcondition Q . Then we assume that all premises of **RULE** holds. We also assume that for all premises $\vdash \{P'\}C'\{Q'\}$, it holds that $\models \{P'\}C'\{Q'\}$. Here, P' is a precondition, C' an executable and Q' is a postcondition. Using these assumptions, we need to prove that $\models \{P\}\mathcal{X}\{Q\}$ holds. During these proofs, we define states $(\alpha_1, h_1, \varsigma)$ such that all premises for **RULE** hold to be *premise AuDaLogic states*.

In the rest of this section, we do this for every rule separately, divided into subsections reflecting the categories of proof rules as presented in Section 7.4.

8.3.1 Soundness of the statement rules

In this section, we prove the statement rules of AuDaLogic. As the proofs are dense, we first give some intuition on how the proofs work.

As these are proofs over statements, we use the statements predicate. Through the definition of AuDaLogic triples, this gives us a framework to prove the proof triples in the conclusions of the proof rules. For proof rules with as their conclusion a statement triple of the form $\vdash \{P\}S\{Q\}$, two things need to be proven for all premise AuDaLogic states $(\alpha_1, h_1, \varsigma)$ and all possible executions π of S such that $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$. Firstly, there must exist a state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, S, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$. Secondly, for this existing state $(\alpha_2, h_2, \varsigma)$, it must hold that $(\alpha_2, h_2, \varsigma) \models Q$.

The first proof obligation has two major components. Firstly, the definition of $(\alpha_2, h_2, \varsigma)$. This definition often immediately proves the requirements 2a, 2c (and the recursive element) and 1 from $\text{safeSt}_{\mathcal{P}}$, along with the premises of the proof rule. Secondly, a proof that all transitions in the execution π of S are safe. This requires reasoning about which transitions one can expect in π , using the interpretation function of the AuDaLa semantics, and what the effect of these transitions can be, using Theorem 4.2.21 and its corollaries. This part also often uses the premises as given in the proof rule.

The second proof obligation requires a proof that $(\alpha_2, h_2, \varsigma) \models Q$. This should follow mainly from the definition of $(\alpha_2, h_2, \varsigma)$, but in the case of **CONSTRUCTOR** it reuses some parts from the proof that the transitions of π are safe. With both proof obligations discharged, we then conclude that the proof rule is sound.

Soundness of UPDATE

Let $\mathcal{X}$ be the syntactical statement ' $X.x := E$ ', with a potentially empty variable expression X and a simple expression E . Let the precondition be $P = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} -)$, with P_L a heap-free predicate and P_H a triple-free predicate

such that $P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta)$ and $P_H \Rightarrow A(a_\epsilon, E, X)$. Let the postcondition be $Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} \text{eqi}(a_\epsilon, E))$.

To prove that $\models \{P\}X.x := E\{Q\}$, we need to prove that for all AuDaLogic states (α_1, h_1, ζ) such that $(\alpha_1, h_1, \zeta) \models P$, for all executions π of $X.x := E$ such that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, there exists a state (α_2, h_2, ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ and $(\alpha_2, h_2, \zeta) \models Q$.

Then let (α_1, h_1, ζ) be an arbitrary AuDaLogic state and let π be an arbitrary execution satisfying the requirements of the last paragraph. Let the first state of the execution π be the AuDaLa state p .

To define (α_2, h_2, ζ) , we define ν to be the result value of E executed from p . As $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$ and $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\epsilon, \theta)$ (as $(\alpha_1, h_1, \zeta) \models P$), all transitions in π can only have ℓ as their executing label, so π does not have race conditions. It follows by Corollary 4.3.23 that the result value of E during π must also be ν . We define the AuDaLogic state (α_2, h_2, ζ) as $(\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta)$, with $h_1[(\zeta(a_X), x) \mapsto (1, \nu)]$ a function update of h_1 .

To prove that $\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \zeta), (\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta))$ holds, we prove the requirements of $\text{safeSt}_{\mathcal{P}}$. The first requirement of $\text{safeSt}_{\mathcal{P}}$ is satisfied because $\mathcal{X} \neq \epsilon$. For the second requirement, as $\mathcal{X}$ consists only of $X.x := E$, we can satisfy 2c and the recursive element by taking $(\alpha', h, \zeta) = (\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta)$. We then need to prove that requirements 2a and 2b hold for $(\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta)$:

2a As $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$ and $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\epsilon, \theta)$, we know that $\alpha_1(1) = \{(\ell, \theta)\}$. Additionally, there is only a single struct instance that can execute transitions in π , so we know that all transitions have execution label ℓ .

2b To prove that requirement 2b holds, we need to prove all **ComCons**, **ComRd** and **ComWr** transitions in π are safe:

- As there are only transitions executed by the struct instance of ℓ , there are only transitions stemming from the commands of $\mathcal{X}$ in π . Therefore, there are no **ComCons** transitions in π .
- As $P \Rightarrow A(a_\epsilon, E, X)$ (as $P_H \Rightarrow A(a_\epsilon, E, X)$), E and X are the expressions of $\mathcal{X}$ and $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, the fact that the **ComRd** rules are safe in π follows from Lemma 8.2.11.
- As there are only transitions executed by the struct instance of ℓ , there are only transitions stemming from $\llbracket \mathcal{X} \rrbracket$ in π . Therefore, there is only at most a single **ComWr** transition in π . By the fact that $(\alpha_1, h_1, \zeta) \models P_H * (a_X, x) \xrightarrow{1} -$, we know that x is not a null-parameter: if it was, it would be in h_0 , and then by definition of memory assertions, $(a_X, x) \xrightarrow{1} -$ would not be satisfiable. We then know there is exactly one **ComWr**-transition.

Let the **ComWr**-transition of π be annotated with $(\ell, \ell_2, x, \nu', \theta')$ for

some $\ell_2 \in \mathcal{L}$, $v' \in \mathcal{V}$ and $\theta' \in \Theta_{\mathcal{P}}$. By the fact that the transition is annotated with $(\ell, \ell_2, x, v', \theta')$ and the fact that $\llbracket \mathcal{X} \rrbracket = \llbracket E \rrbracket; \llbracket X \rrbracket; \mathbf{wr}(x)$, it holds that $v' = v$ and that ℓ_2 is the return value of X in $\mathcal{X}$. By applying Lemma 8.2.10 on X , we can conclude that $\varsigma(a_X) = \ell_2$. It follows that $h_2(\ell_2, x) = (1, v')$, from the definition of h_2 .

Additionally, as $(\alpha_1, h_1, \varsigma) \models P * (a_X, x) \xrightarrow{1} -$, we know that $h_1(\ell_2, x) = (1, v'')$ for some $v'' \in \mathcal{V}$.

We conclude that $\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \varsigma), (\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma))$ holds. We then still need to prove that $(\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma) \models P_L \wedge (P_H * (a_X, x) \xrightarrow{1} \text{eqi}(a_\varepsilon, E))$. As P_L is a heap-free predicate and $(\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma)$ only differs from $(\alpha_1, h_1, \varsigma)$ in the memory heap, we know that $(\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma) \models P_L$.

We prove that additionally, $(\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma) \models (P_H * (a_X, x) \xrightarrow{1} \text{eqi}(a_\varepsilon, E))$. As $h_1[(\varsigma(a_X), x) \mapsto (1, v)]$ is only changed in the entry for $(\varsigma(a_X), x)$, this is proven by proving that $\text{res}(\text{eqi}(a_\varepsilon, E)) = v$. This holds due to Lemma 8.2.10, which is applicable in this situation. We conclude that $(\alpha_1, h_1[(\varsigma(a_X), x) \mapsto (1, v)], \varsigma) \models \blacksquare(a_\varepsilon, \theta) \wedge (P'' * (a_X, x) \xrightarrow{1} \text{eqi}(a_\varepsilon, E))$.

It follows that the proof rule UPDATE is sound. $\square$

Soundness of NULL UPDATE

Let $\mathcal{X}$ be the syntactical statement ' $X.x := E'$ ', with a potentially empty variable expression X and a simple expression E . Let the precondition be $P = P_L \wedge (P_H * (a_X, x) \xrightarrow{\zeta_0} E')$, with P_L a heap-free predicate and P_H a triple-free predicate such that $P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$ and $P_H \Rightarrow A(a_\varepsilon, E, X)$.

To prove that $\models \{P\}X.x := E\{P\}$, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P * (a_X, x) \xrightarrow{\zeta_0} E$ and let π be an execution of $\mathcal{X}$ from some AuDaLa p such that $\pi \prec (\alpha_1, h_1, \varsigma)$. We then need to prove that there exists a AuDaLogic state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ and $(\alpha_2, h_2, \varsigma) \models P * (a_X, x) \xrightarrow{\zeta_0} E$.

We do this by taking $(\alpha_2, h_2, \varsigma) = (\alpha_1, h_1, \varsigma)$, from which $(\alpha_2, h_2, \varsigma) \models P * (a_X, x) \xrightarrow{\zeta_0} E$ follows, and proving that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_1, h_1, \varsigma))$. For this, we need to prove that all requirements of $\text{safeSt}_{\mathcal{P}}$ hold.

The first requirement of $\text{safeSt}_{\mathcal{P}}$ is satisfied because $\mathcal{X} \neq \varepsilon$. To prove the second requirement, by the assumptions on P , we know that $(\alpha_1, h_1, \varsigma) = \{(\varsigma(a_\varepsilon), \theta)\}$. As $\pi \prec (\alpha_1, h_1, \varsigma)$, requirement 2a holds.

For 2c, as $\mathcal{X}$ is not an if-statement and $\mathcal{X} = \mathcal{X}; \varepsilon$, we know that $\pi = \pi_C; \pi_0$. We can then satisfy requirement 2c (and the recursive element) by taking $(\alpha', h', \varsigma) = (\alpha_1, h_1, \varsigma)$.

Then for 2b, we need to prove all transitions in π safe for $(\alpha_1, h_1, \varsigma)$ and $(\alpha_1, h_1, \varsigma)$. By definition of $\mathcal{X}$, $\llbracket \mathcal{X} \rrbracket$ does not generate **cons**-commands, so there are no **ComCons** transitions. Furthermore, as x is the parameter of a null-instance (as $(a_x, x) \xrightarrow{\zeta_0} E$ can only hold for parameters of null-instances), the transition that deals with the **wr**-command generated by $\llbracket \mathcal{X} \rrbracket$ must be a **ComWrNSkip** transition, which is automatically safe. As $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$ and $\alpha_1(1) = \{(\varsigma(a_\varepsilon), \theta)\}$, we know that all **wr**-commands that are executed in π come from $\llbracket \mathcal{X} \rrbracket$. It follows that there are no **ComWr**-transitions in π . We then only need to prove that all **ComRd**-transitions are safe, which holds through Lemma 8.2.11.

Therefore $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_1, h_1, \varsigma))$ holds, from which we conclude that that NULL UPDATE is sound. $\square$

Soundness of CONSTRUCTOR

Let $\mathcal{X}$ be a syntactical statement ' $\theta' x := \theta'(E_1, \dots, E_e)'$ ', with E_1 to E_e being simple expressions and x a fresh local variable, following constructor normal form. Let $\text{Pars}_{\mathcal{P}}(\theta') = x_1, \dots, x_e$. Let the precondition be $P = (P_L * \text{fresh}(l')) \wedge P_H$ such that P_L is a heap-free predicate, P_H is a triple-free predicate, $P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$ and $P_H \Rightarrow \wedge \neg(a_\varepsilon, x) \hookrightarrow - \wedge A(a_\varepsilon, E_1, \dots, E_e)$.

Let Q be the postcondition

$$Q = (P_L * \blacksquare_2(l', \theta')) \wedge (P_H * (a_\varepsilon, x) \xrightarrow{1} l' * \otimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} \text{eqi}(a_\varepsilon, E_i))$$

To prove that $\models \{P\} \theta' x := \theta'(E_1, \dots, E_e) \{Q\}$, we need to prove that for all AuDaLogic states $(\alpha_1, h_1, \varsigma)$ such that $(\alpha_1, h_1, \varsigma) \models P$, for all executions π of $\mathcal{X}$ from some AuDaLa state p such that $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$, there exists a state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, C, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ and $(\alpha_2, h_2, \varsigma) \models Q$.

Then let $(\alpha_1, h_1, \varsigma)$ be an arbitrary AuDaLogic state and let π be an arbitrary execution satisfying the requirements in the last paragraph. Let π start in AuDaLa state p .

To define $(\alpha_2, h_2, \varsigma)$, we define $v_1, \dots, v_e$ to be the result values of $E_1, \dots, E_e$ executed from p . As $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$ and $(\alpha_1, h_1, \varsigma) \models \blacksquare_1(a_\varepsilon, \theta)$, all transitions in π can only have ℓ as their executing label, so π does not have race conditions. It follows by Corollary 4.3.23 that the result values of $E_1, \dots, E_e$ during π must also be $v_1, \dots, v_e$ respectively.

We define α_2 such that $\alpha_2(1) = \alpha_1(1)$, $\alpha_2(2) = \{\varsigma(l'), \theta'\} \cup \alpha_1(2)$ and $\alpha_2(3) = \alpha_1(3) \setminus \{\varsigma(l')\}$. Furthermore, we define $h_2 = h_1[\{(\varsigma(l'), x_1) \mapsto (1, v_1), \dots, (\varsigma(l'), x_e) \mapsto (1, v_e)\}][(\varsigma(a_\varepsilon), x) \mapsto (1, \varsigma(l'))]$. This gives us the state $(\alpha_2, h_2, \varsigma)$.

We have to prove that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ holds. As $\mathcal{X} \neq \varepsilon$, the first requirement of $\text{safeSt}_{\mathcal{P}}$ is satisfied.

For the second requirement, as $\mathcal{X}$ is not an if-statement, $\mathcal{X}$ is the last statement to be executed. It follows that we can satisfy the recursive element and

2c by taking $(\alpha', h', \zeta) = (\alpha_2, h_2, \zeta)$. Furthermore, requirement 2a holds due to $\alpha_2(1) = \alpha_1(1) = \{(\zeta(a_\epsilon), \theta)\}$. As $\pi \rightsquigarrow (\alpha_2, h_2, \zeta)$, we know that there will be only transitions executed by a_ϵ in π .

To prove requirement 2b, we need to prove that all **ComRd**, **ComCons** and **ComWr** transitions executed by $\zeta(a_\epsilon)$ in π are safe:

- The **ComRd** transitions of π are safe as proven in Lemma 8.2.11.
- As $\mathcal{X}$ is in constructor normal form and π contains only transitions executed by a_ϵ , there is only a single **ComCons** transition, as there is only a single **cons**-command in the commands of $\mathcal{X}$.

Let the **ComCons**-transition t of π be annotated with

$(\zeta(a_\epsilon), \ell', \theta'', v'_1, \dots, v'_e)$ for some $\ell' \in \mathcal{L}$, $v'_1, \dots, v'_e \in \mathcal{V}$ and $\theta'' \in \Theta_{\mathcal{P}}$. Due to the definition of $\llbracket \mathcal{X} \rrbracket$, we know that the transition will have had argument θ' during π , so $\theta'' = \theta'$. As the annotations reflect the values used during the **ComCons** transition, we know that $v'_i = v_i$ for all $1 \leq i \leq e$.

As $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, we know that $\ell' \in \alpha_1(3)$. As *fresh*(ℓ') holds for $\alpha_1(3)$ (otherwise the split $P_L * \text{fresh}(\ell')$ would not be possible), $\zeta(\ell') \in \alpha_1(3)$. As there are no restrictions on ζ and $\zeta(\ell') \in \alpha(3)$, let w.l.o.g. $\zeta(\ell') = \ell'$. It follows that $\ell' \in \alpha_2(2)$, by definition of α_2 , and that $h_2(\ell', x_i) = (1, v_i)$ for all $1 \leq i \leq e$, as per the definition of h_2 , from which it follows that the **ComCons** transition is safe.

- By the definition of $\mathcal{X}$ and the fact that π contains only transitions executed by a_ϵ , there is only a single **wr**-command in $\llbracket \mathcal{X} \rrbracket$. As x is a local variable, we know that the write to x will be a **ComWr**-transition.

Let this **ComWr**-transition be annotated in π with $(\zeta(a_\epsilon), \ell_2, x, v', \theta')$ for some $\ell_2 \in \mathcal{L}$, $v' \in \mathcal{V}$ and $\theta' \in \Theta_{\mathcal{P}}$. Per the annotation of the **ComCons** transition, the return value of $\theta'(E_1, \dots, E_e)$ must be ℓ' . It follows that for the annotations to be valid, it must hold that $v' = \ell'$. From $\llbracket \mathcal{X} \rrbracket$ and the fact that x is a single variable in the syntax, we know that $\ell_2 = \zeta(a_\epsilon)$. It then follows that $h_2(\ell_2, x) = (1, v')$, from the definition of h_2 . From the assumption that $\mathcal{P}$ is in constructor normal form, we know that $x \notin \text{Pars}_{\mathcal{P}}(\theta)$, and from the fact that x is in a variable assignment statement and $\mathcal{P}$ is well-formed, we know that x has not been used before, so we know that $h_1(\zeta(a_\epsilon), x) = \perp$.

With this, we know that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds. We then need to prove that $(\alpha_2, h_2, \zeta) \models Q$.

As P_L is a heap-free predicate, it holds that $(\alpha_1, h_1, \zeta) \models P_L * \text{fresh}(\ell')$ and by the definition of α_2 as $\alpha_2(1) = \alpha_1(1)$, $\alpha_2(2) = \{\zeta(\ell'), \theta'\} \cup \alpha_1(2)$ and $\alpha_2(3) = \alpha_1(3) \setminus \{\zeta(\ell')\}$, we know that $(\alpha_2, h_2, \zeta) \models P_L * \blacksquare_2(\ell', \theta')$. As x is fresh (constructor normal form) and not used in h_1 , we know that $h_2 = h_1 \oplus h'$ for a heap h' defined as $h_0[\{(\zeta(\ell'), x_1) \mapsto (1, v_1), \dots, (\zeta(\ell'), x_e) \mapsto (1, v_e)\}][(\zeta(a_\epsilon), x) \mapsto (1, \zeta(\ell'))]$. As P_H is a

triple-free predicate, it follows that

$$(\alpha_2, h_2, \zeta) \models (P_H * (a_\varepsilon, x) \xrightarrow{1} l' * \otimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} \text{eqi}(a_\varepsilon, E_i)).$$

By the definition of h_2 , we know that $h_2 = h_1 \oplus h' \oplus h''$, with $h' = h_0[(\zeta(a_\varepsilon), x) \mapsto (1, \zeta(l'))]$ and $h'' = h_0[\{(\zeta(l'), x_1) \mapsto (1, v_1), \dots, (\zeta(l'), x_e) \mapsto (1, v_e)\}]$. As P_H is a triple-free predicate, we know that $(\alpha_2, h_1, \zeta) \models P_H$. As we set $\zeta(l') = \ell$, we know that $(\alpha_2, h', \zeta) \models (a_\varepsilon, x) \xrightarrow{1} l'$. As $(\alpha_1, h_1, \zeta) \models A(a_\varepsilon, E_1, \dots, E_e)$, we can apply Lemma 8.2.10 to establish that $\text{res}(\text{eqi}(a_\varepsilon, E_i)) = v_i$, from which it follows that $(\alpha_2, h'', \zeta) \models \otimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} \text{eqi}(a_\varepsilon, E_i)$. We then conclude that $(\alpha_2, h_2, \zeta) \models (P_H * (a_\varepsilon, x) \xrightarrow{1} l' * \otimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} \text{eqi}(a_\varepsilon, E_i))$.

As this means that $(\alpha_2, h_2, \zeta) \models Q$, the rule CONSTRUCTOR is sound. $\square$

Soundness of ASSIGNMENT-1

Let $\mathcal{X}$ be a syntactical statement $T x := E$, with E a simple expression and T a syntactic type that is not a struct type. Let the precondition be $P = P_L \wedge P_H$ such that $P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$ and $P_H \Rightarrow \neg(a_\varepsilon, x) \hookrightarrow -$, and let $\models \{P_L \wedge (P_H * (a_\varepsilon, x) \xrightarrow{1} -)\} x := E\{Q\}$ hold. To prove that $\models \{P\} T x := E\{Q\}$ holds, let (α_1, h_1, ζ) be an arbitrary AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P$ and let π be an arbitrary execution of $\mathcal{X}$ from some AuDaLa state p such that $\pi \prec (\alpha_1, h_1, \zeta)$.

As $(\alpha_1, h_1, \zeta) \models P$, let $h'_1 = h_1[(\zeta(a_\varepsilon), x) \mapsto (1, v)]$ for some arbitrary value $v \in \mathcal{V}$. As $P \Rightarrow \neg(l, x) \hookrightarrow -$, it holds that $h_1(\zeta(a_\varepsilon), x) = \perp$. It follows that $(\alpha_1, h'_1, \zeta) \models P * (a_\varepsilon, x) \xrightarrow{1} -$. We then know from $\models \{P * (a_\varepsilon, x) \xrightarrow{1} -\} x := E\{Q\}$ that for all executions π of $x := E$ there exists a (α_2, h_2, ζ) such that $\text{safeSt}_P(\pi, \mathcal{X}, (\alpha_1, h'_1, \zeta), (\alpha_2, h_2, \zeta))$ and $(\alpha_2, h_2, \zeta) \models Q$.

As $\llbracket T x := E \rrbracket = \llbracket x := E \rrbracket$, we know that all executions of $x := E$ are also executions of $T x := E$. Then for an arbitrary execution π of $\mathcal{X}$ we choose (α_2, h_2, ζ) . It follows that in this case, $\text{safeSt}_P(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ iff $\text{safeSt}_P(\pi, x := E, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$, as $x \notin \text{Pars}_P(\theta)$, with θ the struct type of $\zeta(a_\varepsilon)$. Then $\text{safeSt}_P(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds, and as we chose (α_2, h_2, ζ) such that $(\alpha_2, h_2, \zeta) \models Q$, the proof rule ASSIGNMENT is sound.

Soundness of ASSIGNMENT-2

Let $\mathcal{X}$ be a syntactical statement $T x := E$, with E a simple expression and T a syntactic type that is not a struct type. Let the precondition be $P = P_L \wedge P_H$ such that $P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$ and $P_H \Rightarrow (a_\varepsilon, x) \hookrightarrow -$, and let $\models \{P\} x := E\{Q\}$ hold. To prove that $\models \{P\} T x := E\{Q\}$ holds, let (α_1, h_1, ζ) be an arbitrary AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P$ and let π be an arbitrary execution of $\mathcal{X}$ from some AuDaLa state p such that $\pi \prec (\alpha_1, h_1, \zeta)$.

Then as $\llbracket \mathcal{X} \rrbracket = \llbracket x := E \rrbracket$, π is also an execution of $x := E$. It follows from $\models \{P\}x := E\{Q\}$ that there exists a (α_2, h_2, ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds and $(\alpha_2, h_2, \zeta) \models Q$. Therefore, ASSIGNMENT-2 is sound.

Soundness of IF-STATEMENT

Let $\mathcal{X}$ be a syntactical statement ‘if E then $\{S\}$ ’, with E a simple expression and S a list of statements. Let the precondition be P , such that $P \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \wedge A(a_\varepsilon, E)$ and $P \wedge \neg \text{eqi}(a_\varepsilon, E) \Rightarrow Q$. Assume that $\models \{P \wedge \text{eqi}(a_\varepsilon, E)\}S\{Q\}$ holds. Let (α_1, h_1, ζ) be an AuDaLogic state, with $(\alpha_1, h_1, \zeta) \models P$, and let π be an execution of $\mathcal{X}$ from a state p s.t. $\pi \prec (\alpha_1, h_1, \zeta)$.

Let v be the result value of E executed from p . As $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$ and $\pi \prec (\alpha_1, h_1, \zeta)$, π contains only transitions executed by the struct instance of label $\zeta(a_\varepsilon)$, there are no race conditions in π . It follows that the result value of E as executed in π must also be v by Corollary 4.3.23.

As $\mathcal{P}$ is well-formed, either $v = \text{true}$ or $v = \text{false}$ (following Theorem 4.2.21):

- If $v = \text{false}$, let (α_2, h_2, ζ) be (α_1, h_1, ζ) . In this case, we need to prove that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds and that $(\alpha_2, h_2, \zeta) \models Q$. As $v = \text{false}$, the execution of the first statement of the if-statement is also the full execution of $\mathcal{X}$, as the statements S are not executed in this case.

For π , requirement 2a holds, as $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$. Due to the definition of $\llbracket \mathcal{X} \rrbracket$ and the fact that $\mathcal{P}$ is in constructor normal form, π does not contain **ComCons** or **ComWr** transitions, so proving the transitions of π safe is equivalent to proving all **ComRd** transitions safe. For this, we can apply Lemma 8.2.11.

Then to satisfy 2c and the recursive requirement 1, note that π does not contain a **ComIfT** transition (as $v = \text{false}$), so taking (α_2, h_2, ζ) to be (α_1, h_1, ζ) is sufficient.

To prove that $(\alpha_2, h_2, \zeta) \models Q$, we can apply Lemma 8.2.10 to assert that $\neg \text{res}(\text{eqi}(a_\varepsilon, E), \zeta) = \text{true}$. It follows from our assumption $P \wedge \neg \text{eqi}(a_\varepsilon, E) \Rightarrow Q$ that $(\alpha_1, h_1, \zeta) \models Q$.

- If $v = \text{true}$, then due to Lemma 8.2.10, $\text{res}(\text{eqi}(a_\varepsilon, E), \zeta) = \text{true}$. As π is an execution of $\mathcal{X}$ for which E is true, it can be split into $\pi_1; \pi_2$, such that π_1 is the execution of the first statement of $\mathcal{X}$ to a state p_1 and π_2 is the execution of S from p_1 .

As if-statement executions do not contain **ComWr** or **ComCons** statements by $\llbracket \mathcal{X} \rrbracket$ and the fact that $\mathcal{P}$ is in constructor normal form, it follows that $p_1 > (\alpha_1, h_1, \zeta)$. By the fact that $\models \{P \wedge \text{eqi}(a_\varepsilon, E)\}S\{Q\}$ holds, there then exists a state (α', h', ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi_2, S, (\alpha_1, h_1, \zeta), (\alpha', h', \zeta))$ and $(\alpha', h', \zeta) \models Q$.

We then take $(\alpha_2, h_2, \varsigma) = (\alpha', h', \varsigma)$, and prove that $\text{safeStP}(\pi, S, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ holds. As $\mathcal{X} \neq \varepsilon$, requirement 1 of safeStP holds. Requirement 2a holds because $(\alpha_1, h_1, \varsigma) \models \blacksquare_1(a_\varepsilon, \theta)$. Requirement 2b holds because we can apply Lemma 8.2.11 to assert that all **ComRd** transitions are safe in π_1 . As earlier asserted, π_1 does not contain **ComWr** and **ComCons** transitions, so it follows that all transitions in π_1 are safe. Requirement 2c holds because $\text{safeStP}(\pi_2, S, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ holds.

As $(\alpha', h', \varsigma) \models Q$, we know that $(\alpha_2, h_2, \varsigma) \models Q$.

As in both cases, there exists a AuDaLogic state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeStP}(\pi, S, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ holds and $(\alpha_2, h_2, \varsigma) \models Q$, the proof rule IF-STATEMENT is sound. $\square$

Soundness of SEQUENCE

Let $\mathcal{X}$ be the list of statements $S;S$. Let P be the precondition such that $P \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$ and let $\models \{P\}S\{R\}$ and $\models \{R\}S\{Q\}$ hold. To prove $\models \{P\}\mathcal{X}\{Q\}$, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P$, and let π be an execution of $\mathcal{X}$ from a state p such that $\pi \prec (\alpha_1, h_1, \varsigma)$.

As $\mathcal{X} = S;S$, α_1 contains only one active struct instance and $\pi \prec (\alpha_1, h_1, \varsigma)$, we know we can split π into two executions $\pi_S; \pi_S$, where π_S is exactly the execution of S in π from state p to p_1 and π_S is exactly the execution of S in π from p_1 . As $\models \{P\}S\{R\}$, we know that there exists an AuDaLogic state (α', h', ς) such that $\text{safeStP}(\pi_S, S, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ and $(\alpha', h', \varsigma) \models R$. From Lemma 8.1.7 we then know that $p_1 > (\alpha', h', \varsigma)$, from which it follows that $\pi_S \prec (\alpha', h', \varsigma)$ (as $\pi \prec (\alpha_1, h_1, \varsigma)$).

As $(\alpha', h', \varsigma) \models R$ and $\models \{R\}S\{Q\}$, we also know that $(\alpha', h', \varsigma) \models \{R\}S\{Q\}$, so there exists an AuDaLogic state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeStP}(\pi_S, S, (\alpha', h', \varsigma), (\alpha_2, h_2, \varsigma))$ and $(\alpha_2, h_2, \varsigma) \models Q$. It follows that $\text{safeStP}(\pi, S;S, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$: requirement 2a and 2b are satisfied because $\text{safeStP}(\pi_S, S, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ holds and 2c is satisfied because $\text{safeStP}(\pi_S, S, (\alpha', h', \varsigma), (\alpha_2, h_2, \varsigma))$ holds. As $(\alpha_2, h_2, \varsigma) \models Q$, it follows that SEQUENCE is sound. $\square$

8.3.2 Soundness of the parallel rules

In this section, we prove the parallel rules of the proof system of AuDaLogic. To prove the proof rules in this section, we need to work with the safety predicate for schedules.

Considering that the soundness proof of SPLIT PARALLEL is one of the most complicated proofs using the safety predicate for schedules, we first give an intuition on how soundness proofs for rules dealing with the schedule work.

This not only applies for this subsection, but also for the proofs of the schedule rules in Subsection 8.3.3.

For proof rules with as their conclusion a statement triple of the form $\vdash \{P\}\Delta\{Q\}$ with a schedule Δ , for all premise AuDaLogic states $(\alpha_1, h_1, \varsigma)$ and all possible executions π of Δ , we need to prove that the safety predicate for schedules $\text{safeScP}(\pi, \Delta, (\alpha_1, h_1, \varsigma), Q)$ holds. To do this, we need to prove two things, denoted by the requirements of safeScP . Firstly, we need to prove that π is finite; if π is infinite, it does not end, so it does not strictly have a postcondition. Secondly, we have to prove that after the execution π of Δ , we end up in a state that satisfies Q .

This second proof obligation corresponds to requirements 2 and 3 of the safety predicate for schedules. While this is an inductive proof, we can often leave the exact induction implicit, as many proofs have only a single element in the schedule, or can simplify the induction using their premises. We therefore mostly deal with the first step executed in the schedule, or we can create the full inductive proof using some of our premises.

The idea is that for every step F , one needs to be able to specify a postcondition state $(\alpha_t, h_t, \varsigma)$ which reflects the state after executing F . To do this, we consider the execution of F , which consists of an **Init**-transition and the execution π_F of the statements of F by all active struct instances. If π_F deviates from $(\alpha_1, h_1, \varsigma)$, the AuDaLogic framework cannot account for π_F , and we consider the execution from there on safe. This is safe to do, as explained in Section 8.1. We therefore assume that π_F does not deviate.

Furthermore, one needs to define a split among the precondition state for F , the postcondition state and π_F such that conditions 3a-3e holds. As t is an **Init**-transition, it makes struct instances created during the last step active, which is signified by using the maximally active label triple of the precondition label triple for π_F . The requirements 3a to 3c then often follow from the definition of the split, or from the premises. These requirements assert that the split correctly splits the AuDaLogic states and execution π_F into separate executions for individual struct instances contained in the precondition state. Requirement 3d requires us to prove that the individual executions are safe, and that the postcondition state is a correct representation of all individual postcondition states. This often follows from the premises, and we can also often leave the exact postcondition state abstract and define it as a combination of the individual postcondition states. Requirement 3e then requires that the execution of all steps following F is also safe (or deviates). In many cases, the schedule has only a single element, in which case we can satisfy this case by proving that the postcondition state of F satisfies Q . Other proofs allow us to conclude 3e from the premises. Then $\text{safeScP}(\pi, \Delta, (\alpha_1, h_1, \varsigma), Q)$ holds and we conclude that the proof rule is sound.

Soundness of SPLIT PARALLEL

Let $\mathcal{X}$ be a schedule consisting of the step F . Let $P_1 * P_2$ be the precondition and let $Q_1 * Q_2$ be the postcondition. Additionally, let $\models \{P_1\}F\{Q_1\}$ and $\models \{P_2\}F\{Q_2\}$ hold. Then to prove that $\models \{P_1 * P_2\}F\{Q_1 * Q_2\}$, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P_1 * P_2$ and let π be an execution of F from p such that $p > (\alpha_1, h_1, \varsigma)$. As F is a single step, we know that the step sequence of π is $(F, \top)$. We then need to prove that $\text{safeScP}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$.

As the step sequence of π is $(F, \top)$, it holds that π is finite, which satisfies the first requirement of safeScP . As the step sequence is not empty, the second requirement of safeScP is also satisfied.

Then, for requirement 3, from the fact that the schedule is F , it follows that $\pi = t; \pi_F$, with t the **Init**-transition of F . Let p_1 be the state before executing π_F . Due to the definition of AuDaLogic state correspondence, the fact that $p \in \Xi(\alpha_1, h_1, \varsigma)$ and the fact that t is an **Init**-transition (which loads commands into struct instances), we know that $p_1 \in \Xi(\alpha_u, h_1, \varsigma)$, with $\alpha_u = m(\alpha_1)$. (Recall that this means that $\alpha_u(1) = \alpha_1(1) \cup \alpha_1(2)$ and $\alpha_u(2) = \emptyset$.) If $\pi_F \not\sim (\alpha_u, h_1, \varsigma)$, the safety predicate is satisfied, and the rule is sound. Therefore, assume that $\pi_F \sim (\alpha_u, h_1, \varsigma)$.

To prove requirement 3, we need to prove that there exists a state $(\alpha_2, h_2, \varsigma)$ such that there exist the splits outlined in safeScP for which 3a to 3e hold. As there is only one step in the step sequence, proving 3e is done by establishing that $(\alpha_2, h_2, \varsigma) \models Q$, which satisfies the recursive element.

To establish the state $(\alpha_2, h_2, \varsigma)$ and the split for which 3a-3e hold, we use the premises, as they imply the existence of such a split. In the rest of this proof, we first define the state $(\alpha_2, h_2, \varsigma)$ and the splits as implied by the premises, which we then combine to get a *combined split* which satisfies requirement 3 for the whole.

As the precondition is $P_1 * P_2$, we know there exists a split $(\alpha_1, h_1, \varsigma) = (\alpha_1^1, h_1^1, \varsigma) \star (\alpha_1^2, h_1^2, \varsigma)$ such that $(\alpha_1^1, h_1^1, \varsigma) \models P_1$ and $(\alpha_1^2, h_1^2, \varsigma) \models P_2$. It follows by definition of α_u that there also exists a split $(\alpha_u, h_1, \varsigma) = (\alpha_u^1, h_1^1, \varsigma) \star (\alpha_u^2, h_1^2, \varsigma)$ such that $m(\alpha_1^1) = \alpha_u^1$ and $m(\alpha_1^2) = \alpha_u^2$.

As $\pi_F \sim (\alpha_u, h_1, \varsigma)$, we also have the split $\pi_F = \pi_F^1 \parallel \pi_F^2$ such that $\pi_F^i \sim (\alpha_u^i, h_1^i, \varsigma)$ for $i \in \{1, 2\}$. By the definition of π_F^1 and π_F^2 , both start in AuDaLa state p_1 . From the proof rule premises $\models \{P_i\}F\{Q_i\}$, with $i \in \{1, 2\}$, and as $m(\alpha_1^i) = \alpha_u^i$, we know that $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds. Furthermore, as $\pi_F^i \sim (\alpha_u^i, h_1^i, \varsigma)$, we know that this is because there exists a state $(\alpha_2^i, h_2^i, \varsigma)$ that satisfies requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ and for which $(\alpha_2^i, h_2^i, \varsigma) \models Q_i$ holds.

Then as $(\alpha_u, h_1, \varsigma) = (\alpha_u^1, h_1^1, \varsigma) \star (\alpha_u^2, h_1^2, \varsigma)$, as $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds and $\pi_F^i \sim (\alpha_u^i, h_1^i, \varsigma)$ for $i \in \{1, 2\}$, we can conclude that $\text{def}((\alpha_2^1, h_2^1, \varsigma) \star (\alpha_2^2, h_2^2, \varsigma))$, as the executions π_F^1 and π_F^2 will not go outside their

own heap. We fix $(\alpha_2, h_2, \varsigma) = (\alpha_2^1, h_2^1, \varsigma) \star (\alpha_2^2, h_2^2, \varsigma)$.

With $(\alpha_2, h_2, \varsigma)$ established, we define the *individual splits* for both separate states. As $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds for all $i \in \{1, 2\}$, for $(\alpha_u^i, h_1^i, \varsigma)$, $(\alpha_2^i, h_2^i, \varsigma)$ and π_F^i there exist the individual splits

$$(\alpha_u^i, h_1^i, \varsigma) = ((\alpha_u^i)_r, (h_1^i)_r, \varsigma) \star ((\alpha_u^i)_1, (h_1^i)_1, \varsigma) \star \dots \star ((\alpha_u^i)_m, (h_1^i)_m, \varsigma),$$

$$(\alpha_2^i, h_2^i, \varsigma) = ((\alpha_u^i)_r, (h_1^i)_r, \varsigma) \star ((\alpha_2^i)_1, (h_2^i)_1, \varsigma) \star \dots \star ((\alpha_2^i)_m, (h_2^i)_m, \varsigma)$$

and $\pi_F^i = \tau_1^i \parallel \dots \parallel \tau_m^i$, such that requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds.

These splits split the execution π_F^i in m individual executions, and splits the heaps $(\alpha_u^i, h_1^i, \varsigma)$ and $(\alpha_2^i, h_2^i, \varsigma)$ in $m+1$ parts, where the first part $((\alpha_u^i)_r, (h_1^i)_r, \varsigma)$ does not execute anything and the other m parts all have an individual execution. As requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ (for all $i \in \{1, 2\}$) holds for their respective splits, more specifically, we know that the following properties hold for these splits:

1. For every $(\ell, \theta) \in \alpha_u^i(1)$, there exists a $1 \leq j \leq m$, $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$.
2. For every transition π_F^i annotated by an execution label ℓ , there exists a $1 \leq j \leq m$ such that $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$ for some struct type θ .
3. For all $1 \leq j \leq m$, with π_F^i the execution of τ_j^i from p_1 , $\pi_F^i \rightsquigarrow ((hl_u^i)_j, (h_1^i)_j, \varsigma)$ and $\text{safeStP}(\pi_F^i, \text{StepStatP}, ((\alpha_u^i)_j, (h_1^i)_j, \varsigma), ((\alpha_2^i)_j, (h_2^i)_j, \varsigma))$ holds.

Due to the existence of the above splits for $(\alpha_u^1, h_1^1, \varsigma)$, $(\alpha_2^1, h_2^1, \varsigma)$ and π_F^1 and for $(\alpha_u^2, h_1^2, \varsigma)$, $(\alpha_2^2, h_2^2, \varsigma)$ and π_F^2 , there exists the *combined splits* of $(\alpha_u, h_1, \varsigma)$, $(\alpha_2, h_2, \varsigma)$ and π_F defined as follows:

$$(\alpha_u, h_1, \varsigma) = (\alpha'_u, h'_1, \varsigma) \star ((\alpha_u^1)_1, (h_1^1)_1, \varsigma) \star \dots \star ((\alpha_u^1)_m, (h_1^1)_m, \varsigma) \star ((\alpha_u^2)_1, (h_1^2)_1, \varsigma) \star \dots \star ((\alpha_u^2)_m, (h_1^2)_m, \varsigma),$$

with $\alpha'_u = (\alpha_u^1)_r \diamond (\alpha_u^2)_r$ and $h'_1 = (h_1^1)_r \oplus (h_1^2)_r$,

$$(\alpha_2, h_2, \varsigma) = (\alpha'_u, h'_1, \varsigma) \star ((\alpha_2^1)_1, (h_2^1)_1, \varsigma) \star \dots \star ((\alpha_2^1)_m, (h_2^1)_m, \varsigma) \star ((\alpha_2^2)_1, (h_2^2)_1, \varsigma) \star \dots \star ((\alpha_2^2)_m, (h_2^2)_m, \varsigma),$$

and $\pi_F = \tau_1^1 \parallel \dots \parallel \tau_m^1 \parallel \tau_1^2 \parallel \dots \parallel \tau_m^2$.

For these splits, the requirements 3a-3e hold for $\text{safeScP}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$:

- 3a This requirement holds by the definition of the combined splits.
- 3b For every pair $(\ell, \theta) \in \alpha_u(1)$, we know that $(\ell, \theta) \in \alpha_u^1(1)$ or $(\ell, \theta) \in \alpha_u^2(1)$. It follows that for some $1 \leq j \leq m$, $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$. As $(\alpha_u^i)_j$ is also formed in the combined split, it follows that 3b holds.

- 3c From property 2 for the individual splits we know that for every command transition contained in π_F^i annotated with an execution label ℓ , there exists a $1 \leq j \leq m$ such that $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$ for some struct type θ . As $\pi_F = (\pi_F')^1 \parallel (\pi_F')^2$ and all elements $(\alpha_u^i)_j$ are also in the combined split of (α_u, h_1, ζ) , this property still holds for π_F and the combined splits of (α_u, h_1, ζ) .
- 3d As all τ_j^i from p_1 are included in the big split, as well as all $(hl_u^i)_j$ and $(h_1^i)_j$, this property holds directly due to property 3 for the individual splits.
- 3e As $(\alpha_2^1, h_2^1, \zeta) \models Q_1$ and $(\alpha_2^2, h_2^2, \zeta) \models Q_2$ and by the definition of the separator, we know that $(\alpha_2, h_2, \zeta) \models Q_1 * Q_2$. This satisfies 3e and the recursive element, as the schedule consists of only F .

It follows that $\text{safeScP}(t; \pi_F, F, (\alpha_1, h_1, \zeta), Q)$, so the rule SPLIT PARALLEL is sound. $\square$

Soundness of GEN PARALLEL

The proof for the soundness of GEN PARALLEL is the analogous to the proof of SPLIT PARALLEL, but expanded for the pre- and postcondition $P(1) * \dots * P(n)$ and $Q(1) * \dots * Q(n)$, instead of $P_1 * P_2$ and $Q_1 * Q_2$. Though the premise is different, as i is a free variable in the premise, it can be taken to be any value between 1 and n , and as such applies on any part of the split of P and Q . $\square$

8.3.3 Soundness of the schedule rules

For this section of proof rules, the intuition presented in Section 8.3.2 for how soundness proofs for schedule rules work applies.

Soundness of BARRIER

Let $\mathcal{X}$ be a schedule of the form $\Delta_1 < \Delta_2$. Let the precondition be P , let the postcondition be Q and let R be a predicate. Let $\models \{P\}\Delta_1\{R\}$ and $\models \{R\}\Delta_2\{Q\}$ hold. To prove that $\models \{P\}\Delta_1 < \Delta_2\{Q\}$, let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P$ and let π be an execution of $\mathcal{X}$ from an AuDaLa state p such that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$. Let $\mathcal{F}$ be the step sequence of π .

As $\models \{P\}\Delta_1\{R\}$ holds, we know from the definition of AuDaLogic triples and safeScP that any execution of Δ_1 is finite and thus terminates. It follows that $\pi = \pi_1; \pi_2$, where π_1 is the execution until after the last transition of the last step executed for Δ_1 , and it follows that $\mathcal{F} = \mathcal{F}_1; \mathcal{F}_2$, with $\mathcal{F}_1$ the step sequence of Δ_1 . Let π_1 end at the AuDaLa state p_1 .

We know from our premise $\models \{P\}\Delta_1\{R\}$ that $\text{safeScP}(\pi_1, \Delta_1, (\alpha_1, h_1, \varsigma), R)$ holds. Then either π_1 deviates from the AuDaLogic states between every step of Δ_1 , or it does not. If it deviates, then for one of the subcalls of $\text{safeScP}(\pi_1, \Delta_1, (\alpha_1, h_1, \varsigma), R)$, requirement 3 is vacuously true, from which it follows that $\text{safeScP}(\pi_1; \pi_2, \Delta_1 < \Delta_2, (\alpha_1, h_1, \varsigma), Q)$ is also true, as the same requirement 3 will also be true. Therefore, assume that π_1 never deviates for any step in Δ_1 .

As π_1 never deviates, it follows by multiple applications of Lemma 8.1.7 for all subcalls of $\text{safeScP}(\pi_1, \Delta_1, (\alpha_1, h_1, \varsigma), R)$ that in the final state p_1 of π_1 , there exists a AuDaLogic state (α', h', ς) such that $p_1 > (\alpha', h', \varsigma)$ and $(\alpha', h', \varsigma) \models R$. Then by our premise $\models \{R\}\Delta_2\{Q\}$, we know that for all executions π_2 , $\text{safeScP}(\pi_2, \Delta_2, (\alpha', h', \varsigma), Q)$ holds.

Note that at the end of π_1 , the recursive element in requirement 3e of $\text{safeScP}(\pi_1, \Delta_1, (\alpha_1, h_1, \varsigma), R)$ requires that $(\alpha', h', \varsigma) \models R$ holds. We substitute this recursive element with the fact that $\text{safeScP}(\pi_2, \Delta_2, (\alpha', h', \varsigma), Q)$ holds, from which we can conclude that $\text{safeScP}(\pi_1; \pi_2, \Delta_1 < \Delta_2, (\alpha_1, h_1, \varsigma), Q)$ holds. Therefore, the BARRIER rule is sound. $\square$

Soundness of STEP CALL

Let $\mathcal{X}$ be a schedule of the form F . Let the precondition be P such that there exists a struct type θ and an $a_\epsilon \in \mathfrak{A}$ such that $S(P, P')$ and $P' \Rightarrow \blacksquare_1(a_\epsilon, \theta)$. Let $\text{StepStatP}(F, \theta) = \mathcal{S}$ and let $\models \{P\}\mathcal{S}\{Q\}$ hold.

to prove that $\models \{P\}F\{Q\}$, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P$ and let π be an execution of F from an AuDaLa state $p > (\alpha_1, h_1, \varsigma)$. As the schedule is F , we know that the step sequence of F is $(F, \top)$ (as F is a global step call). We then need to prove that $\text{safeScP}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$ holds.

As the schedule is F , we know that $\pi = t; \pi_F$, so requirement 1 of $\text{safeScP}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$ holds. Requirement 2 holds because the step sequence of π is $(F, \top)$. As in the safety predicate for schedules, let $\alpha_u = m(\alpha_1)$. As $S(P, P')$, it follows that $(\alpha_u, h_1, \varsigma) \models P' \wedge \emptyset_2$.

To prove requirement 3, assume that $\pi_F \frown (\alpha_u, h_1, \varsigma)$ (as otherwise requirement 3 of $\text{safeScP}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$ is vacuously true). We need to prove that there exists a state $(\alpha_2, h_2, \varsigma)$ such that there exist the splits outlined in safeScP for which 3a to 3e hold. As there is only one step in the step sequence, proving 3e is done by establishing that $(\alpha_2, h_2, \varsigma) \models Q$, which satisfies the recursive element.

We make the split $(\alpha_u, h_1, \varsigma) \star (\alpha_0, h_0, \varsigma)$ for requirement 3, where α_0 is the empty label triple. Intuitively, this can be considered as not splitting $(\alpha_u, h_1, \varsigma)$ and just adding on an empty state. To satisfy requirement 3a, we will make the same trivial split $(\alpha_2, h_2, \varsigma) \star (\alpha_0, h_0, \varsigma)$ for our currently abstract state $(\alpha_2, h_2, \varsigma)$.

Requirement 3b holds by definition of (α_u, h_1, ζ) . Requirement 3c holds because $\pi \rightsquigarrow (\alpha_u, h_1, \zeta)$ and $(\alpha_u, h_1, \zeta) \models \blacksquare_1(l, \theta) \wedge \emptyset_2$, which means that any command transition in π must be executed by $\zeta(a_\epsilon)$.

From the fact that $\pi \rightsquigarrow (\alpha_u, h_1, \zeta)$, the fact that $\alpha_u(1) = \{\zeta(a_\epsilon), \theta\}$ and the fact that $\alpha_u(2) = \emptyset$, it holds that π_F is exactly an execution of $\text{StepStat}_{\mathcal{P}}(F, \theta)$. Additionally, due to the definition of AuDaLogic state correspondence, the fact that t is an **Init**-transition (which loads commands into struct instances), we know that $\pi_F \rightsquigarrow (\alpha_u, h_1, \zeta)$. Then by the premise $\models \{P\}\mathcal{S}\{Q\}$, we know that there exists a state (α', h', ζ) such that $\text{safeScP}(\pi_F, \mathcal{S}, (\alpha_u, h_1, \zeta), (\alpha', h', \zeta))$ and $(\alpha', h', \zeta) \models Q$. This satisfies requirement 3d.

We fix $(\alpha_2, h_2, \zeta) = (\alpha', h', \zeta)$. This satisfies requirement 3e and the recursive element, as this means that $(\alpha_2, h_2, \zeta) \models Q$. Therefore, the proof rule **STEP CALL** is sound. $\square$

Soundness of TYPED CALL

Let $\mathcal{X}$ be a schedule of the form $\theta.F$. Let the precondition be $P_1 * P_2$ and let the postcondition be $Q_1 * P_2$, such that $P_1 \Rightarrow \blacksquare \theta$ and $P_2 \Rightarrow \neg \Box \theta$. Let $\models \{P_1\}F\{Q\}$ hold.

To prove $\models \{P_1 * P_2\}F\{Q * P_2\}$, let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P_1 * P_2$ and let π be an execution of $\mathcal{X}$ from an AuDaLa state $p \in \Xi(\alpha_1, h_1, \zeta)$. We prove that $\text{safeScP}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), Q * P_2)$ holds. As $\mathcal{X} = \theta.F$, we know that $\pi = t; \pi_F$, with π_F the command transitions of $\theta.F$ and t the transition **InitL** for θ . We then know that π is finite (as steps are finite), and that $\mathcal{F}$ must be (F, θ) . It follows that the first two requirements are satisfied.

For the third requirement, we have to prove that if $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, there exists a state (α_2, h_2, ζ) such that requirements 3a-3e hold. Assume that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, as otherwise this requirement is vacuously satisfied.

According to the precondition, there exists a split $(\alpha_1, h_1, \zeta) = (\alpha_1^1, h_1^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$ where $(\alpha_1^1, h_1^1, \zeta) \models P_1$ and $(\alpha_1^2, h_1^2, \zeta) \models P_2$. As **InitL** for θ only gives struct instances of type θ command transitions and $(\alpha_1^2, h_1^2, \zeta) \models \neg \Box \theta$ as defined in the premise, we know that $\pi \rightsquigarrow (\alpha_1^1, h_1^1, \zeta)$, as all struct elements with transitions in $\theta.F$ must be in α_1^1 . As $(\alpha_1^1, h_1^1, \zeta) \models P_1$, we can conclude from $\models \{P_1\}F\{Q\}$ that $\text{safeScP}(t; \pi_F, F, (\alpha_1^1, h_1^1, \zeta), Q)$. It follows that there exists a state $(\alpha_2^1, h_2^1, \zeta)$ and a split of $(\alpha_1^1, h_1^1, \zeta)$, $(\alpha_2^1, h_2^1, \zeta)$ and $\pi_F = \tau_1 \parallel \dots \parallel \tau_n$ such that requirements 3a-d hold for these splits.

Then as $(\alpha_1, h_1, \zeta) = (\alpha_1^1, h_1^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$, as $\text{safeScP}(t; \pi_F, F, (\alpha_1^1, h_1^1, \zeta), Q)$ holds and $t; \pi_F \rightsquigarrow (\alpha_1^1, h_1^1, \zeta)$, we can conclude that $\text{def}((\alpha_2^1, h_2^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta))$, as the execution $t; \pi_F$ will not touch the heap h_1^2 .

We then fix $(\alpha_2, h_2, \zeta) = (\alpha_2^1, h_2^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$. To prove requirement 3, we can then take the split as used in $\text{safeScP}(t; \pi_F, F, (\alpha_1^1, h_1^1, \zeta), Q)$, and add $(\alpha_1^2, h_1^2, \zeta)$ to the part of the split that does not execute. Then 3a to 3d hold, as

they hold in $\text{safeScP}(t; \pi_F, F, (\alpha_1^1, h_1^1, \zeta), Q)$, as there are no command executions for any of the struct instances in α_1^2 in π_F . By definition of the separator, it also holds that $(\alpha_2, h_2, \zeta) \models Q * P_2$, so $\exists e$ and the recursive element hold as well. It follows that the proof rule **TYPED CALL** is sound. $\square$

Soundness of **FIXPOINT**

Let $\mathcal{X}$ be a schedule of the form $\text{Fix}(\Delta)$. Let the precondition be $P \wedge \text{Va}(0)$ and let the postcondition be $Q = P \wedge \text{Va}(n+1) \wedge \text{stable}_{\mathcal{P}}(\Delta)$.

Let there exist an $n \geq 0$ such that for all $k \geq n$, it holds that $P \wedge \text{Va}(k) \Rightarrow \text{stable}_{\mathcal{P}}(\Delta)$. Let there furthermore exist some fresh AuDaLogic variable j such that $\models \{P \wedge \text{Va}(j) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge \text{Va}(j+1)\}$ hold. As j stands for an abstract natural number, due to the requirements on Va , we can conclude from this premise that for any natural number i , $\models \{P \wedge \text{Va}(i) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge \text{Va}(i+1)\}$ holds.

To prove $\models \{P \wedge \text{Va}(0)\} \text{Fix}(\Delta) \{P \wedge \text{Va}(n+1) \wedge \text{stable}_{\mathcal{P}}(\Delta)\}$, let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P \wedge \text{Va}(0)$ and let π be an execution of $\mathcal{X}$ from an AuDaLa state $p \in \Xi(\alpha_1, h_1, \zeta)$. We then prove that $\text{safeScP}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), Q)$ holds.

As $\mathcal{X} = \text{Fix}(\Delta)$, we know that we can partition π into iterations of the fixpoint. The schedule $\text{Fix}(\Delta)$ can then also be represented by $\Delta <_1 \Delta <_2 \dots$, a chain of executions of Δ of unknown length (by the semantics of AuDaLa). Then, by induction, we prove the following:

For all $i > 1$, let π_i be i executions of Δ , separated by a barrier, such that $\pi_1 = \Delta$, $\pi_2 = \Delta < \Delta$ and so on. Then $\text{safeScP}(\pi_i, \Delta_i, (\alpha_1, h_1, \zeta), P \wedge a \geq 0 \wedge \text{Va}(i))$ holds.

As induction hypothesis, we assume that $\text{safeScP}(\pi_{i-1}, \Delta_{i-1}, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(i))$ holds.

- As the base case, π_1 is the execution of Δ starting from $p \in \Xi(\alpha_1, h_1, \zeta)$. As $(\alpha_1, h_1, \zeta) \models P \wedge \text{Va}(0)$, and $\models \{P \wedge \text{Va}(0) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge \text{Va}(1)\}$ holds, it follows that $\text{safeScP}(\pi_1, \Delta_1, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(1))$ holds.
- Let π_i be the execution of Δ_i . Then from the induction hypothesis we know that $\text{safeScP}(\pi_{i-1}, \Delta_{i-1}, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(i-1))$ holds. Assume that this is not because π_{i-1} diverges from the active labels, as then $\text{safeScP}(\pi_i, \Delta_i, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(i))$ holds vacuously. Then by $i-1$ applications of Lemma 8.1.7, we know that in the final state p_{i-1} of Δ_{i-1} there exists a state (α_i, h_i, ζ) such that $p_i > (\alpha_i, h_i, \zeta)$ and $(\alpha_i, h_i, \zeta) \models P \wedge \text{Va}(i-1)$. Then, as $\pi_i = \pi_{i-1}; \pi'$, we know that π' starts in p_{i-1} .

Then, by the fact that $\models \{P \wedge \text{Va}(j) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge \text{Va}(j+1)\}$ holds, we know that $\models \{P \wedge \text{Va}(i-1) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge \text{Va}(i)\}$ holds. It follows that

$\text{safeScP}(\pi', \Delta, (\alpha_i, h_i, \zeta), P \wedge \text{Va}(i))$ holds. If we add that to the end of the proof of $\text{safeScP}(\pi_{i-1}, \Delta_{i-1}, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(i-1))$ we can use that full proof to conclude that $\text{safeScP}(\pi_i, \Delta_i, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(i))$ holds.

To extend this to prove that $\text{safeScP}(\pi, \text{Fix}(\Delta), (\alpha_1, h_1, \zeta), Q)$ holds, we use that for all $k \geq n$, it holds that $P \wedge \text{Va}(k) \Rightarrow \text{stable}_P(\Delta)$. As discussed in Section 8.2, $\text{stable}_P(\Delta)$ can be split up in subpredicates for every step in the schedule, and then into even smaller predicates for every struct instance executing the step. By application of Lemma 8.2.13, we then know that for each struct instance executing the step from some AuDaLogic state (α, h, ζ) , it holds that for some heap h' such that h and h' are identical for the parameters, $\text{safeStStrP}(\pi_S, \mathcal{S}, (\alpha, h, \zeta), (\alpha, h', \zeta))$, with $\mathcal{S}$ the statements of the step and π_S the execution of the step by the struct instance. Then that execution will not change any parameters, nor will it create new struct instances. We can then generalize this back to all struct instances and all steps. It follows that the fixpoint is stable after iteration n .

It follows that $\text{Fix}(\Delta) = \Delta_{n+1}$ (the extra iteration is due to AuDaLa semantics) and that π is an execution of Δ_{n+1} . By our induction proof above, we then know that $\text{safeScP}(\pi, \Delta_{n+1}, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(n+1))$ holds. Then, as $P \wedge \text{Va}(n+1) \Rightarrow \text{stable}_P(\Delta)$, we know that $\text{safeScP}(\pi, \Delta_{n+1}, (\alpha_1, h_1, \zeta), P \wedge \text{Va}(n+1) \wedge \text{stable}_P(\Delta))$ holds. We conclude that $\text{safeScP}(\pi_{n+1}, \text{Fix}(\Delta), (\alpha_1, h_1, \zeta), P \wedge \text{Va}(n+1) \wedge \text{stable}_P(\Delta))$ holds, so the rule **FIXPOINT** is sound. $\square$

8.3.4 Soundness of the general rules

In this section, we prove the general proof rules of AuDaLogic correct. The proofs in this section generally prove that the conclusion of a proof rule holds due to the premises given from that proof rule.

Soundness of **EMPTY**

Let $\mathcal{X}$ be the empty statement or schedule ε . Then, let the precondition and postcondition be P .

To prove that $\models \{P\}\varepsilon\{P\}$, let (α_1, h_1, ζ) such that $(\alpha_1, h_1, \zeta) \models P$ and let π be an execution of ε starting in a AuDaLa state p such that $p \succ (\alpha_1, h_1, \zeta)$.

Then as $\pi = \varepsilon$, $\mathcal{F}$ is ε . By the definition of safeScP , we then know $\models \{P\}\varepsilon\{P\}$ holds if ε is the empty schedule due to the fact that $(\alpha_1, h_1, \zeta) \models P$. Furthermore, if ε is the empty statement, we can choose $(\alpha_2, h_2, \zeta) = (\alpha_1, h_1, \zeta)$, for which the statements predicate holds and $(\alpha_2, h_2, \zeta) \models P$ also holds. Then, if ε is the empty statement, we also conclude that $\models \{P\}\varepsilon\{P\}$. It follows that **EMPTY** is sound. $\square$

Soundness of WEAKENING

Let $\mathcal{X}$ be an executable and let P , P' , Q and Q' be predicates such that $P' \Rightarrow P$ and $Q \Rightarrow Q'$. Let $\models \{P\}\mathcal{X}\{Q\}$ hold.

Then either $\mathcal{X}$ is a schedule or $\mathcal{X}$ is a list of statements. If $\mathcal{X}$ is a list of statements, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P'$ and let π be an AuDaLogic execution of $\mathcal{X}$ from a state p such that $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$. As $(\alpha_1, h_1, \varsigma) \models P$ and $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$, we know due to $\models \{P\}\mathcal{X}\{Q\}$ that there exists a state (α', h', ς) such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$ holds and $(\alpha', h', \varsigma) \models Q$ holds. Then from the assumptions we know that $(\alpha', h', \varsigma) \models Q'$, so we fix $(\alpha_2, h_2, \varsigma) = (\alpha', h', \varsigma)$. It follows that there exists a state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ holds and $(\alpha_2, h_2, \varsigma) \models Q'$, so in this case, $\models \{P'\}\mathcal{X}\{Q'\}$ holds.

If $\mathcal{X}$ is a schedule, again, as $(\alpha_1, h_1, \varsigma) \models P'$, $(\alpha_1, h_1, \varsigma) \models P$, so by our assumption $\models \{P\}\mathcal{X}\{Q\}$, we know that $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), Q)$ holds. Then as $Q \Rightarrow Q'$, $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), Q')$ also holds. Then $\models \{P'\}\mathcal{X}\{Q'\}$ holds.

It follows that the proof rule WEAKENING is sound. $\square$

Soundness of DISJUNCTION

Let $\mathcal{X}$ be an executable, let $P_1 \vee P_2$ be a precondition and let $Q_1 \vee Q_2$ be a postcondition. Assume that $\models \{P_1\}\mathcal{X}\{Q_1\}$ and $\models \{P_2\}\mathcal{X}\{Q_2\}$ both hold.

Then either $\mathcal{X}$ is a schedule or $\mathcal{X}$ is a list of statements. If $\mathcal{X}$ is a list of statements, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P_1 \vee P_2$ and let π be an AuDaLogic execution of $\mathcal{X}$ from a state p such that $\pi \rightsquigarrow (\alpha_1, h_1, \varsigma)$. To prove that there exists a state $(\alpha_2, h_2, \varsigma)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha_2, h_2, \varsigma))$ holds and $(\alpha_2, h_2, \varsigma) \models Q_1 \vee Q_2$, we do a case distinction. If $(\alpha_1, h_1, \varsigma) \models P_1$, then by assumption $\models \{P_1\}\mathcal{X}\{Q_1\}$, we know there exists a state $(\alpha', h', \varsigma) \models Q_1$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \varsigma), (\alpha', h', \varsigma))$. Then by implication, $(\alpha', h', \varsigma) \models Q_1 \vee Q_2$, so we fix $(\alpha_2, h_2, \varsigma) = (\alpha', h', \varsigma)$. If $(\alpha_1, h_1, \varsigma) \models P_2$, then we do the same, but with assumption $\models \{P_2\}\mathcal{X}\{Q_2\}$. Then for this case $\models \{P_1 \vee P_2\}\mathcal{X}\{Q_1 \vee Q_2\}$ holds.

If $\mathcal{X}$ is a schedule, the proof follows the same reasoning as the proof for $\mathcal{X}$ as a list of statements (assuming that π does not diverge from the active labels, in which case the proof rule holds vacuously). We conclude that the proof rule DISJUNCTION is sound. $\square$

Soundness of CONJUNCTION

Let $\mathcal{X}$ be an executable, let $P_1 \wedge P_2$ be a precondition and let $Q_1 \wedge Q_2$ be a postcondition. Assume that $\models \{P_1\}\mathcal{X}\{Q_1\}$ and $\models \{P_2\}\mathcal{X}\{Q_2\}$ both hold. Then either $\mathcal{X}$ is a schedule or $\mathcal{X}$ is a list of statements. If $\mathcal{X}$ is a list of statements,

let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P_1 \wedge P_2$ and let π be an AuDaLogic execution of $\mathcal{X}$ from a state p such that $\pi \prec (\alpha_1, h_1, \zeta)$.

We prove that there exists a state (α_2, h_2, ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ holds and $(\alpha_2, h_2, \zeta) \models Q_1 \wedge Q_2$. As $(\alpha_1, h_1, \zeta) \models P_1 \wedge P_2$, by the premises, there exists a state (α', h', ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha', h', \zeta))$ holds and $(\alpha', h', \zeta) \models Q_1$, and there exists a state (α'', h'', ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha'', h'', \zeta))$ holds and $(\alpha'', h'', \zeta) \models Q_2$.

Let p' be the final state of π . Then by Lemma 8.1.7, we know that $p' > (\alpha', h', \zeta)$ and that $p' > (\alpha'', h'', \zeta)$. It follows by the definition of state correspondence (Definition 8.1.1), that there exists a state (α_2, h_2, ζ) such that $(\alpha_2, h_2, \zeta) \simeq p'$, for which it must hold by definition of $\simeq$ (Definition 7.2.8) that $(\alpha_2, h_2, \zeta) \models Q_1 \wedge Q_2$. For this state, $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ also holds, as otherwise either $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha', h', \zeta))$ or $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha'', h'', \zeta))$ would not hold. We then conclude that $\models \{P_1 \wedge P_2\} \mathcal{X} \{Q_1 \wedge Q_2\}$.

The case where $\mathcal{X}$ is a schedule follows the same reasoning (assuming that π does not diverge from the active labels, in which case the proof rule holds vacuously). The existence of the states (α', h', ζ) and (α'', h'', ζ) follow from $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), Q_1)$ and $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), Q_2)$ holding, if π does not deviate. It follows that the proof rule CONJUNCTION is sound. $\square$

Soundness of EXAMPLE

Let $\mathcal{X}$ be an executable, let the precondition be $\exists a.P$ and let the postcondition be $\exists a.Q$. Let $\models \{P\} \mathcal{X} \{Q\}$ hold.

Then either $\mathcal{X}$ is a schedule or $\mathcal{X}$ is a list of statements. If $\mathcal{X}$ is a list of statements, let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models \exists a.P$ and let π be an execution such that $\pi \prec (\alpha_1, h_1, \zeta)$. From the semantic definition of $(\alpha_1, h_1, \zeta) \models \exists a.P$, we know that there exists a $v \in \Theta_{\mathcal{P}} \cup \mathcal{V} \cup \bigcup_n 2^{\mathcal{V}^n}$ such that $(\alpha_1, h_1, \zeta[a \mapsto v]) \models P$.

From the fact that $\models \{P\} \mathcal{X} \{Q\}$ holds, there exists a state $(\alpha_2, h_2, \zeta[a \mapsto v])$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta[a \mapsto v]), (\alpha_2, h_2, \zeta[a \mapsto v]))$ and $(\alpha_2, h_2, \zeta[a \mapsto v]) \models Q$ holds. Then $(\alpha_2, h_2, \zeta) \models \exists a.Q$, by the semantics of AuDaLogic. As $\text{safeSt}_{\mathcal{P}}$ does not impose requirements on the AuDaLogic environment, we also know that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ also holds. It follows that in this case, $\models \{\exists a.P\} \mathcal{X} \{\exists a.Q\}$ holds.

If $\mathcal{X}$ is a schedule, the reasoning is largely the same, but we derive $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta), \exists a.Q)$ from $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta[a \mapsto v]), Q)$ instead, with $(\alpha_2, h_2, \zeta[a \mapsto v])$ defined as the state that must exist to satisfy requirement 2 at the end of $\text{safeSc}_{\mathcal{P}}(\pi, \mathcal{X}, (\alpha_1, h_1, \zeta[a \mapsto v]), Q)$, if π does not deviate. It follows that the proof rule EXAMPLE is sound. $\square$

Soundness of FRAME STATEMENTS

Let $\mathcal{S}$ be a list of statements, let the precondition be $P * R$ and let the postcondition be $Q * R$. Let $R \Rightarrow \emptyset_1$, and let the free variables of R and P not overlap. Let $\models \{P\}\mathcal{S}\{Q\}$ hold.

For this proof of $\models \{P * R\}\mathcal{S}\{Q * R\}$, let $(\alpha_1, h_1, \zeta) \models P * R$ be an AuDaLogic state and let π be an execution of $\mathcal{S}$ such that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$. We prove that there exists an AuDaLogic state (α_2, h_2, ζ) such that $(\alpha_2, h_2, \zeta) \models Q * R$ and $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$.

By the precondition, there exists a split $(\alpha_1, h_1, \zeta) = (\alpha_1^1, h_1^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$ such that $(\alpha_1^1, h_1^1, \zeta) \models P$ and $(\alpha_1^2, h_1^2, \zeta) \models R$. As $R \Rightarrow \emptyset_1$, we know that $\pi \rightsquigarrow (\alpha_1^1, h_1^1, \zeta)$ (if not, then we can simply adjust the split of $\alpha(3)$).

From $\models \{P\}\mathcal{S}\{Q\}$, we know there exists a state $(\alpha_2^1, h_2^1, \zeta)$ such that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha_1^1, h_1^1, \zeta), (\alpha_2^1, h_2^1, \zeta))$ and $(\alpha_2^1, h_2^1, \zeta) \models Q$. As $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha_1^1, h_1^1, \zeta), (\alpha_2^1, h_2^1, \zeta))$ holds, we know that π does not access h_1^2 and does not use labels already used in α_1^2 . It then follows that $(\alpha_2^1, h_2^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$ is defined, so we fix $(\alpha_2, h_2, \zeta) = (\alpha_2^1, h_2^1, \zeta) \star (\alpha_1^2, h_1^2, \zeta)$. Then we know that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$, directly from the fact that $\text{safeSt}_{\mathcal{P}}(\pi, \mathcal{S}, (\alpha_1^1, h_1^1, \zeta), (\alpha_2^1, h_2^1, \zeta))$. As P and R have no overlap in free variables, we then also know that $(\alpha_2, h_2, \zeta) \models Q * R$, as nothing has changed in the meaning of R . It follows that FRAME STATEMENTS is sound. $\square$

Soundness of FRAME SCHEDULE

Most of this proof is analogous to the proof for FRAME STATEMENTS, with the change that due to the definition of $\text{safeSc}_{\mathcal{P}}$, all elements of $\alpha_1(2)$ will also execute the schedule. This is then solved by the fact that $R \Rightarrow \emptyset$ instead of $R \Rightarrow \emptyset_1$. It follows that FRAME SCHEDULE is sound. $\square$

8.4 Conclusion

In this chapter, we have connected AuDaLogic to AuDaLa, based on the approach by Vafeiadis [173]. We have related AuDaLogic and AuDaLa states, defined some context and defined two safety predicates for executions, which check whether a postcondition holds in AuDaLogic when steps or a schedule is executed. With this, we have defined the semantics of AuDaLogic triples.

We have used this to prove our expression translation and our stability predicate generator function, as defined in Chapter 7, correct. We then used that to also prove the soundness for all the proof rules defined in Section 7.4. With this, AuDaLogic is sound for AuDaLa.

Currently, however, the proof system can only be used to prove programs without write-write and read-write race conditions. As seen in Chapter 3, it

can be beneficial to include write-write race conditions in AuDaLa programs. To that extent, in the next chapter, we shall give an extension of AuDaLogic to make it compatible with write-write race conditions.

Chapter 9

Adapting AuDaLogic for Write-Write Race Conditions

So far, we have defined AuDaLogic and proven the proof system sound. However, this definition and the proof system are only applicable on programs without read-write and write-write race conditions. The definition of AuDaLa does not prohibit race conditions, and the properties and proofs of Chapter 4 and Chapter 5 specifically allow for write-write race conditions to occur during steps. To use AuDaLogic in the same capacity, we adapt AuDaLogic in this chapter to include write-write race conditions during the execution of steps.

Section 9.1 adapts AuDaLogic for write-write race conditions occurring during the execution of steps. For the adaptation, the scope being steps is important; we allow two struct instances writing to the same parameter during a step, and we allow this parameter to be read from during a later step, but we do not allow this parameter being read from during the same step as the two writes.

We present the adaptation as changes to Chapter 7 and Chapter 8, ranging from changes to the AuDaLogic state and some operations to changes in the proof rules and their soundness. As these chapters already adapt work by Vafeiadis [173] to work for AuDaLa and introduce new concepts to aid that goal, we have decided to not integrate these changes directly with those chapters but to present them separately, to separate the concerns of the two adaptations and therefore make them more comprehensible.

Section 9.2 gives an example of a program for which we can prove a post-condition using the adapted system. We then conclude the chapter in Section 9.3.

9.1 Adapting AuDaLogic for Write-Write Race Conditions

In this section, we adapt AuDaLogic for write-write race conditions occurring in the scope of step executions. We first give the approach and design decisions of our extension, and then give the required changes to Chapter 7 and Chapter 8. To not repeat Chapter 7 and Chapter 8 in this section, we have decided to only highlight the changes. However, we present this adaptation not as conjecture but as a result, especially when we claim that the soundness proofs are minimally impacted by our changes. To that end, we will include the proofs for our adapted version of the UPDATE rule and our adapted version of the STEP CALL rule. We also give an updated proof for the SPLIT PARALLEL rule, our most complicated proof. These serve as examples for how the soundness proofs of proof rules need to be adapted for our changes. We consider this to demonstrate that the proofs are minimally changed and that integrating the changes into the previous two sections will lead to a fully functioning and sound proof system.

Summary of the changes

To add write-write race conditions to AuDaLogic, we adapt the memory heap to be able to contain sets of values, which represent non-determinism in the value of a certain memory cell. To do this, we adapt the permissions, the $\oplus$ operator and the merging of memory heaps to be able to merge values of memory cells if and only if those cells have a write permission. We also add change booleans to the memory heap to keep track of whether a value was updated. This is beneficial in a write-write race conditions in AuDaLa, where the winning value is a value newly written to the heap cell, and never a value that was already there before the race condition. The merging of memory heaps is then further augmented to remove old values when merging them in new values.

Here, we intentionally decided to not be able to split or merge non-deterministic memory cells with read permissions. This reflects that in AuDaLa programs without read-write race condition, variables cannot be read in the same step they are written to, and in the following steps, there will always be a single, possibly unknown value that is deterministically read from the memory cell. Allowing for non-deterministic reads therefore complicates AuDaLogic far more than necessary to represent AuDaLa.

We adapt the syntax and semantics of memory assertions straightforwardly and add some shorthands. We only adapt the access predicate A to work with read permissions, which means it only holds when all memory cells read from are deterministic and in the heap, which further reflects the way AuDaLa treats

write-write race conditions. This however means that we need to add two more relations, like the relation $S(P, P')$, to be able to reset the predicates for the next step to be executed.

When relating our changes back to AuDaLa, we introduce the notion of a *corresponding deterministic heap* for states and heaps. This reflects that non-determinism is reflected by the value of a variable during an execution instead of non-deterministic variable environments, and that therefore we can always lift our non-deterministic heaps to deterministic heaps with unknown values. The notion of the corresponding deterministic heap, as well as a change boolean reset function r , is then used to adapt the safety predicate for schedules.

In the next subsections, we give the changes to AuDaLogic to allow for write-write race conditions with the provisions for reading from memory cells and the safety predicates as outlined above.

9.1.1 Changes to the syntax and semantics of AuDaLogic

In this section, we give the changes to AuDaLogic in more detail. We first discuss the changes to permissions (and their corresponding AuDaLogic expressions) and the heap definition and an added relation to help reasoning about the changed heaps. We then give the adaptations the null-heap, heap ordering and the heap merge. Lastly, we give the changes to the memory assertion definition.

Changes to permissions and the heap definition

We redefine the set of permissions Z as the set $((0, 1] \times \{R, W\}) \cup \{(\zeta_0, R)\}$, where every permission is a pair (ζ, X) with a designation $X \in \{R, W\}$, where the designation determines whether it is a read or a write permission and ζ determines the value of the permission (between $(0, 1]$ or ζ_0). We define that the permissions $(1, R)$ and $(1, W)$ are equivalent; they represent the full permission, which we also denote with 1 . In the syntax of expressions of AuDaLogic, we then change E_Z to $E_Z ::= (\zeta, X) \mid a$, with $X \in \{R, W\}$ and $\zeta \in (0, 1] \cup \{\zeta_0\}$. Furthermore, we change the resolution function and the AuDaLogic variable environment to reflect this straightforwardly.

To deal with the multiple values returned from a write-write race condition, we redefine the heap as $h: \mathcal{L} \times ID_V \rightarrow Z \times 2^{\mathcal{V}} \times \mathbb{B}$, to allow the heap to hold the multiple potential values of a race condition. The boolean is there to keep note of whether the value set has been written to this cell in the last step, which we use to eliminate old values when combining heaps. As a requirement on these heaps, we do not allow any value set in the heap to be empty.

We then call a heap *deterministic* iff for all $(\ell, x) \in \text{dom}(h)$, with $h(\ell, x) = ((\zeta, X), V, b)$ for some ζ, X, b , $V = \{v\}$ for some $v \in \mathcal{V}$, and *non-deterministic* oth-

erwise. We relate non-deterministic heaps to deterministic heaps using the binary relation $\gg$:

Definition 9.1.1 (deterministic relation $\gg$). With h' a non-deterministic heap and h a deterministic heap, we define that $h' \gg h$ holds iff $\text{dom}(h) = \text{dom}(h')$ and for all (ℓ, x) such that $h'(\ell, x) = ((\zeta, X), V, b)$ where $V = \{v_1, \dots, v_n\}$ for $n \geq 1$, $h(\ell, x) = ((\zeta, X), \{v_i\}, b)$ for some $v_i \in V$.

Intuitively, this relation relates non-deterministic heaps to deterministic heaps which contain only values also found in the non-deterministic heap.

Changes to the null-heap, heap ordering and the heap merge

We adapt the definitions which were built on the heap definition to reflect the changes in the heap definition. For Definition 7.2.7, the null-heap has the change booleans set to false. For Definition 7.2.8, we keep the ordering strict with regards to values. If, for two heaps h_1 and h_2 , there exists an entry $h_1(\ell, x) = ((\zeta, X), V, b)$ such that $h_2(\ell, x) = ((\zeta, X), V', b)$ with $V \subsetneq V'$, then $h_1 \not\leq h_2$.

We then give an alternative definition for Definition 7.2.4, which defines permission and heap merging behaviour:

Definition 9.1.2 (Merging permissions and heaps with write-write race conditions). We define $\oplus$ over the set $(0, 1] \times \{R, W\}$

$$(\zeta_1, X) \oplus (\zeta_2, Y) = \begin{cases} (\zeta_1 + \zeta_2, X) & \text{if } X = Y \text{ and } \zeta_1 + \zeta_2 \in (0, 1] \\ \text{undefined} & \text{otherwise} \end{cases}$$

extended for (ζ_0, R) such that $\forall_{(\zeta_1, R) \in (0, 1] \times \{R\}} (\zeta_0, R) \oplus (\zeta_1, R) = (\zeta_1, R)$ and $(\zeta_0, R) \oplus (\zeta_0, R) = (\zeta_0, R)$.

We then extend $\oplus$ to permission/value sets pairs as follows:

$$\begin{aligned}
 & ((\zeta_1, X), V_1, b_1) \oplus ((\zeta_2, Y), V_2, b_2) = \begin{cases} ((\zeta_1 \oplus \zeta_2, X), \{v\}, false) & \text{if } X = Y \\ & \wedge def((\zeta_1, X) \oplus (\zeta_2, X)) \\ & \wedge V_1 = V_2 = \{v\} \\ & \wedge b_1 = b_2 = false \\ ((\zeta_1 \oplus \zeta_2, W), V_1, true) & \text{if } X = Y = W \\ & \wedge def((\zeta_1, W) \oplus (\zeta_2, W)) \\ & \wedge b_1 = true \wedge b_2 = false \\ ((\zeta_1 \oplus \zeta_2, W), V_2, true) & \text{if } X = Y = W \\ & \wedge def((\zeta_1, W) \oplus (\zeta_2, W)) \\ & \wedge b_2 = true \wedge b_1 = false \\ ((\zeta_1 \oplus \zeta_2, W), V_1 \cup V_2, true) & \text{if } X = Y = W \\ & \wedge def((\zeta_1, W) \oplus (\zeta_2, W)) \\ & \wedge b_1 = b_2 = true \\ undefined & \text{otherwise} \end{cases}
 \end{aligned}$$

Merging heaps is defined the same as in Definition 7.2.4: We define that $h_1 \oplus h_2$ is defined iff $h_1(a) \oplus h_2(a)$ is defined for all $a \in dom(h_1) \cap dom(h_2)$. Then $h_1 \oplus h_2$ is defined with $dom(h_1 \oplus h_2) = dom(h_1) \cup dom(h_2)$ and for all $a \in dom(h_1 \oplus h_2)$:

$$(h_1 \oplus h_2)(a) = \begin{cases} h_1(a) \oplus h_2(a) & \text{if } a \in (dom(h_1) \cap dom(h_2)) \\ h_1(a) & \text{if } a \in (dom(h_1) \setminus dom(h_2)) \\ h_2(a) & \text{if } a \in (dom(h_2) \setminus dom(h_1)). \end{cases}$$

Changes to the memory assertion

We change the syntax of a memory assertion to $(E_L, x) \xrightarrow{E_Z} (\{E_1, \dots, E_n\}, E_B)$ to reflect the heap now being possibly non-deterministic and having change booleans, where $E_B \in \mathfrak{E}_B$.

We then change the memory assertion semantics to:

$$\begin{aligned}
 (\alpha, h, \zeta) \models (E_L, E_X) \xrightarrow{E_Z} (\{E_1, \dots, E_n\}, E_B) \text{ iff } \exists \ell, x, \zeta, X, v_i, b. (\\
 & res(E_L, \zeta) = \ell \wedge res(E_X, \zeta) = x \wedge res(E_Z, \zeta) = (\zeta, X) \wedge \\
 & (\zeta = \zeta_0 \Rightarrow (X = R \wedge h = h_0)) \wedge \\
 & (\zeta \neq \zeta_0 \Rightarrow (h_0 \sqsubseteq h \wedge dom(h) = \{(\ell, x)\} \cup dom(h_0))) \wedge \\
 & \forall 1 \leq i \leq n. res(E_i, \zeta) = v_i \wedge res(E_B, \zeta) = b \wedge \\
 & h(\ell, x) = ((\zeta, X), \{v_1, \dots, v_n\}, b))
 \end{aligned}$$

This allows us to assert which possible values are in a non-deterministic memory cell.

We add the shorthands to reflect the new read-write designations in permissions and the change boolean. We furthermore add a shorthand that reflects that one can split the possible values of a non-deterministic heap cell using the separator, and a shorthand which reflects that the heap is deterministic. The exact types have been omitted from the universal quantifier of the shorthand **det**, but can be derived from the semantics of memory assertions.

$$\begin{aligned}
(E_L, E_X) &\xrightarrow{E_Z} (E, E_B) &= (E_L, E_X) &\xrightarrow{E_Z} (\{E\}, E_B) \\
(E_L, E_X) &\xrightarrow{E_Z} \{E_1, \dots, E_n\} &= \exists a \in \mathfrak{A}. (E_L, E_X) &\xrightarrow{E_Z} (\{E_1, \dots, E_n\}, a) \\
(E_L, E_X) &\xrightarrow{X} (\{E_1, \dots, E_n\}, E_B) &= \exists a \in \mathfrak{A}. a = (\zeta, X) \wedge \\
&&& (E_L, E_X) \xrightarrow{a} (\{E_1, \dots, E_n\}, E_B) \\
\mathbf{det}(E_L, E_X) &&= \exists a_1, a_2 \in \mathfrak{A}. (E_L, E_X) \xrightarrow{a_1}_s a_2 \\
\mathbf{det} &&= \forall (l, a) \in \mathfrak{L} \times \mathfrak{A}. \\
&& ((l, a) \hookrightarrow - \Rightarrow \mathbf{det}(l, a))
\end{aligned}$$

The existing shorthands are straightforwardly augmented for the new syntax of memory assertions. Like in the original definition, shorthands can be freely combined.

With this, we have all changes to the syntax and semantics of AuDaLogic required. We continue with the changes to the syntax methods of AuDaLogic.

9.1.2 Changes to expression correspondence and the proof rules

In this section, we detail the changes to the expression correspondence between AuDaLa and AuDaLogic, as well as the changes to the proof rules. The changes to the proof rules include two new relations we use to define the new proof rules.

Expression correspondence

To adapt expression conversion, we need to change Definition 7.3.22 to the following definition:

Definition 9.1.3 (Adapted expression correspondence requirement). Let $E_1, \dots, E_n$ be AuDaLa expressions. We define the *expression correspondence*

requirement function for an AuDaLogic variable a_ϵ and expressions $E_1, \dots, E_n$ as

$$A(a_\epsilon, E_1, \dots, E_n) = \bigwedge_{Y \in \text{occ}(E_1, \dots, E_n)} ((\bigwedge_{x \in \text{prefixes}(Y)} (a_\epsilon, x) \xrightarrow{R} a_x) \wedge (\bigwedge_{X.x \in \text{prefixes}(Y)} (a_X, x) \xrightarrow{R} a_{X.x})).$$

This definition now specifies that if $A(E_1, \dots, E_n)$ holds, all resources required for E_1 to E_n are in the heap and *can be read from the heap*. Furthermore, this read must be deterministic, due to the notation of the memory inclusions. Though we do not change Definition 7.3.23, this change also changes the meaning of that definition, now requiring that all resources that are accessed for an expression E have read permissions.

The stability function is then almost unchanged, except that in PrS_P any occurrence of $(a_X, x) \xrightarrow{1} \text{eqi}(E', a_\epsilon)$ is replaced with $(a_X, x) \xrightarrow{W} \text{eqi}(E', a_\epsilon)$. Though a write permission less than 1 in this extension of AuDaLogic means that the struct instance(s) to which the heap applies cannot read from a memory cell, we are not prohibited from reasoning about the values of memory cells with write-only permissions.

We do not change the proofs Lemma 8.2.10 and Lemma 8.2.13 beyond updating them to work with our new permissions. It is however important to emphasize that due to the definition of the access predicate A , it does not hold if it includes a memory assertion on a cell in the heap which is non-deterministic. Therefore, Lemma 8.2.10 and Lemma 8.2.13 hold for deterministic heaps only.

New relations for the proof rules

For the proof rules, we define two binary relations between predicates, starting with the deterministic weakening relation D :

Definition 9.1.4 (Deterministic weakening relation). We define the *deterministic weakening* relation D such that $D(P, P')$ is true iff $P' \Rightarrow \mathbf{det}$ and for all (α, h, ζ) such that $(\alpha, h, \zeta) \models P$, for all h' such that $h \gg h'$, it holds that $(\alpha, h', \zeta) \models P'$.

Recall that by definition of $\gg$, there is always at least one heap h' for every heap h such that $h \gg h'$, and that for deterministic heaps h_d , $h_d \gg h_d$. It follows that for any $P \Rightarrow \mathbf{det}$, $D(P, P)$.

Generally, P can be adapted into P' using the fact that $(E_L, E_X) \mapsto \{E_1, \dots, E_n\}$ can be weakened into $\exists a \in \mathcal{A}. (E_L, E_X) \mapsto a \wedge (a = E_1 \vee \dots \vee a = E_n)$. This is not semantically equivalent in AuDaLogic, as the first requires non-determinism and the second does not, but the second does maintain all the possible values from the non-deterministic heap cell while allowing a deterministic heap to satisfy the formula.

Furthermore, we define the change reset relation C :

Definition 9.1.5 (Change reset relation). We define the *change reset* relation C such that $C(P, P')$ is true iff for all (α, h, ζ) such that $(\alpha_1, h, \zeta) \models P$, it holds that $(\alpha_1, h', \zeta) \models P'$, where h is identical to h' except that all change booleans have been set to *false* in h' .

Changes to the proof rules

The changes to most proof rules in Chapter 7 are minor, as the change to the expression correspondence requirement relation A is sufficient for them. In the rule NULL UPDATE, note that the change boolean at the end will also be *false*, as the write is ignored.

Two more major changes are to the update rule and the step call rule. We change the update rule into the following rule, which does not need full permission but only a write permission, and which requires the change boolean to be set to true at the end:

$$\begin{array}{c}
 P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
 P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta) \quad P_H \Rightarrow A(a_\epsilon, E, X) \\
 P = P_L \wedge (P_H * (a_X, x) \xrightarrow{(\zeta, W)} -) \\
 \text{(UPDATE-WW)} \frac{Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{(\zeta, W)} (eqi(a_\epsilon, E), true))}{\vdash \{P\} X.x := E\{Q\}}
 \end{array}$$

We change the step call rule into the following rule, which resets all change booleans and which makes the heap deterministic:

$$\begin{array}{c}
 S(P, P_1) \quad C(P_1, P_2) \quad D(P_2, P_3) \\
 P_3 \Rightarrow \blacksquare_1(a_\epsilon, \theta) \wedge \oslash_2 \\
 \text{(STEP CALL-WW)} \frac{StepStat_{\mathcal{P}}(F, \theta) = S \quad \vdash \{P_3\} S\{Q\}}{\vdash \{P\} F\{Q\}}
 \end{array}$$

The addition flattens the current non-deterministic heap to a deterministic one. It also limits the change booleans to step executions. This encodes that in AuDaLa, write-write race conditions are limited to their step, and that after the step all variables should be readable again (even if the exact value may be unknown). This concludes all adaptations to the definition of AuDaLogic, so we move on to the adaptations for the soundness proof of AuDaLogic.

9.1.3 Changes to the AuDaLa-AuDaLogic connection

In this section, we give the changes to the AuDaLa-AuDaLogic connection given in Section 8.1. For this, we start with the changes to state correspondence, including a new notion of a *corresponding deterministic heap* and an

additional relation to reset a heap back to deterministic. We then adapt the notion of transition safety to adapt the safety predicate for statements, after which we adapt the safety predicate for schedules.

State correspondence

The first change is in the definition of state correspondence (Definition 8.1.1). We adapt the definition to use value sets and change booleans straightforwardly, by making the following change to the requirement for memory-equivalence:

Definition 9.1.6 (State correspondence adaptation). Let $p = \langle \Delta, \sigma, s\chi \rangle$ be an AuDaLa state and let (α, h, ς) be an AuDaLogic state. We additionally define that p is (weakly) memory-equivalent to h , notation $p \rightleftharpoons h$, iff the following holds:

1. $h(\ell, x) = (\zeta, V)$ iff $\sigma(\ell) = \langle \theta, \gamma, \chi, \xi \rangle$ for some $\theta, \gamma, \chi, \xi$ such that $\xi(x) \in V$

This is extended as in Definition 8.1.1 to a weak derivation from AuDaLa to AuDaLogic states, denoted using $>$ and Ξ .

From this point on, we consider the classic struct and memory-equivalence as defined in Definition 8.1.1 to be *strict*, and change the symbols used for memory equivalence and derivation to $\rightleftharpoons_s$, $>_s$. We also use Ξ_s to denote all the states strictly derivable from a heap h .

For a deterministic heap cell in a AuDaLogic state, weak memory-equivalence and memory-equivalence are the same equivalence. For a non-deterministic heap cell in a AuDaLogic state, this relates all possible AuDaLa states implied by the non-determinism of the heap cell to the AuDaLogic state. We keep the transition and execution correspondence the same.

While we have a notion of non-determinism in AuDaLogic heaps, during executions, AuDaLa variable environments do not have non-determinism in that there is always a single value saved for every variable. We therefore define the *corresponding deterministic heap* of an AuDaLa state and a heap h .

Definition 9.1.7 (Corresponding Deterministic Heap). Let $\mathcal{X}$ be a list of statements or a step and let p be an AuDaLa state. Let h_1 be some heap such that $p > h_1$. We define the *corresponding deterministic heap* of p and h_1 be the heap h such that $p >_s h$ and $h_1 \gg h$. We also denote this heap as $det(p, h_1)$ for p and h_1 . Additionally, for every execution π starting in p , we consider h to be the corresponding deterministic heap of π and h_1 as well. We overcharge the notation to allow $det(\pi, h_1)$ as well.

We prove that the existence of a corresponding deterministic heap is guaranteed:

Lemma 9.1.8 (Existence Guaranteed). For all combinations of state p and memory heap h_1 such that $p > h_1$, there exists a corresponding deterministic heap h for p and h_1 .

Proof. By the definition of AuDaLa p will have defined values for all entries in all variable environments in p . If we combine these in a deterministic heap h_p , then $p \rightleftharpoons_s h_p$. Then as $p > h_1$, for every heap cell $(\ell, x) \in \text{dom}(h_1)$, $(\ell, x) \in \text{dom}(h_p)$ and $h_p(\ell, x) = ((\zeta, X), \{v\})$ iff $h_1(\ell, x) = ((\zeta, X), V)$ such that $v \in V$. Let h be the restriction of h_p to $\text{dom}(h_1)$. Then h is deterministic, $h_1 \gg h$, by definition of $\gg$, and $p >_s h$, by definition of h_p and $>_s$. It follows that h is a corresponding heap of p and h_1 . $\square$

We implicitly use this lemma when asserting a corresponding deterministic heap.

Lastly, to use the change booleans to reflect a change in the previous step in our heaps, we define the heap reset function r , where $r(h)$ is the heap h' such that for every $(\ell, x) \in \text{dom}(h)$ such that $h(\ell, x) = ((\zeta, X), V, b)$ for some ζ, X, V and b , $h'(\ell, x) = ((\zeta, X), V, \text{false})$. Like the m function, we use the r function to reset our state after every step.

Safety of transitions and the safety predicate for statements

We then use the deterministic corresponding heaps to define the meaning of AuDaLogic triples with write-write race conditions. This can be done for statements by changing the notion of transition safety, which amounts to changing Definition 8.1.5 to reflect our new heap:

Definition 9.1.9 (Safety of Transitions with Write-Write Race Conditions). We define transitions to be *safe* for two AuDaLogic states (α_1, h_1, ζ) and (α_2, h_2, ζ) iff one of the following holds:

1. The transition is not a **ComRd**, **ComWr** or **ComCons** transition.
2. The transition is a **ComRd** transition annotated with $(\ell, \ell_2, x, v, \theta')$ and $h_1(\ell_2, x) = ((\zeta, R), \{v\}, b)$ for some $\zeta \in (0, 1] \cup \{\zeta_0\}$ and some $b \in \mathbb{B}$.
3. The transition is a **ComWr** transition annotated with $(\ell, \ell_2, x, v, \theta')$ and $h_2(\ell_2, x) = ((\zeta, W), \{v\}, \text{true})$ for some $\zeta \in (0, 1]$, and if $x \in \text{ParsP}(\theta)$, $h_1(\ell_2, x) = ((\zeta, W), V, b)$ for some $V \subset \mathcal{V}$ and $b \in \mathbb{B}$.
4. The transition is a **ComCons** transition annotated with $(\ell, \ell', \theta', v_1, \dots, v_e)$ and $\ell' \in \alpha_1(3)$, $\ell' \in \alpha_2(2)$ and for all parameters $x_i \in \text{ParsP}(\theta')$, $h_2(\ell', x_i) = (1, \{v_i\}, \text{true})$.

This definition of transition safety does not allow non-deterministic reading from a variable. This means that to prove statement triples, one has to

take care to make sure the memory heap that satisfies the precondition is deterministic in the values that need to be read.

We do not need to change the safety predicate for statements as given in Definition 8.1.6. Due to the requirements on permissions in Definition 9.1.9, the safety predicate still does not allow read-write race conditions. Note that the safety predicate requires reads to be deterministic, so the precondition needs to reflect that to be able to prove a triple for a list of statements. We also do not need to change Lemma 8.1.7.

Safety predicate for schedules

We change the safety predicate for schedules minimally to make the heap deterministic between each step and to reset the change booleans, as given below. Changes are highlighted:

Definition 9.1.10 (Safety Predicate for Schedules). Let Δ be a schedule, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state and let π be an execution of Δ from some AuDaLa state $p \in \Xi(\alpha_1, h_1, \varsigma)$. Let $\mathcal{F}$ be the step sequence of π . Then $\text{safeScP}(\pi, \Delta, (\alpha_1, h_1, \varsigma), Q)$ holds iff

$\text{safeScP}(\pi, \mathcal{F}, (\alpha_1, h_1, \varsigma), Q)$ holds, and $\text{safeScP}(\pi, \mathcal{F}, (\alpha_1, h_1, \varsigma), Q)$ holds iff all of the following hold:

1. π is finite.
2. If $\mathcal{F} = \varepsilon$, $(\alpha_1, h_1, \varsigma) \models Q$.
3. If $\mathcal{F} = (F, q); \mathcal{F}'$, let $\pi = \pi_\Delta; t; \pi_F; \pi'$, where π_Δ consists of all schedule transitions until the execution of F in π , $t; \pi_F$ is the execution of F in π , with t the **Init**-transition of F and π_F containing all the command transitions in the execution of F and π' as the rest of the execution of π . Let p_1 be the AuDaLa state after the transitions $\pi_\Delta; t$. Let $\alpha_u = m(\alpha_1)$ and let $h_d = r(\text{det}(\pi_F, h_1))$.

Then if $\pi_F \rightsquigarrow (\alpha_u, h_d, \varsigma)$, there exist a state $(\alpha_t, h_t, \varsigma)$ and there exists the splits $(\alpha_u, h_d, h_1) = (\alpha'_u, h'_d, \varsigma) \star (\alpha_u^1, h_d^1, \varsigma) \star \cdots \star (\alpha_u^n, h_d^n, \varsigma)$ and $(\alpha_t, h_t, \varsigma) = (\alpha'_t, h'_t, \varsigma) \star (\alpha_t^1, h_t^1, \varsigma) \star \cdots \star (\alpha_t^n, h_t^n, \varsigma)$ and $\pi_F = \tau^1 \parallel \cdots \parallel \tau^n$ such that all of the following hold:

- (a) $\alpha'_u = \alpha'_t$ and $h'_d = h'_t$.
- (b) For every $(\ell, \theta) \in \alpha_u(1)$, if $q = \theta \vee q = \top$, then for some $1 \leq i \leq n$, $\alpha_u^i(1) = \{(\ell, \theta)\}$.
- (c) For every transition in π_F annotated with an execution label ℓ , there exists a $1 \leq i \leq n$ such that $\alpha_u^i(1) = \{(\ell, \theta)\}$ for some struct type θ .

- (d) For all $1 \leq i \leq n$, with π^i be the execution from p_1 consisting of the transitions τ^i . Then $\pi^i \rightsquigarrow (\alpha_u^i, h_d^i, \zeta)$ and $\text{safeSt}_{\mathcal{P}}(\pi^i, \text{StepStat}_{\mathcal{P}}(F, \theta), (\alpha_u^i, h_d^i, \zeta), (\alpha_t^i, h_t^i, \zeta))$.
- (e) $\text{safeSc}_{\mathcal{P}}(\pi', \mathcal{F}', (\alpha_t, h_t, \zeta), Q)$ holds.

This concludes the changes to the connection between AuDaLa and AuDaLogic. Both safety predicates still do not admit read-write race conditions, due to the semantics of heap splits and permissions.

In the next section, we give some soundness proofs as a demonstration that the proofs are not impacted a lot.

9.1.4 Demonstration of soundness proofs for AuDaLogic adapted for write-write race conditions

In this section, we give the soundness proofs for some AuDaLogic rules adapted for write-write races. Though we do not give the proofs for all AuDaLogic rules, we estimate that the proofs given will sufficiently demonstrate the changes to proving the soundness of AuDaLogic proof rules. We shall give the proofs for UPDATE-WW and STEP CALL-WW, as given in Section 9.1.2 and the rules SPLIT PARALLEL and BARRIER, as given in Section 7.4.2 and Section 7.4.3 respectively. We keep Lemma 8.2.11 the same.

Soundness of UPDATE-WW

This proof is almost identical to the proof for the soundness of the proof rule UPDATE, as given in Section 8.3.1. The only change is that any occurrence of $(a_X, x) \xrightarrow{1} E$ is changed into $(a_X, x) \xrightarrow{W} E$, and that the definition of h_2 reflects our new permissions and the change booleans. We give the proof to demonstrate this. The heap h_1 is allowed to be non-deterministic for the application of this rule, but not in the variables read to execute ' $X.x := E$ ', which is enforced by $A(a_e, E, X)$.

Let $\mathcal{X}$ be the syntactical statement ' $X.x := E$ ', with a potentially empty variable expression X and a simple expression E . Let the precondition be $P = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} -)$, with P_L a heap-free predicate and P_H a triple-free predicate such that $P_L \Rightarrow \blacksquare_1(a_e, \theta)$ and $P_H \Rightarrow A(a_e, E, X)$. Let the postcondition be $Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{W} (eqi(a_e, E), true))$.

To prove that $\models \{P\} X.x := E \{Q\}$, we need to prove that for all AuDaLogic states (α_1, h_1, ζ) such that $(\alpha_1, h_1, \zeta) \models P$, for all executions π of $X.x := E$ such that $\pi \rightsquigarrow (\alpha_1, h_1, \zeta)$, there exists a state (α_2, h_2, ζ) such that $\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \zeta), (\alpha_2, h_2, \zeta))$ and $(\alpha_2, h_2, \zeta) \models Q$.

Then let (α_1, h_1, ζ) be an arbitrary AuDaLogic state and let π be an arbitrary execution satisfying the requirements of the last paragraph. Let π start in AuDaLa state p .

To define (α_2, h_2, ζ) , we define v to be the result value of E executed from p . As $\pi \sim (\alpha_1, h_1, \zeta)$ and $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$ (as $(\alpha_1, h_1, \zeta) \models P$), all transitions in π can only have ℓ as their executing label, so π does not have race conditions. It follows by Corollary 4.3.23 that the result value of E during π must also be v .

As $(\alpha_1, h_1, \zeta) \models P * (a, x) \xrightarrow{W} -$, we know that $h_1(\zeta(a), x) = ((\zeta, W), v_o, b)$ for some $v_o \in \mathcal{V}$ and some $b \in \mathbb{B}$. We use this to define the AuDaLogic state (α_2, h_2, ζ) as $(\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, \text{true})], \zeta)$, with $h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, \text{true})]$ a function update of h_1 .

To prove $\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \zeta), (\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, \text{true})], \zeta))$ to be true, we prove the requirements of $\text{safeSt}_{\mathcal{P}}$. The first requirement of $\text{safeSt}_{\mathcal{P}}$ is satisfied because $\mathcal{X} \neq \varepsilon$. For the second requirement, as $\mathcal{X}$ consists only of $X.x := E$, we can satisfy 2c and the recursive element by taking $(\alpha', h, \zeta) = (\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, \text{true})], \zeta)$. We then need to prove that requirements 2a and 2b hold for $(\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, \text{true})], \zeta)$:

- 2a As $\pi \sim (\alpha_1, h_1, \zeta)$ and $(\alpha_1, h_1, \zeta) \models \blacksquare_1(a_\varepsilon, \theta)$, we know that $\alpha_1(1) = \{(\ell, \theta)\}$. Additionally, there is only a single struct instance that can execute transitions in π , so we know that all transitions have execution label ℓ .
- 2b To prove that requirement 2b holds, we need to prove all **ComCons**, **ComRd** and **ComWr** transitions in π are safe:

- As there are only transitions executed by the struct instance of ℓ , there are only transitions stemming from $\llbracket \mathcal{X} \rrbracket$ in π . Therefore, there are no **ComCons** transitions in π .
- As $P \Rightarrow A(a_\varepsilon, E, X)$ (as $P_H \Rightarrow A(a_\varepsilon, E, X)$), E and X are the expressions of $\mathcal{X}$ and $\pi \sim (\alpha_1, h_1, \zeta)$, the fact that the **ComRd** rules are safe in π follows from Lemma 8.2.11.
- As there are only transitions executed by the struct instance of ℓ , there are only transitions stemming from $\llbracket \mathcal{X} \rrbracket$ in π . Therefore, there is only at most a single **ComWr** transition in π . By the fact that $(\alpha_1, h_1, \zeta) \models P_H * (a_X, x) \xrightarrow{W} -$, we know that the variable written to is not a parameter of a null-instance: if it was, it would be in h_0 , and then by definition of memory assertions, $(a, x) \xrightarrow{W} -$ would not be satisfiable. We then know there is exactly one **ComWr**-transition.

Let the **ComWr**-transition of π be annotated with $(\ell, \ell_2, x, v', \theta')$ for some $\ell_2 \in \mathcal{L}$, $v' \in \mathcal{V}$ and $\theta' \in \Theta_{\mathcal{P}}$. By the fact that the transition is annotated with $(\ell, \ell_2, x, v', \theta')$ and the fact that $\llbracket \mathcal{X} \rrbracket = \llbracket E \rrbracket; \llbracket X \rrbracket; \mathbf{wr}(x)$, it holds that $v' = v$ and that ℓ_2 is the return value of X in $\mathcal{X}$. By applying Lemma 8.2.10 on $X.x$, we can conclude that $\zeta(a) = \ell_2$. It

follows that $h_2(\ell_2, x) = ((\zeta, W), v', true)$, from the definition of h_2 .

Additionally, as $(\alpha_1, h_1, \zeta) \models P_H * (a_X, x) \xrightarrow{W} -$, we know that $h_1(\ell_2, x) = (1, \nu_o, b)$ for some $\nu_o \in \mathcal{V}$.

We conclude that

$\text{safeSt}_{\mathcal{P}}(\pi, X.x := E, (\alpha_1, h_1, \zeta), (\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, true)], \zeta))$ holds. We then still need to prove that $(\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, true)], \zeta) \models P_L \wedge (P_H * (a_X, x) \xrightarrow{W} (eqi(a_\varepsilon, E), true))$. As P_L is a heap-free predicate and $(\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta)$ only differs from (α_1, h_1, ζ) in the memory heap, we know that $(\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta) \models P_L$.

We prove that additionally, $(\alpha_1, h_1[(\zeta(a_X), x) \mapsto (1, \nu)], \zeta) \models (P_H * (a_X, x) \xrightarrow{W} (eqi(a_\varepsilon, E), true))$. As $h_1[(\zeta(a_X), x) \mapsto (1, \nu)]$ is only changed in the entry for $(\zeta(a_X), x)$, this is proven by proving that $\text{res}(eqi(a_\varepsilon, E)) = v$. This holds due to Lemma 8.2.10, which is applicable in this situation. We conclude that $(\alpha_1, h_1[(\zeta(a), x) \mapsto ((\zeta, W), v, true)], \zeta) \models \blacksquare_1(a_\varepsilon, \theta) \wedge (P'' * (a_X, x) \xrightarrow{W} (eqi(E, a_\varepsilon), true))$.

It follows that the proof rule UPDATE is sound. $\square$

Soundness of STEP CALL-WW

This proof is almost identical to the proof found in Section 8.3.3. The main change is that we now use the corresponding deterministic heap with the change booleans reset, which needs to be reflected in the proof when we use the statements triple in the premises.

Let $\mathcal{X}$ be a schedule of the form F . Let the precondition be P such that there exist predicates P_1, P_2 and P_3 such that $S(P, P_1)$, $C(P_1, P_2)$ and $D(P_2, P_3)$. Let θ be a struct type and $a_\varepsilon \in \mathcal{A}$ be a AuDaLogic variable such that $P_3 \Rightarrow \blacksquare_1(a_\varepsilon, \theta)$. Let $\text{StepStat}_{\mathcal{P}}(F, \theta) = S$ and let $\models \{P\}S\{Q\}$ hold.

to prove that $\models \{P\}F\{Q\}$, let (α_1, h_1, ζ) be an AuDaLogic state such that $(\alpha_1, h_1, \zeta) \models P$ and let π be an execution of F from an AuDaLa state $p > (\alpha_1, h_1, \zeta)$. As the schedule is F , we know that the step sequence of F is $(F, \top)$ (as F is a global step call). We then need to prove that $\text{safeSc}_{\mathcal{P}}(\pi, F, (\alpha_1, h_1, \zeta), Q)$ holds.

As the schedule is F , we know that $\pi = t; \pi_F$, so requirement 1 of $\text{safeSc}_{\mathcal{P}}(\pi, F, (\alpha_1, h_1, \zeta), Q)$ holds. Requirement 2 holds because the step sequence of π is $(F, \top)$. As in the safety predicate for schedules, let $\alpha_u = m(\alpha_1)$ and let $h_d = r(\text{det}(\pi_F, h_1))$. As $S(P, P_1)$, $C(P_1, P_2)$ and $D(P_2, P_3)$, it follows that $(\alpha_u, h_d, \zeta) \models P_3 \wedge \mathcal{O}_2$.

To prove requirement 3, assume that $\pi_F \rightsquigarrow (\alpha_u, h_d, \zeta)$ (as otherwise requirement 3 of $\text{safeSc}_{\mathcal{P}}(\pi, F, (\alpha_1, h_1, \zeta), Q)$ is vacuously true). We need to prove that there exists a state (α_2, h_2, ζ) such that there exist the splits outlined in $\text{safeSc}_{\mathcal{P}}$ for which 3a to 3e hold. As there is only one step in the step sequence, proving 3e is done by establishing that $(\alpha_2, h_2, \zeta) \models Q$, which satisfies the recursive

element.

We make the split $(\alpha_u, h_d, \varsigma) \star (\alpha_0, h_0, \varsigma)$ for requirement 3, where α_0 is the empty label triple. Intuitively, this can be considered as not splitting $(\alpha_u, h_d, \varsigma)$ and just adding on an empty state. To satisfy requirement 3a, we will make the same trivial split $(\alpha_2, h_2, \varsigma) \star (\alpha_0, h_0, \varsigma)$ for our currently abstract state $(\alpha_2, h_2, \varsigma)$. Requirement 3b holds by definition of $(\alpha_u, h_d, \varsigma)$. Requirement 3c holds because $\pi \rightsquigarrow (\alpha_u, h_d, \varsigma)$ and $(\alpha_u, h_d, \varsigma) \models \blacksquare_1(l, \theta) \wedge \emptyset_2$, which means that any command transition in π must be executed by $\varsigma(a_\epsilon)$.

From the fact that $\pi \rightsquigarrow (\alpha_u, h_d, \varsigma)$, the fact that $\alpha_u(1) = \{\varsigma(a_\epsilon), \theta\}$ and the fact that $\alpha_u(2) = \emptyset$, it holds that π_F is exactly an execution of $\text{StepStat}_{\mathcal{P}}(F, \theta)$. Additionally, due to the definition of AuDaLogic state correspondence, the fact that t is an **Init**-transition (which loads commands into struct instances), we know that $\pi_F \rightsquigarrow (\alpha_u, h_d, \varsigma)$. Then by the premise $\models \{P\}S\{Q\}$, we know that there exists a state (α', h', ς) such that $\text{safeSt}_{\mathcal{P}}(\pi_F, S, (\alpha_u, h_d, \varsigma), (\alpha', h', \varsigma))$ and $(\alpha', h', \varsigma) \models Q$. This satisfies requirement 3d.

We fix $(\alpha_2, h_2, \varsigma) = (\alpha', h', \varsigma)$. This satisfies requirement 3e and the recursive element, as this means that $(\alpha_2, h_2, \varsigma) \models Q$. Therefore, the proof rule **STEP CALL** is sound. $\square$

Soundness of SPLIT-PARALLEL

This proof is adapted from the soundness proof in Section 8.3.2. The changes are mostly to facilitate our use of the corresponding deterministic heap, mostly in the splits.

Let $\mathcal{X}$ be a schedule consisting of the step F . Let $P_1 * P_2$ be the precondition and let $Q_1 * Q_2$ be the postcondition. Additionally, let $\models \{P_1\}F\{Q_1\}$ and $\models \{P_2\}F\{Q_2\}$ hold. Then to prove that $\models \{P_1 * P_2\}F\{Q_1 * Q_2\}$, let $(\alpha_1, h_1, \varsigma)$ be an AuDaLogic state such that $(\alpha_1, h_1, \varsigma) \models P_1 * P_2$ and let π be an execution of F from p such that $p > (\alpha_1, h_1, \varsigma)$. As F is a single step, we know that the step sequence of π is $(F, \top)$. We then need to prove that $\text{safeSc}_{\mathcal{P}}(\pi, F, (\alpha_1, h_1, \varsigma), Q)$.

As the step sequence of π is $(F, \top)$, it holds that π is finite, which satisfies the first requirement of $\text{safeSc}_{\mathcal{P}}$. As the step sequence is not empty, the second requirement of $\text{safeSc}_{\mathcal{P}}$ is also satisfied.

Then, for requirement 3, from the fact that the schedule is F , it follows that $\pi = t; \pi_F$, with t the **Init**-transition of F . Let p_1 be the state before executing π_F . Due to the definition of AuDaLogic state correspondence, the fact that $p \in \Xi(\alpha_1, h_1, \varsigma)$ and the fact that t is an **Init**-transition (which loads commands into struct instances), we know that $p_1 \in \Xi(\alpha_u, h_1, \varsigma)$, with $\alpha_u = m(\alpha_1)$. (Recall that this means that $\alpha_u(1) = \alpha_1(1) \cup \alpha_1(2)$ and $\alpha_u(2) = \emptyset$.) Let h_d be $r(\text{det}(\pi_F, h_1))$.

If $\pi_F \not\rightsquigarrow (\alpha_u, h_d, \varsigma)$, the safety predicate is satisfied, and the rule is sound. Therefore, assume that $\pi_F \rightsquigarrow (\alpha_u, h_d, \varsigma)$.

To prove requirement 3, we need to prove that there exists a state $(\alpha_2, h_2, \varsigma)$ such that there exist the splits outlined in $\text{safeSc}_{\mathcal{P}}$ for which 3a to 3e hold. As

there is only one step in the step sequence, proving 3e is done by establishing that $(\alpha_2, h_2, \varsigma) \models Q$, which satisfies the recursive element.

To establish the state $(\alpha_2, h_2, \varsigma)$ and the split for which 3a-3e hold, we use the premises, as they imply the existence of such a split. In the rest of this proof, we first define the state $(\alpha_2, h_2, \varsigma)$ and the splits as implied by the premises, which we then combine to get a *combined split* which satisfies requirement 3 for the whole.

As the precondition is $P_1 * P_2$, we know there exists a split $(\alpha_1, h_1, \varsigma) = (\alpha_1^1, h_1^1, \varsigma) \star (\alpha_1^2, h_1^2, \varsigma)$ such that $(\alpha_1^1, h_1^1, \varsigma) \models P_1$ and $(\alpha_1^2, h_1^2, \varsigma) \models P_2$. It follows by definition of α_u and h_d that there also exists a split $(\alpha_u, h_d, \varsigma) = (\alpha_u^1, h_d^1, \varsigma) \star (\alpha_u^2, h_d^2, \varsigma)$ such that $m(\alpha_1^1) = \alpha_u^1$, $m(\alpha_1^2) = \alpha_u^2$, $r(\det(\pi_F, h_1^1)) = h_d^1$ and $r(\det(\pi_F, h_1^2)) = h_d^2$.

As $\pi_F \rightsquigarrow (\alpha_u, h_d, \varsigma)$, we also have the split $\pi_F = \pi_F^1 \parallel \pi_F^2$ such that $\pi_F^i \rightsquigarrow (\alpha_u^i, h_d^i, \varsigma)$ for $i \in \{1, 2\}$. By the definition of π_F^1 and π_F^2 , both start in AuDaLa state p_1 . From the proof rule premises $\models \{P_i\}F\{Q_i\}$, with $i \in \{1, 2\}$, and as $m(\alpha_1^i) = \alpha_u^i$, we know that $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds. Furthermore, as $\pi_F^i \rightsquigarrow (\alpha_u^i, h_d^i, \varsigma)$, we know that $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds because there exists a state $(\alpha_2^i, h_2^i, \varsigma)$ that satisfies requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ and for which $(\alpha_2^i, h_2^i, \varsigma) \models Q_i$ holds.

Then as $(\alpha_u, h_1, \varsigma) = (\alpha_u^1, h_1^1, \varsigma) \star (\alpha_u^2, h_1^2, \varsigma)$, as $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds and $\pi_F^i \rightsquigarrow (\alpha_u^i, h_d^i, \varsigma)$ for $i \in \{1, 2\}$, we can conclude that $\text{def}((\alpha_2^1, h_2^1, \varsigma) \star (\alpha_2^2, h_2^2, \varsigma))$, as the executions π_F^1 and π_F^2 will not go outside their own heap. We fix $(\alpha_2, h_2, \varsigma) = (\alpha_2^1, h_2^1, \varsigma) \star (\alpha_2^2, h_2^2, \varsigma)$.

With $(\alpha_2, h_2, \varsigma)$ established, we define the *individual splits* for both separate states. As $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds for all $i \in \{1, 2\}$, for $(\alpha_u^i, h_d^i, \varsigma)$, $(\alpha_2^i, h_2^i, \varsigma)$ and π_F^i there exist the individual splits

$$(\alpha_u^i, h_d^i, \varsigma) = ((\alpha_u^i)_r, (h_d^i)_r, \varsigma) \star ((\alpha_u^i)_1, (h_d^i)_1, \varsigma) \star \dots \star ((\alpha_u^i)_m, (h_d^i)_m, \varsigma),$$

$$(\alpha_2^i, h_2^i, \varsigma) = ((\alpha_2^i)_r, (h_2^i)_r, \varsigma) \star ((\alpha_2^i)_1, (h_2^i)_1, \varsigma) \star \dots \star ((\alpha_2^i)_m, (h_2^i)_m, \varsigma)$$

and $\pi_F^i = \tau_1^i \parallel \dots \parallel \tau_m^i$, such that requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ holds.

These splits split the execution π_F^i in m individual executions, and splits the heaps $(\alpha_u^i, h_d^i, \varsigma)$ and $(\alpha_2^i, h_2^i, \varsigma)$ in $m+1$ parts, where the first part $((\alpha_u^i)_r, (h_d^i)_r, \varsigma)$ does not execute anything and the other m parts all have an individual execution. As requirement 3 of $\text{safeScP}(t; \pi_F^i, F, (\alpha_1^i, h_1^i, \varsigma), Q_i)$ (for all $i \in \{1, 2\}$) holds for their respective splits, more specifically, we know that the following properties hold for these splits:

1. For every $(\ell, \theta) \in \alpha_u^i(1)$, there exists a $1 \leq j \leq m$, $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$.

2. For every transition π_F^i annotated by an execution label ℓ , there exists a $1 \leq j \leq m$ such that $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$ for some struct type θ .
3. For all $1 \leq j \leq m$, with π_j^i the execution of τ_j^i from p_1 , $\pi_j^i \prec ((hl_u^i)_j, (h_d^i)_j, \varsigma)$ and $\text{safeSt}_{\mathcal{P}}(\pi_j^i, \text{StepStat}_{\mathcal{P}}, ((\alpha_u^i)_j, (h_d^i)_j, \varsigma), ((\alpha_2^i)_j, (h_2^i)_j, \varsigma))$ holds.

Due to the existence of the above splits for $(\alpha_u^1, h_d^1, \varsigma)$, $(\alpha_2^1, h_2^1, \varsigma)$ and π_F^1 and for $(\alpha_u^2, h_d^2, \varsigma)$, $(\alpha_2^2, h_2^2, \varsigma)$ and π_F^2 , there exists the *combined splits* of $(\alpha_u, h_d, \varsigma)$, $(\alpha_2, h_2, \varsigma)$ and π_F defined as follows:

$$(\alpha_u, h_d, \varsigma) = (\alpha_u', h_d', \varsigma) \star ((\alpha_u^1)_1, (h_d^1)_1, \varsigma) \star \dots \star ((\alpha_u^1)_m, (h_d^1)_m, \varsigma) \star ((\alpha_u^2)_1, (h_d^2)_1, \varsigma) \star \dots \star ((\alpha_u^2)_m, (h_d^2)_m, \varsigma),$$

with $\alpha_u' = (\alpha_u^1)_r \diamond (\alpha_u^2)_r$ and $h_d' = (h_d^1)_r \oplus (h_d^2)_r$,

$$(\alpha_2, h_2, \varsigma) = (\alpha_u', h_d', \varsigma) \star ((\alpha_2^1)_1, (h_2^1)_1, \varsigma) \star \dots \star ((\alpha_2^1)_m, (h_2^1)_m, \varsigma) \star ((\alpha_2^2)_1, (h_2^2)_1, \varsigma) \star \dots \star ((\alpha_2^2)_m, (h_2^2)_m, \varsigma),$$

and $\pi_F = \tau_1^1 \parallel \dots \parallel \tau_m^1 \parallel \tau_1^2 \parallel \dots \parallel \tau_m^2$.

For these splits, the requirements 3a-3e hold for $\text{safeSc}_{\mathcal{P}}(\pi, F, (\alpha_1, h_d, \varsigma), Q)$:

- 3a This requirement holds by the definition of the combined splits.
- 3b For every pair $(\ell, \theta) \in \alpha_u(1)$, we know that $(\ell, \theta) \in \alpha_u^1(1)$ or $(\ell, \theta) \in \alpha_u^2(1)$. It follows that for some $1 \leq j \leq m$, $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$. As $(\alpha_u^i)_j$ is also formed in the combined split, it follows that 3b holds.
- 3c From property 2 for the individual splits we know that for every command transition contained in π_F^i annotated with an execution label ℓ , there exists a $1 \leq j \leq m$ such that $(\alpha_u^i)_j(1) = \{(\ell, \theta)\}$ for some struct type θ . As $\pi_F = (\pi_F^1)^1 \parallel (\pi_F^2)^2$ and all elements $\alpha_u^i_j$ are also in the combined split of $(\alpha_u, h_d, \varsigma)$, this property still holds for π_F and the combined splits of $(\alpha_u, h_d, \varsigma)$.
- 3d As all τ_j^i from p_1 are included in the big split, as well as all $(hl_u^i)_j$ and $(h_d^i)_j$, this property holds directly due to property 3 for the individual splits.
- 3e As $(\alpha_2^1, h_2^1, \varsigma) \models Q_1$ and $(\alpha_2^2, h_2^2, \varsigma) \models Q_2$ and by the definition of the separator, we know that $(\alpha_2, h_2, \varsigma) \models Q_1 * Q_2$. This satisfies 3e and the recursive element, as the schedule consists of only F .

It follows that $\text{safeSc}_{\mathcal{P}}(t; \pi_F, F, (\alpha_1, h_1, \varsigma), Q)$, so the rule SPLIT PARALLEL is sound. $\square$

```

1 struct Elem(decider: Elem, winner: Elem){
2   nominate{
3     decider.winner := this;
4   }
5   update{
6     if decider = this then {
7       decider := winner;
8     }
9   }
10 }
11
12 nominate < update

```

Listing 9.1: AuDaLa code for nomination

9.2 Proof Tree Example for Adapted AuDaLogic

In this section, we will first give an example partial program for AuDaLa which contains race conditions, after which we prove the postcondition of the program using AuDaLogic as adapted above. For proving the program, we use the same style as in Section 7.5. Let any occurrence of ∇ be a reference to another part of a proof tree. Let proof trees labeled with **Finished:** be finished proof trees. In partial proof trees, let question marks denote unknown predicates, and let ... denote that we will fill that part of the proof tree in later.

9.2.1 The example program

As an example for proving a program with write-write race conditions, we create a nomination program. For this program, we assume that we initialize a set of elements, amongst which one *decider* element. We assume that every element can access the decider element. During the execution of the program, a nomination will take place via a write-write race condition on a reference to an element, which one of the elements in the set will win. The decider element then updates then makes that element the next decider. The Listing is shown in Listing 9.1.

For ease of use, we define the following statements.

$$S_1 = \text{"winner := this;"}$$

$$S_2 = \text{"if(decider = this) then \{S_3\}"}$$

$$S_3 = \text{"decider := winner;"}$$

$$\begin{array}{c}
\frac{\frac{\nabla_1}{\vdash \{P_2\} \textit{nominate}\{?\}} \quad \frac{\dots}{\vdash \{?\} \textit{update}\{?\}}}{\vdash \{P_2\} \textit{nominate} < \textit{update}\{?\}} \text{BARRIER} \\
\frac{\vdash \{P_2\} \textit{nominate} < \textit{update}\{?\}}{\vdash \{P_1\} \textit{nominate} < \textit{update}\{?\}} \text{WEAKENING} \\
\frac{\vdash \{P_1\} \textit{nominate} < \textit{update}\{?\}}{\vdash \{P_0\} \textit{nominate} < \textit{update}\{?\}} \text{EXAMPLE}
\end{array}$$

Figure 9.1: Proof Tree ∇_0 , first unfinished version

We formalize the initialization requirements in the predicate P_0 :

$$\begin{aligned}
P_0 := & \exists \textit{Set} \subset \mathcal{L}.\textit{Set} = \{l_1, \dots, l_n\} \wedge \\
& \bigotimes_{1 \leq i \leq n} ((\blacksquare_1(l_i, \textit{Elem} \wedge \emptyset_2) \wedge \emptyset_2) \wedge (l_i, \textit{decider}) \xrightarrow{1} l_1 * (l_i, \textit{winner}) \xrightarrow{1} \ell_{\textit{Elem}}^0) \\
& * (\blacksquare_1(\ell_{\textit{Elem}}^0, \textit{Elem}) \wedge \emptyset_2 \wedge \textbf{emp})
\end{aligned}$$

This predicate also encodes the existence of the null-instance.

We then create a proof tree to determine the postcondition of executing Listing 9.1.

9.2.2 Initial schedule decomposition

We start the proof tree by removing the existential quantifier for the set $\textit{Set}$, resulting in P_1 , and then distributing the set over the big separator, using the rule $P_A \wedge (P_B * P_C) \Leftrightarrow (P_A \wedge P_B) * P_C$, as given by Reynolds [159], as the first line of P_0 (and P_1) is pure. This results in P_2 :

$$\begin{aligned}
P_2 := & \\
& \bigotimes_{1 \leq i \leq n} ((\blacksquare_1(l_i, \textit{Elem} \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_i, \textit{decider}) \xrightarrow{1} (l_1, \textit{false}) * (l_i, \textit{winner}) \xrightarrow{1} (\ell_{\textit{Elem}}^0, \textit{false})) \\
& \wedge \textit{Set} = \{l_1, \dots, l_n\}) * (\blacksquare_1(\ell_{\textit{Elem}}^0, \textit{Elem}) \wedge \emptyset_2 \wedge \textbf{emp})
\end{aligned}$$

We then split the schedule using the BARRIER rule. This results in the proof tree ∇_0 as given in Figure 9.1.

9.2.3 Resolving the *nominate* step

We then need to split all active elements to be able to call the step *nominate*. For this, we first redistribute the permissions in the big separator to facilitate

$$\begin{array}{c}
\frac{\dots}{\vdash \{P_5\} S_1 \{?\}} \text{ STEP CALL-WW} \\
\frac{\vdash \{P_5\} \text{nominate}\{?\}}{\vdash \{P_4\} \text{nominate}\{?\}} \text{ GEN PARALLEL} \quad \frac{\dots}{\vdash \{P_N\} \text{nominate}\{?\}} \text{ SPLIT PARALLEL} \\
\frac{\vdash \{P_3\} \text{nominate}\{?\}}{\vdash \{P_2\} \text{nominate}\{?\}} \text{ WEAKENING}
\end{array}$$

Figure 9.2: Proof Tree ∇_1 , first unfinished version

the write that happens in *nominate*. This results in the predicate P_3 :

$$\begin{aligned}
P_3 := & \\
& \circledast_{1 \leq i \leq n} (\blacksquare_1(l_i, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_i, \text{decider}) \xrightarrow{1} (l_1, \text{false}) * \circledast_{1 \leq j \leq n} ((l_j, \text{winner}) \xrightarrow{(1/n, W)} (\ell_{Elem}^0, \text{false}))) \\
& \wedge Set = \{l_1, \dots, l_n\} * (\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp})
\end{aligned}$$

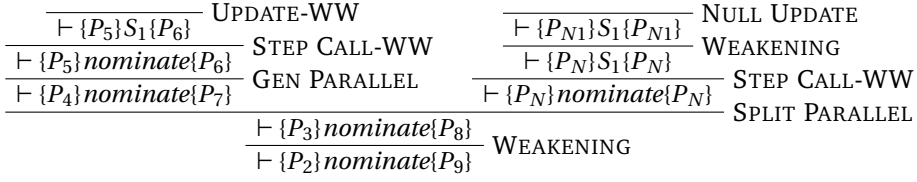
We then split off the null-instance with SPLIT PARALLEL, which leads to P_4 and P_N , with P_N defined as $(\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp})$. This is followed by an application of GEN PARALLEL to split the rest of the struct instances, which leads to P_5 :

$$\begin{aligned}
P_5 := & \\
& (\blacksquare_1(l_i, Elem \wedge \emptyset_2) \wedge \\
& ((l_i, \text{decider}) \xrightarrow{1} (l_1, \text{false}) * \circledast_{1 \leq j \leq n} ((l_j, \text{winner}) \xrightarrow{(1/n, W)} (\ell_{Elem}^0, \text{false}))) \\
& \wedge Set = \{l_1, \dots, l_n\})
\end{aligned}$$

Then, we use STEP CALL-WW, using the fact that $S(P_5, P_5)$, $C(P_5, P_5)$ and $D(P_5, P_5)$, by definition of P_5 : there are no newly created struct instances, all the change booleans are set to false and there are no non-deterministic heaps in P_5 . This results in the proof tree ∇_1 , as given in Figure 9.2

With our active elements split up, we locally prove the result of S_1 . For this, we use the UPDATE-WW rule, as the clause $(l_1, \text{winner}) \xrightarrow{(1/n, W)} (\ell_{Elem}^0, \text{false})$ is contained in the big separator of the *winner*-parameters in P_5 . This results in resulting in P_6 .

$$\begin{aligned}
P_6 := & \\
& (\blacksquare_1(l_i, Elem \wedge \emptyset_2) \wedge \\
& ((l_i, \text{decider}) \xrightarrow{1} (l_1, \text{false}) * (l_1, \text{winner}) \xrightarrow{(1/n, W)} (l_i, \text{true})) * \\
& \circledast_{2 \leq j \leq n} ((l_j, \text{winner}) \xrightarrow{(1/n, W)} (\ell_{Elem}^0, \text{false})) \\
& \wedge Set = \{l_1, \dots, l_n\})
\end{aligned}$$

Figure 9.3: **Finished:** Proof Tree ∇_1

We then return the other struct instances according to GEN PARALLEL, resulting in P_7 . For P_N , we use STEP CALL-WW, and then reform P_N into $P_{N1} = (\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp} * (\ell_{Elem}^0, winner) \xrightarrow{(\zeta_0, R)} (\ell_{Elem}^0, false))$. We then use the rule NULL UPDATE, which results in P_{N1} , which we reform into P_N and process down.

Readding P_N to P_7 results in P_8 . Through weakening, we then redistribute the permissions of contained in the big separator dividing the struct instances back to our earlier form, where every struct instance has full permissions for their own parameters. To do this, we merge the results of the race condition, which results into a non-deterministic heap predicate P_9 :

$$\begin{aligned}
P_9 := & ((\blacksquare_1(l_1, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_1, decider) \xrightarrow{1} (l_1, false) * (l_1, winner) \xrightarrow{1} (\{l_1, \dots, l_n\}, true)) \\
& \wedge Set = \{l_1, \dots, l_n\}) * \\
& \otimes_{2 \leq i \leq n} ((\blacksquare_1(l_i, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_i, decider) \xrightarrow{1} (l_i, false) * (l_i, winner) \xrightarrow{1} (\ell_{Elem}^0, false)) \\
& \wedge Set = \{l_1, \dots, l_n\}) * (\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp})
\end{aligned}$$

Note that here, due to the change booleans, ℓ_{Elem}^0 could never be included in the set of values for $(l_1, winner)$.

We can then finish proof tree ∇_1 , as seen in Figure 9.3.

9.2.4 Resolving the *update* step

To continue tree ∇_0 , we then use P_9 as the precondition for the *update* step, as shown in Figure 9.4. We continue the proof tree in ∇_2 , where we process the *update* step.

To process the *update* step, we first split off the null-instance with SPLIT PARALLEL, which results in P_{10} and P_N . Then, we split off the struct instance

$$\begin{array}{c}
\frac{\frac{\nabla_1}{\vdash \{P_2\} \text{nominate}\{P_9\}} \quad \frac{\nabla_2}{\vdash \{P_9\} \text{update}\{?\}}}{\vdash \{P_2\} \text{nominate} < \text{update}\{?\}} \text{BARRIER} \\
\frac{\vdash \{P_2\} \text{nominate} < \text{update}\{?\}}{\vdash \{P_1\} \text{nominate} < \text{update}\{?\}} \text{WEAKENING} \\
\frac{\vdash \{P_1\} \text{nominate} < \text{update}\{?\}}{\vdash \{P_0\} \text{nominate} < \text{update}\{?\}} \text{EXAMPLE}
\end{array}$$

Figure 9.4: Proof Tree ∇_0 , second unfinished version

$$\begin{array}{c}
\frac{\vdash \{P_{12} \wedge l_1 = l_i\} S_3\{P_{12}\}}{\vdash \{P_{12}\} S_2\{P_{12}\}} \text{FP} \\
\frac{\vdash \{P_{12}\} S_2\{P_{12}\}}{\vdash \{P_{12}\} \text{update}\{P_{12}\}} \text{IS} \\
\frac{\vdash \{P_{12}\} \text{update}\{P_{12}\}}{\vdash \{P_{11}\} \text{update}\{P_{11}\}} \text{SC-WW} \\
\frac{\vdash \{P_{11}\} \text{update}\{P_{11}\}}{\vdash \{P_{10}\} \text{update}\{?\}} \text{GP} \quad \frac{\dots}{\vdash \{P_F\} \text{update}\{?\}} \text{SP} \quad \frac{\dots}{\vdash \{P_N\} \text{update}\{?\}} \text{SP} \\
\frac{\vdash \{P_{10}\} \text{update}\{?\} \quad \vdash \{P_F\} \text{update}\{?\} \quad \vdash \{P_N\} \text{update}\{?\}}{\vdash \{P_9\} \text{update}\{?\}} \text{SP}
\end{array}$$

Figure 9.5: Proof Tree ∇_2 , first unfinished version. Here, we shorten the proof rule names to the first letters and possible -WW suffixes.

for the label variable l_1 using SPLIT PARALLEL as well, which results in P_{11} and P_F , where P_F is defined as:

$$\begin{aligned}
P_F := & \\
& ((\blacksquare_1(l_1, \text{Elem} \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_1, \text{decider}) \xrightarrow{1} (l_1, \text{false}) * (l_1, \text{winner}) \xrightarrow{1} (\{l_1, \dots, l_n\}, \text{true})) \\
& \wedge \text{Set} = \{l_1, \dots, l_n\})
\end{aligned}$$

The rest of the struct instances we split using GEN PARALLEL. This results in P_{12} .

For the rest of the struct instances we use the STEP CALL-WW rule to initialize the step to S_2 . Then the condition of S_2 resolves to $l_1 = l_i$. We take the postcondition to be P_{12} , and note that $P_{12} \wedge l_1 \neq l_i = P_{12} \wedge \text{true}$ (as $2 \leq i \leq n$), so $P_{12} \wedge l_1 \neq l_i \Rightarrow P_{12}$. Then we need to prove the triple $\vdash \{P_{12} \wedge l_1 = l_i\} S_3\{P_{12}\}$, which holds because $l_1 = l_i$ is false, so we use the FALSE PRECONDITION rule. We then reverse the effects of GEN PARALLEL, to once again gain P_{11} .

This results in the proof tree ∇_2 , as shown in Figure 9.5.

For the first struct instance, we again use the STEP CALL-WW rule, but now we have to find a precondition such that $C(P_F, P')$ and $D(P', P_{F1})$ (note

$$\begin{array}{c}
\frac{}{\vdash \{P_{F2}\} S_3 \{P_{F3}\}} \text{UP-WW} \\
\frac{}{\vdash \{P_{F2}\} S_2 \{P_{F3}\}} \text{IF-STATEMENT} \\
\frac{}{\vdash \{P_{F1}\} S_2 \{P_{F4}\}} \text{EXAMPLE} \\
\hline
\vdash \{P_F\} \text{update} \{P_{F4}\} \text{ STEP CALL-WW}
\end{array}$$

Figure 9.6: **Finished:** Proof Tree ∇_3 .

that $S(P_F, P_F)$. This is the case for the predicate P_{F1} defined as follows:

$$\begin{aligned}
P_{F1} := & ((\blacksquare_1(l_1, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \exists l_w \in \mathcal{L}.(\\
& ((l_1, decider) \xrightarrow{1} (l_1, false) * l_w \in \{l_1, \dots, l_n\} \wedge (l_1, winner) \xrightarrow{1} (l_w, false)) \\
& \wedge Set = \{l_1, \dots, l_n\})
\end{aligned}$$

We use the example rule to remove the existential quantifier, leading to P_{F2} . Then, we use the IF-STATEMENT rule. Here, $P_{F2} \wedge l_1 \neq l_1$ is false, so $P_{F2} \wedge l_1 \neq l_1 \Rightarrow Q$ holds for any postcondition Q . We can then continue with the precondition $P_{F2} \wedge (l_1 = l_1) = P_{F2}$. We use the UPDATE-WW rule, resulting in P_{F3} , with the changes highlighted:

$$\begin{aligned}
P_{F3} := & ((\blacksquare_1(l_1, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \exists l_w \in \mathcal{L}.(\\
& ((l_1, decider) \xrightarrow{1} (\mathbf{l}_w, true) * l_w \in \{l_1, \dots, l_n\} \wedge (l_1, winner) \xrightarrow{1} (l_w, false)) \\
& \wedge Set = \{l_1, \dots, l_n\})
\end{aligned}$$

We then read the existential quantifier removed from P_{F1} to get P_{F4} . This one we use as the postcondition of the if-statement as well. This results in proof tree ∇_3 , as shown in Figure 9.6.

Lastly, for P_N , we use STEP CALL-WW to initialize the step, and then find that the if-condition is true. We then weaken P_N in $P_{N2} = (\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp} * (\ell_{Elem}^0, decider) \xrightarrow{(\zeta_0, R)} (\ell_{Elem}^0, false))$, and use the NULL UPDATE rule to resolve S_3 . We refactor P_{N2} back to P_N , and use P_N as the postcondition for the if-statement. This results in proof tree ∇_4 , as shown in Figure 9.7.

We then merge P_{F4} with P_{11} , which results in P_{12} , and we merge P_N back

$$\begin{array}{c}
\frac{}{\vdash \{P_{N2}\}S_3\{P_{N2}\}} \text{ NULL UPDATE} \\
\frac{}{\vdash \{P_N\}S_3\{P_N\}} \text{ WEAKENING} \\
\frac{}{\vdash \{P_N\}S_2\{P_N\}} \text{ IF-STATEMENT} \\
\hline
\vdash \{P_N\}\text{update}\{P_N\} \text{ STEP CALL-WW}
\end{array}$$

Figure 9.7: **Finished:** Proof Tree ∇_4 .

$$\begin{array}{c}
\frac{}{\vdash \{P_{12} \wedge l_1 = l_i\}S_3\{P_{12}\}} \text{ FP} \\
\frac{}{\vdash \{P_{12}\}S_2\{P_{12}\}} \text{ IS} \\
\frac{}{\vdash \{P_{12}\}\text{update}\{P_{12}\}} * \\
\frac{}{\vdash \{P_{11}\}\text{update}\{P_{11}\}} \text{ GP} \\
\hline
\vdash \{P_{10}\}\text{update}\{P_{10}\}
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash \{P_F\}\text{update}\{P_{F4}\}} \nabla_3 \\
\hline
\vdash \{P_{11}\}\text{update}\{P_{13}\}
\end{array}
\quad
\begin{array}{c}
\frac{}{\vdash \{P_N\}\text{update}\{P_N\}} \nabla_4 \\
\hline
\vdash \{P_{11}\}\text{update}\{P_{13}\}
\end{array}
\text{ SP}$$

Figure 9.8: **Finished:** Proof Tree ∇_2 . Here, we shorten the proof rule names to the first letters for conciseness. The occurrence of ‘*’ is short for SC-WW.

in with P_{12} , which results in P_{13} as given below:

$$\begin{aligned}
P_{13} := & ((\blacksquare_1(l_1, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \exists l_w \in \mathcal{L}. (\\
& ((l_1, decider) \xrightarrow{1} (l_w, true) * l_w \in \{l_1, \dots, l_n\} \wedge (l_1, winner) \xrightarrow{1} (l_w, false)) \\
& \wedge Set = \{l_1, \dots, l_n\}) * \\
& \otimes_{2 \leq i \leq n} ((\blacksquare_1(l_i, Elem \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_i, decider) \xrightarrow{1} (l_1, false) * (l_i, winner) \xrightarrow{1} (\ell_{Elem}^0, false)) \\
& \wedge Set = \{l_1, \dots, l_n\}) * (\blacksquare_1(\ell_{Elem}^0, Elem) \wedge \emptyset_2 \wedge \mathbf{emp})
\end{aligned}$$

This results in the finished proof tree ∇_2 , as given in Figure 9.8.

9.2.5 Concluding the proof

We also use P_{13} to finish proof tree ∇_0 , as given in Figure 9.9. We extract Set from the struct instances big separator again, resulting in P_{14} , and we read

$$\begin{array}{c}
\frac{\nabla_1}{\vdash \{P_2\} \textit{nominate}\{P_9\}} \quad \frac{\nabla_2}{\vdash \{P_9\} \textit{update}\{P_{13}\}} \quad \text{BARRIER} \\
\frac{\vdash \{P_2\} \textit{nominate} < \textit{update}\{P_{13}\}}{\vdash \{P_1\} \textit{nominate} < \textit{update}\{P_{14}\}} \quad \text{WEAKENING} \\
\frac{\vdash \{P_1\} \textit{nominate} < \textit{update}\{P_{14}\}}{\{P_0\} \textit{nominate} < \textit{update}\{P_{15}\}} \quad \text{EXAMPLE}
\end{array}$$

Figure 9.9: **Finished:** Proof Tree ∇_0

the existential quantifiers, which results in P_{15} :

$$\begin{aligned}
P_{15} := & \exists \textit{Set} \subset \mathcal{L}. \textit{Set} = \{l_1, \dots, l_n\} \wedge \\
& ((\blacksquare_1(l_1, \textit{Elem} \wedge \emptyset_2) \wedge \emptyset_2) \wedge \exists l_w \in \mathcal{L}. (\\
& ((l_1, \textit{decider}) \xrightarrow{1} (l_w, \textit{true}) * l_w \in \{l_1, \dots, l_n\} \wedge (l_1, \textit{winner}) \xrightarrow{1} (l_w, \textit{false}))) * \\
& \otimes_{2 \leq i \leq n} ((\blacksquare_1(l_i, \textit{Elem} \wedge \emptyset_2) \wedge \emptyset_2) \wedge \\
& ((l_i, \textit{decider}) \xrightarrow{1} (l_i, \textit{false}) * (l_i, \textit{winner}) \xrightarrow{1} (\ell_{\textit{Elem}}^0, \textit{false}))) \\
& * (\blacksquare_1(\ell_{\textit{Elem}}^0, \textit{Elem}) \wedge \emptyset_2 \wedge \textit{emp})
\end{aligned}$$

We conclude that in the program from nomination as given in Listing 9.1, the elements succesfully nominate a winner, and the decider, generalized to the first element, overwrites its own *decider*-reference to refer to the winner. With this, we have demonstrated how to prove the results of a write-write race condition in AuDaLogic with the adaptations given in this section.

9.3 Conclusion

In this chapter, we have adapted AuDaLogic to allow for write-write race conditions, chiefly by adapting the memory heap to allow for multiple values per memory cell, and by adapting the safety predicate to reset non-deterministic heaps back to deterministic heaps after every step.

We have adapted the most difficult soundness proofs from Section 8.3 that would be affected by the adaptations, to demonstrate that not much changes for the soundness proofs. Furthermore, we have given a small example of how AuDaLogic works with the current adaptations.

In the next chapter, we shall conclude the thesis.

Chapter 10

Conclusion and future work

In this chapter, we reflect on the research questions given in Section 1.2 and discuss their implications and possible future work.

In Section 1.2, we gave the following two main research questions:

Research Question 1. What does a parallel programming language following the data-autonomous paradigm look like, and what properties does that language have?

Research Question 2. Can we create formal methods to prove programs of AuDaLa correct?

The first question we answered by creating AuDaLa, and the second question we answered by some work on semantic verification using the semantics of AuDaLa and the creation of AuDaLogic. We created several subquestions to answer in the two parts of this thesis, which we shall discuss separately.

10.1 Discussion of Part I

As the questions for Part I are pretty varied, in this section, we shall discuss the subquestions defined in Section 1.2.1 in turn.

10.1.1 Theoretical Basis

Subquestion 1. What is the theoretical basis of AuDaLa?

We solve this question in Chapter 2. To do so, we first focused on the concepts of the language, including the presentation of a motivating example. We then formally defined the syntax, the type system and the operational

semantics of AuDaLa. Through the concepts and the design of the syntax and program structure of AuDaLa, we have created AuDaLa as a language that approaches parallelism using structure and separation of concerns. While AuDaLa takes a rather hands-off approach to race conditions in function execution, by separating loops from function execution as well, AuDaLa separates parallelism from looping behaviour, which makes it easier to consider the impact of parallel execution in AuDaLa. Furthermore, through the focus on the environment as opposed to process execution and the requirement that everything executes in parallel by default, AuDaLa shifts the focus of programming from a more process-focused, sequential point of view to a more environment and parallel point of view. We consider AuDaLa to be very suited for intuitive parallel programming.

We have used an operational semantics to express AuDaLa, with two distinct layers: the commands and the schedule. Using an operational semantics fits our focus on programmability and understanding parallelism: as opposed to a denotational or axiomatic semantics, the operational semantics allowed us to describe what the programming language does concretely, without abstraction. Through the use of commands, we were furthermore able to keep this description small, which makes it easier to understand the semantics completely. This, in turn, allows for a better understanding of the programming language, which aids in understanding the behaviour of the language and creating an intuition for programming in it.

As a direct answer to the research question, we consider the contents of Chapter 2 to give a solid theoretical basis, which furthermore fits our intent for the language AuDaLa. Future work consists of using and adapting the theoretical basis of AuDaLa to improve, verify and implement AuDaLa. We have made a start on this by exploring the other subquestions of Research Question 1.

10.1.2 Simplicity and Structure

Subquestion 2. To what extent is AuDaLa simple and structured?

The answer to this question follows from Chapter 2 and Chapter 3: AuDaLa is simple and structured due to a few reasons. AuDaLa is structured due to the separation of concerns built into the design of the language. The structs, steps and schedule are distinctly separate in the syntax, type system and the semantics in the language. Though the design of the program can jump back and forth between them, the three different sections of the program have clear and distinct functions. This is also supported by Chapter 3, where the design of the program is discussed in a structured way, in which the structure as depicted in Figure 2.1 is clearly visible.

AuDaLa is simple due to a few properties of the language. From Chapter 2, the behaviour of an AuDaLa execution, as encoded in the operational semantics, is reasonably straightforward, with only fifteen operational rules. Furthermore, by separating steps and schedules, we separate loops from steps in AuDaLa program, which separates finding race conditions from loops. This allows for a simpler conceptualization of the race conditions in a step, as one does not need to take multiple loop iterations into account. Because of this, we think that AuDaLa stays simple, even though step executions may happen asynchronously and the language does not explicitly control race conditions. Lastly, as supported by Chapter 3 in our opinion, the structure of AuDaLa and the lack of hardware concerns in AuDaLa code makes the code relatively easy to follow. This is further supported by the proofs in Chapter 5, which also roughly follow the structure as outlined in Figure 2.1.

Currently, there is one language construct which is, as far as we know, superfluous in AuDaLa: the local step call. We have so far not been able to find an example in which the presence of this construct is required. One could therefore make AuDaLa even simpler in future work by removing that construct. We have currently not done this, due to its potential use if one were to extend AuDaLa to support packages and inheritance, which is future work discussed in the next subsection.

Other future work would have to figure out what one can even consider ‘simple’ and ‘structured’ when designing languages and why, by adding to the currently small body of programmability research. As this is an entire project on its own, we consider this future work to be well out of the scope of this thesis, though important for studying programming language programmability.

10.1.3 Expressivity

Subquestion 3. How expressive is AuDaLa?

We explored this subquestion by proving Theorem 5.3.34, and by giving semantic adaptations for AuDaLa in Section 6.1. From this, we conclude that in some ways, AuDaLa is similarly expressive as many other programming languages, as it is Turing complete. In other ways, AuDaLa does not support certain kinds of behaviours, like singling out parameters to be stable in fixpoints and allowing for arrays to be used, due to its structured and simple design, as shown in Section 6.1.

Of the extensions presented in Section 6.1, we think that the extension for parameter-specific fixpoints as presented in Section 6.1.1 has the most merit, as it directly increases the expressivity of AuDaLa, while still keeping AuDaLa simple. Due to its simplicity, we also think that this extension would probably not lead to too much overhead in the execution time of AuDaLa programs, compared to programs executed with the original semantics.

While the iterator as presented in Section 6.1.1, does add a more powerful language feature to AuDaLa, the iterator does not allow for the creation of AuDaLa programs that cannot be expressed otherwise. We therefore consider the addition to the expressivity of AuDaLa to be small. It is, however, simple and it would be very interesting to implement it, as it will probably remove overhead when used correctly. In return, verifying programs with an iterator using the methods in Chapter 4 would be a lot harder, as would verifying the programs with AuDaLogic as presented in Part II.

In future work, we could investigate allowing for the nesting of fixpoints and iterators into iterators. More interestingly, we could investigate some kind of asynchronous separator, which separates steps without synchronisation in between. However, both of these will probably require a fundamental change to AuDaLa's semantic structure, and will require a thorough analysis of the impact of adding it, which is outside of the scope of this thesis.

Lastly, the array extension does make AuDaLa more expressive, and is directly and theoretically useful to convert older parallel algorithms to AuDaLa and because many programmers will already be used to them. However, the impact on the efficiency of AuDaLa will have to be tested. Additionally, using arrays may lead to less parallel code: traversing an array is a sequential process, which can lead to a delayed application of new information to elements in the array.

For example, where in the reachability example of Section 3.1, the edges propagated reachability information obtained in the next step without extra delay, the first Node in Listing 6.3 will only try to set Node 3 to reachable in the second step. As separating steps from loops is a fundamental design choice of AuDaLa, we furthermore do not think it is a good idea to solve this problem by introducing limited loops in the steps. However, even if it were solved that way, such a simple array traversal loop would make step execution less uniform and more sequential, as the struct instance with the longest array walked through now has to finish before the next step can be executed, and it could be that this array is significantly larger than the other arrays.

Furthermore, our current array extension considers all elements of any array to be parameters, which is a problem for fixpoints if a step uses a local array. Solving this would require a more sophisticated way to keep track of which arrays are local. As arrays are special struct instances, this would require a way to keep track of which parameters an array has been set as the value of, which would lead to more complicated extensions and read and write procedures.

We therefore estimate that to make an array-like structure work in AuDaLa, more extensive extensions outside of the scope of this thesis would be required. As the concepts and design philosophy of AuDaLa is not directly compatible with arrays, implementing arrays should require more considera-

tion on how the concept of arrays would work with AuDaLa and how arrays should be integrated with AuDaLa without sacrificing the benefits of AuDaLa.

A similar potential extension to AuDaLa would be the inclusions of locks in AuDaLa, to deal with the fact that as it is currently defined, AuDaLa assumes atomic registers (by assuming the atomicity of the **rd**- and **wr**-commands). While locks are a solution, they may not be the most fitting solution to AuDaLa, or they may have to be adapted conceptually to work with AuDaLa's design and focus on simplicity. To do this, like with arrays, a study of the integration of locks in AuDaLa should be done in future work, as well as research into possible alternatives.

As additional future work for this question we propose an extension to include inheritance and packages in AuDaLa. This will allow us to work with AuDaLa as it is to create new constructs, without having to fundamentally change the semantics or requiring us to add in certain code for every new program, making AuDaLa more expressive by making it easier to express more complicated programs. To make this extension, it is first important to consider whether inheritance and packages should be implemented as functions on the AuDaLa syntax, which would allow us to keep the current semantics, or whether this requires a semantics adaptation as well.

Other, more general future work is to further work on theoretical expressivity, for example by establish more defined expressivity classes for parallel programming, and to establish which class AuDaLa has in those classes. However, this task is hard and out of scope for this thesis.

10.1.4 Semantic Verification

Subquestion 4. How can we verify AuDaLa programs using the semantics and how difficult is doing so?

We answered this question in Chapter 4 and Chapter 5. In Chapter 4, we defined multiple properties of AuDaLa programs, and notably *well-formed* AuDaLa programs. For the AuDaLa programs, the properties we proved basically give the pre- and postconditions of every possible transition relative to the state the transition is taken from, as well as some guarantees on how the steps and the schedule interact. One notable result is that we used these properties to prove AuDaLa's type system sound semantically. This was more convenient and required less formal reasoning than proving the type system sound separately.

The properties of Chapter 4 reflect our approach to proving parallel programs correct. In parallel programs, we consider it far more important to establish what a program actually does than to establish only whether it does what we expect or not. This is particularly visible in our approach for proving steps in Chapter 5: we work from the start of the step to the end, instead of

defining our intended postcondition first. While we still need to define the intended postcondition up front for the loops, we think moving towards a more observational approach on the behaviour of parallel programming would be beneficial.

Though Chapter 4 describes some properties about race conditions for AuDaLa programs, these have so far not been used to develop algorithms for actually finding the race conditions in AuDaLa, which we leave to future work. Furthermore, the proofs in Chapter 4 are lengthy and complicated, and would benefit from being implemented in a theorem prover to be absolutely sure there are no holes.

In Chapter 5, we give semantical proofs for two example programs. Though without much of a framework, these proofs already indicate some properties of the proof structure for AuDaLa proofs. The proofs for steps and schedule are clearly distinct and the proof has a focus on pre- and postconditions, which can be combined. For step proofs, the struct instance which executes the step is relevant, due to the decentralised environment of AuDaLa, and sometimes this ‘perspective’ can be inverted to make it easier to deal with race condition effects (as seen in Lemma 5.2.15). Furthermore, termination is important for proving that a program actually achieves any goal at all, and to do so, one will always have to prove that a fixpoint becomes stable at some point.

However, the semantic proofs are lengthy, mostly because we need to establish the context and environment for every step contract and fixpoint proof that we do. Furthermore, the fixpoint proofs are full-on inductions, which can get quite complicated. To solve this, we have created a proof system in Part II, AuDaLogic, to give a framework and standardise the proofs more, which makes proving AuDaLa programs more straightforward. In future work, the semantic proofs could be used to get an intuition for proof structure and standard proof techniques which should be added into more abstract proving methods, like proof systems.

10.1.5 Hardware and Optimisations

Subquestion 5. How can we improve AuDaLa’s semantics to be more compatible with common compiler and hardware optimisations?

In Section 6.2 and Section 6.3, we give one way to make AuDaLa more compatible with common compiler and hardware optimisations. As also stated in those sections, the weak-memory semantics that we present is based on, but changed from, a version of the semantics by Leemrijse *et al.* [110]. We consider our results to be straightforwardly extendable to that version, as discussed in Section 6.2.

The most important notion discussed in Section 6.2 and Section 6.3 in our opinion is the semantic web approach to programming language semantics. In this approach, instead of treating the semantics of a language as a monolith, one develops separate, fully realised semantics for different purposes and connects them formally. This has the main benefit of separating different concerns in the analysis of semantics, compilers and programs.

By using this method, concerns for the semantics can be separated in analysis and program execution. If a property or a specific hardware has to be explored, it can be done separately from other versions of the semantics that explore other properties and hardware. For example, to implement arrays in AuDaLa, we do not have to implement it on top of the weak-memory semantics to start with. This reduces the complexity of extending the semantics, and allows us to consider specific extensions separately from each other to start with. Even if we want to combine extensions, a web approach allows us to pick the extensions we want to combine and to ignore extensions which are not relevant to the intended purpose of the new extended semantics.

Furthermore, by creating these kinds of connections, one can simplify analysis of programs, for example for verification. As an example, with the results from Section 6.2 and Section 6.3, we can prove programs which fulfill the race condition requirement correct using the original semantics, and then execute them using the weak-memory semantics. As the original semantics is simpler than the weak-memory semantics, this is preferable over having to prove the program in the weak-memory semantics. Implementation and compiler correctness, in turn, also benefits from this. We can implement both the original semantics and the weak-memory semantics, and verify the program using the original semantics. We can then use the results of executing the program in both implementations to find errors in the implementations and the compilers.

These are powerful benefits for language design and the design of compilers. However, it does take some more work when creating semantics to maintain the web, as seen in Section 6.3, and it does require a fully specified semantics to start with.

Future work for this subquestion can then be divided into more hardware-oriented future work and more semantics-oriented future work. Hardware-oriented future work in the direction of the weak-memory semantics would build upon the work by Leemrijse *et al.* [109, 110] to further develop a compiler for AuDaLa. A prototype has already been created, and by developing a formalised memory model for AuDaLa and comparing it to the PTX memory model as used by Nvidia for CUDA, the first steps have been made for further optimising that compiler. Other possibilities are to directly implement AuDaLa on hardware, or to make compilers to other languages.

As further hardware-oriented future work for making AuDaLa more compatible with hardware, some of the proofs in Chapter 4 also depend on the

fact that AuDaLa cannot destroy struct instances after they are created. It would be worthwhile for the implementations of AuDaLa to instead specify some requirements on what struct instances are considered obsolete and can be removed, and use that to replace any assumptions on that struct instances cannot be removed. This will then allow for the implementation of garbage collection without complications. Furthermore, as already discussed in the last subsection, AuDaLa assumes atomic registers, and making a new semantics to also be able to use AuDaLa without atomic registers would be worthwhile for applying AuDaLa on current-day hardware. Lastly, another avenue of future work is to investigate adapting AuDaLa for message passing, potentially as a new semantics to add to the semantics web of AuDaLa.

For semantics-oriented future work, it would be interesting to find programming languages with a similar web approach to different semantics, use the web approach on other languages, and to expand the web of AuDaLa for extensions like inheritance and packages. Work could also be done on standardizing the way these connections between semantics are made, for example by proving meta-theorems that make it easier to establish the connections.

10.2 Discussion of Part II

In Part II, we discussed the design of the proof system AuDaLogic for AuDaLa. In Section 1.2.2, we defined three subquestions for this question:

Subquestion 6. How can we define a proof system for AuDaLa?

Subquestion 7. Is AuDaLogic *sound*?

Subquestion 8. How can we verify AuDaLa programs with write-write race conditions?

We answer the research question and the subquestions in Chapter 7, Chapter 8 and Chapter 9 of this thesis. In Chapter 7, we define the formal language and the proof rules for the proof system AuDaLogic. This proof system is based on concurrent separation logic (CSL) and specifically the work by Vafeiadis [173].

The proof system adapts CSL to the setting of AuDaLa by casting the decentralised parameters of AuDaLa to a fictional memory heap and by adding a new heap-like structure to also be able to reason about which elements are currently considered active. Furthermore, as AuDaLa has a decentralised environment, connected by references, reasoning about the structure of the environment is required, for which we added higher-order separation logic to the formal language of AuDaLogic.

As AuDaLa is highly *relative*, as a single struct instance's perception of the environment depends on its references, we also added a high level of abstraction to the formal language of AuDaLogic through the use of AuDaLogic variables on top of AuDaLa variables. Though the use of AuDaLogic variables and the use of references in AuDaLa led to the concern of aliasing, we dealt with this concern by making the update rules and the constructor rules more specific, and by keeping it in mind during the design of the semantics of AuDaLogic.

We also defined functions to give AuDaLogic equivalents of AuDaLa expressions in AuDaLogic and to generate the predicate that needs to be fulfilled for a step or schedule execution to be stable. When designing the proof rules in general, we kept an eye on the conclusions we took from the semantic proofs, and notably kept the clear separation of the schedule and the step executions. We consider the proof system AuDaLogic to answer Subquestion 6.

In Chapter 8, we connected AuDaLa to AuDaLogic by formalising how AuDaLa states and executions connect to AuDaLogic states, as well as what we expect from an AuDaLogic triple in the context of AuDaLa executions in the safety predicate functions $\text{safeSt}_{\mathcal{P}}$ for statements and $\text{safeSc}_{\mathcal{P}}$ for schedules. Here, race conditions are taken into account only after the individual executions of steps, as they lead to merge conflicts when merging the heaps of individual struct instances back together.

We then used these notions to prove the expression transformation from AuDaLa to AuDaLogic and the stability predicate generator function correct. After this, we proved the proof rules as defined in Chapter 7 sound, using the semantic definition of AuDaLogic triples. With this, we answer Subquestion 7.

In Chapter 9, prompted by the proofs of Chapter 5, we extended AuDaLogic to also be able to process write-write race conditions, which we did by extending the memory heap and the merging procedures. This change we then propagated through the entire proof system. A notable change here is that after every step execution, we reset the heap to be deterministic in the safety predicate for statements $\text{safeSc}_{\mathcal{P}}$, to reflect that one value has won the race condition. However, as we do this non-deterministically, we do not fix the end result of the race condition and the safety predicate is correctly extended for write-write race conditions, while still allowing us to reason about deterministic heaps in the next step execution. With this, we answer Subquestion 8.

There are several directions for future work, however. Firstly, AuDaLogic currently uses the separator for splitting both the active struct instances of the program and the memory resources of the program. In future work, we can study whether it would be beneficial to create separate separators for these two resources. While using only one separator leads to a smaller syntax and semantics and works for our purposes so far, it does create some overlap in concerns, as we always have to consider both the implications on the label

triple and on the memory heap.

AuDaLogic and its extension for write-write race conditions currently also uses the more abstract relations S, D and C . In future work, these can be expanded upon, by instead finding a way to make predicate generator functions out of these, like the *stable*-function.

The design of AuDaLogic and its soundness proof have so far only been done on paper. In future work, we can encode AuDaLogic in a proof assistant, like for example also done by Vafeiadis [173], to truly guarantee the proof is sound. This we leave to future work, where it can probably be combined with looking into formalising ghost variables better for AuDaLogic.

Encoding AuDaLogic in a proof assistant will also help with another direction of future work: axiomatising AuDaLogic. Generally, proof systems include an axiomatisation, which defines some standard truths in the system. In this thesis, we focused on the proof rules and the formal logic of AuDaLogic. This means, however, that during AuDaLogic proofs one needs to understand the semantics and be able to reason about the semantics as defined, as also demonstrated in Section 7.5 and Section 9.2. Creating a full axiomatisation of AuDaLogic would allow us to prove the methods used in these examples correct formally, and lead to an easier application of the proof system as a whole.

Another major research direction is to do some more analysis on AuDaLogic, its expressivity with regards to conventional CSL and how to improve upon it if possible. This analysis can start with an analysis of the role of critical sections and locks in (concurrent) separation logic, and whether these would be logical additions for AuDaLogic. This can be done together with the future work of determining whether locks would be a good extension for AuDaLa. For this, some more delving into the origins and original purpose of resource invariants, and how that would translate to AuDaLa, would be required.

Currently, AuDaLogic currently uses higher-order logic. We estimate that AuDaLogic does require this, as we are reasoning about groups of struct instances and their relations. However, a direction for future work is to figure out whether we can constrain AuDaLogic to a subset of higher-order logic. Furthermore, the definition of AuDaLogic expressions should be fine-tuned further to fit the needs of AuDaLogic.

Lastly, AuDaLogic currently uses standard set theory. However, due to AuDaLa allowing for arbitrary choice of new labels for new struct instances, this means we will have to keep track of which labels are used and not used via the use of $\alpha(3)$ in the current version of AuDaLogic. We could possibly simplify this by using Fraenkel-Mostowski set theory [2], which allows for easier handling of variables for which only the distinction matters and not the exact names. We leave investigating this and what it would mean for encoding AuDaLogic in a proof assistant for future work.

Appendices

Appendix A

Rules Summary

A.1 Type System Rules

A.1.1 Environment Type Rules

$$\begin{array}{c} \text{(Empty)} \frac{}{\emptyset, \emptyset \vdash \diamond} \\[2ex] \text{(AddType)} \frac{\Gamma, \Omega \vdash \diamond \quad \theta \notin \text{dom}(\Omega) \quad \theta \notin K \quad \forall 1 \leq i < j \leq n. (x_i \neq x_j)}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n) \vdash \diamond} \quad \text{(AddVar)} \frac{\Gamma, \Omega \vdash T \quad x \notin \text{dom}(\Gamma) \quad x \notin K}{\Gamma; x : T, \Omega \vdash \diamond} \end{array}$$

A.1.2 Types Type Rules

$$\begin{array}{c} \text{(TBasic)} \frac{\Gamma, \Omega \vdash \diamond \quad T \in \{\text{Nat}, \text{Int}, \text{Bool}, \text{String}, \text{Step}\}}{\Gamma, \Omega \vdash T} \\[2ex] \text{(TStruct)} \frac{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \diamond}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta} \\[2ex] \text{(DefStruct)} \frac{\Gamma, \Omega \vdash \theta \quad \forall 1 \leq i \leq n. \Gamma, \Omega \vdash T_i}{\Gamma, \Omega \vdash \theta(x_1 : T_1, \dots, x_n : T_n)} \end{array}$$

A.1.3 Standard Values Type Rules

$\text{(Bool)} \frac{\Gamma, \Omega \vdash \diamond \quad b \in \{true, false\}}{\Gamma, \Omega \vdash b : \text{Bool}}$	$\text{(String)} \frac{\Gamma, \Omega \vdash \diamond}{\Gamma, \Omega \vdash \text{"Str"} : \text{String}}$
$\text{(NInt)} \frac{\Gamma, \Omega \vdash \diamond \quad c < 0}{\Gamma, \Omega \vdash c : \text{Int}}$	$\text{(PNat)} \frac{\Gamma, \Omega \vdash \diamond \quad c \geq 0}{\Gamma, \Omega \vdash c : \text{Nat}}$
$\text{(Null)} \frac{\Gamma, \Omega \vdash T}{\Gamma, \Omega \vdash \text{null} : T}$	$\text{(This)} \frac{\Gamma, \Omega \vdash \theta}{\Gamma, \Omega \vdash \text{this}_\theta : \theta}$

A.1.4 Expressions Type Rules

$\text{(Eq)} \frac{\Gamma, \Omega \vdash E_1 : T \quad \Gamma, \Omega \vdash E_2 : T \quad \circ \in \{=, !=\}}{\Gamma, \Omega \vdash E_1 = E_2 : \text{Bool}}$	$\text{(Comp)} \frac{\Gamma, \Omega \vdash E_1 : N_1 \quad \Gamma, \Omega \vdash E_2 : N_2 \quad \circ \in \{<=, >=, <, >, =, !=\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Bool}}$
$\text{(NArith)} \frac{\Gamma, \Omega \vdash E_1 : \text{Nat} \quad \Gamma, \Omega \vdash E_2 : \text{Nat} \quad \circ \in \{*, /, \%, +, ^\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Nat}}$	$\text{(IArith)} \frac{\Gamma, \Omega \vdash E_1 : N_1 \quad \Gamma, \Omega \vdash E_2 : N_2 \quad \circ \in \{*, /, \%, +, ^, -\}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Int}}$
$\text{(BinLog)} \frac{\Gamma, \Omega \vdash E_1 : \text{Bool} \quad \Gamma, \Omega \vdash E_2 : \text{Bool} \quad \circ \in \{\&\&, \}}{\Gamma, \Omega \vdash E_1 \circ E_2 : \text{Bool}}$	$\text{(Brackets)} \frac{\Gamma, \Omega \vdash E : T}{\Gamma, \Omega \vdash (E) : T}$
$\text{(Neg)} \frac{\Gamma, \Omega \vdash E : \text{Bool}}{\Gamma, \Omega \vdash !E : \text{Bool}}$	$\text{(VarS)} \frac{\Gamma_1; (x : T); \Gamma_2, \Omega \vdash \diamond}{\Gamma_1; (x : T); \Gamma_2, \Omega \vdash x : T}$
$\text{(VarR)} \frac{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_i : T_i, \dots, x_n : T_n); \Omega' \vdash X : \theta \quad \Gamma, \Omega; \theta(x_1 : T_1, \dots, x_i : T_i, \dots, x_n : T_n); \Omega' \vdash x_i : T_i}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_i : T_i, \dots, x_n : T_n); \Omega' \vdash X.x_i : T_i}$	
$\text{(ConsE)} \frac{\forall_{0 \leq i \leq n}. \Gamma, \Omega \vdash E_i : T_i}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta(E_1, \dots, E_n) : \theta}$	

A.1.5 Statements Type Rules

$$\begin{array}{c}
\frac{\Gamma, \Omega \vdash E : \text{Bool} \quad \Gamma, \Omega \vdash \mathcal{S}}{(\text{IfThen}) \quad \Gamma, \Omega \vdash \text{if } E \text{ then } \{\mathcal{S}\}} \quad \frac{\Gamma, \Omega \vdash E : T \quad \Gamma_1; (x : T); \Gamma_2, \Omega \vdash \diamond}{(\text{LVar}) \quad \Gamma, \Omega \vdash Tx := E} \\
\\
\frac{\Gamma, \Omega \vdash X : T \quad \Gamma, \Omega \vdash E : T}{(\text{Update}) \quad \Gamma, \Omega \vdash X := E} \quad \frac{S \neq 'Tx := E' \quad \Gamma, \Omega \vdash S}{(\text{Seq}) \quad \Gamma, \Omega \vdash S; \mathcal{S}} \\
\\
\frac{\Gamma, \Omega \vdash Tx := E \quad \Gamma; (x : T), \Omega \vdash \mathcal{S}}{(\text{SeqLVar}) \quad \Gamma, \Omega \vdash Tx := E; \mathcal{S}} \quad (\text{ConsS}) \quad \frac{\forall_{0 \leq i \leq n}. \Gamma, \Omega \vdash E_i : T_i}{\Gamma, \Omega; \theta(x_1 : T_1, \dots, x_n : T_n); \Omega' \vdash \theta(E_1, \dots, E_n)}
\end{array}$$

A.1.6 Schedule Type Rules

$$\begin{array}{c}
(\text{SBar}) \quad \frac{\Gamma, \Omega \vdash \Delta_1 \quad \Gamma, \Omega \vdash \Delta_2}{\Gamma, \Omega \vdash \Delta_1 < \Delta_2} \quad (\text{SFix}) \quad \frac{\Gamma, \Omega \vdash \Delta}{\Gamma, \Omega \vdash \text{Fix}(\Delta)} \\
\\
(\text{STypCall}) \quad \frac{\Gamma, \Omega \vdash F_\theta : \text{Step}}{\Gamma, \Omega \vdash \theta.F} \quad (\text{SNormCall}) \quad \frac{\Gamma, \Omega \vdash F_\theta : \text{Step}}{\Gamma, \Omega \vdash F}
\end{array}$$

A.1.7 Program Type Rule

$$(\text{Prog}) \quad \frac{\begin{array}{l} \mathcal{P} = D_{\mathcal{P}} \Delta_{\mathcal{P}} \\ \forall \theta(x_1 : T_1, \dots, x_n : T_n) \in \text{Structs}(D). \Gamma, \Omega; \text{Structs}(D) \vdash \theta(x_1 : T_1, \dots, x_n : T_n) \\ \forall (F_\theta, \mathcal{S}) \in \text{StepDef}(D). \Gamma; \text{Pars}_{\mathcal{P}}(\theta), \Omega; \text{Structs}(D) \vdash \mathcal{S} \quad \Gamma; \text{Steps}(D), \Omega \vdash \Delta \end{array}}{\Gamma, \Omega \vdash \mathcal{P}}$$

A.2 Operational Semantics Rules

A.2.1 Command Rules

$$\begin{array}{c}
(\text{ComPush}) \quad \frac{\sigma(\ell) = \langle \theta, \text{push}(v); \gamma, \chi, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; v, \xi \rangle], s\chi \rangle} \\
\\
(\text{ComPushThis}) \quad \frac{\sigma(\ell) = \langle \theta, \text{push}(\text{this}); \gamma, \chi, \xi \rangle}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \ell, \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{rd}(x); \gamma, \chi; \ell', \xi \rangle \\
\sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\
\text{(ComRd)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \xi'(x), \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{wr}(x); \gamma, \chi; v; \ell', \xi \rangle \\
\sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\
\ell' \notin \mathcal{L}^0 \vee x \notin \text{Pars}_{\mathcal{P}}(\theta') \\
su = (x \notin \text{Pars}_{\mathcal{P}}(\theta') \vee \xi'(x) = v) \\
\text{(ComWr)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle][\ell', 4 \mapsto \xi'[x \mapsto v]], \\
s\chi_1 \wedge su; \dots; s\chi_{|s\chi|} \wedge su \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{wr}(x); \gamma, \chi; v; \ell', \xi \rangle \\
\sigma(\ell') = \langle \theta', \gamma', \chi', \xi' \rangle \\
\ell' \in \mathcal{L}^0 \wedge x \in \text{Pars}_{\mathcal{P}}(\theta') \\
\text{(ComWrNSkip)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{not}; \gamma, \chi; b, \xi \rangle \\
\text{(ComNot)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; \neg b, \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{op}(\circ); \gamma, \chi; a; b, \xi \rangle \\
\text{(ComOp)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi; (a \circ b), \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{cons}(\theta'); \gamma, \chi; v_1; \dots; v_n, \xi \rangle \\
\text{Pars}_{\mathcal{P}}(\theta') = x_1 : T_1; \dots; x_n : T_n \\
\sigma(\ell') = \perp \\
\text{(ComCons)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \gamma, \chi; \ell', \xi \rangle, \\
\ell' \mapsto \langle \theta', \varepsilon, \varepsilon, \xi_{\theta'}^0[\{x_1 \mapsto v_1, \dots, x_n \mapsto v_n\}]\}], false^{|s\chi|} \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{if}(C); \gamma, \chi; true, \xi \rangle \\
\text{(ComIfT)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, C; \gamma, \chi, \xi \rangle], s\chi \rangle}
\end{array}$$

$$\begin{array}{c}
\sigma(\ell) = \langle \theta, \mathbf{if}(C); \gamma, \chi; false, \xi \rangle \\
\text{(ComIfF)} \frac{}{\langle \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\ell \mapsto \langle \theta, \gamma, \chi, \xi \rangle], s\chi \rangle}
\end{array}$$

A.2.2 Schedule Rules

$$\begin{array}{c}
\text{(InitG)} \frac{Done(\sigma)}{\langle F < \Delta, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta, \sigma[\{\ell \mapsto \langle \theta, \llbracket S_{\theta}^F \rrbracket, \varepsilon, \xi \rangle \mid \sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle\}], s\chi \rangle} \\
\\
\text{(InitL)} \frac{Done(\sigma)}{\langle \theta.F < sc, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle sc, \sigma[\{\ell \mapsto \langle \theta, \llbracket S_{\theta}^F \rrbracket, \varepsilon, \xi \rangle \mid \sigma(\ell) = \langle \theta, \varepsilon, \chi, \xi \rangle\}], s\chi \rangle} \\
\\
\text{(FixInit)} \frac{Done(\sigma)}{\langle Fix(\Delta) < \Delta_1, \sigma, s\chi \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle} \\
\\
\text{(FixIter)} \frac{Done(\sigma)}{\langle aFix(\Delta) < \Delta_1, \sigma, s\chi; false \rangle \Rightarrow_{\mathcal{P}} \langle \Delta < aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle} \\
\\
\text{(FixTerm)} \frac{Done(\sigma)}{\langle aFix(\Delta) < \Delta_1, \sigma, s\chi; true \rangle \Rightarrow_{\mathcal{P}} \langle \Delta_1, \sigma, s\chi \rangle}
\end{array}$$

A.3 AuDaLogic Proof Rules

A.3.1 Statement Rules

$$\begin{array}{c}
\begin{array}{l}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
P_L \Rightarrow \blacksquare_1(a_{\varepsilon}, \theta) \quad P_H \Rightarrow A(a_{\varepsilon}, E, X) \\
P = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} -) \\
Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{1} eqi(a_{\varepsilon}, E))
\end{array} \\
\text{(UPDATE)} \frac{}{\vdash \{P\} X.x := E \{Q\}} \\
\\
\begin{array}{l}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
P_L \Rightarrow \blacksquare_1(a_{\varepsilon}, \theta) \quad P_H \Rightarrow A(a_{\varepsilon}, E, X) \\
P = P_L \wedge (P_H * (a_X, x) \xrightarrow{\zeta_0} E')
\end{array} \\
\text{(NULL UPDATE)} \frac{}{\vdash \{P\} X.x := E \{P\}} \\
\\
\begin{array}{l}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
Pars_{\mathcal{P}}(\theta) = x_1, \dots, x_e \quad P_L \Rightarrow \blacksquare_1(a_{\varepsilon}, \theta) \\
P_H \Rightarrow \wedge \neg(a_{\varepsilon}, x) \hookrightarrow - \wedge A(a_{\varepsilon}, E_1, \dots, E_e) \\
Q = (P_L * \blacksquare_2(l', \theta')) \wedge (P_H * (a_{\varepsilon}, x) \xrightarrow{1} l' * Q') \\
Q' = \bigotimes_{1 \leq i \leq e} (l', x_i) \xrightarrow{1} eqi(a_{\varepsilon}, E_i)
\end{array} \\
\text{(CONSTRUCTOR)} \frac{}{\vdash \{(P_L * fresh(l')) \wedge P_H\} \theta \ x := \theta'(E_1, \dots, E_e) \{Q\}}
\end{array}$$

$$\begin{array}{c}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \quad P_H \Rightarrow \neg(a_\varepsilon, x) \hookrightarrow - \\
\vdash \{P_L \wedge (P_H * (a_\varepsilon, x) \xrightarrow{1} -)\} x := E\{Q\} \\
\text{(ASSIGNMENT-1)} \frac{}{\vdash \{P_L \wedge P_H\} T \ x := E\{Q\}}
\end{array}$$

$$\begin{array}{c}
P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
P_L \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \quad P_H \Rightarrow (a_\varepsilon, x) \hookrightarrow - \\
\vdash \{P_L \wedge P_H\} x := E\{Q\} \\
\text{(ASSIGNMENT-2)} \frac{}{\vdash \{P_L \wedge P_H\} T \ x := E\{Q\}}
\end{array}$$

$$\begin{array}{c}
P \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \wedge A(a_\varepsilon, E) \\
P \wedge \neg eqi(a_\varepsilon, E) \Rightarrow Q \\
\vdash \{P \wedge eqi(a_\varepsilon, E)\} S\{Q\} \\
\text{(IF-STATEMENT)} \frac{}{\vdash \{P\} \text{if } E \text{ then } \{S\}\{Q\}}
\end{array}$$

$$\begin{array}{c}
P \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \\
\vdash \{P\} S\{R\} \quad \vdash \{R\} S\{Q\} \\
\text{(SEQUENCE)} \frac{}{\vdash \{P\} S; S\{Q\}}
\end{array}$$

A.3.2 Parallel Rules

$$\begin{array}{c}
\vdash \{P_1\} F\{Q_1\} \quad \vdash \{P_2\} F\{Q_2\} \\
\text{(SPLIT PARALLEL)} \frac{}{\vdash \{P_1 * P_2\} F\{Q_1 * Q_2\}}
\end{array}$$

$$\begin{array}{c}
\vdash \{P(i)\} F\{Q(i)\} \\
\text{(GEN PARALLEL)} \frac{}{\vdash \{\otimes_{1 \leq i \leq n} (P(i))\} F\{\otimes_{1 \leq i \leq n} (Q(i))\}}
\end{array}$$

A.3.3 Schedule Rules

$$\begin{array}{c}
\vdash \{P\} \Delta_1 \{R\} \quad \vdash \{R\} \Delta_2 \{Q\} \\
\text{(BARRIER)} \frac{}{\vdash \{P\} \Delta_1 < \Delta_2 \{Q\}}
\end{array}$$

$$\begin{array}{c}
S(P, P') \quad P' \Rightarrow \blacksquare_1(a_\varepsilon, \theta) \\
StepStat_{\mathcal{P}}(F, \theta) = S \quad \vdash \{P'\} S\{Q\} \\
\text{(STEP CALL)} \frac{}{\vdash \{P\} F\{Q\}}
\end{array}$$

$$\begin{array}{c}
\vdash \{P_1\} F\{Q\} \quad P_1 \Rightarrow \blacksquare \theta \\
P_2 \Rightarrow \neg \sqsupset \theta \\
\text{(TYPED CALL)} \frac{}{\vdash \{P_1 * P_2\} \theta.F\{Q * P_2\}}
\end{array}$$

$$\begin{array}{c}
 \forall_{j \in \mathbb{N}}. j > 0 \Rightarrow (\vdash \{P \wedge Va(j) \wedge \neg \text{stable}_{\mathcal{P}}(\Delta)\} \Delta \{P \wedge Va(j+1)\}) \\
 \exists n > 0. \forall k \geq n. P \wedge Va(k) \Rightarrow \text{stable}_{\mathcal{P}}(\Delta) \\
 \text{(FIXPOINT)} \frac{}{\vdash \{P \wedge Va(1)\} \text{Fix}(\Delta) \{P \wedge Va(n+2) \wedge \text{stable}_{\mathcal{P}}(\Delta)\}}
 \end{array}$$

A.3.4 General Rules

$$\text{(EMPTY)} \frac{}{\vdash \{P\} \varepsilon \{P\}}$$

$$\text{(FALSE PRECONDITION)} \frac{}{\vdash \{false\} \mathcal{X} \{P\}}$$

$$\begin{array}{c}
 \vdash \{P\} \mathcal{X} \{Q\} \\
 P' \Rightarrow P \quad Q \Rightarrow Q' \\
 \text{(WEAKENING)} \frac{}{\vdash \{P'\} \mathcal{X} \{Q'\}}
 \end{array}$$

$$\begin{array}{c}
 \vdash \{P_1\} \mathcal{X} \{Q_1\} \\
 \vdash \{P_2\} \mathcal{X} \{Q_2\} \\
 \text{(DISJUNCTION)} \frac{}{\vdash \{P_1 \vee P_2\} \mathcal{X} \{Q_1 \vee Q_2\}}
 \end{array}$$

$$\begin{array}{c}
 \vdash \{P_1\} \mathcal{X} \{Q_1\} \\
 \vdash \{P_2\} \mathcal{X} \{Q_2\} \\
 \text{(CONJUNCTION)} \frac{}{\vdash \{P_1 \wedge P_2\} \mathcal{X} \{Q_1 \wedge Q_2\}}
 \end{array}$$

$$\begin{array}{c}
 \vdash \{P\} \mathcal{X} \{Q\} \\
 \text{(EXAMPLE)} \frac{}{\vdash \{\exists a. P\} \mathcal{X} \{\exists a. Q\}}
 \end{array}$$

$$\begin{array}{c}
 \vdash \{P\} \mathcal{S} \{Q\} \\
 fv(P) \cap fv(R) = \emptyset \\
 R \Rightarrow \emptyset_1 \\
 \text{(FRAME STATEMENTS)} \frac{}{\vdash \{P * R\} \mathcal{S} \{Q * R\}}
 \end{array}$$

$$\begin{array}{c}
 \vdash \{P\} \Delta \{Q\} \\
 fv(P) \cap fv(R) = \emptyset \\
 R \Rightarrow \emptyset \\
 \text{(FRAME SCHEDULE)} \frac{}{\vdash \{P * R\} \Delta \{Q * R\}}
 \end{array}$$

A.3.5 Write-Write Race Condition Adapted Rules

$$\begin{array}{c}
 P_L \text{ is heap-free} \quad P_H \text{ is triple-free} \\
 P_L \Rightarrow \blacksquare_1(a_\epsilon, \theta) \quad P_H \Rightarrow A(a_\epsilon, E, X) \\
 P = P_L \wedge (P_H * (a_X, x) \xrightarrow{W} -) \\
 \text{(UPDATE-WW)} \frac{Q = P_L \wedge (P_H * (a_X, x) \xrightarrow{W} (eqi(a_\epsilon, E), true))}{\vdash \{P\} X.x := E\{Q\}}
 \end{array}$$

$$\begin{array}{c}
 S(P, P_1) \quad C(P_1, P_2) \quad D(P_2, P_3) \\
 P_3 \Rightarrow \blacksquare_1(a_\epsilon, \theta) \wedge \emptyset_2 \\
 \text{(STEP CALL-WW)} \frac{StepStat_{\mathcal{P}}(F, \theta) = S \quad \vdash \{P_3\} S\{Q\}}{\vdash \{P\} F\{Q\}}
 \end{array}$$

Appendix B

Glossaries

B.1 Part I - AuDaLa

AuDaLa Terms

Notation	Description
(Turing machine/Implementation) configuration	An abstraction over Turing machine states and the states of the Turing machine implementation, used to prove their executions equivalent. See Definition 3.5.5 and Definition 5.3.25.
AuDaLa	The language we create in Part I of the thesis. It stands for ‘Autonomous Data Language’.
AuDaLa state	A semantic state in AuDaLa consists of a schedule, a struct environment and a stability stack. See Definition 2.4.7.
abstract syntax tree	A tree representation of a piece of code, with variables and constants as leaves and other constructs as nodes. Nodes and leaves can be annotated with additional information.
access	An access of a variable x is defined as a read from or a write to variable x . See also Definition 4.1.3.

Notation	Description
asynchronous execution	Execution during which the parallel elements of a program do not take into account the progress of other executing parallel programs.
atomic actions	Actions which are executed without any outside interference from other actions, for example due to parallelism.
atomic propositions	Propositions that express information about states. For example, an atomic proposition may express how many other states may be reached from the current state with a single transition, or it may extract relevant data from the current state. In this thesis, we abstract away from the full state using the C -labeling function (Chapter 6) and configurations (Section 5.3), the outputs of which can be considered atomic propositions.
barrier	Separates schedule elements and acts like a synchronisation point: the schedule execution only proceeds past a barrier when all computations that precede the barrier have concluded.
big O -notation	When considering functions $f(x)$ and $g(x)$, we state that $f(x)$ is bounded by $O(g(x))$ if $f(x)$ does not grow faster than $g(x)$. For example, for any constant c , c is bounded by $O(1)$, as these functions never change their value. The function x is bounded by $O(x)$ and $O(x^2)$, but not by $O(1)$, as the function x is not constant.
breadth-first search	A graph traversal method that considers the nodes in order of their depth: how many edges need to be traversed before the node is reached.
command	A command, defined formally in Definition 2.4.4, are the atomic actions of the semantics of AuDaLa.
data-autonomous	We consider a programming language data-autonomous if in it, the data executes their own computations and all computations are always carried out in parallel by default. Furthermore, the language must abstract away from explicit processors and memory, like the use of threads.
deterministic	Something is deterministic when it will always have the same result relative to the input. See Definition 4.1.4 for a definition relative to AuDaLa.

Notation	Description
execution from p	A sequence of semantics rule transitions starting from AuDaLa state p . See Definition 4.1.2. Extended to expression, statement, command list, step and program executions. Executions of size i contain exactly i transitions.
expressivity	A measure of what can and cannot be expressed in a language.
fixpoint	Looping mechanism of AuDaLa, which executes the steps contained in it until the entire system is stable. When a fixpoint terminates, it will remove itself during the schedule.
identifiers	Names given to program elements in the code by the programmer.
implicit/ explicit parallelism	If a programming language is explicitly parallel, it leaves it to the user to divide the tasks and its resources over the parallel elements throughout the whole program. In contrast, implicitly parallel programming languages rely on their definition and semantics to divide the tasks and resources.
incoherence	A memory consistency model is considered incoherent if it does not require all the processors to always agree on the value of
inference rule notation	A notation we use for type rules, operational rules and proof rules which states that if the premises above the line hold, so does the conclusion below the line. For example, the below inference rule NAME states that if P_1, P_2 and P_3 hold, P holds: $(NAME) \frac{P_1 \quad P_2 \quad P_3}{P}$ Operational rules, instead of asserting that the conclusion holds, instead assert that the transition depicted in the conclusion is enabled.
keywords	Words protected from use by the programmer due to meaning associated with them in the programming language. AuDaLa uses the keywords <code>this</code> , <code>if</code> , <code>then</code> , <code>null</code> , <code>Fix</code> and <code>struct</code> .

Notation	Description
label	A semantic struct instance identifier. Every struct instance has a unique label in the semantics of AuDaLa.
local variables	Local variables of a struct as defined during a step. Local variables can only be accessed by the struct instance for which they are defined. Local variables do not carry over between steps.
Memory Consistency Model	A specification on how memory accesses can appear appear to be executed to the programmer when executing the program.
null-instance	The default instantiation of an AuDaLa struct which represents the undefined instances of that struct.
paradigm	A category of programming languages characterised by a shared set of principles and assumptions in the hardware model.
parameters	Global variables of a struct as defined in the struct definition. Parameters of a struct instance can be used by all struct instances with the right reference. Parameter values carry over between steps.
path	A sequence of semantics states, often generated during an execution. See Definition 5.3.22.
potential race condition	A pair of statements which may during execution lead to a race condition. See Definition 4.1.9.
prefix	A path starting in the initial state. See Definition 5.3.22.
programmability	A measure of how easy and intuitive it is to program in a language.
race condition	A race condition in which the two accesses happen at the same time. The result of these overlapping accesses are generally unpredictable, which makes them undesirable. Can be prevented by correctly implementing the atomicity of certain actions.

Notation	Description
race condition	A conflict between two executing struct instances where two struct instances want to access the same variable, and at least one struct instance wants to write to it. See also Definition 4.1.5. Race conditions without undesirable side effects are considered benign.
reference notation	The notation $s.x$ to denote to the variable x as owned by the struct instance s , or $\ell_s.x$ to denote the same. See Definition 4.1.1.
references	Links to other struct instances, used to access other struct instances.
schedule	The part of the AuDaLa program in which the execution order is defined.
separation of concerns	Making sure that different concerns during program design (dealing with hardware, implementing the concept, synchronisation etc.) do not have to be dealt with in the same sections of code.
sequential consistency	A memory consistency model that assumes that an execution of a parallel program can be expressed as the interleaving of the sequential executions of the parallel elements executing the program.
shared memory language	A language which assumes that all memory is accessible by all processors through a shared memory space.
span	The largest amount of operations that have to be performed sequentially when executing a parallel program. The span limits the speed of a program.
spanning tree	A spanning tree of a graph G is a subgraph G' of G such that G' is a tree and G' contains all the nodes of G .
stability stack	A part of the AuDaLa state which saves whether (nested) fixpoints are stable. See Definition 2.4.7.

Notation	Description
stable	An AuDaLa fixpoint is considered stable iff during the last iteration of the contents of the fixpoint, no additional struct instances have been created nor have any parameters been changed. In AuDaLogic, a schedule is considered stable iff it will not create any struct instances nor change any parameters during execution. If the schedule in a fixpoint is stable, then after the next iteration, the fixpoint will be stable.
step	A simple AuDaLa function defined for one or more structs and called in the schedule.
struct	An AuDaLa struct type definition. These can be considered data type definitions.
struct environment	A part of the semantic AuDaLa state which contains all struct instances that exist during that state. Accessing struct instances is done via labels. See Definition 2.4.7.
struct instance	An instantiation of an AuDaLa struct during runtime. These can be considered individual data elements. The semantics of struct instances are defined in Definition 2.4.6.
stutter trace equivalence	An equivalence between two semantics where two paths are equivalent if they go through states with comparable atomic propositions in the same order, but not necessarily at the same time. See also Section 5.3.
termination	A piece of code terminates iff at some point it finishes execution.
trace	A sequence of atomic propositions as derived from semantic states. Sequences of the output of the $\mathcal{C}$ -labeling function (Chapter 6) and of configurations (Section 5.3) are traces defined over AuDaLa and Turing machine executions.
type error	An error which occurs when there is a mismatch between the expected type of an expression or variable and the actual type presented in the code.
unbound variables	Variables not bound in the scope of a predicate. In our inference rules, unbound variables are bound globally. Also called <i>free</i> variables.

Notation	Description
weak-memory semantics	A semantics for AuDaLa which allows for the reordering of the execution of commands, where commands may now be executed the moment all of their dependencies have been resolved. Additionally, the semantics allows for incoherence between the variable values of struct instances.
well-formed	An AuDaLa program is well-formed if it satisfies the type system. See Definition 2.3.2
well-formed modulo <i>aFix</i>	A schedule is well-formed modulo <i>aFix</i> iff the schedule with all occurrences of <i>aFix</i> replaced by <i>Fix</i> is well-formed. See Definition 4.2.13.
work	The amount of operations performed when executing a parallel program.

AuDaLa Notations

Notation	Description
C	Used to denote an arbitrary list of commands.
$D_{\mathcal{P}}$	Used to denote the definitions of $\mathcal{P}$, which is the part of the program $\mathcal{P}$ where struct types and steps are defined.
E	Used to denote an arbitrary expression.
F	Used to denote an arbitrary step name.
K	Set of keywords. Used in the type system of Section 2.3.
S	Used to denote an arbitrary statement.
T	Used to denote an arbitrary syntactic type generally.
W	Used to refer to a random statement or expression in Section 6.3.
γ	The command list of a struct instance, holding the commands that still need to be executed by that struct instance. See Definition 2.4.6.
ξ	A variable environment of a struct instance, which saves the values owned by that struct instance. See Definition 2.4.6.
EX	Used to denote the set of all AuDaLa expressions.
Γ	Syntax variable environment, which saves the existing variable names and their types. Used in the type system of Section 2.3.
ID	Used to denote the set of all AuDaLa identifiers.
$\mathcal{L}$	Used to denote a sufficiently large set containing all labels used in the context of an AuDaLa program. Has a notable subset $\mathcal{L}_0$, which is the set of all null-labels in the context of an AuDaLa program.
LT	Used to denote the set of all AuDaLa literals.
Ω	Struct type environment, which saves the declared structs and their parameters. Used in the type system of Section 2.3.

Notation	Description
$Pars_{\mathcal{P}}(\theta),$ $Pars_{\mathcal{P}}(\ell)$	A function which returns the parameters declared for a struct type θ or for the struct type of the struct instance of a label ℓ , including the types of these parameters. We consider $x \in Pars_{\mathcal{P}}(\theta)$ to be true if x is a parameter declared for θ , which we straightforwardly extend to $x \in Pars_{\mathcal{P}}(\ell)$.
$\mathbb{P}$	Used to denote the set of all AuDaLa programs.
Δ	Used to denote an arbitrary schedule.
SC	Used to denote the set of all AuDaLa schedules.
$\mathcal{O}$	Used to denote the set of all AuDaLa semantic binary operators. Defined as $\mathcal{O} = \{=, \neq, \leq, \geq, <, >, *, /, \%, +, -, \wedge, \vee\}$ for this thesis.
$\mathbb{T}$	Used to denote the set of semantic types, defined as $\mathbb{T} = \{\mathcal{L}, \mathbb{N}, \mathbb{Z}, \mathbb{B}, String\}$.
$s\chi$	Used to denote a stability stack of an AuDaLa state.
χ	The value stack of a struct instance, holding the values computed during an execution by that struct instance. See Definition 2.4.6.
ST	Used to denote the set of all AuDaLa statements.
θ	Used to denote an arbitrary struct type.
$\Theta, \Theta_{\mathcal{P}}$	Used to denote a set of struct types. The annotation $\mathcal{P}$ denotes that these are the struct types as defined in $\mathcal{P}$.
σ	Used to denote a struct environment of an AuDaLa state.
$\mathbb{S}$	The set of all possible struct instances.
O	Used to denote the set of all AuDaLa syntactic binary operators. Defined as $O = \{<=, >=, <, >, =, !=, *, /, \%, +, -, \wedge, \&\&, \}$ for this thesis.
$\mathcal{T}$	Used to denote the set of all AuDaLa syntactic types. Defined as $\mathcal{T} = \{\text{Nat}, \text{Int}, \text{Bool}, \text{String}\} \cup ID$.
$\mathcal{V}$	Used to denote the set of semantic values, defined as $\mathcal{V} = \mathcal{L} \sqcup \mathbb{Z} \sqcup \mathbb{B} \sqcup String$ (where $\sqcup$ denotes the disjoint union).
$\circ, o$	Used to denote the a syntactic binary operator and a semantic binary operator respectively.

Notation	Description
$defaultVal(T)$	Used to denote the default value of a type T , as defined in Section 2.4.
$\diamond$	Used to denote well-formedness of the environments in the type rules given in Section 2.3.
ℓ	Used to denote an arbitrary label.
$\llbracket \cdot \rrbracket$	Notation for the application of the interpretation function of AuDaLa, as defined in Definition 2.4.5.
$\mathcal{P}$	Used to denote an arbitrary AuDaLa program.
S	Used to denote an arbitrary list of statements.
$aFix$	A purely semantic schedule element which indicates an initiated fixpoint in the semantics of AuDaLa. See page 43.
sc	Used to denote arbitrary AuDaLa schedule elements where required for clarity. Schedule elements designated with sc cannot be of the form $\Delta_1 < \Delta_2$.
$val(g)$	Used to denote the semantic value of a syntactic literal $g \in LT$.
π	Used to denote arbitrary AuDaLa executions.
ϱ	Used to denote arbitrary AuDaLa paths.
$\Rightarrow_{\mathcal{P}}$	The transition relation of the semantics of AuDaLa. Dependent on $\mathcal{P}$ because it uses struct type and parameter information from $\mathcal{P}$.
ε	Signifies the empty element. Often used for stacks and lists. For example, an empty stack is denoted ε , and we use ε in lists to denote empty list elements, most notably the one at the end, which is used to signify that the list is empty.
$f[a \mapsto b]$	A notation we use to denote a function update. Here, with $f' = f[a \mapsto b]$, it holds that $f'(x) = f(x)$ iff $x \neq a$ and $f'(a) = b$. We extend this notation for multiupdates of the form $f[\{a_1 \mapsto b_1, a_2 \mapsto b_2, \dots\}]$, which are equivalent to $f[a_1 \mapsto b_1][a_2 \mapsto b_2] \dots$ and require the set $a_1, \dots, a_n$ to be pairwise distinct. Furthermore, we extend this notation for index updates $f[a, i \mapsto b]$ such that if $f(a) = \langle b_1, \dots, b_n \rangle$, then $f[a, i \mapsto b](a) = \langle b_1, \dots, b_{i-1}, b, b_{i+1}, \dots, b_n \rangle$.

Notation	Description
g	Used to denote an arbitrary literal.
p	Used to denote arbitrary AuDaLa states.
$p_{\mathcal{P}}^0$	Used to denote the initial state of an AuDaLa program $\mathcal{P}$.
s	Used to denote arbitrary AuDaLa struct instances.
v	Used to denote an arbitrary value.
x	Used to denote an arbitrary variable.

Commands

Notation	Description
if (·)	An <i>if</i> -command. The argument represents the commands that will be executed if the top of the struct instance stack holds the value <i>true</i> . See Definition 2.4.4.
not	A <i>not</i> -command. See Definition 2.4.4.
op (·)	An <i>operator</i> -command. The argument represents the operator that is applied. See Definition 2.4.4.
cons (·)	A <i>constructor</i> -command. The argument represents the struct type that is created. See Definition 2.4.4.
push (·)	A <i>push</i> -command. The argument represents the value that is pushed to the stack of a struct instance. See Definition 2.4.4.
push (this)	A <i>push</i> -command with the special argument this , representing the currently executing struct instance. See Definition 2.4.4.
rd (·)	A <i>read</i> -command. The argument represents the variable that is read from. See Definition 2.4.4.
wrP (·)	A <i>writeP</i> -command, used in the weak memory semantics, as given in Section 6.2. The argument represents the variable that is written to.
wr (·)	A <i>write</i> -command. The argument represents the variable that is written to. See Definition 2.4.4.

Notations used for AuDaLa Adaptations

Notation	Description
Z_p	The contents of the state p , consisting of the contents of all its struct instances, as used in the stutter trace equivalence proof of the standard semantics and the weak semantics. See Definition 6.3.18.
ξ	An environment that saves the variables read by the current struct instance, with their values.
χ	A list of commands and values. During execution, the commands and the values required for those commands are processed and replaced with other values.
ι	The initial value function in the state of the weak memory semantics. Saves the initial values when executing a step.
$\mathcal{C}(p)$	The label of state p , as used in the stutter trace equivalence proof of the standard semantics and the weak semantics. See Definition 6.3.19.
Con_ℓ	The contents of the struct instance with label ℓ , consisting of the label, struct type and parameter/value function of ℓ , as used in the stutter trace equivalence proof of the standard semantics and the weak semantics. See Definition 6.3.18.
μ	The initial value function in the state of the weak memory semantics. Saves the initial values when executing a step.
z_ℓ	The parameter/value function of the struct instance with label ℓ , as used in the stutter trace equivalence proof of the standard semantics and the weak semantics. See Definition 6.3.18.

Turing machine Notations

Notation	Description
A	Used to denote the set of accepting states of a Turing machine.
B	Used to denote the blank symbol of a Turing machine.
M	Used to denote an arbitrary Turing machine.
Q	Used to denote the set of control states of a Turing machine.
Λ	Used to denote the set tape symbols of a Turing machine.
Σ	Used to denote the set of input symbols of a Turing machine.
δ	Used to denote the transition function of a Turing machine.
$\mathcal{I}$	Used to denote an arbitrary input string of a Turing machine.
ς	Used to denote an arbitrary tape symbol.
q_0	Used to denote the initial state of a Turing machine.

B.2 Part II - AuDaLogic

AuDaLogic Terms

Notation	Description
safeStStr_P	Used to denote the strict safety predicate for statements. See Definition 8.2.12.
(Hoare) triple	A predicate $\{P\}\mathcal{X}\{Q\}$, which denotes that if we execute some executable $\mathcal{X}$ from a state satisfying precondition P , we end up in a state satisfying Q . If we are talking about Hoare triples in the context of AuDaLogic, we call them AuDaLogic triples. The semantics of AuDaLogic triples is defined in Definition 8.1.9.
(concurrent) separation logic	See Section 7.1. An extension of Hoare logic to reason about (concurrent) programs by using a memory heap.
AuDaLogic	The proof system for AuDaLa we create in Part II of the thesis. It stands for ‘Autonomous Data Logic’.
abstract	A variable introduced in the scope of an AuDaLogic which represents some explicit value. Often used when the exact value is not relevant, but its origin is.
annotation	To reason about the correspondence between AuDaLa executions and AuDaLogic, we annotate AuDaLa command transitions with the execution label and some other information depending on the transition. See page 215.
axioms	An axiom is a basic assumption which is assumed to always hold.
concurrency rule (CSL)	The extension of the frame rule where memory is split between two separate processes, which allows one to consider the processes apart from each other in CSL.
conforms	An AuDaLa execution π conforms to an AuDaLogic state (α, h, ς) if τ respects (α, h, ς) and π starts from an AuDaLa state p from which (α, h, ς) can be derived. See Definition 8.1.2.

Notation	Description
constructor normal form	An AuDaLa program is in constructor normal form when constructor expressions only occur in special variable assignment statements. See also Section 7.3.1.
correctly maintained	A rewrite function used for proving the stability predicate is correctly maintained for an AuDaLogic state (α, h, ζ) with only one active struct instance represented by a_ϵ , if for any input x rewritten to expression E , $(\alpha, h, \zeta) \models A(a_\epsilon, E)$ holds.
correctly maintained	A rewrite function used for proving the stability predicate is correctly maintained for an AuDaLa state p if for any input x rewritten to expression E , the result values of resolving x and E from p are the same.
corresponding deterministic heap	In Chapter 9, the corresponding deterministic heap of an AuDaLa state p and a heap h is the deterministic variant h' of h such that h can be strictly derived from P . Also used for AuDaLa executions, where we take p to be the starting state of the execution. See Definition 9.1.7.
derived	An memory heap h is derived from p iff it is weaker than the heap h' memory-equivalent to p according to the heap ordering $\sqsubseteq$. This essentially means that h' can be extracted from p . See also Definition 8.1.1. In Chapter 9, this relation is weakened to allow for non-deterministic heaps. The original variant is renamed to strict derivation.
deterministic heap	In Chapter 9, we consider a heap <i>deterministic</i> when it contains exactly one value per memory cell, and non-deterministic when it contains at least one cell with multiple values.
equivalent	An AuDaLa state p is equivalent to an AuDaLogic state (α, h, ζ) iff p is struct-equivalent to α and memory-equivalent to h . See Definition 8.1.1.
execution label	For any command transition, the execution label is the label of the struct instance which executes the transition. In the operational rules for commands, this is the label of the struct instance which contains the command which causes the transition to be enabled.

Notation	Description
expression correspon- dence re- quirement	The requirement that when replacing AuDaLa variable expressions with AuDaLogic variables, the mapping from AuDaLa variables to AuDaLogic variables retains the structure of the references of the AuDaLa variable expressions. This is encoded in predicates generated by the expression correspondence requirement function A . See also Section 7.3.2.
frame rule (separation logic)	The rule which allows separation logic to discard parts of the memory context not relevant for the currently executed code, which simplifies reasoning about programs which use pointers.
fresh	We consider a variable to be fresh if it is not presently already in use. In the case of new labels this means the label is not yet in use, while in the case of the fixpoint rule in Section 7.4.3 this means that it is not already in use in any of the predicates related to that rule.
heap deter- minism	We consider a memory heap h deterministic if every entry of the heap can contain at most one value. This is used in Chapter 9, where we adapt heaps to be able to contain more than one value for every entry.
heap-free predicate	A predicate which does not contain any assertions on the memory heap. It can therefore be considered pure with regards to the memory heap. See also Definition 7.2.13.
Hoare logic	See Section 7.1. A formal system for reasoning about the correctness of computer programs.
label triple	The part of the AuDaLogic state that keeps track of the relevant struct instances. It contains three sets: the first contains the labels of the active struct instances, the second contains the labels of the newly made struct instances and the third contains the labels that are available to be used for new struct instances. See also Definition 7.2.1.
memory as- sertion	A memory assertion. An integral part of separation logic and AuDaLogic. See Sections 7.1 and 7.2. The memory assertion $x \mapsto a$ states that our memory consists of a single memory cell (or address) x , containing value a . Distinct from the function update notation.

Notation	Description
memory heap	A representation of the memory accessible by a processor, modeled as a heap which can be split and combined. We represent the heap by a partial function, which takes a variable and returns the value, if the function is defined for the variable.
memory inclusion	A memory inclusion, a weakened form of a memory assertion which only asserts (in the case $x \mapsto a$) that the memory should <i>contain</i> a memory cell x which contains a . Logically equivalent to $(x \mapsto a) * \text{true}$.
memory-equivalent	An AuDaLa state p and a heap h are memory-equivalent iff all defined inputs in h are reflected in p and all parameters in p are reflected in h , with the correct values. See also Definition 8.1.1. In Chapter 9, this relation is weakened to allow for non-deterministic heaps. The original variant is renamed strict memory-equivalence.
null-heap	The <i>default</i> memory heap, which stores the values of parameters of null-instances of a program $\mathcal{P}$. As these parameters cannot be written to, the null-heap can safely be shared between all active struct instances, which is expressed by the null-permission ζ_0 . See also Definition 7.2.7.
permissions	An extension of CSL which allows for memory cells to be present in both sides of a split, but only to be read. This allows for read-read race conditions, which makes proving concurrent programs with CSL easier.
precision	In conventional separation logic, a predicate is <i>precise</i> if it is true for only one exact definition of the memory heap. Not used in AuDaLogic.
premise AuDaLogic state	During the soundness proof of a proof rule, a premise AuDaLogic state is an AuDaLogic state for which all premises hold.
proof rule	A proof rule specifies a number of premises and a conclusion, and asserts that if the premises hold, the conclusion also holds.
proof system	The combination of a logic, inference rules and axioms used to prove properties about statements and formulas. Also called a formal system.

Notation	Description
pure	A predicate is <i>pure</i> in AuDaLogic iff it is free from assertions on the memory heap and on the label triple.
resource invariant	In conventional separation logic, a resource invariant is a guarantee on the effects and properties of a resource accessed by a process. Not used in AuDaLogic.
respects	A sequence of command transitions τ respects an AuDaLogic state (α, h, ζ) iff all execution labels of τ are contained in $\alpha(1)$ and all newly introduced labels are contained in $\alpha(3)$. See Definition 8.1.2.
schedule predicate	The safety predicate for schedules, as defined in Definition 8.1.8, used to define the semantic meaning of AuDaLogic triples. For AuDaLogic with write-write race conditions, see Definition 9.1.10.
semi-semantic expression	An intermediate form between an AuDaLa and an AuDaLogic expression. AuDaLa expressions are transformed into semi-semantic expressions before fully being converted to AuDaLogic expressions. See also Section 7.3.2.
simple expression	Expressions in which no constructor expression occurs (either as a subexpression or as the expression itself).
soundness	A proof system is sound iff whatever can be proved in the proof system is actually true. For AuDaLogic, this means that AuDaLogic is sound iff any triple $\{P\}\mathcal{X}\{Q\}$ we can prove using the proof rules of AuDaLogic reflects the semantics and executions of AuDaLa correctly, which is the case iff every AuDaLa execution of the executable $\mathcal{X}$ from an AuDaLa state which satisfies P leads to an AuDaLa state that satisfies Q . See Definition 8.3.15.
statements predicate	The safety predicate for lists of statements, as defined in Definition 8.1.6, used to define the semantic meaning of AuDaLogic triples.
step sequence	The step sequence of an execution π is the possibly infinite ordered list $\mathcal{F}_\pi$ consisting of the steps executed during π and by which struct instances the step was executed. See Definition 8.1.4.

Notation	Description
strict safety predicate for statements	A stronger version of the safety predicate for schedules, which also requires that for any intermediate AuDaLogic state chosen for requirement 2 of the original safety predicate for schedules, the parameters are unchanged and no additional parameters have been made. Used to prove the stability predicate correct. See Definition 8.2.12.
struct-equivalent	An AuDaLa state p and a label triple α are struct-equivalent if for all labels in $\alpha(1)$ a struct instance is defined in p , for all labels in $\alpha(2)$ there is a struct instance defined in p with empty commands and stack and there is no struct instance defined in p for any label in $\alpha(3)$. See also Definition 8.1.1.
transition safety	A transition is safe iff it satisfies the requirements in Definition 8.1.5. When considering AuDaLogic with write-write race conditions in Chapter 9, a transition needs to satisfy the requirements in Definition 9.1.9 instead.
triple-free predicate	A predicate which does not contain any assertions on the label triple. It can therefore be considered pure with regards to the label triple. See also Definition 7.2.13.
variant	A predicate which encodes what changes. Opposite of an invariant. In AuDaLogic, variants are used to prove fixpoints totally correct.
variant generator function	A function which takes as input the current iteration of a fixpoint and which outputs a variant predicate corresponding to the end of that iteration. The predicate $\forall a(0)$ should hold before the execution of a fixpoint.

AuDaLogic Notations

Notation	Description
(α, h, ζ)	An AuDaLogic state. See Definition 7.2.1.
$(l, x) \xrightarrow{\zeta} v$	An example of an AuDaLogic memory assertion. The memory inclusion corresponding to $(l, x) \xrightarrow{\zeta} v * \text{true}$ would be $(l, x) \xrightarrow{\zeta} v$. See also Section 7.2.
$*$	The separating conjunction, or separator. An integral part of separation logic and AuDaLogic. See Sections 7.1 and 7.2. Used for separating resources between active elements in AuDaLogic.
$A(a_\epsilon, E)$	An application of the expression correspondence requirement function A , as defined in Definition 7.3.22, from the perspective of a struct instance referenced by label variable a_ϵ which is executing expression E .
$D(P, P')$	The deterministic weakening relation, where $D(P, P')$ iff $P' \Rightarrow \mathbf{det}$ and for all (α, h, ζ) such that $(\alpha, h, \zeta) \models P$, for all h' such that $h \gg h'$, it holds that $(\alpha, h', \zeta) \models P'$. Defined in Definition 9.1.4.
R, W, X	The notations used in Chapter 9 as the categories of permissions, such that a permission (ζ, R) is a read-permission, (ζ, W) is a write-permission and (ζ, X) is a read or a write permission.
$S(P, P')$	The activation relation, where $S(P, P')$ iff $P' \Rightarrow \emptyset_2$ and $(\alpha, h, \zeta) \models P$ iff $(m(\alpha), h, \zeta) \models P'$. Syntactically, P' can be created from P by changing all occurrences of $\blacksquare$ and $\blacksquare_2$ into $\blacksquare_1$ and all occurrences of $\boxplus$ and $\boxplus_2$ to $\boxplus_1$. Defined in Definition 7.2.18.
Z	The set of all AuDaLogic permissions, defined as $(0, 1] \cup \{\zeta_0\}$.
EX_S	The set of simple expressions. Simple expressions do not contain constructor statements.
$\mathcal{L}$	The set of all AuDaLogic label variables.

Notation	Description
$\mathfrak{E}$	The set of all AuDaLogic expressions. Has the notable subsets $\mathfrak{E}_E$, $\mathfrak{E}_M$, $\mathfrak{E}_B$, $\mathfrak{E}_L$, $\mathfrak{E}_T$, $\mathfrak{E}_Z$ and $\mathfrak{E}_X$, which collect the AuDaLogic execution, math, boolean, label, struct type, permission and variable expressions respectively. See also Definition 7.2.10.
$\star$	The merge operator for states. See Definition 7.2.5.
$\mathfrak{A}$	The set of all AuDaLogic variables.
$\Xi(h)$	The set of AuDaLa states p such that $p > h$. See Definition 8.1.1.
$\blacksquare_i, \square_i$	Symbols used as struct type quantifiers for the label triple. with $\blacksquare_i$ being comparable to forall and $\square_i$ being comparable to exists. If $i \in \{1, 2\}$, the predicate $\blacksquare_i \theta$ states that the set $\alpha(i)$ contains only active labels of which the struct instance has type θ , while the predicate $\square_i \theta$ states that the set $\alpha(i)$ contains at least one active label of which the struct instance has type θ .
$def(f(x)),$ $def(a \circ b),$ $def(E)$	Denotes whether function f is defined for input a , whether binary operator $\circ$ is defined for a and b and whether AuDaLogic expression $E \in \mathfrak{E}$ is defined. Here, $def(E)$ is defined iff for all subexpressions $E_1 \circ E_2$ in E , $def(E_1 \circ E_2)$.
$dom(f)$	The domain of a function f .
emp	A separation logic and AuDaLogic constant denoting the empty memory heap.
$\blacksquare_i(l)$	An AuDaLogic label triple assertion which asserts that the i th set of the label triple (with $i \in \{1, 2\}$) consists only of the label represented by the label variable l . This can be extended to both sets through the notation $\blacksquare(l)$.
$\mathcal{X}$	Used to denote an arbitrary executable, which a schedule or a list of statements.
$fresh(l)$	An AuDaLogic predicate which asserts that the label represented by the label variable l is in the third set of the label triple.
$\gg$	the deterministic relation $\gg$ as defined in Definition 9.1.1. Relates a non-deterministic heap to potential deterministic variants of that heap.

Notation	Description
ς	An AuDaLogic variable environment. See Definition 7.2.1.
α	An AuDaLogic label triple. See Definition 7.2.1.
$\emptyset_i$	An AuDaLogic constant, with $i \in \{1,2\}$, denoting that the i th set in the label triple is empty. If both the first and the second set must be empty, then the notation $\emptyset$ can be used.
det	The predicate which asserts that the memory heap of the current state is deterministic.
$\mathcal{F}$	Used to denote a step sequence. See Definition 8.1.4.
$ConsElim, ConsElimE$	The functions used to transform an AuDaLa program to constructor normal form. See also Section 7.3.1.
Va	Used to denote variant generator functions.
$Va(i)$	A variant predicate as used for fixpoints. Here, i is a number depicting the iteration of the fixpoint.
$eqi(a_\epsilon, E)$	The predicate equivalent to executing E from the struct instance referenced through the AuDaLogic label variable a_ϵ . See Definition 7.3.23.
fv	A function which takes one or more AuDaLogic predicates and returns the set of unbound variables in those predicates.
$occ(E_1, \dots, E_n)$	An application of the occurrence function on the AuDaLa expressions $E_1, \dots, E_n$, which returns all occurrences of maximal variable expressions in $E_1, \dots, E_n$. For example, for an AuDaLa expression $x.y.z + z.y$, the occurrence function returns the set $\{x.y.z, z.y\}$.
$prefixes(E_V)$	An application of the prefixes function on the AuDaLa variable expression E , which returns all non-empty prefixes of E . For example, for an AuDaLa variable expression $x.z$, the prefixes function returns the set $\{x, x.z\}$.
rew	The rewrite function used in the stability predicate generator functions. It rewrites AuDaLa expressions to unfold local variables back into parameter variables. See also Section 7.3.3.
$\diamond$	The merge operator for label triples. See Definition 7.2.3.

Notation	Description
$\oplus$	The merge operator for heaps. See Definition 7.2.4.
$\sqsubseteq$	A heap ordering. For two heaps h_1 and h_2 , $h_1 \sqsubseteq h_2$ iff h_2 contains all entries of h_1 with the same values assigned to them and with permissions equal or larger than in h_1 . See also Definition 7.2.8.
$\pi \rightsquigarrow (\alpha, h, \varsigma)$	Used to denote that AuDaLa execution π conforms to AuDaLogic state (α, h, ς) . See Definition 8.1.2.
$PrSp$	The stability function for predicates, which, given a statement, a label variable which denotes the struct instance the statement is executed from, a struct type and a rewrite function, returns a predicate that is true iff the execution of the statement from the struct instance is stable. See also Section 7.3.3.
res	The resolution function for AuDaLogic expressions. See Definition 7.2.11.
$safeScp$	Used to denote the safety predicate for schedules, as defined in Definition 8.1.8, used to define the semantic meaning of AuDaLogic triples. For AuDaLogic with write-write race conditions, see Definition 9.1.10.
$safeStp$	Used to denote the safety predicate for lists of statements, as defined in Definition 8.1.6, used to define the semantic meaning of AuDaLogic triples.
$-*$	The separating implication. An integral part of separation logic and AuDaLogic. See Sections 7.1 and 7.2.
$stablep$	The stability predicate generator function. Given a schedule, it generates a predicate that checks whether the next execution of the schedule is stable. See also Section 7.3.3.
$stableFp$	The stability predicate generator function for steps. Given a step, it generates a predicate that checks whether the next execution of the step is stable. See also Section 7.3.3.
$\tau = \tau_1 \parallel \tau_2$	Used to denote that sequence of AuDaLa transitions τ is an interleaving of τ_1 and τ_2 . τ is defined iff τ_1 and τ_2 respect mergeable states. This can scale to multiple sequences of AuDaLa transitions. See Definition 8.1.3.

Notation	Description
$\tau \sim (\alpha, h, \zeta)$	Used to denote that the sequence of AuDaLa transitions τ respects AuDaLogic state (α, h, ζ) . See Definition 8.1.2.
∇	A sign used to name proof trees and to refer to other proof trees in Section 7.5.
ζ	A permission
ζ_0	The null-permission, a special permission for parameters of null-instances.
$a : T$	This notation asserts that AuDaLogic variable a refers to a value of type T .
h	An AuDaLogic memory heap. See Definition 7.2.1.
h_0	The null-heap. See Definition 7.2.7.
$m(\alpha)$	The maximally active label triple corresponding to α : the label triple α' such that $\alpha(1) \cup \alpha(2) = \alpha'(1)$ and $\alpha'(2) = \emptyset$. Defined in Definition 7.2.17.
$p \in \Xi(\alpha, h, \zeta)$	Equivalent to $p \in \Xi(h)$. See Definition 8.1.1.
$p \Leftrightarrow (\alpha, h, \zeta)$	Used to denote that p is equivalent to (α, h, ζ) . See Definition 8.1.1.
$p \Leftrightarrow \alpha$	Used to denote that AuDaLa state p is struct-equivalent to label triple α . See Definition 8.1.1.
$p \Leftrightarrow h$	Used to denote that AuDaLa state p is memory-equivalent to label triple h . See Definition 8.1.1. In Chapter 9, this notation denotes that p is weakly memory-equivalent to h , with $\Leftrightarrow_s$ denoting strict memory-equivalence, which is the same as the definition of memory-equivalence in Chapter 8.
$p > (\alpha, h, \zeta)$	Equivalent to $p \Leftrightarrow \alpha$ and $p > h$. See Definition 8.1.1.
$p > h$	Used to denote that h can be derived from AuDaLa state p . See Definition 8.1.1. In Chapter 9, this notation denotes that h is weakly derived from p , with $>_s$ denoting strict derivation, which is the same as the definition of derivation in Chapter 8.

Bibliography

- [1] M. Ajtai, J. Komlós, and E. Szemerédi. “An $O(n \log n)$ sorting network”. In: *Proceedings of the fifteenth annual ACM symposium on Theory of computing*. STOC '83. New York, NY, USA: Association for Computing Machinery, 1983, pp. 1–9. ISBN: 978-0-89791-099-6. DOI: 10.1145/800061.808726.
- [2] A. Alexandru and G. Ciobanu. “Fraenkel-Mostowski Set Theory: A Framework for Finitely Supported Mathematics”. In: *Finitely Supported Mathematics: An Introduction*. Cham: Springer International Publishing, 2016, pp. 11–47. ISBN: 978-3-319-42282-4. DOI: 10.1007/978-3-319-42282-4_2.
- [3] J. Alglave, L. Maranget, and M. Tautschnig. “Herding cats: Modelling, simulation, testing, and data mining for weak memory”. In: *ACM Transactions on Programming Languages and Systems* 36.2 (2014). Publisher: ACM, pp. 1–74. DOI: 10.1145/2627752.
- [4] P. America and F. van der Linden. “A parallel object-oriented language with inheritance and subtyping”. In: *SIGPLAN Not.* 25.10 (1990), pp. 161–168. DOI: 10.1145/97946.97966.
- [5] K. R. Apt, F. S. De Boer, and E.-R. Olderog. *Verification of Sequential and Concurrent Programs*. Ed. by D. Gries and F. B. Schneider. Texts in Computer Science. Springer, 2009. ISBN: 978-1-84882-745-5. DOI: 10.1007/978-1-84882-745-5.
- [6] K. R. Apt and E.-R. Olderog. “Assessing the Success and Impact of Hoare’s Logic”. In: *Theories of Programming: The Life and Works of Tony Hoare*. Association for Computing Machinery, 2021, pp. 41–76. ISBN: 978-1-4503-8728-6. DOI: 10.1145/3477355.3477359.

- [7] K. R. Apt and E.-R. Olderog. “Fifty years of Hoare’s logic”. In: *Form. Asp. Comput.* 31.6 (Dec. 2019), pp. 751–807. ISSN: 0934-5043. DOI: 10.1007/s00165-019-00501-3.
- [8] F. Arbab, P. Ciancarini, and C. Hankin. “Coordination languages for parallel programming”. In: *Parallel Computing* 24.7 (1998), pp. 989–1004. DOI: 10.1016/S0167-8191(98)00039-8.
- [9] F. Arbab. “Reo: a channel-based coordination model for component composition”. In: *Mathematical Structures in Computer Science* 14.3 (2004), pp. 329–366. DOI: 10.1017/S0960129504004153.
- [10] T. Baba and T. Yoshinaga. “A-NETL: a language for massively parallel object-oriented computing”. In: *PMMPC Proc.* IEEE, 1995, pp. 98–105. DOI: 10.1109/PMMP.1995.504346.
- [11] S. W. A.-H. Baddar and K. E. Batchier. *Designing Sorting Networks: A New Paradigm*. Springer Publishing Company, Incorporated, 2011. ISBN: 978-1-4614-1850-4.
- [12] C. Baier and J.-P. Katoen. *Principles of model checking*. MIT press, 2008.
- [13] R. Baldoni, E. Coppa, D. C. D’elia, C. Demetrescu, and I. Finocchi. “A Survey of Symbolic Execution Techniques”. In: *ACM Computing Surveys* (2018), 50:1–50:39. ISSN: 0360-0300. DOI: 10.1145/3182657.
- [14] J.-P. Banâtre, A. Coutant, and D. Le Metayer. “A parallel machine for multiset transformation and its programming style”. In: *Future Generation Computer Systems* 4.2 (1988), pp. 133–144. DOI: 10.1016/0167-739X(88)90012-X.
- [15] J.-P. Banâtre, P. Fradet, and D. Le Métayer. “Gamma and the Chemical Reaction Model: Fifteen Years After”. In: *Multiset Processing*. Springer, 2001, pp. 17–44. ISBN: 978-3-540-45523-3. DOI: 10.1007/3-540-45523-X_2.
- [16] J.-P. Banâtre and D. Le Métayer. “The gamma model and its discipline of programming”. In: *Science of Computer Programming* 15.1 (1990), pp. 55–77. DOI: 10.1016/0167-6423(90)90044-E.
- [17] M. Bauer, S. Treichler, E. Slaughter, and A. Aiken. “Legion: Expressing locality and independence with logical regions”. In: *SC ’12*. 2012, pp. 1–11. DOI: 10.1109/SC.2012.71.
- [18] J. Berdine, C. Calcagno, and P. W. O’Hearn. “Symbolic Execution with Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2005, pp. 52–68. ISBN: 978-3-540-32247-4. DOI: 10.1007/11575467_5.

- [19] G. Berry and G. Boudol. “The chemical abstract machine”. In: *Theor. Comput. Sci.* 96.1 (Apr. 1992), pp. 217–248. DOI: 10.1016/0304-3975(92)90185-I.
- [20] G. E. Bluelloch and B. M. Maggs. “Parallel algorithms”. In: *ACM Comput. Surv.* 28.1 (1996), pp. 51–54. ISSN: 0360-0300. DOI: 10.1145/234313.234339.
- [21] S. Blom, S. Darabi, M. Huisman, and W. Oortwijn. “The VerCors Tool Set: Verification of Parallel and Concurrent Software”. In: *Integrated Formal Methods*. Cham: Springer International Publishing, 2017, pp. 102–110. ISBN: 978-3-319-66845-1. DOI: 10.1007/978-3-319-66845-1_7.
- [22] H.-J. Boehm. “How to miscompile programs with "benign" data races”. In: *USENIX 2011 Proc.* USENIX Association, 2011.
- [23] F. de Boer et al. “A Survey of Active Object Languages”. In: *ACM Comput. Surv.* 50.5 (2017), 76:1–76:39. DOI: 10.1145/3122848.
- [24] F. S. de Boer et al. “Decidability Problems for Actor Systems”. In: *CONCUR 2012 – Concurrency Theory*. Vol. 10. Logical Methods in Computer Science. Springer, 2012, pp. 562–577. DOI: 10.1007/978-3-642-32940-1_39.
- [25] F. S. de Boer, D. Clarke, and E. B. Johnsen. “A Complete Guide to the Future”. In: *Programming Languages and Systems*. Vol. 4421. LNCS. Springer, 2007, pp. 316–330. DOI: 10.1007/978-3-540-71316-6_22.
- [26] F. S. d. Boer, H.-D. A. Hiep, and S. d. Gouw. “Dynamic Separation Logic”. In: *Electronic Notes in Theoretical Informatics and Computer Science* Volume 3 - Proceedings of MFPS XXXIX (2023). ISSN: 2969-2431. DOI: 10.46298/entics.12297.
- [27] M. Bohr. “A 30 Year Retrospective on Dennard’s MOSFET Scaling Paper”. In: *IEEE Solid-State Circuits Society Newsletter* 12.1 (2007), pp. 11–13. ISSN: 1098-4232. DOI: 10.1109/N-SSC.2007.4785534.
- [28] R. Bornat. “Proving Pointer Programs in Hoare Logic”. In: *Mathematics of Program Construction*. Ed. by R. Backhouse and J. N. Oliveira. Berlin, Heidelberg: Springer, 2000, pp. 102–126. ISBN: 978-3-540-45025-2. DOI: 10.1007/10722010_8.
- [29] S. Brookes. “A Revisionist History of Concurrent Separation Logic”. In: *Electronic Notes in Theoretical Computer Science*. Twenty-seventh Conference on the Mathematical Foundations of Programming Semantics (MFPS XXVII) 276 (2011), pp. 5–28. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2011.09.013.

- [30] S. Brookes. “A Semantics for Concurrent Separation Logic”. In: *CONCUR 2004 - Concurrency Theory*. Berlin, Heidelberg: Springer, 2004, pp. 16–34. ISBN: 978-3-540-28644-8. DOI: 10.1007/978-3-540-28644-8_2.
- [31] S. Brookes. “A semantics for concurrent separation logic”. In: *Theoretical Computer Science*. Festschrift for John C. Reynolds’s 70th birthday 375.1 (2007), pp. 227–270. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2006.12.034.
- [32] S. Brookes and P. W. O’Hearn. “Concurrent separation logic”. In: *ACM SIGLOG News* 3.3 (2016), pp. 47–65. DOI: 10.1145/2984450.2984457.
- [33] P.-J. Cagnard. “The ParCeL-2 Programming Language”. In: *Euro-Par 2000*. Vol. 1900. LNCS. Springer, 2000, pp. 767–770. DOI: 10.1007/3-540-44520-X_106.
- [34] Q. Cao, S. Cuellar, and A. W. Appel. “Bringing Order to the Separation Logic Jungle”. In: *Programming Languages and Systems*. Cham: Springer International Publishing, 2017, pp. 190–211. ISBN: 978-3-319-71237-6. DOI: 10.1007/978-3-319-71237-6_10.
- [35] L. Cardelli. “Type Systems”. In: *Computer Science Handbook, Second Edition*. Ed. by A. B. Tucker. Accessed through <http://lucacardelli.name/indexPapers.html>. Chapman & Hall/CRC, 2004. Chap. 97. ISBN: 9781584883609.
- [36] L. Cardelli. “Type systems”. In: *ACM Comput. Surv.* 28.1 (1996), pp. 263–264. ISSN: 0360-0300. DOI: 10.1145/234313.234418.
- [37] N. Carriero and D. Gelernter. “How to write parallel programs: a guide to the perplexed”. In: *ACM Comput. Surv.* 21.3 (1989), pp. 323–357. ISSN: 0360-0300. DOI: 10.1145/72551.72553.
- [38] N. Carriero and D. Gelernter. “Linda in context”. In: *Commun. ACM* 32.4 (Apr. 1, 1989), pp. 444–458. ISSN: 0001-0782. DOI: 10.1145/63334.63337.
- [39] B. Chamberlain, D. Callahan, and H. Zima. “Parallel Programmability and the Chapel Language”. In: *The International Journal of High Performance Computing Applications* 21.3 (2007), pp. 291–312. ISSN: 1094-3420. DOI: 10.1177/1094342007078442.
- [40] N. Chong, A. F. Donaldson, and J. Ketema. “A Sound and Complete Abstraction for Reasoning about Parallel Prefix Sums”. In: *SIGPLAN Not.* 49.1 (2014), pp. 397–409. DOI: 10.1145/2578855.2535882.
- [41] F. Ciccozzi et al. “A Comprehensive Exploration of Languages for Parallel Computing”. In: *ACM Comput. Surv.* 55.2 (2022), 24:1–24:39. DOI: 10.1145/3485008.

- [42] L. Clarke, I. Glendinning, and R. Hempel. “The MPI Message Passing Interface Standard”. In: *Programming Environments for Massively Parallel Distributed Systems*. Monte Verità. Birkhäuser, 1994, pp. 213–218. DOI: 10.1007/978-3-0348-8534-8_21.
- [43] R. Cole. “Parallel Merge Sort”. In: *SIAM J. Comput.* 17 (1988), pp. 770–785. DOI: 10.1137/0217049.
- [44] S. A. Cook and P. W. Dymond. “Parallel pointer machines”. In: *computational complexity* 3 (1993), pp. 19–30. ISSN: 1420-8954. DOI: 10.1007/BF01200405.
- [45] B. J. Copeland. “The Church-Turing Thesis”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta and U. Nodelman. Winter 2024. Metaphysics Research Lab, Stanford University, 2024.
- [46] B. Cosenza et al. “OpenABL: A Domain-Specific Language for Parallel and Distributed Agent-Based Simulations”. In: *Euro-Par 2018*. Vol. 11014. LNCS. Springer, 2018, pp. 505–518. DOI: 10.1007/978-3-319-96983-1_36.
- [47] “Cramming more components onto integrated circuits”. In: *Electronics* 38.8 (1965), pp. 114–117.
- [48] P. Czarnul, J. Proficz, and K. Drypczewski. “Survey of Methodologies, Approaches, and Challenges in Parallel Programming Using High-Performance Computing Systems”. In: *Scientific Programming* 2020.1 (2020), p. 4176794. ISSN: 1875-919X. DOI: 10.1155/2020/4176794.
- [49] J. R. Davidson. “An information theoretic approach to the expressiveness of programming languages”. PhD Thesis. University of Glasgow, 2016.
- [50] R. De Nicola, G. Ferrari, and R. Pugliese. “KLAIM: a kernel language for agents interaction and mobility”. In: *IEEE Transactions on Software Engineering* 24.5 (May 1998), pp. 315–330. ISSN: 1939-3520. DOI: 10.1109/32.685256.
- [51] R. Dennard, F. Gaensslen, H.-N. Yu, V. Rideout, E. Bassous, and A. LeBlanc. “Design of ion-implanted MOSFET’s with very small physical dimensions”. In: *IEEE Journal of Solid-State Circuits* 9.5 (1974), pp. 256–268. ISSN: 1558-173X. DOI: 10.1109/JSSC.1974.1050511.
- [52] J. Detrey and O. Diessel. *A Constructive Proof of the Turing Completeness of Circal*. School of Computer Science and Engineering, University of New South Wales, Australia, 2002.
- [53] A. V. Deursen and P. Klint. “Little languages: little maintenance?”. In: *Journal of Software Maintenance: Research and Practice* 10 (1998), pp. 75–92. ISSN: 1096-908X. DOI: 10.1002/(SICI)1096-908X(199803/04)10:2<75::AID-SMR168>3.0.CO;2-5.

- [54] C. Di Giusto. “Expressiveness of Concurrent Languages”. PhD thesis. Alma Mater Studiorum Università di Bologna, 2009.
- [55] J. Diaz, C. Muñoz-Caro, and A. Niño. “A Survey of Parallel Programming Models and Tools in the Multi and Many-Core Era”. In: *IEEE Transactions on Parallel and Distributed Systems* 23.8 (2012), pp. 1369–1386. ISSN: 1558-2183. DOI: 10.1109/TPDS.2011.308.
- [56] H. Esmailzadeh, E. Blem, R. S. Amant, K. Sankaralingam, and D. Burger. “Power challenges may end the multicore era”. In: *Commun. ACM* 56.2 (2013), pp. 93–102. ISSN: 0001-0782. DOI: 10.1145/2408776.2408797.
- [57] W. H. J. Feijen and A. J. M. v. Gasteren. *On a Method of Multiprogramming*. Springer Science & Business Media, 2013. ISBN: 978-1-4757-3126-2.
- [58] M. Felleisen. “On the expressive power of programming languages”. In: *Science of Computer Programming* 17.1 (1991), pp. 35–75. ISSN: 0167-6423. DOI: 10.1016/0167-6423(91)90036-W.
- [59] R. Ferreira, X. Feng, and Z. Shao. “Parameterized Memory Models and Concurrent Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2010, pp. 267–286. ISBN: 978-3-642-11957-6. DOI: 10.1007/978-3-642-11957-6_15.
- [60] R. W. Floyd. “Assigning Meanings to Programs”. In: *Program Verification: Fundamental Issues in Computer Science*. Dordrecht: Springer Netherlands, 1993, pp. 65–81. ISBN: 978-94-011-1793-7. DOI: 10.1007/978-94-011-1793-7_4.
- [61] S. Fortune and J. Wyllie. “Parallelism in random access machines”. In: *Proceedings of the tenth annual ACM symposium on Theory of computing*. STOC ’78. New York, NY, USA: Association for Computing Machinery, 1978, pp. 114–118. ISBN: 978-1-4503-7437-8. DOI: 10.1145/800133.804339.
- [62] T. T. P. Franken and T. Neele. “AuDaLa is Turing Complete”. In: *Formal Techniques for Distributed Objects, Components, and Systems*. Cham: Springer Nature Switzerland, 2024, pp. 221–229. ISBN: 978-3-031-62645-6. DOI: 10.1007/978-3-031-62645-6_12.
- [63] T. T. P. Franken and T. Neele. “Expressivity of AuDaLa: Turing Completeness and Possible Extensions”. In: *Logical Methods in Computer Science* Volume 22, Issue 2 (2026). ISSN: 1860-5974. DOI: 10.46298/lmcs-22(2:5)2026.

- [64] T. T. P. Franken, T. Neele, and J. F. Groote. “An Autonomous Data Language”. In: *Theoretical Aspects of Computing – ICTAC 2023*. Cham: Springer Nature Switzerland, 2023, pp. 158–177. ISBN: 978-3-031-47963-2. DOI: 10.1007/978-3-031-47963-2_11.
- [65] T. T. P. Franken, T. Neele, and J. F. Groote. “The Autonomous Data Language – Concepts, design and formal verification”. In: *Theoretical Computer Science* 1057 (2025), p. 115560. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2025.115560>.
- [66] A. Gajentaan and M. H. Overmars. “On a class of $O(n^2)$ problems in computational geometry”. In: *Computational Geometry* 5.3 (1995), pp. 165–185. ISSN: 0925-7721. DOI: 10.1016/0925-7721(95)00022-2.
- [67] G. Galante, R. da Rosa Righi, and C. de Andrade. “Extending parallel programming patterns with adaptability features”. In: *Cluster Computing* 27.9 (2024), pp. 12547–12568. ISSN: 1573-7543. DOI: 10.1007/s10586-024-04622-0.
- [68] S. Gannouni. “A Gamma-calculus GPU-based parallel programming framework”. In: *WSWAN Proc. IEEE*, 2015, pp. 1–4. DOI: 10.1109/WSWAN.2015.7210299.
- [69] M. Garland et al. “Parallel Computing Experiences with CUDA”. In: *IEEE Micro* 28.4 (2008), pp. 13–27. DOI: 10.1109/MM.2008.57.
- [70] A. Geist and D. A. Reed. “A survey of high-performance computing scaling challenges”. EN. In: *The International Journal of High Performance Computing Applications* 31.1 (2017), pp. 104–113. ISSN: 1094-3420. DOI: 10.1177/1094342015597083.
- [71] D. Gelernter and N. Carriero. “Coordination languages and their significance”. In: *Commun. ACM* 35.2 (Feb. 1, 1992), pp. 96–107. ISSN: 0001-0782. DOI: 10.1145/129630.376083.
- [72] F. Ghanim, U. Vishkin, and R. Barua. “Easy PRAM-Based High-Performance Parallel Programming with ICE”. In: *IEEE Transactions on Parallel and Distributed Systems* 29.2 (2018), pp. 377–390. ISSN: 1558-2183. DOI: 10.1109/TPDS.2017.2754376.
- [73] K. Gharachorloo, S. V. Adve, A. Gupta, J. L. Hennessy, and M. D. Hill. “Programming for different memory consistency models”. In: *Journal of Parallel and Distributed Computing* 15.4 (Aug. 1992), pp. 399–407. ISSN: 0743-7315. DOI: 10.1016/0743-7315(92)90052-0.
- [74] J. Gibbons. “Functional Programming for Domain-Specific Languages”. In: *CEFP 2013*. LNCS. Springer International Publishing, 2015, pp. 1–28. ISBN: 978-3-319-15940-9. DOI: 10.1007/978-3-319-15940-9_1.

- [75] M. B. Giles and I. Reguly. “Trends in high-performance computing for engineering calculations”. In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 372.2022 (2014), p. 20130319. ISSN: 1364-503X. DOI: 10.1098/rsta.2013.0319.
- [76] B. Gilles, H. Justin, and L. Kevin. “A probabilistic separation logic”. In: *Proceedings of the ACM on Programming Languages* (2019). DOI: 10.1145/3371123.
- [77] L. Gonnord, L. Henrio, L. Morel, and G. Radanne. “A Survey on Parallelism and Determinism”. In: *ACM Comput. Surv.* 55.10 (2023), 210:1–210:28. ISSN: 0360-0300. DOI: 10.1145/3564529.
- [78] M. T. Goodrich and S. R. Kosaraju. “Sorting on a parallel pointer machine with applications to set expression evaluation”. In: *J. ACM* 43.2 (1996), pp. 331–361. DOI: 10.1145/226643.226670.
- [79] A. Gotsman, J. Berdine, and B. Cook. “Precision and the Conjunction Rule in Concurrent Separation Logic”. In: *Electronic Notes in Theoretical Computer Science*. Twenty-seventh Conference on the Mathematical Foundations of Programming Semantics (MFPS XXVII) 276 (2011), pp. 171–190. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2011.09.021.
- [80] W. Haensch. “Scaling is over — What now?” In: *2017 75th Annual Device Research Conference (DRC)*. 2017, pp. 1–2. DOI: 10.1109/DRC.2017.7999480.
- [81] S. W. Al-Haj Baddar and K. E. Batchner. “The AKS Sorting Network”. In: *Designing Sorting Networks: A New Paradigm*. New York, NY: Springer, 2011, pp. 73–80. ISBN: 978-1-4614-1851-1. DOI: 10.1007/978-1-4614-1851-1_11.
- [82] M. Harris, S. Sengupta, and J. D. Owens. “Parallel Prefix Sum (Scan) with CUDA”. In: *GPU gems* 3.39 (2007), pp. 851–876.
- [83] J. Hayman and G. Winskel. “Independence and concurrent separation logic”. In: *Logical Methods in Computer Science* Volume 4, Issue 1 (2008). ISSN: 1860-5974. DOI: 10.2168/LMCS-4(1:6)2008.
- [84] A. Henderson, R. Nicolescu, M. J. Dinneen, T. N. Chan, H. Happe, and T. Hinze. “Turing completeness of water computing”. In: *J Membr Comput* 3 (2021), pp. 182–193. ISSN: 2523-8914. DOI: 10.1007/s41965-021-00081-3.
- [85] T. Henriksen et al. “Futhark: purely functional GPU-programming with nested parallelism and in-place array updates”. In: *PLDI 2017*. PLDI 2017. ACM, 2017, pp. 556–571. DOI: 10.1145/3062341.3062354.

- [86] C. Hewitt. “Viewing control structures as patterns of passing messages”. In: *Artificial Intelligence* 8.3 (1977), pp. 323–364. ISSN: 0004-3702. DOI: 10.1016/0004-3702(77)90033-9.
- [87] C. Hewitt, P. B. Bishop, and R. Steiger. “A Universal Modular ACTOR Formalism for Artificial Intelligence”. In: *Proceedings of the 3rd International Joint Conference on Artificial Intelligence. Stanford, CA, USA, August 20-23, 1973*. William Kaufmann, 1973, pp. 235–245.
- [88] H.-D. A. Hiep and F. S. d. Boer. “A sound and complete proof system for separation logic (part 1)”. In: *dr. heap 2.2* (2024). ISSN: 3050-5224. DOI: 10.59350/2gkd1-c0k49.
- [89] W. D. Hillis. *The connection machine*. Cambridge, Mass: MIT Press, 1989.
- [90] W. D. Hillis and G. L. Steele. “Data parallel algorithms”. In: *Commun. ACM* 29.12 (1986), pp. 1170–1183. DOI: 10.1145/7902.7903.
- [91] C. A. R. Hoare. “An axiomatic basis for computer programming”. In: *Commun. ACM* 12.10 (1969), pp. 576–580. ISSN: 0001-0782. DOI: 10.1145/363235.363259.
- [92] A. Hobor, A. W. Appel, and F. Z. Nardelli. “Oracle Semantics for Concurrent Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2008, pp. 353–367. ISBN: 978-3-540-78739-6. DOI: 10.1007/978-3-540-78739-6_27.
- [93] A. Hobor and C. Gherghina. “Barriers in Concurrent Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2011, pp. 276–296. ISBN: 978-3-642-19718-5. DOI: 10.1007/978-3-642-19718-5_15.
- [94] M. Hoffman. *Visibility Graphs and 3-Sum*. <https://ti.inf.ethz.ch/ew/courses/CG09/materials/v12.pdf>. Accessed: 2025-12-01.
- [95] S. Hooker. “The hardware lottery”. In: *Commun. ACM* 64.12 (2021), pp. 58–65. ISSN: 0001-0782. DOI: 10.1145/3467017.
- [96] J. E. Hopcroft, R. Motwani, and J. D. Ullman. *Introduction to automata theory, languages, and computation*. 2nd. Boston: Addison-Wesley, 2001. ISBN: 978-0-201-44124-6.
- [97] J. JáJá. *An introduction to parallel algorithms*. USA: Addison Wesley Longman Publishing Co., Inc., 1992. ISBN: 0201548569.
- [98] J. B. Jensen and L. Birkedal. “Fictional Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2012, pp. 377–396. ISBN: 978-3-642-28869-2. DOI: 10.1007/978-3-642-28869-2_19.

- [99] E. B. Johnsen, R. Hähnle, J. Schäfer, R. Schlatte, and M. Steffen. “ABS: A Core Language for Abstract Behavioral Specification”. In: *Formal Methods for Components and Objects*. Berlin, Heidelberg: Springer, 2012, pp. 142–164. ISBN: 978-3-642-25271-6. DOI: 10.1007/978-3-642-25271-6_8.
- [100] R. Jung, R. Krebbers, J.-H. Jourdan, A. Bizjak, L. Birkedal, and D. Dreyer. “Iris from the ground up: A modular foundation for higher-order concurrent separation logic”. In: *Journal of Functional Programming* 28 (2018), e20. ISSN: 0956-7968, 1469-7653. DOI: 10.1017/S0956796818000151.
- [101] H. Kasim, V. March, R. Zhang, and S. See. “Survey on Parallel Programming Model”. In: *Network and Parallel Computing*. Berlin, Heidelberg: Springer, 2008, pp. 266–275. ISBN: 978-3-540-88140-7. DOI: 10.1007/978-3-540-88140-7_24.
- [102] J. C. King. “Symbolic execution and program testing”. In: *Communications of the ACM* (1976). DOI: 10.1145/360248.360252.
- [103] V. E. Kotov. “Theory of parallel programming. Part I: Survey of practical aspects”. In: *Cybernetics* 10.1 (1974), pp. 1–18. ISSN: 1573-8337. DOI: 10.1007/BF01069013.
- [104] D. Kozen. “On parallelism in Turing machines”. In: *17th Annual Symposium on Foundations of Computer Science (sfcs 1976)*. IEEE, 1976, pp. 89–97. DOI: 10.1109/SFCS.1976.20.
- [105] R. Krebbers, R. Jung, A. Bizjak, J.-H. Jourdan, D. Dreyer, and L. Birkedal. “The Essence of Higher-Order Concurrent Separation Logic”. In: *Programming Languages and Systems*. Berlin, Heidelberg: Springer, 2017, pp. 696–723. ISBN: 978-3-662-54434-1. DOI: 10.1007/978-3-662-54434-1_26.
- [106] L. Lamport. “The ‘Hoare logic’ of concurrent programs”. In: *Acta Informatica* (1980), pp. 21–37. ISSN: 1432-0525. DOI: 10.1007/BF00289062.
- [107] E. Lee. “The problem with threads”. In: *Computer* 39.5 (2006), pp. 33–42. ISSN: 1558-0814. DOI: 10.1109/MC.2006.180.
- [108] S. Lee et al. “Comparing Programming Language Comprehension between Novice and Expert Programmers Using EEG Analysis”. In: *2016 IEEE 16th International Conference on Bioinformatics and Bioengineering (BIBE)*. ISSN: 2471-7819. 2016, pp. 350–355. DOI: 10.1109/BIBE.2016.30.
- [109] G. Leemrijse. *Towards relaxed memory semantics for the Autonomous Data Language*. MSc. thesis, Eindhoven University of Technology. 2023.

- [110] G. P. Leemrijse, T. T. P. Franken, and T. Neele. “Formalisation of a New Weak Semantics for AuDaLa”. In: *Automated Technology for Verification and Analysis*. Cham: Springer Nature Switzerland, 2025, pp. 93–116. ISBN: 978-3-031-78750-8. DOI: 10.1007/978-3-031-78750-8_5.
- [111] C. E. Leiserson. “Setting a Course for Post-Moore Software Performance”. In: *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*. PPOPP ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1. DOI: 10.1145/3710848.3717824.
- [112] C. E. Leiserson et al. “There’s plenty of room at the Top: What will drive computer performance after Moore’s law?” In: *Science* 368.6495 (June 2020), eaam9744. ISSN: 0036-8075, 1095-9203. DOI: 10.1126/science.aam9744.
- [113] S. Lerner, T. Millstein, and C. Chambers. “Automatically proving the correctness of compiler optimizations”. In: *Proceedings of the ACM SIGPLAN 2003 Conference on Programming Language Design and Implementation*. PLDI ’03. New York, NY, USA: Association for Computing Machinery, 2003, pp. 220–231. ISBN: 1581136625. DOI: 10.1145/781131.781156.
- [114] G. Liu. *Petri Nets: Theoretical Models and Analysis Methods for Concurrent Systems*. Springer Nature, 2022. ISBN: 978-981-19-6309-4. DOI: 10.1007/978-981-19-6309-4.
- [115] H. A. López et al. “Protocol-based verification of message-passing parallel programs”. In: *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. OOPSLA 2015. New York, NY, USA: Association for Computing Machinery, 2015, pp. 280–298. ISBN: 9781450336895. DOI: 10.1145/2814270.2814302.
- [116] L. Lu and M. L. Scott. “Toward a Formal Semantic Framework for Deterministic Parallel Programming”. In: *Distributed Computing*. Berlin, Heidelberg: Springer, 2011, pp. 460–474. ISBN: 978-3-642-24100-0. DOI: 10.1007/978-3-642-24100-0_43.
- [117] E. Mahe, C. Gaston, and P. L. Gall. “Equivalence of Denotational and Operational Semantics for Interaction Languages”. In: *Theoretical Aspects of Software Engineering*. Cham: Springer International Publishing, 2022, pp. 113–130. ISBN: 978-3-031-10363-6. DOI: 10.1007/978-3-031-10363-6_8.

- [118] Marc Snir, Steve W. Otto, Steven Huss-Lederman, David W. Walker, and Jack Dongarra. *MPI - The Complete Reference : Volume 1, The MPI Core*. 2nd ed. Cambridge, Mass: The MIT Press, 1998. ISBN: 978-0-262-69215-1.
- [119] A. Marcoux, C. Maurel, and P. Salle. “AL 1: a language for distributed applications”. In: *FTDCS1990 Workshop Proc.* IEEE, 1988, pp. 270–276. DOI: 10.1109/FTDCS.1988.26707.
- [120] M. Maresca and P. Baglietto. “A programming model for reconfigurable mesh based parallel computers”. In: *PMMPC Workshop Proc.* IEEE, 1993, pp. 124–133. DOI: 10.1109/PMMP.1993.315547.
- [121] A. Marowka. “Towards Standardization of Measuring the Usability of Parallel Languages”. In: *Parallel Processing and Applied Mathematics*. Berlin, Heidelberg: Springer, 2014, pp. 65–74. ISBN: 978-3-642-55195-6. DOI: 10.1007/978-3-642-55195-6_6.
- [122] J. Martens, J. F. Groote, L. van den Haak, P. Hijma, and A. Wijs. “A Linear Parallel Algorithm to Compute Bisimulation and Relational Coarsest Partitions”. In: *Formal Aspects of Component Software*. Cham: Springer International Publishing, 2021, pp. 115–133. ISBN: 978-3-030-90636-8. DOI: 10.1007/978-3-030-90636-8_7.
- [123] I. A. Mason and C. L. Talcott. “Actor languages their syntax, semantics, translation, and equivalence”. In: *Theoretical Computer Science* 220.2 (1999), pp. 409–467. ISSN: 0304-3975. DOI: 10.1016/S0304-3975(99)00009-2.
- [124] T. Mattson and M. Wrinn. “Parallel programming: can we PLEASE get it right this time?” In: *Proceedings of the 45th annual Design Automation Conference*. DAC '08. New York, NY, USA: Association for Computing Machinery, 2008, pp. 7–11. ISBN: 978-1-60558-115-6. DOI: 10.1145/1391469.1391474.
- [125] A. McCreight. “Practical Tactics for Separation Logic”. In: *Theorem Proving in Higher Order Logics*. Berlin, Heidelberg: Springer, 2009, pp. 343–358. ISBN: 978-3-642-03359-9. DOI: 10.1007/978-3-642-03359-9_24.
- [126] P.-A. Melliès and L. Stefanescu. “A Game Semantics of Concurrent Separation Logic”. In: *Electronic Notes in Theoretical Computer Science*. The Thirty-third Conference on the Mathematical Foundations of Programming Semantics (MFPS XXXIII) 336 (2018), pp. 241–256. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2018.03.026.
- [127] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard Version 5.0*. 2025.

- [128] R. Milner. “A theory of type polymorphism in programming”. In: *Journal of Computer and System Sciences* 17.3 (Dec. 1978), pp. 348–375. ISSN: 0022-0000. DOI: 10.1016/0022-0000(78)90014-4.
- [129] J. C. Mitchell. “CHAPTER 8 - Type Systems for Programming Languages”. In: *Formal Models and Semantics*. Handbook of Theoretical Computer Science. Amsterdam: Elsevier, 1990, pp. 365–458. ISBN: 978-0-444-88074-1. DOI: 10.1016/B978-0-444-88074-1.50013-5.
- [130] J. C. Mitchell. “On abstraction and the expressive power of programming languages”. In: *Science of Computer Programming* 21.2 (1993), pp. 141–163. ISSN: 0167-6423. DOI: 10.1016/0167-6423(93)90004-9.
- [131] G. E. Moore et al. “Progress in digital integrated electronics”. In: *Electron devices meeting*. Vol. 21. Washington, DC. 1975, pp. 11–13.
- [132] G. Mudalige, M. Giles, I. Reguly, C. Bertolli, and P. Kelly. “OP2: An active library framework for solving unstructured mesh-based applications on multi-core and many-core architectures”. In: *InPar 2012*. 2012, pp. 1–12. DOI: 10.1109/InPar.2012.6339594.
- [133] D. G. Murray and S. Hand. “Non-deterministic parallelism considered useful”. In: *Proceedings of the 13th USENIX conference on Hot topics in operating systems*. HotOS’13. USA: USENIX Association, 2011, p. 19.
- [134] B. A. Myers et al. “Usability of Programming Languages: Special Interest Group (SIG) Meeting at CHI 2016”. In: *Proceedings of the 2016 CHI Conference Extended Abstracts on Human Factors in Computing Systems*. CHI EA ’16. New York, NY, USA: Association for Computing Machinery, 2016, pp. 1104–1107. ISBN: 978-1-4503-4082-3. DOI: 10.1145/2851581.2886434.
- [135] V. Nagarajan, D. J. Sorin, M. D. Hill, and D. A. Wood. *A Primer on Memory Consistency and Cache Coherence*. Springer International Publishing, 2020. ISBN: 978-3-031-01764-3. DOI: 10.1007/978-3-031-01764-3.
- [136] S. Nanz, F. Torshizi, M. Pedroni, and B. Meyer. “Design of an empirical study for comparing the usability of concurrent programming languages”. In: *Information and Software Technology* 55.7 (2013), pp. 1304–1315. ISSN: 0950-5849. DOI: 10.1016/j.infsof.2012.08.013.
- [137] S. Narayanasamy, Z. Wang, J. Tigani, A. Edwards, and B. Calder. “Automatically classifying benign and harmful data races using replay analysis”. In: *PLDI 2007 Proc*. Association for Computing Machinery, 2007, pp. 22–31. ISBN: 978-1-59593-633-2. DOI: 10.1145/1250734.1250738.

- [138] R. H. B. Netzer and B. P. Miller. “What are race conditions? Some issues and formalizations”. In: *ACM Lett. Program. Lang. Syst.* 1.1 (1992), pp. 74–88. ISSN: 1057-4514. DOI: 10.1145/130616.130623.
- [139] H. R. Nielson and F. Nielson. *Semantics with Applications: An Appetizer*. Undergraduate Topics in Computer Science. London: Springer, 2007. ISBN: 978-1-84628-692-6. DOI: 10.1007/978-1-84628-692-6.
- [140] S. Nishimura. “Reasoning About Data-Parallel Pointer Programs in a Modal Extension of Separation Logic”. In: *Algebraic Methodology and Software Technology*. Ed. by M. Johnson and V. Vene. Berlin, Heidelberg: Springer, 2006, pp. 293–307. ISBN: 978-3-540-35636-3. DOI: 10.1007/11784180_23.
- [141] P. O’Hearn. “Separation logic”. In: *Commun. ACM* 62.2 (Jan. 2019), pp. 86–95. ISSN: 0001-0782. DOI: 10.1145/3211968.
- [142] G. O’Regan. *A Brief History of Computing*. 1st ed. Springer Publishing Company, Incorporated, Jan. 2008. ISBN: 978-1-84800-083-4.
- [143] P. W. O’Hearn. “Resources, concurrency, and local reasoning”. In: *Theor. Comput. Sci.* 375.1-3 (2007), pp. 271–307. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2006.12.035.
- [144] A. Opler. “Procedure-oriented language statements to facilitate parallel processing”. In: *Commun. ACM* 8.5 (1965), pp. 306–307. ISSN: 0001-0782. DOI: 10.1145/364914.364947.
- [145] S. Owicki and D. Gries. “An axiomatic proof technique for parallel programs I”. In: *Acta Informatica* (1976), pp. 319–340. ISSN: 1432-0525. DOI: 10.1007/BF00268134.
- [146] S. Owicki and D. Gries. “Verifying properties of parallel programs: an axiomatic approach”. In: *Communications of the ACM* (1976), pp. 279–285. ISSN: 0001-0782. DOI: 10.1145/360051.360224.
- [147] J. Pagel and F. Zuleger. “Strong-separation Logic”. In: *ACM Trans. Program. Lang. Syst.* 44.3 (2022), 16:1–16:40. ISSN: 0164-0925. DOI: 10.1145/3498847.
- [148] G. Papadopoulos and F. Arbab. “Coordination Models and Languages”. In: *Advances in Computers*. Vol. 46. Elsevier, Jan. 1, 1998, pp. 329–400. DOI: 10.1016/S0065-2458(08)60208-9.
- [149] M. Parkinson. “The Next 700 Separation Logics”. In: *Verified Software: Theories, Tools, Experiments*. Berlin, Heidelberg: Springer, 2010, pp. 169–182. ISBN: 978-3-642-15057-9. DOI: 10.1007/978-3-642-15057-9_12.

- [150] J. Parrow. “Expressiveness of Process Algebras”. In: *Electronic Notes in Theoretical Computer Science*. Proceedings of the LIX Colloquium on Emerging Trends in Concurrency Theory (LIX 2006) 209 (2008), pp. 173–186. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2008.04.011.
- [151] *Past events | Express/SOS*.
- [152] B. C. Pierce. *Types and programming languages*. MIT press, 2002. ISBN: 9780262162098.
- [153] E. G. Pinho and F. H. de Carvalho. “An object-oriented parallel programming language for distributed-memory parallel computing platforms”. In: *Science of Computer Programming* 80 (2014), pp. 65–90. DOI: 10.1016/j.scico.2013.03.014.
- [154] D. Pym, J. M. Spring, and P. O’Hearn. “Why Separation Logic Works”. In: *Philosophy & Technology* 32.3 (2019), pp. 483–516. ISSN: 2210-5441. DOI: 10.1007/s13347-018-0312-8.
- [155] P. Qu, J. Yan, Y.-H. Zhang, and G. R. Gao. “Parallel Turing Machine, a Proposal”. In: *J. Comput. Sci. Technol.* 32 (2017), pp. 269–285. ISSN: 1860-4749. DOI: 10.1007/s11390-017-1721-3.
- [156] A. Raad, J. Berdine, D. Dreyer, and P. W. O’Hearn. “Concurrent incorrectness separation logic”. In: *Proceedings of the ACM on Programming Languages* (2022). DOI: 10.1145/3498695.
- [157] J. Ragan-Kelley et al. “Halide: decoupling algorithms from schedules for high-performance image processing”. In: *Commun. ACM* 61 (2017), pp. 106–115. DOI: 10.1145/3150211.
- [158] F. Raimbault and D. Lavenier. “RELACS for systolic programming”. In: *ASAP Proc. IEEE*, 1993, pp. 132–135. DOI: 10.1109/ASAP.1993.397128.
- [159] J. Reynolds. “Separation logic: a logic for shared mutable data structures”. In: *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*. 2002, pp. 55–74. DOI: 10.1109/LICS.2002.1029817.
- [160] J. C. Reynolds. “An Overview of Separation Logic”. In: *Verified Software: Theories, Tools, Experiments*. Berlin, Heidelberg: Springer, 2008, pp. 460–469. ISBN: 978-3-540-69149-5. DOI: 10.1007/978-3-540-69149-5_49.
- [161] J. J. M. M. Rutten. “Semantic Correctness for a Parallel Object-Oriented Language”. In: *SIAM Journal on Computing* (2006). Publisher: Society for Industrial and Applied Mathematics. DOI: 10.1137/0219023.

- [162] C. Sadowski and A. Shewmaker. “The last mile: parallel programming and usability”. In: *Proceedings of the FSE/SDP workshop on Future of software engineering research*. FoSER ’10. New York, NY, USA: Association for Computing Machinery, 2010, pp. 309–314. ISBN: 978-1-4503-0427-6. DOI: 10.1145/1882362.1882426.
- [163] P. Sanders, K. Mehlhorn, M. Dietzfelbinger, and R. Dementiev. *Sequential and Parallel Algorithms and Data Structures: The Basic Toolbox*. Cham: Springer International Publishing, 2019. ISBN: 978-3-030-25209-0. DOI: 10.1007/978-3-030-25209-0.
- [164] D. B. Skillicorn and D. Talia. “Models and languages for parallel computation”. In: *ACM Comput. Surv.* 30.2 (1998), pp. 123–169. ISSN: 0360-0300. DOI: 10.1145/280277.280278.
- [165] P. Soares, A. Ravara, and S. Melo De Sousa. “Revisiting Concurrent Separation Logic and Operational Semantics”. In: *2015 23rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing*. ISSN: 2377-5750. 2015, pp. 484–491. DOI: 10.1109/PDP.2015.85.
- [166] C. Stirling. “A generalization of Owicki-Gries’s Hoare logic for a concurrent while language”. In: *Theoretical Computer Science* (1988), pp. 347–359. ISSN: 0304-3975. DOI: 10.1016/0304-3975(88)90033-3.
- [167] A. Stromme, R. Carlson, and T. Newhall. “Chestnut: a GPU programming language for non-experts”. In: *PMAM Proc.* ACM, 2012, pp. 156–167. DOI: 10.1145/2141702.2141720.
- [168] K. Sudipta, T. Zachary, and L. Sorin. “Proving optimizations correct using parameterized program equivalence”. EN. In: *ACM SIGPLAN Notices* (June 2009). Publisher: ACM/PUB27 New York, NY, USA. DOI: 10.1145/1543135.1542513.
- [169] J. Sun, H. Vandierendonck, and D. S. Nikolopoulos. “GraphGrind: addressing load imbalance of graph partitioning”. In: *ICS Proc.* ACM, 2017, pp. 1–10. DOI: 10.1145/3079079.3079097.
- [170] J. Tassarotti and R. Harper. “A separation logic for concurrent randomized programs”. In: *Artifact for Article: A Separation Logic for Concurrent Randomized Programs* 3.POPL (2019), 64:1–64:30. DOI: 10.1145/3290377.
- [171] V. Tran, L. Hluchy, and G. Nguyen. “Parallel programming with data driven model”. In: *EMPDP Proc.* IEEE, 2000, pp. 205–211. DOI: 10.1109/EMPDP.2000.823413.

- [172] D. Ungar and S. S. Adams. “Harnessing emergence for manycore programming: early experience integrating ensembles, adverbs, and object-based inheritance”. In: *OOPSLA Proc. ACM*, 2010, pp. 19–26. DOI: 10.1145/1869542.1869546.
- [173] V. Vafeiadis. “Concurrent Separation Logic and Operational Semantics”. In: *Electronic Notes in Theoretical Computer Science*. Twenty-seventh Conference on the Mathematical Foundations of Programming Semantics (MFPS XXVII) 276 (Sept. 29, 2011), pp. 335–351. ISSN: 1571-0661. DOI: 10.1016/j.entcs.2011.09.029.
- [174] P. Van Roy et al. “Programming paradigms for dummies: What every programmer should know”. In: *New computational paradigms for computer music* 104 (2009), pp. 616–621.
- [175] S. Vialle, T. Cornu, and Y. Lallement. “ParCeL-1: a parallel programming language based on autonomous and synchronous actors”. In: *SIGPLAN Not.* 31.8 (1996), pp. 43–51. DOI: 10.1145/242903.242945.
- [176] E. de Vries and V. Koutavas. “Reverse Hoare Logic”. In: *Software Engineering and Formal Methods*. Vol. 7041. LNPSE. Springer, 2011, pp. 155–171. ISBN: 978-3-642-24690-6. DOI: 10.1007/978-3-642-24690-6_12.
- [177] J. Wiedermann. *Parallel Turing machines*. Department of Computer Science, University of Utrecht The Netherlands, 1984.
- [178] A. Wijs, J.-P. Katoen, and D. Bošnački. “GPU-Based Graph Decomposition into Strongly Connected and Maximal End Components”. In: *Computer Aided Verification*. Springer International Publishing, 2014, pp. 310–326. ISBN: 978-3-319-08867-9. DOI: 10.1007/978-3-319-08867-9_20.
- [179] J. J. Willcock, T. Hoefler, N. G. Edmonds, and A. Lumsdaine. “Active pebbles: parallel programming for data-driven applications”. In: *ICS Proc. ACM*, 2011, p. 235. DOI: 10.1145/1995896.1995934.
- [180] A. K. Wright and M. Felleisen. “A Syntactic Approach to Type Soundness”. In: *Information and Computation* 115.1 (Nov. 1994), pp. 38–94. ISSN: 0890-5401. DOI: 10.1006/inco.1994.1093.
- [181] X. Xia, L. Bao, D. Lo, Z. Xing, A. E. Hassan, and S. Li. “Measuring Program Comprehension: A Large-Scale Field Study with Professionals”. In: *IEEE Transactions on Software Engineering* 44.10 (2018), pp. 951–976. ISSN: 1939-3520. DOI: 10.1109/TSE.2017.2734091.

- [182] T. Yamashita et al. “Turing-Completeness of Asynchronous Non-camouflage Cellular Automata”. In: *Cellular Automata and Discrete Complex Systems*. LNCS. Springer International Publishing, 2017, pp. 187–199. ISBN: 978-3-319-58631-1. DOI: 10.1007/978-3-319-58631-1_15.
- [183] C.-S. D. Yang and L. L. Pollock. “All-uses testing of shared memory parallel programs”. In: *Software Testing, Verification and Reliability* 13.1 (2003), pp. 3–24. ISSN: 1099-1689. DOI: 10.1002/stvr.262.
- [184] H. Yang. “Relational separation logic”. In: *Theoretical Computer Science*. Festschrift for John C. Reynolds’s 70th birthday 375.1 (2007), pp. 308–334. ISSN: 0304-3975. DOI: 10.1016/j.tcs.2006.12.036.
- [185] M. Zheng, M. S. Rogers, Z. Luo, M. B. Dwyer, and S. F. Siegel. “CIVL: Formal Verification of Parallel Programs”. In: *2015 30th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2015, pp. 830–835. DOI: 10.1109/ASE.2015.99.

Summary

Since the advent of the computer, there has been a consistent effort to optimize computer programs for their execution speed, or performance. These days, a main method for doing so is exploiting parallelism, where multiple processors work at the same time to execute a program as opposed to one processor executing a program sequentially. To facilitate this, many languages and frameworks aimed at parallel programming have been created. Of these languages and frameworks, many of these put a large focus on the individual process, which is often considered separate from the whole when accessing hardware or interacting with other processes.

Parallel programming is inherently difficult. The behaviour of a parallel program is far harder to specify than the behaviour of a sequential program, due to the simultaneous behaviour of parallel processes. These behaviours may for example lead to *race conditions*, which are bugs caused by simultaneous but clashing accesses of the memory by processes. This makes it harder to create a *correct* parallel program and to verify that it is correct.

To solve this, many process-focused parallel languages require an explicit mapping and handling of the communication between multiple processes and between processes and the hardware, to limit unwanted behaviours. However, as this makes the language more complicated, this makes parallel programs in these languages less intuitive, more difficult to make and harder to verify.

We set out to investigate a solution for these problems. To do so, we introduce the data-autonomous paradigm, in which data elements execute their own code. It abstracts away from processes and hardware as much as possible. The paradigm focuses on local interactions between data and a robust design of the whole parallel system which executes the program. In this thesis, we develop the first data-autonomous parallel programming language: AuDaLa, the Autonomous Data Language. We claim that this language is understandable and feasible in practice, we show that the language has a compact but com-

plete semantics and we demonstrate that the language is verifiable in practice. Furthermore, to demonstrate the verifiability of AuDaLa, we develop AuDaLogic, a proof system for AuDaLa based on separation logic.

In the first part of this thesis, we give the theoretical foundations of AuDaLa, including a syntax, type system and a semantics. We discuss these foundations by giving some standard example programs, including programs for prefix sum and sorting. We further analyse these foundations by proving AuDaLa to be Turing complete, and discussing the limitations of the language. Furthermore, we demonstrate that the semantics of AuDaLa lends itself to adaptation by extending AuDaLa to overcome some of the discussed limitations.

Furthermore, we prove some semantic properties of AuDaLa programs, which we can use to verify AuDaLa programs with the syntax and the semantics. We demonstrate this using examples given in the thesis.

We also give an adaptation of the standard semantics which is compatible with conventional hardware, which allows for feasible implementations of AuDaLa on current machines. We prove that this adaptation does not allow for any behaviour the original semantics did not already allow as long as there are no read-write race conditions, allowing us to keep using the simpler standard semantics for verification.

In the second part of this thesis we introduce AuDaLogic, a proof system for AuDaLa based on separation logic. We give the syntax, semantics and proof rules of AuDaLogic and prove the proof system sound for AuDaLa. We demonstrate the proof system using an extended example. also give a further adaptation of the proof system to handle write-write race conditions, which we again demonstrate.

Samenvatting

Sinds de opkomst van de computer hebben mensen zich consistent ingespannen om computer programma's te optimaliseren voor executiesnelheid, hier efficiëntie genoemd. Op dit moment is een van de voornaamste manieren om dit te doen het gebruiken van parallelisme. In parallelle executie werken meerdere processoren tegelijk om samen een programma uit te voeren, in tegenstelling tot sequentiële executie, waar het programma door maar een processor wordt uitgevoerd. Om parallelle executie te bewerkstelligen zijn er veel programmeertalen en frameworks gemaakt voor parallel programmeren. Veel van deze programmeertalen en frameworks leggen de nadruk op individuele processen, welke vaak apart van het geheel worden beschouwd wanneer ze hardware aanroepen of communiceren met andere processen.

Parallel programmeren is inherent moeilijk. Het gedrag van een parallel programma is veel moeilijker om te specificeren vergeleken met het gedrag van een sequentieel programma, doordat het gedrag van de parallelle processen gelijktijdig plaatsvindt. Dit gedrag kan bijvoorbeeld leiden tot *race conditions*, bugs die worden veroorzaakt door het gelijktijdig aanroepen van het geheugen door verschillende processen, waar de twee processen elkaar in de weg zitten. Dit maakt het moeilijker om een *correct* parallel programma te maken, en ook om dat te verifiëren.

Om dit op te lossen vereisen veel proces-gefoceerde parallelle programmeertalen dat de communicatie tussen verschillende processen en de hardware expliciet wordt geregeld door de programmeur, om ongewenst gedrag te verminderen. Dit maakt zulke talen echter nog moeilijker, wat parallelle programma's in deze talen minder intuïtief, moeilijker te maken en moeilijker te verifiëren maakt.

Wij zijn begonnen met het zoeken naar een oplossing voor deze problemen. Om dit te doen introduceren wij de data-autonome programmeerstijl, waarin data elementen hun eigen code uitvoert. De stijl abstraheert weg van

processen en hardware waar mogelijk. De programmeerstijl focust op lokale interacties tussen data en een robuust design van het hele parallelle systeem dat een programma uitvoert.

In dit proefschrift ontwikkelen wij de eerste data-autonome programmeertaal: AuDaLa, wat staat voor Autonomous Data Language (Autonome Data Taal). Wij zijn van mening dat deze taal begrijpelijk en bruikbaar is in de praktijk, we laten zien dat de taal een compacte maar complete semantiek heeft en we demonstreren dat programma's in de taal te verifiëren zijn in de praktijk. Om dit laatste aan te tonen hebben wij AuDaLogic (Autonomous Data Logic, Autonome Data Logica) ontwikkeld, een bewijssysteem voor AuDaLa gebaseerd op separation logic.

In het eerste deel van dit proefschrift geven wij de theoretische basis van AuDaLa, inclusief een syntax, een typesysteem en een semantiek. Deze basis wordt besproken door middel van standaard voorbeeldprogramma's, inclusief programma's voor het prefix sum probleem en sorteren. Wij analyseren de basis verder door AuDaLa Turing compleet te bewijzen en de door sommige limitaties van de taal te bespreken. Daarna demonstreren wij dat de semantiek van AuDaLa zich leent voor aanpassing door AuDaLa uit te breiden om de besproken limitaties op te lossen.

Wij gebruiken de semantiek ook om sommige semantische eigenschappen van AuDaLa programma's te bewijzen die gebruikt kunnen worden om AuDaLa programma's te verifiëren via de syntax en de semantiek. Dit wordt gedemonstreerd in voorbeelden gegeven in het proefschrift.

Om te laten zien dat de taal bruikbaar is in de praktijk geven wij een uitbreiding van de standaard semantiek die compatibel is met de huidige hardware en kan worden gebruikt voor het implementeren van de taal op huidige hardware. Wij bewijzen dat de uitbreiding geen nieuw gedrag toestaat vergeleken met de standaard semantiek zolang er geen lees-schrijf race condities zijn, wat ons toestaat de simpelere standaard semantiek te gebruiken voor verificatie.

In het tweede deel van het proefschrift introduceren wij AuDaLogic, een bewijssysteem voor AuDaLa, gebaseerd op separation logic. Wij geven de syntax, semantiek en de bewijsregels van AuDaLogic en bewijzen de correctheid van dit bewijssysteem voor AuDaLa. Wij demonstreren het bewijssysteem met een uitgebreid voorbeeld. Ook geven wij een verdere uitbreiding voor dit bewijssysteem om schrijf-schrijf race condities toe te laten, die wij wederom demonstreren.

Curriculum Vitae

Thomas (Tom) Theo Paul Franken was born on July 2nd 1997 in Rotterdam. After finishing high school in 2015, he studied the combined major Software Science & Web Science at Eindhoven University of Technology. He obtained his bachelor's degree in 2018. He continued with a master's in Computer Science at Eindhoven University of Technology, for which he obtained his degree in 2021. Tom completed his master's thesis on the topic of creating an oracle data structure to use in an algorithm for subtrajectory clustering under the supervision of dr. Kevin Buchin and prof. dr. Anne Driemel (University of Bonn). During his master's, he also completed an internship on translating between different types of equation systems as used in formal verification, supervised by Tim Willemse.

In October 2021, Tom started as a PhD at the Eindhoven University of Technology. Under the supervision of Thomas Neele and Jan Friso Groote, he developed the parallel programming language AuDaLa and its proof system AuDaLogic, which among other things involved research on parallel programming design, semantics, parallel verification and separation logic. Tom's dissertation is titled "A Parallel Programming Perspective - Designing AuDaLa and AuDaLogic".

After his PhD, in fall 2026, Tom will start as a post-doctoral research fellow at the National University of Singapore, where he will investigate agentic verification using theorem provers.

Titles in the IPA Dissertation Series since 2023

S. Kochanthara. *A Changing Landscape: On Safety & Open Source in Automated and Connected Driving.* Faculty of Mathematics and Computer Science, TU/e. 2023-01

L.M. Ochoa Venegas. *Break the Code? Breaking Changes and Their Impact on Software Evolution.* Faculty of Mathematics and Computer Science, TU/e. 2023-02

N. Yang. *Logs and models in engineering complex embedded production software systems.* Faculty of Mathematics and Computer Science, TU/e. 2023-03

J. Cao. *An Independent Timing Analysis for Credit-Based Shaping in Ethernet TSN.* Faculty of Mathematics and Computer Science, TU/e. 2023-04

K. Dokter. *Scheduled Protocol Programming.* Faculty of Mathematics and Natural Sciences, UL. 2023-05

J. Smits. *Strategic Language Workbench Improvements.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-06

A. Arslanagić. *Minimal Structures for Program Analysis and Verification.* Faculty of Science and Engineering, RUG. 2023-07

M.S. Bouwman. *Supporting Railway Standardisation with Formal Verification.* Faculty of Mathematics and Computer Science, TU/e. 2023-08

S.A.M. Lathouwers. *Exploring Annotations for Deductive Verification.* Faculty of Electrical Engineering,

Mathematics & Computer Science, UT. 2023-09

J.H. Stoel. *Solving the Bank, Lightweight Specification and Verification Techniques for Enterprise Software.* Faculty of Mathematics and Computer Science, TU/e. 2023-10

D.M. Groenewegen. *WebDSL: Linguistic Abstractions for Web Programming.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2023-11

D.R. do Vale. *On Semantical Methods for Higher-Order Complexity Analysis.* Faculty of Science, Mathematics and Computer Science, RU. 2024-01

M.J.G. Olsthoorn. *More Effective Test Case Generation with Multiple Tribes of AI.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-02

B. van den Heuvel. *Correctly Communicating Software: Distributed, Asynchronous, and Beyond.* Faculty of Science and Engineering, RUG. 2024-03

H.A. Hiep. *New Foundations for Separation Logic.* Faculty of Mathematics and Natural Sciences, UL. 2024-04

C.E. Brandt. *Test Amplification For and With Developers.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-05

J.I. Hejderup. *Fine-Grained Analysis of Software Supply Chains.* Fac-

ulty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-06

J. Jacobs. *Guarantees by construction.* Faculty of Science, Mathematics and Computer Science, RU. 2024-07

O. Bunte. *Cracking OIL: A Formal Perspective on an Industrial DSL for Modelling Control Software.* Faculty of Mathematics and Computer Science, TU/e. 2024-08

R.J.A. Erkens. *Automaton-based Techniques for Optimized Term Rewriting.* Faculty of Mathematics and Computer Science, TU/e. 2024-09

J.J.M. Martens. *The Complexity of Bisimilarity by Partition Refinement.* Faculty of Mathematics and Computer Science, TU/e. 2024-10

L.J. Edixhoven. *Expressive Specification and Verification of Choreographies.* Faculty of Science, OU. 2024-11

J.W.N. Paulus. *On the Expressivity of Typed Concurrent Calculi.* Faculty of Science and Engineering, RUG. 2024-12

J. Denkers. *Domain-Specific Languages for Digital Printing Systems.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-13

L.H. Applis. *Tool-Driven Quality Assurance for Functional Programming and Machine Learning.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2024-14

P. Karkhanis. *Driving the Future: Facilitating C-ITS Service Deployment for Connected and Smart Roadways.* Faculty of Mathematics and Computer Science, TU/e. 2024-15

N.W. Cassee. *Sentiment in Software Engineering.* Faculty of Mathematics and Computer Science, TU/e. 2024-16

H. van Antwerpen. *Declarative Name Binding for Type System Specifications.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-01

I.N. Mulder. *Proof Automation for Fine-Grained Concurrent Separation Logic.* Faculty of Science, Mathematics and Computer Science, RU. 2025-02

T.S. Badings. *Robust Verification of Stochastic Systems: Guarantees in the Presence of Uncertainty.* Faculty of Science, Mathematics and Computer Science, RU. 2025-03

A.M. Mir. *Machine Learning-assisted Software Analysis.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-04

L.T. Vinkhuijzen. *Data Structures for Quantum Circuit Verification and How To Compare Them.* Faculty of Mathematics and Natural Sciences, UL. 2025-05

D. van der Wal. *What is the Point? Single-Input-Change Testing a EULYNX Controller.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-06

A. Rosset. *Uniform Monad Presentations and Graph Quasitoposes.*

Faculty of Sciences, Department of Computer Science, VU. 2025-07

L. Guo. *Higher-Order Termination with Logical Constraints.* Faculty of Science, Mathematics and Computer Science, RU. 2025-08

D.A. Manrique Negrin. *A Model Orchestra in Digital Twins: A Model-Driven Approach to Integration and Orchestration.* Faculty of Mathematics and Computer Science, TU/e. 2025-09

C.A. Esterhuyse. *Specification-Centric Multi-Agent Systems.* Faculty of Science, UvA. 2025-10

H.M. Muctadir. *Consistency Matters: Building Consistent Digital Twin Virtual Entities.* Faculty of Mathematics and Computer Science, TU/e. 2025-11

A. Stramaglia. *Model Checking Machine-Control Applications.* Faculty of Mathematics and Computer Science, TU/e. 2025-12

M. Saeedi Nikoo. *Supporting business process management: clone detection and recommendation techniques.* Faculty of Mathematics and Computer Science, TU/e. 2025-13

R.B. Rubbens. *Bridging the Implementation Gap: Advances in Model-Based Concurrent Program Verification.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2025-14

F.I. van der Berg. *DMC Model Checker: Delta-Driven Variable-Length Next-State Generation via Recursive Compression.* Faculty of Elec-

trical Engineering, Mathematics & Computer Science, UT. 2025-15

D.G. Sprokholt. *Correct Translation between Weak Memory Model Architectures.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2025-16

B. Kohlen. *Verified & Fast Probabilistic Verification: Stepwise Refinement of Probabilistic Model Checking into an IEEE 754 Floating-Point Implementation.* Faculty of Electrical Engineering, Mathematics & Computer Science, UT. 2026-01

D. Frolich. *Exploratory Language Development.* Faculty of Science, UvA. 2026-02

L.B. van den Haak. *Verifying Optimised Parallel Code.* Faculty of Mathematics and Computer Science, TU/e. 2026-03

A. Papotti. *Human Acceptance of Machine Generated Code Security Analysis.* Faculty of Sciences, Department of Computer Science, VU. 2026-04

N. Doorn. *O bug, where art thou? Improving software testing education.* Faculty of Science, OU. 2026-05

C.R. van der Rest. *Reusable Programming Language Components.* Faculty of Electrical Engineering, Mathematics, and Computer Science, TUD. 2026-06

T.T.P. Franken. *A Parallel Programming Perspective – Designing AuDaLa and AuDaLogic.* Faculty of Mathematics and Computer Science, TU/e. 2026-07